

ORACLE®

Полностью обновлено для JDK 7

Java



руководство для начинающих

Пятое издание

Незаменимое учебное пособие по написанию, компилированию и выполнению современных программ на Java

Герберт Шилдт

Java

руководство для начинающих

Пятое издание

Java

A Beginner's Guide

Fifth Edition

HERBERT SCHILDT

McGraw Hill

Java

руководство для начинающих

Пятое издание

ГЕРБЕРТ ШИЛДТ



Москва · Санкт-Петербург · Киев
2012

ББК 32.973.26-018.2.75

Ш57

УДК 681.3.07

Издательский дом “Вильямс”

Зав. редакцией *С.Н. Тригуб*

Перевод с английского и редакция *И.В. Берштейна*

По общим вопросам обращайтесь в Издательский дом “Вильямс” по адресу:
info@williamspublishing.com, <http://www.williamspublishing.com>

Шилдт, Герберт.

Ш57 Java: руководство для начинающих, 5-е изд. : Пер. с англ. — М. : ООО “И.Д. Вильямс”, 2012. — 624 с. : ил. — Парал. тит. англ.

ISBN 978-5-8459-1770-6 (рус.)

ББК 32.973.26-018.2.75

Все названия программных продуктов являются зарегистрированными торговыми марками соответствующих фирм.

Никакая часть настоящего издания ни в каких целях не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами, будь то электронные или механические, включая фотокопирование и запись на магнитный носитель, если на это нет письменного разрешения издательства Osborne Media.

Authorized translation from the English language edition published by McGraw-Hill Companies, Copyright © 2012.

All rights reserved. Except as permitted under the Copyright Act of 1976, no part of this publication may be reproduced or distributed in any form or by any means, or stored in a database or retrieval system, without the prior written permission of Publisher, with the exception that the program listings may be entered, stored, and executed in a computer system, but they may not be reproduced for publication.

Oracle and Java are registered trademarks of Oracle Corporation and/or its affiliates.

Russian language edition published by Williams Publishing House according to the Agreement with R&I Enterprises International, Copyright © 2012

Научно-популярное издание

Герберт Шилдт

Java: руководство для начинающих

5-е издание

Литературный редактор *И.А. Попова*

Верстка *М.А. Удалов*

Художественный редактор *В.Г. Павлютин*

Корректор *Л.А. Гордиенко*

Подписано в печать 19.04.2012. Формат 70х100/16.

Гарнитура Times. Печать офсетная.

Усл. печ. л. 50,3. Уч.-изд. л. 32,05.

Тираж 1500 экз. Заказ № 232.

Первая Академическая типография “Наука”

199034, Санкт-Петербург, 9-я линия, 12/28.

ООО “И. Д. Вильямс”, 127055, г. Москва, ул. Лесная, д. 43, стр. 1

ISBN 978-5-8459-1770-6 (рус.)

ISBN 978-0-07-160632-5 (англ.)

© Издательский дом “Вильямс”, 2012

© by The McGraw-Hill Companies, Inc., 2012



Оглавление

Введение	17
Глава 1. Основы Java	23
Глава 2. Введение в типы данных и операторы	55
Глава 3. Управляющие операторы	85
Глава 4. Введение в классы, объекты и методы	123
Глава 5. Дополнительные сведения о типах данных и операторах	155
Глава 6. Дополнительные сведения о методах и классах	199
Глава 7. Наследование	241
Глава 8. Пакеты и интерфейсы	283
Глава 9. Обработка исключений	309
Глава 10. Ввод-вывод данных	339
Глава 11. Многопоточное программирование	383
Глава 12. Перечисления, автоупаковка, статический импорт и аннотации	421
Глава 13. Обобщения	451
Глава 14. Апплеты, события и прочее	487
Глава 15. Введение в Swing	517
Приложение А. Ответы на вопросы для самопроверки	555
Приложение Б. Применение документирующих комментариев в Java	603
Предметный указатель	611



Содержание

Введение	17
Структура книги	20
Основные навыки и понятия	20
Упражнения для самопроверки	20
Обращение к знатоку	20
Примеры для опробования	20
Навыки программирования	20
Необходимое программное обеспечение	21
Исходный код примеров программ	21
Особая благодарность	21
Другая литература на данную тему	21
От издательства	22
Глава 1. Основы Java	23
Основные навыки и понятия	24
Истоки Java	25
Взаимосвязь Java с языками C и C++	26
Взаимосвязь Java с языком C#	26
Вклад Java в развитие Интернета	27
Java-апплеты	27
Безопасность	28
Переносимость	28
Волшебный байт-код Java	28
Основные свойства Java	30
Объектно-ориентированное программирование	31
Инкапсуляция	32
Полиморфизм	32
Наследование	33
Установка комплекта Java Development Kit	33

Первая программа на Java	34
Ввод исходного кода программ	35
Компиляция программы	35
Построчный анализ исходного кода первого примера программы	36
Обработка синтаксических ошибок	39
Еще одна простая программа	39
Другие типы данных	41
Пример для опробования 1.1. Преобразование галлонов в литры	43
Два управляющих оператора	44
Оператор if	44
Цикл for	46
Создание кодовых блоков	47
Точки с запятой и оформление исходного текста программы	48
Практическое применение отступов	49
Пример для опробования 1.2. Усовершенствованный вариант преобразования галлонов в литры	49
Ключевые слова Java	51
Идентификаторы в Java	51
Библиотеки классов Java	52
Упражнение для самопроверки по материалу главы 1	52
Глава 2. Введение в типы данных и операторы	55
Основные навыки и понятия	56
Особое значение типов данных	56
Простые типы данных в Java	56
Целочисленные типы данных	57
Типы данных с плавающей точкой	58
Символы	59
Логический тип данных	61
Пример для опробования 2.1. Расчет расстояния до удара молнии	61
Литералы	62
Шестнадцатеричные, восьмеричные и двоичные литералы	63
Символьные управляющие последовательности	64
Строковые литералы	64
Подробнее о переменных	65
Инициализация переменных	66
Динамическая инициализация	66
Область и срок действия переменных	67
Операторы	69
Арифметические операторы	70
Операторы инкремента и декремента	71
Операторы отношения и логические операторы	72
Укороченные логические операторы	73
Оператор присваивания	75
Укороченные операторы присваивания	75
Преобразование типов при присваивании	77
Приведение несовместимых типов	78

Предшествование операторов	79
Пример для опробования 2.2. Отображение таблицы истинности для логических операторов	80
Выражения	81
Преобразование типов в выражениях	82
Пробелы и круглые скобки	83
Упражнение для самопроверки по материалу главы 2	84
Глава 3. Управляющие операторы	85
Основные навыки и понятия	86
Ввод символов с клавиатуры	86
Условный оператор if	87
Вложенные условные операторы if	89
Многоступенчатая конструкция if-else-if	90
Оператор switch	91
Вложенные операторы switch	94
Пример для опробования 3.1. Начало построения справочной системы Java	94
Цикл for	97
Некоторые разновидности цикла for	98
Пропуск отдельных частей в определении цикла for	99
Бесконечный цикл	101
Циклы без тела	101
Объявление управляющих переменных в цикле for	102
Расширенный цикл for	102
Цикл while	103
Цикл do-while	104
Пример для опробования 3.2. Расширение справочной системы Java	106
Применение оператора break для выхода из цикла	109
Оператор break в качестве оператора goto	111
Применение оператора continue	114
Пример для опробования 3.3. Завершение построения справочной системы Java	116
Вложенные циклы	119
Упражнение для самопроверки по материалу главы 3	120
Глава 4. Введение в классы, объекты и методы	123
Основные навыки и понятия	124
Основные положения о классах	124
Общая форма определения класса	125
Определение класса	125
Порядок создания объектов	128
Переменные ссылочного типа и присваивание	129
Методы	130
Добавление метода в класс Vehicle	130
Возврат из метода	133
Возврат значения	133
Использование параметров	135
Добавление параметризованного метода в класс Vehicle	137

Пример для опробования 4.1. Создание справочного класса	138
Конструкторы	144
Параметризованные конструкторы	145
Добавление конструктора в класс Vehicle	146
Еще раз об операторе new	147
“Сборка мусора” и методы завершения	148
Метод finalize()	148
Пример для опробования 4.2. Демонстрация “сборки мусора” и завершения кода	149
Ключевое слово this	151
Упражнение для самопроверки по материалу главы 4	153
Глава 5. Дополнительные сведения о типах данных и операторах	155
Основные навыки и понятия	156
Массивы	156
Одномерные массивы	157
Пример для опробования 5.1. Сортировка массива	160
Многомерные массивы	161
Двумерные массивы	162
Нерегулярные массивы	162
Трехмерные, четырехмерные и многомерные массивы	164
Инициализация многомерных массивов	164
Альтернативный синтаксис объявления массивов	165
Присваивание ссылок на массивы	166
Применение переменной экземпляра length	167
Пример для опробования 5.2. Создание класса очереди	169
Разновидность for-each цикла for	173
Циклическое обращение к многомерным массивам	176
Применение расширенного цикла for	177
Символьные строки	178
Построение строк	178
Операции над символьными строками	179
Массивы символьных строк	181
Постоянство строк	182
Применение строк для управления оператором switch	183
Употребление аргументов командной строки	184
Поразрядные операторы	185
Поразрядные операторы И, ИЛИ, исключающее ИЛИ и НЕ	186
Операторы сдвига	190
Поразрядные составные операторы присваивания	192
Пример для опробования 5.3. Создание класса ShowBits	193
Оператор ?	195
Упражнение для самопроверки по материалу главы 5	197
Глава 6. Дополнительные сведения о методах и классах	199
Основные навыки и понятия	200
Управление доступом к членам класса	200

Модификаторы доступа в Java	201
Пример для опробования 6.1. Усовершенствование класса Queue	205
Передача объектов методам	206
Способы передачи аргументов методу	208
Возврат объектов	210
Перегрузка методов	212
Перегрузка конструкторов	217
Пример для опробования 6.2. Перегрузка конструктора класса Queue	218
Рекурсия	221
Применение ключевого слова static	224
Статические блоки	226
Пример для опробования 6.3. Быстрая сортировка	227
Вложенные и внутренние классы	230
Аргументы переменной длины	233
Общие положения об аргументах переменной длины	234
Перегрузка методов с аргументами переменной длины	236
Аргументы переменной длины и неоднозначность	238
Упражнение для самопроверки по материалу главы 6	239
Глава 7. Наследование	241
Основные навыки и понятия	242
Основы наследования	242
Доступ к членам класса и наследование	245
Конструкторы и наследование	248
Применение ключевого слова super для вызова конструктора суперкласса	249
Применение ключевого слова super для доступа к членам суперкласса	253
Пример для опробования 7.1. Расширение класса Vehicle	254
Создание многоуровневой иерархии классов	257
Порядок вызова конструкторов	260
Ссылки на суперкласс и объекты подклассов	261
Переопределение методов	266
Поддержка полиморфизма в переопределяемых методах	268
Причины для переопределения методов	270
Демонстрация механизма переопределения методов на примере класса TwoDShape	270
Применение абстрактных классов	274
Использование ключевого слова final	278
Предотвращение переопределения методов	278
Предотвращение наследования	279
Применение ключевого слова final к переменным экземпляра	279
Класс Object	281
Упражнение для самопроверки по материалу главы 7	282
Глава 8. Пакеты и интерфейсы	283
Основные навыки и понятия	284
Пакеты	284
Определение пакета	285

Поиск пакета и переменная окружения CLASSPATH	286
Простой пример применения пакета	286
Пакеты и доступ к членам классов	288
Пример доступа к пакету	289
Представление о защищенных членах классов	290
Импорт пакетов	292
Библиотечные классы Java, содержащиеся в пакетах	293
Интерфейсы	294
Реализация интерфейсов	295
Применение интерфейсных ссылок	299
Пример для опробования 8.1. Создание интерфейса для очереди	300
Переменные в интерфейсах	306
Наследование интерфейсов	307
Упражнение для самопроверки по материалу главы 8	308
Глава 9. Обработка исключений	309
Основные навыки и понятия	310
Иерархия исключений	311
Общее представление об обработке исключений	311
Использование ключевых слов try и catch	312
Простой пример обработки исключений	312
Последствия перехвата исключений	314
Обработка исключений — изящный способ устранения программных ошибок	316
Применение нескольких операторов catch	317
Перехват исключений, генерируемых подклассами	318
Вложенные блоки try	319
Генерирование исключений	320
Повторное генерирование исключений	321
Подробнее о классе Throwable	322
Использование ключевого слова finally	324
Использование ключевого слова throws	325
Новые средства обработки исключений, внедренные в версии JDK 7	327
Встроенные в Java исключения	328
Создание подклассов, производных от класса Exception	331
Пример для опробования 9.1. Добавление исключений в класс очереди	333
Упражнение для самопроверки по материалу главы 9	336
Глава 10. Ввод-вывод данных	339
Основные навыки и понятия	340
Организация системы ввода-вывода в Java на потоках	341
Байтовые и символьные потоки	341
Классы байтовых потоков	341
Классы символьных потоков	342
Встроенные потоки	343
Применение байтовых потоков	343
Консольный ввод	345
Вывод на консоль	346

Чтение и запись в файлы из байтовых потоков	347
Ввод данных из файла	347
Вывод в файл	351
Автоматическое закрытие файлов	353
Чтение и запись двоичных данных	356
Пример для опробования 10.1. Утилита сравнения файлов	359
Файлы с произвольным доступом	361
Применение символьных потоков в Java	363
Консольный ввод из символьных потоков	365
Консольный вывод в символьные потоки	367
Ввод-вывод в файлы через символьные потоки	369
Применение класса FileWriter	369
Применение класса FileReader	370
Применение оболочек типов для преобразования символьных строк в числа	371
Пример для опробования 10.2. Создание справочной системы, находящейся на диске	374
Упражнение для самопроверки по материалу главы 10	381
Глава 11. Многопоточное программирование	383
Основные навыки и понятия	384
Общее представление о многопоточной обработке	384
Класс Thread и интерфейс Runnable	385
Создание потока	386
Несложные усовершенствования многопоточной программы	389
Пример для опробования 11.1. Расширение класса Thread	391
Создание нескольких потоков	393
Определение момента завершения потока	396
Приоритеты потоков	399
Синхронизация	401
Применение синхронизированных методов	402
Синхронизированные блоки	405
Организация взаимодействия потоков с помощью методов notify() , wait() и notifyAll()	407
Пример применения методов wait() и notify()	408
Приостановка, возобновление и остановка потоков	414
Пример для опробования 11.2. Применение основного потока	418
Упражнение для самопроверки по материалу главы 11	419
Глава 12. Перечисления, автоупаковка, статический импорт и аннотации	421
Основные навыки и понятия	422
Перечисления	422
Основные положения о перечислениях	423
Перечисления в Java относятся к типам классов	425
Методы values() и valueOf()	425
Конструкторы, методы, переменные экземпляра и перечисления	427
Два важных ограничения	429

Наследование перечислений от класса Enum	429
Пример для опробования 12.1. Автоматизированный светофор	431
Автоупаковка	436
Оболочки типов	436
Основные положения о об автоупаковке	438
Автоупаковка и методы	439
Автоупаковка и автораспаковка в выражениях	440
Предупреждение относительно автоупаковки и автораспаковки	442
Статический импорт	442
Аннотации (метаданные)	445
Упражнение для самопроверки по материалу главы 12	448
Глава 13. Обобщения	451
Основные навыки и понятия	452
Основные положения об обобщениях	452
Простой пример обобщений	453
Действие обобщений распространяется только на объекты	457
Различение обобщений по аргументам типа	457
Обобщенный класс с двумя параметрами типа	457
Общая форма обобщенного класса	459
Ограниченные типы	459
Использование метасимвольных аргументов	462
Ограниченные метасимвольные аргументы	465
Обобщенные методы	468
Обобщенные конструкторы	470
Обобщенные интерфейсы	471
Пример для опробования 13.1. Создание обобщенного класса очереди	473
Базовые типы и устаревший код	477
Выводимость типов с помощью ромбовидного оператора	480
Стирание	481
Ошибки неоднозначности	482
Ограничения, накладываемые на обобщения	483
Невозможность получить экземпляры параметров типа	483
Ограничения, накладываемые на статические члены класса	483
Ограничения, накладываемые на обобщенные массивы	484
Ограничения, накладываемые на обобщенные исключения	485
Дальнейшее изучение обобщений	485
Упражнение для самопроверки по материалу главы 13	485
Глава 14. Апплеты, события и прочее	487
Основные навыки и понятия	488
Общее представление об апплетах	488
Организация апплета и его основные элементы	491
Архитектура апплетов	491
Заготовка апплета	492
Инициализация и завершение апплетов	493
Запрос на перерисовку окна апплета	494

Метод update()	495
Пример для опробования 14.1. Простой апплет для формирования крупного заголовка на веб-странице	495
Применение строки состояния	499
Передача параметров апплету	500
Класс Applet	501
Обработка событий	503
Модель делегирования событий	503
События	504
Источники событий	504
Приемники событий	504
Классы событий	505
Интерфейсы приемников событий	505
Применение модели делегирования событий	506
Обработка событий от мыши	507
Простой пример апплета, демонстрирующий обработку событий от мыши	508
Другие ключевые слова Java	510
Модификаторы transient и volatile	511
Ключевое слово instanceof	511
Ключевое слово strictfp	511
Ключевое слово assert	512
Собственные методы	513
Упражнение для самопроверки по материалу главы 14	514
Глава 15. Введение в Swing	517
Основные навыки и понятия	518
Происхождение и принципы конструирования Swing	519
Компоненты и контейнеры	521
Компоненты	521
Контейнеры	522
Панели контейнеров верхнего уровня	522
Диспетчеры компоновки	523
Первая простая Swing-программа	523
Построчный анализ первой Swing-программы	525
Применение компонента JButton	529
Работа с компонентом JTextField	533
Создание флажков с помощью компонента JCheckBox	537
Работа с компонентом JList	540
Пример для опробования 15.1. Утилита сравнения файлов, создаваемая на основе Swing	544
Применение анонимных внутренних классов для обработки событий	549
Создание апплета средствами Swing	550
Что делать дальше	552
Упражнения для самопроверки по материалу главы 15	553

Приложение А. Ответы на вопросы для самопроверки	555
Глава 1. Основы Java	556
Глава 2. Введение в типы данных и операторы	558
Глава 3. Управляющие операторы	559
Глава 4. Введение в классы, объекты и методы	562
Глава 5. Дополнительные сведения о типах данных и операторах	563
Глава 6. Дополнительные сведения о методах и классах	567
Глава 7. Наследование	571
Глава 8. Пакеты и интерфейсы	573
Глава 9. Обработка исключений	575
Глава 10. Ввод-вывод данных	578
Глава 11. Многопоточное программирование	581
Глава 12. Перечисления, автоупаковка, статический импорт и аннотации	583
Глава 13. Обобщения	587
Глава 14. Апплеты, события и прочее	591
Глава 15. Введение в Swing	596
Приложение Б. Применение документирующих комментариев в Java	603
Дескрипторы javadoc	604
Дескриптор <code>@author</code>	605
Дескриптор <code>{@code}</code>	605
Дескриптор <code>@deprecated</code>	605
Дескриптор <code>{@docRoot}</code>	606
Дескриптор <code>@exception</code>	606
Дескриптор <code>{@inheritDoc}</code>	606
Дескриптор <code>{@link}</code>	606
Дескриптор <code>{@linkplain}</code>	606
Дескриптор <code>{@literal}</code>	606
Дескриптор <code>@param</code>	606
Дескриптор <code>@return</code>	607
Дескриптор <code>@see</code>	607
Дескриптор <code>@serial</code>	607
Дескриптор <code>@serialData</code>	607
Дескриптор <code>@serialField</code>	607
Дескриптор <code>@since</code>	608
Дескриптор <code>@throws</code>	608
Дескриптор <code>@value</code>	608
Дескриптор <code>@version</code>	608
Общая форма документирующих комментариев	608
Результат, выводимый утилитой <code>javadoc</code>	609
Пример применения документирующих комментариев	609
Предметный указатель	611

Об авторе

Герберт Шилдт — признанный авторитет в области языков программирования C, C++, Java и C#. Его книги по программированию переведены на многие языки и разошлись по всему миру миллионными тиражами. Шилдт — автор целого ряда книг по Java, включая *Полный справочник по Java*, *Swing: руководство для начинающих*, *Искусство программирования на Java*. Его перу принадлежат также популярные книги *Полный справочник по C++*, *Полный справочник по C*, *Полный справочник по C#*. И хотя ему не чужды все аспекты вычислительной техники, главный интерес для него представляют языки программирования, в том числе компиляторы, интерпретаторы и языки управления робототехническими системами. Он также живо интересуется вопросами стандартизации языков программирования. Шилдт закончил университет штата Иллинойс. А связаться с ним можно по адресу www.HerbSchildt.com.

О научном рецензенте

Доктор **Дэнни Кауэрд** занимается платформами Java с 1997 года. Работая в компании Sun Microsystems Inc., он был одним из основателей группы экспертов по Java EE, членом исполнительного комитета, действовавшего в рамках Процесса сообщества пользователей Java (Java Community Process), а также одним из ведущих специалистов, участвовавших в создании всех платформ Java: Java SE, Java ME и Java EE. Кроме того, он организовал первоначальную группу разработчиков платформы JavaFX.



Введение

Java считается одним из самых значительных и широко распространенных языков программирования, поскольку ему принадлежит исключительная роль в разработке приложений для Интернета. Для того чтобы профессионально заниматься разработкой веб-приложений, нужно досконально знать Java. Поэтому, если хотите связать свое будущее с программированием веб-приложений, вам нужно выбрать подходящий для этого язык и как следует изучить его. Кроме того, Java принадлежит ведущая роль в программировании смартфонов на платформе Android. Проще говоря, программирование на Java составляет основу большей части современной вычислительной техники.

Цель этой книги — обучить вас, читатель, основам программирования на языке Java. В ней применяется поэтапный подход к освоению языковых средств на многочисленных примерах, упражнениях на самопроверку и несложных проектах. Для изучения Java по этой книге не обязательно иметь какой-то опыт программирования. Книга начинается с самых основных понятий, включая компилирование и выполнение программ на Java. Затем речь пойдет о ключевых словах и языковых средствах и конструкциях, составляющих основу Java. После этого рассматриваются более сложные языковые средства Java, в том числе многопоточное программирование и обобщения. И завершается книга введением в библиотеку Swing. Все это позволит вам получить ясное представление об основах программирования на Java.

Но эта книга — лишь первый шаг на пути к освоению Java, поскольку для профессионального программирования на Java нужно знать не только составные элементы этого языка, но и многочисленные библиотеки и инструменты, существенно упрощающие процесс разработки программ. Проработав материал этой книги, вы получите достаточно знаний, чтобы приступить к изучению всех остальных аспектов Java.

Эволюция Java

Немногие языки могут похвастаться тем, что им удалось изменить общее представление о программировании. Но и в этой “элитной” группе один язык выделяется среди прочих. Его влияние очень быстро почувствовали все программисты. Речь, конечно же, идет о Java. Не будет преувеличением сказать, что выпуск в 1995 году компанией Sun Microsystems Inc. версии Java 1.0 вызвал настоящую революцию в программировании.

В результате Всемирная паутина стала по-настоящему интерактивной средой. Между тем Java установил новый стандарт в разработке языков программирования.

Со временем Java усовершенствовался. В отличие от многих других языков, в которых новые средства внедрялись относительно медленно, Java всегда находился на переднем крае разработки языков программирования. Одной из причин, позволивших добиться этого, послужило создание вокруг Java плодотворной атмосферы, способствовавшей внедрению новых идей. В результате язык Java постоянно совершенствовался: одни его изменения были незначительными, а другие — весьма существенными.

Первым существенным обновлением Java стала версия 1.1. Изменения в ней были более значительны, чем это обычно подразумевает переход к новой версии языка программирования. В версии Java 1.1 были добавлены многие библиотечные элементы, переопределены средства обработки событий, перекомпонованы многие функциональные средства исходной библиотеки версии 1.0.

Следующим этапом развития данного языка стала платформа Java 2, где цифра 2 обозначает “второе поколение”. Ее создание стало поворотным событием, ознаменовавшим начало “новой эпохи” Java. Первым выпуском Java 2 стала версия 1.2. На первый взгляд, несоответствие номеров в обозначениях Java 2 и версии 1.2 может показаться странным. Дело в том, что номером 1.2 сначала обозначались библиотеки Java и только затем весь выпуск. Компания Sun перекомпоновала программный продукт Java в J2SE (Java 2 Platform Standard Edition — Стандартная версия платформы Java 2), и с тех пор номера версии стали относиться именно к этому продукту.

Затем появилась версия J2SE 1.3, в которую были внесены первые значительные изменения по сравнению с первоначальным выпуском Java 2. Новые функциональные средства были в основном добавлены к уже существующим и более тесно связаны со средой разработки. Версия J2SE 1.4 стала очередным этапом в развитии Java. Она содержала новые важные средства, в том числе цепочки исключений, канальный ввод-вывод и ключевое слово **assert**.

Следующая версия J2SE 5, по сути, стала вторым революционным преобразованием Java. В отличие от большинства предыдущих модернизаций, которые сводились к важным, но предсказуемым усовершенствованиям, в J2SE 5 были существенно расширены рамки применения и функциональные возможности языка, а также повышена его производительность. Для более ясного представления о масштабах изменений, внесенных в версии J2SE 5, ниже приводится перечень новых средств, которые рассматриваются в этой книге.

- Обобщения
- Автоупаковка и автораспаковка
- Перечисления
- Усовершенствованный вариант `for-each` цикла `for`
- Аргументы переменной длины
- Статический импорт
- Аннотации

В этот список не вошли несущественные дополнения или поэтапные изменения, характерные для перехода к новой версии. Каждый элемент этого списка представляет собой существенное дополнение Java. Для поддержки одних нововведений, в том числе обобщений, варианта `for-each` цикла `for` и аргументов переменной длины, пришлось ввести новые синтаксические конструкции в язык. А другие нововведения, например

автоупаковка и автораспаковка, повлияли на семантику языка. И наконец, аннотации открыли совершенно новые возможности для программирования.

Особое значение новых языковых средств проявилось в том, что новая версия получила номер 5. Логично предположить, что номером очередной версии Java должен бы стать 1.5. Но нововведения оказались настолько значительными, что переход от версии 1.4 к 1.5 не отражал масштабы внесенных изменений. Поэтому разработчики из компании Sun решили увеличить номер версии до 5, подчеркнув тем самым важность нововведений. В итоге новая версия получила название J2SE 5, а комплект инструментальных средств разработчиков стал называться JDK 5. Но ради согласованности с предыдущими версиями было решено использовать 1.5 в качестве *внутреннего* номера версии, на который можно ссылаться как на номер версии, используемый при *разработке*. Номер 5 в J2SE 5 означает номер версии данного *программного продукта*.

Следующая версия Java называется J2SE 6. Это означает, что в компании Sun вновь решили изменить название платформы Java. Прежде всего, из названия исчезла цифра 2. Теперь платформа называется Java SE, а официальное имя продукта — Java Platform, Standard Edition 6. Как и в J2SE 5, цифра 6 в Java SE 6 означает номер версии программного продукта. А внутренним номером версии для использования при разработке является 1.6.

Версия Java SE 6 была построена на основе J2SE 5, но отличается от последней рядом нововведений. Изменения в этой версии не такие масштабные, как в предыдущей, но в ней были усовершенствованы библиотеки прикладного пользовательского интерфейса (API), добавлен ряд новых пакетов и улучшена исполняющая система. По существу, в версии Java SE 6 были закреплены усовершенствования, внедренные в J2SE 5.

Самая последняя версия Java называется Java SE 7, а соответствующий комплект инструментальных средств разработчиков — JDK 7. Данной версии присвоен внутренний номер 1.7. Java SE 7 — это первая основная версия Java, выпущенная после того, как компания Sun Microsystems Inc. была приобретена компанией Oracle (этот процесс начался в апреле 2009 года и завершился в январе 2010 года). В версии Java SE 7 появилось немало новых средств, в том числе существенные дополнения в сам язык и в библиотеки API. Усовершенствована также исполняющая система Java, в которой теперь поддерживаются программы, написанные на других языках программирования.

Наиболее важные средства, внедренные в версии Java SE 7 и рассматриваемые в этой книге, были разработаны в рамках проекта под названием Project Coin. В этом проекте преследовалась цель определить ряд незначительных изменений в языке Java, которые должны быть внедрены в JDK 7. И хотя эти изменения в целом называются “незначительными”, их последствия довольно значительны для разрабатываемых программ. На самом деле для многих программистов они могут стать самыми важными среди всех новых средств, появившихся в Java SE 7. Ниже перечислены новые языковые средства Java SE 7, рассматриваемые в этой книге.

- Возможность управлять оператором `switch` с помощью объектов типа `String`.
- Двоичные целочисленные литералы.
- Символы подчеркивания в числовых литералах.
- Расширенный оператор `try`, называемый *оператором try с ресурсами* и поддерживающий автоматическое управление ресурсами. (Например, файловый поток может быть теперь закрыт, если он больше не нужен.)
- Выводимость типов (посредством *ромбовидного* оператора) при построении обобщенного экземпляра объекта.

- Усовершенствованная обработка исключений, благодаря которой два или более исключения могут быть перехвачены одним (многозаходным, как мышеловка) оператором `catch`, а также улучшенный контроль типов для исключений, которые генерируются повторно.

Как видите, средства, разработанные в проекте Project Coin в качестве незначительных языковых изменений, сулят выгоды, которые никак нельзя отнести к разряду “незначительных”. В частности, оператор `try` с ресурсами позволяет существенно сократить объем написанного кода.

Материал предыдущего издания этой книги был обновлен и отражает особенности версии Java SE 7. Новые средства, обновления и дополнения Java SE 7 обозначаются соответствующим образом на страницах нового издания книги.

Структура книги

Эта книга представляет собой учебное пособие, разделенное на 15 глав, в каждой из которых рассматриваются отдельные вопросы программирования на Java. Материал каждой последующей главы основывается на предыдущей. Отличительная особенность книги состоит в том, что в ней используется ряд специальных приемов, повышающих эффективность обучения.

Основные навыки и понятия

Каждая глава начинается с рассмотрения самых важных для программирования навыков, которыми нужно овладеть.

Упражнения для самопроверки

В конце каждой главы приведено упражнение для самопроверки, позволяющее читателю проверить приобретенные им знания. Ответы на вопросы к этим упражнениям приведены в приложении А.

Обращение к знатоку

На страницах книги вам будут встречаться врезки “Обращение к знатоку”. Они содержат дополнительные сведения или любопытные замечания по рассматриваемой теме в форме вопросов и ответов.

Примеры для опробования

В каждой главе содержится один или несколько примеров для опробования, которые представляют собой несложные проекты для закрепления полученных знаний на практике. Как правило, это реальные примеры, с которых можно начинать разработку собственных прикладных программ.

Навыки программирования

Для чтения этой книги никаких особых навыков программирования не требуется. Если же вы программировали прежде, вам будет проще усваивать материал этой книги. Но, поскольку Java имеет ряд принципиальных отличий от других распространенных языков программирования, не спешите с выводами. Даже если у вас имеется немалый опыт программирования, внимательно относитесь к изучению материала этой книги.

Необходимое программное обеспечение

Для компиляции и запуска программ, исходные коды которых представлены в этой книге, вам потребуется последняя версия комплекта Java Development Kit — (JDK). На момент написания данной книги это был комплект JDK 7 от компании Oracle для версии Java SE 7. О том, как найти и установить такой комплект, речь пойдет в главе 1.

Если вы пользуетесь более ранней версией Java, например Java 5, то и в этом случае сможете извлечь пользу из этой книги. Но вам не удастся скомпилировать и выполнить те программы, в которых используются новые функциональные возможности Java.

Исходный код примеров программ

Имейте в виду, что исходный код всех примеров программ и проектов, представленных в этой книге, свободно доступен по адресу www.oraclepressbooks.com.

Особая благодарность

Особая благодарность выражается Дэнни Кауэрду — научному рецензенту настоящего издания книги. Его советы, предложения и идеи оказались очень ценными и дельными.

Другая литература на данную тему

Эта книга — лишь одна из целого ряда книг по программированию, написанных Гербертом Шилдтом. Возможно, вас заинтересуют и другие труды этого автора.

Тем, кто хочет больше узнать о программировании на Java, рекомендуются следующие книги данного автора.

- *Полный справочник по Java* (ИД “Вильямс”, 2012 г.)
- *Java. Методики программирования Шилдта* (ИД “Вильямс”, 2008 г.)
- *Искусство программирования на Java* (ИД “Вильямс”, 2005 г.)
- *SWING: руководство для начинающих* (ИД “Вильямс”, 2007 г.)

Если хотите изучить C++, вам помогут перечисленные ниже книги.

- *Полный справочник по C++* (ИД “Вильямс”, 2007 г.)
- *C++. Руководство для начинающих* (ИД “Вильямс”, 2005 г.)
- *STL Programming From the Ground Up* (об основах программирования на C++ средствами стандартной библиотеки шаблонов; издательство Osborne/McGraw-Hill, 1998 г.)
- *C++. Методики программирования Шилдта* (ИД “Вильямс”, 2009 г.)

Если вас интересует язык C#, обратите внимание на следующие книги Г. Шилдта.

- *Полный справочник по C# 4.0* (ИД “Вильямс”, 2010 г.)
- *C#. Руководство для начинающих 3.0* (ИД “Вильямс”, 2008 г.)

И наконец, тем, кто хочет изучить язык C, можно порекомендовать книгу *Полный справочник по C* (ИД “Вильямс”, 2007 г.)

За дополнительными рекомендациями обращайтесь непосредственно к Герберту Шилдту — признанному авторитету в области программирования.

От издательства

Вы, читатель этой книги, и есть главный ее критик и комментатор. Мы ценим ваше мнение и хотим знать, что было сделано нами правильно, что можно было сделать лучше и что еще вы хотели бы увидеть изданным нами. Нам интересно услышать и любые другие замечания, которые вам хотелось бы высказать в наш адрес.

Мы ждем ваших комментариев и надеемся на них. Вы можете прислать нам бумажное или электронное письмо либо просто посетить наш веб-сайт и оставить свои замечания там. Одним словом, любым удобным для вас способом дайте нам знать, нравится или нет вам эта книга, а также выскажите свое мнение о том, как сделать наши книги более интересными для вас.

Посылая письмо или сообщение, не забудьте указать название книги и ее авторов, а также ваш обратный адрес. Мы внимательно ознакомимся с вашим мнением и обязательно учтем его при отборе и подготовке к изданию последующих книг.

Наши электронные адреса:

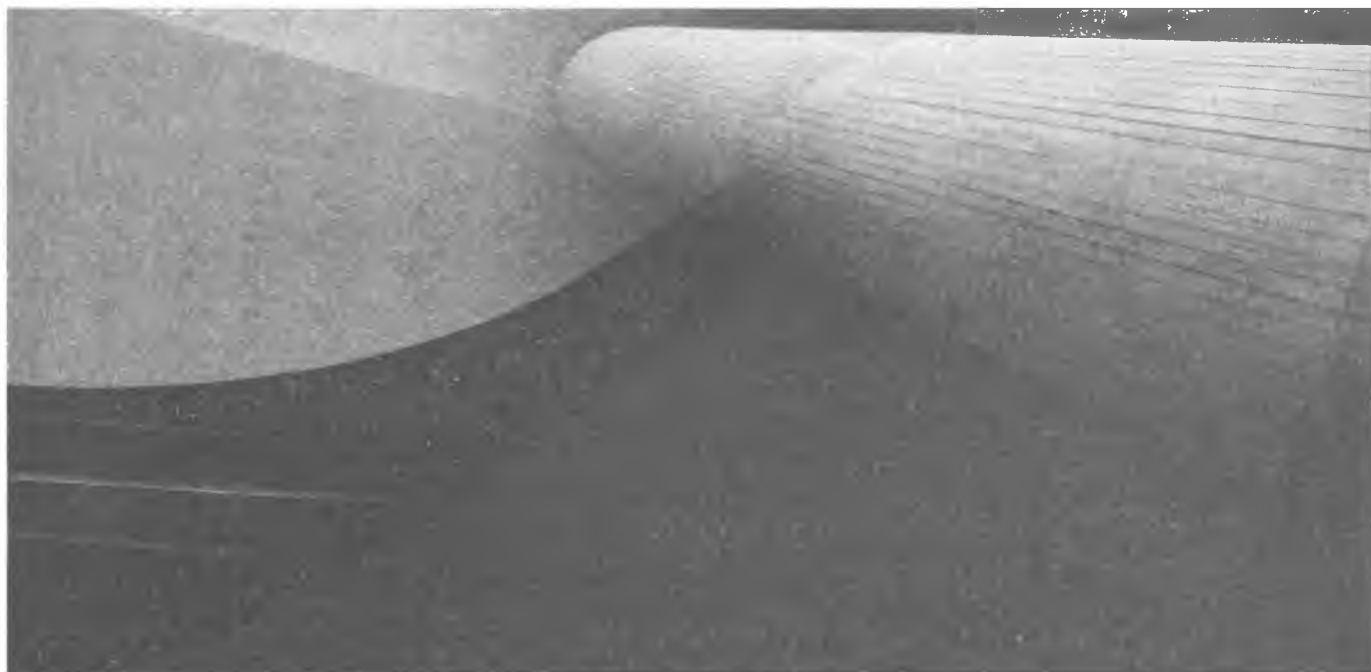
E-mail: info@williamspublishing.com

Веб: <http://www.williamspublishing.com>

Наши почтовые адреса:

России: 127055, г. Москва, ул. Лесная, д. 43, стр. 1

Украины: 03150, Киев, а/я 152



Глава 1

Основы Java

Основные навыки и понятия

- История развития и концепция Java
- Влияние Java на развитие Интернета
- Назначение байт-кода
- Основные свойства Java
- Основные принципы объектно-ориентированного программирования
- Написание, компиляция и выполнение простой программы на Java
- Использование переменных
- Использование управляющих операторов `if` и `for`
- Создание кодовых блоков
- Расположение в строке, отступление от края и завершение операторов
- Ключевые слова Java
- Правила применения идентификаторов Java

Развитие Интернета и, в частности, Всемирной паутины (или просто веб) оказало существенное влияние на состояние вычислительной техники. До появления веб в распоряжении большинства пользователей были лишь персональные компьютеры, работавшие независимо друг от друга. В настоящее время почти все они подключены к Интернету, который в свою очередь претерпел существенные изменения. Если первоначально Интернет предоставлял лишь удобную возможность обмениваться информацией в файлах или иным способом, то теперь он превратился в разветвленную среду распределенных вычислений. Подобные изменения привели к появлению нового подхода к программированию, основанного на языке Java.

Язык Java особенно удобен для написания интернет-приложений, но область его применения этим не ограничивается. Появление Java стало коренным переломом в программировании, поскольку изменило наше представление о форме и назначении программ. В настоящее время программист не может считать себя профессионалом, если он не умеет писать программы на Java. Проработав материал настоящей книги, вы сможете овладеть основами этого языка программирования.

Данная глава служит введением в язык Java. В ней представлена краткая история развития Java, концепция разработки программ и некоторые наиболее важные средства этого языка. Начинающим труднее всего уяснить, что в языке программирования не существует независимых элементов, но все они взаимосвязаны. Такая взаимосвязанность характерна и для Java. В самом деле, очень трудно рассматривать какой-то один аспект Java в отрыве от других. И для того чтобы преодолеть подобное затруднение, в этой главе дается краткий обзор ряда языковых средств Java, включая общую форму программы на Java, основные управляющие структуры и операторы. Не вдаваясь в детали, уделим основное внимание общим понятиям и принципам построения всех программ на Java.

Истоки Java

Главными причинами, побуждающими к созданию нового языка программирования, служат совершенствование искусства программирования или изменения в вычислительной среде. И Java не является исключением из этого правила. Опираясь на богатое наследие С и С++, этот язык программирования уточнен и дополнен средствами, отражающими текущее положение дел в программировании. Отвечая потребностям возникшей интерактивной среды, Java предоставляет средства, упрощающие создание прикладных программ с сильно распределенной архитектурой.

Язык Java был задуман в 1991 году сотрудниками компании Sun Microsystems Джеймсом Гослингом (James Gosling), Патриком Нотоном (Patrick Naughton), Крисом Уортом (Chris Warth), Эдом Фрэнком (Ed Frank) и Майком Шериданом (Mike Sheridan). Сначала этот язык получил имя Oak, но в 1995 году он был переименован в Java. Как ни странно, разработчики языка первоначально не предназначали его для разработки интернет-приложений. Они стремились создать платформенно-независимый язык, на котором можно было бы создавать встраиваемое программное обеспечение для различной бытовой аппаратуры с микропроцессорным управлением, в том числе тостеров, микроволновых печей и пультов дистанционного управления. В таких устройствах применялись контроллеры на микропроцессорах разной архитектуры, а исполняемый код, генерируемый компиляторами большинства существовавших в то время языков программирования, был ориентирован на конкретный процессор. Характерным тому примером может служить язык С++.

Несмотря на то что программу, написанную на С++, можно выполнить на процессоре практически любого типа, сделать это можно, лишь скомпилировав ее в исполняемый код команд конкретного процессора. Создание компиляторов — длительный и трудоемкий процесс, поэтому в поисках оптимального решения Гослинг и другие члены рабочей группы остановились на межплатформенном языке, для которого компилятор генерировал бы код, способный выполняться на разных процессорах в различных вычислительных средах. В конце концов их усилия увенчались созданием языка, известного теперь под названием Java.

В то время как разработчики Java уточняли детали созданного ими языка, начала постепенно развиваться Всемирная паутина (веб), во много определившая его будущее. Если бы не веб, Java, вероятно, нашел бы лишь ограниченное применение, главным образом в разработке программ, встраиваемых в специализированные контроллеры. Но как только веб стал реальностью, появилась острая потребность в переносимых программах, что и послужило основной побудительной причиной для выдвижения Java на передний план в качестве основного языка разработки подобных программ.

Набираясь опыта, многие программисты очень быстро приходят к выводу, что переносимость программ — это нечто вроде труднодостижимого идеала. Задача создания межплатформенных программ возникла едва ли не вместе с появлением первых компьютеров, но взяться за ее решение так и не удавалось из-за необходимости решать другие более важные и неотложные задачи. Тем не менее с появлением Интернета и веб задача переносимости программ перешла в разряд совершенно неотложных. Ведь Интернет состоит из множества разнотипных компьютеров с различной архитектурой процессоров и разными операционными системами.

В итоге увлекательная, но маловажная задача неожиданно стала чрезвычайно актуальной. В 1993 году разработчикам Java стало ясно, что задачу переносимости нужно решать не только при программировании микропроцессорных устройств, но и при создании кода

для интернет-приложений. Иными словами, сфера применения языка Java внезапно расширилась. И если программирование микроконтроллеров стало побудительной причиной для создания Java, то Интернет способствовал широкому распространению этого языка.

Взаимосвязь Java с языками C и C++

Язык Java очень похож на языки C и C++. От C язык Java унаследовал синтаксис, а от C++ — объектную модель. Сходство Java с языками C и C++ очень важно. Во-первых, многие программисты знакомы с синтаксисом C и C++, что упрощает изучение языка Java. Те же, кто освоил Java, могут без труда изучить C и C++.

И во-вторых, программирующим на Java не приходится изобретать колесо. Они могут успешно применять известные и хорошо зарекомендовавшие себя подходы. Современная эпоха в программировании, по существу, началась с языка C. Затем появился язык C++, а после него — Java. Имея такое богатое наследство, Java предоставляет программистам производительную и логически согласованную среду, в которой были реализованы лучшие из уже известных решений и добавлены новые средства, необходимые для интерактивной среды. Очень важно отметить тот факт, что вследствие своей схожести языки C, C++ и Java сформировали концептуальную основу для профессионального программирования. При переходе от одного языка к другому программистам не приходится преодолевать глубокие пропасти принципиального характера.

Один из принципов проектирования, положенных в основу C и C++, отдает ведущую роль программисту. Разработчики Java также следовали этому принципу. Если не учитывать ограничения, накладываемые средой Интернета, то следует признать, что Java предоставляет программисту полный контроль над кодом. Если вы умеете грамотно программировать, это будет видно по вашим программам. Недостаток опыта также отразится на ваших программах. Одним словом, Java — язык не для дилетантов, а для профессионалов.

У Java имеется еще одно сходство с языками C и C++: все эти языки были задуманы, разработаны, проверены и уточнены программистами-практиками. В их основу положены реальные потребности их создателей. При таком подходе к разработке языка программирования велика вероятность получить качественный продукт, способный найти признание у специалистов.

Из-за сходства языков Java и C++, в особенности из-за подобия предоставляемых ими средств для объектно-ориентированного программирования, возникает соблазн рассматривать Java как своего рода версию C++ для Интернета. Но это было бы ошибкой. У Java имеется целый ряд существенных отличий от C++ как в концептуальном, так и в практическом плане. Несмотря на то что C++ оказал очень сильное влияние на Java, последний совсем не является расширенной версией первого. В частности, эти языки не совместимы ни сверху вниз, ни снизу вверх. Конечно, сходство с языком C++ очень важно, и если у вас имеется опыт программирования на C++, вы будете чувствовать себя как дома, программируя на Java. Но не следует забывать, что Java был разработан не на замену C++, а для решения вполне определенного круга задач, отличающихся от тех, что решаются с помощью C++. Именно поэтому мирное сосуществование Java и C++ будет продолжаться еще многие годы.

Взаимосвязь Java с языком C#

Через несколько лет после создания Java корпорация Microsoft разработала язык C#. И это очень важно, поскольку C# тесно связан с Java. В самом деле, у многих языковых средств C# имеются свои аналоги в Java. В Java и C# используется единый общий син-

таксис, похожий на C++, поддерживается распределенное программирование и применяется одна и та же объектная модель. Разумеется, у Java и C# имеются отличия, но внешне эти языки очень похожи. Это означает, что, зная C#, вы сможете относительно просто изучить Java, и, наоборот, если вам предстоит изучить C#, знание Java может очень пригодиться.

Учитывая сходство Java и C#, может возникнуть вопрос: “Заменит ли C# язык Java?” Ответ, безусловно, отрицательный. Java и C# оптимизированы для абсолютно разных типов вычислительных сред. Как и вместе с языком C++, Java будет мирно сосуществовать с языком C# еще многие годы.

Вклад Java в развитие Интернета

Развитие Интернета послужило основной побудительной причиной для выхода Java на передний край программирования. В свою очередь, Java оказал благотворное влияние на развитие Интернета. Этот язык не только упростил веб-программирование, но и положил начало новой разновидности сетевых программ, называемых *апплетами* и полностью изменивших представление о содержимом веб. Java также позволил решить наиболее сложные задачи, возникающие при создании сетевых программ: обеспечение переносимости и безопасности. Рассмотрим подробнее апплеты и их основные свойства.

Java-апплеты

Апплет — это специальная программа на Java, предназначенная для передачи по Интернету и автоматического выполнения в среде, формируемой совместимым с Java браузером. Апплет загружается по требованию клиентской программы, а для его передачи по сети вмешательство пользователя не требуется. Если пользователь щелкает на ссылке, указывающей на документ, который содержит апплет, последний будет автоматически скопирован и запущен браузером. Большинство апплетов невелики по размерам. Обычно они служат для отображения информации, предоставляемой серверами, или поддержки ввода данных пользователем. Иногда с их помощью реализуются несложные функции. Например, калькулятор удобнее разместить в виде апплета на стороне клиента, чем выполнять вычисления на стороне сервера. Таким образом, апплет позволяет переносить некоторые функции с сервера на машину клиента.

С появлением апплетов расширился круг объектов, пригодных для свободной передачи в сетевой среде. Существуют две категории объектов, которыми сервер может обмениваться с клиентом: статические, пассивные данные и исполняемые программы (динамические данные). Например, просматривая электронную почту, вы имеете дело со статическими данными. Даже если в почтовом отправлении пересылается программа, ее код не активизируется до тех пор, пока не получит управление. Апплет, напротив, является динамической, самостоятельно выполняющейся программой, для запуска которой не приходится принимать никаких мер. Такие программы выполняют роль активных агентов на клиентских машинах, но инициализируются сервером.

Несмотря на то что с помощью динамических программ можно реализовать полезные функции, они могут вызвать серьезные затруднения, имеющие отношение к безопасности и переносимости. Очевидно, что программа, которая загружается на машину клиента и автоматически запускается, не должна наносить вред клиентской системе. Кроме того, она должна допускать выполнение в разнотипных средах: на разных платформах под управлением различных операционных систем. Как будет показано далее, в Java эти затруднения изящно и эффективно преодолены.

Безопасность

Как известно, запуск обычной программы, загруженной через Интернет, сопряжен с риском, поскольку она может быть заражена вирусом или служить своего рода “троянским конем” для злонамеренного проникновения в систему. А злонамеренные действия такой программы возможны из-за того, что она получает несанкционированный доступ к системным ресурсам. Так, вирус, анализируя содержимое файловой системы локального компьютера, может собирать секретные сведения, например номера платежных карточек, сведения о банковских счетах и пароли. Для безопасной загрузки и запуска апплетов на клиентской машине необходимо устранить саму возможность атаки на систему со стороны апплета.

Защита от атак реализуется путем создания специальной среды для выполнения апплета, не позволяющей ему обращаться к ресурсам компьютера. При отсутствии такой среды апплет выполняться не может. (Далее будет показано, как решается подобная задача.) Возможность загружать апплет с уверенностью в том, что он не нанесет вреда системе, относится к одним из самых привлекательных особенностей Java.

Переносимость

Переносимость является важным свойством сетевых программ. Она имеет большое значение потому, что в сети могут присутствовать разнотипные компьютеры, работающие под управлением различных операционных систем. Если программа на Java предназначена для выполнения на произвольном компьютере, подключенном к Интернету, то должны существовать способы обеспечения работы этой программы в различных системах. Например, один и тот же апплет должен работать на машинах с разнотипными процессорами, в разных операционных системах и с различными браузерами. Хранить разные версии апплета для разнотипных компьютеров слишком сложно, если вообще возможно. *Один и тот же* код должен работать на *всех* машинах. Таким образом, необходима поддержка процесса генерации переносимого исполняемого кода. Как станет ясно в дальнейшем, те же самые средства, которые обеспечивают безопасность, помогают добиться переносимости программ.

Волшебный байт-код Java

Добиться безопасности и переносимости программ по сети позволяет генерируемый компилятором Java код, не являющийся исполняемым. Такой код называется *байт-кодом*. Это оптимизированный набор команд, предназначенных для выполнения в исполняющей системе Java, называемой *виртуальной машиной Java* (Java Virtual Machine — JVM). Виртуальная машина Java, по существу, представляет собой *интерпретатор байт-кода*. Такой подход может показаться не совсем обычным, поскольку для повышения производительности компиляторы большинства современных языков генерируют исполняемый код. Но выполнение программы под управлением виртуальной машины позволяет разрешить многие затруднения, возникающие в работе веб-приложений.

Преобразование исходного кода Java в байт-код существенно упрощает перенос программ из одной среды в другую, поскольку для обеспечения работоспособности кода достаточно реализовать на каждой платформе виртуальную машину. Если на компьютере присутствует пакет исполняющей системы, то на нем может работать любая программа, написанная на Java. Несмотря на то что виртуальные машины на различных платформах могут быть реализованы по-разному, они должны одинаково интерпретировать байт-код. Если бы исходный текст программы на Java компилировался в собственный код,

для каждого типа процессора, взаимодействующего с Интернетом, необходимо было бы предусмотреть отдельную версию данной программы. Такое решение нельзя назвать приемлемым. Следовательно, выполнение байт-кода под управлением виртуальной машины — самый простой путь к обеспечению переносимости программ.

Выполнение программы под управлением виртуальной машины помогает также обеспечить безопасность. Виртуальная машина может запретить программе выполнять операции, побочные эффекты которых способны повлиять на ресурсы за пределами исполняющей системы. Кроме того, безопасность достигается посредством некоторых ограничений, предусмотренных в языке Java.

Как правило, интерпретируемая программа выполняется медленнее, чем скомпилированная в машинный код. Но для кода Java отличия в быстродействии не очень существенны. Ведь байт-код оптимизирован, и поэтому программа выполняется под управлением виртуальной машины значительно быстрее, чем следовало бы ожидать.

Несмотря на то что Java был задуман как интерпретируемый язык, ничто не мешает преобразовать байт-код по ходу компиляции в собственный для повышения производительности. С этой целью сразу же после первой реализации JVM компания Sun Microsystems начала работу над технологией HotSpot, в рамках которой был разработан *динамический компилятор* байт-кода. Если в состав виртуальной машины входит динамический компилятор, байт-код по частям преобразуется в собственный исполняемый код. Преобразовывать сразу всю программу на Java в исполняемый код нецелесообразно из-за разнообразных проверок, которые могут производиться только на этапе выполнения программы. Поэтому динамический компилятор выполняет преобразования кода частями по мере необходимости (отсюда и его другое название — *JIT-компилятор*, т.е. компилятор, вступающий в действие лишь в нужный момент времени). Более того, компиляции подвергаются не все фрагменты байт-кода, а лишь те, скорость выполнения которых можно повысить благодаря компиляции, а остальной код интерпретируется. Несмотря на все ограничения, присущие динамической компиляции, она, тем не менее, позволяет существенно повысить производительность программ. И невзирая на динамическое преобразование байт-кода в исполняемый код, переносимость и защита сохраняются, поскольку JVM по-прежнему участвует в процессе выполнения программ.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Мне приходилось слышать о специальных программах на Java, называемых сервлетами. Что это такое?

ОТВЕТ. *Сервлет* — это небольшая программа, выполняемая на стороне сервера. Подобно тому как апплеты динамически расширяют функциональные возможности веб-браузера, сервлеты динамически реализуют дополнительные функции веб-сервера. Как бы ни были важны апплеты, они реализуют только одну часть архитектуры “клиент-сервер”. Прошло совсем немного времени с момента появления Java, как специалистам стало ясно, что этот язык может оказаться полезным и на стороне сервера. В результате появились сервлеты. Благодаря сервлетам процесс взаимодействия клиентов и серверов может полностью поддерживаться в программах на Java. Вопросы функционирования сервлетов выходят за рамки книги для начинающих, но в дальнейшем вам будет полезно изучить этот тип программ. Более подробно о сервлетах можно узнать из книги *Полный справочник по Java*; ИД “Вильямс”, 2012 г.

Основные свойства Java

Даже самый краткий обзор языка Java будет неполным без упоминания его основных свойств. И хотя главной причиной, побудившей к разработке Java, послужила потребность в языке, позволяющем создавать переносимые и защищенные программы, заметное влияние на оформление Java в окончательном виде оказали и другие факторы. Ниже вкратце перечислены основные свойства этого языка программирования.

Простота	Java обладает лаконичными, тесно связанными друг с другом и легко усваиваемыми языковыми средствами
Безопасность	Предоставляет безопасные средства для создания интернет-приложений
Переносимость	Программы на Java могут выполняться в любой среде, для которой имеется исполняющая система Java
Объектно-ориентированный характер	Воплощает современную концепцию объектно-ориентированного программирования
Надежность	Стимулирует безошибочное программирование благодаря строгому контролю типов и проверкам во время выполнения программ
Многопоточность	Обеспечивает встроенную поддержку многопоточного программирования
Архитектурная независимость	Не привязан к конкретному типу вычислительной машины или архитектуре операционной системы
Интерпретируемость	Предоставляет байт-код, обеспечивающий независимость от платформы
Высокая производительность	Байт-код Java сильно оптимизирован на повышение скорости выполнения
Распределенность	Разработан для применения в распределенной среде Интернета
Динамичность	Программы на Java содержат значительную долю информации, используемой во время выполнения для проверки и разрешения доступа к объектам

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Если задачи переносимости и безопасности оказались столь актуальными, то зачем для их решения нужно было создавать новый язык программирования вроде Java. Не проще ли было адаптировать для этой цели уже существующий язык, например C++? Иными словами, можно ли создать компилятор C++, который генерировал бы байт-код?

ОТВЕТ. В самом деле, вполне возможно создать компилятор C++, который вместо исполняемого кода генерировал бы байт-код, но язык C++ обладает рядом свойств, препятствующих его использованию для разработки интернет-приложений. Самым важным из них является поддержка указателей. *Указатель* содержит адрес некоторого объекта, находящегося в памяти. С помощью указателей можно обращаться к ресурсам за пределами программы, что создает брешь в системе защиты. В Java указатели не поддерживаются, а следовательно, и не возникает никаких осложнений, связанных с нарушением защиты.

Объектно-ориентированное программирование

Одним из главных свойств Java является поддержка объектно-ориентированного программирования (ОП). Объектная методология неотделима от Java, а все программы на Java в той или иной степени являются объектно-ориентированными. Поэтому имеет смысл кратко рассмотреть принципы ОП, прежде чем переходить к написанию даже самой простой программы на Java.

Объектно-ориентированный подход к программированию позволяет разрабатывать достаточно сложные программы. С момента появления первого компьютера методология программирования претерпела ряд существенных изменений, связанных с возрастанием сложности программ. На заре вычислительной техники процесс программирования представлял собой ввод машинных команд в двоичной форме с пульта управления ЭВМ. В то время размеры программ не превышали нескольких сотен команд, и поэтому такой подход считался вполне приемлемым. Затем появился язык ассемблера. Символьное представление машинных команд и процедура компиляции позволили перейти к созданию более сложных программ. В связи с дальнейшим увеличением объема программного кода появились языки высокого уровня. Они стали теми инструментами, которые позволили программистам справиться с постепенным усложнением программ. Первым из широко распространенных языков высокого уровня стал FORTRAN. Разработка FORTRAN стала важным этапом в развитии языков программирования, но этот язык не вполне подходил для создания удобочитаемых программ.

В 1960-е годы начало зарождаться структурное программирование. Впоследствии для поддержки данного подхода были созданы такие языки, как C и Pascal. Благодаря структурированным языкам программирования появилась возможность очень просто создавать программы средней сложности. Главными свойствами структурированных языков стали поддержка независимых подпрограмм, локальных переменных, наличие расширенного набора управляющих конструкций и отсутствие оператора GOTO. Но, несмотря на то что структурированные языки стали мощными инструментами программирования, с увеличением объема и сложности проектов их возможности были быстро исчерпаны.

На каждом очередном этапе развития методологии и инструментальных средств программирования разработчики получали возможность создавать все более сложные программы. На этом пути очередной подход наследовал лучшие черты своих предшественников, а кроме того, он приобретал новые качества, позволявшие двигаться вперед. К моменту разработки принципов ОП многие проекты стали настолько сложными, что управлять ими средствами структурного программирования уже не представлялось возможным. Объектно-ориентированная методология позволила разработчикам преодолеть эти препятствия.

Создатели объектно-ориентированного программирования переняли лучшие идеи у структурного программирования и дополнили их новыми понятиями. В результате возник новый способ организации программ. В принципе программы могут создаваться двумя путями: на основе кода (выполняющего действия) и на основе данных (подвергающихся обработке). При использовании только принципов структурного программирования программы организуются на основе кода. Такой подход можно рассматривать как код, воздействующий на данные.

Объектно-ориентированное программирование подразумевает другой подход. Программы организуются на основе данных по следующему главному принципу: данные управляют доступом к коду. В объектно-ориентированных языках программирования определяются данные и процедуры, которым разрешается обрабатывать эти данные. Таким образом, тип данных определяет те операции, которые применимы к этим данным.

Во всех объектно-ориентированных языках программирования, в том числе и в Java, поддерживаются три основных принципа ООП: инкапсуляция, полиморфизм и наследование. Рассмотрим каждый из этих принципов в отдельности.

Инкапсуляция

Инкапсуляция представляет собой механизм программирования, объединяющий код и данные, которыми он манипулирует. Он предотвращает несанкционированный доступ к данным извне и их некорректное использование. В объектно-ориентированных языках программирования код и данные организуются в некое подобие “черного ящика”. В результате такого объединения кода и данных создается объект. Иными словами, объект — это компонент, поддерживающий инкапсуляцию.

Данные и код внутри объекта могут быть *закрытыми* (*private*) или *открытыми* (*public*). Закрытый код или данные доступны только элементам, содержащимся в том же самом объекте. Поэтому обратиться к такому коду или данным вне пределов объекта невозможно. Если код или данные являются открытыми, то к ним можно обращаться из любой части программы (несмотря на то, что они находятся внутри объекта). Как правило, открытые элементы объекта используются для создания управляемого интерфейса к его закрытым элементам.

Основной языковой конструкцией, поддерживающей инкапсуляцию в Java, является *класс*. Классы будут подробнее рассматриваться далее в этой книге, но о них нужно сказать несколько слов уже теперь. Класс определяет тип объекта. В нем определяются как данные, так и код, выполняющий определенные действия над этими данными. В Java определение или так называемая *спецификация* класса служит для построения *объектов*. Объекты представляют собой экземпляры классов. Следовательно, класс — это ряд “чертежей”, по которым строится объект.

Код и данные в составе класса называются *членами* класса. Данные, определенные в составе класса, принято называть *переменными-членами* или *переменными экземпляра*, а код, выполняющий действия над этими данными, — *методами-членами*, или просто *методами*. *Метод* — это термин, которым в Java принято обозначать подпрограмму. Если вы знакомы с языками C/C++, то, вероятно, знаете, что в этих языках для той же самой цели служит термин *функция*.

Полиморфизм

Полиморфизм (от греческого слова, означающего “много форм”) — это свойство, позволяющее с помощью одного интерфейса обращаться к общему ряду действий. Конкретное действие определяется ситуацией. В качестве примера, позволяющего лучше понять принцип полиморфизма, можно привести руль автомобиля. Руль (т.е. интерфейс) остается одним и тем же, независимо от того, какой именно механизм рулевого управления применяется в автомобиле, будь то зубчатая, реечная передача или гидроусилитель. Таким образом, зная, как обращаться с рулем, вы можете управлять автомобилем любого типа.

Тот же самый принцип применяется и в программировании. Рассмотрим в качестве примера стек (структуру данных, организованных по принципу “последним пришел — первым обслужен”). Допустим, в программе требуются три разнотипных стека. Первый стек служит для хранения целочисленных значений, второй — для хранения значений с плавающей точкой и третий — для хранения символьных значений. Каждый стек реализуется с помощью одного и того же алгоритма, несмотря на то, что в стеках хранятся разнотипные данные. На языке, не поддерживающем ООП, пришлось бы создавать три

разных ряда процедур управления стеками, присвоив им разные имена. А на Java благодаря полиморфизму можно создать один общий ряд процедур управления стеками, который будет действовать по-разному в зависимости от конкретного типа стека. Таким образом, зная, как работать с одним стеком, можно обращаться со всеми тремя стеками.

Принцип полиморфизма хорошо иллюстрирует следующее выражение: “один интерфейс — множество методов”. Это означает возможность создания универсального интерфейса для группы взаимосвязанных действий. Полиморфизм упрощает программу благодаря возможности определить *общий класс действий* с помощью одного и того же интерфейса. Выбрать определенное действие (т.е. *метод*) — задача компилятора, и он решает ее в зависимости от конкретных условий. Как программисту вам не приходится выбирать метод вручную. Нужно лишь помнить основные принципы использования общего интерфейса.

Наследование

Наследование — это процесс, в ходе которого один объект приобретает свойства другого объекта. Наследование имеет очень большое значение, поскольку с его помощью поддерживается иерархическая классификация. Если вдуматься, то знания чаще всего имеют иерархическую структуру. Например, яблоки конкретного сорта относятся к классу яблок, который в свою очередь относится к классу фруктов, а тот — к более обширному классу съедобных плодов. В данном случае съедобные плоды обладают определенными свойствами (они питательны, не ядовиты и т.д.). Эти же свойства автоматически присущи подклассу фруктов. Кроме того, класс фруктов обладает дополнительными свойствами (сочные, сладкие и т.д.), что отличает его от класса других съедобных плодов. Яблоки имеют более конкретные свойства (растут на деревьях, не являются тропическими плодами и т.д.) И наконец, отдельный сорт яблок наследует все свойства описанных выше классов и, кроме того, обладает особыми свойствами, отличающими его от других сортов яблок.

Без такой иерархической структуры для каждого объекта пришлось бы заново определять весь набор свойств. Благодаря наследованию для объекта достаточно указать те свойства, которые отличают его от других классов, а остальные общие атрибуты он наследует от своих родительских классов. Таким образом, благодаря наследованию возможно создать объект, являющийся экземпляром более общего класса.

Установка комплекта Java Development Kit

Итак, усвоив теоретические основы Java, можно приступать к написанию программ на Java. Для того чтобы написанную программу можно было скомпилировать и запустить на выполнение, вам нужно установить на своем компьютере комплект Java Development Kit, поддерживающий процесс разработки программ на Java. Этот комплект свободно предоставляется компанией Oracle. На момент написания данной книги доступна была его версия JDK 7, применяемая на платформе Java SE 7, где SE — стандартная версия. Комплект JDK 7 содержит немало новых средств, которые не поддерживаются в предыдущих версиях Java, и поэтому для компиляции и выполнения примеров программ, представленных в этой книге, потребуется JDK 7 или более поздняя версия данного комплекта.

Комплект JDK можно загрузить по адресу www.oracle.com/technetwork/java/javase/downloads/index.html. Для этого достаточно перейти на страницу загрузки по указанному адресу и следовать инструкциям для вашего типа компьютера и операционной системы. После установки JDK вы сможете приступить к компиляции и запуску

программ на выполнение. Среди многих программ, входящих в состав JDK, вам в первую очередь понадобятся две. Первая из них, **javac**, служит в качестве компилятора Java, а вторая, **java**, — в качестве стандартного интерпретатора Java и зачастую называется *загрузчиком приложений*.

Следует, однако, иметь в виду, что программы, входящие в состав JDK, запускаются из командной строки. Они не являются оконными приложениями и не составляют интегрированную среду разработки.

На заметку

Помимо основных инструментальных средств JDK, доступных из командной строки, для разработки программ на Java имеются высококачественные интегрированные среды вроде NetBeans и Eclipse. Интегрированная среда может оказаться очень удобной для разработки и развертывания коммерческих приложений. Как правило, в интегрированной среде можно по желанию скомпилировать и выполнить примеры программ, представленных в этой книге. Но инструкции по компиляции и выполнению этих примеров программ приводятся в книге только для инструментальных средств JDK, доступных из командной строки, ради простоты усвоения по целому ряду причин. Во-первых, комплект JDK легко доступен для всех читателей книги. Во-вторых, инструкции по применению JDK одинаковы для всех пользователей. А кроме того, простые примеры программ, представленные в этой книге, проще всего компилировать и выполнять инструментальными средствами JDK из командной строки. Если же вы пользуетесь интегрированной средой разработки, вам придется следовать ее инструкциям. А в силу отличий, имеющих у разных интегрированных сред разработки, дать общие инструкции по компиляции и выполнению программ, написанных на Java, не представляется возможным.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как пояснялось выше, объектно-ориентированное программирование — эффективный способ управления крупными проектами. Но его применение в небольших проектах влечет за собой значительные накладные расходы. Все программы на Java в той или иной степени являются объектно-ориентированными. Означает ли это, что данный язык неэффективен для написания небольших программ?

ОТВЕТ. Нет, не означает. Как будет показано далее, для небольших программ объектно-ориентированные средства Java практически прозрачны. Несмотря на то что язык Java строго придерживается объектной модели, вам решать, в какой степени применять его объектно-ориентированные средства. Так, для написания небольших программ объектно-ориентированный подход вряд ли целесообразен. А для создания крупных программ вы сможете без труда применять более развитые средства ООП.

Первая программа на Java

Попробуйте скомпилировать и запустить простую программу, исходный код которой приведен ниже.

/*

Это пример простой программы на Java.

Присвоить ее исходному файлу имя **Example.java**.

```
*/  
class Example {  
    // Выполнение всякой программы на Java начинается  
    // с вызова метода main().  
    public static void main(String args[]) {  
        System.out.println("Java drives the Web.");  
    }  
}
```

Итак, выполните следующие действия.

1. Введите исходный код программы.
2. Скомпилируйте программу.
3. Запустите программу на выполнение.

Ввод исходного кода программ

Исходные коды примеров программ, представленных в этой книге, доступны на веб-сайте издательства McGraw-Hill по адресу www.oraclepressbooks.com. Перейдите по указанному адресу на главную страницу этого веб-сайта, а затем по ссылке Downloads⇒Code⇒Java A Beginner's Guide, Fifth Edition и далее следуйте инструкциям по загрузке архивного файла с исходными кодами программ. Но исходный код можно ввести и вручную, воспользовавшись простым текстовым редактором, а не текстовым процессором вроде Word, поскольку вместе с исходным текстом программы он записывает в файл сведения о форматировании текста, которые будут восприняты компилятором как недопустимые языковые конструкции. Если вы работаете на платформе Windows, вам вполне подойдет WordPad или другой простой редактор текста.

В большинстве языков программирования допускается присваивать произвольное имя файлу, содержащему исходный код программы. Но в Java действуют иные правила. Для программирования на Java следует знать, что *имя исходного файла* имеет очень большое значение. В рассматриваемом здесь примере файлу, содержащему исходный код программы, нужно присвоить имя `Example.java`. Ниже поясняются причины, по которым выбирается именно такое имя файла.

В Java исходный файл формально называется *единицей компиляции*. Это текстовый файл, содержащий определения одного или нескольких классов. (Исходные файлы первых примеров программ будут содержать только один класс.) Компилятор Java требует, чтобы исходный файл имел расширение `.java`. Анализируя исходный код первого примера программы, вы заметите, что класс имеет имя `Example`. И это не случайно. В программах на Java весь код должен находиться внутри класса. Условно принято, что имя класса должно совпадать с именем файла, содержащего исходный текст программы. Нетрудно убедиться, что имя файла в точности соответствует имени класса вплоть до регистра. Дело в том, что в Java имена и другие идентификаторы зависят от регистра символов. На первый взгляд, условие соответствия имен классов и файлов может показаться слишком строгим, но оно упрощает организацию и сопровождение программ, как станет ясно из последующих примеров.

Компиляция программы

Для компиляции программы `Example` запустите компилятор `javac`, указав в командной строке имя исходного файла, как показано ниже.

```
C:\>javac Example.java
```


Компилятор **javac** создаст файл `Example.class`, содержащий байт-код программы. Напомним, что байт-код не является исполняемым, но интерпретируется виртуальной машиной Java. Таким образом, результат компиляции в **javac** нельзя запускать на выполнение непосредственно.

Для запуска скомпилированной программы следует воспользоваться интерпретатором **java**. В качестве параметра ему нужно передать имя класса `Example` в командной строке:

```
C:\>java Example
```

В результате выполнения программы на экран будет выведена следующая строка:

```
Java drives the Web.
```

При компиляции исходного кода Java каждый класс помещается в отдельный выходной файл, называемый по имени класса и получающий расширение `.class`. Именно по этой причине и выдвигается требование, чтобы имя исходного файла программы на Java в точности соответствовало имени содержащегося в нем класса, а по существу, — имени файла с расширением `.class`. При вызове интерпретатора Java ему передается имя класса, предназначенного для выполнения (пример такого вызова приведен выше). В результате интерпретатор автоматически находит файл с указанным именем и расширением `.class`. Обнаружив этот файл, интерпретатор выполняет код, указанный в классе.

На заметку

Если при попытке скомпилировать программу компилятор **javac** не будет найден, вам придется указать полный путь к инструментальным средствам JDK, доступным из командной строки, при условии, что комплект JDK установлен правильно. В Windows это, например, означает, что вам нужно ввести путь к доступным из командной строки инструментальным средствам JDK в переменной окружения `PATH`. Так, если комплект JDK 7 установлен в выбираемых по умолчанию каталогах, путь к его инструментальным средствам будет следующим: `C:\Program Files\Java\jdk1.7.0\bin`. (Разумеется, в этом пути может быть указана другая версия JDK.) Чтобы узнать, как задавать путь к файлам, обращайтесь к справочной системе конкретной операционной системы, поскольку в разных операционных системах эта процедура может отличаться. Но если вам не терпится скомпилировать и выполнить свою первую программу на Java, то укажите полный путь к соответствующим инструментальным средствам прямо в командной строке, как показано в приведенном ниже примере.

```
C:\>"Program Files"\Java\jdk1.7.0\bin\javac Example.java
```

или

```
C:\>"Program Files"\Java\jdk1.7.0\bin\java Example
```

Впрочем, такой способ не совсем практичен. Намного удобнее задать путь к нужным файлам раз и навсегда.

Построчный анализ исходного кода первого примера программы

Несмотря на то что программа `Example.java` очень проста и объем ее кода невелик, она обладает рядом свойств, общих для всех программ на Java. Проанализируем эту программу по частям.

Ее исходный код начинается следующими строками:

```
/*  
    Это пример простой программы на Java.  
    Присвоить ее исходному файлу имя Example.java.  
*/
```

Это *комментарии*. Как и в большинстве других языков программирования, в Java разрешается комментировать исходный текст программы. Комментарии игнорируются компилятором. Текст комментариев описывает действия, выполняемые программой, а в итоге исходный код становится более простым для восприятия. В данном случае комментарии сообщают, что программа проста, а ее исходному файлу следует присвоить имя **Example.java**. Очевидно, что в реальных приложениях комментарии описывают, какие именно функции выполняют отдельные фрагменты программы или зачем используются те или иные языковые конструкции.

В Java поддерживаются три вида комментариев. Первый из них приведен в начале рассматриваемой здесь программы и позволяет задавать *многострочные комментарии*. Они начинаются символами `/*` и оканчиваются символами `*/`. Любое содержимое, находящееся между этими ограничителями, игнорируется компилятором.

Далее следует приведенная ниже строка кода.

```
class Example {
```

В ней присутствует ключевое слово **class**, с помощью которого определяется новый класс. Как упоминалось ранее, класс является основной языковой конструкцией Java, поддерживающей инкапсуляцию. А **Example** — это имя класса. Определение класса начинается открывающей фигурной скобкой (`{`) и заканчивается закрывающей фигурной скобкой (`}`). Элементы, находящиеся между ними, являются членами класса. Не пытайтесь пока что разобраться в особенностях классов, но имейте лишь в виду, что любой код, какие бы действия он ни выполнял, находится внутри класса. Такая организация свидетельствует о том, что любая программа на Java является в той или иной мере объектно-ориентированной.

Следующая строка кода содержит *однострочные комментарии*, как показано ниже.

```
// Выполнение каждой программы на Java начинается  
// с вызова метода main().
```

Это второй вид комментариев, поддерживаемых в Java. Однострочные комментарии начинаются с символов `//` и продолжаются до конца строки. Как правило, многострочные комментарии служат для длинных примечаний, а однострочные — для коротких заметок.

Следующая анализируемая строка кода выглядит так, как показано ниже.

```
public static void main (String args[]) {
```

В этой строке определяется метод **main()**. Как упоминалось ранее, в терминологии Java подпрограммы принято называть *методами*. Именно с данной строки начинается работа программы, и приведенные выше комментарии подтверждают это. Выполнение любой прикладной программы на Java начинается с вызова метода **main()**. Мы не будем пока что касаться назначения каждого элемента анализируемой строки кода. Ведь для этого нужно сначала рассмотреть ряд других языковых средств Java. Но поскольку многие примеры программ, представленные в этой книге, содержат строку с подобным определением метода **main()**, рассмотрим вкратце ее составные части.

Ключевое слово **public** называется *модификатором доступа*. Модификатор доступа определяет правила обращения к членам класса из других частей программы. Если член класса предваряется ключевым словом **public**, то к нему можно обращаться за преде-

лами класса. (В отличие от `public`, модификатор доступа **`private`** запрещает доступ к членам класса за его пределами.) В данном случае метод `main()` должен быть объявлен как `public`, поскольку при выполнении программы он вызывается за пределами своего класса. Ключевое слово **`static`** допускает вызов метода `main()` до создания объекта класса. Указывать его необходимо, поскольку метод `main()` вызывается виртуальной машиной еще до того, как будут созданы какие-либо объекты. Ключевое слово **`void`** лишь сообщает компилятору о том, что метод `main()` не возвращает значение. Как будет показано далее, для некоторых методов предусматривается возвращаемое значение. Если вам пока еще не все понятно в анализируемой строке кода, не отчаивайтесь. Упомянутые здесь языковые средства Java будут подробно рассмотрены в последующих главах книги.

Как упоминалось выше, метод `main()` вызывается в начале выполнения программы на Java. Любые данные, которые требуется передать этому методу, задаются с помощью переменных, указываемых в круглых скобках после имени метода. Эти переменные называются *параметрами*. Если для какого-нибудь метода параметры не предусмотрены, то после его имени указывается лишь пара круглых скобок. В данном случае для метода `main()` под именем **`args`** задается единственный параметр `String args[]`. Это массив объектов типа `String`. (*Массивы* представляют собой наборы однотипных объектов.) В объектах типа `String` хранятся последовательности символов. В данном случае в массиве `args` методу `main()` передаются в виде аргументов параметры, указываемые в командной строке при запуске программы. В анализируемой здесь программе данные, получаемые из командной строки, не используются, но в других примерах программ будет показано, каким образом эти данные обрабатываются.

И завершается анализируемая строка кода символом `{`. Этот символ обозначает начало тела метода `main()`. Весь код, содержащийся в методе, располагается между открывающей и закрывающей фигурными скобками.

Очередная анализируемая строка кода приведена ниже. Обратите внимание на то, что она содержится в методе `main()`.

```
System.out.println("Java drives the Web.");
```

В этой строке кода на экран сначала выводится символьная строка "Java drives the Web." (Java приводит в действие веб), а затем происходит переход на новую строку. Вывод на экран осуществляется встроенным методом **`println()`**. В данном случае метод `println()` выводит на экран переданную ему символьную строку. Как будет показано далее, с помощью метода `println()` можно выводить на экран не только символьные строки, но и данные других типов. Анализируемая строка кода начинается с имен `System.out`. В настоящий момент объяснить их назначение нелегко, поэтому достаточно будет сказать, что **`System`** — это предопределенный класс, предоставляющий доступ к системным ресурсам, а **`out`** — поток вывода на консоль. Таким образом, **`System.out`** — это объект, инкапсулирующий вывод на консоль. Тот факт, что для определения консоли в Java используется объект, лишний раз свидетельствует об объектно-ориентированном характере этого языка.

Как вы, вероятно, уже догадались, в реальных программах и апплетах Java вывод на консоль (как, впрочем, и ввод с консоли) используется очень редко. Современные вычислительные среды, как правило, носят оконный и графический характер, и поэтому обмен данными с консолью применяется лишь в простых служебных и демонстрационных программах. Далее в этой книге будут рассмотрены другие способы формирования выходных данных средствами Java, а до тех пор ограничимся использованием методов ввода-вывода на консоль в представленных примерах программ.

Обратите внимание на то, что оператор, содержащий вызов метода `println()`, оканчивается точкой с запятой. Этим символом завершаются все операторы в Java. В других строках кода точка с запятой отсутствует лишь потому, что они формально не являются операторами.

Первая закрывающая фигурная скобка завершает метод `main()`, а вторая такая же скобка указывает на окончание определения класса `Example`.

И последнее замечание: в Java *учитывается* регистр символов. Забвение этого правила влечет за собой серьезные осложнения. Так, если вместо `main` вы случайно наберете **Main**, а вместо `println` — **PrintLn**, исходный код программы окажется некорректным. И хотя компилятор Java скомпилирует классы, не содержащие метод `main()`, у него не окажется средств для их выполнения. Следовательно, если вы допустите ошибку в имени `main`, компилятор скомпилирует программу и не сообщит об ошибке. Но о ней вас уведомит интерпретатор, когда не сумеет обнаружить метод `main()`.

Обработка синтаксических ошибок

Введите текст рассмотренной выше программы, скомпилируйте и запустите ее (если вы не сделали этого раньше). Если у вас имеется некоторый опыт программирования, то вам, безусловно, известно, что никто не гарантирован от ошибок при наборе исходного кода программы, а также о том, насколько просто допустить такие ошибки. Если вы наберете исходный код программы неправильно, компилятор, пытаясь обработать этот код, выведет сообщение об ошибке. Компилятор Java следует формальным правилам, и поэтому строка в исходном коде, на которую он указывает, не всегда соответствует истинному местоположению источника ошибки. Допустим, в рассмотренной ранее программе вы забыли ввести открывающую фигурную скобку после метода `main()`. В таком случае компилятор сообщит вам о следующих ошибках:

```
Example.java:8: ';' expected
    public static void main(String args[])
                               ^
Example.java:11: class, interface, or enum expected
}
^
```

Очевидно, что первое сообщение не соответствует действительности: пропущена не точка с запятой, а фигурная скобка.

Таким образом, если ваша программа содержит синтаксические ошибки, не стоит бездумно следовать рекомендациям компилятора. Сообщение об ошибке составляется по формальным правилам. Вам же нужно разобраться, каковы истинные причины ошибки. Желательно проанализировать строки кода программы, предшествующие строке, обозначенной компилятором. Иногда ошибку удастся распознать лишь после анализа нескольких строк кода, следующих за той, в которой она была на самом деле обнаружена.

Еще одна простая программа

Вероятно, наиболее широко употребительной конструкцией в любом языке программирования является оператор присваивания значения переменной. *Переменная* — это именованная ячейка памяти, в которой можно хранить присваиваемое значение. В процессе выполнения программы значение переменной может изменяться. Это означает, что содержимое переменной не фиксировано. В приведенной ниже программе используются две переменные — `var1` и `var2`.

```

/*
Продemonстрировать применение переменной.

Присвоить исходному файлу имя Example2.java.
*/
class Example2 {
    public static void main(String args[]) {
        int var1; // Объявление переменной.
        int var2; // Объявление еще одной переменной.

        var1 = 1024; // Присваивание значения 1024 переменной var1.

        System.out.println("var1 contains " + var1);

        var2 = var1 / 2;

        System.out.print("var2 contains var1 / 2: ");
        System.out.println(var2);
    }
}

```

Запустив эту программу на выполнение, вы получите следующий результат:

```

var1 contains 1024
var2 contains var1 / 2: 512

```

В этой программе представлен ряд новых понятий. Сначала в операторе

```
int var1; // Объявление переменной.
```

объявляется переменная `var1` целочисленного типа.

В Java каждая переменная должна быть объявлена перед ее использованием. При объявлении переменной задается ее тип, т.е. тип данных, которые могут в ней содержаться. В данном случае переменная `var1` может содержать целочисленные значения. Для этой цели при объявлении переменной перед ее именем указывается ключевое слово **int**. Таким образом, в приведенной выше строке кода объявляется переменная `var1` типа `int`.

В следующей строке кода объявляется вторая переменная `var2`:

```
int var2; // Объявление еще одной переменной.
```

Как видите, в этой строке кода используется та же самая форма объявления переменной, что и в предыдущей. А отличается она только именем переменной.

В общем, для объявления переменной служит следующий оператор:

```
тип имя_переменной;
```

где *тип* обозначает конкретный тип объявляемой переменной, а *имя_переменной* — ее наименование. Помимо `int`, в Java поддерживаются и другие типы данных.

В следующей строке кода переменной `var1` присваивается значение 1024:

```
var1 = 1024; // Присваивание значения 1024 переменной var1.
```

В Java оператор присваивания обозначается знаком равенства. Он копирует значение, находящееся справа от него, в переменную, указанную слева.

В следующей строке кода значение переменной `var1` выводится на экран после символической строки "var1 contains" (Переменная `var1` содержит):

```
System.out.println("var1 contains " + var1);
```

В этом операторе знак `+` указывает на то, что значение переменной `var1` должно быть выведено после символьной строки. Это правило имеет общий характер. Знаком `+` можно объединить несколько элементов в одном операторе, передав их в качестве параметра методу `println()`.

В следующей строке кода переменной `var2` присваивается значение переменной `var1`, разделенное на два:

```
var2 = var1 / 2;
```

После выполнения этого оператора переменная `var2` будет содержать значение **512**, тогда как значение переменной `var1` останется без изменения. Как и в большинстве других языков программирования, в Java поддерживаются арифметические операторы, в том числе и перечисленные ниже.

<code>+</code>	Сложение
<code>-</code>	Вычитание
<code>*</code>	Умножение
<code>/</code>	Деление

Рассмотрим следующие две строки кода:

```
System.out.print("var2 contains var1 / 2: ");
System.out.println(var2);
```

В первой из них вызывается метод `print()`, выводящий символьную строку `"var2 contains var1 / 2:"` (Переменная `var2` содержит значение переменной `var1 / 2:`) без последующего символа перевода строки. Это означает, что очередные данные будут отображены в той же строке. Метод `print()` действует таким же образом, как и метод `println()`, за исключением того, что его выполнение не завершается переходом на следующую строку. Во второй строке методу `println()` в качестве параметра передается переменная `var2`. С помощью обоих методов, `print()` и `println()`, можно выводить на экран значения любого встроенного в Java типа данных.

Прежде чем переходить к изучению других аспектов Java, необходимо обратить внимание на следующую особенность: в одном операторе можно объявить две и более переменные. Для этого достаточно разделить их имена запятыми. Например, переменные `var1` и `var2` можно объявить так, как показано ниже.

```
int var1, var2; // обе переменные объявляются в одном операторе
```

Другие типы данных

В предыдущем примере программы были использованы переменные типа `int`. Но они могут содержать только целые числа и не пригодны для хранения дробной части. Так, переменная типа `int` может содержать значение **18**, но не **18,3**. Но `int` — это лишь один из нескольких типов данных, определенных в Java. Для хранения чисел, содержащих дробную часть, в Java предусмотрены типы с плавающей точкой, `float` и `double`, которые представляют значения с одинарной и двойной точностью соответственно. Тип `double` используется чаще, чем `float`.

Для объявления переменной типа `double` следует использовать оператор, аналогичный приведенному ниже.

```
double x;
```

где **x** обозначает имя переменной типа `double`. А поскольку **x** объявлена как переменная с плавающей точкой, в ней можно хранить значения **122,23**, **0,034**, **-19,0** и т.д.

Для того чтобы стали понятнее отличия между типами `int` и `double`, рассмотрим следующую программу:

```
/*
  Эта программа демонстрирует отличия между типами int и double.
  Присвоить ее исходному файлу имя Example3.java.
*/
class Example3 {
    public static void main(String args[]) {
        int var; // объявить целочисленную переменную
        double x; // объявить переменную с плавающей точкой

        var = 10; // присвоить переменной var значение 10

        x = 10.0; // присвоить переменной x значение 10.0

        System.out.println("Original value of var: " + var);
        System.out.println("Original value of x: " + x);
        System.out.println(); // вывести пустую строку

        // а теперь разделить значения обеих переменных на 4
        var = var / 4;
        x = x / 4;

        System.out.println("var after division: " + var);
        System.out.println("x after division: " + x);
    }
}
```

Выполнение данной программы дает следующий результат:

Original value of var: 10

Original value of x: 10.0

var after division: 2 ← Дробная часть числа теряется

x after division: 2.5 ← Дробная часть числа сохраняется

Как видите, при делении значения переменной `var` на четыре результат получается дробным и программа выводит числовое значение **2**, а дробная его часть теряется. В то же время при делении на четыре значения переменной `x` дробная часть ее числового значения сохраняется и программа отображает правильный результат.

В этой программе показан еще один прием. Для вывода пустой строки на экран достаточно вызвать метод `println()` без параметров.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Почему для хранения целых и дробных значений используются разные типы? Не проще ли создать один универсальный тип для всех числовых значений?

ОТВЕТ. Для написания эффективных программ предусмотрены разные типы данных. Например, целочисленные вычисления производятся быстрее, чем вычисления

с плавающей точкой. Следовательно, если вы не пользуетесь дробными значениями, то и незачем навлекать дополнительные издержки, связанные с обработкой содержимого переменной `float` или `double`. Кроме того, для хранения переменных разных типов требуется различный объем памяти. Предоставляя разработчику разные типы данных, Java создает благоприятные условия для оптимального использования системных ресурсов. И наконец, для некоторых алгоритмов требуются конкретные типы данных (во всяком случае, они работают с ними более эффективно). Вообще говоря, разные встроенные типы предусмотрены в Java для обеспечения большей гибкости программирования.

Пример для опробования 1.1.

Преобразование галлонов в литры

Исходный файл: `GalToLit.java`

Предыдущие примеры программ иллюстрируют некоторые важные особенности языка программирования Java, но они вряд ли применимы на практике. Несмотря на то что ваши знания Java пока еще весьма ограничены, вам вполне по силам написать программу, выполняющую нечто полезное, в частности преобразование галлонов в литры.

В этой программе объявляются две переменные типа `double`. Одна из них содержит объем жидкости в галлонах, а вторая — тот же самый объем, но преобразованный в литры. В галлоне содержится 3,7854 литра. Поэтому для преобразования галлонов в литры следует умножить число галлонов на 3,7854. В результате выполнения данной программы объем жидкости отображается как в галлонах, так и в литрах.

Последовательность действий

1. Создайте новый файл `GalToLit.java`.
2. Введите в этот файл следующий код программы:

```
/*
    Пример для опробования 1.1

    Эта программа преобразует галлоны в литры.

    Присвоить ее исходному файлу имя GalToLit.java.
*/
class GalToLit {
    public static void main(String args[]) {
        double gallons; // в этой переменной хранится объем жидкости в галлонах
        double liters;   // в этой переменной хранится объем жидкости в литрах

        gallons = 10;    // исходное значение равно 10 галлонам

        liters = gallons * 3.7854; // преобразовать в литры

        System.out.println(gallons + " gallons is " + liters + " liters.");
    }
}
```

3. Скомпилируйте программу, указав в командной строке следующее:

```
C>javac GalToLit.java
```


4. Запустите программу на выполнение из командной строки следующим образом:

```
C>java GalToLit
```

В результате выполнения данной программы на экране появится следующий результат:

```
10.0 gallons is 37.854 liters.
```

5. Как видите, программа преобразует в литры объем жидкости, равный десяти галлонам. Изменив значение переменной `gallons`, обозначающее объем жидкости в галлонах, вы получите другой ее объем в литрах.

Два управляющих оператора

Операторы в методе выполняются по порядку сверху вниз. Но этот порядок можно изменить, воспользовавшись управляющими операторами, предусмотренными в Java. Подробнее об управляющих операторах речь пойдет далее в книге, а до тех пор ограничимся кратким рассмотрением двух таких операторов. Они будут использованы при написании примеров программ.

Оператор `if`

Используя условный оператор `if`, можно выборочно выполнять отдельные части программы. В Java этот оператор действует таким же образом, как и оператор `IF` из любого другого языка программирования. Ниже приведена простейшая форма оператора `if`.

```
if (условие) оператор;
```

где *условие* обозначает логическое выражение. Если *условие* истинно (принимает логическое значение `true`), *оператор* выполняется. А если *условие* ложно (принимает логическое значение `false`), *оператор* не выполняется. Ниже приведен простой пример употребления оператора `if` в коде.

```
if(10 < 11) System.out.println("10 is less than 11");
```

В данном примере числовое значение **10** меньше **11**, и поэтому условное выражение принимает логическое значение `true`, а следовательно, выполняется метод `println()`. Рассмотрим еще один пример с противоположным условием.

```
if(10 < 9) System.out.println("this won't be displayed");
```

Теперь числовое значение **10** не меньше **9**, а следовательно, метод `println()` не вызывается и на экран ничего не выводится.

В Java определен полный набор операторов сравнения, которые могут быть использованы при составлении условных выражений. Эти операторы приведены ниже.

Оператор	Значение
<code><</code>	Меньше
<code><=</code>	Меньше или равно
<code>></code>	Больше
<code>>=</code>	Больше или равно
<code>==</code>	Равно
<code>!=</code>	Неравно

Обратите внимание на то, что для проверки на равенство следует указать два знака равенства.

Ниже приведен пример программы, демонстрирующий применение оператора `if`.

Продемонстрировать применение оператора `if`.

Присвоить исходному файлу имя `IfDemo.java`.

```
*/
class IfDemo {
    public static void main(String args[]) {
        int a, b, c;
        a = 2;
        b = 3;

        if(a < b) System.out.println("a is less than b");
        // следующая строка не должна выводиться
        if(a == b) System.out.println("you won't see this");

        System.out.println();

        c = a - b; // переменная c содержит значение -1

        System.out.println("c contains -1");
        if(c >= 0) System.out.println("c is non-negative");
        if(c < 0) System.out.println("c is negative");

        System.out.println();

        c = b - a; // переменная c теперь содержит значение 1

        System.out.println("c contains 1");
        if(c >= 0) System.out.println("c is non-negative");
        if(c < 0) System.out.println("c is negative");
    }
}
```

Ниже приведен результат выполнения данной программы.

a is less than b

c contains -1

c is negative

c contains 1

c is non-negative

Обратите внимание на еще одну особенность данной программы. В следующей строке объявляются три переменные, `a`, `b` и `c`, имена которых разделяются запятыми:

```
int a, b, c;
```

Как упоминалось ранее, если требуется несколько переменных одного типа, их можно объявить в одном операторе. Для этого имена переменных следует разделить запятыми.

Цикл `for`

Циклы позволяют многократно выполнять последовательность кода. В Java для этой цели предоставляется целый ряд эффективных конструкций циклов. Рассмотрим для начала цикл `for`. Ниже приведена простейшая форма цикла `for`.

for(инициализация; условие; итерация) оператор;

В самой общей форме исходное значение переменной цикла задается в части *инициализация*, а в части *условие* — логическое выражение, которое, как правило, проверяет значение переменной цикла. Если логическое выражение принимает значение `true`, цикл `for` продолжает выполняться. А если оно принимает значение `false`, то цикл прекращается. И, наконец, в части *итерация* задается порядок изменения переменной цикла на каждом шаге цикла. Ниже приведен пример простой программы, демонстрирующий применение цикла `for` в коде.

```
/*
Продemonстрировать применение цикла for.

Присвоить исходному файлу имя ForDemo.java.
*/
class ForDemo {
    public static void main(String args[]) {
        int count;

        // Этот цикл выполняется пять раз.
        for(count = 0; count < 5; count = count+1)
            System.out.println("This is count: " + count);

        System.out.println("Done!");
    }
}
```

Ниже приведен результат выполнения данной программы.

```
This is count: 0
This is count: 1
This is count: 2
This is count: 3
This is count: 4
Done!
```

В данном примере `count` — это переменная цикла. При инициализации ей присваивается значение 0. В начале каждого шага цикла (включая и первый) проверяется условие `count < 5`. Если это условное выражение принимает логическое значение `true`, вызывается метод `println()`, после чего выполняется итерационная часть цикла. Данная последовательность действий повторяется до тех пор, пока условное выражение не примет логическое значение `false`. В этом случае управление передается следующему после цикла оператору.

Следует иметь в виду, что в профессионально написанных программах очень редко можно встретить итерационную часть цикла, написанную так, как это сделано в приведенном выше примере программы и показано ниже.

```
count = count + 1;
```

Дело в том, что в Java имеется специальный и более эффективный оператор инкремента. Для обозначения оператора инкремента служат два знака “плюс” (`++`) без про-

бела между ними. В результате выполнения этого оператора значение операнда увеличивается на единицу. Используя оператор инкремента, предыдущее итерационное выражение можно переписать следующим образом:

```
count++;
```

Следовательно, оператор цикла `for` из предыдущего примера программы можно переписать так, как показано ниже.

```
for(count = 0; count < 5; count++)
```

Попробуйте внести это изменение в рассматриваемый здесь цикл, и вы увидите, что результаты выполнения программы останутся прежними.

В Java предусмотрен также оператор декремента, обозначаемый двумя знаками “минус” (`--`) без пробела между ними. При выполнении этого оператора значение операнда уменьшается на единицу.

Создание кодовых блоков

Еще одним ключевым элементом Java является кодовый блок. Он представляет собой группу, в которую входят два или более оператора. Для оформления в виде блока они помещаются между открывающей и закрывающей фигурными скобками. Созданный кодовый блок становится единым логическим блоком и может использоваться в любом месте программы, как и одиночный оператор. В частности, кодовый блок можно использовать как адресат (или цель выполнения) операторов `if` и `for`. Рассмотрим следующий пример применения кодового блока в операторе `if`:

```
if(w < h) { ← Это начало кодового блока
    v = w * h;
    w = 0;
} ← Это конец кодового блока
```

В данном примере оба оператора в блоке выполняются в том случае, если значение переменной `w` меньше значения переменной `h`. Эти операторы составляют единый логический блок, и ни один из них не может быть выполнен без другого. Таким образом, для объединения нескольких выражений в единое логическое целое нужно создать кодовый блок. Кодовые блоки позволяют оформлять многие алгоритмы в удобном для восприятия виде.

Ниже приведен пример программы, в которой кодовый блок служит для того, чтобы предотвратить деление на ноль.

```
/*
```

```
    Продемонстрировать применение кодового блока.
```

```
    Присвоить исходному файлу имя BlockDemo.java.
```

```
*/
```

```
class BlockDemo {
    public static void main(String args[]) {
        double i, j, d;

        i = 5;
        j = 10;

        // Адресатом этого оператора if является целый кодовый блок.
        if(i != 0) {
```

```

        System.out.println("i does not equal zero");
        d = j / i;
        System.out.print("j / i is " + d);
    }
}
}

```

Результат выполнения данной программы выглядит следующим образом:

```

i does not equal zero
j / i is 2.0

```

В данном примере адресатом оператора `if` служит не один оператор, а целый кодовый блок. Если условие, управляющее оператором `if`, принимает логическое значение `true`, как в данном случае, то выполняются все три оператора в кодовом блоке. Попробуйте присвоить переменной `i` нулевое значение и запустить программу на выполнение, и вы увидите, что ни один из операторов из кодового блока не будет выполнен, т.е. весь этот блок будет пропущен.

Как будет показано далее в книге, кодовые блоки обладают и другими свойствами и находят иное применение. Но главное их назначение — создание логически неделимых единиц кода.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Приводит ли использование кодовых блоков к снижению эффективности программы? Иными словами, выполняются ли какие-то дополнительные действия, когда в коде Java встречаются символы `{ }`?

ОТВЕТ. Нет. Кодовые блоки кода не влекут за собой никаких дополнительных издержек. Более того, кодовые блоки повышают быстродействие, поскольку они упрощают код и позволяют создавать более эффективные алгоритмы. Символы `{ }` существуют только в исходном коде программы, но не подразумевают выполнение каких-то дополнительных действий.

Точки с запятой и оформление исходного текста программы

В Java точка с запятой служит в качестве разделителя операторов. Это означает, что каждый оператор должен оканчиваться точкой с запятой. Точка с запятой обозначает окончание одного логического объекта.

Как вы уже знаете, кодовый блок — это совокупность логически связанных операторов, помещаемых между открывающей и закрывающей фигурными скобками. Кодовый блок не завершается точкой с запятой. Он представляет собой группу операторов, и поэтому точка с запятой ставится в конце каждого оператора, а сам блок завершается лишь закрывающей фигурной скобкой.

В Java конец строки не считается окончанием оператора. Поэтому не имеет значения, где именно он находится в строке кода. Например, в Java строки кода

```

x = y;
y = y + 1;
System.out.println(x + " " + y);

```

означают то же самое, что и строка кода

```
x = y; y = y + 1; System.out.println(x + " " + y);
```

Более того, каждый элемент оператора можно разместить в отдельной строке. Например, следующая запись вполне допустима:

```
System.out.println("This is a long line of output" +
    x + y + z +
    "more output");
```

Перенос на новую строку позволяет избегать длинных строк и делать код программы более удобочитаемым. Кроме того, разбивая оператор на несколько строк, вы предотвращаете автоматический перенос, который зачастую портит внешний вид исходного текста программы.

Практическое применение отступов

Вы, вероятно, заметили, что некоторые операторы в предыдущих примерах программ располагаются с отступом от начала строки. В Java допускается произвольное оформление исходного кода, а это означает, что расположение операторов относительно друг друга не имеет особого значения. Но как показывает опыт программирования, располагая некоторые операторы с отступом от начала строки, можно сделать исходный текст программы более удобным для восприятия. В этой книге широко применяются отступы. Внимательно проанализировав представленные в ней примеры программ, вы, скорее всего, согласитесь, что подобный стиль представления исходного кода вполне оправдан. Так, операторы, заключенные в фигурные скобки, желательно располагать с отступом. А некоторым операторам требуется дополнительный отступ. Подробнее об этом речь пойдет далее в книге.

Пример для опробования 1.2.

Усовершенствованный вариант преобразования галлонов в литры

Исходный файл: GalToLitTable.java

В рассматриваемой здесь усовершенствованной версии программы, преобразующей галлоны в литры и созданной в рамках первого проекта, используются цикл `for`, условный оператор `if` и кодовые блоки. В новой версии программы на экран выводится таблица преобразования значений от 1 до 100 галлонов. После каждых десяти галлонов отображается пустая строка. Это достигается благодаря переменной `counter`, которая подсчитывает число выведенных строк. Обратите внимание на особенности применения этой переменной.

Последовательность действий

1. Создайте новый файл `GalToLitTable.java`.
2. Введите в этот файл следующий код программы:

```
/*
```

```
    Пример для опробования 1.2
```

```
    Эта программа отображает таблицу преобразования галлоны в литры.
```

```
    Присвоить ее исходному файлу имя GalToLitTable.java.
```

```
*/
```

```
class GalToLitTable {
```

```

public static void main(String args[]) {
    double gallons, liters;
    int counter;

    // Первоначально в счетчике строк устанавливается нулевое значение.
    counter = 0;
    for(gallons = 1; gallons <= 100; gallons++) {
        liters = gallons * 3.7854; // преобразовать в литры
        System.out.println(gallons + " gallons is " +
                           liters + " liters.");

        // Приращение счетчика строк происходит на каждом шаге цикла,
        // а через каждые десять строк выводится пустая строка.
        counter++;
        if(counter == 10) {
            System.out.println();
            counter = 0; // сбросить счетчик строк
        }
    }
}

```

3. Скомпилируйте программу, указав в командной строке следующее:

C>javac GalToLitTable.java

4. Запустите программу на выполнение из командной строки следующим образом:

C>java GalToLitTable

В результате выполнения данной программы на экране появится следующий результат:

```

1.0 gallons is 3.7854 liters.
2.0 gallons is 7.5708 liters.
3.0 gallons is 11.356200000000001 liters.
4.0 gallons is 15.1416 liters.
5.0 gallons is 18.927 liters.
6.0 gallons is 22.712400000000002 liters.
7.0 gallons is 26.4978 liters.
8.0 gallons is 30.2832 liters.
9.0 gallons is 34.0686 liters.
10.0 gallons is 37.854 liters.

11.0 gallons is 41.6394 liters.
12.0 gallons is 45.424800000000005 liters.
13.0 gallons is 49.2102 liters.
14.0 gallons is 52.9956 liters.
15.0 gallons is 56.781 liters.
16.0 gallons is 60.5664 liters.
17.0 gallons is 64.3518 liters.
18.0 gallons is 68.1372 liters.
19.0 gallons is 71.9226 liters.
20.0 gallons is 75.708 liters.

21.0 gallons is 79.49340000000001 liters.
22.0 gallons is 83.2788 liters.
23.0 gallons is 87.0642 liters.

```

```

24.0 gallons is 90.84960000000001 liters.
25.0 gallons is 94.635 liters.
26.0 gallons is 98.4204 liters.
27.0 gallons is 102.2058 liters.
28.0 gallons is 105.9912 liters.
29.0 gallons is 109.7766 liters.
30.0 gallons is 113.562 liters.

```

Ключевые слова Java

В настоящее время в языке Java определено пятьдесят ключевых слов (табл. 1.1). Вместе с синтаксисом операторов и разделителями они образуют определение языка Java. Ключевые слова нельзя использовать в качестве имен переменных, классов или методов.

Ключевые слова `const` и `goto` зарезервированы, но не используются. На ранних этапах развития Java для дальнейшего использования были зарезервированы и другие ключевые слова. Но в текущем определении (так называемой спецификации) Java определены только те ключевые слова, которые представлены в табл. 1.1.

Помимо ключевых слов, в Java зарезервированы также логические значения `true`, `false` и `null`. Их нельзя использовать для обозначения переменных, классов и других элементов программ.

Таблица 1.1. Ключевые слова Java

<code>abstract</code>	<code>assert</code>	<code>boolean</code>	<code>break</code>	<code>byte</code>	<code>case</code>
<code>catch</code>	<code>char</code>	<code>class</code>	<code>const</code>	<code>continue</code>	<code>default</code>
<code>do</code>	<code>double</code>	<code>else</code>	<code>enum</code>	<code>extends</code>	<code>final</code>
<code>finally</code>	<code>float</code>	<code>for</code>	<code>goto</code>	<code>if</code>	<code>implements</code>
<code>import</code>	<code>instanceof</code>	<code>int</code>	<code>interface</code>	<code>long</code>	<code>native</code>
<code>new</code>	<code>package</code>	<code>private</code>	<code>protected</code>	<code>public</code>	<code>return</code>
<code>short</code>	<code>static</code>	<code>strictfp</code>	<code>super</code>	<code>switch</code>	<code>synchronized</code>
<code>this</code>	<code>throw</code>	<code>throws</code>	<code>transient</code>	<code>try</code>	<code>void</code>
<code>volatile</code>	<code>while</code>				

Идентификаторы в Java

В Java *идентификатор* обозначает имя метода, переменной или элемента, определяемых пользователем. Идентификатор может содержать один или несколько символов. Имя переменной может начинаться с любой буквы, знака подчеркивания или денежной единицы. Далее могут следовать буквы, цифры, знак подчеркивания или денежной единицы. Знак подчеркивания обычно применяется для того, чтобы сделать имя более удобным для восприятия, например `line_count`.

В Java символы нижнего и верхнего регистра различаются, т.е. `myvar` и `MyVar` — это имена разных переменных. Ниже приведен ряд примеров допустимых в Java идентификаторов.

<code>Test</code>	<code>x</code>	<code>y2</code>	<code>MaxLoad</code>
<code>\$up</code>	<code>_top</code>	<code>my_var</code>	<code>sample23</code>

Как упоминалось выше, идентификатор не может начинаться с цифры. Например, идентификатор 12x недопустим.

В качестве идентификаторов нельзя использовать ключевые слова Java. Для этой цели нельзя использовать также имена стандартных методов, например `println`. Эти ограничения необходимо соблюдать. Кроме того, профессиональный стиль программирования предполагает употребление имен, отражающих назначение соответствующих элементов.

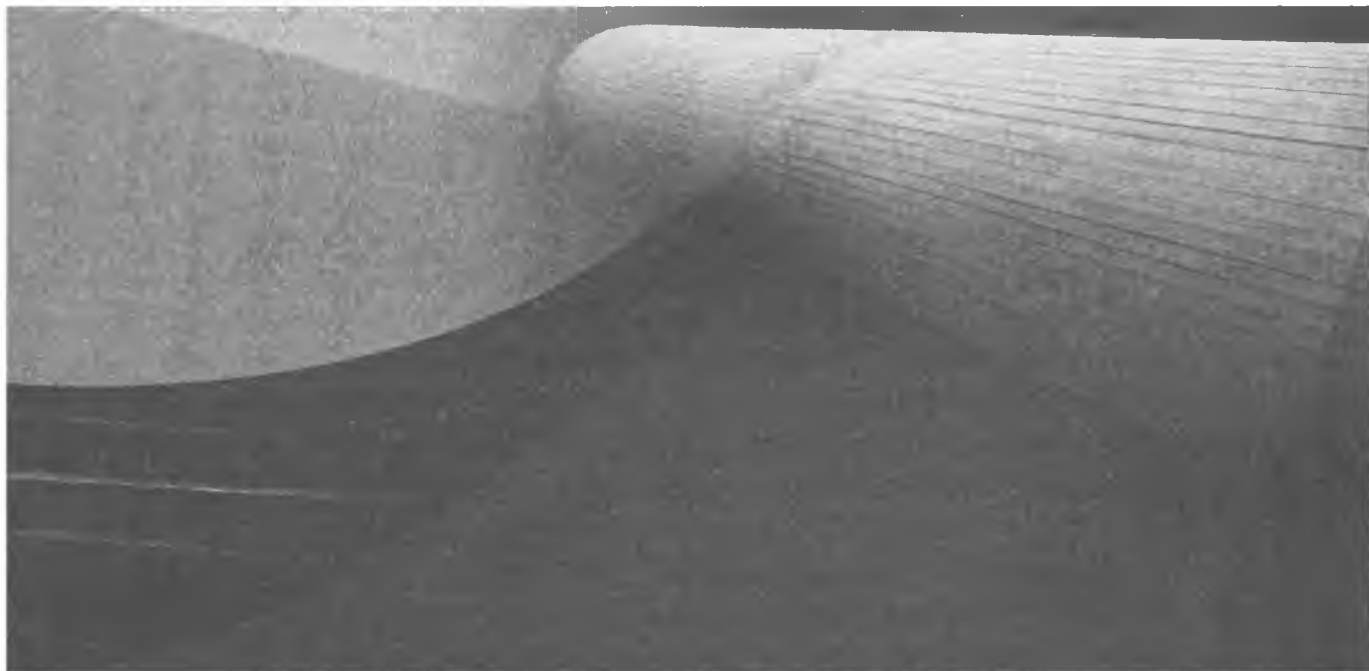
Библиотеки классов Java

В примерах программ, представленных в этой главе, применяются два встроенных в Java метода: `println()` и `print()`. Эти методы являются членами класса `System`, который предопределен в Java и автоматически включается в состав любой программы. В более широком смысле среда Java включает в себя ряд встроенных библиотек классов, содержащих большое количество методов. Они обеспечивают поддержку ввода-вывода, операций с символьными строками, сетевого взаимодействия и отображения графики. Стандартные классы также реализуют оконный вывод. Таким образом, Java представляет собой сочетание собственно языка и стандартных классов. Как станет ясно в дальнейшем, многими своими функциональными возможностями язык Java обязан именно библиотекам классов. Поэтому научиться грамотно программировать Java невозможно, не усвоив стандартные классы. На протяжении всей остальной части книги вам будет встречаться описание различных классов и методов из стандартных библиотек. Но в одной книге невозможно описать все библиотеки, поэтому полученные знания основ Java вам придется пополнить в процессе самостоятельной работы.

Упражнение для самопроверки по материалу главы 1

1. Что такое байт-код и почему он так важен для интернет-программирования на языке Java?
2. Каковы три основных принципа объектно-ориентированного программирования?
3. С чего начинается выполнение программы на Java?
4. Что такое переменная?
5. Какое из перечисленных ниже имен переменных недопустимо?
 - A. `count`
 - B. `$count`
 - C. `count27`
 - D. `67count`
6. Как создать однострочный комментарий? И как создать многострочный комментарий?
7. Как выглядит общая форма условного оператора `if`? И как выглядит общая форма цикла `for`?
8. Как создать кодовый блок?
9. Сила тяжести на Луне составляет около 17% земной. Напишите программу, которая вычисляла бы ваш вес на Луне.

10. Видоизмените программу, созданную в примере для опробования 1.2, таким образом, чтобы она выводила таблицу преобразования дюймов в метры. Выводите значения длины до 12 футов через каждый дюйм. После каждых 12 дюймов выводите пустую строку. (Один метр приблизительно равен 39,37 дюйма.)
11. Если при вводе кода программы вы допустите опечатку, то какого рода сообщение об ошибке получите?
12. Имеет ли значение, в каком именно месте строки находится оператор?



Глава 2

Введение в типы данных и операторы

Основные навыки и понятия

- Представление о простых типах данных в Java
- Применение литералов
- Инициализация переменных
- Правила соблюдения области действия переменных в методе
- Применение арифметических операторов
- Применение операторов сравнения и логических операторов
- Представление об операторах присваивания
- Представление об укороченных операторах присваивания
- Преобразование типов при присваивании
- Представление о неприводимых типах данных
- Преобразование типов в выражениях

Основу любого языка программирования составляют типы данных и операторы, и Java не является исключением из этого правила. Типы данных и операторы определяют область применимости языка и круг задач, которые можно успешно решать с его помощью. В Java поддерживаются самые разные типы данных и операторы, что делает этот язык универсальным и пригодным для написания любых программ.

Значение типов данных и операций нельзя недооценивать. Эта глава начинается с анализа основных типов данных и наиболее часто используемых операторов. А кроме того, в ней будут подробно рассмотрены переменные и выражения.

Особое значение типов данных

В связи с тем, что Java относится к категории строго типизированных языков программирования, типы данных имеют в нем очень большое значение. В процессе компиляции проверяются типы операндов во всех операциях. И если в программе встречаются недопустимые операции, ее исходный код не преобразуется в байт-код. Контроль типов позволяет сократить количество ошибок и повысить надежность программы. В отличие от других языков программирования, где допускается не указывать типы данных, хранящихся в переменных, в Java все переменные, выражения и значения строго контролируются на соответствие типов данных. Более того, тип переменной определяет, какие именно операции могут быть выполнены над ней. Операции, разрешенные для одного типа данных, могут оказаться недопустимы для другого.

Простые типы данных в Java

Встроенные типы данных в Java разделяются на две категории: объектно-ориентированные и неobjектно-ориентированные. Объектно-ориентированные типы данных определяются в классах, о которых речь пойдет далее в книге. В основу язы-

ка Java положено восемь простых типов данных, приведенных в табл. 2.1 (их также называют *элементарными*, или *примитивными*). Термин *простые* указывает на то, что эти типы данных являются не объектами, а обычными двоичными значениями. Такие типы данных предусмотрены в языке для того, чтобы увеличить эффективность работы программ. Все остальные типы данных Java образуются на основе простых типов.

В Java четко определены области действия простых типов и диапазон допустимых для них значений. Эти правила должны соблюдаться при создании всех виртуальных машин. А поскольку программы на Java должны быть переносимыми, точное следование этим правилам является одним из основных требований языка. Например, тип `int` остается неизменным в любой исполняющей среде, благодаря чему удастся обеспечить реальную переносимость программ. Это означает, что при переходе с одной платформы на другую не придется переписывать код. И хотя строгий контроль типов может привести к незначительному снижению производительности в некоторых исполняющих средах, он является обязательным условием переносимости программ.

Таблица 2.1. Простые типы, встроенные в Java

Тип	Описание
<code>boolean</code>	Представляет логические значения <code>true</code> и <code>false</code>
<code>byte</code>	8-разрядное целое число
<code>char</code>	Символ
<code>double</code>	Числовое значение с плавающей точкой двойной точности
<code>float</code>	Числовое значение с плавающей точкой одинарной точности
<code>int</code>	Целое число
<code>long</code>	Длинное целое число
<code>short</code>	Короткое число

Целочисленные типы данных

В Java определены четыре целочисленных типа данных: `byte`, `short`, `int` и `long`. Их краткое описание приведено ниже.

Тип	Разрядность, бит	Диапазон допустимых значений
<code>byte</code>	8	от -128 до 127
<code>short</code>	16	от -32,768 до 32,767
<code>int</code>	32	от -2,147,483,648 до 2,147,483,647
<code>long</code>	64	от -9,223,372,036,854,775,808 до 9,223,372,036,854,775,807

Как следует из приведенной выше таблицы, целочисленные типы данных предполагают как положительные, так и отрицательные значения. В Java не поддерживаются целочисленные значения без знака, т.е. только положительные целые числа. Во многих других языках программирования широко применяются целочисленные типы данных без знака, но создатели Java посчитали их излишними.

На заметку

В исполняющей системе Java для хранения простых типов может формально выделяться любой объем памяти, но диапазон допустимых значений остается неизменным.

Из всех целочисленных типов данных чаще всего применяется `int`. Переменные типа `int` нередко используются в качестве переменных циклов, индексов массивов и, конечно же, для выполнения универсальных операций над целыми числами.

Если диапазон значений, допустимых для типа `int`, не устраивает, можно выбрать тип `long`. Ниже приведен пример программы для расчета числа кубических дюймов в кубе, длина, ширина и высота которого равны одной миле.

```
/*
 * Расчет числа кубических дюймов в кубе объемом в 1 миле.
 */
class Inches {
    public static void main(String args[]) {
        long ci;
        long im;

        im = 5280 * 12;

        ci = im * im * im;

        System.out.println("There are " + ci +
                           " cubic inches in cubic mile.");
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
There are 254358061056000 cubic inches in cubic mile.
```

Очевидно, что результирующее значение не умещается в переменной типа `int`.

Наименьшим диапазоном допустимых значений среди всех целочисленных типов обладает тип `byte`. Переменные типа `byte` очень удобны для обработки исходных двоичных данных, которые могут оказаться несовместимыми с другими встроенными в Java типами данных. Тип `short` предназначен для хранения небольших целых чисел. Переменные данного типа пригодны для хранения значений, изменяющихся в относительно небольших пределах по сравнению со значениями типа `int`.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как упоминалось выше, существуют четыре целочисленных типа данных: `int`, `short`, `long` и `byte`. Но говорят, что тип `char` в Java также считается целочисленным. В чем здесь дело?

ОТВЕТ. Формально в спецификации Java определена категория целочисленных типов данных, в которую входят типы `byte`, `short`, `int`, `long` и `char`. Такие типы называют *целочисленными*, поскольку они могут хранить целые двоичные значения. Первые четыре типа предназначены для представления целых чисел, а тип `char` — для представления символов. Таким образом, тип `char` принципиально отличается от остальных четырех целых типов данных. Учитывая это отличие, тип `char` рассматривается в данной книге отдельно.

Типы данных с плавающей точкой

Как пояснялось в главе 1, типы с плавающей точкой могут представлять числовые значения с дробной частью. Существуют два типа данных с плавающей точкой: `float`

и `double`. Они представляют числовые значения с одинарной и двойной точностью соответственно. Разрядность данных типа `float` составляет 32 бита, а разрядность данных типа `double` — 64 бита.

Тип `double` употребляется намного чаще, чем `float`, поскольку во всех математических функциях из библиотек классов Java используются значения типа `double`. Например, метод `sqrt()`, определенный в стандартном классе `Math`, возвращает значение `double`, являющееся квадратным корнем значения аргумента этого метода, также представленного типом `double`. Ниже приведен фрагмент кода, в котором метод `sqrt()` используется для расчета длины гипотенузы треугольника, при условии, что заданы длины катетов.

```
/*
    Определение длины гипотенузы, исходя из длины катетов по теореме Пифагора.
*/
class Hypot {
    public static void main(String args[]) {
        double x, y, z;

        x = 3;
        y = 4;

        // Обратите внимание на вызов метода sqrt(). Перед именем метода
        // указывается имя класса, членом которого он является.
        z = Math.sqrt(x*x + y*y);

        System.out.println("Hypotenuse is " + z);
    }
}
```

Выполнение этого фрагмента кода дает следующий результат:

```
Hypotenuse is 5.0
```

Как упоминалось выше, метод `sqrt()` определен в стандартном классе `Math`. Обратите внимание на вызов этого метода в приведенном выше фрагменте кода: перед его именем указывается имя класса. Аналогичным образом перед именем метода `println()` указывается имена классов `System.out`. Имя класса указывается не перед всеми стандартными методами, но для некоторых из них целесообразно применять именно такой способ.

Символы

В отличие от других языков в Java символы не являются 8-битовыми значениями. Вместо этого в Java символы всех языков мира представлены в уникоде (Unicode). Таким образом, в Java тип `char` представляет 16-разрядное значение без знака в диапазоне от 0 до 65536. Стандартный набор 8-разрядных символов в коде ASCII является подмножеством уникода. В нем коды символов находятся в пределах от 0 до 127. Следовательно, символы в коде ASCII по-прежнему допустимы в Java.

Переменной символьного типа может быть присвоено значение, которое записывается в виде символа, заключенного в одинарные кавычки. В приведенном ниже фрагменте кода показано, каким образом переменной `ch` присваивается буква `S`.

```
char ch;
ch = 'S';
```


Отобразить значение типа `char` можно с помощью метода `println()`. В приведенной ниже строке кода показано, каким образом этот метод вызывается для вывода на экран значения символа, хранящегося в переменной `ch`.

```
System.out.println("This is ch:" + ch);
```

Тип `char` представляет 16-разрядное значение без знака, а это означает, что над переменной символьного типа можно производить различные арифметические операции. Рассмотрим в качестве примера следующую программу:

```
// С символьными переменными можно обращаться, как с целочисленными.
class CharArithDemo {
    public static void main(String args[]) {
        char ch;

        ch = 'X';
        System.out.println("ch contains " + ch);

        ch++; // переменную типа char можно инкрементировать
        System.out.println("ch is now " + ch);

        ch = 90; // переменной типа char можно присвоить целочисленное значение
        System.out.println("ch is now " + ch);
    }
}
```

Ниже приведен результат выполнения данной программы.

```
ch contains X
ch is now Y
ch is now Z
```

В приведенной выше программе переменной `ch` сначала присваивается значение кода буквы **X**. Затем содержимое `ch` увеличивается на единицу, в результате чего оно превращается в код буквы **Y** — следующего по порядку символа в коде ASCII (а также в уникоде). После этого переменной `ch` присваивается значение **90**, представляющее букву **Z** в коде ASCII (и в уникоде). А поскольку символам в коде ASCII соответствуют первые 127 значений в уникоде, то все приемы, обычно применяемые для манипулирования символами в других языках программирования, вполне работоспособны и в Java.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Почему в Java для кодировки символов применяется уникод?

ОТВЕТ. Язык Java изначально разрабатывался для международного применения.

Поэтому возникла необходимость в наборе символов, способном представлять все языки мира. Именно для этой цели и был разработан уникод. Очевидно, что применение уникода в программах на таких языках, как английский, немецкий, испанский и французский, сопряжено с дополнительными издержками, поскольку для символа, который вполне может быть представлен восьмью битами, приходится выделять 16 битов. Это та цена, которую приходится платить за обеспечение переносимости программ. Но эти издержки вполне оправданны для разработки пользовательских интерфейсов программ на русском, украинском, польском, греческом и других языках, буквы которых не могут быть представлены первыми 127 значениями в уникоде.

Логический тип данных

Тип `boolean` представляет логические значения “истина” и “ложь”, для которых в Java зарезервированы слова `true` и `false` соответственно. Следовательно, переменная или выражение типа `boolean` может принимать одно из этих двух значений.

Ниже приведен пример программы, демонстрирующий применение типа `boolean` в коде.

```
// Демонстрация обращения с логическими значениями.
class BoolDemo {
    public static void main(String args[]) {
        boolean b;

        b = false;
        System.out.println("b is " + b);
        b = true;

        System.out.println("b is " + b);
        // Логическое значение можно использовать для
        // управления условным оператором if.
        if(b) System.out.println("This is executed.");

        b = false;
        if(b) System.out.println("This is not executed.");

        // Логическое значение получается в результате
        // выполнения оператора отношения.
        System.out.println("10 > 9 is " + (10 > 9));
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
b is false
b is true
This is executed.
10 > 9 is true
```

Анализируя приведенную выше программу, необходимо отметить следующее. Во-первых, нетрудно заметить, что метод `println()`, обрабатывая логическое значение, отображает символьные строки `"true"` и `"false"`. Во-вторых, значение логической переменной может быть само использовано для управления условным оператором `if`. Это означает, что отпадает необходимость в выражениях вроде следующего:

```
if(b == true) ...
```

И в-третьих, результатом выполнения оператора отношения, например `<`, является логическое значение. Именно поэтому при передаче методу `println()` выражения `(10 > 9)` отображается логическое значение `true`. Скобки в данном случае необходимы, потому что оператор `+` имеет более высокий приоритет, чем оператор `>`.

Пример для опробования 2.1.

Расчет расстояния до удара молнии

Исходный файл: `Sound.java`

В данном проекте предстоит написать программу, вычисляющую расстояние в футах до источника звука при ударе молнии. Звук распространяется в воздухе со скоростью, приблизительно равной 1100 футам в секунду. Следовательно, зная промежуток времени между теми

моментами, когда наблюдатель увидит вспышку молнии и услышит сопровождающий ее раскат грома, можно рассчитать расстояние до нее. Допустим, что этот промежуток времени составляет 7,2 секунды.

Последовательность действий

1. Создайте новый файл `Sound.java`.
2. Для расчета искомого расстояния потребуются числовые значения с плавающей точкой. Почему? А потому, что упомянутое выше числовое значение промежутка времени содержит дробную часть. И хотя для расчета достаточно точности, обеспечиваемой типом `float`, в данном примере будет использован тип `double`.
3. Для расчета искомого расстояния умножьте значение **7,2** на **1100**, а полученный результат сохраните в переменной типа `double`.
4. Выведите результат расчета на экран.

Ниже приведен исходный код программы из файла `Sound.java`.

```
/*
    Пример для опробования 2.1.

    Рассчитать расстояние до удара молнии, звук от которого
    доходит до наблюдателя через 7,2 с.
*/
class Sound {
    public static void main(String args[]) {
        double dist;

        dist = 7.2 * 1100;

        System.out.println("The lightning is " + dist +
                           " feet away.");
    }
}
```

5. Скомпилируйте программу и запустите ее на выполнение, чтобы вывести на экран следующий результат:

```
The lightning is 7920.0 feet away.
```

6. Усложним задачу. Рассчитать расстояние до крупного объекта, например скалы, можно по времени прихода эхо. Так, если вы хлопнете в ладоши, время, через которое вернется эхо, будет равно времени прохождения звука в прямом и обратном направлении. Разделив этот промежуток времени на два, вы получите время прохождения звука от вас до объекта. Это время можно затем использовать для расчета расстояния до объекта. Видоизмените рассмотренную выше программу, используя в качестве заданного промежутка время прихода эха.

Литералы

В Java *литералы* применяются для представления постоянных значений в форме, удобной для восприятия. Например, число **100** является литералом. Литералы часто называют *константами*. Как правило, структура литералов и их использование интуитив-

но понятны. Они уже встречались в рассмотренных ранее примерах программ, а теперь пришло время дать им формальное определение. В Java предусмотрены литералы для всех простых типов. Способ представления литерала зависит от типа данных. Как пояснялось ранее, константы, соответствующие символам, заключаются в одинарные кавычки. Например, и 'a', и '%' являются символьными константами.

Целочисленные константы записываются как числа без дробной части. Например, целочисленными константами являются 10 и -100. При формировании константы с плавающей точкой необходимо указывать десятичную точку, после которой следует дробная часть. Например, 11,123 — это константа с плавающей точкой. В Java поддерживается и так называемый экспоненциальный формат представления чисел с плавающей точкой.

По умолчанию целочисленные литералы относятся к типу `int`. Если же требуется определить литерал типа `long`, после числа следует указать букву **L** или **l**. Например, 12 — это константа типа `int`, а 12L — литерал типа `long`.

По умолчанию к типу `double` относятся и литералы с плавающей точкой. А для того чтобы задать литерал типа `float`, следует указать после числа букву **F** или **f**. Так, например, к типу `float` относится литерал 10,19F.

Несмотря на то что целочисленные литералы по умолчанию создаются как значения типа `int`, их можно присваивать переменным типа `char`, `byte`, `short` и `long`. Присваиваемое значение приводится к целевому типу. Переменной типа `long` можно также присвоить любое значение, представленное целочисленным литералом.

Начиная с версии JDK 7 появилась возможность вставлять в литералы (как целочисленные, так и с плавающей точкой) знаки подчеркивания. Благодаря этому упрощается восприятие числовых значений, состоящих из нескольких цифр. А при компиляции знаки подчеркивания просто удаляются из литерала. Ниже приведен пример литерала со знаком подчеркивания.

```
123_45_1234
```

Этот литерал задает числовое значение 123451234. Пользоваться знаками подчеркивания особенно удобно при кодировании номеров деталей, идентификаторов заказчиков и кодов состояния, которые обычно состоят из целых групп цифр.

Шестнадцатеричные, восьмеричные и двоичные литералы

Вам, вероятно, известно, что при написании программ бывает удобно пользоваться числами, представленными в системе счисления, отличающейся от десятичной. Для этой цели чаще всего выбирается восьмеричная (с основанием 8) и шестнадцатеричная (с основанием 16) системы счисления. В *восьмеричной* системе используются цифры от 0 до 7, а число 10 соответствует числу 8 в десятичной системе. В *шестнадцатеричной* системе используются цифры от 0 до 9, а также буквы от **A** до **F**, которыми обозначаются числа 10, 11, 12, 13, 14 и 15 в десятичной системе, тогда как число 10 в шестнадцатеричной системе соответствует десятичному числу 16. Восьмеричная и шестнадцатеричная системы используются очень часто в программировании, и поэтому в языке Java предусмотрена возможность представления целочисленных констант (или литералов) в восьмеричной и шестнадцатеричной форме. Шестнадцатеричная константа должна начинаться с символов **0x** (цифры 0, после которой следует буква **x**). А восьмеричная константа начинается с нуля. Ниже приведены примеры таких констант.

```
hex = 0xFF; // Соответствует десятичному числу 255
oct  = 011;  // Соответствует десятичному числу 9
```

Любопытно, что в Java допускается также задавать шестнадцатеричные литералы в формате с плавающей точкой, хотя они употребляются очень редко. Начиная с версии JDK 7 появилась также возможность задавать целочисленный литерал в двоичной форме. Для этого перед целым числом достаточно указать символы **0b** или **0B**. Например, следующий литерал определяет целое значение **12** в двоичной форме:

```
0b1100.
```

Символьные управляющие последовательности

Заключая большинство печатных символов в одинарные кавычки, можно обращаться с ними как с символьными константами, но для непечатных символов, например возврата каретки, требуется специальное символьное представление. Кроме того, некоторые знаки, например одинарные и двойные кавычки, имеют специальное назначение, и поэтому их нельзя непосредственно указывать в качестве литерала. По этой причине в языке Java предусмотрены специальные *управляющие последовательности*, начинающиеся с обратной косой черты (и нередко называемые *константами с обратной косой чертой*). Эти последовательности перечислены в табл. 2.2. Они используются в литералах вместо непечатных символов, которые они представляют.

Таблица 2.2. Символьные управляющие последовательности

Управляющая последовательность	Описание
<code>\'</code>	Одинарная кавычка
<code>\"</code>	Двойная кавычка
<code>\\</code>	Обратная косая черта
<code>\r</code>	Возврат каретки
<code>\n</code>	Перевод строки
<code>\f</code>	Перевод страницы
<code>\t</code>	Горизонтальная табуляция
<code>\b</code>	Возврат на одну позицию
<code>\ddd</code>	Восьмеричная константа (где <i>ddd</i> —восьмеричное число)
<code>\uxxxx</code>	Шестнадцатеричная константа (где <i>xxxx</i> —шестнадцатеричное число)

Ниже приведен пример присваивания переменной `ch` символа табуляции.

```
ch = '\t';
```

А в следующем примере переменной `ch` присваивается одинарная кавычка:

```
ch = '\'';
```

Строковые литералы

В Java предусмотрены также литералы для представления символьных строк. *Символьная строка* — это набор символов, заключенных в двойные кавычки, как в приведенном ниже примере.

```
"this is a test"
```

Примеры строковых литералов не раз встречались в рассмотренных ранее примерах программ. В частности, они передавались в качестве аргументов методу `println()`.

Помимо обычных символов, строковый литерал может также содержать упоминавшиеся выше управляющие последовательности. Рассмотрим в качестве примера следующую программу, в которой применяются управляющие последовательности `\n` и `\t`:

// Демонстрация управляющих последовательностей в символьных строках.

```
class StrDemo {  
    public static void main(String args[]) {  
        // В следующей строке кода в символьную строку введена  
        // управляющая последовательность \n для перевода строки.  
        System.out.println("First line\nSecond line");  
        // В двух последующих строках кода в символьную строку введена  
        // управляющая последовательность \t для выравнивания выводимых  
        // результатов с помощью табуляции.  
        System.out.println("A\tB\tC");  
        System.out.println("D\tE\tF") ;  
    }  
}
```

Ниже приведен результат выполнения данной программы.

```
First line  
Second line  
A      B      C  
D      E      F
```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Одно ли и то же строка, состоящая из одного символа, и символьный литерал? Например, есть ли разница между обозначениями `"k"` и `'k'`?

ОТВЕТ. Нет, не одно и то же. Строки следует отличать от символов. Символьный литерал представляет один символ и относится к типу `char`. А строка, содержащая даже один символ, все равно остается строкой. Несмотря на то что строки состоят из символов, они относятся к разным типам данных.

Обратите внимание на использование управляющей последовательности `\n` для перевода строки в приведенном выше примере программы. Для вывода на экран нескольких символьных строк, совсем не обязательно вызывать метод `println()` несколько раз подряд. Достаточно ввести в строку символы `\n`, и при выводе в этом месте произойдет переход на новую строку.

Подробнее о переменных

О переменных уже шла речь в главе 1. А здесь они будут рассмотрены более подробно. Как вы уже знаете, переменная объявляется в такой форме:

тип имя_переменной;

где *тип* обозначает конкретный тип объявляемой переменной, а *имя_переменной* — ее наименование. Объявить можно переменную любого допустимого типа, включая рассмотренные ранее простые типы. Когда объявляется переменная, создается экземпляр соответствующего типа. Следовательно, возможности переменной определяются ее типом. Например, переменную типа `boolean` нельзя использовать для хранения значения с плавающей точкой. На протяжении всего срока действия переменной ее тип остается неизменным. Так, переменная `int` не может превратиться в переменную `char`.

В Java каждая переменная должна быть непременно объявлена перед ее употреблением. Ведь компилятору необходимо знать, данные какого именно типа содержит переменная, и лишь тогда он сможет правильно скомпилировать оператор, в котором используется переменная. Объявление переменных позволяет также осуществлять строгий контроль типов в Java.

Инициализация переменных

Прежде чем использовать переменную в выражении, ей нужно присвоить значение. Сделать это можно, в частности, с помощью уже знакомого вам оператора присваивания. Существует и другой способ: инициализировать переменную при ее объявлении. Для этого достаточно указать после имени переменной знак равенства и требуемое значение. Ниже приведена общая форма инициализации переменной.

тип переменная = значение;

где *значение* обозначает конкретное значение, которое получает *переменная* при ее создании. А *тип* значения должен быть совместим с типом переменной. Ниже приведен ряд примеров инициализации переменных.

```
int count = 10; // присвоить переменной count начальное значение 10
char ch = 'S'; // инициализировать переменную ch буквой S
float f = 1.2F; // инициализировать переменную f числовым значением 1,2
```

Присваивать начальные значения переменным можно и в том случае, если в одном операторе объявляется несколько переменных, как в приведенном ниже примере.

```
int a, b = 8, c = 19, d; // инициализируются переменные b и c
```

В данном случае инициализируются переменные **b** и **c**.

Динамическая инициализация

В приведенных выше примерах в качестве значений, присваиваемых переменным, использовались только константы. Но в Java поддерживается также динамическая инициализация, при которой можно использовать любые выражения, допустимые в момент объявления переменной. Ниже приведен пример простой программы, в которой объем цилиндра рассчитывается, исходя из его радиуса и высоты.

```
// Демонстрация динамической инициализации.
class DynInit {
    public static void main(String args[]) {
        double radius = 4, height = 5;

        // Переменная volume инициализируется динамически
        // во время выполнения программы.
        double volume = 3.1416 * radius * radius * height;

        System.out.println("Volume is " + volume);
    }
}
```

В данном примере используются три локальные переменные: *radius*, *height* и *volume*. Первые две из них инициализируются константами, а для присвоения значения переменной *volume* применяется динамическая инициализация, в ходе которой вычисляется объем цилиндра. В выражении динамической инициализации можно использовать любой определенный к этому моменту элемент, в том числе вызовы методов, другие переменные и литералы.

Область и срок действия переменных

Все использовавшиеся до сих пор переменные объявлялись в начале метода `main()`. Но в Java можно объявлять переменные в любом кодовом блоке. Как пояснялось в главе 1, кодовый блок начинается с открывающей фигурной скобки и оканчивается закрывающей фигурной скобкой. Блок определяет *область действия* переменных. Начиная новый блок, вы всякий раз создаете новую область действия. По существу, область действия определяет доступность объектов из других частей программы и срок их действия.

Во многих языках программирования поддерживаются две общие категории областей действия: глобальная и локальная. И хотя они поддерживаются и в Java, тем не менее не относятся к самым лучшим средствам для определения пределов действия объектов. Намного большее значение в Java имеют области, определяемые классом и методом. Об областях действия, определяемых классом (и объявляемых в них переменных), речь пойдет далее в этой книге, когда дойдет черед до рассмотрения классов. А до тех пор исследуем только те области действия, которые определяются методами или в самих методах.

Начало области действия, определяемой методом, обозначает открывающая фигурная скобка. Если для метода предусмотрены параметры, они также входят в область его действия.

Как правило, переменные, объявленные в некоторой области действия, не видны (а следовательно, недоступны) за ее пределами. Следовательно, объявляя переменную в некоторой области действия, вы тем самым ограничиваете пределы ее действия и защищаете ее от нежелательного доступа и видоизменения. На самом деле правила определения области действия служат основанием для инкапсуляции.

Области действия могут быть вложенными. Открывая новый кодовый блок, вы создаете новую, вложенную область действия. Такая область заключена во внешней области. Это означает, что объекты, объявленные во внешней области действия, будут доступны для кода во внутренней области, но не наоборот. Объекты, объявленные во внутренней области действия, недоступны во внешней области.

Для того чтобы лучше понять принцип действия вложенных областей действия, рассмотрим следующий пример программы:

```
// Демонстрация области действия кодового блока.
class ScopeDemo {
    public static void main(String args[]) {
        int x; // эта переменная доступна для всего кода в методе main.

        x = 10;
        if(x == 10) { // Начало новой области действия.

            int y = 20; // Эта переменная доступна только в данном кодовом блоке.

            // Обе переменные, x и y, доступны в данном кодовом блоке.

            System.out.println("x and y: " + x + " " + y);
            x = y * 2;
        }
        // y = 100; // Ошибка! Переменная y недоступна за пределами
                // области своего действия

        // А переменная x по-прежнему доступна.
```



```

        System.out.println("x is " + x);
    }
}

```

Как следует из комментариев к приведенной выше программе, переменная `x` определяется в начале области действия метода `main()` и доступна для всего кода, содержащегося в этом методе. В блоке условного оператора `if` объявляется переменная `y`. Этот блок определяет область действия переменной `y`, и, следовательно, она доступна только в нем. Именно поэтому закомментирована строка кода `y = 100;`, находящаяся за пределами данного блока. Если удалить символы комментариев, то при компиляции программы появится сообщение об ошибке, поскольку переменная `y` недоступна для кода за пределами ее блока. В то же время в блоке условного оператора `if` можно пользоваться переменной `x`, потому что код в блоке, который определяет вложенную область действия, имеет доступ к переменным из внешней, охватывающей его области действия.

Переменные можно объявлять в любом месте кодового блока, но сделать это следует непременно перед тем, как пользоваться ими. Именно поэтому переменная, определенная в начале метода, доступна для всего кода этого метода. А если объявить переменную в конце блока, то такое объявление окажется, по существу, бесполезным, поскольку переменная станет вообще недоступной для кода.

Следует также иметь в виду, что переменные, созданные в области их действия, удаляются, как только управление в программе передается за пределы этой области. Следовательно, после выхода из области действия переменной содержащееся в ней значение теряется. В частности, переменные, объявленные в теле метода, не хранят значения в промежутках между последовательными вызовами этого метода. Таким образом, срок действия переменной ограничивается областью ее действия.

Если при объявлении переменной осуществляется ее инициализация, то переменная будет повторно инициализироваться при каждом входе в тот блок, в котором она объявлена. Рассмотрим в качестве примера следующую программу:

```

// Демонстрация срока действия переменной.
class VarInitDemo {
    public static void main(String args[]) {
        int x;

        for(x = 0; x < 3; x++) {
            int y = -1; // переменная y инициализируется при каждом входе в блок
            System.out.println("y is: " + y); // всегда выводится значение -1
            y = 100;
            System.out.println("y is now: " + y);
        }
    }
}

```

Ниже приведен результат выполнения данной программы.

```

y is: -1
y is now: 100
y is: -1
y is now: 100
y is: -1
y is now: 100

```

Как видите, на каждом шаге цикла `for` переменная `y` инициализируется значением `-1`. Затем ей присваивается значение `100`, но по завершении кодового блока данного цикла оно теряется.

Для правил области действия в Java характерна следующая особенность: имя переменной во вложенном блоке не может совпадать с именем переменной во внешнем блоке. Рассмотрим следующий пример программы. В ней предпринимается попытка объявить две переменные с одним и тем же именем в разных областях действия, и поэтому она не подлежит компиляции.

```
/*
  В этой программе предпринимается попытка объявить переменную
  во внутренней области действия с таким же именем, как и у переменной,
  определенной во внешней области действия.

  *** Эта программа не подлежит компиляции. ***
  */
class NestVar {
    public static void main(String args[]) {
        int count; // Первое объявление переменной count

        for(count = 0; count < 10; count = count+1) {
            System.out.println("This is count: " + count);

            int count; // Второе объявление переменной count.
                       // Недопустимо, поскольку точно такая же
                       // переменная объявлена раньше!!!
            for(count = 0; count < 2; count++)
                System.out.println("This program is in error!");
        }
    }
}
```

Если у вас имеется некоторый опыт программирования на C или C++, то вам, вероятно, известно, что в этих языках отсутствуют какие-либо ограничения на имена переменных, объявляемых во внутренней области действия. Так, в C и C++ объявление переменной `count` в блоке внешнего цикла `for` из приведенного выше примера программы вполне допустимо, несмотря на то, что такая же переменная уже объявлена во внешнем блоке. В этом случае переменная во внутреннем блоке скрывает переменную из внешнего блока. Создатели Java решили, что подобное сокрытие имен переменных может привести к программным ошибкам, и поэтому запретили его.

Операторы

В Java предусмотрено большое количество операторов. *Оператор* — это знак, указывающий компилятору на необходимость выполнения определенного математического или логического действия. Операторы в Java делятся на четыре общие категории: арифметические, поразрядные, логические и отношения. Кроме того, в Java определены некоторые дополнительные операторы для применения в особых случаях. В этой главе будут рассмотрены арифметические и логические операторы, а также операторы отношения присваивания. А о поразрядных и специальных операторах речь пойдет далее в этой книге.

Арифметические операторы

В языке Java определены следующие арифметические операторы.

Оператор	Выполняемое действие
+	Сложение (а также унарный плюс)
-	Вычитание (а также унарный минус)
*	Умножение
/	Деление
%	Деление по модулю
++	Инкремент
--	Декремент

Операторы +, -, * и / действуют в Java таким же образом, как и в любом другом языке программирования в частности и математике вообще, т.е. выполняют обычные арифметические операции. Их можно применять к числовым данным встроенных типов. Они также применимы к объектам типа char.

Несмотря на то что арифметические операции хорошо всем известны, у них имеются некоторые особенности, требующие специального пояснения. Во-первых, если оператор / применяется к целым числам, остаток от деления отбрасывается. Например, результат целочисленного деления 10 / 3 равен 3. А для получения остатка от деления следует применить оператор деления по модулю %. В Java он действует таким же образом, как и в других языках программирования. Например, в результате операции 10 % 3 будет получено значение 1. Оператор % применяется не только к целым числам, но и к числам с плавающей точкой. Следовательно, в результате операции 10,0 % 3,0 также будет получено значение 1. Ниже приведен пример программы, демонстрирующий применение оператора %.

```
// Демонстрация оператора %.
class ModDemo {
    public static void main(String args[]) {
        int iresult, irem;
        double dresult, drem;

        iresult = 10 / 3;
        irem = 10 % 3;

        dresult = 10.0 / 3.0;
        drem = 10.0 % 3.0;

        System.out.println("Result and remainder of 10 / 3: " +
                           iresult + " " + irem);
        System.out.println("Result and remainder of 10.0 / 3.0: " +
                           dresult + " " + drem);
    }
}
```

Выполнение этой программы дает следующий результат:

```
Result and remainder of 10 / 3: 3 1
Result and remainder of 10.0 / 3.0: 3.3333333333333335 1.0
```

Как видите, оператор `%` возвращает остаток от деления как целых чисел, так и чисел с плавающей точкой.

Операторы инкремента и декремента

Операторы `++` и `--`, выполняющие положительное (инкремент) и отрицательное (декремент) приращение значений, были уже представлены в главе 1. Как будет показано ниже, у этих операторов имеется ряд интересных особенностей. Рассмотрим подробнее действия, выполняемые операторами инкремента и декремента.

Оператор инкремента добавляет к своему операнду единицу, а оператор декремента вычитает единицу из операнда. Следовательно, оператор

```
x = x + 1;
```

дает такой же результат, как и оператор

```
x++;
```

а оператор

```
x = x - 1;
```

дает такой же результат, как и оператор

```
--x;
```

Оба оператора инкремента и декремента могут быть как префиксным (предварять операнд), так и постфиксным (следовать за операндом). Например, оператор

```
x = x + 1;
```

можно переписать так:

```
++x; // префиксная форма
```

или так:

```
x++; // постфиксная форма
```

В приведенных выше примерах не имело особого значения, применять ли оператор инкремента в префиксной или постфиксной форме. Но когда операторы инкремента и декремента являются частью более крупного выражения, отличия между этими формами уже имеют значение. Так, если оператор инкремента или декремента предшествует операнду, то сначала выполняется соответствующее приращение, а затем получается значение операнда для последующего его использования в других элементах выражения. А если оператор инкремента или декремента следует за операндом, то сначала получается значение операнда, а затем выполняется инкремент или декремент. Рассмотрим следующие строки кода:

```
x = 10;
y = ++x;
```

После их выполнения значение переменной `y` будет равно **11**. Но если изменить код так, как показано ниже, результат будет другим.

```
x = 10;
y = x++;
```

Теперь значение переменной `y` равно **10**. Но в обоих случаях значение переменной `x` будет равно **11**. Возможность контролировать момент выполнения операции инкремента или декремента дает немало преимуществ при написании программ.

Операторы отношения и логические операторы

Операторы *отношения* отличаются от *логических* операторов тем, что первые определяют отношения между значениями, а вторые связывают вместе логические значения (`true` или `false`), получаемые в результате определения отношений между значениями. Операторы отношения возвращают логическое значение `true` или `false`, и поэтому они нередко используются совместно с логическими операторами. По этой причине они и рассматриваются вместе.

Ниже перечислены операторы отношения.

Оператор	Значение
<code>==</code>	Равно
<code>!=</code>	Неравно
<code>></code>	Больше
<code><</code>	Меньше
<code>>=</code>	Больше или равно
<code><=</code>	Меньше или равно

Далее перечислены логические операторы.

Оператор	Значение
<code>&</code>	И
<code> </code>	ИЛИ
<code>^</code>	Исключающее ИЛИ
<code> </code>	Укороченное ИЛИ
<code>&&</code>	Укороченное И
<code>!</code>	НЕ

Результатом выполнения оператора отношения или логического оператора является логическое значение типа `boolean`.

В Java все объекты могут быть проверены на равенство или неравенство с помощью операторов `==` и `!=` соответственно. Но операторы `<`, `>`, `<=` и `>=` могут быть применены только к тем типам данных, для которых определено отношение порядка. Следовательно, все операторы отношения можно применять к данным числовых типов и типа `char`. А логические значения типа `boolean` можно проверить только на равенство или неравенство, поскольку истинные (`true`) и ложные (`false`) значения не имеют отношения порядка. Например, выражение `true > false` не имеет смысла в Java.

Операнды логических операторов должны иметь тип `boolean`, как, впрочем, и результаты выполнения этих операторов. Логические операторы `&`, `|`, `^` и `!` выполняют логические операции И, ИЛИ, исключающее ИЛИ и НЕ в соответствии со следующей таблицей истинности.

<code>p</code>	<code>q</code>	<code>p & q</code>	<code>p q</code>	<code>p ^ q</code>	<code>!p</code>
false	false	false	false	false	true
true	false	false	true	true	false

false	true	false	true	true	true
true	true	true	true	false	false

Как следует из приведенной выше таблицы, результатом выполнения логической операции **исключающее ИЛИ** будет истинное значение (`true`), если один и только один ее операнд имеет логическое значение `true`.

Приведенный ниже пример программы демонстрирует применение некоторых операторов отношения и логических операторов.

// Демонстрация операторов отношения и логических операторов.

```
class RelLogOps {
    public static void main(String args[]) {
        int i, j;
        boolean b1, b2;

        i = 10;
        j = 11;
        if(i < j) System.out.println("i < j");
        if(i <= j) System.out.println("i <= j");
        if(i != j) System.out.println("i != j");
        if(i == j) System.out.println("this won't execute");
        if(i >= j) System.out.println("this won't execute");
        if(i > j) System.out.println("this won't execute");

        b1 = true;
        b2 = false;
        if(b1 & b2) System.out.println("this won't execute");
        if(!(b1 & b2)) System.out.println("!(b1 & b2) is true");
        if(b1 | b2) System.out.println("b1 | b2 is true");
        if(b1 ^ b2) System.out.println("b1 ^ b2 is true");
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
i < j
i <= j
i != j
!(b1 & b2) is true
b1 | b2 is true
b1 ^ b2 is true
```

Укороченные логические операторы

В Java предусмотрены также специальные, *укороченные* варианты логических операторов **И** и **ИЛИ**, предназначенные для получения более эффективного кода. Поясним это на следующих примерах логических операций. Если первый операнд логической операции **И** имеет ложное значение (`false`), то ее результат будет иметь ложное значение независимо от значения второго операнда. Если же первый операнд логической операции **ИЛИ** имеет истинное значение (`true`), то ее результат будет иметь истинное значение независимо от значения второго операнда. Благодаря тому что значение второго операнда в этих операциях вычислять не нужно, экономится время и повышается эффективность кода.

Укороченная логическая операция **И** выполняется с помощью оператора **&&**, а укороченная логическая операция **ИЛИ** — с помощью оператора **||**. Этим укороченным логическим операторам соответствуют обычные логические операторы **&** и **|**. Единственное отличие укороченного логического оператора от обычного заключается в том, что второй его операнд вычисляется только по мере необходимости.

В приведенном ниже примере программы демонстрируется применение укороченного логического оператора **И**. В этой программе с помощью операции деления по модулю определяется следующее: делится ли значение переменной *d* на значение переменной *n* нацело. Если остаток от деления *n/d* равен нулю, то *n* делится на *d* нацело. Но поскольку данная операция подразумевает деление, то для проверки условия деления на нуль служит укороченный логический оператор **И**.

```
// Демонстрация укороченных логических операторов.
class SCops {
    public static void main(String args[]) {
        int n, d, q;

        n = 10;
        d = 2;
        if(d != 0 && (n % d) == 0)
            System.out.println(d + " is a factor of " + n);

        d = 0; // установить нулевое значение в переменной d

        // Второй операнд не вычисляется, поскольку значение
        // переменной d равно нулю. Таким образом, укороченный
        // логический оператор предотвращает деление на нуль.
        if(d != 0 && (n % d) == 0) //
            System.out.println(d + " is a factor of " + n);

        /* А теперь те же самые действия выполняются без укороченного
           логического оператора. На этот раз вычисляются оба операнда,
           в результате чего возникает ошибка деления на нуль.
        */
        if(d != 0 & (n % d) == 0)
            System.out.println(d + " is a factor of " + n);
    }
}
```

Для предотвращения возможности деления на нуль в условном операторе **if** сначала проверяется, равно ли нулю значение переменной *d*. Если эта проверка дает истинный результат, вычисление второго операнда укороченного логического оператора **И** не выполняется. Так, если значение переменной *d* равно 2, вычисляется остаток от деления по модулю. Если же значение переменной *d* равно нулю, операция деления по модулю пропускается, а следовательно, предотвращается деление на нуль. В конце рассматриваемой здесь программы применяется обычный логический оператор **И**, в котором вычисляются оба операнда, а это может привести к делению на нуль при выполнении данной программы.

И последнее замечание: в формальной спецификации Java укороченный оператор **И** называется *условным логическим оператором И*, а укороченный оператор **ИЛИ** — *условным логическим оператором ИЛИ*, но чаще подобные операторы называются укороченными.

Оператор присваивания

Оператор присваивания уже не раз применялся в примерах программ, начиная с главы 1. И теперь настало время дать ему формальное определение. *Оператор присваивания* обозначается одиночным знаком равенства (=). В Java он выполняет те же действия, что и в других языках программирования. Ниже приведена общая форма этого оператора.

переменная = *выражение*

где *переменная* и *выражение* должны иметь совместимые типы.

У оператора присваивания имеется одна интересная особенность, о которой вам будет полезно знать: он позволяет создавать цепочку операций присваивания. Рассмотрим, например, следующий фрагмент кода:

```
int x, y, z;  
x = y = z = 100; // присвоить значение 100 переменным x, y и z
```

В приведенном выше фрагменте кода одно и то же значение **100** задается для переменных *x*, *y* и *z* с помощью единственного оператора присваивания **=**, получающего всякий раз значение от операнда из правой части выражения. Таким образом, значение **100** присваивается сначала переменной *z*, затем переменной *y* и, наконец, переменной *x*. Такой способ присваивания по цепочке удобен для задания общего значения целой группе переменных.

Укороченные операторы присваивания

В Java предусмотрены специальные *укороченные* операторы присваивания, упрощающие программирование некоторых операций присваивания. Обратимся сначала к простому примеру. Приведенный ниже оператор присваивания

```
x = x + 10;
```

можно переписать, используя следующий укороченный оператор присваивания:

```
x += 10;
```

Пара операторов **+=** указывает компилятору на то, что переменной *x* должно быть присвоено ее первоначальное значение, увеличенное на **10**.

Рассмотрим еще один пример. Оператор

```
x = x - 100;
```

и оператор

```
x -= 100;
```

выполняют одни и те же действия. Оба оператора присваивают переменной *x* ее первоначальное значение, уменьшенное на **100**.

Для многих двоичных операций в Java, т.е. операций, требующих наличия двух операндов, существуют отдельные укороченные операторы присваивания. Общая форма всех этих операторов имеет следующий вид:

переменная op = *выражение*

где *op* — арифметический или логический оператор, применяемый вместе с оператором присваивания.

Ниже перечислены укороченные операторы присваивания для арифметических и логических операций.

<code>+=</code>	<code>-=</code>	<code>*=</code>	<code>/=</code>
<code>%=</code>	<code>&=</code>	<code> =</code>	<code>^=</code>

Каждый из перечисленных выше операторов состоит из двух знаков, обозначающих арифметическую или логическую операцию, а также операцию присваивания. Поэтому их часто еще называют *составными* операторами присваивания.

У составных операторов присваивания имеются два главных преимущества. Во-первых, они более компактны, чем их неукороченные эквиваленты. И во-вторых, дают более эффективный исполняемый код, поскольку левый операнд этих операторов вычисляется только один раз. Именно по этим причинам составные операторы присваивания чаще всего применяются в программах, профессионально написанных на Java.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Если укороченные операторы более эффективны, то почему наряду с ними в Java используются также обычные логические операторы?

ОТВЕТ. В некоторых случаях требуется вычислять оба операнда логического оператора, чтобы проявились побочные эффекты. Рассмотрим следующий пример:

```
// Демонстрация особого значения побочных эффектов.
class SideEffects {
    public static void main(String args[]) {
        int i;

        i = 0;

        /* Значение переменной i все равно инкрементируется, несмотря
           на то, что проверяемое условие в операторе if ложно. */
        if(false & (++i < 100))
            System.out.println("this won't be displayed");
        System.out.println("if statements executed: " + i); // выводится 1

        /* В данном случае значение переменной i не инкрементируется,
           поскольку второй операнд укороченного логического оператора
           не вычисляется, а следовательно, инкремент пропускается. */
        if(false && (++i < 100))
            System.out.println("this won't be displayed");
        System.out.println("if statements executed: " + i); // still 1 !!
    }
}
```

Как следует из приведенного выше фрагмента кода и комментариев к нему, в первом условном операторе `if` значение переменной `i` должно увеличиваться на единицу независимо от того, выполняется ли условие этого оператора. Но когда во втором условном операторе `if` применяется укороченный логический оператор, значение переменной `i` не инкрементируется, поскольку первый операнд в проверяемом условии имеет логическое значение `false`. Следовательно, если логика программы требует, чтобы второй операнд логического оператора непременно вычислялся, следует применять обычные, а не укороченные формы логических операторов.

Преобразование типов при присваивании

При написании программ очень часто возникает потребность в присваивании значения, хранящегося в переменной одного типа, переменной другого типа. Например, значение `int`, возможно, потребуется присвоить переменной `float`, как показано ниже.

```
int i;
float f;

i = 10;
f = i; // присвоить значение переменной типа int переменной типа float
```

Если типы данных являются совместимыми, значение из правой части оператора присваивания автоматически преобразуется к типу данных в левой его части. Так, в приведенном выше фрагменте кода значение переменной `i` преобразуется в тип `float`, а затем присваивается переменной `f`. Но ведь Java — язык со строгим контролем типов, и далеко не все типы данных в нем совместимы, поэтому неявное преобразование типов выполняется не всегда. В частности, типы `boolean` и `int` не являются совместимыми.

Автоматическое преобразование типов в операции присваивания выполняется при соблюдении следующих условий:

- оба типа являются совместимыми;
- целевой тип обладает более широким диапазоном допустимых значений, чем исходный тип.

Если оба перечисленных выше условия соблюдаются, происходит расширяющее преобразование типов. Например, диапазона значений, допустимых для типа `int`, совершенно достаточно, чтобы представить любое значение типа `byte`, а кроме того, оба эти типа данных являются целочисленными. Поэтому и происходит автоматическое преобразование типа `byte` в тип `int`.

С точки зрения расширяющего преобразования типов целочисленные типы и типы с плавающей точкой совместимы друг с другом. Например, приведенная ниже программа написана корректно, поскольку преобразование типа `long` в тип `double` является расширяющим и выполняется автоматически.

```
// Демонстрация автоматического преобразования типа long в тип double.
class LtoD {
    public static void main(String args[]) {
        long L;
        double D;

        L = 100123285L;
        // Автоматическое преобразование типа long в тип double.
        D = L;

        System.out.println("L and D: " + L + " " + D);
    }
}
```

В то же время тип `double` не может быть автоматически преобразован в тип `long`, поскольку такое преобразование уже не является расширяющим. Следовательно, приведенный ниже вариант той же самой программы оказывается некорректным.

```
// *** Эта программа не подлежит компиляции. ***
class LtoD {
    public static void main(String args[]) {
        long L;
        double D;

        D = 100123285.0;
        // Тип double не преобразуется автоматически в тип long.
        L = D; // Ошибка!!!

        System.out.println("L and D: " + L + " " + D);
    }
}
```

Автоматическое преобразование числовых типов в тип `char` или `boolean` не производится. Кроме того, типы `char` и `boolean` несовместимы друг с другом. Тем не менее переменной `char` может быть присвоено значение, представленное целочисленным литералом.

Приведение несовместимых типов

Несмотря на всю полезность неявных автоматических преобразований типов, они неспособны удовлетворить все потребности в программировании, поскольку допускают лишь расширяющие преобразования совместимых типов. А во всех остальных случаях приходится обращаться к приведению типов. *Приведение* — это команда компилятору преобразовать результат вычисления выражения в указанный тип. А для этого требуется явное преобразование типов. Ниже показана общая форма приведения типов.

(целевой_тип) выражение

где *целевой_тип* обозначает тот тип, в который желательно преобразовать указанное выражение. Так, если требуется привести значение, возвращаемое выражением `x / y`, к типу `int`, это можно сделать следующим образом:

```
double x, y;
// ...
(int) (x / y)
```

В данном случае приведение типов обеспечит преобразование результатов выполнения выражения в тип `int`, несмотря на то, что переменные `x` и `y` принадлежат к типу `double`. Выражение `x / y` следует непременно заключить в круглые скобки, иначе будет преобразован не результат деления, а только значение переменной `x`. Приведение типов в данном случае требуется потому, что автоматическое преобразование типа `double` в тип `int` не выполняется.

Если приведение типов приводит к *сужающему преобразованию*, то часть информации может быть потеряна. Например, в результате приведения типа `long` к типу `int` часть информации потеряется, если значение типа `long` окажется больше диапазона представления чисел для типа `int`, поскольку старшие разряды этого числового значения отбрасываются. Когда же значение с плавающей точкой приводится к целочисленному, в результате усечения теряется дробная часть этого числового значения. Так, если присвоить значение `1,23` целочисленной переменной, то в результате в ней останется лишь целая часть исходного числа (`1`), а дробная его часть (`0,23`) будет потеряна.

Ниже приведен пример программы, демонстрирующий некоторые виды преобразований, требующие явного приведения типов.

```
// Демонстрация приведения типов.
class CastDemo {
    public static void main(String args[]) {
        double x, y;
        byte b;
        int i;
        char ch;

        x = 10.0;
        y = 3.0;
        // В данном случае теряется дробная часть числа.
        i = (int) (x / y); // привести тип double к типу int
        System.out.println("Integer outcome of x / y: " + i);

        i = 100;
        // А в этом случае данные не теряются. Тип byte может
        // содержать значение 100.
        b = (byte) i;
        System.out.println("Value of b: " + b);

        i = 257;
        // На этот раз данные теряются. Тип byte не может
        // содержать значение 257.
        b = (byte) i;
        System.out.println("Value of b: " + b);

        b = 88; // Представление символа X в коде ASCII.
        // И снова требуется явное приведение несовместимых типов.
        ch = (char) b;
        System.out.println("ch: " + ch);
    }
}
```

Выполнение этой программы дает следующий результат:

```
Integer outcome of x / y: 3
Value of b: 100
Value of b: 1
ch: X
```

В данной программе приведение выражения (x / y) к типу `int` означает потерю дробной части числового значения результата деления. Когда переменной `b` присваивается значение **100** из переменной `i`, данные не теряются, поскольку диапазон допустимых значений у типа `byte` достаточен для представления этого значения. Далее при попытке присвоить переменной `b` значение **257** снова происходит потеря данных, поскольку значение **257** оказывается за пределами диапазона допустимых значений для типа `byte`. И наконец, когда переменной `char` присваивается содержимое переменной типа `byte`, данные не теряются, но явное приведение типов все же требуется.

Предшествование операторов

В табл. 2.3 приведен порядок предшествования всех операторов в Java: от самого высокого до самого низкого. В эту таблицу включен ряд операторов, рассматриваемых да-

лее в этой книге. Формально разделители `[]`, `()` и `.` могут действовать как операторы, и в этом случае они будут иметь наивысший порядок предшествования.

Таблица 2.3. Предшествование операторов в Java

Наивысший порядок						
<code>++</code> (постфиксный)	<code>--</code> (постфиксный)					
<code>++</code> (префиксный)	<code>--</code> префиксный)	<code>~</code>	<code>!</code>	<code>+</code> (унарный плюс)	<code>-</code> (унарный минус)	(приведение типов)
<code>*</code>	<code>/</code>	<code>%</code>				
<code>+</code>	<code>-</code>					
<code>>></code>	<code>>>></code>	<code><<</code>				
<code>></code>	<code>>=</code>	<code><</code>	<code><=</code>	<code>instanceof</code>		
<code>==</code>	<code>!=</code>					
<code>&</code>						
<code>^</code>						
<code> </code>						
<code>&&</code>						
<code> </code>						
<code>?:</code>						
<code>=</code>	<code>op=</code>					
Наинизший порядок						

Пример для опробования 2.2.

Отображение таблицы истинности для логических операторов

Исходный файл: `LogicalOpTable.java`

В этом проекте предстоит создать программу, которая отображает таблицу истинности для логических операторов Java.

Для удобства восприятия отображаемой информации следует выровнять столбцы таблицы. В данном проекте используется ряд языковых средств, рассмотренных ранее в этой главе, включая управляющие последовательности и логические операторы, а также демонстрируются отличия в предшествовании арифметических и логических операторов.

Последовательность действий

1. Создайте новый файл `LogicalOpTable.java`.
2. Для того чтобы обеспечить выравнивание столбцов таблицы, в каждую выводимую строку следует ввести символы `\t`. В качестве примера ниже приведен вызов метода `println()` для отображения заголовков таблицы.

```
System.out.println("P\tQ\tAND\tOR\tXOR\tNOT");
```
3. Для того чтобы сведения об операторах располагались под соответствующими заголовками, в каждую последующую строку таблицы должны быть введены символы табуляции.
4. Введите в файл `LogicalOpTable.java` исходный код программы, как показано ниже.

```
// Пример для опробования 2.2.
// Отображение таблицы истинности для логических операторов.
class LogicalOpTable {
    public static void main(String args[]) {
        boolean p, q;

        System.out.println("P\tQ\tAND\tOR\tXOR\tNOT");

        p = true; q = true;
        System.out.print(p + "\t" + q + "\t");
        System.out.print((p&q) + "\t" + (p|q) + "\t");
        System.out.println((p^q) + "\t" + (!p));

        p = true; q = false;
        System.out.print(p + "\t" + q + "\t");
        System.out.print((p&q) + "\t" + (p|q) + "\t");
        System.out.println((p^q) + "\t" + (!p));

        p = false; q = true;
        System.out.print(p + "\t" + q + "\t");
        System.out.print((p&q) + "\t" + (p|q) + "\t");
        System.out.println((p^q) + "\t" + (!p));

        p = false; q = false;
        System.out.print(p + "\t" + q + "\t");
        System.out.print((p&q) + "\t" + (p|q) + "\t");
        System.out.println((p^q) + "\t" + (!p));
    }
}
```

Обратите внимание на то, что в операторах с вызовами метода `println()` логические операторы заключены в круглые скобки. Эти скобки необходимы для соблюдения предшествования операторов. В частности, арифметический оператор `+` имеет более высокий порядок предшествования, чем логические операторы.

5. Скомпилируйте программу и запустите ее на выполнение, чтобы вывести на экран следующий результат:

P	Q	AND	OR	XOR	NOT
true	true	true	true	false	false
true	false	false	true	true	false
false	true	false	true	true	true
false	false	false	false	false	true

6. Попробуйте видоизменить программу таким образом, чтобы вместо логических значений `true` и `false` отображались значения **1** и **0**. Это потребует больших усилий, чем кажется на первый взгляд!

Выражения

Операторы, переменные и литералы являются составными частями *выражений*. Выражением в Java может стать любое допустимое сочетание этих составных частей. Выражения должны быть уже знакомы вам по предыдущим примерам программ. Более

того, вы изучали их в школьном курсе алгебры. Но некоторые их особенности все же нуждаются в обсуждении.

Преобразование типов в выражениях

В выражении можно свободно употреблять два или несколько типов данных, при условии их совместимости друг с другом. Например, в одном выражении допускается применение типов `short` и `long`, поскольку оба типа являются числовыми. Когда в выражении употребляются разные типы данных, они преобразуются в один и тот же тип по принятым в Java *правилам продвижения типов*.

Сначала все значения типа `char`, `byte` и `short` продвигаются к типу `int`. Затем все выражение продвигается к типу `long`, если хотя бы один из его операндов принадлежит к типу `long`. Далее все выражение продвигается к типу `float`, если хотя бы один из операндов относится к типу `float`. А если какой-нибудь из операндов относится к типу `double`, то результат также относится к типу `double`.

Очень важно иметь в виду, что правила продвижения типов применяются только к значениям, над которыми выполняются действия по мере вычисления выражения. Так, если значение переменной типа `byte` при вычислении выражения продвигается к типу `int`, за пределами выражения эта переменная будет по-прежнему относиться к типу `byte`. Следовательно, продвижение типов затрагивает только вычисление выражения.

Но продвижение типов может иногда привести к неожиданным результатам. Если, например, в арифметической операции используются два значения типа `byte`, то происходит следующее. Сначала операнды типа `byte` продвигаются к типу `int`. А затем выполняется операция, дающая результат типа `int`. Следовательно, результат выполнения операции, в которой участвуют два значения типа `byte`, будет иметь тип `int`. Но ведь это не тот результат, который можно было бы с очевидностью предположить. Рассмотрим следующий пример программы:

```
// Неожиданный результат продвижения типов!
class PromDemo {
    public static void main(String args[]) {
        byte b;
        int i;

        b = 10;
        // В данном случае приведение типов не требуется, так как
        // результат вычисления выражения уже относится к типу int.
        i = b * b;

        b = 10;
        // А в этом случае приведение типов требуется для
        // присваивания значения int переменной типа byte!
        b = (byte) (b * b); // cast needed!!

        System.out.println("i and b: " + i + " " + b);
    }
}
```

Как ни странно, но когда результат вычисления выражения `b*b` присваивается обратно переменной `b`, то возникает потребность в приведении к типу `byte`! Объясняется это тем, что в выражении `b*b` значение переменной `b` продвигается к типу `int` и поэтому не может быть присвоено переменной типа `byte` без приведения типов. Имейте это об-

стоятельство в виду, если получите неожиданное сообщение об ошибке несовместимости типов в выражениях, которые, на первый взгляд, кажутся совершенно правильными.

Аналогичная ситуация возникает при выполнении операций с символьными операндами. Например, в следующем фрагменте кода требуется обратное приведение к типу `char`, поскольку операнды `ch1` и `ch2` в выражении продвигаются к типу `int`:

```
char ch1 = 'a', ch2 = 'b';  
ch1 = (char) (ch1 + ch2);
```

Без приведения типов результат сложения операндов `ch1` и `ch2` будет иметь тип `int`, и поэтому его нельзя присвоить переменной типа `char`.

Приведение типов необходимо не только при присваивании значения переменной. Рассмотрим в качестве примера следующую программу. В ней приведение типов выполняется для того, чтобы дробная часть числового значения типа `double` не была утеряна. В противном случае операция деления будет выполняться над целыми числами.

```
// Приведение типов для правильного вычисления выражения.  
class UseCast {  
    public static void main(String args[]) {  
        int i;  
  
        for(i = 0; i < 5; i++) {  
            System.out.println(i + " / 3: " + i / 3);  
            System.out.println(i + " / 3 with fractions: "  
                               + (double) i / 3);  
            System.out.println();  
        }  
    }  
}
```

Ниже приведен результат выполнения данной программы.

```
0 / 3: 0  
0 / 3 with fractions: 0.0  
  
1 / 3: 0  
1 / 3 with fractions: 0.3333333333333333  
  
2 / 3: 0  
2 / 3 with fractions: 0.6666666666666666  
  
3 / 3: 1  
3 / 3 with fractions: 1.0  
  
4 / 3: 1  
4 / 3 with fractions: 1.3333333333333333
```

Пробелы и круглые скобки

Для того чтобы сделать выражения в коде Java более удобными для восприятия, в них можно использовать символы табуляции и пробелы. Например, ниже приведены два варианта одного и того же выражения, но второй вариант читается гораздо легче.

```
x=10/y*(127/x);  
  
x = 10 / y * (127/x);
```


Круглые скобки повышают порядок предшествования содержащихся в них операций. (Аналогичное правило применяется и в алгебре.) Избыточные скобки допустимы. Они не приводят к ошибке и не замедляют выполнение программы. В некоторых случаях лишние скобки даже желательны. Они поясняют порядок вычисления выражения как для вас, так и для тех, кто будет разбирать исходный код вашей программы. Какое из приведенных ниже двух выражений воспринимается легче?

```
x = y/3-34*temp+127;
```

```
x = (y/3) - (34*temp) + 127;
```

Упражнение для самопроверки по материалу главы 2

1. Почему в Java строго определены диапазоны допустимых значений и области действия простых типов?
2. Что собой представляет символьный тип в Java и чем он отличается от символьного типа в ряде других языков программирования?
3. Переменная типа `boolean` может иметь любое значение, поскольку любое ненулевое значение интерпретируется как истинное. Верно или неверно?
4. Допустим, результат выполнения программы выглядит следующим образом:

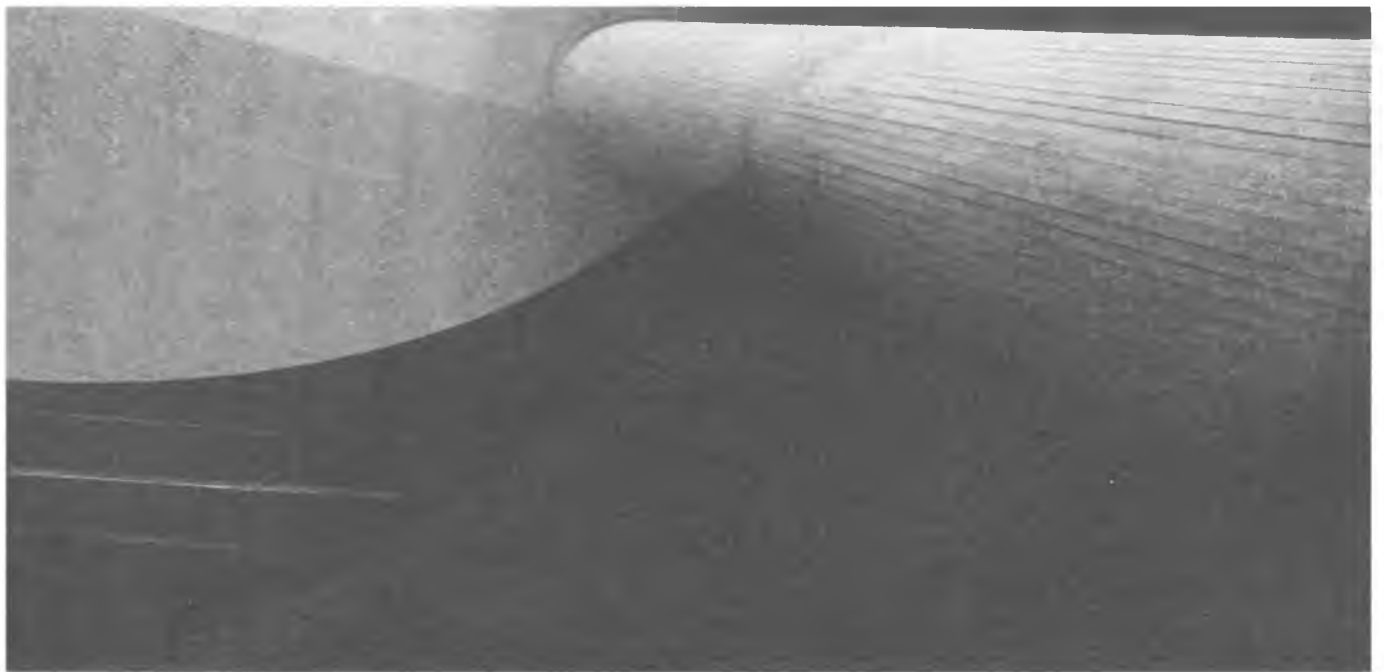
```
One
Two
Three
```

Напишите строку кода с вызовом метода `println()`, где этот результат выводится в одной символьной строке.

5. Какая ошибка допущена в следующем фрагменте кода:

```
for(i = 0; i < 10; i++) {
    int sum;
    sum = sum + i;
}
System.out.println("Sum is: " + sum);
```

6. Поясните отличие между префиксной и постфиксной формами оператора инкремента.
7. Покажите, каким образом укороченный логический оператор `И` может предотвратить деление на нуль.
8. К какому типу продвигаются типы `byte` и `short` при вычислении выражения?
9. Когда возникает потребность в явном приведении типов?
10. Напишите программу, которая находила бы простые числа в пределах от 2 до 100.
11. Оказывают ли лишние скобки влияние на эффективность выполнения программ?
12. Определяет ли кодовый блок область действия переменных?



Глава 3

Управляющие операторы

Основные навыки и понятия

- Ввод символов с клавиатуры
- Полная форма условного оператора `if`
- Применение оператора `switch`
- Полная форма цикла `for`
- Применение цикла `while`
- Применение цикла `do-while`
- Применение оператора `break` для выхода из цикла
- Использование оператора `break` в качестве оператора `goto`
- Применение оператора `continue`
- Вложенные циклы

В этой главе вы ознакомитесь с операторами, управляющими ходом выполнения программы. Существуют три категории управляющих операторов: операторы *выбора*, к числу которых относятся операторы `if` и `switch`, *итерационные* операторы, в том числе операторы цикла `for`, `while`, `do-while`, а также операторы *перехода*, включая `break`, `continue` и `return`. Все эти управляющие операторы, кроме оператора `return`, обсуждаемого далее в книге, подробно рассматриваются в этой главе, в начале которой будет показано, каким образом организуется простой ввод данных с клавиатуры.

Ввод символов с клавиатуры

Прежде чем приступить к рассмотрению управляющих операторов в Java, уделим немного внимания средствам, которые позволяют писать интерактивные программы. В рассмотренных до сих пор примерах программ данные *выводились* на экран, но у пользователя не было возможности *вводить* данные. В этих программах, в частности, применялся консольный вывод, но не консольный ввод (с клавиатуры). И объясняется это тем, что возможности ввода данных с клавиатуры в Java опираются на языковые средства, рассматриваемые далее в этой книге. Кроме того, большинство реальных программ на Java и апплетов имеют графический и оконный, а не консольный интерфейс. Именно по этим причинам консольный ввод нечасто применяется в примерах программ, представленных в данной книге. Но имеется один вид консольного ввода, который реализуется очень просто. Это чтение символов с клавиатуры. А поскольку ввод символов применяется в ряде примеров, представленных в этой главе, мы и начнем ее с обсуждения данного вопроса.

Для чтения символа с клавиатуры достаточно вызвать метод `System.in.read()`, где `System.in` — объект ввода (с клавиатуры), дополняющий объект вывода `System.out`. Метод `read()` ожидает нажатия пользователем клавиш, после чего возвращает результат. Возвращаемый им символ представлен целочисленным значением, и поэтому, прежде чем присвоить его символьной переменной, следует выполнить явное его приведе-

ние к типу `char`. По умолчанию данные, вводимые с консоли, буферизуются *построчно*. Под термином *буфер* здесь подразумевается небольшая область памяти, выделяемая для хранения символов перед тем, как они будут прочитаны программой. В данном случае в буфере хранится целая текстовая строка, и поэтому для передачи программе любого введенного с клавиатуры символа следует нажать клавишу `<Enter>`. Ниже приведен пример программы, читающей символы, вводимые с клавиатуры.

```
// Чтение символа с клавиатуры.
class KbIn {
    public static void main(String args[])
        throws java.io.IOException {

        char ch;

        System.out.print("Press a key followed by ENTER: ");
        // Ввод символа с клавиатуры.
        ch = (char) System.in.read(); // получить значение типа char

        System.out.println("Your key is: " + ch);
    }
}
```

Выполнение этой программы может дать, например, следующий результат:

```
Press a key followed by ENTER: t
Your key is: t
```

Обратите внимание на то, что метод `main()` начинается со следующих строк кода:

```
public static void main(String args[])
    throws java.io.IOException {
```

В рассматриваемой здесь программе применяется метод `System.in.read()`, и поэтому в ее код следует ввести оператор `throws java.io.IOException`. Этот оператор требуется для обработки ошибок, которые могут возникнуть в процессе ввода данных. Он является частью механизма обработки исключений в Java, более подробно рассматриваемого в главе 9. А до тех пор не обращайтесь особого внимания на этот оператор, принимая во внимание лишь его назначение.

Построчная буферизация вводимых данных средствами `System.in` часто приводит к недоразумениям. При нажатии клавиши `<Enter>` в поток ввода записывается последовательность, состоящая из символов возврата каретки и перевода строки. Эти символы ожидают чтения из буфера ввода. Поэтому в некоторых приложениях, возможно, потребуется удалить символы возврата каретки и перевода строки, прежде чем переходить к следующей операции ввода. Для этого достаточно прочитать их из буфера ввода. Соответствующий пример реализации подобного решения на практике будет представлен далее в главе.

Условный оператор `if`

Этот условный оператор уже был представлен в главе 1, а здесь он будет рассмотрен более подробно. Ниже приведена полная форма условного оператора `if`.

```
if(условие) оператор;
else оператор;
```

где *условие* — это некоторое условное выражение, а *оператор* — адресат операторов `if` и `else`. Оператор `else` не является обязательным. Адресатами обоих операторов, `if` и `else`, могут также служить блоки операторов. Ниже приведена общая форма условного оператора `if`, в котором используются блоки операторов.

```
if(условие)
{
    последовательность операторов
}
else
{
    последовательность операторов
}
```

Если условное выражение оказывается истинным, то выполняется адресат оператора `if`. В противном случае выполняется адресат оператора `else`, если таковой существует. Но одновременно не может выполняться и то и другое. Условное выражение, управляющее оператором `if`, должно давать результат типа `bool`.

Для того чтобы продемонстрировать применение оператора `if` (и ряда других управляющих операторов) на практике, разработаем простую игру, основанную на угадывании. Возможно, она понравится вашим детям. В первой версии этой игры программа предложит пользователю угадать задуманную букву от **A** до **Z**. Если пользователь правильно угадает букву и нажмет на клавиатуре соответствующую клавишу, программа выведет сообщение **** Right **** (Правильно). Ниже приведен исходный код программы, реализующей эту игру.

```
// Игра в угадывание букв.
class Guess {
    public static void main(String args[])
        throws java.io.IOException {
        char ch, answer = 'S';

        System.out.println("I'm thinking of a letter between A and Z.");
        System.out.print("Can you guess it: ");

        ch = (char) System.in.read(); // прочитать символ с клавиатуры

        if(ch == answer) System.out.println("** Right **");
    }
}
```

Эта программа выводит на экран сообщение с предложением угадать букву, а затем читает символ с клавиатуры. Используя условный оператор `if`, она сравнивает введенный символ с правильным ответом (в данном случае это буква **S**). Если введена буква **S**, то отображается сообщение об угадывании буквы. Опробуя эту программу, не забывайте, что угадываемую букву следует вводить в верхнем регистре.

В следующей версии программы оператор `else` используется для вывода сообщения о том, что буква не была угадана.

```
// Игра в угадывание букв, вторая версия
class Guess2 {
    public static void main(String args[])
        throws java.io.IOException {
```

```
char ch, answer = 'S';

System.out.println("I'm thinking of a letter between A and Z.");
System.out.print("Can you guess it: ");

ch = (char) System.in.read(); // ввести символ с клавиатуры

if(ch == answer) System.out.println("*** Right ***");
else System.out.println("...Sorry, you're wrong.");
}
}
```

Вложенные условные операторы **if**

Вложенные операторы **if** представляют собой условные операторы **if**, являющиеся адресатами оператора **else**. Подобные условные операторы очень часто встречаются в программах. Но, пользуясь ими, следует помнить, что в Java оператор **else** всегда связан с ближайшим к нему оператором **if**, находящимся в том же кодовом блоке и не связанном с другим оператором **else**. Рассмотрим следующий пример:

```
if(i == 10) {
    if(j < 20) a = b;
    if(k > 100) c = d;
    else a = c; // этот оператор else относится к оператору if(k > 100)
}
else a = d; // а этот оператор else относится к оператору if(i == 10)
```

Как следует из комментариев к приведенному выше фрагменту кода, последний оператор **else** не имеет отношения к оператору **if(j < 20)**, поскольку он не находится с ним в одном кодовом блоке, несмотря на то, что это ближайший оператор **if**, не имеющий себе в пару оператор **else**. Следовательно, последний оператор **else** относится к оператору **if(i == 10)**. А в кодовом блоке оператор **else** связан с оператором **if(k > 100)**, поскольку это самый близкий из всех находящихся к нему операторов **if** в том же самом блоке.

Используя вложенные условные операторы **if**, можно усовершенствовать игру, рассматриваемую здесь в качестве примера. Теперь при неудачной попытке угадать букву пользователю предоставляется дополнительная информация, подсказывающая, насколько он далек от правильного ответа.

// Игра в угадывание букв, третья версия.

```
class Guess3 {
    public static void main(String args[])
        throws java.io.IOException {

        char ch, answer = 'S';

        System.out.println("I'm thinking of a letter between A and Z.");
        System.out.print("Can you guess it: ");

        ch = (char) System.in.read(); // ввести символ с клавиатуры

        if(ch == answer) System.out.println("*** Right ***");
        else {
```

```

        System.out.print("...Sorry, you're ");
        // вложенный оператор if
        if(ch < answer) System.out.println("too low");
        else System.out.println("too high");
    }
}

```

Выполнение этой программы может дать, например, следующий результат:

```

I'm thinking of a letter between A and Z.
Can you guess it: Z
...Sorry, you're too high

```

Многоступенчатая конструкция **if-else-if**

В программировании часто применяется *многоступенчатая конструкция if-else-if*, состоящая из вложенных условных операторов **if**. Ниже приведена ее общая форма.

```

if(условие)
    оператор;
else if (условие)
    оператор;
else if (условие)
    оператор;
.
.
.
else
    оператор;

```

Условные выражения в такой конструкции вычисляются сверху вниз. Как только обнаружится истинное условие, выполняется связанный с ним оператор, а все остальные операторы в многоступенчатой конструкции опускаются. Если ни одно из условий не является истинным, то выполняется последний оператор **else**, который зачастую служит в качестве условия, устанавливаемого по умолчанию. Когда же последний оператор **else** отсутствует, а все остальные проверки по условию дают ложный результат, никаких действий вообще не выполняется.

Ниже приведен пример программы, демонстрирующий применение многоступенчатой конструкции **if-else-if**.

```

// Демонстрация многоступенчатой конструкции if-else-if.
class Ladder {
    public static void main(String args[]) {
        int x;

        for(x=0; x<6; x++) {
            if(x==1)
                System.out.println("x is one");
            else if(x==2)
                System.out.println("x is two");
            else if(x==3)
                System.out.println("x is three");
            else if(x==4)
                System.out.println("x is four");

```

```
    else
    // Условие, выполняемое по умолчанию.
    System.out.println("x is not between 1 and 4");
}
}
```

Выполнение этой программы дает следующий результат:

```
x is not between 1 and 4
x is one
x is two
x is three
x is four
x is not between 1 and 4
```

Как видите, устанавливаемый по умолчанию условный оператор `else` выполняется лишь в том случае, если проверки по условию всех предыдущих операторов `if` дают ложный результат.

Оператор `switch`

Вторым оператором выбора в Java является оператор `switch`, который обеспечивает многонаправленное ветвление программы. Следовательно, этот оператор позволяет сделать выбор среди нескольких альтернативных вариантов дальнейшего выполнения программы. Несмотря на то что многонаправленная проверка может быть организована с помощью последовательного ряда вложенных условных операторов `if`, во многих случаях более эффективным оказывается применение оператора `switch`. Этот оператор действует следующим образом. Значение выражения последовательно сравнивается с константами выбора из заданного списка. Как только будет обнаружено совпадение с одним из условий выбора, выполняется последовательность операторов, связанных с этим условием. Ниже приведена общая форма оператора `switch`.

```
switch(выражение) {
    case константа1:
        последовательность операторов
        break;
    case константа2:
        последовательность операторов
        break;
    case константа3:
        последовательность операторов
        break;
    .
    .
    .
    default:
        последовательность операторов
}
```

В версиях Java, предшествующих JDK 7, выражение, управляющее оператором `switch`, должно быть типа `byte`, `short`, `int`, `char` или перечислением. (Подробнее о перечислениях речь пойдет в главе 12.)

Начиная с версии JDK 7, *выражение* может относиться к типу `String`. Это означает, что в современных версиях Java для управления оператором `switch` можно пользоваться символьной строкой. (Этот прием программирования демонстрируется в главе 5 при рассмотрении класса `String`.) Но зачастую в качестве выражения, управляющего оператором `switch`, вместо сложного выражения употребляется простая переменная.

Последовательность операторов из ветви `default` выполняется в том случае, если ни одна из констант выбора не совпадает с заданным выражением. Ветвь `default` не является обязательной. Если же она отсутствует и выражение не совпадает ни с одним из условий выбора, то никаких действий вообще не выполняется. Если же происходит совпадение с одним из условий выбора, то выполняются операторы, связанные с этим условием, вплоть до оператора `break`.

Ниже приведен пример программы, демонстрирующий применение оператора `switch`.

```
// Демонстрация оператора switch.
class SwitchDemo {
    public static void main(String args[]) {
        int i;

        for(i=0; i<10; i++)
            switch(i) {
                case 0:
                    System.out.println("i is zero");
                    break;
                case 1:
                    System.out.println("i is one");
                    break;
                case 2:
                    System.out.println("i is two");
                    break;
                case 3:
                    System.out.println("i is three");
                    break;
                case 4:
                    System.out.println("i is four");
                    break;
                default:
                    System.out.println("i is five or more");
            }
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
i is zero
i is one
i is two
i is three
i is four
i is five or more
i is five or more
i is five or more
```

```
i is five or more
i is five or more
```

Как видите, на каждом шаге цикла выполняются операторы, связанные с совпадающей константой выбора в одной из ветвей `case`, в обход всех остальных ветвей. Когда же значение переменной `i` становится равным или больше пяти, оно не совпадает ни с одной из констант выбора, и поэтому управление получает выражение, следующее за оператором `default`.

Формально оператор `break` может отсутствовать, но, как правило, в реальных приложениях он применяется. При выполнении оператора `break` оператор `switch` завершает работу и управление передается следующему за ним оператору. Если же в последовательности операторов, связанных с совпадающей константой выбора в одной из ветвей `case`, не содержится оператор `break`, то сначала выполняются все операторы в *этой* ветви, а затем операторы, *совпадающие* с константой выбора в следующей ветви `case`. Этот процесс продолжается до тех пор, пока не встретится оператор `break` или же будет достигнут конец оператора `switch`.

В качестве упражнения проанализируйте исходный код приведенной ниже программы. Сможете ли вы предсказать, как будет выглядеть результат ее выполнения?

// Демонстрация оператора **switch** без оператора **break**.

```
class NoBreak {
    public static void main(String args[]) {
        int i;

        for(i=0; i<=5; i++) {
            switch(i) {
                case 0:
                    // Далее следует "провал" в ветвях case оператора switch.
                    System.out.println("i is less than one");
                case 1:
                    System.out.println("i is less than two");
                case 2:
                    System.out.println("i is less than three");
                case 3:
                    System.out.println("i is less than four");
                case 4:
                    System.out.println("i is less than five");
            }
            System.out.println();
        }
    }
}
```

Выполнение этой программы дает следующий результат:

```
i is less than one
i is less than two
i is less than three
i is less than four
i is less than five

i is less than two
i is less than three
i is less than four
```

```

i is less than five

i is less than three
i is less than four
i is less than five

i is less than four
i is less than five

i is less than five

```

Как демонстрирует приведенный выше пример, выполнение программы будет продолжено в следующей ветви `case` в отсутствие оператора `break`. А в следующем примере кода показано, что в операторе `switch` могут присутствовать пустые ветви `case`:

```

switch(i) {
    case 1:
    case 2:
    case 3: System.out.println("i is 1, 2 or 3");
        break;
    case 4: System.out.println("i is 4");
        break;
}

```

Если в приведенном выше фрагменте кода переменная `i` имеет значение 1, 2 или 3, то вызывается первый метод `println()`. А если ее значение равно 4, вызывается второй метод `println()`. Такое расположение нескольких пустых ветвей `case` подряд нередко используется в тех случаях, когда нескольким ветвям должен соответствовать один и тот же общий код.

Вложенные операторы `switch`

Один оператор `switch` может быть частью последовательности операторов другого, внешнего оператора `switch`. И такой оператор `switch` называется *вложенным*. Константы выбора внутреннего и внешнего операторов `switch` могут содержать общие значения, не вызывая никаких конфликтов. Например, следующий фрагмент кода является вполне допустимым:

```

switch(ch1) {
    case 'A': System.out.println("This A is part of outer switch.");
        switch(ch2) {
            case 'A':
                System.out.println("This A is part of inner switch");
                break;
            case 'B': // ...
        } // конец внутреннего оператора switch
        break;
    case 'B': // ...
}

```

Пример для опробования 3.1.

Начало построения справочной системы Java

Исходный файл: `Help.java`

В этом проекте предстоит создать простую справочную систему, предоставляющую сведения о синтаксисе

управляющих операторов Java. Программа, реализующая эту справочную систему, отображает меню с названиями операторов и ожидает выбора одного из них. Как только пользователь выберет один из пунктов меню, на экран будут выведены сведения о синтаксисе соответствующего оператора. В первой версии данной программы предоставляются сведения только об операторах `if` и `switch`. А в последующих проектах будут добавлены справочные данные об остальных управляющих операторах.

Последовательность действий

1. Создайте новый файл `Help.java`.
2. В начале работы программы отображается следующее меню:

```
Help on:
1. if
2. switch
Choose one:
```

Для этой цели потребуется приведенная ниже последовательность операторов.

```
System.out.println("Help on:");
System.out.println(" 1. if");
System.out.println(" 2. switch");
System.out.print("Choose one: ");
```

3. Далее программа получает данные о выборе пользователя. С этой целью вызывается метод `System.in.read()`, как показано ниже.

```
choice = (char) System.in.read();
```

4. После этого в программе используется оператор `switch` для отображения сведений о синтаксисе выбранного оператора.

```
switch(choice) {
    case '1':
        System.out.println("The if:\n");
        System.out.println("if(condition) statement;");
        System.out.println("else statement;");
        break;
    case '2':
        System.out.println("The switch:\n");
        System.out.println("switch(expression) {");
        System.out.println(" case constant:");
        System.out.println(" statement sequence");
        System.out.println(" break;");
        System.out.println(" // ...");
        System.out.println("}");
        break;
    default:
        System.out.print("Selection not found.");
}
```

Обратите внимание на то, как в ветви `default` перехватываются сведения о неправильно сделанном выборе. Так, если пользователь введет значение `3`, оно не совпадет ни с одной из констант в ветвях `case` оператора `switch`, и тогда управление будет передано коду в ветви `default`.

5. Ниже приведен весь исходный код программы из файла `Help.java`.

```

/*
  Пример для опробования 3.1. Простая справочная система.
*/
class Help {
    public static void main(String args[])
        throws java.io.IOException {
        char choice;

        System.out.println("Help on:");
        System.out.println(" 1. if");
        System.out.println(" 2. switch");
        System.out.print("Choose one: ");
        choice = (char) System.in.read();

        System.out.println("\n");

        switch(choice) {
            case '1':
                System.out.println("The if:\n");
                System.out.println("if(condition) statement;");
                System.out.println("else statement;");
                break;
            case '2':
                System.out.println("The switch:\n");
                System.out.println("switch(expression) {");
                System.out.println(" case constant:");
                System.out.println(" statement sequence");
                System.out.println(" break;");
                System.out.println(" // ...");
                System.out.println("}");
                break;
            default:
                System.out.print("Selection not found.");
        }
    }
}

```

6. Выполнение этой программы дает следующий результат:

```

Help on:
1. if
2. switch
Choose one: 1

The if:

if(condition) statement;
else statement;

```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. В каких случаях вместо оператора `switch` следует использовать многоступенчатую конструкцию `if-else-if`?

ОТВЕТ. Вообще говоря, многоступенчатая конструкция `if-else-if` уместна в тех случаях, когда проверка условий не сводится к выяснению совпадения или несовпадения выражения с одиночным значением. Рассмотрим для примера следующую последовательность условных операторов:

```
if(x < 10) // ...  
else if(y != 0) // ...  
else if(!done) // ...
```

Данную последовательность нельзя заменить оператором `switch`, поскольку в трех ее условиях используются разные переменные и разные виды сравнения. Многоступенчатую конструкцию `if-else-if` целесообразно также применять для проверки значений с плавающей точкой и других объектов, типы которых отличаются от типов, предусмотренных для ветвей оператора `switch`.

Цикл `for`

Цикл `for` уже был представлен в главе 1, а здесь он рассматривается более подробно. Вас должны приятно удивить эффективность и гибкость этого цикла. Прежде всего обратимся к самым основным и традиционным формам цикла `for`.

Ниже приведена общая форма цикла `for` для повторного выполнения единственного оператора.

```
for(инициализация; условие; итерация) оператор;
```

А вот как выглядит его форма для повторного выполнения кодового блока.

```
for(инициализация; условие; итерация)  
{  
    последовательность операторов;  
}
```

где *инициализация*, как правило, представлена оператором присваивания, задающим первоначальное значение переменной, которая выполняет роль счетчика и управляет циклом; *условие* — это логическое выражение, определяющее необходимость повторения цикла; а *итерация* — выражение, определяющее величину, на которую должно изменяться значение переменной, управляющей циклом, на каждом шаге цикла. Обратите внимание на то, что эти три основные части оператора цикла `for` должны быть разделены точкой с запятой. Выполнение цикла `for` будет продолжаться до тех пор, пока проверка условия дает истинный результат. Как только эта проверка даст ложный результат, цикл завершится, а выполнение программы будет продолжено с оператора, следующего после цикла `for`.

Ниже приведен пример программы, где цикл `for` служит для вывода на экран значений квадратного корня чисел в пределах от 1 до 99. В данной программе отображается также ошибка округления, допущенная при вычислении квадратного корня.

```
// Вывод квадратных корней чисел от 1 до 99 вместе с ошибкой округления.  
class SqrRoot {  
    public static void main(String args[]) {
```

```

double num, sroot, rerr;

for(num = 1.0; num < 100.0; num++) {
    sroot = Math.sqrt(num);
    System.out.println("Square root of " + num +
        " is " + sroot);

    // вычислить ошибку округления
    rerr = num - (sroot * sroot);
    System.out.println("Rounding error is " + rerr);
    System.out.println();
}
}
}

```

Обратите внимание на то, что ошибка округления вычисляется путем возведения в квадрат квадратного корня числа. Полученное значение отнимается от исходного числа.

Переменная цикла может как увеличиваться, так и уменьшаться, а величина приращения может выбираться произвольно. Например, в приведенном ниже фрагменте кода выводятся числа в пределах от 100 до -95, и на каждом шаге переменная цикла уменьшается на 5.

```

// Цикл for, выполняющийся с отрицательным приращением переменной.
class DecrFor {
    public static void main(String args[]) {
        int x;

        // На каждом шаге цикла управляющая им переменная уменьшается на 5.
        for(x = 100; x > -100; x -= 5)
            System.out.println(x);
    }
}

```

В отношении циклов `for` следует особо подчеркнуть, что условное выражение всегда проверяется в самом начале цикла. Это означает, что код в цикле может вообще не выполняться, если проверяемое условие с самого начала оказывается ложным. Рассмотрим следующий пример:

```

for(count=10; count < 5; count++)
    x += count; // этот оператор не будет выполнен

```

Этот цикл вообще не будет выполняться, поскольку первоначальное значение переменной `count`, которая им управляет, сразу же оказывается больше 5. А это означает, что условное выражение `count < 5` оказывается ложным с самого начала, т.е. еще до выполнения первого шага цикла.

Некоторые разновидности цикла `for`

Цикл `for` относится к наиболее универсальным операторам языка Java, поскольку он допускает самые разные варианты применения. Например, для управления циклом можно использовать несколько переменных. Рассмотрим следующий пример программы:

```

// Применение запятых в операторе цикла for.
class Comma {

```

```
public static void main(String args[]) {
    int i, j;

    // Для управления этим циклом используются две переменные.
    for(i=0, j=10; i < j; i++, j--)
        System.out.println("i and j: " + i + " " + j);
}
```

Выполнение этой программы дает следующий результат:

```
i and j: 0 10
i and j: 1 9
i and j: 2 8
i and j: 3 7
i and j: 4 6
```

В данном примере запятыми разделяются два оператора инициализации и еще два итерационных выражения. Когда цикл начинается, инициализируются обе переменные, *i* и *j*. Всякий раз, когда цикл повторяется, переменная *i* инкрементируется, а переменная *j* декрементируется. Применение нескольких переменных управления циклом нередко оказывается удобным и упрощает некоторые алгоритмы. Теоретически в цикле `for` может быть указано любое количество операторов инициализации и итерации, но на практике такой цикл получается слишком громоздким, если применяется больше двух подобных операторов.

Условным выражением, управляющим циклом `for`, может быть любое действительное выражение, дающее результат типа `bool`. В него не обязательно должна входить переменная управления циклом. В следующем примере программы цикл будет выполняться до тех пор, пока пользователь не введет с клавиатуры букву *S*.

```
// Выполнение цикла до тех пор, пока с клавиатуры
// не будет введена буква S.
class ForTest {
    public static void main(String args[])
        throws java.io.IOException {

        int i;

        System.out.println("Press S to stop.");
        for(i = 0; (char) System.in.read() != 'S'; i++)
            System.out.println("Pass #" + i);
    }
}
```

Пропуск отдельных частей в определении цикла `for`

Ряд интересных разновидностей цикла `for` получается в том случае, если оставить пустыми отдельные части в определении цикла. В Java допускается оставлять пустыми любые или же все части инициализации, условия и итерации в операторе цикла `for`. В качестве примера рассмотрим следующую программу:

```
// Пропуск отдельных частей в определении цикла for.
class Empty {
```



```

public static void main(String args[]) {
    int i;

    // В определении этого цикла отсутствует итерационное выражение.
    for(i = 0; i < 10; ) {
        System.out.println("Pass #" + i);
        i++; // инкрементировать переменную управления циклом
    }
}

```

В данном примере итерационное выражение в определении цикла `for` оказывается пустым, т.е. вообще отсутствует. Вместо этого переменная `i`, управляющая циклом, инкрементируется в теле самого цикла. Это означает, что всякий раз, когда цикл повторяется, значение переменной `i` проверяется на равенство числу **10**, но никаких других действий при этом не происходит. А поскольку переменная `i` инкрементируется в теле цикла, то сам цикл выполняется обычным образом, выводя приведенный ниже результат.

```

Pass #0
Pass #1
Pass #2
Pass #3
Pass #4
Pass #5
Pass #6
Pass #7
Pass #8
Pass #9

```

В следующем примере программы из определения цикла `for` исключена инициализирующая часть.

```

// Пропуск дополнительных частей в определении цикла for.
class Empty2 {
    public static void main(String args[]) {
        int i;

        // Из определения этого цикла исключено не только
        // итерационное, но и инициализирующее выражение.
        i = 0;
        for(; i < 10; ) {
            System.out.println("Pass #" + i);
            i++; // инкрементировать переменную управления циклом
        }
    }
}

```

В данном примере переменная `i` инициализируется перед началом цикла, а не в самом цикле `for`. Как правило, переменная управления циклом инициализируется в цикле `for`. Выведение инициализирующей части за пределы цикла обычно делается лишь в том случае, если первоначальное значение управляющей им переменной получается в результате сложного процесса, который нецелесообразно вводить в само определение цикла `for`.

Бесконечный цикл

Если оставить пустым выражение условия в определении цикла `for`, то получится *бесконечный цикл*, т.е. такой цикл, который никогда не завершается. В качестве примера в следующем фрагменте кода показано, каким образом в Java обычно создается бесконечный цикл:

```
for(;;) // цикл, намеренно сделанный бесконечным
{
    //...
}
```

Этот цикл будет выполняться бесконечно. Несмотря на то что бесконечные циклы требуются для решения некоторых задач программирования, например при разработке командных процессоров операционных систем, большинство так называемых “бесконечных” циклов на самом деле представляют собой циклы со специальными требованиями к завершению. (Подробнее об этом речь пойдет ближе к концу главы, но, как правило, выход из бесконечного цикла осуществляется с помощью оператора `break`.)

Циклы без тела

В Java допускается оставлять пустым тело цикла `for` или любого другого цикла, поскольку *пустой оператор* с точки зрения синтаксиса этого языка считается действительным. Циклы без тела нередко оказываются полезными. Например, в следующей программе цикл без тела служит для получения суммы чисел от 1 до 5:

```
// Тело цикла for может быть пустым.
class Empty3 {
    public static void main(String args[]) {
        int i;

        int sum = 0;
        // Несмотря на отсутствие тела, в этом цикле
        // производится суммирование чисел от 1 до 5!
        for(i = 1; i <= 5; sum += i++) ;
        System.out.println("Sum is " + sum);
    }
}
```

Выполнение этой программы дает следующий результат:

```
Sum is 15
```

Обратите внимание на то, что процесс суммирования чисел выполняется полностью в операторе цикла `for`, и для этого тело цикла не требуется. В этом цикле особое внимание обращает на себя итерационное выражение.

```
sum += i++
```

Подобные операторы не должны вас смущать. Они часто встречаются в программах, профессионально написанных на Java, и становятся вполне понятными, если разобрать их по частям. Дословно приведенный выше оператор означает следующее: сложить со значением переменной `sum` результат суммирования значений переменных `sum` и `i`, а затем инкрементировать значение переменной `i`. Следовательно, данный оператор равнозначен следующей последовательности операторов:

```
sum = sum + i;
i++;
```

Объявление управляющих переменных в цикле `for`

Нередко переменная, управляющая циклом `for`, требуется только для выполнения самого цикла и нигде больше не используется. В таком случае управляющую переменную можно объявить в инициализирующей части оператора цикла `for`. Например, в приведенной ниже программе вычисляется сумма и факториал чисел от 1 до 5, а переменная `i`, управляющая циклом `for`, объявляется в этом цикле.

```
// Объявление переменной управления циклом в самом цикле for.
class ForVar {
    public static void main(String args[]) {
        int sum = 0;
        int fact = 1;

        // Вычисление факториала чисел от 1 до 5.
        // Переменная управления объявляется в этом цикле for.
        for(int i = 1; i <= 5; i++) {
            sum += i; // Она доступна во всем цикле.
            fact *= i;
        }

        // Но недоступна за пределами цикла.

        System.out.println("Sum is " + sum);
        System.out.println("Factorial is " + fact);
    }
}
```

Объявляя переменную в цикле `for`, не следует забывать о том, что область действия этой переменной ограничивается пределами оператора цикла `for`. Это означает, что за пределами цикла действие данной переменной прекращается. Так, в приведенном выше примере переменная `i` оказывается недоступной за пределами цикла `for`. Для того чтобы использовать переменную управления циклом в каком-нибудь другом месте программы, ее нельзя объявлять в цикле `for`.

Прежде чем переходить к чтению последующих разделов этой главы, поэкспериментируйте с собственными разновидностями цикла `for`. В ходе эксперимента вы непременно обнаружите замечательные свойства этого цикла.

Расширенный цикл `for`

С недавних пор в распоряжение программирующих на Java предоставлен так называемый *расширенный* цикл `for`, обеспечивающий специальные средства для перебора объектов из коллекции, например из массива. Расширенный цикл `for` будет представлен в главе 5 при рассмотрении массивов.

Цикл **while**

Еще одной разновидностью циклов в Java является **while**. Ниже приведена общая форма этого цикла.

```
while (условие) оператор;
```

где *оператор* — это единственный оператор или блок операторов, а *условие* означает конкретное условие управления циклом и может быть любым логическим выражением. В этом цикле *оператор* выполняется до тех пор, пока *условие* истинно. Как только *условие* становится ложным, управление программой передается строке кода, следующей непосредственно после цикла.

Ниже приведен простой пример использования цикла **while** для вывода на экран букв английского алфавита.

// Демонстрация цикла **while**.

```
class WhileDemo {
    public static void main(String args[]) {
        char ch;

        // вывести буквы английского алфавита, используя цикл while
        ch = 'a';
        while(ch <= 'z') {
            System.out.print(ch);
            ch++;
        }
    }
}
```

В данном примере переменная *ch* инициализируется кодом буквы **a**. На каждом шаге цикла на экран сначала выводится значение переменной *ch*, а затем это значение увеличивается на единицу. Процесс продолжается до тех пор, пока значение переменной *ch* не станет больше кода буквы **z**.

Как и в цикле **for**, в цикле **while** проверяется условное выражение, указываемое в самом начале цикла. Это означает, что код в теле цикла может вообще не выполняться, а кроме того, избавляет от необходимости производить отдельную проверку перед самим циклом. Данное свойство цикла **while** демонстрируется в следующем примере программы, где вычисляются целые степени числа 2 от 0 до 9.

// Вычисление целых степеней числа 2.

```
class Power {
    public static void main(String args[]) {
        int e;
        int result;

        for(int i=0; i < 10; i++) {
            result = 1;
            e = i;
            while(e > 0) {
                result *= 2;
                e--;
            }

            System.out.println("2 to the " + i +
                               " power is " + result);
        }
    }
}
```

```

    }
}
}

```

Ниже приведен результат выполнения данной программы.

```

2 to the 0 power is 1
2 to the 1 power is 2
2 to the 2 power is 4
2 to the 3 power is 8
2 to the 4 power is 16
2 to the 5 power is 32
2 to the 6 power is 64
2 to the 7 power is 128
2 to the 8 power is 256
2 to the 9 power is 512

```

Обратите внимание на то, что цикл `while` выполняется только в том случае, если значение переменной `e` больше нуля. А когда оно равно нулю, как это имеет место на первом шаге цикла `for`, цикл `while` пропускается.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. В языке Java циклы обладают большой гибкостью. Как же в таком случае выбрать цикл, наиболее подходящий для решения конкретной задачи?

ОТВЕТ. Если число итераций известно заранее, то лучше выбрать цикл `for`. Цикл `do-while` подходит в тех случаях, если требуется, чтобы была выполнена как минимум одна итерация. А цикл `while` оказывается наиболее удобным тогда, когда число повторений цикла заранее неизвестно.

Цикл `do-while`

Третьей и последней разновидностью циклов в Java является `do-while`. В отличие от циклов `for` и `while`, в которых условие проверялось в самом начале, в цикле `do-while` условие выполнения проверяется в самом конце. Это означает, что цикл `do-while` всегда выполняется хотя бы один раз. Ниже приведена общая форма цикла `do-while`.

```

do {
    операторы;
} while (условие);

```

При наличии лишь одного оператора фигурные скобки в данной форме записи необязательны. Тем не менее они зачастую используются для того, чтобы сделать конструкцию `do-while` более удобочитаемой и не путать ее с конструкцией цикла `while`. Цикл `do-while` выполняется до тех пор, пока условное выражение истинно.

```

// Демонстрация цикла do-while.
class DWDemo {
    public static void main(String args[])
        throws java.io.IOException {

        char ch;

        do {

```

```

        System.out.print("Press a key followed by ENTER: ");
        ch = (char) System.in.read(); // ввести символ с клавиатуры
    } while(ch != 'q');
}
}

```

Используя цикл `do-while`, мы можем усовершенствовать игру в угадывание букв, созданную в начале этой главы. На этот раз выполнение цикла будет продолжаться до тех пор, пока пользователь не угадает букву.

// Игра в угадывание букв, четвертая версия.

```

class Guess4 {
    public static void main(String args[])
        throws java.io.IOException {

        char ch, ignore, answer = 'S';

        do {
            System.out.println("I'm thinking of a letter between A and Z.");
            System.out.print("Can you guess it: ");

            // ввести символ с клавиатуры
            ch = (char) System.in.read();
            // отвергнуть все остальные символы во входном буфере
            do {
                ignore = (char) System.in.read();
            } while(ignore != '\n');

            if(ch == answer) System.out.println("*** Right ***");
            else {
                System.out.print("...Sorry, you're ");
                if(ch < answer) System.out.println("too low");
                else System.out.println("too high");
                System.out.println("Try again!\n");
            }
        } while(answer != ch);
    }
}

```

Ниже приведен один из возможных вариантов выполнения данной программы в интерактивном режиме.

```

I'm thinking of a letter between A and Z.
Can you guess it: A
...Sorry, you're too low
Try again!

```

```

I'm thinking of a letter between A and Z.
Can you guess it: Z
...Sorry, you're too high
Try again!

```

```

I'm thinking of a letter between A and Z.
Can you guess it: S
** Right **

```

Обратите внимание на еще одну интересную особенность данной программы: в ней применяются два цикла `do-while`. Первый цикл выполняется до тех пор, пока пользователь не введет правильную букву. А второй цикл приведен ниже и требует дополнительных пояснений.

```
// отвергнуть все остальные символы во входном буфере
do {
    ignore = (char) System.in.read();
} while(ignore != '\n');
```

Как пояснялось ранее, консольный ввод буферизуется построчно. Это означает, что для передачи символов, вводимых с клавиатуры, приходится нажимать клавишу `<Enter>`, что приводит к формированию последовательности символов перевода строки и возврата каретки. Эти символы сохраняются во входном буфере вместе с введенными с клавиатуры. Кроме того, если ввести с клавиатуры несколько символов подряд, не нажав клавишу `<Enter>`, они так и останутся во входном буфере.

В рассматриваемом здесь цикле эти символы отвергаются до тех пор, пока не будет достигнут конец строки. Если не сделать этого, лишние символы будут передаваться программе как отгадываемые, что не соответствует правилам игры в отгадывание. (Для того чтобы убедиться в этом, попробуйте закомментировать внутренний цикл `do-while` в исходном коде программы.) После представления ряда других языковых средств Java в главе 10 рассматриваются некоторые более совершенные способы обработки консольного ввода. Но применение метода `read()` в данной программе дает элементарное представление о принципе действия системы ввода-вывода в Java. А кроме того, в данной программе демонстрируется еще один пример применения циклов в практике программирования на Java.

Пример для опробования 3.2.

Расширение справочной системы Java

Исходный файл: `Help2.java`

В этом проекте предстоит расширить справочную систему Java, созданную в примере для опробования 3.1. В эту версию программы будут добавлены сведения о синтаксисе циклов `for`, `while` и `do-while`. Кроме того, будет реализована проверка действий пользователя, работающего с меню. Цикл будет повторяться до тех пор, пока пользователь не введет допустимое значение.

Последовательность действий

1. Скопируйте файл `Help.java` в новый файл `Help2.java`.
2. Измените часть программы, ответственную за отображение вариантов, предлагаемых пользователю на выбор. Реализуйте ее с помощью циклов так, как показано ниже.

```
public static void main(String args[])
    throws java.io.IOException {
    char choice, ignore;

    do {
        System.out.println("Help on:");
        System.out.println(" 1. if");
        System.out.println(" 2. switch");
        System.out.println(" 3. for");
        System.out.println(" 4. while");
```

```

System.out.println(" 5. do-while\n");
System.out.print("Choose one: ");

choice = (char) System.in.read();

do {
    ignore = (char) System.in.read();
} while(ignore != '\n');
} while( choice < '1' | choice > '5');

```

3. Обратите внимание на вложенные циклы do-while, используемые с целью избавиться от нежелательных символов, оставшихся во входном буфере. После внесения приведенных выше изменений программа будет отображать меню в цикле до тех пор, пока пользователь не введет числовое значение в пределах от 1 до 5.
4. Дополните оператор switch выводом на экран сведений о синтаксисе циклов for, while и do-while, как показано ниже.

```

switch(choice) {
    case '1':
        System.out.println("The if:\n");
        System.out.println("if(condition) statement;");
        System.out.println("else statement;");
        break;
    case '2':
        System.out.println("The switch:\n");
        System.out.println("switch(expression) {");
        System.out.println(" case constant;");
        System.out.println(" statement sequence");
        System.out.println(" break;");
        System.out.println(" // ...");
        System.out.println("}");
        break;
    case '3':
        System.out.println("The for:\n");
        System.out.print("for(init; condition; iteration)");
        System.out.println(" statement;");
        break;
    case '4':
        System.out.println("The while:\n");
        System.out.println("while(condition) statement;");
        break;
    case '5':
        System.out.println("The do-while:\n");
        System.out.println("do {");
        System.out.println(" statement;");
        System.out.println("} while (condition);");
        break;
}

```

5. Обратите внимание на то, что в данном варианте оператора switch отсутствует ветвь default. А поскольку цикл отображения меню будет выполняться до тех пор, пока пользователь не введет допустимое значение, необходимость в обработке неправильных значений отпадает.
6. Ниже приведен весь исходный код программы из файла Help2.java.

/*

Пример для опробования 3.2.

Расширенная справочная система, в которой для обработки результатов выбора из меню используется цикл **do-while**.

*/

```

class Help2 {
    public static void main(String args[])
        throws java.io.IOException {
        char choice, ignore;

        do {
            System.out.println("Help on:");
            System.out.println(" 1. if");
            System.out.println(" 2. switch");
            System.out.println(" 3. for");
            System.out.println(" 4. while");
            System.out.println(" 5. do-while\n");
            System.out.print("Choose one: ");

            choice = (char) System.in.read();

            do {
                ignore = (char) System.in.read();
            } while(ignore != '\n');
        } while( choice < '1' | choice > '5');

        System.out.println("\n");

        switch(choice) {
            case '1':
                System.out.println("The if:\n");
                System.out.println("if(condition) statement;");
                System.out.println("else statement;");
                break;
            case '2':
                System.out.println("The switch:\n");
                System.out.println("switch(expression) {");
                System.out.println(" case constant:");
                System.out.println(" statement sequence");
                System.out.println(" break;");
                System.out.println(" // ...");
                System.out.println("}");
                break;
            case '3':
                System.out.println("The for:\n");
                System.out.print("for(init; condition; iteration)");
                System.out.println(" statement;");
                break;
            case '4':
                System.out.println("The while:\n");
                System.out.println("while(condition) statement;");
                break;
            case '5':

```

```

        System.out.println("The do-while:\n");
        System.out.println("do {");
        System.out.println(" statement;");
        System.out.println("} while (condition);");
        break;
    }
}

```

Применение оператора **break** для выхода из цикла

С помощью оператора **break** можно специально организовать немедленный выход из цикла в обход любого кода, оставшегося в теле цикла, а также минуя проверку условия цикла. Когда в теле цикла встречается оператор **break**, цикл завершается, а выполнение программы возобновляется с оператора, следующего после этого цикла. Рассмотрим следующий краткий пример программы:

```

// Применение оператора break для выхода из цикла.
class BreakDemo {
    public static void main(String args[]) {
        int num;

        num = 100;

        // выполнять цикл до тех пор, пока квадрат значения
        // переменной i меньше значения переменной num
        for(int i=0; i < num; i++) {
            if(i*i >= num) break; // прекратить цикл, если i*i >= 100
            System.out.print(i + " ");
        }
        System.out.println("Loop complete.");
    }
}

```

Выполнение этой программы дает следующий результат:

```
0 1 2 3 4 5 6 7 8 9 Loop complete.
```

Как видите, цикл **for** организован для выполнения в пределах значений переменной **num** от 0 до 100. Но, несмотря на это, оператор **break** прерывает этот цикл раньше, когда квадрат значения переменной **i** становится больше значения переменной **num**.

Оператор **break** можно применять в любых циклах, предусмотренных в Java, включая и те, что намеренно организованы бесконечными. В качестве примера ниже приведен простой пример программы, в которой вводимые данные читаются до тех пор, пока пользователь не введет с клавиатуры букву **q**.

```

// Чтение вводимых данных до тех пор, пока не будет получена буква q.
class Break2 {
    public static void main(String args[])
        throws java.io.IOException {
        char ch;

```

```
// "Бесконечный" цикл, завершаемый с помощью оператора break.
for( ; ; ) {
    ch = (char) System.in.read(); // ввести символ с клавиатуры
    if(ch == 'q') break;
}
System.out.println("You pressed q!");
}
```

Если оператор `break` применяется в целом ряде вложенных циклов, то он прерывает выполнение только самого внутреннего цикла. В качестве примера рассмотрим следующую программу:

```
// Применение оператора break во вложенных циклах.
class Break3 {
    public static void main(String args[]) {

        for(int i=0; i<3; i++) {
            System.out.println("Outer loop count: " + i);
            System.out.print(" Inner loop count: ");

            int t = 0;
            while(t < 100) {
                if(t == 10) break; // прервать цикл, если t = 10
                System.out.print(t + " ");
                t++;
            }
            System.out.println();
        }
        System.out.println("Loops complete.");
    }
}
```

Выполнение этой программы дает следующий результат:

```
Outer loop count: 0
    Inner loop count: 0 1 2 3 4 5 6 7 8 9
Outer loop count: 1
    Inner loop count: 0 1 2 3 4 5 6 7 8 9
Outer loop count: 2
    Inner loop count: 0 1 2 3 4 5 6 7 8 9
Loops complete.
```

Как видите, оператор `break` из внутреннего цикла вызывает прерывание только этого цикла. А на выполнение внешнего цикла он не оказывает никакого влияния.

В отношении оператора `break` необходимо также иметь в виду следующее. Во-первых, в теле цикла может присутствовать несколько операторов `break`, но применять их следует очень аккуратно, поскольку чрезмерное количество операторов `break` обычно приводит к нарушению нормальной структуры кода. И во-вторых, оператор `break`, выполняющий выход из оператора `switch`, оказывает воздействие только на этот оператор, но не на объемлющие его циклы.

Оператор `break` в качестве оператора `goto`

Помимо оператора `switch` и циклов, оператор `break` может быть использован как “цивилизованный” вариант оператора `goto`. В языке Java оператор `goto` отсутствует, поскольку он дает возможность произвольного перехода в любую точку программы, что способствует созданию плохо структурированного кода. Программы, в которых часто используется оператор `goto`, обычно сложны для восприятия и сопровождения. Но в некоторых случаях оператор `goto` может оказаться полезным. Его удобно, например, использовать для экстренного выхода из многократно вложенных циклов.

Для решения подобных задач в Java определена расширенная форма оператора `break`. Используя этот вариант оператора `break`, можно, например, выйти за пределы одного или нескольких кодовых блоков. Эти блоки должны быть частью циклов или оператора `switch`. Кроме того, нужно явно указать точку, с которой должно быть продолжено выполнение программы. Для этой цели в расширенной форме оператора `break` предусмотрена метка. Как будет показано далее, оператор `break` позволяет воспользоваться всеми преимуществами оператора `goto` и в то же время избежать сложностей, связанных с его применением.

Ниже приведена общая форма оператора `break` с меткой.

```
break метка;
```

Как правило, *метка* — это имя, обозначающее кодовый блок. При выполнении расширенного оператора `break` управление передается за пределы именованного кодового блока. Оператор `break` с меткой может содержаться непосредственно в именованном кодовом блоке или в одном из блоков, входящих в состав именованного блока. Следовательно, рассматриваемый здесь вариант оператора `break` можно использовать для выхода из ряда вложенных блоков. Но это языковое средство не позволяет передать управление в кодовый блок, не содержащий оператор `break`.

Для того чтобы присвоить имя кодовому блоку, нужно поставить перед ним метку. Именовывать можно как независимый блок, так и оператор, адресатом которого является кодовый блок. Роль *метки* может выполнять любой допустимый в Java идентификатор с двоеточием. Пометив кодовый блок, метку можно использовать в качестве адресата оператора `break`. Благодаря этому выполнение программы возобновляется с *конца* именованного блока. Например, в приведенном ниже фрагменте кода используются три вложенных блока.

```
// Применение оператора break с меткой.
class Break4 {
    public static void main(String args[]) {
        int i;

        for(i=1; i<4; i++) {
            one: {
                two: {
                    three: {
                        System.out.println("\ni is " + i);
                        // Переходы по меткам.
                        if(i==1) break one;
                        if(i==2) break two;
                        if(i==3) break three;

                        // Эта строка кода никогда не будет достигнута.
```

```

        System.out.println("won't print");
    }
    System.out.println("After block three.");
}
    System.out.println("After block two.");
}
    System.out.println("After block one.");
}
    System.out.println("After for.");
}
}

```

Выполнение этого фрагмента кода дает следующий результат:

```

i is 1
After block one.

```

```

i is 2
After block two.
After block one.

```

```

i is 3
After block three.
After block two.

```

```

After block one.
After for.

```

Рассмотрим подробнее приведенный выше фрагмент кода, чтобы лучше понять, каким образом получается именно такой результат его выполнения. Когда значение переменной *i* равно 1, условие первого оператора *if* становится истинным и управление передается в конец блока с меткой *one*. В результате выводится сообщение "After block one." (После первого блока). Если значение переменной *i* равно 2, то успешно завершается проверка условия во втором операторе *if* и выполнение программы продолжается с конца блока с меткой *two*. В результате выводятся по порядку сообщения "After block two." (После второго блока) и "After block one.". Когда же переменная *i* принимает значение 3, истинным становится условие в третьем операторе *if* и управление передается в конец блока с меткой *three*. В этом случае выводятся все три сообщения: два упомянутых выше, а также сообщение "After block three." (После третьего блока), а затем еще и сообщение "After for." (После цикла *for*).

Обратимся к еще одному примеру. На этот раз оператор *break* будет использован для выхода за пределы нескольких вложенных циклов. Когда во внутреннем цикле выполняется оператор *break*, управление передается в конец блока внешнего цикла. Этот блок помечен меткой *done*. В результате происходит выход из всех трех циклов.

```

// Еще один пример применения оператора break с меткой.
class Break5 {
    public static void main(String args[]) {

```

```

        done:
        for(int i=0; i<10; i++) {
            for(int j=0; j<10; j++) {
                for(int k=0; k<10; k++) {
                    System.out.println(k + " ");

```

```

        if(k == 5) break done; // переход по метке done
    }
    System.out.println("After k loop"); // не выполнится
}
System.out.println("After j loop"); // не выполнится
}
System.out.println("After i loop");
}
}

```

Ниже приведен результат выполнения данного фрагмента кода.

```

0
1
2
3
4
5
After i loop

```

Расположение метки имеет большое значение, особенно когда речь идет о циклах. Рассмотрим в качестве примера следующий фрагмент кода:

```

// Расположение метки имеет большое значение.
class Break6 {
    public static void main(String args[]) {
        int x=0, y=0;

        // Здесь метка располагается перед оператором.
stop1: for(x=0; x < 5; x++) {
            for(y = 0; y < 5; y++) {
                if(y == 2) break stop1;
                System.out.println("x and y: " + x + " " + y);
            }
        }

        System.out.println();

        // А здесь метка располагается непосредственно перед
        // открывающей фигурной скобкой.
        for(x=0; x < 5; x++)
stop2: {
            for(y = 0; y < 5; y++) {
                if(y == 2) break stop2;
                System.out.println("x and y: " + x + " " + y);
            }
        }
    }
}

```

Ниже приведен результат выполнения данного фрагмента кода.

```

x and y: 0 0
x and y: 0 1

x and y: 0 0
x and y: 0 1

```

```

x and y: 1 0
x and y: 1 1
x and y: 2 0
x and y: 2 1
x and y: 3 0
x and y: 3 1
x and y: 4 0
x and y: 4 1

```

Ряды вложенных циклов в данном фрагменте кода идентичны за исключением того, что в первом ряду метка находится перед внешним циклом `for`. В таком случае при выполнении оператора `break` управление передается в конец всего блока цикла `for`, а оставшиеся итерации внешнего цикла пропускаются. Во втором ряду метка находится перед открывающей фигурной скобкой кодового блока, определяющего тело внешнего цикла. Поэтому при выполнении оператора `break stop2` управление передается в конец тела внешнего цикла `for`, и далее выполняется очередной его шаг.

Не следует, однако, забывать, что в операторе `break` нельзя использовать метку, не определенную в объемлющем его цикле. Например, приведенный ниже фрагмент кода некорректен и не подлежит компиляции.

```

// Этот фрагмент кода содержит ошибку.
class BreakErr {
    public static void main(String args[]) {

        one: for(int i=0; i<3; i++) {
            System.out.print("Pass " + i + ": ");
        }

        for(int j=0; j<100; j++) {
            if(j == 10) break one; // НЕВЕРНО!
            System.out.print(j + " ");
        }
    }
}

```

Кодовый блок, обозначенный меткой `one`, не содержит оператор `break`, и поэтому управление не может быть передано этому блоку.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как пояснялось выше, оператор `goto` нарушает структуру программы, и поэтому более подходящим вариантом является оператор `break` с меткой. Но не нарушает ли структуру программы и переход в конец внешнего цикла, т.е. в точку, далеко отстоящую от оператора `break`?

ОТВЕТ. Действительно, нарушает. Но если явный переход все-таки необходим, то передача управления в конец кодового блока сохраняет хоть какое-то подобие структуры, чего нельзя сказать об операторе `goto`.

Применение оператора `continue`

С помощью оператора `continue` можно организовать преждевременное завершение шага итерации цикла в обход обычной структуры управления циклом. Оператор

`continue` осуществляет принудительный переход к следующему шагу цикла, пропуская любой код, оставшийся невыполненным. Таким образом, оператор `continue` служит своего рода дополнением оператора `break`. В приведенном ниже примере программы оператор `continue` используется в качестве вспомогательного средства для вывода четных чисел в пределах от 0 до 100.

```
// Применение оператора continue.
class ContDemo {
    public static void main(String args[]) {
        int i;

        // вывести четные числа в пределах от 0 до 100
        for(i = 0; i<=100; i++) {
            if((i%2) != 0) continue; // завершить шаг итерации цикла
            System.out.println(i);
        }
    }
}
```

В данном примере выводятся только четные числа, поскольку при обнаружении нечетного числа шаг итерации цикла завершается преждевременно в обход вызова метода `println()`.

В циклах `while` и `do-while` оператор `continue` вызывает передачу управления непосредственно условному выражению, после чего продолжается процесс выполнения цикла. А в цикле `for` сначала вычисляется итерационное выражение, затем условное выражение, после чего цикл продолжается.

Как и в операторе `break`, в операторе `continue` может присутствовать метка, обозначающая тот объемлющий цикл, выполнение которого должно быть продолжено. Ниже приведен пример программы, демонстрирующий применение оператора `continue` с меткой.

```
// Применить оператор continue с меткой.
class ContToLabel {
    public static void main(String args[]) {

outerloop:
        for(int i=1; i < 10; i++) {
            System.out.print("\nOuter loop pass " + i +
                             ", Inner loop: ");
            for(int j = 1; j < 10; j++) {
                if(j == 5) continue outerloop; // продолжить внешний цикл
                System.out.print(j);
            }
        }
    }
}
```

Выполнение этой программы дает следующий результат:

```
Outer loop pass 1, Inner loop: 1234
Outer loop pass 2, Inner loop: 1234
Outer loop pass 3, Inner loop: 1234
Outer loop pass 4, Inner loop: 1234
Outer loop pass 5, Inner loop: 1234
```



```
Outer loop pass 6, Inner loop: 1234
Outer loop pass 7, Inner loop: 1234
Outer loop pass 8, Inner loop: 1234
Outer loop pass 9, Inner loop: 1234
```

Как следует из приведенного выше примера, при выполнении оператора `continue` управление передается внешнему циклу, и оставшиеся итерации внутреннего цикла пропускаются.

В реальных программах оператор `continue` применяется очень редко. И объясняется это, в частности, богатым набором в Java операторов цикла, удовлетворяющих большую часть потребностей в написании прикладных программ. Но в особых случаях, когда требуется преждевременное прекращение цикла, оператор `continue` позволяет сделать это, не нарушая структуру кода.

Пример для опробования 3.3.

Завершение построения справочной системы Java

Исходный файл: `Help3.java`

В этом проекте предстоит завершить построение справочной системы Java, начатое в предыдущих проектах. Данная версия будет дополнена сведениями о синтаксисе операторов `break` и `continue`, а также даст пользователю возможность запрашивать сведения о синтаксисе нескольких операторов. Эта цель достигается путем добавления внешнего цикла, который выполняется до тех пор, пока пользователь не введет с клавиатуры букву **q** вместо номера пункта меню.

Последовательность действий

1. Скопируйте файл `Help2.java` в новый файл `Help3.java`.
2. Поместите весь исходный код программы в бесконечный цикл `for`. Выход из этого цикла будет осуществляться с помощью оператора `break`, который получит управление тогда, когда пользователь введет с клавиатуры букву **q**. А поскольку этот цикл включает в себя весь код, то выход из него означает завершение программы.
3. Измените цикл отображения меню так, как показано ниже.

```
do {
    System.out.println("Help on:");
    System.out.println(" 1. if");
    System.out.println(" 2. switch");
    System.out.println(" 3. for");
    System.out.println(" 4. while");
    System.out.println(" 5. do-while");
    System.out.println(" 6. break");
    System.out.println(" 7. continue\n");
    System.out.print("Choose one (q to quit): ");

    choice = (char) System.in.read();
    do {
        ignore = (char) System.in.read();
    } while(ignore != '\n');
} while( choice < '1' | choice > '7' & choice != 'q');
```

Как видите, цикл теперь включает в себя операторы `break` и `continue`. Кроме того, буква **q** воспринимается в нем как допустимый вариант выбора.

4. Дополните оператор switch операторами break и continue, как показано ниже.

```

case '6':
    System.out.println("The break:\n");
    System.out.println("break; or break label;");
    break;
case '7':
    System.out.println("The continue:\n");
    System.out.println("continue; or continue label;");
    break;

```

5. Ниже приведен весь исходный код программы из файла Help3.java.

```

/*
    Пример для опробования 3.3.

    Завершенная справочная система по управляющим
    операторам Java, обрабатывающая многократные запросы.
*/
class Help3 {
    public static void main(String args[])
        throws java.io.IOException {
        char choice, ignore;

        for(;;) {
            do {
                System.out.println("Help on:");
                System.out.println(" 1. if");
                System.out.println(" 2. switch");
                System.out.println(" 3. for");
                System.out.println(" 4. while");
                System.out.println(" 5. do-while");
                System.out.println(" 6. break");
                System.out.println(" 7. continue\n");
                System.out.print("Choose one (q to quit): ");

                choice = (char) System.in.read();

                do {
                    ignore = (char) System.in.read();
                } while(ignore != '\n');
            } while( choice < '1' | choice > '7' & choice != 'q');

            if(choice == 'q') break;

            System.out.println("\n");

            switch(choice) {
                case '1':
                    System.out.println("The if:\n");
                    System.out.println("if(condition) statement;");
                    System.out.println("else statement;");
                    break;
                case '2':

```

```

        System.out.println("The switch:\n");
        System.out.println("switch(expression) {");
        System.out.println(" case constant:");
        System.out.println(" statement sequence");
        System.out.println(" break;");
        System.out.println(" // ...");
        System.out.println("}");
        break;
    case '3':
        System.out.println("The for:\n");
        System.out.println("for(init; condition; iteration)");
        System.out.println(" statement;");
        break;
    case '4':
        System.out.println("The while:\n");
        System.out.println("while(condition) statement;");
        break;
    case '5':
        System.out.println("The do-while:\n");
        System.out.println("do {");
        System.out.println(" statement;");
        System.out.println("} while (condition);");
        break;
    case '6':
        System.out.println("The break:\n");
        System.out.println("break; or break label;");
        break;
    case '7':
        System.out.println("The continue:\n");
        System.out.println("continue; or continue label;");
        break;
    }
    System.out.println();
}
}
}

```

6. Ниже приведен один из возможных вариантов выполнения данной программы в диалоговом режиме.

Help on:

1. if
2. switch
3. for
4. while
5. do-while
6. break
7. Continue

Choose one (q to quit): 1

The if:

```
if(condition) statement;
```

```
else statement;
```

```
Help on:
```

1. if
2. switch
3. for
4. while
5. do-while
6. break
7. Continue

```
Choose one (q to quit): 6
```

```
The break:
```

```
break; or break label;
```

```
Help on:
```

1. if
2. switch
3. for
4. while
5. do-while
6. break
7. Continue

```
Choose one (q to quit): q
```

Вложенные циклы

Как следует из предыдущих примеров программ, один цикл может быть вложен в другой. С помощью вложенных циклов решаются самые разные задачи. Поэтому, прежде чем завершить рассмотрение циклов в Java, уделим еще немного внимания вложенным циклам. Ниже приведен пример программы, содержащей вложенные циклы `for`. С помощью этих циклов для каждого числа от 2 до 100 определяется ряд множителей, на которые данное число делится без остатка.

```
/*
   Использовать вложенные циклы для выявления
   множителей чисел от 2 до 100.
*/
class FindFac {
    public static void main(String args[]) {

        for(int i=2; i <= 100; i++) {
            System.out.print("Factors of " + i + ": ");
            for(int j = 2; j < i; j++)
                if((i%j) == 0) System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

Ниже приведена часть результата выполнения данной программы.

```
Factors of 2:
Factors of 3:
Factors of 4: 2
Factors of 5:
Factors of 6: 2 3
Factors of 7:
Factors of 8: 2 4
Factors of 9: 3
Factors of 10: 2 5
Factors of 11:
Factors of 12: 2 3 4 6
Factors of 13:
Factors of 14: 2 7
Factors of 15: 3 5
Factors of 16: 2 4 8
Factors of 17:
Factors of 18: 2 3 6 9
Factors of 19:
Factors of 20: 2 4 5 10
```

В данной программе переменная `i` из внешнего цикла последовательно принимает значения до **2** до **100**. А во внутреннем цикле для каждого числа от **2** до текущего значения переменной `i` выполняется проверка, делится ли это значение без остатка. В качестве упражнения попробуйте сделать данную программу более эффективной. (Подсказка: число итераций во внутреннем цикле можно уменьшить.)

Упражнение для самопроверки по материалу главы 3

1. Напишите программу, которая вводила бы символы с клавиатуры до тех пор, пока не встретится точка. Предусмотрите в программе счетчик числа пробелов. Сведения о количестве пробелов должны выводиться в конце программы.
2. Какова общая форма многоступенчатой конструкции `if-else-if`?
3. Допустим, имеется следующий фрагмент кода:

```
if(x < 10)
    if(y > 100) {
        if(!done) x = z;
        else y = z;
    }
else System.out.println("error"); // что если?
```

4. С каким из операторов `if` связан последний оператор `else`?
5. Напишите цикл `for`, в котором перебирались бы значения от **1000** до **0** с шагом **-2**.
6. Корректен ли следующий фрагмент кода?

```
for(int i = 0; i < num; i++)
    sum += i;

count = i;
```

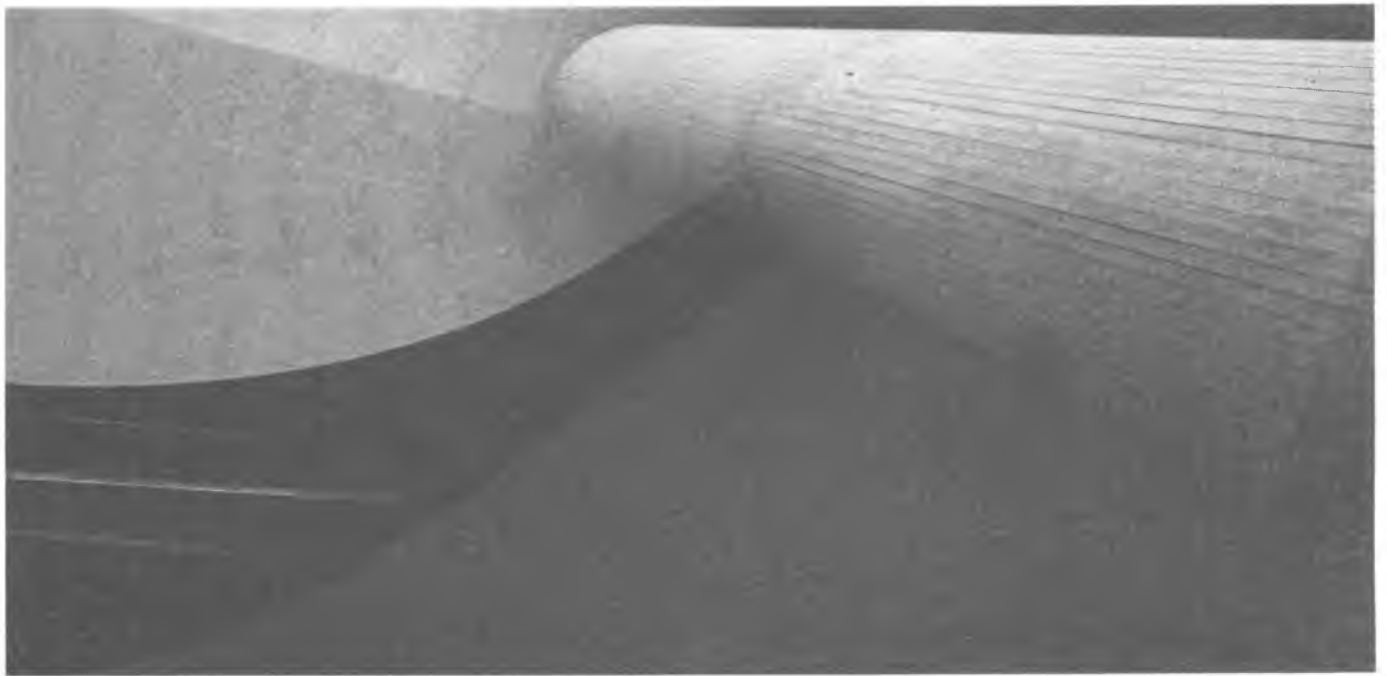
7. Какие действия выполняет оператор **break**? Опишите оба варианта этого оператора.
8. Какое сообщение будет выведено после выполнения оператора **break** в приведенном ниже фрагменте кода?

```
for(i = 0; i < 10; i++) {  
    while(running) {  
        if(x<y) break;  
        // ...  
    }  
    System.out.println("after while");  
}  
System.out.println("After for");
```

9. Что будет выведено на экран в результате выполнения следующего фрагмента кода?

```
for(int i = 0; i<10; i++) {  
    System.out.print(i + " ");  
    if((i%2) == 0) continue;  
    System.out.println();  
}
```

10. Итерационное выражение для цикла **for** не обязательно должно изменять переменную цикла на фиксированную величину. Эта переменная может принимать произвольные значения. Напишите программу, использующую цикл **for** для вывода чисел в геометрической прогрессии 1, 2, 4, 8, 16, 32 и т.д.
11. Код ASCII символов нижнего регистра отличается от кода соответствующих символов верхнего регистра на величину 32. Следовательно, для преобразования строчной буквы в прописную нужно уменьшить ее код на 32. Используйте это обстоятельство для написания программы, осуществляющей ввод символов с клавиатуры. При выводе результатов данная программа должна преобразовывать строчные буквы в прописные, а прописные — в строчные. Остальные символы не должны изменяться. Работа программы должна завершаться после того, как пользователь введет с клавиатуры точку. И наконец, сделайте так, чтобы программа отображала число символов, для которых был изменен регистр.
12. Что такое бесконечный цикл?
13. Должна ли метка, используемая вместе с оператором **break**, быть определена в кодовом блоке, содержащем этот оператор?



Глава 4

**Введение в классы,
объекты и методы**

Основные навыки и понятия

- Основные положения о классах
- Создание объектов
- Присваивание ссылок на объекты
- Создание методов, возврат значений и использование параметров
- Применение ключевого слова `return`
- Возврат значения из метода
- Добавление параметров в метод
- Применение конструкторов
- Создание параметризованных конструкторов
- Представление об операторе `new`
- Представление о “сборке мусора” и методах завершения
- Применение ключевого слова `this`

Прежде чем продолжить изучение Java, следует ознакомиться с классами. Классы, по существу, составляют самые основы языка Java, поскольку класс определяет характер объекта. Следовательно, классы служат прочным основанием для объектно-ориентированного программирования на Java. В классе определяются данные и код, который выполняет действия над этими данными. Код находится внутри методов. Эта глава посвящена классам, объектам и методам, т.е. самим основам Java. Имея представление о классах, объектах и методах, вы сможете писать более сложные программы и лучше уяснить те элементы языка Java, которые будут описаны в последующих главах.

Основные положения о классах

Все действия программ на Java производятся в пределах классов, поэтому мы вынуждены были пользоваться классами, начиная с первых же примеров программ в этой книге. Разумеется, мы ограничивались лишь самыми простыми классами и не пользовались большинством их возможностей. Как станет ясно в дальнейшем, классы — намного более эффективное языковое средство, чем можно было бы предположить, имея о них лишь самое ограниченное представление, почерпнутое из предыдущих глав.

Начнем рассмотрение классов с основных положений. *Класс* представляет собой шаблон, по которому определяется форма объекта. В нем указываются данные и код, который будет оперировать этими данными. В Java используется спецификация класса для построения *объектов*, которые являются *экземплярами* класса. Следовательно, класс, по существу, представляет собой ряд схематических описаний способа построения объекта. При этом очень важно подчеркнуть, что класс является логической абстракцией. Физическое представление класса появится в оперативной памяти лишь после того, как будет создан объект этого класса.

Следует также иметь в виду, что методы и переменные, составляющие класс, принято называть *членами* класса. А члены данных называются *переменными экземпляра*.

Общая форма определения класса

Определяя класс, вы объявляете его конкретную форму и поведение. Для этого указываются содержащиеся в нем переменные экземпляра, а также методы оперирования этими переменными. Если самые простые классы могут содержать только код или только данные, то большинство настоящих классов содержат и то и другое.

Класс создается с помощью ключевого слова `class`. Ниже приведена упрощенная общая форма определения класса.

```
class имя_класса {
    // Объявление переменных экземпляра.
    тип переменная1;
    тип переменная2;
    //...
    тип переменнаяN;

    // Объявление методов.
    тип метод1(параметры) {
        // тело метода
    }
    тип метод2(параметры) {
        // тело метода
    }

    //...
    тип методN(параметры) {
        // тело метода
    }
}
```

Несмотря на отсутствие соответствующего правила в синтаксисе Java, правильно сконструированный класс должен определять одну и только одну логическую сущность. Например, класс, в котором хранятся Ф.И.О. и номера телефонов, обычно не содержит сведения о фондовом рынке, среднем уровне осадков, циклах солнечных пятен или другую не относящуюся к делу информацию. Таким образом, в правильно сконструированном классе должна быть сгруппирована логически связанная информация. Если же в один и тот же класс помещается логически несвязанная информация, то структурированность кода быстро нарушается.

Классы, использовавшиеся в приведенных ранее примерах программ, содержали только один метод: `main()`. Но в представленной выше общей форме определения класса метод `main()` не указывается. Этот метод требуется указывать в классе лишь в том случае, если программа начинается с данного класса. Кроме того, в некоторых приложениях Java, в том числе в апплетах, метод `main()` вообще не требуется.

Определение класса

Для того чтобы проиллюстрировать особенности создания классов, построим класс, инкапсулирующий сведения о транспортных средствах, например о легковых автомобилях, фургонах и грузовиках. Назовем этот класс `Vehicle`. В нем будут храниться сле-

дующие сведения: количество пассажиров, емкость топливного бака и среднее потребление топлива (в милях на галлон).

Ниже приведен первый вариант класса `Vehicle`. В нем определены три переменные экземпляра: `passengers`, `fuelcap` и `mpg`. Обратите внимание на то, что в классе `Vehicle` пока еще отсутствуют методы. Они будут добавлены в последующих разделах, а до тех пор в этом классе содержатся только данные.

```
class Vehicle {
    int passengers; // количество пассажиров
    int fuelcap;    // емкость топливного бака
    int mpg;        // потребление топлива в милях на галлон
}
```

Объявление `class` указывает на создание нового типа данных. В данном случае этот тип называется `Vehicle`. Пользуясь этим именем, можно теперь создавать объекты типа `Vehicle`. Но не следует забывать, что объявление `class` — это всего лишь описание типа данных, а реальный объект при этом не создается. Следовательно, приведенный выше код не приводит к появлению объектов типа `Vehicle`.

Для того чтобы создать реальный объект `Vehicle`, потребуется оператор, аналогичный следующему:

```
Vehicle minivan = new Vehicle(); // создать объект minivan типа Vehicle
```

После выполнения этого оператора объект `minivan` станет экземпляром класса `Vehicle`. Иными словами, класс обретет физическое воплощение. Не пытайтесь вникнуть пока что в детали приведенного выше оператора. В дальнейшем он станет вам полностью понятным.

Всякий раз, когда создается экземпляр класса, строится объект, содержащий копии всех переменных экземпляра, определенных в классе. Иными словами, каждый объект типа `Vehicle` будет содержать копии переменных `passengers`, `fuelcap` и `mpg`. Для обращения к этим переменным используется *оператор-точка* (`.`). Этот оператор связывает имя объекта с именем члена класса. Ниже приведена общая форма записи этого оператора.

объект.член

В этой форме *объект* указывается слева, а *член* — справа от точки. Так, если переменной `fuelcap` из объекта `minivan` требуется присвоить значение `16`, это можно сделать следующим образом:

```
minivan.fuelcap = 16;
```

Вообще говоря, оператором-точкой можно пользоваться для обращения как к переменным экземпляра, так и к методам.

Ниже приведен пример программы, в которой используется класс `Vehicle`.

```
/* Программа, в которой используется класс Vehicle.
```

```
    Присвоить ее исходному файлу имя VehicleDemo.java
*/
class Vehicle {
    int passengers; // количество пассажиров
    int fuelcap;    // емкость топливного бака
    int mpg;        // потребление топлива в милях на галлон
}
```

```
// В этом классе объявляется объект типа Vehicle.
class VehicleDemo {
    public static void main(String args[]) {
        Vehicle minivan = new Vehicle();
        int range;

        // присвоить значения полям в объекте minivan
        // Обратите внимание на применение оператора-точки
        // для доступа к переменным экземпляра данного объекта.
        minivan.passengers = 7;
        minivan.fuelcap = 16;
        minivan.mpg = 21;

        // рассчитать дальность действия транспортного средства,
        // исходя из того, что топливный бак заполнен
        range = minivan.fuelcap * minivan.mpg;
        System.out.println("Minivan can carry " + minivan.passengers +
            " with a range of " + range);
    }
}
```

Файлу, содержащему приведенный выше код, следует присвоить имя `VehicleDemo.java`, поскольку метод `main()` находится не в классе `Vehicle`, а в классе `VehicleDemo`. В результате компиляции программы будут созданы два файла с расширением `.class`: один — для класса `Vehicle`, а другой — для класса `VehicleDemo`. Компилятор Java автоматически помещает каждый класс в отдельный файл с расширением `.class`. Совсем не обязательно, чтобы классы `Vehicle` и `VehicleDemo` находились в одном и том же исходном файле. Их можно расположить в двух файлах — `Vehicle.java` и `VehicleDemo.java`.

Для того чтобы привести эту программу в действие, следует запустить на выполнение файл `VehicleDemo.class`. В итоге на экране появится следующий результат:

```
Minivan can carry 7 with a range of 336
```

А теперь самое время рассмотреть следующий основополагающий принцип: каждый объект содержит свои копии переменных экземпляра, определенные в его классе. Следовательно, содержимое переменных в одном объекте может отличаться от содержимого тех же самых переменных в другом объекте. Между объектами нет никакой связи, за исключением того, что они относятся к одному и тому же типу. Так, если имеются два объекта типа `Vehicle`, каждый из них содержит собственную копию переменных `passengers`, `fuelcap` и `mpg`, причем значения одноименных переменных в этих двух объектах могут отличаться. Этот факт демонстрирует следующий пример программы (обратите внимание на то, что класс, содержащий метод `main()`, на этот раз назван `TwoVehicles`):

```
// В этой программе создаются два объекта класса Vehicle.
class Vehicle {
    int passengers; // количество пассажиров
    int fuelcap;    // емкость топливного бака
    int mpg;        // потребление топлива в милях на галлон }

// В этом классе объявляется объект типа Vehicle.
class TwoVehicles {
    public static void main(String args[]) {
```

```

// Помните, что переменные minivan и sportscar
// ссылаются на разные объекты.
Vehicle minivan = new Vehicle();
Vehicle sportscar = new Vehicle();

int range1, range2;

// присвоить значения полям в объекте minivan
minivan.passengers = 7;
minivan.fuelcap = 16;
minivan.mpg = 21;

// присвоить значения полям в объекте sportscar
sportscar.passengers = 2;
sportscar.fuelcap = 14;
sportscar.mpg = 12;

// рассчитать дальность действия транспортного средства,
// исходя из того, что топливный бак заполнен
range1 = minivan.fuelcap * minivan.mpg;
range2 = sportscar.fuelcap * sportscar.mpg;

System.out.println("Minivan can carry " + minivan.passengers +
    " with a range of " + range1);
System.out.println("Sportscar can carry " + sportscar.passengers +
    " with a range of " + range2);
}
}

```

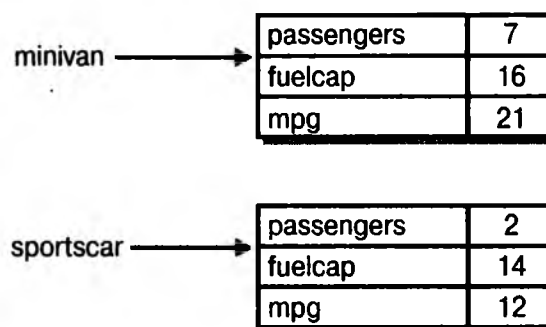
Ниже приведен результат выполнения данной программы.

```

Minivan can carry 7 with a range of 336
Sportscar can carry 2 with a range of 168

```

Как видите, данные из объекта `minivan` отличаются от соответствующих данных из объекта `sportscar`. Это обстоятельство иллюстрирует приведенный ниже рисунок.



Порядок создания объектов

В рассмотренных ранее примерах программ для объявления объекта типа `Vehicle` использовалась следующая строка кода:

```
Vehicle minivan = new Vehicle();
```

Эта строка кода выполняет две функции. Во-первых, в ней объявляется переменная класса `Vehicle` под именем `minivan`. Эта переменная еще не определяет объект, она лишь имеет возможность *ссылаться* на объект. И во-вторых, в этой строке кода созда-

ется физическая копия объекта, а ссылка на него присваивается переменной `minivan`. И делается это с помощью оператора `new`.

Оператор `new` динамически (т.е. в процессе выполнения программы) выделяет память для объекта и возвращает ссылку на него. Эта ссылка, по существу, представляет собой адрес области памяти, выделяемой для объекта оператором `new`. Ссылка на объект сохраняется в переменной. Таким образом, память для объектов всех классов в Java выделяется динамически.

Обе упомянутые функции приведенной выше строки кода можно записать следующим образом, чтобы разделить их и показать по отдельности:

```
Vehicle minivan; // объявить ссылку на объект
minivan = new Vehicle(); // выделить память для объекта типа Vehicle
```

В первой строке кода `minivan` объявляется как ссылка на объект типа `Vehicle`. Следует иметь в виду, что `minivan` — это переменная, которая может ссылаться на объект, а не сам объект. В данный момент переменная `minivan` пока еще не ссылается на объект. Во второй строке кода создается новый объект типа `Vehicle`, а ссылка на него присваивается переменной `minivan`. С этого момента переменная `minivan` оказывается связанной с объектом.

Переменные ссылочного типа и присваивание

В операции присваивания переменные ссылочного типа действуют иначе, чем переменные такого простого типа, как, например, `int`. Когда одна переменная простого типа присваивается другой, ситуация оказывается довольно простой. Переменная, находящаяся в левой части оператора присваивания, получает *копию значения* переменной, находящейся в правой части этого оператора. Когда же одна переменная ссылки на объект присваивается другой, ситуация несколько усложняется, поскольку такое присваивание приводит к тому, что переменная, находящаяся в левой части оператора присваивания, ссылается на тот же самый объект, на который ссылается переменная, находящаяся в правой части этого оператора. Сам же объект не копируется. В силу этого отличия присваивание переменных ссылочного типа может привести к несколько неожиданным результатам. В качестве примера рассмотрим следующий фрагмент кода:

```
Vehicle car1 = new Vehicle();
Vehicle car2 = car1;
```

На первый взгляд, переменные `car1` и `car2` ссылаются на совершенно разные объекты, но на самом деле это не так. Переменные `car1` и `car2`, напротив, ссылаются на один и тот же объект. Когда переменная `car1` присваивается переменной `car2`, в конечном итоге переменная `car2` просто ссылается на тот же самый объект, что и переменная `car1`. Следовательно, этим объектом можно оперировать с помощью переменной `car1` или `car2`. Например, после очередного присваивания

```
car1.mpg = 26;
```

оба метода `println()` в операторах их вызова

```
System.out.println(car1.mpg);
System.out.println(car2.mpg);
```

выводят одно и то же значение: **26**.

Несмотря на то что обе переменные, `car1` и `car2`, ссылаются на один и тот же объект, они никак иначе не связаны друг с другом. Например, в результате следующей

последовательности операций присваивания просто изменяется объект, на который ссылается переменная `car2`:

```
Vehicle car1 = new Vehicle();
Vehicle car2 = car1;
Vehicle car3 = new Vehicle();
car2 = car3; // Теперь переменные car2 и car3
              // ссылаются на один и тот же объект.
```

После выполнения этой последовательности операций присваивания переменная `car2` ссылается на тот же самый объект, что и переменная `car3`. А ссылка на объект в переменной `car1` не меняется.

Методы

Как пояснялось выше, переменные экземпляра и методы являются двумя основными составляющими классов. До сих пор класс `Vehicle`, рассматриваемый здесь в качестве примера, содержал только данные, но не методы. Хотя классы, содержащие только данные, вполне допустимы, у большинства классов должны быть также методы. *Методы* представляют собой подпрограммы, которые манипулируют данными, определенными в классе, а во многих случаях они предоставляют доступ к этим данным. Как правило, другие части программы взаимодействуют с классом посредством его методов.

Метод состоит из одного или нескольких операторов. В грамотно написанной программе на Java каждый метод выполняет только одну функцию. У каждого метода имеется свое имя, по которому он вызывается. В общем, методу в качестве имени можно присвоить любой действительный идентификатор. Следует, однако, иметь в виду, что идентификатор `main()` зарезервирован для метода, с которого начинается выполнение программы. Кроме того, в качестве имен методов нельзя использовать ключевые слова Java.

В этой книге методы именуются в соответствии с условными обозначениями, принятыми в литературе по Java. В частности, после имени метода следуют круглые скобки. Так, если методу присвоено имя `getval`, то в тексте книги он упоминается в следующем виде: `getval()`. Такая форма записи помогает отличать имена методов от имен переменных при чтении книги.

Ниже приведена общая форма объявления метода.

```
возвращаемый_тип имя(список_параметров) {
    // тело метода
}
```

где *возвращаемый_тип* обозначает тип данных, возвращаемых методом. Этот тип должен быть действительным, в том числе и типом создаваемого класса. Если метод не возвращает значение, то в качестве возвращаемого для него следует указать тип `void`. Далее, *имя* обозначает конкретное имя, присваиваемое методу. В качестве имени метода может служить любой допустимый идентификатор, не приводящий к конфликтам в текущей области объявлений. И наконец, *список_параметров* — это последовательность пар, состоящих из типа и идентификатора и разделенных запятыми. Параметры представляют собой переменные, получающие значение *аргументов*, передаваемых методу при его вызове. Если у метода отсутствуют параметры, то список параметров оказывается пустым.

Добавление метода в класс `Vehicle`

Как пояснялось ранее, методы класса обычно выполняют действия над данными в составе класса и предоставляют доступ к этим данным. Напомним, что метод `main()`

в предыдущих примерах вычислял дальность действия транспортного средства, умножая емкость топливного бака на число миль, которые оно может проехать, потребовав единичный объем топлива (в данном случае — галлон). И хотя такой расчет формально считается правильным, его лучше всего производить в пределах самого класса `Vehicle`. Аргументы в пользу такого решения очевидны: дальность действия транспортного средства зависит от потребления топлива в милях на галлон и емкости топливного бака, а обе эти величины инкапсулированы в классе `Vehicle`. Благодаря добавлению в класс `Vehicle` метода, предназначенного для расчета дальности, улучшается объектно-ориентированная структура кода.

Для того чтобы добавить метод в класс `Vehicle`, его следует объявить в пределах этого класса. Например, приведенный ниже вариант класса `Vehicle` содержит метод `range()`, определяющий и отображающий дальность действия транспортного средства.

```
// Добавление метода range в класс Vehicle.
class Vehicle {
    int passengers; // количество пассажиров
    int fuelcap;    // емкость топливного бака
    int mpg;        // потребление топлива в милях на галлон

    // отобразить дальность действия транспортного средства
    // Метод range() относится к классу Vehicle.
    // Обратите внимание на то, что переменные fuelcap и mpg
    // указываются напрямую без имени объекта и оператора-точки.
    void range() {
        System.out.println("Range is " + fuelcap * mpg);
    }
}

class AddMeth {
    public static void main(String args[]) {
        Vehicle minivan = new Vehicle();
        Vehicle sportscar = new Vehicle();

        int range1, range2;

        // присвоить значения полям в объекте minivan
        minivan.passengers = 7;
        minivan.fuelcap = 16;
        minivan.mpg = 21;

        // присвоить значения полям в объекте sportscar
        sportscar.passengers = 2;
        sportscar.fuelcap = 14;
        sportscar.mpg = 12;

        System.out.print("Minivan can carry " +
            minivan.passengers + ". ");

        minivan.range(); // отобразить дальность действия мини-фургона

        System.out.print("Sportscar can carry " +
            sportscar.passengers + ". ");
    }
}
```



```
sportscar.range(); // отобразить дальность действия спортивной машины
}
}
```

Ниже приведен результат выполнения данной программы.

```
Minivan can carry 7. Range is 336
Sportscar can carry 2. Range is 168
```

Рассмотрим основные элементы данной программы. Начнем с метода `range()`. Первая строка этого метода выглядит так:

```
void range() {
```

В этой строке объявляется метод `range`, для которого не предусмотрены параметры. В качестве типа, возвращаемого этим методом, указано ключевое слово `void`. Таким образом, метод `range()` не возвращает никаких данных вызывающей части программы. И завершается рассматриваемая здесь строка открывающей фигурной скобкой, обозначающей начало тела метода. Тело метода `range()` состоит из следующей единственной строки кода:

```
System.out.println("Range is " + fuelcap * mpg);
```

В этой строке на экран выводится дальность действия транспортного средства как результат перемножения переменных `fuelcap` и `mpg`. А поскольку у каждого объекта типа `Vehicle` имеются свои копии переменных `fuelcap` и `mpg`, то при вызове метода `range()` используются данные текущего объекта.

Действие метода `range()` завершается по достижении закрывающей фигурной скобки его тела. При этом управление передается обратно вызывающей части программы.

А теперь рассмотрим подробнее следующую строку кода в методе `main()`:

```
minivan.range();
```

В этой строке кода вызывается метод `range()` для объекта `minivan`. Для вызова метода относительно объекта перед его именем указываются имя объекта и оператор-точка. При вызове метода ему передается управление. Когда метод завершит свое действие, управление будет возвращено вызывающей части программы, и ее выполнение продолжится со строки кода, следующей за вызовом этого метода.

В данном случае в результате вызова `minivan.range()` отображается дальность действия транспортного средства, определяемого объектом `minivan`. Аналогично при вызове `sportscar.range()` на экран выводится дальность действия транспортного средства, определяемого объектом `sportscar`. При каждом вызове метода `range()` выводится дальность действия для указанного объекта.

Необходимо отметить следующую особенность метода `range()`: в нем выполняется непосредственное обращение к переменным экземпляра `fuelcap` и `mpg`, т.е. перед ними не указываются имя объекта и оператор-точка. Если в методе используется переменная экземпляра, определенная в его классе, обращаться к ней можно напрямую, не указывая объект. По зрелом размышлении следует признать, что такой подход вполне логичен. Ведь метод всегда вызывается относительно некоторого объекта своего класса, а следовательно, при вызове метода объект известен и нет никакой необходимости определять его еще раз. Это означает, что переменные `fuelcap` и `mpg`, встречающиеся в теле метода `range()`, неявно обозначают их копии, находящиеся в том объекте, для которого вызывается метод `range()`.

Возврат из метода

Возврат из метода осуществляется при выполнении одного из двух условий. Первое из них вам уже знакомо по методу `range()`, а именно: признаком завершения метода и возврата из него служит закрывающая круглая скобка. Вторым условием является выполнение оператора `return`. Существуют две разновидности оператора `return`: одна — для методов типа `void`, не возвращающих значение, а другая — для методов, возвращающих значение вызывающей части программы. Здесь мы рассмотрим первую разновидность оператора `return`, а о возвращаемых значениях речь пойдет в следующем разделе.

Организовать немедленное завершение метода типа `void` и возврат из него можно с помощью следующей формы оператора `return`:

```
return ;
```

При выполнении этого оператора управление будет возвращено вызывающей части программы, а оставшийся в методе код будет проигнорирован. Рассмотрим в качестве примера следующий метод:

```
void myMeth() {
    int i;
    for(i=0; i<10; i++) {
        if(i == 5) return; // завершить цикл на значении 5
        System.out.println();
    }
}
```

Здесь переменная цикла `for` принимает лишь значения от 0 до 5. Как только значение переменной `i` становится равным 5, цикл завершается и происходит возврат из метода. В одном методе допускается несколько операторов `return`. Необходимость в них возникает в том случае, если в методе организовано несколько ветвей выполнения, как в приведенном ниже примере.

```
void myMeth() {
    // ...
    if(done) return;
    // ...
    if(error) return;
    // ...
}
```

В данном примере метод возвращает управление вызывающей части программы либо по завершении всех необходимых действий, либо при появлении ошибки. Применяя операторы `return`, следует соблюдать осторожность: слишком большое количество точек возврата из метода нарушает структуру кода. В грамотно написанном методе точки возврата четко определены.

Итак, метод типа `void` может быть завершен одним из двух способов: по достижении закрывающей фигурной скобки тела метода или при выполнении оператора `return`.

Возврат значения

Несмотря на то что методы типа `void` встречаются довольно часто, большинство методов все же возвращают значения. Способность возвращать значение относится к одним из самых полезных свойств метода. Пример возврата значения уже встречался ранее в этой книге, когда для вычисления квадратного корня использовался метод `sqrt()`.

В программировании возвращаемые значения применяются для самых разных целей. В одних случаях, как, например, при обращении к методу `sqrt()`, возвращаемое значение представляет собой результат некоторых расчетов. А в других случаях это значение лишь сообщает, успешно ли были выполнены действия, предусмотренные в методе. При этом возвращаемое значение нередко содержит код состояния. Независимо от конкретного способа применения, возвращаемые значения являются неотъемлемой частью программирования на Java.

Методы возвращают значения вызывающей части программы, используя следующую форму оператора `return`:

```
return значение;
```

где *значение* — конкретное возвращаемое значение. Данная форма оператора `return` может быть использована только в тех методах, тип которых отличается от типа `void`. Более того, подобные методы *обязаны* возвращать значение, используя данную форму оператора `return`.

Теперь мы можем немного видоизменить метод `range()` с учетом возвращаемых значений. Вместо того чтобы отображать дальность в методе `range()`, лучше ограничиться ее расчетом и возвратом полученного значения. Преимущество такого подхода заключается, в частности, в том, что возвращаемое значение может быть использовано при выполнении других расчетов. В приведенном ниже примере метод `range()` видоизменен, возвращая значение, которое затем выводится на экран.

```
// Использование возвращаемого значения.
class Vehicle {
    int passengers; // количество пассажиров
    int fuelcap;    // емкость топливного бака
    int mpg;        // потребление топлива в милях на галлон

    // Возврат дальности действия для заданного транспортного средства.
    int range() {
        return mpg * fuelcap;
    }
}

class RetMeth {
    public static void main(String args[]) {
        Vehicle minivan = new Vehicle();
        Vehicle sportscar = new Vehicle();

        int range1, range2;

        // присвоить значения полям в объекте minivan
        minivan.passengers = 7;
        minivan.fuelcap = 16;
        minivan.mpg = 21;

        // присвоить значения полям в объекте sportscar
        sportscar.passengers = 2;
        sportscar.fuelcap = 14;
        sportscar.mpg = 12;

        // получить дальности действия разных транспортных средств
        // Возвращаемое методом значение присваивается переменной.
```

```

range1 = minivan.range();
range2 = sportscar.range();

System.out.println("Minivan can carry " + minivan.passengers +
    " with range of " + range1 + " Miles");
System.out.println("Sportscar can carry " + sportscar.passengers +
    " with range of " + range2 + " miles");
}
}

```

Ниже приведен результат выполнения данной программы.

```

Minivan can carry 7 with range of 336 Miles
Sportscar can carry 2 with range of 168 miles

```

Обратите внимание на то, что вызов метода `range()` в данной программе указывается в правой части оператора присваивания, тогда как в левой его части — переменная, которая принимает значение, возвращаемое методом `range()`. Таким образом, после выполнения следующей строки кода значение дальности действия для объекта `minivan` сохраняется в переменной `range1`:

```
range1 = minivan.range();
```

Следует иметь в виду, что в данном случае метод `range()` возвращает значение типа `int`, т.е. вызывающая часть программы получает целочисленное значение. Тип возвращаемого значения — очень важная характеристика метода, поскольку возвращаемые данные должны соответствовать типу, указанному в определении метода. Иными словами, если метод должен возвращать значение типа `double`, то именно таким и следует объявить его тип.

Несмотря на то что приведенная выше программа компилируется и выполняется без ошибок, ее эффективность можно повысить. В частности, переменные `range1` и `range2` в ней не нужны. Вызов метода `range()` можно непосредственно указать в качестве параметра метода `println()`, как показано ниже.

```

System.out.println("Minivan can carry " + minivan.passengers +
    " with range of " + minivan.range() + " Miles");

```

В данном случае при выполнении метода `println()` будет автоматически осуществляться вызов `minivan.range()`, а полученное в итоге значение — передаваться методу `println()`. Более того, к методу `range()` можно обратиться в любой момент, когда понадобится значение дальности действия для объекта типа `Vehicle`. В качестве примера ниже приведено выражение, в котором сравнивается дальность действия двух транспортных средств.

```
if(v1.range() > v2.range()) System.out.println("v1 has greater range");
```

Использование параметров

При вызове метода ему можно передать одно или несколько значений. Значение, передаваемое методу, называется *аргументом*. А переменная, получающая аргумент, называется *формальным параметром*, или просто *параметром*. Параметры объявляются в скобках после имени метода. Синтаксис объявления параметров такой же, как и у переменных. А областью действия параметров является тело метода. За исключением особых случаев передачи аргументов методу параметры действуют так же, как и любые другие переменные.

Ниже приведен простой пример программы, демонстрирующий использование параметров. В классе `ChkNum` метод `isEven()` возвращает логическое значение `true`, если значение, передаваемое при вызове этого метода, является четным числом. В противном случае метод возвращает логическое значение `false`. Таким образом, метод `isEven()` возвращает значение типа `boolean`.

// Простой пример применения параметра в методе.

```
class ChkNum {
    // вернуть логическое значение true,
    // если x содержит четное число
    // Здесь x — целочисленный параметр метода isEven().
    boolean isEven(int x) {
        if((x%2) == 0) return true;
        else return false;
    }
}

class ParmDemo {
    public static void main(String args[]) {
        ChkNum e = new ChkNum();

        // В следующих строках кода передаются аргументы методу isEven().
        if(e.isEven(10)) System.out.println("10 is even.");

        if(e.isEven(9)) System.out.println("9 is even.");

        if(e.isEven(8)) System.out.println("8 is even.");
    }
}
```

Выполнение этой программы дает следующий результат:

```
10 is even.
8 is even.
```

В данной программе метод `isEven()` вызывается трижды, и каждый раз ему передается новое значение. Рассмотрим подробнее ее исходный код. Обратите сначала внимание на то, каким образом вызывается метод `isEven()`. Его параметр указывается в круглых скобках. При первом вызове методу `isEven()` передается значение `10`. Следовательно, когда метод `isEven()` начинает выполняться, параметр `x` получает значение `10`. При втором вызове в качестве аргумента этому методу передается значение `9`, которое и принимает параметр `x`. А при третьем вызове методу `isEven()` передается значение `8`, которое опять же присваивается параметру `x`. Какое бы значение ни указать при вызове метода `isEven()`, его все равно получит параметр `x`.

В методе может быть определено несколько параметров, и в этом случае они разделяются запятыми. Допустим, в классе `Factor` имеется метод `isFactor()`, который определяет, является ли первый его параметр множителем второго, как показано ниже.

```
class Factor {
    // В этом методе предусмотрены два параметра.
    boolean isFactor(int a, int b) {
        if( (b % a) == 0) return true;
        else return false;
    }
}
```

```
}  
}  
  
class IsFact {  
    public static void main(String args[]) {  
        Factor x = new Factor();  
        // При вызове методу isFactor() передаются два аргумента.  
        if(x.isFactor(2, 20)) System.out.println("2 is factor");  
        if(x.isFactor(3, 20)) System.out.println("this won't be displayed");  
    }  
}
```

Обратите внимание на то, что при вызове метода `isFactor()` передаваемые ему значения также разделяются запятыми.

При использовании нескольких параметров для каждого из них определяется тип, причем типы параметров могут отличаться. Например, следующее объявление метода является корректным:

```
int myMeth(int a, double b, float c) {  
    // ...  
}
```

Добавление параметризованного метода в класс `Vehicle`

Параметризованный метод позволяет реализовать в классе `Vehicle` новую возможность: расчет объема топлива, необходимого для преодоления заданного расстояния. Назовем этот новый метод `fuelneeded()`. Он получает в качестве параметра расстояние в милях, которое должно проехать транспортное средство, а возвращает необходимое для этого количество галлонов топлива. Метод `fuelneeded()` определяется следующим образом:

```
double fuelneeded(int miles) {  
    return (double) miles / mpg;  
}
```

Обратите внимание на то, что этот метод возвращает значение типа `double`. Это важно, поскольку объем потребляемого топлива не всегда можно выразить целым числом. Ниже приведен исходный код программы для расчета дальности действия транспортных средств с классом `Vehicle`, содержащим метод `fuelneeded()`.

```
/*  
    Добавление параметризованного метода, в котором производится  
    расчет объема топлива, необходимого транспортному средству для  
    преодоления заданного расстояния.  
*/  
  
class Vehicle {  
    int passengers; // количество пассажиров  
    int fuelcap;    // емкость топливного бака  
    int mpg;        // потребление топлива в милях на галлон }  
  
    // вернуть дальность действия транспортного средства  
    int range() {  
        return mpg * fuelcap;  
    }  
}
```

```

// рассчитать объем топлива, необходимого транспортному
// средству для преодоления заданного расстояния
double fuelneeded(int miles) {
    return (double) miles / mpg;
}
}

class CompFuel {
    public static void main(String args[]) {
        Vehicle minivan = new Vehicle();
        Vehicle sportscar = new Vehicle();
        double gallons;
        int dist = 252;

        // присвоить значения полям в объекте minivan
        minivan.passengers = 7;
        minivan.fuelcap = 16;
        minivan.mpg = 21;

        // присвоить значения полям в объекте sportscar
        sportscar.passengers = 2;
        sportscar.fuelcap = 14;
        sportscar.mpg = 12;

        gallons = minivan.fuelneeded(dist);

        System.out.println("To go " + dist + " miles minivan needs " +
            gallons + " gallons of fuel.");

        gallons = sportscar.fuelneeded(dist);

        System.out.println("To go " + dist + " miles sportscar needs " +
            gallons + " gallons of fuel.");
    }
}

```

Выполнение этой программы дает следующий результат:

```

To go 252 miles minivan needs 12.0 gallons of fuel.
To go 252 miles sportscar needs 21.0 gallons of fuel.

```

Пример для опробования 4.1.

Создание справочного класса

Исходный файл: HelpClassDemo.java

Если попытаться кратко выразить сущность классов, то следовало бы сказать, что класс инкапсулирует функциональные возможности. Иногда трудно определить, где оканчиваются одни функциональные возможности и начинаются другие. Класс должен быть стандартным блоком для компоновки приложения. Для этой цели класс необходимо спроектировать таким образом, чтобы он представлял собой одну функциональную единицу, выполняющую строго определенные действия. Следовательно, нужно стремиться к тому, чтобы классы были как можно более компактными, но в разумных пределах! Ведь классы, реализующие лишние функциональные возможности, делают код сложным для понимания и плохо структурированным, но классы со слишком огра-

ниченными функциональными возможностями приводят к тому, что программа становится неоправданно фрагментированной. Как же найти золотую середину? В поисках ее наука программирования превращается в *искусство* программирования. Многие программисты считают, что такая способность приходит с опытом.

В качестве упражнения для приобретения нужных навыков работы с классами в этом проекте вам предстоит преобразовать в класс `Help` справочную систему, созданную в примере для опробования 3.3. Но прежде рассмотрим, какие для этого имеются основания. Во-первых, справочная система представляет собой один логический блок. Эта система лишь отображает синтаксис управляющих операторов Java. Ее функциональные возможности четко определены. Во-вторых, реализация справочной системы в виде класса представляет собой довольно изящное решение. Всякий раз, когда требуется отобразить подсказку для пользователя, достаточно создать экземпляр объекта справочной системы. И наконец, справочную информацию можно дополнить или изменить, не затрагивая остальные части программы, поскольку она инкапсулирована в классе.

Последовательность действий

1. Создайте новый файл `HelpClassDemo.java`. Ради экономии времени и труда на ввод исходного текста скопируйте файл `Help3.java`, созданный вами во время проработки примера для опробования 3.3, и сохраните его под именем `HelpClassDemo.java`.
2. Для того чтобы преобразовать справочную систему в класс, нужно сначала четко определить ее составные части. Так, в исходном файле `Help3.java` программы, реализующей справочную систему, имеется код, отвечающий за отображение меню, получение информации от пользователя, проверку достоверности ответа и отображение данных, соответствующих выбранному пункту меню. В этой программе имеется также цикл, который завершается при вводе символа `q`. По зрелом размышлении становится ясно, что средства организации меню, проверки достоверности ответа и отображения информации являются составными частями справочной системы. В то же время порядок получения данных от пользователя и обработки многократных запросов не имеет к системе непосредственного отношения. Таким образом, нужно создать класс, который отображает справочную информацию, меню для ее выбора и проверяет правильность сделанного выбора. Соответствующие методы класса можно назвать `helpon()`, `showmenu()` и `isvalid()`.
3. Создайте метод `helpon()`, исходный код которого приведен ниже.

```
void helpon(int what) {
    switch(what) {
        case '1':
            System.out.println("The if:\n");
            System.out.println("if(condition) statement;");
            System.out.println("else statement;");
            break;
        case '2':
            System.out.println("The switch:\n");
            System.out.println("switch(expression) {");
            System.out.println(" case constant;");
            System.out.println(" statement sequence");
            System.out.println(" break;");
            System.out.println(" // ...");
```



```

        System.out.println("}");
        break;
    case '3':
        System.out.println("The for:\n");
        System.out.print("for(init; condition; iteration)");
        System.out.println(" statement;");
        break;
    case '4':
        System.out.println("The while:\n");
        System.out.println("while(condition) statement;");
        break;
    case '5':
        System.out.println("The do-while:\n");
        System.out.println("do {");
        System.out.println(" statement;");
        System.out.println("} while (condition);");
        break;
    case '6':
        System.out.println("The break:\n");
        System.out.println("break; or break label;");
        break;
    case '7':
        System.out.println("The continue:\n");
        System.out.println("continue; or continue label;");
        break;
    }
    System.out.println();
}

```

4. Далее создайте метод `showmenu()`, исходный код которого приведен ниже.

```

void showmenu() {
    System.out.println("Help on:");
    System.out.println(" 1. if");
    System.out.println(" 2. switch");
    System.out.println(" 3. for");
    System.out.println(" 4. while");
    System.out.println(" 5. do-while");
    System.out.println(" 6. break");
    System.out.println(" 7. continue\n");
    System.out.print("Choose one (q to quit): ");
}

```

5. И наконец, создайте метод `isvalid()`, исходный код которого приведен ниже.

```

boolean isvalid(int ch) {
    if(ch < '1' | ch > '7' & ch != 'q') return false;
    else return true;
}

```

6. Добавьте созданные выше методы в класс `Help`, как показано ниже.

```

class Help {
    void helpon(int what) {
        switch(what) {
            case '1':
                System.out.println("The if:\n");

```

```

        System.out.println("if(condition) statement;");
        System.out.println("else statement;");
        break;
    case '2':
        System.out.println("The switch:\n");
        System.out.println("switch(expression) {");
        System.out.println(" case constant;");
        System.out.println(" statement sequence");
        System.out.println(" break;");
        System.out.println(" // ...");
        System.out.println("}");
        break;
    case '3':
        System.out.println("The for:\n");
        System.out.println("for(init; condition; iteration)");
        System.out.println(" statement;");
        break;
    case '4':
        System.out.println("The while:\n");
        System.out.println("while(condition) statement;");
        break;
    case '5':
        System.out.println("The do-while:\n");
        System.out.println("do {");
        System.out.println(" statement;");
        System.out.println("} while (condition);");
        break;
    case '6':
        System.out.println("The break:\n");
        System.out.println("break; or break label;");
        break;
    case '7':
        System.out.println("The continue:\n");
        System.out.println("continue; or continue label;");
        break;
    }
    System.out.println();
}

void showmenu() {
    System.out.println("Help on:");
    System.out.println(" 1. if");
    System.out.println(" 2. switch");
    System.out.println(" 3. for");
    System.out.println(" 4. while");
    System.out.println(" 5. do-while");
    System.out.println(" 6. break");
    System.out.println(" 7. continue\n");
    System.out.print("Choose one (q to quit): ");
}

boolean isValid(int ch) {
    if(ch < '1' | ch > '7' & ch != 'q') return false;
}

```

```

        else return true;
    }
}

```

7. Перепишите метод `main()` из примера для опробования 3.3 таким образом, чтобы использовать в нем новый класс `Help`. Сохраните новый исходный код в файле `HelpClassDemo.java`. Ниже приведен весь исходный код программы, реализующей справочную систему в файле `HelpClassDemo.java`.

```
/*
```

```
    Пример для опробования 4.1.
```

```

    Преобразование в класс Help справочной системы
    из примера для опробования 3.3.

```

```
*/
```

```

class Help {
    void helpon(int what) {
        switch(what) {
            case '1':
                System.out.println("The if:\n");
                System.out.println("if(condition) statement;");
                System.out.println("else statement;");
                break;
            case '2':
                System.out.println("The switch:\n");
                System.out.println("switch(expression) {");
                System.out.println(" case constant;");
                System.out.println(" statement sequence");
                System.out.println(" break;");
                System.out.println(" // ...");
                System.out.println("}");
                break;
            case '3':
                System.out.println("The for:\n");
                System.out.print("for(init; condition; iteration)");
                System.out.println(" statement;");
                break;
            case '4':
                System.out.println("The while:\n");
                System.out.println("while(condition) statement;");
                break;
            case '5':
                System.out.println("The do-while:\n");
                System.out.println("do {");
                System.out.println(" statement;");
                System.out.println("} while (condition);");
                break;
            case '6':
                System.out.println("The break:\n");
                System.out.println("break; or break label;");
                break;
            case '7':
                System.out.println("The continue:\n");

```

```
        System.out.println("continue; or continue label;");
        break;
    }
    System.out.println();
}

void showmenu() {
    System.out.println("Help on:");
    System.out.println(" 1. if");
    System.out.println(" 2. switch");
    System.out.println(" 3. for");
    System.out.println(" 4. while");
    System.out.println(" 5. do-while");
    System.out.println(" 6. break");
    System.out.println(" 7. continue\n");
    System.out.print("Choose one (q to quit): ");
}

boolean isvalid(int ch) {
    if(ch < '1' | ch > '7' & ch != 'q') return false;
    else return true;
}

}

class HelpClassDemo {
    public static void main(String args[])
        throws java.io.IOException {
        char choice, ignore;
        Help hlpobj = new Help();

        for(;;) {
            do {
                hlpobj.showmenu();

                choice = (char) System.in.read();

                do {
                    ignore = (char) System.in.read();
                } while(ignore != '\n');
            } while( !hlpobj.isvalid(choice) );

            if(choice == 'q') break;

            System.out.println("\n");

            hlpobj.helpon(choice);
        }
    }
}
```

Запустив эту программу на выполнение, вы увидите, что она ведет себя точно так же, как и предыдущая ее версия. Преимущество текущей ее версии заключается лишь

в том, что теперь справочная система может быть использована повторно всякий раз, когда в этом возникнет потребность.

Конструкторы

В предыдущем примере программы мы вынуждены были вручную устанавливать значения переменных экземпляра для каждого объекта типа `Vehicle`, как показано ниже.

```
minivan.passengers = 7;
minivan.fuelcap = 16;
minivan.mpg = 21;
```

Но такой подход вообще не применяется в программах, профессионально написанных на Java, по следующим причинам. Во-первых, существует большая вероятность допустить ошибку (можно, например, забыть установить значение одного из полей). И во-вторых, существует гораздо более простой и надежный способ решения подобной задачи: использовать конструктор.

Конструктор инициализирует объект при его создании. У конструктора такое же имя, как и у его класса, а с точки зрения синтаксиса он подобен методу. Но у конструкторов нет возвращаемого типа, указываемого явно. Как правило, конструкторы используются для задания первоначальных значений переменных экземпляра, определенных в классе, или же для выполнения любых других установочных процедур, которые требуются для создания полностью сформированного объекта.

У всех классов имеются конструкторы, независимо от того, определите вы их или нет, поскольку в Java автоматически предоставляется конструктор, используемый по умолчанию и инициализирующий все переменные экземпляра их значениями по умолчанию. Для большинства типов данных значением по умолчанию является нулевое, для типа `bool` — логическое значение `false`, а для ссылочных типов — пустое значение `null`. Но как только вы определите свой собственный конструктор, конструктор по умолчанию больше не используется.

Ниже приведен простой пример, демонстрирующий применение конструктора.

```
// Простой конструктор.

class MyClass {
    int x;

    // Конструктор класса MyClass.
    MyClass() {
        x = 10;
    }
}

class ConsDemo {
    public static void main(String args[]) {
        MyClass t1 = new MyClass();
        MyClass t2 = new MyClass();

        System.out.println(t1.x + " " + t2.x);
    }
}
```

В данном примере конструктор класса `MyClass` объявляется следующим образом:

```
MyClass() {
    x = 10;
}
```

В этом конструкторе переменной экземпляра `x`, определяемой в классе `MyClass`, присваивается значение `10`. Этот конструктор вызывается оператором `new` при создании объекта данного класса. Ниже приведена строка кода, в которой используется оператор `new`.

```
MyClass t1 = new MyClass();
```

В этой строке кода для объекта `t1` вызывается конструктор `MyClass()`, в котором переменной экземпляра `t1.x` присваивается значение `10`. То же самое происходит и для объекта `t2`. После вызова данного конструктора переменная экземпляра `t2.x` также получает значение `10`. Таким образом, выполнение приведенного выше примера программы дает следующий результат:

```
10 10
```

Параметризованные конструкторы

В предыдущем примере использовался конструктор без параметров. В некоторых случаях этого оказывается достаточно, но зачастую конструктор должен принимать один или несколько параметров. В конструктор параметры вводятся таким же образом, как и в метод. Для этого достаточно объявить их в скобках после имени конструктора. Ниже приведен пример применения параметризованного конструктора класса `MyClass`.

```
// Параметризованный конструктор.
```

```
class MyClass {
    int x;

    // У этого конструктора имеется параметр.
    MyClass(int i) {
        x = i;
    }
}

class ParmConsDemo {
    public static void main(String args[]) {
        MyClass t1 = new MyClass(10);
        MyClass t2 = new MyClass(88);

        System.out.println(t1.x + " " + t2.x);
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
10 88
```

В данной версии программы в конструкторе класса `MyClass` определяется единственный параметр `i`, который используется для инициализации переменной экземпляра `x`. При выполнении следующей строки кода значение `10` сначала передается параметру `i` данного конструктора, а затем присваивается переменной `x`:

```
MyClass t1 = new MyClass(10);
```

Добавление конструктора в класс Vehicle

Теперь мы можем усовершенствовать класс `Vehicle`, добавив в него конструктор, в котором будут автоматически инициализироваться поля `passengers`, `fuelcap` и `mpg` при построении объекта. Обратите особое внимание на то, каким образом создаются объекты типа `Vehicle`.

// Добавление конструктора.

```
class Vehicle {
    int passengers; // количество пассажиров
    int fuelcap;    // емкость топливного бака
    int mpg;        // потребление топлива в милях на галлон

    // Это конструктор класса Vehicle.
    Vehicle(int p, int f, int m) {
        passengers = p;
        fuelcap = f;
        mpg = m;
    }

    // вернуть дальность действия транспортного средства
    int range() {
        return mpg * fuelcap;
    }

    // рассчитать объем топлива, необходимого транспортному
    // средству для преодоления заданного расстояния
    double fuelneeded(int miles) {
        return (double) miles / mpg;
    }
}

class VehConsDemo {
    public static void main(String args[]) {

        // сконструировать полностью объекты транспортных средств
        Vehicle minivan = new Vehicle(7, 16, 21);
        Vehicle sportscar = new Vehicle(2, 14, 12);
        double gallons;
        int dist = 252;

        gallons = minivan.fuelneeded(dist);

        System.out.println("To go " + dist + " miles minivan needs " +
            gallons + " gallons of fuel.");

        gallons = sportscar.fuelneeded(dist);

        System.out.println("To go " + dist + " miles sportscar needs " +
            gallons + " gallons of fuel.");
    }
}
```

При создании объекты `minivan` и `sportscar` инициализируются конструктором `Vehicle()`. Каждый такой объект инициализируется параметрами, указанными в конструкторе его класса. Например, в строке кода

```
Vehicle minivan = new Vehicle(7, 16, 21);
```

значения **7**, **16** и **21** передаются конструктору `Vehicle()` в качестве параметров при создании нового объекта `minivan` с помощью оператора `new`.

В итоге копии переменных `passengers`, `fuelcap` и `mpg` в объекте `minivan` будут содержать значения **7**, **16** и **21** соответственно. Рассмотренная здесь версия программы дает такой же результат, как и ее предыдущая версия.

Еще раз об операторе `new`

Теперь, когда вы ближе ознакомились с классами и их конструкторами, вернемся к оператору `new`, чтобы рассмотреть его более подробно. Ниже приведена общая форма этого оператора в контексте присваивания.

```
переменная_класса = new имя_класса(список_аргументов)
```

где *переменная_класса* обозначает тип переменной создаваемого класса, а *имя_класса* — конкретное имя класса, реализуемого в виде экземпляра его объекта. Имя класса и *список_аргументов* в скобках, который может быть пустым, обозначают конструктор этого класса. Если в классе не определен его собственный конструктор, то в операторе `new` будет использован конструктор, предоставляемый в Java по умолчанию. Следовательно, оператор `new` может быть использован для создания объекта, относящегося к классу любого типа. Оператор `new` возвращает ссылку на вновь созданный объект, который получает *переменная_класса* в результате присваивания в данной форме записи.

Оперативная память не бесконечна, и поэтому вполне возможно, что оператору `new` не удастся выделить память для объекта из-за нехватки доступной памяти. В этом случае возникает исключительная ситуация во время выполнения (подробнее об обработке исключительных ситуаций речь пойдет в главе 9). В примерах программ, представленных в этой книге, ситуация, связанная с исчерпанием оперативной памяти, не учитывается, но при написании реальных программ такую возможность, вероятно, придется принимать во внимание.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Почему оператор `new` не указывается для переменных таких простых типов, как `int` или `float`?

ОТВЕТ. В Java простые типы не реализованы в виде объектов. Ради повышения эффективности кода они реализованы как “обычные” переменные. Переменная простого типа фактически содержит присвоенное ей значение. Как пояснялось ранее, объектные переменные ссылочного типа представляют собой ссылки на объекты. Такая косвенная адресация (а также другие особенности объектов) вносят дополнительные издержки в обращение с объектами. Но подобные издержки не возникают при обращении с простыми типами данных.

“Сборка мусора” и методы завершения

Как было показано выше, при использовании оператора `new` свободная память для создаваемых объектов динамически выделяется из доступной буферной области оперативной памяти. Разумеется, оперативная память не бесконечна, и поэтому свободно доступная память рано или поздно исчерпывается. Это может привести к неудачному выполнению оператора `new` из-за нехватки свободной памяти для создания требуемого объекта. Именно по этой причине одной из главных функций любой схемы динамического распределения памяти является своевременное освобождение памяти от неиспользуемых объектов, чтобы сделать ее доступной для последующего перераспределения. Во многих языках программирования освобождение распределенной ранее памяти осуществляется вручную. Например, в C++ для этой цели служит оператор `delete`. Но в Java применяется другой, более надежный подход: *“сборка мусора”*.

Система “сборки мусора” в Java освобождает память от лишних объектов автоматически, действуя подспудно, незаметно и без всякого вмешательства со стороны программиста. “Сборка мусора” происходит следующим образом. Если ссылки на объект отсутствуют, то такой объект считается больше ненужным, и занимаемая им память в итоге освобождается. Эта утилизированная память может быть затем распределена для других объектов.

“Сборка мусора” происходит лишь время от времени по ходу выполнения программы. Она не состоится только потому, что существует один или несколько объектов, которые больше не используются. Следовательно, нельзя заранее знать или предположить, когда именно произойдет “сборка мусора”.

Метод `finalize()`

Существует возможность определить метод, который будет вызван непосредственно перед окончательным удалением объекта из памяти. Этот метод называется `finalize()`. Он позволяет убедиться, что объект больше не существует. Этот метод можно, например, использовать для закрытия файла, открытого проверяемым объектом.

Для того чтобы добавить в класс метод завершения, достаточно определить метод `finalize()`. Исполняющая система Java вызовет этот метод перед фактическим удалением объекта. В теле метода `finalize()` следует предусмотреть действия, которые должны быть выполнены непосредственно перед удалением объекта.

Ниже приведена общая форма метода `finalize()`.

```
protected void finalize()
{
    // здесь указывается код метода завершения
}
```

Ключевое слово `protected` является здесь спецификатором, предотвращающим обращение к методу `finalize()` за пределами класса. Этот и другие спецификаторы доступа подробнее рассматриваются в главе 6.

Следует, однако, иметь в виду, что метод `finalize()` вызывается непосредственно перед операцией “сборки мусора” по отношению к удаляемому объекту. Но он не вызывается в том случае, если объект оказывается за пределами области действия. Поэтому заранее невозможно предсказать, когда и при каких условиях метод `finalize()` будет выполнен. Так, если программа завершится до того, как будет запущена процедура “сборки мусора”, метод `finalize()` не будет выполнен. Таким образом, данный метод не пригоден для корректного освобождения занятых ресурсов или для других специаль-

ных целей, а тем более для нормального завершения работы программы. Короче говоря, метод `finalize()` имеет специальное назначение и поэтому редко требуется для завершения большинства программ.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Мне известно, что в языке C++ определены так называемые *деструкторы*, автоматически выполняющиеся при удалении объектов. Подобен ли метод `finalize()` деструктору?

ОТВЕТ. В языке Java деструкторы не определены. Метод `finalize()` действительно выполняет функции деструктора, но не равнозначен ему. Например, деструктор C++ всегда вызывается перед тем, как объект исчезает из области действия, а предсказать момент выполнения метода `finalize()` для конкретного объекта невозможно. Но, поскольку в Java используется автоматическая “сборка мусора”, особой надобности в деструкторе не существует.

Пример для опробования 4.2.

Демонстрация “сборки мусора” и завершения кода

Исходный файл: `Finalize.java`

В связи с тем что “сборка мусора” начинается в произвольные моменты времени и выполняется в фоновом режиме, продемонстрировать ее действие не так-то просто, но это можно все же сделать с помощью метода `finalize()`. Напомним, этот метод вызывается в тот момент, когда объект должен быть удален. Но, как пояснялось ранее, объект не обязательно удаляется именно тогда, когда необходимость в нем отпадает. Вместо этого система “сборки мусора” дожидается того момента, когда освобождение памяти может быть произведено наиболее эффективно. Чаще всего для этого должно накопиться достаточно большое количество неиспользуемых объектов. Поэтому для демонстрации “сборки мусора” с помощью метода `finalize()` нужно создать и удалить как можно больше объектов, что и предстоит сделать в данном проекте.

Последовательность действий

1. Создайте новый файл `Finalize.java`.
2. Создайте класс `FDemo`, как показано ниже.

```
class FDemo {
    int x;

    FDemo(int i) {
        x = i;
    }

    // вызывается при утилизации объекта
    protected void finalize() {
        System.out.println("Finalizing " + x);
    }

    // формирует объект, который тотчас уничтожается
    void generator(int i) {
```

```

        FDemo o = new FDemo(i);
    }
}

```

3. В конструкторе данного класса устанавливается значение переменной экземпляра `x`, определяемое передаваемым параметром. В данном примере переменная экземпляра `x` служит в качестве идентификатора объекта. При утилизации объекта метод `finalize()` отображает значение переменной `x`. Обратите особое внимание на метод `generator()`. В нем создается объект типа `FDemo`, который сразу же уничтожается. Этот метод будет использован в дальнейшем.

4. Создайте класс `Finalize`, как показано ниже.

```

class Finalize {
    public static void main(String args[]) {
        int count;

        FDemo ob = new FDemo(0);

        /* А теперь сформировать большое количество объектов.
           В какой-то момент должна начаться "сборка мусора".
           Примечание: количество формируемых объектов, возможно,
           придется увеличить, чтобы принудить "сборку мусора". */
        for(count=1; count < 100000; count++)
            ob.generator(count);
    }
}

```

5. В классе `Finalize` сначала создается исходный объект `ob` типа `FDemo`. Затем из этого объекта формируется 100000 других аналогичных объектов. С этой целью вызывается метод `generator()` для объекта `ob`. На различных этапах данного процесса вступает в действие процедура "сборки мусора". Частота активизации данной процедуры зависит от целого ряда факторов, в том числе от объема свободной памяти и типа операционной системы. Но в любом случае в какой-то момент вы увидите сообщения, выводимые на экран в процессе выполнения метода `finalize()`. Если ни одного сообщения не появится, попробуйте увеличить число создаваемых объектов, изменив условие завершения цикла `for`.

6. Ниже приведен весь исходный код программы из файла `Finalize.java`.

```

/*
   Пример для опробования 4.2.

   Демонстрация "сборки мусора" и метода finalize().
*/

class FDemo {
    int x;

    FDemo(int i) {
        x = i;
    }

    // вызывается при утилизации объекта
    protected void finalize() {

```

```

        System.out.println("Finalizing " + x);
    }

    // формирует объект, который тотчас уничтожается
    void generator(int i) {
        FDemo o = new FDemo(i);
    }
}

class Finalize {
    public static void main(String args[]) {
        int count;

        FDemo ob = new FDemo(0);

        /* А теперь сформировать большое количество объектов.
           В какой-то момент должна начаться "сборка мусора".
           Примечание: количество формируемых объектов, возможно,
           придется увеличить, чтобы принудить "сборку мусора". */
        for(count=1; count < 100000; count++)
            ob.generator(count);
    }
}

```

Ключевое слово **this**

И в завершение этой главы рассмотрим ключевое слово **this**. Когда метод вызывается, ему автоматически передается ссылка на вызывающий объект, т.е. тот объект, для которого вызывается данный метод. Эта ссылка обозначается ключевым словом **this**. Следовательно, ключевое слово **this** обозначает именно тот объект, по ссылке на который действует вызываемый метод. Для того чтобы стало яснее назначение ключевого слова **this**, рассмотрим сначала пример программы, в которой создается класс **Pwr**, в котором вычисляется результат возведения числа в некоторую целочисленную степень.

```

class Pwr {
    double b;
    int e;
    double val;

    Pwr(double base, int exp) {
        b = base;
        e = exp;

        val = 1;
        if(exp==0) return;
        for( ; exp>0; exp--) val = val * base;
    }

    double get_pwr() {
        return val;
    }
}

```

```

}

class DemoPwr {
    public static void main(String args[]) {
        Pwr x = new Pwr(4.0, 2);
        Pwr y = new Pwr(2.5, 1);
        Pwr z = new Pwr(5.7, 0);

        System.out.println(x.b + " raised to the " + x.e +
            " power is " + x.get_pwr());
        System.out.println(y.b + " raised to the " + y.e +
            " power is " + y.get_pwr());
        System.out.println(z.b + " raised to the " + z.e +
            " power is " + z.get_pwr());
    }
}

```

Как вам должно быть уже известно, в теле метода можно непосредственно обращаться к другим членам класса, не указывая имя объекта или класса. Так, в методе `get_pwr()` имеется следующий оператор:

```
return val;
```

Он означает, что из данного метода должна быть возвращена копия значения переменной `val`, связанной с вызывающим объектом. Этот оператор можно переписать следующим образом:

```
return this.val;
```

где ключевое слово `this` ссылается на объект, для которого был вызван метод `get_pwr()`. Следовательно, `this.val` — это ссылка на копию переменной `val` в данном объекте. Так, если бы метод `get_pwr()` был вызван для объекта `x`, ключевое слово `this` в приведенном выше операторе ссылалось бы на объект `x`. Оператор, в котором отсутствует ключевое слово `this`, на самом деле является не более чем сокращенной записью.

Ниже приведен исходный код класса `Pwr`, написанный с использованием ключевого слова `this`.

```

class Pwr {
    double b;
    int e;
    double val;

    Pwr(double base, int exp) {
        this.b = base;
        this.e = exp;

        this.val = 1;
        if(exp==0) return;
        for( ; exp>0; exp--) this.val = this.val * base;
    }

    double get_pwr() {
        return this.val;
    }
}

```

На самом деле ни один программирующий на Java не напишет класс `Pwr` подобным образом, поскольку, добавляя ключевое слово `this`, он не получит никаких преимуществ. В то же время стандартная форма записи тех же самых операторов выглядит намного проще. Но в ряде случаев ключевое слово `this` может оказаться очень полезным. Например, синтаксис языка Java не запрещает использовать имена параметров или локальных переменных, совпадающие с именами глобальных переменных. В этом случае локальная переменная или параметр *скрывает* переменную экземпляра. А доступ к скрытой переменной экземпляра позволяет получить ключевое слово `this`. Так, приведенный ниже пример конструктора класса `Pwr()` синтаксически правилен, но подобного стиля программирования рекомендуется все же избегать.

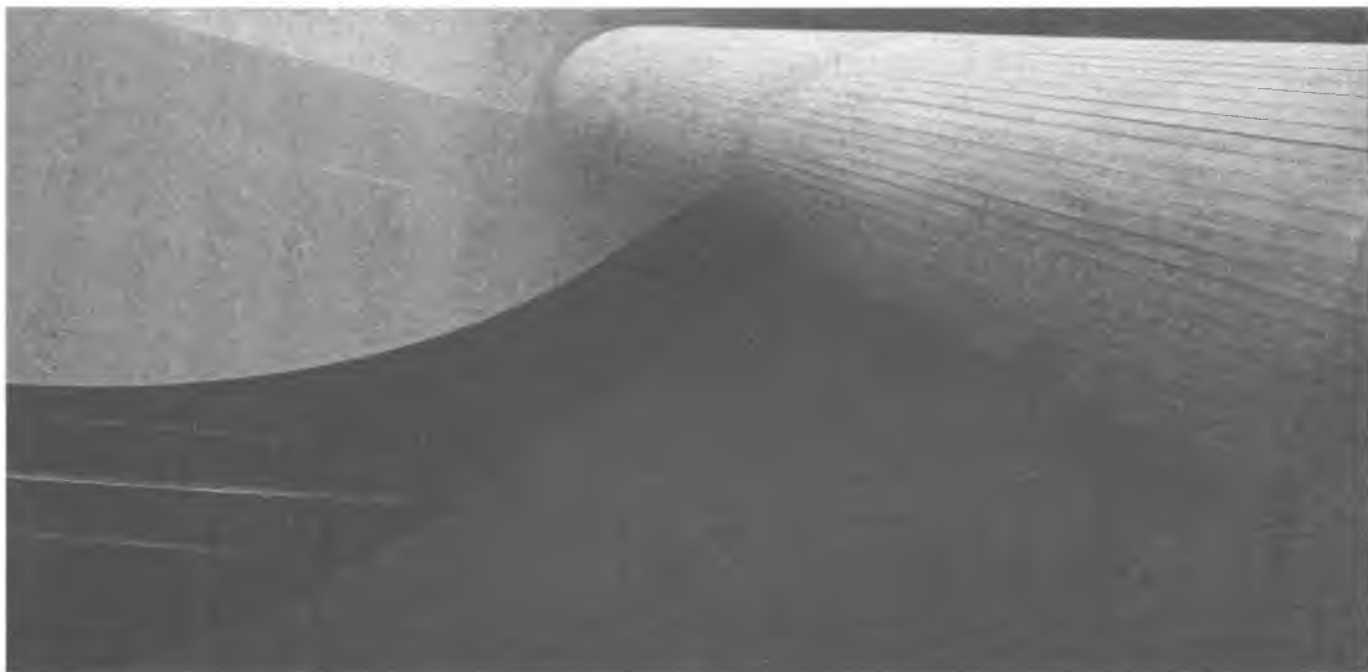
```
Pwr(double b, int e) {
    // Здесь ключевое слово this обозначает ссылку
    // на переменные b и e, а не на параметры.
    this.b = b;
    this.e = e;

    val = 1;
    if(e==0) return;
    for( ; e>0; e--) val = val * b;
}
```

В данной версии конструктора класса `Pwr` имена параметров совпадают с именами переменных экземпляра, скрывая их. А ключевое слово `this` используется здесь для того, чтобы “раскрыть” переменные экземпляра.

Упражнение для самопроверки по материалу главы 4

1. В чем отличие класса от объекта?
2. Как определяется класс?
3. Чью собственную копию содержит каждый объект?
4. Покажите, как объявить объект `counter` класса `MyCounter`, используя два отдельных оператора.
5. Как должен быть объявлен метод `myMeth`, принимающий два параметра, `a` и `b`, типа `int` и возвращающий значение типа `double`?
6. Как должно завершаться выполнение метода, возвращающего некоторое значение?
7. Каким должно быть имя конструктора?
8. Какие действия выполняет оператор `new`?
9. Что такое “сборка мусора” и какие действия она выполняет? Зачем нужен метод `finalize()`?
10. Что означает ключевое слово `this`?
11. Может ли конструктор иметь один или несколько параметров?
12. Если метод не возвращает значения, то как следует объявить тип этого метода?



Глава 5

**Дополнительные
сведения о типах
данных и операторах**

Основные навыки и понятия

- Представление о массивах
- Создание многомерных массивов
- Создание нерегулярных массивов
- Представление об альтернативном синтаксисе объявления массивов
- Присваивание ссылок на массивы
- Применение переменной экземпляра `length`
- Использование разновидности `for-each` цикла `for`
- Манипулирование символьными строками
- Употребление аргументов командной строки
- Использование поразрядных операторов
- Применение оператора `?`

В этой главе мы возвращаемся к обсуждению типов данных и операторов Java. В ней речь пойдет, в частности, о массивах, классе `String`, поразрядных операторах и тернарном операторе `?`. Кроме того, мы рассмотрим разновидность цикла `for`, реализованную лишь в самых последних версиях языка. Внимание также будет уделено аргументам командной строки.

Массивы

Массив представляет собой совокупность однотипных переменных с общим для обращения к ним именем. В Java массивы могут быть как одномерными, так и многомерными, хотя чаще всего применяются одномерные массивы. Массивы служат самым разным целям, поскольку они предоставляют удобные средства для объединения связанных вместе переменных. Например, в массиве можно хранить максимальные суточные температуры, зарегистрированные в течение месяца, перечень биржевых курсов или же названия книг по программированию из домашней библиотеки.

Главное преимущество массива — в организации данных таким образом, чтобы ими было проще манипулировать. Так, если имеется массив, содержащий дивиденды, выплачиваемые по избранной группе акций, то, организовав циклическое обращение к элементам этого массива, можно без особого труда рассчитать средний доход от этих акций. Кроме того, массивы позволяют организовать данные таким образом, чтобы легко отсортировать их.

Массивами в Java можно пользоваться практически так же, как и в других языках программирования. Тем не менее у них имеется одна особенность: они реализованы в виде объектов. Именно поэтому их рассмотрение было отложено до тех пор, пока в этой книге не были представлены объекты. Реализация массивов в виде объектов дает ряд существенных преимуществ, и далеко не самым последним среди них является возможность утилизировать неиспользуемые массивы средствами “сборки мусора”.

Одномерные массивы

Одномерный массив представляет собой список связанных переменных. Такие списки часто применяются в программировании. Например, в одномерном массиве можно хранить учетные номера активных пользователей сети или текущие средние уровни достижений бейсбольной команды.

Для объявления одномерного массива обычно применяется следующая общая форма:

```
тип имя_массива[] = new тип[размер];
```

где *тип* объявляет конкретный тип элемента массива. Тип элемента, называемый также *базовым*, определяет тип данных каждого элемента, составляющего массив. А *размер* определяет число элементов массива. В связи с тем что массивы реализованы в виде объектов, создание массива происходит в два этапа. Сначала объявляется переменная, ссылающаяся на массив, затем выделяется память для массива, а ссылка на нее присваивается переменной массива. Следовательно, память для массивов в Java динамически распределяется с помощью оператора `new`.

Проиллюстрируем все сказанное выше на конкретном примере. В следующей строке кода создается массив типа `int`, состоящий из 10 элементов, а ссылка на него присваивается переменной `sample`:

```
int sample[] = new int[10];
```

Объявление массива происходит так же, как и объявление объекта. В переменной `sample` сохраняется ссылка на область памяти, выделяемую для массива оператором `new`. Этой памяти должно быть достаточно для размещения в ней 10 элементов типа `int`.

Как и объявление объектов, приведенное выше объявление массива можно разделить на две отдельные составляющие следующим образом:

```
int sample[];  
sample = new int[10];
```

В данном случае сначала создается переменная `sample`, которая пока что не ссылается на конкретный объект. А затем переменная `sample` получает ссылку на конкретный массив.

Доступ к отдельным элементам массива осуществляется с помощью индексов. *Индекс* обозначает положение элемента в массиве. В Java индекс первого элемента массива равен нулю. Так, если массив `sample` содержит 10 элементов, их индексы находятся в пределах от 0 до 9. Индексирование массива осуществляется по номерам его элементов, заключенным в квадратные скобки. Например, для доступа к первому элементу массива `sample` следует указать `sample[0]`, а для доступ к последнему элементу этого массива — `sample[9]`. В приведенном ниже примере программы в массиве `sample` сохраняются числа от 0 до 9.

```
// Демонстрация одномерного массива.  
class ArrayDemo {  
    public static void main(String args[]) {  
        int sample[] = new int[10];  
        int i;  
  
        // Массивы индексируются с нуля, как показано ниже.  
        for(i = 0; i < 10; i = i+1)  
            sample[i] = i;  
  
        for(i = 0; i < 10; i = i+1)
```

```

        System.out.println("This is sample[" + i + "]: " + sample[i]);
    }
}

```

Результат выполнения данной программы выглядит следующим образом:

```

This is sample[0]: 0
This is sample[1]: 1
This is sample[2]: 2
This is sample[3]: 3
This is sample[4]: 4
This is sample[5]: 5
This is sample[6]: 6
This is sample[7]: 7
This is sample[8]: 8
This is sample[9]: 9

```

Структура массива `sample` наглядно показана на приведенном ниже рисунке.

0	1	2	3	4	5	6	7	8	9
sample [0]	sample [1]	sample [2]	sample [3]	sample [4]	sample [5]	sample [6]	sample [7]	sample [8]	sample [9]

Массивы часто употребляются в программировании, поскольку они позволяют обрабатывать в цикле большое количество переменных. Например, в результате выполнения следующего примера программы определяется минимальное и максимальное значения из всех, хранящихся в массиве `nums`. Элементы этого массива перебираются в цикле `for`.

// Нахождение минимального и максимального значений в массиве.

```

class MinMax {
    public static void main(String args[]) {
        int nums[] = new int[10];
        int min, max;

        nums[0] = 99;
        nums[1] = -10;
        nums[2] = 100123;
        nums[3] = 18;
        nums[4] = -978;
        nums[5] = 5623;
        nums[6] = 463;
        nums[7] = -9;
        nums[8] = 287;
        nums[9] = 49;
        min = max = nums[0];
        for(int i=1; i < 10; i++) {
            if(nums[i] < min) min = nums[i];
            if(nums[i] > max) max = nums[i];
        }
        System.out.println("min and max: " + min + " " + max);
    }
}

```

Ниже приведен результат выполнения данной программы.

```
min and max: -978 100123
```

В приведенном выше примере программы массив `nums` заполняется вручную в десяти операторах присваивания. И хотя в этом нет ничего неверного, существует более простой способ решения той же самой задачи. Массивы можно инициализировать в процессе их создания. Для этой цели служит приведенная ниже общая форма инициализации массива.

```
тип имя_массива[] = {val1, val2, val3, ..., valN};
```

где `val1-valN` обозначают первоначальные значения, которые присваиваются элементам массива по очереди, слева направо и по порядку индексирования. Для хранения инициализаторов массива в Java автоматически распределяется достаточный объем памяти. А необходимость пользоваться оператором `new` явным образом отпадает сама собой. В качестве примера ниже приведена улучшенная версия программы, в которой определяются максимальное и минимальное значения в массиве.

```
// Применение инициализаторов массива.
class MinMax2 {
    public static void main(String args[]) {
        // Инициализаторы массива.
        int nums[] = { 99, -10, 100123, 18, -978,
                      5623, 463, -9, 287, 49 };
        int min, max;

        min = max = nums[0];
        for(int i=1; i < 10; i++) {
            if(nums[i] < min) min = nums[i];
            if(nums[i] > max) max = nums[i];
        }
        System.out.println("Min and max: " + min + " " + max);
    }
}
```

Границы массива в Java строго соблюдаются. Если границы массива не достигаются или же превышаются, при выполнении программы возникает ошибка. Для того чтобы убедиться в этом, попробуйте выполнить приведенную ниже программу, в которой намеренно превышаются границы массива.

```
// Демонстрация превышения границ массива.
class ArrayErr {
    public static void main(String args[]) {
        int sample[] = new int[10];
        int i;

        // воссоздать превышение границ массива
        for(i = 0; i < 100; i = i+1)
            sample[i] = i;
    }
}
```

Как только значение переменной `i` достигнет **10**, будет сгенерировано исключение `ArrayIndexOutOfBoundsException` и выполнение программы прекратится.

Пример для опробования 5.1.**Сортировка массива****Исходный файл:** `Bubble.java`

Как пояснялось выше, данные в одномерном массиве организованы в виде индексруемого линейного списка. Такая структура как нельзя лучше подходит для сортировки. В этом проекте предстоит реализовать простой алгоритм сортировки массива. Вам, вероятно, известно, что существуют разные алгоритмы сортировки, в том числе быстрая сортировка, сортировка перемешиванием, сортировка методом Шелла. Но самым простым и общеизвестным алгоритмом является пузырьковая сортировка. Этот алгоритм не очень эффективен, но отлично подходит для сортировки небольших массивов.

Последовательность действий

1. Создайте новый файл `Bubble.java`.
2. В алгоритме пузырьковой сортировки соседние элементы массива сравниваются и меняются, если требуется, местами. При этом малые значения сдвигаются к одному краю массива, а большие значения — к другому. Этот процесс напоминает всплывание пузырьков воздуха на разные уровни в емкости с жидкостью, откуда и произошло название данного алгоритма. Пузырьковая сортировка предполагает обработку массива в несколько проходов. Элементы, взаимное расположение которых отличается от требуемого, меняются местами. Число проходов должно быть таким, чтобы все элементы непременно встали на свои места. Максимальное количество проходов должно быть на один меньше, чем число элементов в массиве.

Ниже приведен исходный код, составляющий основу алгоритма пузырьковой сортировки. Сортируемый массив называется `nums`.

```
// Это пример реализации алгоритма пузырьковой сортировки.
for(a=1; a < size; a++)
    for(b=size-1; b >= a; b--) {
        if(nums[b-1] > nums[b]) { // если требуемый порядок следования
                                // не соблюдается, поменять элементы местами
            t = nums[b-1];
            nums[b-1] = nums[b];
            nums[b] = t;
        }
    }
}
```

Как видите, в приведенном выше фрагменте кода используются два цикла `for`. Во внутреннем цикле сравниваются соседние элементы массива и выявляются элементы, находящиеся не на своих местах. Если обнаружен элемент, положение которого отличается от требуемого, то два соседних элемента меняются местами. На каждом проходе наименьший элемент передвигается на один шаг в нужное положение. Внешний цикл обеспечивает повторение описанного выше процесса до тех пор, пока сортировка массива не будет завершена.

3. Ниже приведен весь исходный код программы из файла `Bubble.java`.

```
/*
    Пример для опробования 5.1.

    Демонстрация алгоритма пузырьковой сортировки.
*/
```

```

class Bubble {
    public static void main(String args[]) {
        int nums[] = { 99, -10, 100123, 18, -978,
                       5623, 463, -9, 287, 49 };

        int a, b, t;
        int size;

        size = 10; // Количество элементов для сортировки

        // отобразить исходный массив
        System.out.print("Original array is:");
        for(int i=0; i < size; i++)
            System.out.print(" " + nums[i]);
        System.out.println();

        // реализовать алгоритм пузырьковой сортировки
        for(a=1; a < size; a++)
            for(b=size-1; b >= a; b--) {
                if(nums[b-1] > nums[b]) { // если требуемый порядок
                    // следования не соблюдается, поменять элементы местами
                    t = nums[b-1];
                    nums[b-1] = nums[b];
                    nums[b] = t;
                }
            }

        // отобразить отсортированный массив
        System.out.print("Sorted array is:");
        for(int i=0; i < size; i++)
            System.out.print(" " + nums[i]);
        System.out.println();
    }
}

```

Ниже приведен результат выполнения данной программы.

```

Original array is: 99 -10 100123 18 -978 5623 463 -9 287 49
Sorted array is: -978 -10 -9 18 49 99 287 463 5623 100123

```

4. Как упоминалось выше, пузырьковая сортировка отлично подходит для обработки мелких массивов, но при большом числе элементов массива она становится неэффективной. Более универсальным является алгоритм быстрой сортировки, но для его эффективной реализации необходимы языковые средства Java, которые рассматриваются далее в этой книге.

Многомерные массивы

Несмотря на то что одномерные массивы употребляются чаще всего, в программировании, безусловно, применяются и многомерные (двух-, трехмерные и т.д.) массивы. В Java многомерные массивы представляют собой массивы массивов.

Двумерные массивы

Среди многомерных массивов наиболее простыми являются двумерные массивы. Двумерный массив, по существу, представляет собой ряд одномерных массивов. Для того чтобы объявить двумерный целочисленный табличный массив `table` размерами 10×20 , следует написать такое выражение:

```
int table[][] = new int[10][20];
```

Обратите особое внимание на объявление этого массива. В отличие от некоторых других языков программирования, где размеры массива разделяются запятыми, в Java они заключаются в отдельные квадратные скобки. Так, для обращения к элементу массива `table` по индексам 3 и 5 следует указать `table[3][5]`.

В следующем примере двумерный массив заполняется числами от 1 до 12:

// Демонстрация двумерного массива.

```
class TwoD {
    public static void main(String args[]) {
        int t, i;
        int table[][] = new int[3][4];

        for(t=0; t < 3; ++t) {
            for(i=0; i < 4; ++i) {
                table[t][i] = (t*4)+i+1;
                System.out.print(table[t][i] + " ");
            }
            System.out.println();
        }
    }
}
```

В данном примере элемент `table[0][0]` будет содержать значение 1, элемент `table[0][1]` — значение 2, элемент `table[0][2]` — значение 3 и так далее, а элемент `table[2][3]` — значение 12. Структура данного массива наглядно показана на рис. 5.1.

	0	1	2	3 ← правый индекс
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

↑ левый индекс

↑
table[1][2]

Рис. 5.1. Структура массива `table` из программы `TwoD`

Нерегулярные массивы

Выделяя память под многомерный массив, достаточно указать лишь первый (крайний слева) размер. А память под остальные размеры массива можно выделять по отдельности. Например, в приведенном ниже фрагменте кода память выделяется только

под первый размер двумерного массива `table`. А под второй его размер она выделяется вручную.

```
int table[][] = new int[3][];
table[0] = new int[4];
table[1] = new int[4];
table[2] = new int[4];
```

Объявляя массив подобным образом, мы не получаем никаких преимуществ, но в некоторых случаях такое объявление оказывается вполне оправданным. Это, в частности, дает возможность установить разную длину массива по каждому индексу. Как упоминалось выше, многомерный массив реализован в виде массива массивов, что позволяет контролировать длину каждого из них. Допустим, требуется написать программу, в процессе работы которой будет сохраняться число пассажиров, перевезенных автобусом-экспрессом в аэропорт. Если автобус-экспресс делает по десять рейсов в будние дни и по два рейса в субботу и воскресенье, то массив `riders` можно объявить так, как показано в приведенном ниже фрагменте кода. Обратите внимание на то, что длина массива по второму размеру для первых пяти элементов равна 10, а для двух последних элементов — 2.

// Указать разную длину по второму размеру массива вручную.

```
class Ragged {
    public static void main(String args[]) {
        int riders[][] = new int[7][];
        // Для первых пяти элементов длина массива
        // по второму размеру равна 10.
        riders[0] = new int[10];
        riders[1] = new int[10];
        riders[2] = new int[10];
        riders[3] = new int[10];
        riders[4] = new int[10];
        // Для остальных двух элементов длина массива
        // по второму размеру равна 2.
        riders[5] = new int[2];
        riders[6] = new int[2];

        int i, j;

        // сформировать произвольные данные
        for(i=0; i < 5; i++)
            for(j=0; j < 10; j++)
                riders[i][j] = i + j + 10;
        for(i=5; i < 7; i++)
            for(j=0; j < 2; j++)
                riders[i][j] = i + j + 10;

        System.out.println("Riders per trip during the week:");
        for(i=0; i < 5; i++) {
            for(j=0; j < 10; j++)
                System.out.print(riders[i][j] + " ");
            System.out.println();
        }
        System.out.println();
    }
}
```



```

System.out.println("Riders per trip on the weekend:");
for(i=5; i < 7; i++) {
    for(j=0; j < 2; j++)
        System.out.print(riders[i][j] + " ");
    System.out.println();
}
}
}

```

Для большинства приложений использовать нерегулярные массивы не рекомендуется, поскольку это затрудняет восприятие кода другими программистами. Но в некоторых случаях такие массивы вполне уместны и могут существенно повысить эффективность программ. Так, если вам требуется создать большой двумерный массив, в котором используются не все элементы, то нерегулярный массив позволит существенно сэкономить память.

Трехмерные, четырехмерные и многомерные массивы

В Java допускаются массивы размерностью больше двух. Ниже приведена общая форма объявления многомерного массива.

```
тип имя_массива[][]...[] = new тип[размер_1][размер_2]...[размер_N];
```

В качестве примера ниже приведено объявление трехмерного целочисленного массива размерами 4×10×3.

```
int multidim[][][] = new int[4][10][3];
```

Инициализация многомерных массивов

Многомерный массив можно инициализировать, заключая инициализирующую последовательность для каждого размера массива в отдельные фигурные скобки, как показано ниже.

```
тип имя_массива[][] = {
    { val, val, val, ..., val },
    { val, val, val, ..., val },
    .
    .
    .
    { val, val, val, ..., val }
};
```

где *val* обозначает начальное значение, которым инициализируются элементы многомерного массива. Каждый внутренний блок многомерного массива соответствует отдельной строке. В каждой строке первое значение сохраняется в первом элементе подмассива, второе значение — во втором элементе и т.д. Обратите внимание на запятые, разделяющие блоки инициализаторов многомерного массива, а также на точку с запятой после закрывающей фигурной скобки.

В следующем фрагменте кода двумерный массив *sqr*s инициализируется числами от 1 до 10 и их квадратами:

```
// Инициализация двумерного массива.
class Squares {
    public static void main(String args[]) {
```

```
// Обратите внимание на то, что у каждой строки свой ряд инициализаторов.
int sqrs[][] = {
    { 1, 1 },
    { 2, 4 },
    { 3, 9 },
    { 4, 16 },
    { 5, 25 },
    { 6, 36 },
    { 7, 49 },
    { 8, 64 },
    { 9, 81 },
    { 10, 100 }
};
int i, j;

for(i=0; i < 10; i++) {
    for(j=0; j < 2; j++)
        System.out.print(sqrs[i][j] + " ");
    System.out.println();
}
}
```

Выполнение этого фрагмента кода дает следующий результат:

```
1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
10 100
```

Альтернативный синтаксис объявления массивов

Помимо рассмотренной выше общей формы, для объявления массива можно также пользоваться следующей синтаксической конструкцией:

тип[] имя_переменной;

Здесь квадратные скобки указываются после спецификатора типа, а не имени переменной. Поэтому следующие два объявления равнозначны:

```
int counter[] = new int[3];
int[] counter = new int[3];
```

Равнозначными являются и приведенные ниже строки кода.

```
char table[][] = new char[3][4];
char[][] table = new char[3][4];
```

Альтернативный синтаксис объявления массива оказывается удобным в тех случаях, если требуется объявить несколько массивов одного типа. Например:

```
int[] nums, nums2, nums3; // создать три массива
```

В этом объявлении создаются три переменные, ссылающиеся на массивы типа `int`. Тот же результат можно получить с помощью следующей строки кода:

```
int nums[], nums2[], nums3[]; // создать три массива
```

Альтернативный синтаксис объявления массива оказывается удобным и в тех случаях, если в качестве типа, возвращаемого методом, требуется указать массив. Например:

```
int[] someMeth( ) { ...
```

В этой строке кода объявляется метод `someMeth()`, возвращающий целочисленный массив.

Обе рассмотренные выше формы объявления массивов широко распространены в программировании на Java, и поэтому они употребляются в примерах, представленных в этой книге.

Присваивание ссылок на массивы

Присваивание значения одной переменной ссылке на массив другой переменной, по существу, означает, что обе переменные ссылаются на один и тот же массив, и в этом отношении массивы ничем не отличаются от любых других объектов. Такое присваивание не приводит ни к созданию копии массива, ни к копированию содержимого одного массива в другой. В качестве примера рассмотрим следующую программу:

```
// Присваивание ссылок на массивы.
class AssignARef {
    public static void main(String args[]) {
        int i;

        int nums1[] = new int[10];
        int nums2[] = new int[10];

        for(i=0; i < 10; i++)
            nums1[i] = i;

        for(i=0; i < 10; i++)
            nums2[i] = -i;

        System.out.print("Here is nums1: ");
        for(i=0; i < 10; i++)
            System.out.print(nums1[i] + " ");
        System.out.println();

        System.out.print("Here is nums2: ");
        for(i=0; i < 10; i++)
            System.out.print(nums2[i] + " ");
        System.out.println();

        // присвоить ссылку на массив
        // Теперь переменные nums2 и nums1 ссылаются
        // на один и тот же массив.
        nums2 = nums1;

        System.out.print("Here is nums2 after assignment: ");
        for(i=0; i < 10; i++)
```

```

        System.out.print(nums2[i] + " ");
        System.out.println();

        // выполнить операции над массивом nums1
        // по ссылке на массив nums2.
        nums2[3] = 99;

        System.out.print("Here is nums1 after change through nums2: ");
        for(i=0; i < 10; i++)
            System.out.print(nums1[i] + " ");
        System.out.println();
    }
}

```

Выполнение этой программы дает следующий результат:

```

Here is nums1: 0 1 2 3 4 5 6 7 8 9
Here is nums2: 0 -1 -2 -3 -4 -5 -6 -7 -8 -9
Here is nums2 after assignment: 0 1 2 3 4 5 6 7 8 9
Here is nums1 after change through nums2: 0 1 2 99 4 5 6 7 8 9

```

Нетрудно заметить, что в результате присваивания ссылки на массив `nums1` переменной `nums2` обе переменные ссылаются на один и тот же массив.

Применение переменной экземпляра `length`

В связи с тем что массивы реализованы в виде объектов, в каждом массиве содержится переменная экземпляра `length`. Значением этой переменной является число элементов, которые может содержать массив. (Иными словами, в переменной `length` содержится размер массива.) Ниже приведен пример программы, демонстрирующий данное свойство массивов.

```

// Использование переменной экземпляра length.
class LengthDemo {
    public static void main(String args[]) {
        int list[] = new int[10];
        int nums[] = { 1, 2, 3 };
        int table[][] = { // таблица со строками переменной длины
            {1, 2, 3},
            {4, 5},
            {6, 7, 8, 9}
        };

        System.out.println("length of list is " + list.length);
        System.out.println("length of nums is " + nums.length);
        System.out.println("length of table is " + table.length);
        System.out.println("length of table[0] is " + table[0].length);
        System.out.println("length of table[1] is " + table[1].length);
        System.out.println("length of table[2] is " + table[2].length);
        System.out.println();

        // использовать переменную length для инициализации списка
        // Переменная length служит для управления циклом for.
        for(int i=0; i < list.length; i++)
            list[i] = i * i;
    }
}

```

```

        System.out.print("Here is list: ");
        for(int i=0; i < list.length; i++)
            System.out.print(list[i] + " ");
        System.out.println();
    }
}

```

Выполнение этой программы дает следующий результат:

```

length of list is 10
length of nums is 3
length of table is 3
length of table[0] is 3
length of table[1] is 2
length of table[2] is 4

```

```

Here is list: 0 1 4 9 16 25 36 49 64 81

```

Обратите особое внимание, каким образом переменная `length` используется в двумерном массиве. Как пояснялось ранее, двумерный массив представляет собой массив массивов. Поэтому приведенное ниже выражение позволяет определить число массивов, содержащихся в массиве `table`.

```

table.length

```

Число таких массивов равно 3. Для того чтобы получить длину отдельного массива, содержащегося в массиве `table`, потребуется выражение, аналогичное следующему:

```

table[0].length

```

Это выражение возвращает длину первого массива.

Анализируя код класса `LengthDemo`, следует также заметить, что выражение `list.length` используется в цикле `for` для определения требуемого количества итераций. Учитывая то, что у каждого подмассива своя длина, пользоваться таким выражением удобнее, чем отслеживать вручную размеры массивов. Но не следует забывать, что переменная `length` не имеет никакого отношения к количеству фактически используемых элементов массива. Она содержит лишь данные о том, сколько элементов может содержать массив.

Использование переменной экземпляра `length` позволяет упростить многие алгоритмы. Так, в приведенном ниже примере программы эта переменная используется при копировании одного массива в другой и предотвращает возникновение при выполнении программы исключительной ситуации в связи с превышением границ массива.

```

// Использование переменной length для копирования массивов.

```

```

class ACopy {
    public static void main(String args[]) {
        int i;
        int nums1[] = new int[10];
        int nums2[] = new int[10];

        for(i=0; i < nums1.length; i++)
            nums1[i] = i;

        // копировать массив nums1 в массив nums2
        // Переменная length служит для сравнения размеров массивов.
        if(nums2.length >= nums1.length)

```

```

    for(i = 0; i < nums2.length; i++)
        nums2[i] = nums1[i];

    for(i=0; i < nums2.length; i++)
        System.out.print(nums2[i] + " ");
}
}

```

В данном примере переменная экземпляра `length` помогает решить две важные задачи. Во-первых, позволяет убедиться в том, что размера целевого массива достаточно для хранения содержимого исходного массива. И во-вторых, с ее помощью формируется условие завершения цикла, в котором выполняется копирование массива. Конечно, в столь простом примере размеры массивов нетрудно отследить и без переменной экземпляра `length`, но подобный подход может быть применен для решения более сложных задач.

Пример для опробования 5.2.

Создание класса очереди

Исходный файл: `QDemo.java`

Вам, вероятно, известно, что структура данных — это способ их организации. Одной из самых простых структур является массив, который представляет собой линейный список элементов, допускающий произвольный доступ к ним. Нередко массивы используются в качестве основания для создания более сложных структур вроде стеков или очередей. *Стек* — это набор элементов с организацией доступа по принципу “первым пришел — последним обслужен”, а *очередь* — это набор элементов с организацией доступа по принципу “первым пришел — первым обслужен”. Стек можно сравнить со стопкой тарелок на столе: первая тарелка снизу стопки используется последней. А очередь можно сравнить с выстроившейся очередью к окошку в банке: клиент, стоящий в очереди первым, обслуживается первым.

В очередях и стеках нас больше всего интересует способ хранения информации и обращения к ней. И стеки, и очереди представляют собой механизмы доступа к данным, в которых хранение и извлечение информации поддерживается самой структурой, а не реализуется в программе. Такое сочетание способов хранения и обработки данных лучше всего реализуется в рамках класса, поэтому в данном проекте предстоит создать простой класс очереди.

В очереди поддерживаются две основные операции: размещение и извлечение. При выполнении операции размещения новый элемент помещается в конец очереди, а при операции извлечения очередной элемент извлекается из начала очереди. Операции с очередью являются *истошающими*: элемент, однажды извлеченный из очереди, не может быть извлечен из нее повторно. Очередь может быть переполнена, когда в ней не остается места для новых элементов. Но очередь может быть и пуста, когда в ней нет ни одного элемента.

И последнее замечание: существуют два типа очередей — *циклические* и *нециклические*. В циклической очереди элементы массива, на основе которого она создана, могут использоваться повторно по мере удаления данных. Нециклическая очередь не позволяет повторно использовать элементы, поэтому со временем пространство для хранения новых элементов исчерпывается. Нециклическую очередь создать намного проще, чем циклическую, поэтому именно ее мы и реализуем в данном примере для опробования. Но если немного подумать, то нециклическую очередь можно без особого труда превратить в циклическую.

Последовательность действий

1. Создайте новый файл `QDemo.java`.
2. Очередь можно организовать разными способами. Мы создадим ее на основании массива, выступающего в роли хранилища данных, помещаемых в очередь. Для доступа к массиву будут использованы два индекса. *Индекс размещения* данных определяет, в каком месте будет размещен следующий элемент очереди. А *индекс извлечения* данных указывает место, откуда должен быть извлечен очередной элемент очереди. Напомним, что операция извлечения является истощающей и не позволяет извлечь один и тот же элемент дважды. Создаваемая здесь очередь предназначена для хранения символов, но та же самая логика ее организации может быть использована для размещения данных любых типов, в том числе объектов. Итак, начните создание класса очереди `Queue` со следующих строк кода:

```
class Queue {
    char q[]; // Массив для хранения элементов очереди
    int putloc, getloc; // Индексы размещения и извлечения элементов
    очереди
}
```

3. Конструктор класса `Queue` создает очередь заданного размера. Его код приведен ниже.

```
Queue(int size) {
    q = new char[size+1]; // выделить память для очереди
    putloc = getloc = 0;
}
```

Обратите внимание на то, что размер очереди оказывается на единицу больше размера, задаваемого параметром `size`. Особенности реализации очереди таковы, что один элемент массива остается неиспользованным, поэтому размер массива должен быть на единицу больше размера очереди, создаваемой на его основе. Первоначально индексы размещения и извлечения данных равны нулю.

4. Метод `put()`, помещающий элемент в очередь, имеет следующий вид:

```
// поместить символ в очередь
void put(char ch) {
    if(putloc==q.length-1) {
        System.out.println(" - Queue is full.");
        return;
    }

    putloc++;
    q[putloc] = ch;
}
```

Прежде всего в теле данного метода проверяется, не переполнена ли очередь. Если значение переменной `putloc` соответствует последнему местоположению в массиве `q`, то места для размещения новых элементов в очереди нет. В противном случае переменная `putloc` инкрементируется и новый элемент располагается в указанном месте массива. Следовательно, переменная `putloc` всегда содержит индекс элемента, помещенного в очередь последним.

5. Для извлечения элементов из очереди служит метод `get()`, код которого приведен ниже.

```
// извлечь символ из очереди
char get() {
    if(getloc == putloc) {
        System.out.println(" - Queue is empty.");
        return (char) 0;
    }

    getloc++;
    return q[getloc];
}
```

Сначала в данном методе проверяется, пуста ли очередь. Если значения индексов в переменных `getloc` и `putloc` совпадают, то в очереди нет ни одного элемента. Именно поэтому в конструкторе `Queue` переменные `getloc` и `putloc` инициализируются нулевыми значениями. Если очередь не пуста, то переменная `getloc` инкрементируется и из нее извлекается очередной элемент. Следовательно, переменная `getloc` содержит индекс последнего извлеченного элемента.

6. Ниже приведен весь исходный код программы из файла `QDemo.java`.

```
/*
    Пример для опробования 5.2.

    Класс, реализующий очередь, для хранения символов.
*/

class Queue {
    char q[];           // Массив для хранения элементов очереди
    int putloc, getloc; // Индексы размещения и извлечения элементов
                        // очереди

    Queue(int size) {
        q = new char[size+1]; // выделить память для очереди
        putloc = getloc = 0;
    }

    // поместить символ в очередь
    void put(char ch) {
        if(putloc==q.length-1) {
            System.out.println(" - Queue is full.");
            return;
        }

        putloc++;
        q[putloc] = ch;
    }

    // извлечь символ из очереди
    char get() {
        if(getloc == putloc) {
            System.out.println(" - Queue is empty.");
            return (char) 0;
        }
    }
}
```



```

        getloc++;
        return q[getloc];
    }
}

// продемонстрировать класс Queue
class QDemo {
    public static void main(String args[]) {
        Queue bigQ = new Queue(100);
        Queue smallQ = new Queue(4);
        char ch;
        int i;

        System.out.println("Using bigQ to store the alphabet.");
        // поместить буквенные символы в очередь bigQ
        for(i=0; i < 26; i++)
            bigQ.put((char) ('A' + i));

        // извлечь буквенные символы из очереди bigQ и отобразить
        System.out.print("Contents of bigQ: ");
        for(i=0; i < 26; i++) {
            ch = bigQ.get();
            if(ch != (char) 0) System.out.print(ch);
        }

        System.out.println("\n");

        System.out.println("Using smallQ to generate errors.");
        // использовать небольшую очередь smallQ для генерации ошибок
        for(i=0; i < 5; i++) {
            System.out.print("Attempting to store " +
                            (char) ('Z' - i));

            smallQ.put((char) ('Z' - i));

            System.out.println();
        }
        System.out.println();

        // дополнительные ошибки при обращении к очереди smallQ
        System.out.print("Contents of smallQ: ");
        for(i=0; i < 5; i++) {
            ch = smallQ.get();

            if(ch != (char) 0) System.out.print(ch);
        }
    }
}

```

7. Ниже приведен результат выполнения данной программы.

```

Using bigQ to store the alphabet.
Contents of bigQ: ABCDEFGHIJKLMNOPQRSTUVWXYZ

```

```
Using smallQ to generate errors.
Attempting to store Z
Attempting to store Y
Attempting to store X
Attempting to store W
Attempting to store V - Queue is full.
```

Contents of smallQ: ZYXW - Queue is empty.

8. Попробуйте самостоятельно усовершенствовать класс Queue таким образом, чтобы в очереди можно было хранить другие типы данных, например значения типа `int` или `double`.

Разновидность `for-each` цикла `for`

При выполнении операций с массивами очень часто возникают ситуации, когда должен быть обработан каждый элемент массива. Например, для расчета суммы всех значений, содержащихся в массиве, нужно обратиться ко всем его элементам. То же самое приходится делать при расчете среднего значения, поиске элемента и решении многих других задач. В связи с тем что задачи, предполагающие обработку всего массива, встречаются очень часто, в Java была реализована еще одна разновидность цикла `for`, рационализирующая подобные операции с массивами.

Вторая разновидность оператора `for` реализует цикл типа `for-each`. В этом цикле происходит последовательное обращение к каждому элементу совокупности объектов (например, массива). За последние годы циклы `for-each` появились практически во всех языках программирования. Изначально в Java подобный цикл не был предусмотрен и был реализован лишь в пакете JDK 5. Разновидность `for-each` цикла `for` называется также *расширенным циклом* `for`. В данной книге употребляются оба эти термина.

Ниже приведена общая форма разновидности `for-each` цикла `for`.

```
for(тип intr_var : коллекция) блок_операторов
```

где *тип* обозначает конкретный тип `intr_var` — *итерационной переменной*, в которой сохраняются перебираемые по очереди элементы набора данных, обозначенного как *коллекция*. В данной разновидности цикла `for` могут быть использованы разные типы коллекций, но в этой книге рассматриваются только массивы. На каждом шаге цикла очередной элемент извлекается из коллекции и сохраняется в итерационной переменной. Выполнение цикла продолжается до тех пор, пока не будут получены все элементы коллекции. Таким образом, при обработке массива размером N в расширенном цикле `for` будут последовательно извлечены элементы с индексами от 0 до $N-1$.

Итерационная переменная получает значения из коллекции, и поэтому ее *тип* должен совпадать (или, по крайней мере, быть совместимым) с типом элементов, которые содержит *коллекция*. В частности, при обработке массива тип итерационной переменной должен совпадать с типом массива.

Для того чтобы стали понятнее причины, побудившие к внедрению разновидности `for-each` цикла `for` в Java, рассмотрим приведенный ниже фрагмент кода, в котором традиционный цикл `for` используется для вычисления суммы значений элементов массива.

```
int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int sum = 0;

for(int i=0; i < 10; i++) sum += nums[i];
```

Для того чтобы вычислить упомянутую выше сумму, придется перебрать все элементы массива `nums` от начала до конца. Перебор элементов осуществляется благодаря использованию переменной цикла `i` в качестве индекса массива `nums`. Кроме того, нужно явно указать начальное значение переменной цикла, шаг приращения и условие завершения цикла.

При использовании разновидности `for-each` данного цикла некоторые перечисленные выше действия выполняются автоматически. В частности, отпадает необходимость в использовании переменной цикла, задании ее исходного значения и условия завершения цикла, а также в индексировании массива. Вместо этого массив автоматически обрабатывается в цикле от начала до конца. Код, позволяющий решить ту же самую задачу с помощью разновидности `for-each` цикла `for`, выглядит следующим образом:

```
int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
int sum = 0;

for(int x: nums) sum += x;
```

На каждом шаге этого цикла переменная `x` автоматически принимает значение, равное очередному элементу массива `nums`. Сначала ее значение равно 1, на втором шаге цикла итерации оно становится равным 2 и т.д. В данном случае не только упрощается синтаксис, но и исключается ошибка, связанная с превышением границ массива.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Какие типы коллекций, помимо массивов, можно обрабатывать с помощью разновидности `for-each` цикла `for`?

ОТВЕТ. Наиболее важное применение разновидность `for-each` цикла `for` находит в обработке содержимого коллекции, определенной в Collections Framework — библиотеке классов, реализующих различные структуры данных, в том числе списки, векторы, множества и отображения. Рассмотрение библиотеки Collections Framework выходит за рамки этой книги, а дополнительные сведения о ней можно найти в книге *Java: The Complete Reference, 8th Edition* (издательство McGraw-Hill, 2011 г.). (Русский перевод: *Java. Полное руководство*, 8-е издание, ИД “Вильямс”, 2012 г.).

Ниже приведен весь исходный код программы, демонстрирующей решение описанной выше задачи с помощью разновидности `for-each` цикла `for`.

```
// Использование разновидности for-each цикла for.
class ForEach {
    public static void main(String args[]) {
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
        int sum = 0;

        // использовать разновидность for-each цикла for
        // для суммирования и отображения значений.
        for(int x : nums) {
            System.out.println("Value is: " + x);
            sum += x;
        }

        System.out.println("Summation: " + sum);
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
Value is: 1
Value is: 2
Value is: 3
Value is: 4
Value is: 5
Value is: 6
Value is: 7
Value is: 8
Value is: 9
Value is: 10
Summation: 55
```

Нетрудно заметить, что в данной разновидности `for-each` цикла `for` элементы массива автоматически извлекаются один за другим в порядке возрастания индекса.

Несмотря на то что в разновидности `for-each` цикла `for` обрабатываются все элементы массива, этот цикл можно завершить преждевременно, используя оператор `break`. Так, в цикле, используемом в следующем примере, вычисляется сумма только пяти элементов массива `nums`:

```
// Суммирование первых 5 элементов массива.
for(int x : nums) {
    System.out.println("Value is: " + x);
    sum += x;
    if(x == 5) break; // прервать цикл по достижении значения 5
}
```

Следует, однако, иметь в виду одну важную особенность разновидности `for-each` цикла `for`. Итерационная переменная в этом цикле обеспечивает только чтение элементов массива, но ее нельзя использовать для записи значения в какой-либо элемент массива. Иными словами, изменить содержимое массива, присвоив итерационной переменной новое значение, не удастся. Рассмотрим в качестве примера следующую программу:

```
// Цикл for-each, предназначенный только для чтения.
class NoChange {
    public static void main(String args[]) {
        int nums[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

        for(int x : nums) {
            System.out.print(x + " ");
            // Следующая операция не оказывает никакого влияния
            // на содержимое массива nums.
            x = x * 10;
        }

        System.out.println();

        for(int x : nums)
            System.out.print(x + " ");

        System.out.println();
    }
}
```

В первом цикле `for` значение итерационной переменной увеличивается на 10, но это не оказывает никакого влияния на содержимое массива `nums`, что и демонстрирует второй цикл `for`. Это же подтверждает и результат выполнения программы.

```
1 2 3 4 5 6 7 8 9 10
1 2 3 4 5 6 7 8 9 10
```

Циклическое обращение к многомерным массивам

Расширенный цикл `for` можно применять и при обращении к многомерным массивам. Как вам должно быть уже известно, в Java многомерный массив представляет собой *массив массивов*. (Например, двумерный массив — это массив, элементами которого являются одномерные массивы.) Эту особенность важно помнить, организуя циклическое обращение к многомерным массивам, поскольку на каждом шаге цикла извлекается *очередной* массив, а не отдельный элемент. Более того, итерационная переменная в расширенном цикле `for` должна иметь тип, совместимый с типом извлекаемого массива. Так, при обращении к двумерному массиву итерационная переменная должна представлять собой ссылку на одномерный массив. При использовании разновидности `for-each` цикла `for` для обработки N -мерного массива извлекаемый объект представляет собой $(N-1)$ -мерный массив. Для того чтобы сказанное стало более понятным, рассмотрим приведенный ниже пример программы, где для извлечения элементов двумерного массива используются вложенные циклы `for`. Обратите внимание на то, каким образом объявляется переменная `x`.

```
// Использование разновидности for-each цикла for
// для обработки двумерного массива.
class ForEach2 {
    public static void main(String args[]) {
        int sum = 0;
        int nums[][] = new int[3][5];

        // ввести ряд значений в массив nums
        for(int i = 0; i < 3; i++)
            for(int j=0; j < 5; j++)
                nums[i][j] = (i+1)*(j+1);

        // использовать разновидность for-each цикла for
        // для суммирования и отображения значений
        // Обратите внимание на объявление переменной x.
        for(int x[] : nums) {
            for(int y : x) {
                System.out.println("Value is: " + y);
                sum += y;
            }
        }
        System.out.println("Summation: " + sum);
    }
}
```

Выполнение этой программы дает следующий результат:

```
Value is: 1
Value is: 2
Value is: 3
```

```

Value is: 4
Value is: 5
Value is: 2
Value is: 4
Value is: 6
Value is: 8
Value is: 10
Value is: 3
Value is: 6
Value is: 9
Value is: 12
Value is: 15
Summation: 90

```

Обратите особое внимание на следующую строку кода:

```
for(int x[] : nums) {
```

Не упустите из виду и то, как объявляется переменная *x*. Она представляет собой ссылку на одномерный целочисленный массив. Это очень важно, поскольку на каждом шаге цикла *for* из двумерного массива *nums* извлекается *очередной* массив, начиная с *nums[0]*. А во внутреннем цикле *for* перебираются элементы полученного массива и отображаются их значения.

Применение расширенного цикла **for**

Разновидность *for-each* цикла *for* обеспечивает лишь последовательный перебор элементов от начала до конца массива, поэтому может создаться впечатление, будто такой цикл имеет ограниченное применение. Но это совсем не так. Данный механизм циклического обращения применяется в самых разных алгоритмах. Один из самых характерных тому примеров — организация поиска. В приведенном ниже примере программы расширенный цикл *for* используется для поиска значения в неотсортированном массиве. Выполнение цикла прерывается, если искомый элемент найден.

// Поиск в массиве с использованием разновидности **for-each** цикла **for**.

```

class Search {
    public static void main(String args[]) {
        int nums[] = { 6, 8, 3, 7, 5, 6, 1, 4 };
        int val = 5;
        boolean found = false;

        // использовать разновидность for-each цикла for
        // для поиска значения переменной val в массиве nums
        for(int x : nums) {
            if(x == val) {
                found = true;
                break;
            }
        }

        if(found)
            System.out.println("Value found!");
    }
}

```

В данном случае применение расширенного цикла `for` вполне оправданно, поскольку найти значение в неотсортированном массиве можно лишь, перебрав все его элементы. (Если бы содержимое массива было предварительно отсортировано, то лучше было бы применить более эффективный алгоритм поиска, например поиск методом дихотомии. В этом случае пришлось бы использовать другой массив.) Расширенным циклом `for` удобно также пользоваться для расчета среднего значения, нахождения минимального и максимального элементов множества, выявления дублирующихся значений и т.д.

Теперь, когда разновидность `for-each` цикла `for` представлена в достаточной степени, она будет еще не раз использоваться там, где это уместно, в примерах программ, представленных в остальной части книги.

Символьные строки

В повседневной работе каждый программист обязательно встречается с объектами типа `String`. Объект типа `String` определяет символьную строку и поддерживает операции над ней. Во многих языках программирования символьная строка — это лишь массив символов, но в Java это совсем не так, где символьная строка, по существу, является объектом.

Возможно, вы и не заметили, но класс `String` фактически уже использовался в примерах программ, начиная с главы 1. При создании строкового литерала на самом деле строился объект типа `String`. Рассмотрим приведенный ниже оператор.

```
System.out.println("In Java, strings are objects.");
```

Наличие в нем символьной строки `"In Java, strings are objects."` (В Java строки являются объектами) автоматически приводит к созданию объекта типа `String`. Таким образом, класс `String` незримо присутствовал в предыдущих примерах программ. В последующих разделах будет показано, как этим классом пользоваться явным образом. Однако в классе `String` предусмотрен обширный ряд методов, поэтому здесь будут рассмотрены лишь некоторые из них. Большую часть функциональных возможностей класса `String` вам предстоит изучить самостоятельно.

Построение строк

Объекты типа `String` создаются таким же образом, как и объекты других типов. Для этой цели используется конструктор, как показано в следующем примере:

```
String str = new String("Hello");
```

В данном примере создается объект `str` типа `String`, содержащий символьную строку `"Hello"` (Привет). Объект типа `String` можно создать и на основе другого объекта такого же типа. Например:

```
String str = new String("Hello");
String str2 = new String(str);
```

После выполнения этих строк кода объект `str2` будет также содержать символьную строку `"Hello"`.

Ниже представлен еще один способ создания объекта типа `String`.

```
String str = "Java strings are powerful.";
```

В данном случае объект `str` инициализируется последовательностью символов `"Java strings are powerful."` (Строки в Java эффективны).

Создав объект типа `String`, можете использовать его везде, где допускается строковый литерал (последовательность символов, заключенная в кавычки). Например, объект типа `String` можно передать в качестве параметра методу `println()` при его вызове, как показано в приведенном ниже примере программы.

```
// Представление класса String.
class StringDemo {
    public static void main(String args[]) {
        // объявить символьные строки разными способами
        String str1 = new String("Java strings are objects.");
        String str2 = "They are constructed various ways.";
        String str3 = new String(str2);

        System.out.println(str1);
        System.out.println(str2);
        System.out.println(str3);
    }
}
```

Выполнение этой программы дает следующий результат:

```
Java strings are objects.
They are constructed various ways.
They are constructed various ways.
```

Операции над символьными строками

Класс `String` содержит ряд методов, предназначенных для выполнения действий над символьными строками. Ниже описаны некоторые из них.

<code>boolean equals(str)</code>	Возвращает логическое значение <code>true</code> , если текущая строка содержит ту же последовательность символов, что и параметр <code>str</code>
<code>int length()</code>	Возвращает длину символьной строки
<code>char charAt(index)</code>	Возвращает символ, находящийся в строке на позиции, указываемой параметром <code>index</code>
<code>int compareTo(String str)</code>	Возвращает отрицательное значение, если текущая строка меньше строки <code>str</code> , нуль, если эти строки равны, и положительное значение, если текущая строка больше строки <code>str</code>
<code>int indexOf(String str)</code>	Производит в текущей строке поиск подстроки, определяемой параметром <code>str</code> . Возвращает индекс первого вхождения подстроки <code>str</code> или <code>-1</code> , если поиск завершается неудачно
<code>int lastIndexOf(String str)</code>	Производит в текущей строке поиск подстроки, определяемой параметром <code>str</code> . Возвращает индекс последнего вхождения подстроки <code>str</code> или <code>-1</code> , если поиск завершается неудачно

В приведенном ниже примере программы демонстрируется применение перечисленных выше методов, оперирующих символьными строками.

```
// Некоторые операции над символьными строками.
class StrOps {
    public static void main(String args[]) {
        String str1 =
```



```

        "When it comes to Web programming, Java is #1.";
String str2 = new String(str1);
String str3 = "Java strings are powerful.";
int result, idx;
char ch;

System.out.println("Length of str1: " +
                    str1.length());

// отобразить строку str1 посимвольно.
for(int i=0; i < str1.length(); i++)
    System.out.print(str1.charAt(i));
System.out.println();

if(str1.equals(str2))
    System.out.println("str1 equals str2");
else
    System.out.println("str1 does not equal str2");

if(str1.equals(str3))
    System.out.println("str1 equals str3");
else
    System.out.println("str1 does not equal str3");

result = str1.compareTo(str3);
if(result == 0)
    System.out.println("str1 and str3 are equal");
else if(result < 0)
    System.out.println("str1 is less than str3");
else
    System.out.println("str1 is greater than str3");

// присвоить переменной str2 новую строку
str2 = "One Two Three One";

idx = str2.indexOf("One");
System.out.println("Index of first occurrence of One: " + idx);
idx = str2.lastIndexOf("One");
System.out.println("Index of last occurrence of One: " + idx);
}
}

```

Выполнение этой программы дает следующий результат:

```

Length of str1: 45
When it comes to Web programming, Java is #1.
str1 equals str2
str1 does not equal str3
str1 is greater than str3
Index of first occurrence of One: 0
Index of last occurrence of One: 14

```

Сцепление (или объединение) двух символьных строк обеспечивает оператор `+`. Например, в приведенной ниже последовательности операторов присваивания переменная `str4` инициализируется символьной строкой `"OneTwoThree"`.

```
String str1 = "One";
String str2 = "Two";
String str3 = "Three";
String str4 = str1 + str2 + str3;
```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Зачем в классе `String` определен метод `equals()`? Не проще ли использовать вместо него оператор `==`?

ОТВЕТ. Метод `equals()` сравнивает последовательности символов, содержащиеся в двух объектах типа `String`, и проверяет, совпадают ли они. А при сравнении двух ссылок на объекты типа `String` в операторе `==` лишь определяет, делаются ли обе ссылки на один и тот же объект.

Массивы символьных строк

Подобно другим типам данных, символьные строки можно объединять в массивы. Ниже приведен характерный тому пример.

```
// Использование массивов символьных строк.
class StringArrays {
    public static void main(String args[]) {
        // Массив символьных строк.
        String strs[] = { "This", "is", "a", "test." };

        System.out.println("Original array: ");
        for(String s : strs)
            System.out.print(s + " ");
        System.out.println("\n");

        // изменить символьную строку
        strs[1] = "was";
        strs[3] = "test, too!";

        System.out.println("Modified array: ");
        for(String s : strs)
            System.out.print(s + " ");
    }
}
```

Выполнение этого фрагмента кода дает следующий результат:

```
Original array:
This is a test.
```

```
Modified array:
This was a test, too!
```

Постоянство строк

Как ни странно, содержимое объекта типа `String` не подлежит изменению. Это означает, что однажды созданную последовательность символов изменить нельзя. Но данное ограничение способствует более эффективной реализации символьных строк. Поэтому этот, на первый взгляд, очевидный недостаток на самом деле превращается в преимущество. Так, если требуется строка в качестве разновидности уже имеющейся строки, для этой цели следует создать новую строку, содержащую все необходимые изменения. А поскольку неиспользуемые строковые объекты автоматически собираются в “мусор”, то о дальнейшей судьбе ненужных строк можно даже не беспокоиться. Следует, однако, иметь в виду, что переменные ссылки на строки (т.е. объекты типа `String`) подлежат изменению, а следовательно, могут ссылаться на другой объект. Но содержимое самого объекта типа `String` не меняется после его создания.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как пояснялось выше, однажды созданный объект типа `String` остается неизменяемым. С практической точки зрения это не является серьезным ограничением, но что если мне нужно создать символьную строку, которая может изменяться?

ОТВЕТ. Можете считать, что вам повезло. В Java предоставляется класс `StringBuffer`, который создает символьные строки, способные изменяться. Так, в дополнение к методу `charAt()`, возвращающему символ из указанного места в строке, в классе `StringBuffer` определен метод `setCharAt()`, вставляющий символ в строку. Но для большинства целей вполне подходит и класс `String`, а в классе `StringBuffer` особой необходимости не возникает.

Для того чтобы стало понятнее, почему постоянство строк не является помехой, воспользуемся еще одним методом обращения со строками из класса `String`. Это метод `substring()`, возвращающий новую строку, содержащую часть вызывающей строки. В итоге создается новый строковый объект, содержащий выбранную подстроку, тогда как исходная строка не меняется, а следовательно, соблюдается принцип постоянства строк. Ниже приведена рассматриваемая здесь общая форма объявления метода `substring()`.

```
String substring(int начальный_индекс, int конечный_индекс)
```

где *начальный_индекс* обозначает начало извлекаемой подстроки, а *конечный_индекс* — ее окончание. Ниже приведен пример программы, демонстрирующий применение метода `substring()` и принцип постоянства строк.

```
// Применение метода substring().
class SubStr {
    public static void main(String args[]) {
        String orgstr = "Java makes the Web move.";

        // сформировать подстроку
        // Здесь создается новая строка, содержащая нужную подстроку.
        String substr = orgstr.substring(5, 18);

        System.out.println("orgstr: " + orgstr);
```

```

        System.out.println("substr: " + substr);
    }
}

```

Результат выполнения данной программы выглядит следующим образом:

```

orgstr: Java makes the Web move.
substr: makes the Web

```

Как видите, исходная строка `orgstr` остается без изменения, а новая строка `substr` содержит сформированную подстроку.

Применение строк для управления оператором `switch`

Как пояснялось в главе 3, до появления версии JDK 7 для управления оператором `switch` приходилось пользоваться константами типа `int` или `char`. Это препятствовало применению оператора `switch` в тех случаях, когда одно из выполняемых действий выбирается из содержимого символьной строки. В качестве выхода из этого положения зачастую приходилось обращаться к многоступенчатой конструкции `if-else-if`. И хотя эта конструкция семантически правильна, для организации подобного выбора более естественным было бы применение оператора `switch`. К счастью, этот недостаток был исправлен. После выпуска комплекта JDK 7 появилась возможность пользоваться объектами типа `String` для управления оператором `switch`. И во многих случаях это способствует написанию более удобочитаемого и рационально организованного кода.

Ниже приведен краткий пример программы, демонстрирующий управление оператором `switch` с помощью объектов типа `String`.

```

// Использование символьной строки для управления оператором switch.
class StringSwitch {
    public static void main(String args[]) {

        String command = "cancel";

        switch(command) {
            case "connect":
                System.out.println("Connecting");
                break;
            case "cancel":
                System.out.println("Canceling");
                break;
            case "disconnect":
                System.out.println("Disconnecting");
                break;
            default:
                System.out.println("Command Error!");
                break;
        }
    }
}

```

Как и следовало ожидать, выполнение этой программы приводит к следующему результату:

```

Canceling

```

Символьная строка, содержащаяся в переменной `command`, а в данном примере это строка `"cancel"` (отмена), проверяется на совпадение с символьными константами

в ветвях `case` оператора `switch`. Если совпадение обнаружено, как это имеет место во второй ветви `case`, выполняется кодовая последовательность, связанная с данной последовательностью действий.

Использование символьных строк в операторе `switch` может оказаться очень удобным и позволяет сделать код более удобочитаемым. В частности, применение оператора `switch`, управляемого строками, является более совершенным решением по сравнению с равнозначной последовательностью операторов `if/else`. Но переход по символьным строкам обходится дороже с точки зрения потребляемых вычислительных ресурсов, чем переход по целочисленным значениям. Поэтому организовывать переход по символьным строкам рекомендуется лишь в тех случаях, когда управление данными уже происходит в строковой форме. Иными словами, пользоваться символьными строками в операторе `switch` без особой надобности не следует.

Употребление аргументов командной строки

А теперь, когда представлен класс `String`, можно пояснить назначение параметра `args` метода `main()` в исходном коде большинства всех рассмотренных ранее примеров программ. Многие программы получают параметры, задаваемые в командной строке. Это так называемы *аргументы командной строки*. Они представляют собой данные, указываемые непосредственно после имени запускаемой на выполнение программы. Для того чтобы получить доступ к аргументам командной строки из программы на Java, достаточно обратиться к массиву объектов типа `String`, который передается методу `main()`. Рассмотрим в качестве примера программу, отображающую параметры командной строки. Ее исходный код приведен ниже.

```
// Отображение всех данных, указываемых в командной строке.
class CLDemo {
    public static void main(String args[]) {
        System.out.println("There are " + args.length +
                           " command-line arguments.");

        System.out.println("They are: ");
        for(int i=0; i<args.length; i++)
            System.out.println("arg[" + i + "]: " + args[i]);
    }
}
```

Допустим, программа `CLDemo` была запущена на выполнение из командной строки следующим образом:

```
java CLDemo one two three
```

Тогда ее выполнение приведет к следующему результату:

```
There are 3 command-line arguments.
They are:
arg[0]: one
arg[1]: two
arg[2]: three
```

Обратите внимание на то, что первый аргумент содержится в строке, представляющей собой элемент массива с индексом **0**. Для доступа ко второму аргументу следует воспользоваться индексом **1** и т.д.

Для того чтобы стало понятнее, как пользоваться аргументами командной строки, рассмотрим приведенный ниже пример программы. Эта программа принимает из командной строки один аргумент, определяющий имя абонента, а затем производит поиск имени в двумерном массиве символьных строк. Если имя найдено, программа отображает телефонный номер обнаруженного абонента.

```
// Простейший автоматизированный телефонный справочник.
class Phone {
    public static void main(String args[]) {
        String numbers[][] = {
            { "Tom", "555-3322" },
            { "Mary", "555-8976" },
            { "Jon", "555-1037" },
            { "Rachel", "555-1400" }
        };
        int i;

        // Для того чтобы воспользоваться программой,
        // ей нужно передать один аргумент командной строки.
        if(args.length != 1)
            System.out.println("Usage: java Phone <name>");
        else {
            for(i=0; i<numbers.length; i++) {
                if(numbers[i][0].equals(args[0])) {
                    System.out.println(numbers[i][0] + ": " +
                        numbers[i][1]);
                    break;
                }
            }
            if(i == numbers.length)
                System.out.println("Name not found.");
        }
    }
}
```

Выполнение этой программы может дать, например, следующий результат:

```
C>java Phone Mary
Mary: 555-8976
```

Поразрядные операторы

В главе 2 были рассмотрены арифметические и логические операторы, а также операторы отношения. Эти три вида операторов используются чаще всего, но в Java предоставляются также поразрядные операторы, которые позволяют расширить границы применения данного языка программирования. Поразрядные операторы можно применять к значениям типа long, int, short, char и byte. А над типами boolean, float, double или типами классов поразрядные операции выполнять нельзя. Эти операторы называются *поразрядными* потому, что они используются в основном для проверки, установки и сдвига отдельных разрядов числа. Поразрядные операции чрезвычайно важны для решения задач системного программирования, в которых требуется анализировать данные, получаемые из устройства, или формировать значения, передаваемые на устройство. Доступные в Java поразрядные операторы перечислены к табл. 5.1.

Таблица 5.1. Поразрядные операторы

Оператор	Выполняемые действия
&	Поразрядное И
	Поразрядное ИЛИ
^	Поразрядное исключающее ИЛИ
>>	Сдвиг вправо
>>>	Сдвиг вправо без знака
<<	Сдвиг влево
~	Дополнение до 1 (унарный оператор НЕ)

Поразрядные операторы И, ИЛИ, исключающее ИЛИ и НЕ

Поразрядные операторы И, ИЛИ, исключающее ИЛИ и НЕ обозначаются следующим образом: `&`, `|`, `^` и `~`. Они выполняют те же функции, что и их логические аналоги, рассмотренные в главе 2. Но в отличие от логических операторов поразрядные операторы действуют на уровне отдельных двоичных разрядов. Ниже приведены результаты поразрядных операций с двоичными единицами и нулями.

p	q	p & q	p q	p ^ q	~p
0	0	0	0	0	1
1	0	0	1	1	0
0	1	0	1	1	1
1	1	1	1	0	0

С точки зрения наиболее распространенного применения поразрядную операцию И можно рассматривать как способ подавления отдельных двоичных разрядов. Это означает, что если какой-нибудь бит в любом из операндов равен 0, то соответствующий бит результата будет сброшен в 0. Например:

```

1101 0011
1010 1010
&
-----
1000 0010

```

Ниже приведен пример программы, демонстрирующий применение оператора `&`. В этой программе строчные буквы английского алфавита преобразуются в прописные путем сброса шестого бита в коде символа. Коды ASCII и уника (Unicode) строчных букв английского алфавита отличаются от аналогичных кодов прописных букв на величину 32. Поэтому для преобразования строчных букв в прописные достаточно сбросить в нуль шестой бит в кодах их символов.

```

// Преобразование строчных букв английского алфавита в прописные.
class UpCase {
    public static void main(String args[]) {
        char ch;

        for(int i=0; i < 10; i++) {
            ch = (char) ('a' + i);

```

```

System.out.print(ch);

// В следующем операторе сбрасывается шестой бит.
// После этого в переменной ch будет храниться код
// символа прописной буквы.
ch = (char) ((int) ch & 65503);

System.out.print(ch + " ");
}
}
}

```

Результат выполнения данной программы выглядит следующим образом:

```
aA bB cC dD eE fF gG hH iI jJ
```

Значение **65503**, используемое в операции поразрядного И, является десятичным представлением двоичного числа **111111111011111**. Таким образом, при выполнении данной операции все биты кода символа в переменной *ch*, за исключением шестого, остаются без изменения, а шестой бит сбрасывается в нуль.

Операция поразрядного И оказывается удобной и в том случае, если требуется выяснить, установлен или сброшен отдельный бит числа. Например, в приведенной ниже строке кода проверяется, установлен ли четвертый бит значения переменной *status*.

```
if(status & 8) System.out.println("bit 4 is on");
```

В данном примере число **8** использует потому, что в его двоичном представлении установлен только четвертый бит. Таким образом, в условном операторе *if* логическое значение *true* будет получено только в том случае, если четвертый бит значения переменной *status* также установлен. Подобный подход можно применить и для преобразования значения типа *byte* в двоичный формат, как показано ниже.

```

// Отображение битов, составляющих байт.
class ShowBits {
    public static void main(String args[]) {
        int t;
        byte val;

        val = 123;
        for(t=128; t > 0; t = t/2) {
            if((val & t) != 0) System.out.print("1 ");
            else System.out.print("0 ");
        }
    }
}

```

Выполнение этой программы дает следующий результат:

```
0 1 1 1 1 0 1 1
```

В цикле *for* последовательно проверяется каждый бит значения переменной *val*. Для того чтобы выяснить, установлен ли бит, выполняется операция поразрядного И. Если бит установлен, отображается цифра **1**, иначе — **0**. В примере для опробования 5.3 будет показано, как расширить этот элементарный принцип для создания класса, в котором будут отображаться биты двоичного представления целого числа любого типа.

Операция поразрядного ИЛИ выполняет действия, противоположные операции поразрядного И, и служит для установки отдельных битов. Любой бит, значение которого

равно единице хотя бы в одном из двух операндов, будет равен единице и в результирующем значении. Например:

```

1101 0011
1010 1010
|
1111 1011

```

Операцию поразрядного ИЛИ можно использовать для преобразования прописных букв английского алфавита в строчные. Ниже приведен пример программы, решающей эту задачу.

```

// Преобразование прописных букв английского алфавита в строчные.
class LowCase {
    public static void main(String args[]) {
        char ch;

        for(int i=0; i < 10; i++) {
            ch = (char) ('A' + i);
            System.out.print(ch);

            // В следующем операторе устанавливается шестой бит,
            // в итоге переменная ch содержит код символа строчной буквы.
            ch = (char) ((int) ch | 32);

            System.out.print(ch + " ");
        }
    }
}

```

Выполнение этой программы дает следующий результат:

```
Aa Bb Cc Dd Ee Ff Gg Hh Ii Jj
```

В приведенном выше примере программы операция поразрядного ИЛИ выполняется над кодом символа и значением 32, имеющим двоичное представление 0000000000100000. Как видите, в двоичном представлении значения 32 установлен только шестой бит. Используя это значение в качестве одного операнда в операции поразрядного ИЛИ с любым другим значением в качестве другого операнда, получим результат, в котором устанавливается шестой бит, а состояние всех остальных битов остается без изменения. Таким образом, любая прописная буква будет преобразована в строчную.

Операция поразрядного исключающего ИЛИ дает результат, в котором отдельный бит устанавливается в том и только в том случае, если соответствующие биты в двух операндах имеют разные значения. Ниже приведен пример выполнения операции поразрядного исключающего ИЛИ.

```

0111 1111
1011 1001
^
1100 0110

```

Операция поразрядного исключающего ИЛИ имеет одну интересную особенность, которая позволяет очень просто кодировать сообщения. Если выполнить данную опе-

рацию сначала над некоторыми значениями X и Y , а затем над ее результатом и значением Y , то снова получится значение X . Например, при выполнении приведенной ниже последовательности операторов переменная $R2$ получит то же значение, что и X . Таким образом, в результате выполнения подряд двух операций поразрядного исключающего ИЛИ восстанавливается исходное значение.

```
R1 = X ^ Y; R2 = R1 ^ Y;
```

Эту особенность операции поразрядного исключающего ИЛИ можно использовать для создания простейшей шифрующей программы, в которой некоторое целое число будет выполнять роль ключа, применяемого как при шифровании, так и дешифровании сообщений. Над всеми символами сообщения и данным числом будет выполняться операция поразрядного исключающего ИЛИ. В первый раз данная операция будет выполняться при шифровании, формируя кодированный текст, а второй раз — при дешифровании, в результате чего восстанавливается исходный текст сообщения. Ниже приведен пример простой программы, выполняющей шифрование и дешифрование коротких сообщений.

```
// Использование операции поразрядного исключающего ИЛИ
// для шифрования и дешифрования сообщений.
class Encode {
    public static void main(String args[]) {
        String msg = "This is a test";
        String encmsg = "";
        String decmsg = "";
        int key = 88;

        System.out.print("Original message: ");
        System.out.println(msg);

        // зашифровать сообщение
        for(int i=0; i < msg.length(); i++)
            // Построение зашифрованной строки сообщения.
            encmsg = encmsg + (char) (msg.charAt(i) ^ key);

        System.out.print("Encoded message: ");
        System.out.println(encmsg);

        // дешифровать сообщение
        for(int i=0; i < msg.length(); i++)
            // Построение дешифрованной строки сообщения.
            decmsg = decmsg + (char) (encmsg.charAt(i) ^ key);

        System.out.print("Decoded message: ");
        System.out.println(decmsg);
    }
}
```

Выполнение этой программы дает следующий результат:

```
Original message: This is a test
Encoded message: 01+x1+x9x,=+,
Decoded message: This is a test
```

Как видите, в результате двух операций поразрядного исключающего ИЛИ с одним и тем же ключом получается дешифрованное сообщение, совпадающее с исходным.

Унарная операция поразрядного НЕ (или дополнения до 1) изменяет на обратное состояние всех битов операнда. Так, если некоторая целочисленная переменная *A* содержит значение с двоичным представлением **10010110**, то в результате поразрядной операции $\sim A$ получится двоичная комбинация **01101001**.

Ниже приведен пример программы, демонстрирующий применение операции поразрядного НЕ. Эта программа отображает число и его дополнение в двоичном представлении.

```
// Демонстрация операции поразрядного НЕ.
class NotDemo {
    public static void main(String args[]) {
        byte b = -34;

        for(int t=128; t > 0; t = t/2) {
            if((b & t) != 0) System.out.print("1 ");
            else System.out.print("0 ");
        }
        System.out.println();

        // изменить на обратное состояние всех битов
        b = (byte) ~b;

        for(int t=128; t > 0; t = t/2) {
            if((b & t) != 0) System.out.print("1 ");
            else System.out.print("0 ");
        }
    }
}
```

Выполнение этой программы дает следующий результат:

```
1 1 0 1 1 1 1 0
0 0 1 0 0 0 0 1
```

Операторы сдвига

В Java предусмотрена возможность сдвига битов, составляющих числовое значение, влево или вправо на заданное количество позиций. Для этой цели в Java предоставляются три перечисленных ниже оператора сдвига.

<<	Сдвиг влево
>>	Сдвиг вправо
>>>	Сдвиг вправо без знака

Ниже приведена общая форма для этих операторов.

```
значение << число_битов
значение >> число_битов
значение >>> число_битов
```

где *число_битов* — это число позиций двоичных разрядов, на которое сдвигается указанное значение.

При сдвиге влево освободившиеся младшие разряды заполняются нулями. А при сдвиге вправо дело обстоит немного сложнее. Как известно, признаком отрицательного целого числа является единица в старшем разряде, поэтому при сдвиге вправо старший (знаковый) разряд сохраняется. Если число положительное, то в него записывается нуль, а если отрицательное — единица.

Помимо сохранения знакового разряда, необходимо помнить еще об одной особенности сдвига вправо. Отрицательные числа в Java (как, впрочем, и в других языках программирования) представлены в виде *дополнения до двух*. Для того чтобы преобразовать положительное число в отрицательное, нужно изменить на обратное состояние всех битов его двоичного представления, а к полученному результату прибавить единицу. Так, значение -1 имеет байтовое представление **11111111**. Сдвинув это значение вправо на любое число позиций, мы снова получим -1 !

Если при сдвиге вправо не требуется сохранять знаковый разряд, то можно воспользоваться сдвигом вправо без знака (оператором `>>>`). В этом случае освободившиеся старшие разряды всегда будут заполняться нулями. Именно поэтому такая операция называется еще *сдвигом с заполнением нулями*. Сдвигом вправо без знака удобно пользоваться для обработки нечисловых значений, в том числе кодов состояния.

При любом сдвиге теряются те биты, которые сдвигаются. Циклический сдвиг в Java не поддерживается, и поэтому нет возможности восстановить потерянные разряды.

Ниже приведен пример программы, демонстрирующий эффект от применения операторов сдвига влево и вправо. В двоичном представлении исходного целочисленного значения **1** установлен лишь младший разряд. К этому значению восемь раз применяется операция сдвига влево. После каждого сдвига на экран выводится восемь младших разрядов числа. Затем единица устанавливается в восьмом двоичном разряде числа и производятся его сдвиги вправо.

```
// Демонстрация операторов << и >>.
class ShiftDemo {
    public static void main(String args[]) {
        int val = 1;

        for(int i = 0; i < 8; i++) {
            for(int t=128; t > 0; t = t/2) {
                if((val & t) != 0) System.out.print("1 ");
                else System.out.print("0 ");
            }
            System.out.println();
            val = val << 1; // сдвинуть влево
        }
        System.out.println();

        val = 128;
        for(int i = 0; i < 8; i++) {
            for(int t=128; t > 0; t = t/2) {
                if((val & t) != 0) System.out.print("1 ");
                else System.out.print("0 ");
            }
            System.out.println();
            val = val >> 1; // сдвинуть вправо
        }
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```

0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 0
0 0 0 0 0 1 0 0
0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0
0 0 1 0 0 0 0 0
0 1 0 0 0 0 0 0
1 0 0 0 0 0 0 0

1 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0
0 0 1 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 1

```

Выполняя сдвиг значений типа `byte` и `short`, необходимо соблюдать осторожность, поскольку исполняющая система Java автоматически преобразует их в тип `int` и лишь затем вычисляет выражение с оператором сдвига. Так, если сдвинуть вправо значение типа `byte`, оно будет сначала продвинуто к типу `int`, а результат сдвига будет также отнесен к типу `int`. Обычно такое преобразование не влечет за собой никаких последствий. Но если попытаться сдвинуть отрицательное значение типа `byte` или `short`, то при продвижении к типу `int` оно будет дополнено знаком, а следовательно, старшие его разряды будут заполнены единицами. Это вполне оправдано при обычном сдвиге вправо. Но при выполнении сдвига с заполнением нулями в байтовом представлении числа неожиданно появятся 24 единицы, которые придется дополнительно сдвинуть, прежде чем в нем появятся нули.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Известно, что каждый последующий разряд двоичного представления числа отличается от предыдущего на величину, равную двум. Значит ли это, что операторы сдвига можно использовать для умножения или деления числа на два?

ОТВЕТ. Да, можно. Операторы сдвига часто используются для этой цели. При сдвиге влево число умножается на два, а при сдвиге вправо — делится на два. Нужно лишь помнить, что при сдвиге могут исчезнуть установленные биты, что приведет к потере точности.

Поразрядные составные операторы присваивания

Все двоичные поразрядные операторы имеют укороченную форму и могут быть использованы в составных операциях присваивания. Например, в двух приведенных ниже операторах переменной `x` присваивается результат выполнения операции исключающее ИЛИ над первоначальным значением переменной `x` и числовым значением `127`.

```

x = x ^ 127;
x ^= 127;

```

Пример для опробования 5.3.**Создание класса ShowBits****Исходный файл:** ShowBitsDemo.java

В данном проекте предстоит создать класс ShowBits, который позволит отображать любое целочисленное значение в двоичном виде. Этот класс может оказаться очень полезным при разработке некоторых программ. Так, если требуется отладить код драйвера устройства, возможность контролировать поток данных в двоичном виде окажется весьма кстати.

Последовательность действий

1. Создайте новый файл ShowBitsDemo.java.
2. Создайте класс ShowBits, начав его со следующего кода:

```
class ShowBits {
    int numbits;

    ShowBits(int n) {
        numbits = n;
    }
}
```

Конструктор класса ShowBits позволяет создавать объекты, отображающие заданное число битов. Например, для создания объекта, отображающего 8 младших битов некоторого значения, служит следующее выражение:

```
ShowBits byteval = new ShowBits(8)
```

Число битов, предназначенных для отображения, сохраняется в переменной экземпляра numbits.

3. Для вывода двоичных значений в классе ShowBits определен метод show(), код которого приведен ниже.

```
void show(long val) {
    long mask = 1;

    // сдвинуть значение 1 влево на нужную позицию
    mask <<= numbits-1;

    int spacer = 0;
    for(; mask != 0; mask >>= 1) {
        if((val & mask) != 0) System.out.print("1");
        else System.out.print("0");
        spacer++;
        if((spacer % 8) == 0) {
            System.out.print(" ");
            spacer = 0;
        }
    }
    System.out.println();
}
```

Обратите внимание на то, что данному методу передается один параметр типа long. Но это совсем не означает, что при вызове ему нужно всегда передавать значение типа long. Правила автоматического продвижения типов в Java допускают передавать методу show() любое целочисленное значение. А количество отображаемых

битов определяется переменной `numbits`. Группы из 8 битов разделяются в методе `show()` пробелами. Это упрощает восприятие длинных двоичных комбинаций.

4. Ниже приведен весь исходный код программы из файла `ShowBitsDemo.java`.

```
/*
    Пример для опробования 5.3

    Создание класса для отображения значений в двоичном виде.
*/

class ShowBits {
    int numbits;

    ShowBits(int n) {
        numbits = n;
    }

    void show(long val) {
        long mask = 1;

        // сдвинуть значения 1 влево на нужную позицию
        mask <=<= numbits-1;

        int spacer = 0;
        for(; mask != 0; mask >>= 1) {
            if((val & mask) != 0) System.out.print("1");
            else System.out.print("0");
            spacer++;
            if((spacer % 8) == 0) {
                System.out.print(" ");
                spacer = 0;
            }
        }
        System.out.println();
    }
}

// продемонстрировать класс ShowBits
class ShowBitsDemo {
    public static void main(String args[]) {
        ShowBits b = new ShowBits(8);
        ShowBits i = new ShowBits(32);
        ShowBits li = new ShowBits(64);

        System.out.println("123 in binary: ");
        b.show(123);

        System.out.println("\n87987 in binary: ");
        i.show(87987);

        System.out.println("\n237658768 in binary: ");
        li.show(237658768);
    }
}
```

```

        // можно также отобразить младшие разряды любого целого числа
        System.out.println("\nLow order 8 bits of 87987 in binary: ");
        b.show(87987);
    }
}

```

5. Результат выполнения программы ShowBitsDemo выглядит следующим образом:

```

123 in binary:
01111011

87987 in binary:
00000000 00000001 01010111 10110011

237658768 in binary:
00000000 00000000 00000000 00000000 00001110 00101010 01100010 10010000

Low order 8 bits of 87987 in binary:
10110011

```

Оператор ?

Оператор ? — один из самых удобных в Java. Он часто используется вместо операторов if-else в следующей общей форме:

```

if (условие)
    var = выражение_1;
else
    var = выражение_2;

```

где значение, присваиваемое переменной `var`, определяет *условие*, управляющее выполнением оператора if. Оператор ? называется *тернарным*, поскольку он обрабатывает три операнда. Этот оператор записывается в следующей общей форме:

```
выражение_1 ? выражение_2 : выражение_3;
```

где *выражение_1* должно быть логическим, т.е. возвращать тип `boolean`, а *выражение_2* и *выражение_3*, разделяемые двоеточием, могут быть любого типа, за исключением `void`. Но типы второго и третьего выражений должны непременно совпадать.

Значение выражения ? определяется следующим образом. Сначала вычисляется *выражение_1*. Если оно дает логическое значение `true`, то вычисляется *выражение_2*, а его значение становится результирующим для всего выражения ?. Если же *выражение_1* дает логическое значение `false`, то вычисляется *выражение_3*, а его значение становится результирующим для всего выражения ?. Рассмотрим пример, в котором сначала вычисляется абсолютное значение переменной `val`, а затем оно присваивается переменной `absval`.

```
absval = val < 0 ? -val : val; // получить абсолютное значение переменной val
```

В данном примере переменной `absval` присваивается значение переменной `val`, если это значение больше или равно нулю. А если значение переменной `val` отрицательное, то переменной `absval` присваивается значение `val` со знаком “минус”, что в итоге дает положительную величину. Код, выполняющий ту же самую функцию, но с помощью логической конструкции if-else, будет выглядеть следующим образом:


```
if(val < 0) absval = -val;
else absval = val;
```

Рассмотрим еще один пример применения оператора `?`. В этом примере программы выполняется деление двух чисел, но не допускается деление на нуль:

```
// Предотвращение деления на ноль с помощью оператора ?.
class NoZeroDiv {
    public static void main(String args[]) {
        int result;

        for(int i = -5; i < 6; i++) {
            // Деление на ноль предотвращается.
            result = i != 0 ? 100 / i : 0;
            if(i != 0)
                System.out.println("100 / " + i + " is " + result);
        }
    }
}
```

Ниже приведен результат выполнения данной программы.

```
100 / -5 is -20
100 / -4 is -25
100 / -3 is -33
100 / -2 is -50
100 / -1 is -100
100 / 1 is 100
100 / 2 is 50
100 / 3 is 33
100 / 4 is 25
100 / 5 is 20
```

Обратите особое внимание на следующую строку кода:

```
result = i != 0 ? 100 / i : 0;
```

где переменной `result` присваивается результат деления числа **100** на значение переменной `i`. Но деление выполняется только в том случае, если значение переменной `i` не равно нулю. В противном случае переменной `result` присваивается нулевое значение.

Значение, возвращаемое оператором `?`, не обязательно присваивать переменной. Его можно, например, использовать в качестве параметра при вызове метода. Если же все три выражения оператора `?` имеют тип `boolean`, то сам оператор `?` может быть использован в качестве условия для выполнения цикла или оператора `if`. Ниже приведена немного видоизмененная версия предыдущего примера программы. Ее выполнение дает такой же результат, как и прежде.

[illegible]

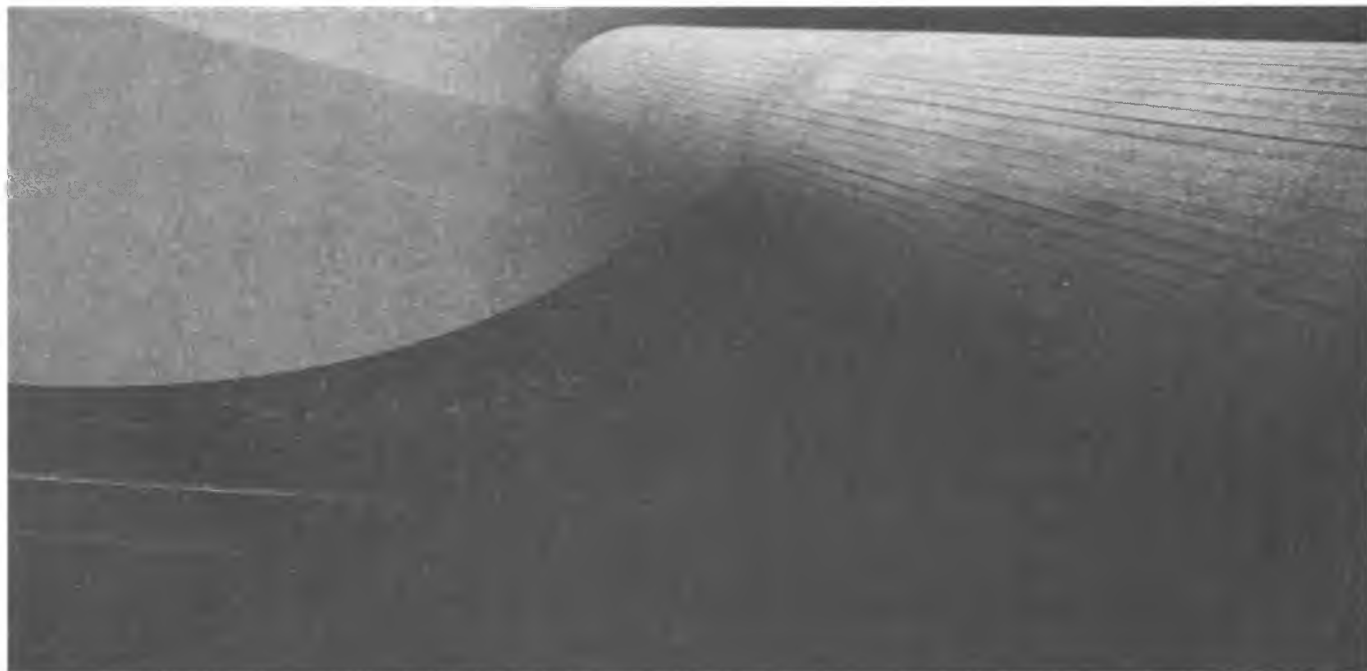
Обратите внимание на выражение, определяющее условие выполнения оператора `if`. Если значение переменной `i` равно нулю, то оператор `?` возвращает логическое значение `false`, что предотвращает деление на нуль, и результат не отображается. В противном случае осуществляется обычное деление.

Упражнение для самопроверки по материалу главы 5

1. Покажите два способа объявления одномерного массива, состоящего из 12 элементов типа `double`.
2. Покажите, как инициализировать одномерный массив целочисленными значениями от 1 до 5.
3. Напишите программу, в которой массив используется для нахождения среднего арифметического десяти значений типа `double`. Используйте любые десять чисел.
4. Измените программу, написанную в примере для опробования 5.1, таким образом, чтобы она сортировала массив символьных строк. Продемонстрируйте ее работоспособность.
5. В чем отличие методов `indexOf()` и `lastIndexOf()` из класса `String`?
6. Все символьные строки являются объектами типа `String`. Покажите, как вызываются методы `length()` и `charAt()` для строкового литерала `"I like Java"` (Мне нравится Java).
7. Расширьте класс `Encode` таким образом, чтобы в качестве ключа шифрования использовалась строка из восьми символов.
8. Можно ли применять поразрядные операторы к значениям типа `double`?
9. Перепишите приведенную ниже последовательность операторов, воспользовавшись оператором `?`.

```
if(x < 0) y = 10;
else y = 20;
```
10. В приведенном ниже фрагменте кода содержится знак `&`. Какой оператор он обозначает: поразрядный или логический? Обоснуйте свой ответ.

```
boolean a, b;
// ...
if(a & b) ...
```
11. Является ли ошибкой превышение верхней границы массива? Является ли ошибкой использование отрицательных значений для доступа к элементам массива?
12. Как обозначается оператор сдвига вправо без знака?
13. Перепишите рассмотренный ранее в этой главе класс `MinMax` таким образом, чтобы в нем использовалась разновидность `for-each` цикла `for`.
14. В примере для опробования 5.1 была реализована пузырьковая сортировка. Можно ли в программе из этого примера заменить обычный цикл `for` его разновидностью `for-each`? Если нельзя, то почему?
15. Можно ли управлять оператором `switch` с помощью объектов типа `String`?



Глава 6

**Дополнительные
сведения о методах
и классах**

Основные навыки и понятия

- Управление доступом к членам классов
- Передача объектов при вызове методов
- Возврат объектов из методов
- Перегрузка методов
- Перегрузка конструкторов
- Применение рекурсии
- Использование ключевого слова `static`
- Применение внутренних классов
- Использование аргументов переменной длины

В этой главе возобновляется рассмотрение классов и методов. Сначала будет показано, каким образом контролируется доступ к членам класса, а затем рассмотрены особенности передачи и возврата объектов из методов, перегрузки методов, использования рекурсии и ключевого слова `static`. Кроме того, в этой главе будут представлены вложенные классы и методы с аргументами переменной длины.

Управление доступом к членам класса

Поддержка свойства инкапсуляции в классе дает два главных преимущества. Во-первых, класс связывает данные с кодом. Это преимущество использовалось в предыдущих примерах программ, начиная с главы 4. И во-вторых, класс предоставляет средства для управления доступом к его членам. Именно эта, вторая преимущественная особенность и будет рассмотрена ниже.

В языке Java, по существу, имеются два типа членов класса: открытые (`public`) и закрытые (`private`), хотя в действительности дело обстоит немного сложнее. Доступ к открытому члену свободно осуществляется из кода, определенного за пределами класса. Именно этот тип члена класса использовался в рассматривавшихся до сих пор примерах программ. А закрытый член класса доступен только методам, определенным в самом классе. С помощью закрытых членов и организуется управление доступом.

Ограничение доступа к членам класса является основополагающей частью объектно-ориентированного программирования, поскольку оно позволяет исключить неверное использование объекта. Разрешая доступ к закрытым данным только с помощью строго определенного ряда методов, можно предупредить присваивание неверных значений этим данным, выполняя, например, проверку диапазона представления чисел. Для закрытого члена класса нельзя задать значение непосредственно в коде за пределами класса. Но в то же время можно полностью управлять тем, как и когда данные используются в объекте. Следовательно, правильно реализованный класс образует некий “черный ящик”, которым можно пользоваться, но внутренний механизм его действия закрыт для вмешательства извне.

В рассмотренных ранее примерах программ не уделялось никакого внимания управлению доступом, поскольку в Java члены класса по умолчанию доступны из остальных частей программы. (Иными словами, они открыты для доступа по умолчанию.) Это удобно для создания небольших программ (в том числе и тех, что служат примерами в данной книге), но недопустимо в большинстве реальных условий эксплуатации программного обеспечения. Ниже будет показано, какими языковыми средствами Java можно пользоваться для управления доступом.

Модификаторы доступа в Java

Управление доступом к членам класса в Java осуществляется с помощью трех *модификаторов доступа* (называемых также *спецификаторами*): `public`, `private` и `protected`. Если модификатор не указан, то принимается тип доступа по умолчанию. В этой главе будут рассмотрены модификаторы `public` и `private`. Модификатор `protected` непосредственно связан с наследованием, и поэтому он будет обсуждаться в главе 8.

Когда член класса обозначается модификатором `public`, он становится доступным из любого другого кода в программе, включая и методы, определенные в других классах. Когда же член класса обозначается модификатором `private`, он может быть доступен только другим членам этого класса. Следовательно, методы из других классов не имеют доступа к закрытому члену данного класса.

Если все классы в программе относятся к одному пакету, то отсутствие модификатора доступа равнозначно указанию модификатора `public` по умолчанию. *Пакет* представляет собой группу классов, предназначенных как для организации классов, так и для управления доступом. Рассмотрение пакетов откладывается до главы 8, а для примеров программ, представленных в этой и предыдущих главах, тип доступа по умолчанию не отличается от `public`.

Модификатор доступа указывается перед остальной частью описания типа отдельного члена класса. Это означает, что именно с него должен начинаться оператор объявления члена класса. Ниже приведены соответствующие примеры.

```
public String errMsg;
private accountBalance bal;
```

```
private boolean isError(byte status) { // ...
```

Для того чтобы стал понятнее эффект от применения модификаторов доступа `public` и `private`, рассмотрим следующий пример программы:

```
// Открытый и закрытый доступ к членам класса.
class MyClass {
    private int alpha; // закрытый доступ
    public int beta;   // открытый доступ
    int gamma; // тип доступа по умолчанию (по существу, открытый)

    /* Методы доступа к переменной alpha. Члены класса могут
       обращаться к закрытым членам того же класса.
    */
    void setAlpha(int a) {
        alpha = a;
    }

    int getAlpha() {
```

```

        return alpha;
    }
}

class AccessDemo {
    public static void main(String args[]) {
        MyClass ob = new MyClass();

        /* Доступ к переменной alpha возможен только с помощью
           специально предназначенных для этой цели методов. */
        ob.setAlpha(-99);
        System.out.println("ob.alpha is " + ob.getAlpha());

        // Обратиться к переменной alpha так, как показано ниже, нельзя.
        // ob.alpha = 10; // Ошибка! alpha - закрытая переменная!

        // Следующие обращения вполне допустимы, так как
        // переменные beta и gamma являются открытыми.
        ob.beta = 88;
        ob.gamma = 99;
    }
}

```

Нетрудно заметить, что в классе `MyClass` переменная `alpha` определена как `private`, переменная `beta` — как `public`, а перед переменной `gamma` модификатор доступа отсутствует, т.е. в данном примере она ведет себя как открытый член класса, которому по умолчанию присваивается модификатор доступа `public`. Переменная `alpha` закрыта, и поэтому к ней невозможно обратиться за пределами ее класса. Следовательно, в классе `AccessDemo` нельзя пользоваться переменной `alpha` непосредственно. Доступ к ней организуется с помощью открытых методов доступа `setAlpha()` и `getAlpha()`, определенных в одном с ней классе. Если удалить комментарии в начале следующей строки кода, то скомпилировать рассматриваемую здесь программу не удастся:

```
// ob.alpha = 10; // Ошибка! alpha - закрытая переменная!
```

Компилятор выдаст сообщение об ошибке, связанной с нарушением правил доступа. Несмотря на то что переменная `alpha` недоступна для кода за пределами класса `MyClass`, пользоваться ею можно с помощью открытых методов доступа `setAlpha()` и `getAlpha()`.

Таким образом, закрытые переменные могут быть свободно использованы другими членами класса, но недоступны за пределами этого класса.

Рассмотрим практическое применение средств управления доступом на примере приведенной ниже программы. Во время ее выполнения предотвращается возникновение ошибок нарушения границ отказоустойчивого целочисленного массива. Это достигается следующим образом. Массив объявляется как закрытый член класса, а доступ к нему осуществляется с помощью специально предназначенных для этой цели методов. Эти методы отслеживают попытки обращения к элементам, не входящим в массив, и вместо аварийной остановки программы возвращают сообщение об ошибке. Массив определяется в классе `FailSoftArray`, код которого приведен ниже.

```

/* В этом классе реализуется отказоустойчивый массив,
   предотвращающий ошибки при выполнении программы.
*/

```

```

class FailSoftArray {
    private int a[];        // Ссылка на массив.
    private int errval;     // Значение, возвращаемое в том случае, если
                            // в методе get() будет обнаружена ошибка.
    public int length;      // Переменная length открыта.

    /* Конструктору данного класса передается размер массива и значение,
       которое должен возвращать метод get() при обнаружении ошибки. */
    public FailSoftArray(int size, int errv) {
        a = new int[size];
        errval = errv;
        length = size;
    }

    // вернуть значение элемента массива по заданному индексу
    public int get(int index) {
        // Отслеживание попытки обращения за границы массива.
        if(ok(index)) return a[index];
        return errval;
    }

    // установить значение элемента по заданному индексу,
    // если возникнет ошибка, вернуть логическое значение false
    public boolean put(int index, int val) {
        // Отслеживание попытки обращения за границы массива.
        if(ok(index)) {
            a[index] = val;
            return true;
        }
        return false;
    }

    // вернуть логическое значение true, если индекс
    // не выходит за границы массива
    private boolean ok(int index) {
        if(index >= 0 & index < length) return true;
        return false;
    }
}

// продемонстрировать обращение к отказоустойчивому массиву
class FSDemo {
    public static void main(String args[]) {
        FailSoftArray fs = new FailSoftArray(5, -1);
        int x;

        // выявить скрытые сбои при обращении к массиву
        System.out.println("Fail quietly.");
        for(int i=0; i < (fs.length * 2); i++)
            // Доступ к массиву должен осуществляться с помощью
            // специально предназначенных для этого методов.
            fs.put(i, i*10);
    }
}

```



```

for(int i=0; i < (fs.length * 2); i++) {
    // Доступ к массиву должен осуществляться с помощью
    // специально предназначенных для этого методов.
    x = fs.get(i);

    if(x != -1) System.out.print(x + " ");
}
System.out.println("");

// а теперь обработать сбои и вывести сообщения об ошибках
System.out.println("\nFail with error reports.");
for(int i=0; i < (fs.length * 2); i++)
    if(!fs.put(i, i*10))
        System.out.println("Index " + i + " out-of-bounds");

for(int i=0; i < (fs.length * 2); i++) {
    x = fs.get(i);
    if(x != -1) System.out.print(x + " ");
    else
        System.out.println("Index " + i + " out-of-bounds");
}
}
}

```

Выполнение этой программы дает следующий результат:

Fail quietly.

0 10 20 30 40

Fail with error reports.

Index 5 out-of-bounds

Index 6 out-of-bounds

Index 7 out-of-bounds

Index 8 out-of-bounds

Index 9 out-of-bounds

0 10 20 30 40 Index 5 out-of-bounds

Index 6 out-of-bounds

Index 7 out-of-bounds

Index 8 out-of-bounds

Index 9 out-of-bounds

Рассмотрим подробнее приведенный выше пример программы. В классе `FailSoftArray` определены три закрытых члена. Первым из них является переменная `a`, в которой содержится ссылка на массив, предназначенный для хранения данных. Вторым членом является переменная `errval`, в которой хранится значение, возвращаемое вызывающей части программы в том случае, если вызов метода `get()` оказывается неудачным. И третьим членом является метод `ok()`, в котором определяется, находится ли индекс в границах массива. Эти три члена могут быть использованы только другими членами класса `FailSoftArray`. Остальные члены данного класса объявлены открытыми и могут быть вызваны из любой части программы, где используется класс `FailSoftArray`.

При построении объекта типа `FailSoftArray` следует указать размер массива и значение, которое должно быть возвращено, если вызов `get()` окажется неудачным. Ошибочное значение должно отличаться от тех значений, которые могут хранить-

ся в массиве. Конкретный массив, обращение к которому осуществляется по ссылке в переменной `a`, а также ошибочное значение в переменной `errval` не могут быть непосредственно доступны пользователям построенного объекта типа `FailSoftArray`, и благодаря этому неправильное их употребление исключается. В частности, пользователь не может непосредственно обратиться к массиву по ссылке в переменной `a`, указав индекс нужного элемента и не нарушив, возможно, при этом границы массива. Это можно сделать только с помощью методов `get()` и `put()`.

Метод `ok()` объявлен как закрытый главным образом для того, чтобы проиллюстрировать управление доступом. Даже если бы он и был открытым, это не представляло бы никакой опасности, поскольку он не видоизменяет объект. Но этот метод используется только членами класса `FailSoftArray`, поэтому он и объявлен закрытым.

Обратите внимание на то, что переменная экземпляра `length` открыта. Это согласуется с правилами реализации массивов в Java. Для того чтобы получить данные о длине массива типа `FailSoftArray`, достаточно прочитать значение переменной экземпляра `length`.

Для сохранения данных в массиве типа `FailSoftArray` по указанному индексу вызывается метод `put()`, тогда как метод `get()` извлекает содержимое элемента этого массива по заданному индексу. Если указанный индекс оказывается вне границ массива, метод `put()` возвращает логическое значение `false`, а метод `get()` — значение `errval`.

Ради простоты в большинстве примеров программ, представленных в этой книге, на члены класса будет в основном распространяться тип доступа по умолчанию. Но не следует забывать, что в реальных объектно-ориентированных программах очень важно ограничивать доступ к членам класса, и в особенности к переменным. Как будет показано в главе 7, при использовании наследования роль средств управления доступом еще более возрастает.

Пример для опробования 6.1.

Усовершенствование класса `Queue`

Исходный файл: `Queue.java`

Модификатор доступа `private` можно использовать для усовершенствования класса `Queue`, разработанного в примере для опробования 5.2 из главы 5. В текущей версии этого класса используется тип доступа по умолчанию, который, по существу, делает все члены этого класса открытыми. Это означает, что другие классы могут непосредственно обращаться к элементам базового массива — и даже вне очереди. А поскольку назначение класса, реализующего очередь, состоит в том, чтобы обеспечить принцип доступа “первым пришел — первым обслужен”, то возможность произвольного обращения к элементам массива явно неуместна. В частности, это давало бы возможность недобросовестным программистам изменять индексы в переменных `putloc` и `getloc`, искажая тем самым организацию очереди. Подобные недостатки нетрудно устранить с помощью модификатора доступа `private`.

Последовательность действий

1. Создайте новый файл `Queue.java`.
2. Добавьте к массиву `q`, а также к переменным `putloc` и `getloc` модификатор доступа `private` в классе `Queue`. В результате код этого класса должен выглядеть так, как показано ниже.

```
// Усовершенствованный класс очереди, предназначенной
// для хранения символьных значений.
class Queue {
```

```

// Следующие члены класса теперь являются закрытыми.
private char q[]; // Массив для хранения элементов очереди
private int putloc, getloc; // Индексы размещения и извлечения
                           // элементов очереди

Queue(int size) {
    q = new char[size+1]; // выделить память для очереди
    putloc = getloc = 0;
}

// поместить символ в очередь
void put(char ch) {
    if(putloc==q.length-1) {
        System.out.println(" - Queue is full.");
        return;
    }

    putloc++;
    q[putloc] = ch;
}

// извлечь символ из очереди
char get() {
    if(getloc == putloc) {
        System.out.println(" - Queue is empty.");
        return (char) 0;
    }

    getloc++;
    return q[getloc];
}
}

```

3. Изменение типа доступа к массиву `q` и переменным `putloc` и `getloc` с выбираемого по умолчанию на закрытый (`private`) никак не скажется на работе тех программ, где класс `Queue` используется правильно. В частности, этот класс будет по-прежнему взаимодействовать с классом `QDemo`, созданным в примере для опробования 5.2. В то же время неправильное обращение к классу `Queue` станет невозможным. Например, следующий фрагмент кода недопустим:

```

Queue test = new Queue(10);

test.q[0] = 99; // Ошибка!
test.putloc = -100; // Не пройдет!

```

4. Теперь, когда массив `q` и переменные `putloc` и `getloc` стали закрытыми, класс `Queue` строго следует принципу “первым пришел — первым обслужен”, по которому действует очередь.

Передача объектов методам

В приведенных ранее примерах программ в качестве параметров методам передавались лишь простые типы. Но параметрами могут быть и объекты. Например, в приве-

денной ниже программе определен класс `Block` для хранения размеров параллелепипеда в трех измерениях.

```
// Методам можно передавать объекты.
class Block {
    int a, b, c;
    int volume;

    Block(int i, int j, int k) {
        a = i;
        b = j;
        c = k;
        volume = a * b * c;
    }

    // вернуть логическое значение true, если
    // параметр ob определяет такой же параллелепипед
    // В качестве параметра методу передается объект.
    boolean sameBlock(Block ob) {
        if((ob.a == a) & (ob.b == b) & (ob.c == c)) return true;
        else return false;
    }

    // вернуть логическое значение true, если
    // параметр ob определяет параллелепипед такого же объема
    // В качестве параметра методу передается объект.
    boolean sameVolume(Block ob) {
        if(ob.volume == volume) return true;
        else return false;
    }
}

class PassOb {
    public static void main(String args[]) {
        Block ob1 = new Block(10, 2, 5);
        Block ob2 = new Block(10, 2, 5);
        Block ob3 = new Block(4, 5, 5);

        // Здесь методам передаются объекты.
        System.out.println("ob1 same dimensions as ob2: " +
                           ob1.sameBlock(ob2));
        System.out.println("ob1 same dimensions as ob3: " +
                           ob1.sameBlock(ob3));
        System.out.println("ob1 same volume as ob3: " +
                           ob1.sameVolume(ob3));
    }
}
```

Выполнение этой программы дает следующий результат:

```
ob1 same dimensions as ob2: true
ob1 same dimensions as ob3: false
ob1 same volume as ob3: true
```

В методах `sameBlock()` и `sameVolume()` объект `Block`, переданный им в качестве параметра, сравнивается с текущим объектом. Метод `sameBlock()` возвращает логиче-

ское значение `true` только в том случае, если все три размера обоих параллелепипедов совпадают. А в методе `sameVolume()` сравниваются лишь объемы двух параллелепипедов. Но в обоих случаях параметр `ob` имеет тип `Block`. Несмотря на то что `Block` — это класс, параметры данного типа используются таким же образом, как и параметры встроенных в Java типов данных.

Способы передачи аргументов методу

Как показывает приведенный выше пример, передача объекта методу производится очень просто. Но в этом примере показаны не все нюансы данного процесса. В некоторых случаях последствия передачи объекта по ссылке будут отличаться от тех результатов, к которым приводит передача значения обычного типа. Для выяснения причин этих отличий рассмотрим два способа передачи аргументов методу.

Первым способом является *вызов по значению*. В этом случае значение аргумента копируется в формальный параметр метода. Следовательно, изменения, вносимые в параметр метода, не оказывают никакого влияния на аргумент, используемый для вызова. А вторым способом передачи аргумента является *вызов по ссылке*. В данном случае параметру метода передается ссылка на аргумент, а не значение аргумента. В методе эта ссылка используется для доступа к конкретному аргументу, указываемому при вызове. Это означает, что изменения, вносимые в параметр, *будут* оказывать влияние на аргумент, используемый для вызова метода. Как будет показано далее, в Java используются оба способа. А выбор конкретного способа зависит от того, что именно передается.

Если методу передается простой тип, например `int` или `double`, он передается по значению. При этом создается копия аргумента, а то, что происходит с параметром, принимающим аргумент, не распространяется за пределы метода. Рассмотрим в качестве примера следующую программу:

```
// Простые типы данных передаются методам по значению.
class Test {
    /* Этот метод не может изменить значения аргументов,
       передаваемых ему при вызове. */
    void noChange(int i, int j) {
        i = i + j;
        j = -j;
    }
}

class CallByValue {
    public static void main(String args[]) {
        Test ob = new Test();

        int a = 15, b = 20;

        System.out.println("a and b before call: " +
                           a + " " + b);

        ob.noChange(a, b);

        System.out.println("a and b after call: " +
                           a + " " + b);
    }
}
```

Ниже приведен результат выполнения данной программы.

```
a and b before call: 15 20
a and b after call: 15 20
```

Нетрудно заметить, что действия, выполняемые в теле метода `noChange()`, не оказывают никакого влияния на значения переменных `a` и `b` в вызывающем методе.

Если методу передается объект, то ситуация меняется коренным образом, поскольку объекты передаются неявно по ссылке. Не следует забывать, что создание переменной типа класса, по существу, означает формирование ссылки на объект этого класса. И методу на самом деле передается только ссылка, а не сам объект. Поэтому при передаче этой ссылки методу принимающий ее параметр будет ссылаться на тот же самый объект, на который ссылается аргумент. Это означает, что и аргумент, и параметр ссылается на один и тот же объект и что объекты, по существу, передаются методам по ссылке. Таким образом, объект в методе *будет* оказывать влияние на объект, используемый в качестве аргумента. Для примера рассмотрим следующую программу:

```
// Объекты передаются методам по ссылке.
class Test {
    int a, b;

    Test(int i, int j) {
        a = i;
        b = j;
    }
    /* Передача объекта методу. Теперь переменные ob.a и ob.b
       из передаваемого объекта можно изменить. */
    void change(Test ob) {
        ob.a = ob.a + ob.b;
        ob.b = -ob.b;
    }
}

class CallByRef {
    public static void main(String args[]) {
        Test ob = new Test(15, 20);

        System.out.println("ob.a and ob.b before call: " +
                           ob.a + " " + ob.b);

        ob.change(ob);

        System.out.println("ob.a and ob.b after call: " +
                           ob.a + " " + ob.b);
    }
}
```

Выполнение этой программы дает следующий результат:

```
ob.a and ob.b before call: 15 20
ob.a and ob.b after call: 35 -20
```

Как видите, в данном случае действия в методе `change()` оказывают влияние на объект, используемый в качестве аргумента этого метода.

Не следует, однако, забывать, что когда объект передается методу по ссылке, сама ссылка на него передается по значению. Но поскольку передаваемое значение лишь

указывает на объект, копия этого значения будет по-прежнему указывать на тот же самый объект в соответствующем аргументе.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Существует ли способ передачи простого типа по ссылке?

ОТВЕТ. Явным образом этого сделать нельзя. Но в Java определен ряд классов, служащих *оболочкой* для простых типов. Это классы `Double`, `Float`, `Byte`, `Short`, `Integer`, `Long` и `Character`. Они не только позволяют передавать простые типы по ссылке, но и содержат ряд методов для манипулирования их значениями. Например, в классах-оболочках числовых типов содержатся методы, преобразующие двоичные значения в символьную строку, а также методы, выполняющие обратное преобразование.

Возврат объектов

Метод может возвращать данные любого типа, включая и типы классов. Например, объект приведенного ниже класса `ErrorMsg` может быть использован для сообщения об ошибке. В этом классе имеется метод `getErrorMsg()`, который возвращает объект типа `String`, описывающий ошибку. Объект типа `String` строится на основании кода ошибки, переданного методу.

```
// Возврат объекта типа String.
class ErrorMsg {
    String msgs[] = {
        "Output Error",
        "Input Error",
        "Disk Full",
        "Index Out-Of-Bounds"
    };

    // вернуть объект типа String в виде сообщения об ошибке
    String getErrorMsg(int i) {
        if(i >= 0 & i < msgs.length)
            return msgs[i];
        else
            return "Invalid Error Code";
    }
}

class ErrMsg {
    public static void main(String args[]) {
        ErrorMsg err = new ErrorMsg();

        System.out.println(err.getErrorMsg(2));
        System.out.println(err.getErrorMsg(19));
    }
}
```

Выполнение этой программы дает следующий результат:

```
Disk Full
Invalid Error Code
```

Разумеется, возвращать можно и объекты создаваемых классов. Например, приведенный ниже фрагмент кода представляет собой переработанную версию предыдущей программы, где создаются два класса формирования ошибок `Err` и `ErrorInfo`. В классе `Err`, помимо кода ошибки, инкапсулируется символьная строка описания ошибки. А в классе `ErrorInfo` содержится метод `getErrorInfo()`, возвращающий объект типа `Err`.

```
// Возврат объекта, определяемого разработчиком программы.
```

```
class Err {
    String msg;    // Сообщение об ошибке
    int severity; // Код, определяющий серьезность ошибки

    Err(String m, int s) {
        msg = m;
        severity = s;
    }
}

class ErrorInfo {
    String msgs[] = {
        "Output Error",
        "Input Error",
        "Disk Full",
        "Index Out-Of-Bounds"
    };
    int howbad[] = { 3, 3, 2, 4 };

    // Возврат объекта типа Err.
    Err getErrorInfo(int i) {
        if(i >=0 & i < msgs.length)
            return new Err(msgs[i], howbad[i]);
        else
            return new Err("Invalid Error Code", 0);
    }
}

class ErrInfo {
    public static void main(String args[]) {
        ErrorInfo err = new ErrorInfo();
        Err e;

        e = err.getErrorInfo(2);
        System.out.println(e.msg + " severity: " + e.severity);

        e = err.getErrorInfo(19);
        System.out.println(e.msg + " severity: " + e.severity);
    }
}
```

Результат выполнения данной версии программы выглядит следующим образом:

```
Disk Full severity: 2
Invalid Error Code severity: 0
```


При каждом вызове метода `getErrorInfo()` создается новый объект типа `Err` и ссылка на него возвращается вызывающему методу. Этот объект затем используется методом `main()` для отображения кода серьезности ошибки и текстового сообщения.

Объект, возвращенный методом, существует до тех пор, пока на него имеется хотя бы одна ссылка. Если ссылки на объект отсутствуют, он уничтожается системой “сборки мусора”. Поэтому при выполнении программы не возникает ситуация, когда объект разрушается лишь потому, что метод, в котором он был создан, завершился.

Перегрузка методов

В этом разделе речь пойдет об одном из самых интересных языковых средств Java — *перегрузке методов*. Несколько методов одного класса могут иметь одно и то же имя, отличаясь лишь набором параметров. Перегрузка методов является одним из способов реализации принципа полиморфизма в Java.

Для того чтобы перегрузить метод, достаточно объявить его новый вариант, отличающийся от уже существующих, а все остальное сделает компилятор. Нужно лишь соблюсти одно условие: тип и/или число параметров в каждом из перегружаемых методов должны быть разными. Некоторые считают, что два метода могут отличаться лишь типом возвращаемого значения, но это заблуждение. Возвращаемый тип не предоставляет достаточных сведений для принятия решения о том, какой именно метод должен быть использован. Конечно, перегружаемые методы *могут* иметь разные возвращаемые типы, но при вызове метода выполняется лишь тот его вариант, в котором параметры соответствуют передаваемым аргументам.

Ниже приведен простой пример программы, демонстрирующий перегрузку методов.

```
// Перегрузка методов.
class Overload {
    // Первый вариант метода.
    void ovlDemo() {
        System.out.println("No parameters");
    }

    // перегрузить метод ovlDemo с одним параметром типа int.
    // Второй вариант метода.
    void ovlDemo(int a) {
        System.out.println("One parameter: " + a);
    }

    // перегрузить метод ovlDemo с двумя параметрами типа int.
    // Третий вариант метода.
    int ovlDemo(int a, int b) {
        System.out.println("Two parameters: " + a + " " + b);
        return a + b;
    }

    // перегрузить метод ovlDemo с двумя параметрами типа double.
    // Четвертый вариант метода.
    double ovlDemo(double a, double b) {
        System.out.println("Two double parameters: " +
            a + " " + b);
        return a + b;
    }
}
```

```

}

class OverloadDemo {
    public static void main(String args[]) {
        Overload ob = new Overload();
        int resI;
        double resD;

        // вызвать все варианты метода ovlDemo()
        ob.ovlDemo();
        System.out.println();

        ob.ovlDemo(2);
        System.out.println();

        resI = ob.ovlDemo(4, 6);
        System.out.println("Result of ob.ovlDemo(4, 6): " +
                           resI);
        System.out.println();

        resD = ob.ovlDemo(1.1, 2.32);
        System.out.println("Result of ob.ovlDemo(1.1, 2.32): " +
                           resD);
    }
}

```

Выполнение этой программы дает следующий результат:

No parameters

One parameter: 2

Two parameters: 4 6

Result of ob.ovlDemo(4, 6): 10

Two double parameters: 1.1 2.32

Result of ob.ovlDemo(1.1, 2.32): 3.42

Как видите, метод `ovlDemo()` перегружается четырежды. В первом его варианте параметры не предусмотрены, во втором — определен один целочисленный параметр, в третьем — два целочисленных параметра, в четвертом — два параметра типа `double`. Обратите внимание на то, что первые два варианта метода `ovlDemo()` имеют тип `void`, а два другие возвращают значение. Как пояснялось ранее, тип возвращаемого значения не учитывается при перегрузке методов. Следовательно, попытка определить два варианта метода `ovlDemo()` так, как показано ниже, приводит к ошибке.

```
// Возможен лишь один вариант метода ovlDemo(int).
```

```
// Возвращаемое значение нельзя использовать
```

```
// для различения перегружаемых методов.
```

```
void ovlDemo(int a) {
    System.out.println("One parameter: " + a);
}
```

```
/* Ошибка! Два варианта метода ovlDemo(int) не могут существовать,
   даже если типы возвращаемых ими значений отличаются.
```

```

*/
int ovlDemo(int a) {
    System.out.println("One parameter: " + a);
    return a * a;
}

```

Как поясняется в комментариях к приведенному выше фрагменту кода, отличия возвращаемых типов недостаточно для перегрузки методов.

Как следует из главы 2, в Java производится автоматическое приведение типов. Это приведение распространяется и на типы параметров перегружаемых методов. В качестве примера рассмотрим следующий фрагмент кода:

```

/* Автоматическое преобразование типов может оказывать влияние
   на выбор перегружаемого метода.
*/
class Overload2 {
    void f(int x) {
        System.out.println("Inside f(int): " + x);
    }

    void f(double x) {
        System.out.println("Inside f(double): " + x);
    }
}

class TypeConv {
    public static void main(String args[]) {
        Overload2 ob = new Overload2();

        int i = 10;
        double d = 10.1;
        byte b = 99;
        short s = 10;
        float f = 11.5F;

        ob.f(i); // Вызов метода ob.f(int)
        ob.f(d); // Вызов метода ob.f(double)
        ob.f(b); // Вызов метода ob.f(int) с преобразованием типов
        ob.f(s); // Вызов метода ob.f(int) с преобразованием типов
        ob.f(f); // Вызов метода ob.f(double) с преобразованием типов
    }
}

```

Выполнение этого фрагмента кода дает следующий результат:

```

Inside f(int): 10
Inside f(double): 10.1
Inside f(int): 99
Inside f(int): 10
Inside f(double): 11.5

```

В данном примере определены только два варианта метода `f()`: один принимает параметр типа `int`, а второй — параметр типа `double`. Но передать методу `f()` можно также значение типа `byte`, `short` или `float`. Значения типа `byte` и `short` исполняющая система Java автоматически преобразует в тип `int`. В результате будет вызван вариант

метода `f(int)`. А если параметр имеет значение типа `float`, то оно преобразуется в тип `double` и далее вызывается вариант метода `f(double)`.

Важно понимать, что автоматическое преобразование типов выполняется лишь в отсутствие прямого соответствия типов параметра и аргумента. В качестве примера ниже представлена другая версия предыдущей программы, в которой добавлен вариант метода `f()` с параметром типа `byte`.

```
// Добавление варианта метода f(byte).
class Overload2 {
    void f(byte x) {
        System.out.println("Inside f(byte): " + x);
    }

    void f(int x) {
        System.out.println("Inside f(int): " + x);
    }

    void f(double x) {
        System.out.println("Inside f(double): " + x);
    }
}

class TypeConv {
    public static void main(String args[]) {
        Overload2 ob = new Overload2();

        int i = 10;
        double d = 10.1;

        byte b = 99;
        short s = 10;
        float f = 11.5F;

        ob.f(i); // Вызов метода ob.f(int)
        ob.f(d); // Вызов метода ob.f(double)

        ob.f(b); // Вызов метода ob.f(byte) без преобразования типов

        ob.f(s); // Вызов метода ob.f(int) с преобразованием типов
        ob.f(f); // Вызов метода ob.f(double) с преобразованием типов
    }
}
```

Выполнение этой версии программы дает следующий результат:

```
Inside f(int): 10
Inside f(double): 10.1
Inside f(byte): 99
Inside f(int): 10
Inside f(double): 11.5
```

В данной версии программы используется вариант метода `f()` с параметром типа `byte`. Так, если при вызове метода `f()` ему передается значение типа `byte`, вызывается вариант метода `f(byte)` и автоматическое приведение к типу `int` не производится.

Перегрузка методов представляет собой механизм воплощения полиморфизма, т.е. способ реализации в Java принципа “один интерфейс — множество методов”. Для того чтобы стало понятнее, как и для чего это делается, необходимо принять во внимание следующее соображение: в языках программирования, не поддерживающих перегрузку методов, каждый метод должен иметь уникальное имя. Но в ряде случаев требуется выполнять одну и ту же последовательность операций над разными типами данных. В качестве примера рассмотрим функцию, определяющую абсолютное значение. В языках, не поддерживающих перегрузку методов, приходится создавать три или более варианта данной функции с именами, отличающимися хотя бы одним символом. Например, в языке C функция `abs()` возвращает абсолютное значение числа типа `int`, функция `labs()` — абсолютное значение числа типа `long`, а функция `fabs()` применяется к значению с плавающей точкой. А поскольку в C не поддерживается перегрузка, то каждая из функций должна иметь свое собственное имя, несмотря на то, что все они выполняют одинаковые действия. Это приводит к неоправданному усложнению процесса написания программ. Разработчику приходится не только представлять себе действия, выполняемые функциями, но и помнить все три их имени. Такая ситуация не возникает в Java, потому что все методы, вычисляющие абсолютное значение, имеют одно и то же имя. В стандартной библиотеке Java для вычисления абсолютного значения предусмотрен метод `abs()`. Его перегрузка осуществляется в классе `Math` для обработки значений всех числовых типов. Решение о том, какой именно вариант метода `abs()` должен быть вызван, исполняющая система Java принимает, исходя из типа аргумента.

Главная ценность перегрузки заключается в том, что она обеспечивает доступ к связанным вместе методам по общему имени. Следовательно, имя `abs` обозначает *общее* выполняемое действие, а компилятор сам выбирает *конкретный* вариант метода по обстоятельствам. Благодаря полиморфизму несколько имен сводятся к одному. Несмотря на всю простоту рассматриваемого здесь примера, продемонстрированный в нем принцип полиморфизма можно расширить, чтобы выяснить, каким образом перегрузка помогает справляться с более сложными ситуациями в программировании.

Когда метод перегружается, каждый его вариант может выполнять какое угодно действие. Для установления взаимосвязи между перегружаемыми методами не существует какого-то твердого правила, но с точки зрения правильного стиля программирования перегрузка методов подразумевает подобную взаимосвязь. Следовательно, использовать одно и то же имя для несвязанных друг с другом методов не следует, хотя это и возможно. Например, имя `sqr` можно было бы выбрать для методов, возвращающих квадрат и квадратный корень числа с плавающей точкой. Но ведь это принципиально разные операции. Такое применение перегрузки методов противоречит ее первоначальному назначению. На практике перегружать следует только тесно связанные операции.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Программирующие на Java употребляют термин *сигнатура*. Что это такое?

ОТВЕТ. Применительно к Java сигнатура обозначает имя метода и список его параметров. При перегрузке методов действует следующее правило: никакие два метода из одного класса не могут иметь одинаковые сигнатуры. Следует, однако, иметь в виду, что сигнатура не включает в себя тип возвращаемого значения, поскольку он не используется в Java при принятии решения о перегрузке.

Перегрузка конструкторов

Как и методы, конструкторы также могут перегружаться. Это дает возможность конструировать объекты самыми разными способами. В качестве примера рассмотрим следующую программу:

```
// Демонстрация перегрузки конструкторов.
class MyClass {
    int x;

    // Конструкторы перегружаются разными способами.
    MyClass() {
        System.out.println("Inside MyClass().");
        x = 0;
    }

    MyClass(int i) {
        System.out.println("Inside MyClass(int).");
        x = i;
    }

    MyClass(double d) {
        System.out.println("Inside MyClass(double).");
        x = (int) d;
    }

    MyClass(int i, int j) {
        System.out.println("Inside MyClass(int, int).");
        x = i * j;
    }
}

class OverloadConsDemo {
    public static void main(String args[]) {
        MyClass t1 = new MyClass();
        MyClass t2 = new MyClass(88);
        MyClass t3 = new MyClass(17.23);
        MyClass t4 = new MyClass(2, 4);

        System.out.println("t1.x: " + t1.x);
        System.out.println("t2.x: " + t2.x);
        System.out.println("t3.x: " + t3.x);
        System.out.println("t4.x: " + t4.x);
    }
}
```

В результате выполнения этой программы получается следующий результат:

```
Inside MyClass().
Inside MyClass(int).
Inside MyClass(double).
Inside MyClass(int, int).
t1.x: 0
t2.x: 88
t3.x: 17
t4.x: 8
```

В данном примере конструктор `MyClass()` перегружается четырежды. Во всех вариантах этого конструктора объект типа `MyClass` строится по-разному. Конкретный вариант конструктора выбирается из тех параметров, которые указываются при выполнении оператора `new`. Перегружая конструктор класса, вы предоставляете пользователю созданного вами класса свободу в выборе способа конструирования объекта.

Перегрузка конструкторов чаще всего производится для того, чтобы дать возможность инициализировать один объект на основании другого объекта. Рассмотрим в качестве примера следующую программу, в которой класс `Summation` используется для вычисления суммы двух целочисленных значений.

```
// Инициализация одного объекта посредством другого.
class Summation {
    int sum;

    // построить объект из целочисленного значения
    Summation(int num) {
        sum = 0;
        for(int i=1; i <= num; i++)
            sum += i;
    }

    // Построение одного объекта из другого.
    Summation(Summation ob) {
        sum = ob.sum;
    }
}

class SumDemo {
    public static void main(String args[]) {
        Summation s1 = new Summation(5);
        Summation s2 = new Summation(s1);

        System.out.println("s1.sum: " + s1.sum);
        System.out.println("s2.sum: " + s2.sum);
    }
}
```

Выполнение этой программы дает следующий результат:

```
s1.sum: 15
s2.sum: 15
```

Как следует из приведенного выше примера, использование одного объекта при инициализации другого нередко оказывается вполне оправданным. В данном случае при конструировании объекта `s2` нет необходимости вычислять сумму. Даже если подобная инициализация не повышает быстродействие программы, зачастую удобно иметь конструктор, создающий копию объекта.

Пример для опробования 6.2.

Перегрузка конструктора класса `Queue`

Исходный файл: `QDemo2.java`

В этом проекте предстоит усовершенствовать класс `Queue`, добавив в него два дополнительных конструктора. В первом из них новая очередь будет конструироваться на основании уже существующей, а во втором — присваиваться начальные значения элементам очереди.

при ее конструировании. Как станет ясно в дальнейшем, добавление этих конструкторов сделает класс `Queue` более удобным для использования.

Последовательность действий

1. Создайте новый файл `QDemo2.java` и скопируйте в него код класса `Queue`, созданный в примере для опробования 6.1.
2. Добавьте сначала в этот класс приведенный ниже конструктор, который будет строить одну очередь на основании другой.

```
// Конструктор, строящий один объект типа Queue на основании другого.
Queue(Queue ob) {
    putloc = ob.putloc;
    getloc = ob.getloc;
    q = new char[ob.q.length];

    // копировать элементы очереди
    for(int i=getloc+1; i <= putloc; i++)
        q[i] = ob.q[i];
}
```

Внимательно проанализируем этот конструктор. Сначала переменные `putloc` и `getloc` инициализируются в нем значениями, содержащимися в объекте `ob`, который передается ему в качестве параметра. Затем в нем организуется новый массив для хранения элементов очереди, которые далее копируются из объекта `ob` в этот массив. Вновь созданная копия очереди будет идентична оригиналу, хотя они и являются совершенно отдельными объектами.

3. Добавьте в данный класс конструктор, инициализирующий очередь данными из символьного массива, как показано ниже.

```
// Конструирование и инициализация объекта типа Queue.
Queue(char a[]) {
    putloc = 0;
    getloc = 0;
    q = new char[a.length+1];

    for(int i = 0; i < a.length; i++) put(a[i]);
}
```

В этом конструкторе создается достаточно большая очередь для хранения символов из массива `a`. В силу особенностей алгоритма, реализующего очередь, длина очереди должна быть на один элемент больше, чем длина исходного массива.

4. Ниже приведен весь код видоизмененного класса `Queue`, а также код класса `QDemo2`, демонстрирующего организацию очереди для хранения символов и обращение с ней.

```
// Класс, реализующий очередь для хранения символов.
class Queue {
    private char q[]; // Массив для хранения элементов очереди.
    private int putloc, getloc; // Индексы размещения и извлечения
                                // элементов очереди.

    // сконструировать пустую очередь заданного размера
    Queue(int size) {
```



```

    q = new char[size+1]; // выделить память для очереди
    putloc = getloc = 0;
}

// сконструировать очередь из существующего объекта типа Queue
Queue(Queue ob) {
    putloc = ob.putloc;
    getloc = ob.getloc;
    q = new char[ob.q.length];

    // копировать элементы в очередь
    for(int i=getloc+1; i <= putloc; i++)
        q[i] = ob.q[i];
}

// сконструировать очередь из массива исходных значений
Queue(char a[]) {
    putloc = 0;
    getloc = 0;
    q = new char[a.length+1];

    for(int i = 0; i < a.length; i++) put(a[i]);
}

// поместить символ в очередь
void put(char ch) {
    if(putloc==q.length-1) {
        System.out.println(" - Queue is full.");
        return;
    }

    putloc++;
    q[putloc] = ch;
}

// извлечь символ из очереди
char get() {
    if(getloc == putloc) {
        System.out.println(" - Queue is empty.");
        return (char) 0;
    }

    getloc++;
    return q[getloc];
}

// продемонстрировать класс Queue в действии
class QDemo2 {
    public static void main(String args[]) {
        // построить пустую очередь для хранения 10 элементов
        Queue q1 = new Queue(10);
    }
}

```

```
char name[] = {'T', 'o', 'm'};
// построить очередь из массива
Queue q2 = new Queue(name);

char ch;
int i;

// поместить ряд символов в очередь q1
for(i=0; i < 10; i++)
    q1.put((char) ('A' + i));

// построить одну очередь из другой очереди
Queue q3 = new Queue(q1);

// показать очереди
System.out.print("Contents of q1: ");
for(i=0; i < 10; i++) {
    ch = q1.get();
    System.out.print(ch);
}

System.out.println("\n");

System.out.print("Contents of q2: ");
for(i=0; i < 3; i++) {
    ch = q2.get();
    System.out.print(ch);
}

System.out.println("\n");

System.out.print("Contents of q3: ");
for(i=0; i < 10; i++) {
    ch = q3.get();
    System.out.print(ch);
}
}
```

Результат выполнения данной программы выглядит следующим образом:

Contents of q1: ABCDEFGHIJ

Contents of q2: Tom

Contents of q3: ABCDEFGHIJ

Рекурсия

В Java допускается, чтобы метод вызывал самого себя. Этот процесс называется *рекурсией*, а метод, вызывающий самого себя, — *рекурсивным*. Вообще говоря, рекурсия представляет собой процесс, в ходе которого нечто определяет самое себя. В этом отношении она чем-то напоминает циклическое определение. Рекурсивный метод отлича-

ется в основном тем, что он содержит оператор, в котором этот метод вызывает самого себя. Рекурсия является эффективным механизмом управления программой.

Классическим примером рекурсии служит вычисление факториала числа. Факториал числа N представляет собой произведение всех целых чисел от 1 до N . Например, факториал числа 3 равен $1 \times 2 \times 3$, или 6. В приведенном ниже примере программы демонстрируется рекурсивный способ вычисления факториала числа. Для сравнения в эту программу включен также нерекурсивный вариант вычисления факториала числа.

```
// Простой пример рекурсии.
class Factorial {
    // Рекурсивный метод.
    int factR(int n) {
        int result;

        if(n==1) return 1;
        // Рекурсивный вызов метода factR().
        result = factR(n-1) * n;
        return result;
    }

    // Вариант программы, вычисляющий факториал итеративным способом.
    int factI(int n) {
        int t, result;

        result = 1;
        for(t=1; t <= n; t++) result *= t;
        return result;
    }
}

class Recursion {
    public static void main(String args[]) {
        Factorial f = new Factorial();

        System.out.println("Factorials using recursive method.");
        System.out.println("Factorial of 3 is " + f.factR(3));
        System.out.println("Factorial of 4 is " + f.factR(4));
        System.out.println("Factorial of 5 is " + f.factR(5));
        System.out.println();

        System.out.println("Factorials using iterative method.");
        System.out.println("Factorial of 3 is " + f.factI(3));
        System.out.println("Factorial of 4 is " + f.factI(4));
        System.out.println("Factorial of 5 is " + f.factI(5));
    }
}
```

Ниже приведен результат выполнения данной программы.

```
Factorials using recursive method.
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

```
Factorials using iterative method.
```

```
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

Действия нерекурсивного метода `factI()` не требуют особых пояснений. В нем используется цикл, в котором числа, начиная с 1, последовательно умножаются друг на друга, постепенно образуя произведение, дающее факториал.

Рекурсивный метод `factR()` действует несколько сложнее. Когда метод `factR()` вызывается с аргументом, равным 1, он возвращает 1, а иначе — произведение, определяемое из выражения `factR(n-1)*n`. Для вычисления этого выражения вызывается метод `factR()` с аргументом `n-1`. Этот процесс повторяется до тех пор, пока значение переменной `n` не окажется равным 1, после чего из предыдущих вызовов данного метода начнут возвращаться полученные значения. Например, при вычислении факториала 2 первый вызов метода `factR()` повлечет за собой второй вызов того же самого метода, но с аргументом 1. В результате метод возвратит значение 1, которое затем умножается на 2 (т.е. исходное значение переменной `n`). В результате всех этих вычислений будет получен факториал, равный 2. По желанию в тело метода `factR()` можно ввести операторы `println()`, чтобы сообщать, на каком именно уровне осуществляется очередной вызов, а также отображать промежуточные результаты вычислений.

Когда метод вызывает самого себя, в системном стеке распределяется память для новых локальных переменных и параметров, и код метода выполняется с этими новыми переменными и параметрами с самого начала. При рекурсивном вызове метода не создается его новая копия, но лишь используются его новые аргументы. А при возврате из каждого рекурсивного вызова старые локальные переменные и параметры извлекаются из стека, и выполнение возобновляется с точки вызова в методе. Рекурсивные методы можно сравнить по принципу действия с постепенно сжимающейся и затем распрямляющейся пружиной.

Рекурсивные варианты многих процедур могут выполняться немного медленнее, чем их итерационные эквиваленты, из-за дополнительных затрат системных ресурсов на неоднократные вызовы метода. Если же таких вызовов окажется слишком много, то в конечном итоге может быть переполнен системный стек. А поскольку параметры и локальные переменные рекурсивного метода хранятся в системном стеке и при каждом новом вызове этого метода создается их новая копия, то в какой-то момент стек может оказаться исчерпанным. Если возникнет подобная ситуация, исполняющая система Java сгенерирует исключение. Но в большинстве случаев об этом не стоит особенно беспокоиться. Как правило, переполнение системного стека происходит тогда, когда рекурсивный метод выходит из под контроля.

Главное преимущество рекурсии заключается в том, что она позволяет реализовать некоторые алгоритмы яснее и проще, чем итерационным способом. Например, алгоритм быстрой сортировки довольно трудно реализовать итерационным способом. А некоторые задачи, например искусственного интеллекта, очевидно, требуют именно рекурсивного решения. При написании рекурсивных методов следует непременно указать в соответствующем месте условный оператор, например `if`, чтобы организовать возврат из метода без рекурсии. В противном случае возврат из вызванного однажды рекурсивного метода может вообще не произойти. Подобного рода ошибка весьма характерна для реализации рекурсии в практике программирования. Поэтому рекомендуется пользоваться операторами, содержащими вызовы метода `println()`, чтобы следить за происходящим в рекурсивном методе и прервать его выполнение, если в нем обнаружится ошибка.

Применение ключевого слова `static`

Иногда требуется определить такой член класса, который будет использоваться независимо от всех остальных объектов этого класса. Как правило, доступ к члену класса организуется посредством объекта этого класса, но в то же время можно создать член класса для самостоятельного применения без ссылки на конкретный экземпляр объекта. Для того чтобы создать такой член класса, достаточно указать в самом начале его объявления ключевое слово `static`. Если член класса объявляется как `static`, он становится доступным до создания любых объектов своего класса и без ссылки на какой-нибудь объект. С помощью ключевого слова `static` можно объявлять как переменные, так и методы. Наиболее характерным примером члена типа `static` служит метод `main()`, который объявляется таковым потому, что он должен вызываться виртуальной машиной Java в самом начале выполняемой программы.

Для того чтобы воспользоваться членом типа `static` за пределами класса, достаточно указать имя этого класса с оператором-точкой. Но создавать объект для этого не нужно. В действительности член типа `static` оказывается доступным не по ссылке на объект, а по имени своего класса. Так, если требуется присвоить значение 10 переменной `count` типа `static`, являющейся членом класса `Timer`, то для этой цели можно воспользоваться следующей строкой кода:

```
Timer.count = 10;
```

Эта форма записи подобна той, что используется для доступа к обычным переменным экземпляра посредством объекта, но в ней указывается имя класса, а не объекта. Аналогичным образом можно вызвать метод типа `static`, используя имя класса и оператор-точку.

Переменные, объявляемые как `static`, по существу являются глобальными. Когда же объекты объявляются в своем классе, копия переменной типа `static` не создается. Вместо этого все экземпляры класса совместно пользуются одной и той же переменной типа `static`. Ниже приведен пример программы, демонстрирующий отличия переменной, объявленной как `static`, от обычной переменной экземпляра.

```
// Применение статической переменной.
class StaticDemo {
    int x; // обычная переменная экземпляра
    static int y; // статическая переменная — это одна копия,
                // совместно используемая всеми объектами.

    // вернуть сумму значений переменной экземпляра x и
    // статической переменной y.
    int sum() {
        return x + y;
    }
}

class SDemo {
    public static void main(String args[]) {
        StaticDemo ob1 = new StaticDemo();
        StaticDemo ob2 = new StaticDemo();

        // У каждого объекта имеется своя копия переменной экземпляра.
        ob1.x = 10;
        ob2.x = 20;
```

```

System.out.println("Of course, ob1.x and ob2.x " +
                  "are independent.");
System.out.println("ob1.x: " + ob1.x +
                  "\nob2.x: " + ob2.x);
System.out.println();

// Все объекты совместно пользуются одной общей
// копией статической переменной.
System.out.println("The static variable y is shared.");
StaticDemo.y = 19;
System.out.println("Set StaticDemo.y to 19.");

System.out.println("ob1.sum(): " + ob1.sum());
System.out.println("ob2.sum(): " + ob2.sum());
System.out.println();

StaticDemo.y = 100;
System.out.println("Change StaticDemo.y to 100");

System.out.println("ob1.sum(): " + ob1.sum());
System.out.println("ob2.sum(): " + ob2.sum());
System.out.println(); }

```

Выполнение этой программы дает следующий результат:

Of course, ob1.x and ob2.x are independent.

ob1.x: 10

ob2.x: 20

The static variable y is shared.

Set StaticDemo.y to 19.

ob1.sum(): 29

ob2.sum(): 39

Change StaticDemo.y to 100

ob1.sum(): 110

ob2.sum(): 120

Нетрудно заметить, что статическая переменная `y` используется как объектом `ob1`, так и объектом `ob2`. Изменения в ней оказывают влияние на весь класс, а не только на его экземпляр.

Метод типа `static` отличается от обычного метода тем, что его можно вызывать по имени его класса, не создавая экземпляр объекта этого класса. Пример такого вызова уже приводился ранее. Это был метод `sqrt()` типа `static`, относящийся к классу `Math` из стандартной библиотеки классов Java. Ниже приведен пример программы, в которой объявляется статическая переменная и создается метод типа `static`.

```

// Применение статического метода.
class StaticMeth {
    static int val = 1024; // статическая переменная

    // Статический метод.
    static int valDiv2() {
        return val/2;
    }
}

```

```

    }
}

class SDemo2 {
    public static void main(String args[]) {

        System.out.println("val is " + StaticMeth.val);
        System.out.println("StaticMeth.valDiv2(): " +
            StaticMeth.valDiv2());

        StaticMeth.val = 4;
        System.out.println("val is " + StaticMeth.val);
        System.out.println("StaticMeth.valDiv2(): " +
            StaticMeth.valDiv2());
    }
}

```

Выполнение этой программы дает следующий результат:

```

val is 1024
StaticMeth.valDiv2(): 512
val is 4
StaticMeth.valDiv2(): 2

```

На применение методов типа `static` накладывается ряд следующих ограничений.

- В методе типа `static` допускается непосредственный вызов только других методов типа `static`.
- Для метода типа `static` непосредственно доступными оказываются только другие данные типа `static`, определенные в его классе.
- В методе типа `static` должна отсутствовать ссылка `this`.

В приведенном ниже классе код статического метода `valDivDenom()` создан некорректно.

```

class StaticError {
    int denom = 3;           // обычная переменная экземпляра
    static int val = 1024;   // статическая переменная

    /* Ошибка! К нестатическим переменным нельзя обращаться
       из статического метода. */
    static int valDivDenom() {
        return val/denom; // не подлежит компиляции!
    }
}

```

В данном примере `denom` является обычной переменной экземпляра, к которой нельзя обращаться из статического метода.

Статические блоки

Иногда для подготовки к созданию объектов в классе должны быть выполнены некоторые инициализирующие действия. В частности, может возникнуть потребность установить соединение с удаленным сетевым узлом или задать значения некоторых статических переменных перед тем, как воспользоваться статическими методами класса. Для

решения подобных задач в Java предусмотрены *статические блоки*. Статический блок выполняется при первой загрузке класса, еще до того, как класс будет использован для каких-нибудь других целей. Ниже приведен пример применения статического блока.

```
// Применение статического блока.
class StaticBlock {
    static double rootOf2;
    static double rootOf3;

    // Этот блок выполняется при загрузке класса.
    static {
        System.out.println("Inside static block.");
        rootOf2 = Math.sqrt(2.0);
        rootOf3 = Math.sqrt(3.0);
    }

    StaticBlock(String msg) {
        System.out.println(msg);
    }
}

class SDemo3 {
    public static void main(String args[]) {
        StaticBlock ob = new StaticBlock("Inside Constructor");

        System.out.println("Square root of 2 is " +
            StaticBlock.rootOf2);
        System.out.println("Square root of 3 is " +
            StaticBlock.rootOf3);
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
Inside static block.
Inside Constructor
Square root of 2 is 1.4142135623730951
Square root of 3 is 1.7320508075688772
```

Как видите, статический блок выполняется еще до того, как будет создан какой-нибудь объект.

Пример для опробования 6.3.

Быстрая сортировка

Исходный файл: QSDemo.java

В главе 5 был рассмотрен простой способ так называемой пузырьковой сортировки, а кроме него, вкратце упоминались и более совершенные способы сортировки. В этом проекте предстоит реализовать один из самых лучших способов: быструю сортировку. Алгоритм быстрой сортировки был разработан Ч. Хоаром и назван его именем. На сегодняшний день это самый лучший универсальный алгоритм сортировки. Он не был продемонстрирован в главе 5 лишь потому, что реализовать быструю сортировку лучше всего с помощью рекурсии. В данном проекте будет создана программа для сортировки символьного массива, но демонстрируемый подход может быть применен к сортировке любых объектов.

Быстрая сортировка опирается на принцип разделения. Сначала выбирается опорное значение (так называемый *компаранд*), и массив разделяется на две части. Все элементы, которые больше или равны разделяемому компаранду, помещаются в одну часть массива, а те элементы, которые меньше компаранда, — в другую часть. Затем процесс рекурсивно повторяется для каждой оставшейся части до тех пор, пока массив не окажется отсортированным. Допустим, имеется массив, содержащий последовательность символов fedacb, а в качестве компаранда выбран символ d. На первом проходе массив будет частично упорядочен следующим образом:

Исходные данные	f e d a c b
Проход 1	b c a d e f

Этот процесс повторяется для каждой части: bca и def. Как видите, процесс рекурсивен по своей сути, поэтому рекурсивный способ лучше всего подходит для реализации быстрой сортировки.

Компаранд можно выбрать двумя способами: случайно либо вычислив среднее значение части элементов массива. Эффективность сортировки окажется оптимальной в том случае, когда компаранд выбран как раз посередине диапазона значений элементов, содержащихся в массиве, но зачастую выбрать такое значение непросто. Если же выбрать компаранд случайным образом, то вполне возможно, что он окажется на краю диапазона. Но и в этом случае алгоритм быстрой сортировки будет действовать правильно. В том варианте быстрой сортировки, который реализуется в данном проекте, в качестве компаранда выбирается элемент, находящийся посередине массива.

Последовательность действий

1. Создайте новый файл QSDemo.java.
2. Создайте сначала класс Quicksort, код которого приведен ниже.

```
// Пример для опробования 6.3. Простая версия класса Quicksort,
// реализующего быструю сортировку.
class Quicksort {

    // организовать вызов конкретного метода быстрой сортировки
    static void qsort(char items[]) {
        qs(items, 0, items.length-1);
    }

    // Рекурсивная версия метода быстрой сортировки символов.
    private static void qs(char items[], int left, int right)
    {
        int i, j;
        char x, y;

        i = left; j = right;
        x = items[(left+right)/2];

        do {
            while((items[i] < x) && (i < right)) i++;
            while((x < items[j]) && (j > left)) j--;

            if(i <= j) {
```

```

        y = items[i];
        items[i] = items[j];
        items[j] = y;
        i++; j--;
    }
    while(i <= j);

    if(left < j) qs(items, left, j);
    if(i < right) qs(items, i, right);
}
}

```

Для упрощения интерфейса в классе `Quicksort` предоставляется метод `qsort()`, из которого вызывается метод `qs()`, фактически выполняющий сортировку. Такой подход позволяет выполнять сортировку, передавая методу лишь имя массива и не осуществляя первоначальное разделение. А поскольку метод `qs()` используется только в классе, он определяется как `private`.

3. Для того чтобы воспользоваться классом `Quicksort`, достаточно вызвать метод `Quicksort.qsort()`. Этот метод определен как `static`, и поэтому для его вызова достаточно указать имя класса, а создавать объект не обязательно. По завершении работы этого метода массив будет отсортирован. Данная версия программы работает только с символьными массивами, но вы можете адаптировать ее для сортировки массивов любого типа.
4. Ниже приведен весь исходный код программы, демонстрирующей применение класса `Quicksort`.

```

// Пример для опробования 6.3. Простая версия класса Quicksort,
// реализующего быструю сортировку.
class Quicksort {

    // организовать вызов конкретного метода быстрой сортировки
    static void qsort(char items[]) {
        qs(items, 0, items.length-1);
    }

    // Рекурсивная версия метода быстрой сортировки символов.
    private static void qs(char items[], int left, int right)
    {
        int i, j;
        char x, y;

        i = left; j = right;
        x = items[(left+right)/2];

        do {
            while((items[i] < x) && (i < right)) i++;
            while((x < items[j]) && (j > left)) j--;

            if(i <= j) {
                y = items[i];
                items[i] = items[j];
                items[j] = y;
                i++; j--;
            }
        }
    }
}

```

```

        } while(i <= j);

        if(left < j) qs(items, left, j);
        if(i < right) qs(items, i, right);
    }
}

class QSDemo {
    public static void main(String args[]) {
        char a[] = { 'd', 'x', 'a', 'r', 'p', 'j', 'i' };
        int i;

        System.out.print("Original array: ");
        for(i=0; i < a.length; i++)
            System.out.print(a[i]);

        System.out.println();

        // отсортировать массив
        Quicksort.qsort(a);

        System.out.print("Sorted array: ");
        for(i=0; i < a.length; i++)
            System.out.print(a[i]);
    }
}

```

Вложенные и внутренние классы

В Java определены *вложенные классы*. Вложенным называется такой класс, который объявляется в другом классе. Вложенные классы не относятся к базовым языковым средствам Java. Они даже не поддерживались до появления версии Java 1.1, хотя с тех пор часто применяются в реальных программах, и поэтому о них нужно знать.

Вложенный класс не может существовать независимо от объемлющего класса, потому что последний ограничивает область его действия. Если вложенный класс объявлен в пределах области действия объемлющего класса, он становится членом последнего. Имеется также возможность объявить вложенный класс, который станет локальным в пределах блока.

Существуют два типа вложенных классов. Одни вложенные классы объявляются с помощью модификатора доступа `static`, а другие — без него. В этой книге будет рассматриваться только нестатический вариант вложенных классов. Классы такого типа называются *внутренними*. Внутренний класс имеет доступ ко всем переменным и методам внешнего (т.е. объемлющего) класса и может обращаться к ним непосредственно, как и все остальные нестатические члены внешнего класса. Иногда внутренний класс используется для предоставления услуг объемлющему классу. Ниже приведен пример применения внутреннего класса для вычисления различных значений в объемлющем его классе.

```

// Применение внутреннего класса.
class Outer {
    int nums[];

```

```

Outer(int n[]) {
    nums = n;
}

void Analyze() {
    Inner inOb = new Inner();

    System.out.println("Minimum: " + inOb.min());
    System.out.println("Maximum: " + inOb.max());
    System.out.println("Average: " + inOb.avg());
}

// Внутренний класс.
class Inner {
    int min() {
        int m = nums[0];

        for(int i=1; i < nums.length; i++)
            if(nums[i] < m) m = nums[i];
        return m;
    }

    int max() {
        int m = nums[0];
        for(int i=1; i < nums.length; i++)
            if(nums[i] > m) m = nums[i];

        return m;
    }

    int avg() {
        int a = 0;
        for(int i=0; i < nums.length; i++)
            a += nums[i];

        return a / nums.length;
    }
}

class NestedClassDemo {
    public static void main(String args[]) {
        int x[] = { 3, 2, 1, 5, 6, 9, 7, 8 };
        Outer outOb = new Outer(x);

        outOb.Analyze();
    }
}

```

Результат выполнения данной программы выглядит следующим образом:

```

Minimum: 1
Maximum: 9
Average: 5

```

В данном примере внутренний класс `Inner` обрабатывает массив `nums`, являющийся членом класса `Outer`. Как упоминалось выше, вложенный класс имеет доступ к членам объемлющего класса, и поэтому он может непосредственно обращаться к массиву `nums`. А вот обратное не справедливо. Так, например, метод `analyze()` не может непосредственно вызвать метод `min()`, не создав объект типа `Inner`.

Как упоминалось выше, класс можно вложить в области действия блока. В итоге получается локальный класс, недоступный за пределами блока. В следующем примере программы класс `ShowBits`, созданный в примере для опробования 5.3, преобразуется таким образом, чтобы стать локальным.

```
// Применение класса ShowBits в качестве локального.
class LocalClassDemo {
    public static void main(String args[]) {

        // Внутренний вариант класса ShowBits.
        // Локальный класс, вложенный в метод.
        class ShowBits {
            int numbits;

            ShowBits(int n) {
                numbits = n;
            }

            void show(long val) {
                long mask = 1;

                // сдвинуть влево для установки единицы в нужной позиции
                mask <<= numbits-1;

                int spacer = 0;
                for(; mask != 0; mask >>= 1) {
                    if((val & mask) != 0) System.out.print("1");
                    else System.out.print("0");
                    spacer++;
                    if((spacer % 8) == 0) {
                        System.out.print(" ");
                        spacer = 0;
                    }
                }
                System.out.println();
            }
        }

        for(byte b = 0; b < 10; b++) {
            ShowBits byteval = new ShowBits(8);

            System.out.print(b + " in binary: ");
            byteval.show(b);
        }
    }
}
```

Выполнение этой программы дает следующий результат:

```

0 in binary: 00000000
1 in binary: 00000001
2 in binary: 00000010
3 in binary: 00000011
4 in binary: 00000100
5 in binary: 00000101
6 in binary: 00000110
7 in binary: 00000111
8 in binary: 00001000
9 in binary: 00001001

```

В данном примере класс `ShowBits` недоступен за пределами метода `main()`, а следовательно, попытка получить доступ к нему из любого метода, кроме `main()`, приведет к ошибке.

И последнее замечание: внутренний класс может быть безымянным. Экземпляр *безымянного*, или *анонимного*, внутреннего класса создается при объявлении класса с помощью оператора `new`. Безымянные внутренние классы будут подробнее рассмотрены в главе 15.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Чем статический вложенный класс отличается от нестатического?

ОТВЕТ. Статический вложенный класс объявляется с помощью модификатора `static`. Являясь статическим, он может непосредственно обращаться к любому статическому члену объемлющего класса. А другие члены объемлющего класса доступны ему посредством ссылки на объект.

Аргументы переменной длины

Иногда оказываются полезными методы, способные принимать переменное число аргументов. Например, методу, устанавливающему соединение с Интернетом, могут понадобиться имя и пароль пользователя, имя файла, протокол и другие параметры. Если при вызове метода какие-нибудь из этих данных не указаны, то должны использоваться значения по умолчанию. В таком случае было бы уместнее передавать только те аргументы, для которых не предусмотрены значения по умолчанию. А для этого требуется метод, поддерживающий список аргументов переменной, не фиксированной длины.

Раньше для поддержки переменного числа аргументов применялись два способа, причем оба были далеки от совершенства. Первый способ состоял в следующем: если максимальное число аргументов невелико и известно заранее, то можно создать разные варианты одного метода. Очевидно, что такой подход применим лишь в отдельных случаях. Если же значений оказывалось слишком много или их максимальное количество не было известно заранее, то применялся второй способ: параметры помещались в массив, который и передавался методу. Обоим этим способам присущи определенные недостатки, и со временем стало ясно, что требуется какой-то другой, более совершенный подход к решению данной задачи.

В версии JDK 5 появились языковые средства Java, упрощающие создание методов, которым требуется переменное число аргументов. Эти средства называются *аргументами переменной длины*, а метод, принимающий переменное число аргументов, — *методом переменной аргументности*, или же *методом с аргументами переменной длины*. Список параме-

тров, соответствующих аргументам переменной длины, имеет не фиксированную, а переменную длину. Поэтому метод с аргументами переменной длины может принимать произвольно изменяющееся число аргументов.

Общие положения об аргументах переменной длины

Для указания на то, что метод может принимать переменное число аргументов, в его объявление включается многоточие (...). Ниже приведен пример метода `vaTest()`, принимающего переменное число аргументов.

```
// Метод vaTest() с аргументами переменной длины.
// Объявление списка аргументов переменной длины.
static void vaTest(int ... v) {
    System.out.println("Number of args: " + v.length);
    System.out.println("Contents: ");

    for(int i=0; i < v.length; i++)
        System.out.println(" arg " + i + ": " + v[i]);

    System.out.println();
}
```

Обратите внимание на следующий синтаксис объявления аргумента `v`:

```
int ... v
```

Этот синтаксис сообщает компилятору, что метод `vaTest()` может вызываться с указанием произвольного числа аргументов, в том числе и совсем без них. Более того, аргумент `v` неявно объявляется как массив типа `int[]`. А в теле метода `vaTest()` доступ к аргументу `v` осуществляется с помощью обычного синтаксиса обращения к массивам.

Ниже приведен весь исходный код примера программы, демонстрирующего метод `vaTest()` в действии.

```
// Демонстрация метода с аргументами переменной длины.
class VarArgs {

    // Метод vaTest() с аргументами переменной длины.
    static void vaTest(int ... v) {
        System.out.println("Number of args: " + v.length);
        System.out.println("Contents: ");

        for(int i=0; i < v.length; i++)
            System.out.println(" arg " + i + ": " + v[i]);

        System.out.println();
    }

    public static void main(String args[])
    {
        // Метод vaTest() может вызываться с переменным числом аргументов.
        vaTest(10);           // 1 аргумент
        vaTest(1, 2, 3);      // 3 аргумента
        vaTest();             // без аргументов
    }
}
```

Выполнение этой программы дает следующий результат:

```
Number of args: 1
Contents:
arg 0: 10
```

```
Number of args: 3
Contents:
arg 0: 1
arg 1: 2
arg 2: 3
```

```
Number of args: 0
Contents:
```

В приведенной выше программе обращает на себя внимание следующее. Во-первых, как пояснялось выше, обращение к аргументу *v* в методе *vaTest()* осуществляется как к массиву. Дело в том, что он действительно является *массивом*. Многоточие в объявлении этого метода указывает компилятору на использование переменного числа аргументов и на необходимость поместить их в массив *v*. Во-вторых, при обращении к методу *vaTest()* в методе *main()* указывается разное число аргументов, включая и вызов данного метода вообще без аргументов. Указываемые аргументы автоматически помещаются в массив *v*. Если же аргументы не указаны, длина этого массива будет равна нулю.

Помимо аргумента переменной длины, в методе можно также указывать и обычные аргументы, но при одном условии: аргумент переменной длины должен быть указан последним. Например, приведенное ниже объявление метода является вполне допустимым.

```
int doIt(int a, int b, double c, int ... vals) {
```

В данном случае первые три аргумента, передаваемые при вызове метода *doIt()*, будут соответствовать первым трем параметрам. А остальные аргументы будут считаться относящимися к параметру переменной длины *vals*.

Ниже приведен переработанный вариант метода *vaTest()*, в котором используются как обычные аргументы, так и аргументы переменной длины.

```
// Использование аргументов переменной длины
// вместе с обычными аргументами.
class VarArgs2 {

    // Здесь msg - обычный аргумент,
    // а v - аргумент переменной длины.
    static void vaTest(String msg, int ... v) {
        System.out.println(msg + v.length);
        System.out.println("Contents: ");

        for(int i=0; i < v.length; i++)
            System.out.println(" arg " + i + ": " + v[i]);

        System.out.println();
    }

    public static void main(String args[])
    {
        vaTest("One vararg: ", 10);
    }
}
```



```

    vaTest("Three varargs: ", 1, 2, 3);
    vaTest("No varargs: ");
}
}

```

Выполнение этого фрагмента кода дает следующий результат:

```

One vararg: 1
Contents:
arg 0: 10

```

```

Three varargs: 3
Contents:
arg 0: 1
arg 1: 2
arg 2: 3

```

```

No varargs: 0
Contents:

```

Не следует, однако, забывать, что аргумент переменной длины должен быть указан последним. Например, следующее объявление метода недопустимо:

```

int doIt(int a, int b, double c, int ... vals,
        boolean stopFlag) { // Ошибка!

```

В данном примере сделана попытка указать обычный аргумент после аргумента переменной длины.

Существует еще одно ограничение, которое следует соблюдать: аргументы переменной длины можно указать в методе только один раз. Например, приведенное ниже объявление метода составлено неверно.

```

int doIt(int a, int b, double c, int ... vals,
        double ... morevals) { // Ошибка!

```

Ошибкой в данном случае является попытка указать два разных типа аргументов переменной длины.

Перегрузка методов с аргументами переменной длины

Если требуется, то метод, принимающий переменное число аргументов, можно перегрузить. Например, в следующей программе трижды перегружается метод `vaTest()`:

```

// Перегрузка метода с аргументами переменной длины.
class VarArgs3 {

    // Первый вариант метода vaTest().
    static void vaTest(int ... v) {
        System.out.println("vaTest(int ...): " +
                           "Number of args: " + v.length);
        System.out.println("Contents: ");

        for(int i=0; i < v.length; i++)
            System.out.println(" arg " + i + ": " + v[i]);

        System.out.println();
    }
}

```

```

}

// Второй вариант метода vaTest().
static void vaTest(boolean ... v) {
    System.out.println("vaTest(boolean ...): " +
        "Number of args: " + v.length);
    System.out.println("Contents: ");

    for(int i=0; i < v.length; i++)
        System.out.println(" arg " + i + ": " + v[i]);

    System.out.println();
}

// Третий вариант метода vaTest().
static void vaTest(String msg, int ... v) {
    System.out.println("vaTest(String, int ...): " +
        msg + v.length);
    System.out.println("Contents: ");
    for(int i=0; i < v.length; i++)
        System.out.println(" arg " + i + ": " + v[i]);

    System.out.println();
}

public static void main(String args[])
{
    vaTest(1, 2, 3);
    vaTest("Testing: ", 10, 20);
    vaTest(true, false, false);
}
}

```

Выполнение этой программы дает следующий результат:

```

vaTest(int ...): Number of args: 3
Contents:
  arg 0: 1
  arg 1: 2
  arg 2: 3

vaTest(String, int ...): Testing: 2
Contents:
  arg 0: 10
  arg 1: 20

vaTest(boolean ...): Number of args: 3
Contents:
  arg 0: true
  arg 1: false
  arg 2: false

```

В приведенном выше примере программы демонстрируются два способа перегрузки методов с аргументами переменной длины. Во-первых, типы параметров аргументов длины у перегружаемых методов могут отличаться. Это демонстрируют варианты метода `vaTest(int ...)` и `vaTest(boolean ...)`. Напомним: многоточие обозначает, что соответствующий аргумент должен рассматриваться как массив указанного типа.

Следовательно, как и при перегрузке обычных методов указываются разные типы параметров, так и перегрузке методов с аргументами переменной длины задаются разные типы подобных аргументов. Исполняющая система Java использует эти данные для правильного выбора вызываемого метода.

Второй способ перегрузки методов с аргументами переменной длины состоит в добавлении обычных аргументов. Он реализован в варианте метода `vaTest(String, int ...)`. В этом случае исполняющая система Java использует для выбора нужного варианта метода данные как о числе параметров, так и об их типах.

Аргументы переменной длины и неоднозначность

При перегрузке методов с аргументами переменной длины может возникнуть довольно неожиданная ошибка. А возникает она вследствие неоднозначности в выборе метода. Рассмотрим в качестве примера следующую программу:

```
// Перегрузка метода с аргументами переменной длины
// и неоднозначность выбора метода.
//
// В этой программе имеется ошибка, и
// поэтому она не будет компилироваться.
class VarArgs4 {

    // Использование аргумента переменной длины типа int.
    static void vaTest(int ... v) {
        // ...
    }

    // Использование аргумента переменной длины типа boolean.
    static void vaTest(boolean ... v) {
        // ...
    }

    public static void main(String args[])
    {
        vaTest(1, 2, 3); // OK
        vaTest(true, false, false); // OK

        vaTest(); // Ошибка вследствие неоднозначности!
    }
}
```

В этой программе перегрузка метода `vaTest()` указана правильно, но она не компилируется. И причиной тому служит следующий вызов:

```
vaTest(); // Ошибка: неоднозначность вызова!
```

Переменное число аргументов подразумевает в том числе и нулевое их число, и поэтому приведенный выше вызов может быть интерпретирован и как `vaTest(int ...)`, и как `vaTest(boolean ...)`. Оба вызова допустимы, и поэтому обращение к данному методу неоднозначно.

Рассмотрим еще один пример неоднозначности при обращении к методу. Из приведенных ниже вариантов метода `vaTest()` невозможно однозначно выбрать требуемый, несмотря на то, что в одном из вариантов метода, помимо аргумента переменной длины, присутствует также обычный аргумент.

```
static void vaTest(int ... v) { // ...

static void vaTest(int n, int ... v) { // ...
```

И хотя списки аргументов у обоих вариантов метода `vaTest()` отличаются, компилятор все равно не может правильно выбрать вариант для следующего вызова:

```
vaTest(1)
```

В самом деле, не понятно, нужно ли преобразовать этот вызов в `vaTest(int ...)` с одним аргументом переменной длины или же в вызов `vaTest(int, int ...)` без аргументов переменной длины? В итоге возникает неоднозначная ситуация.

Вследствие ошибок, подобных описанным выше, в ряде случаев приходится отказываться от перегрузки и присваивать методам разные имена. Кроме того, ошибки неоднозначности вскрывают концептуальные просчеты в программировании, которые можно исправить, более тщательно обдумав структуру программы.

Упражнение для самопроверки по материалу главы 6

1. Допустим, имеется следующий фрагмент кода:

```
class X {
    private int count;
```

Является ли допустимым приведенный ниже фрагмент кода?

```
class Y {
    public static void main(String args[]) {
        X ob = new X();

        ob.count = 10;.
```

2. Модификатор доступа должен _____ объявлению члена класса.
3. Помимо очереди, в программах часто используется структура данных, которая называется стеком. Обращение к стеку осуществляется по принципу “первым пришел — последним обслужен”. Стек можно сравнить со стопкой тарелок, стоящих на столе. Последней берется тарелка, поставленная на стол первой. Создайте класс `Stack`, реализующий стек для хранения символов. Используйте методы `push()` и `pop()` для манипулирования содержимым стека. Пользователь класса `Stack` должен иметь возможность задавать размер стека при его создании. Все члены класса `Stack`, кроме методов `push()` и `pop()`, должны быть объявлены как `private`. (Подсказка: в качестве заготовки можете воспользоваться классом `Queue`, изменив в нем лишь способ доступа к данным.)

4. Допустим, имеется следующий класс:

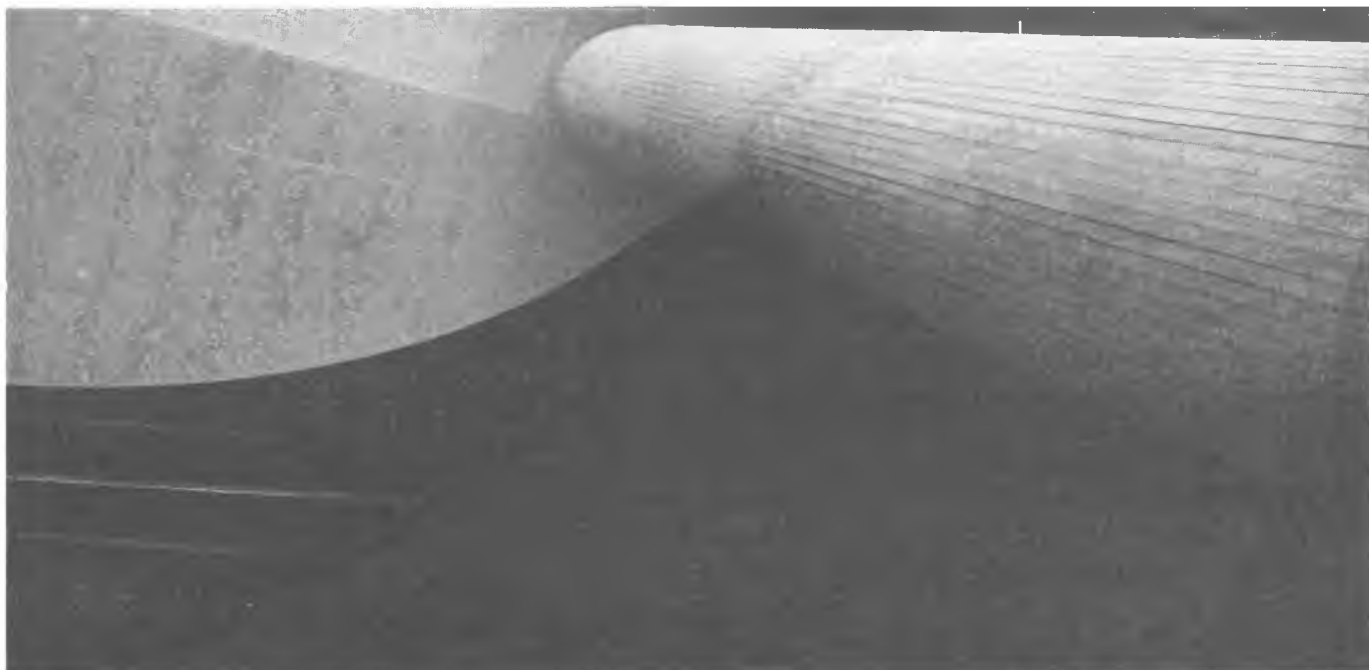
```
class Test {
    int a;
    Test(int i) { a = i; }
}
```

Напишите метод `swap()`, реализующий обмен содержимым между двумя объектами типа `Test`, на которые ссылаются две переменные данного типа.

5. Правильно ли написан следующий фрагмент кода?

```
class X {  
    int meth(int a, int b) { ... }  
    String meth(int a, int b) { ... }
```

6. Напишите рекурсивный метод, отображающий строку задом наперед.
7. Допустим, все объекты класса должны совместно пользоваться одной и той же переменной. Как объявить такую переменную?
8. Для чего может понадобиться статический блок?
9. Что такое внутренний класс?
10. Допустим, требуется член класса, к которому могут обращаться только другие члены этого же класса. Какой модификатор доступа следует использовать в его объявлении?
11. Имя метода и список его параметров вместе составляют _____ метода.
12. Если методу передается значение типа `int`, то в этом случае используется передача параметра по _____.
13. Создайте метод `sum()` с аргументами переменной длины для суммирования передаваемых ему значений типа `int`. Метод должен возвращать результат суммирования. Продемонстрируйте его в действии.
14. Можно ли перегружать метод с аргументами переменной длины?
15. Приведите пример неоднозначного вызова перегружаемого метода с переменным числом аргументов.



Глава 7

Наследование

Основные навыки и понятия

- Основы наследования
- Вызов конструктора суперкласса
- Обращения к членам суперкласса с помощью ключевого слова `super`
- Создание многоуровневой иерархии классов
- Порядок вызова конструкторов
- Представление о ссылках на объекты подкласса из переменной суперкласса
- Переопределение методов
- Применение переопределяемых методов для организации динамического доступа
- Абстрактные классы
- Использование ключевого слова `final`
- Представление о классе `Object`

Наследование является одним из трех основополагающих принципов объектно-ориентированного программирования, поскольку оно допускает создание иерархических классификаций. Благодаря наследованию можно создать общий класс, в котором определяются характерные особенности, присущие множеству связанных элементов. От этого класса могут затем наследовать другие, более конкретные классы, добавляя в него свои индивидуальные особенности.

В языке Java наследуемый класс принято называть *суперклассом*, а наследующий от него класс — *подклассом*. Следовательно, подкласс — это специализированный вариант суперкласса. Он наследует все переменные и методы, определенные в суперклассе, дополняя их своими элементами.

Основы наследования

Наследование одних классов от других отражается в Java при объявлении класса. Для этой цели служит ключевое слово `extends`. Подкласс дополняет суперкласс, расширяя его.

Рассмотрим простой пример программы, демонстрирующий некоторые свойства наследования. В этой программе определен суперкласс `TwoDShape`, хранящий сведения о ширине и высоте двумерного объекта. Там же определен и его подкласс `Triangle`. Обратите внимание на то, что в определении подкласса присутствует ключевое слово `extends`.

```
// Простая иерархия классов.
```

```
// Класс, описывающий двумерные объекты.
```

```
class TwoDShape {
    double width;
    double height;
```

```

void showDim() {
    System.out.println("Width and height are " +
        width + " and " + height);
}
}

// Подкласс класса TwoDShape для представления треугольников.
// Класс Triangle наследует от класса TwoDShape
class Triangle extends TwoDShape {
    String style;

    double area() {
        // Из класса Triangle можно обращаться к членам класса
        // TwoDShape таким же образом, как и к собственным членам.
        return width * height / 2;
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}

class Shapes {
    public static void main(String args[]) {
        Triangle t1 = new Triangle();
        Triangle t2 = new Triangle();

        // Все члены класса Triangle, даже унаследованные от класса
        // TwoDShape, доступны из объектов типа Triangle.
        t1.width = 4.0;
        t1.height = 4.0;
        t1.style = "isosceles";

        t2.width = 8.0;
        t2.height = 12.0;
        t2.style = "right";

        System.out.println("Info for t1: ");
        t1.showStyle();
        t1.showDim();
        System.out.println("Area is " + t1.area());

        System.out.println();

        System.out.println("Info for t2: ");
        t2.showStyle();
        t2.showDim();
        System.out.println("Area is " + t2.area());
    }
}

```

Ниже приведен результат выполнения данной программы.

```

Info for t1:
Triangle is isosceles
Width and height are 4.0 and 4.0

```



```
Area is 8.0
```

```
Info for t2:
```

```
Triangle is right
```

```
Width and height are 8.0 and 12.0
```

```
Area is 48.0
```

Здесь в классе `TwoDShape` определены атрибуты обобщенной двумерной фигуры, конкретным воплощением которой может быть квадрат, треугольник, прямоугольник и т.д. Класс `Triangle` представляет конкретную разновидность объекта типа `TwoDShape`, в данном случае — треугольник. Класс `Triangle` включает в себя все элементы класса `TwoDObject`, а в дополнение к ним — поле `style` и методы `area()` и `showStyle()`. Описание треугольника хранится в переменной экземпляра `style`, метод `area()` вычисляет и возвращает площадь треугольника, а метод `showStyle()` отображает геометрическую форму треугольника.

В класс `Triangle` входят все члены суперкласса `TwoDShape`, и поэтому в теле метода `area()` доступны переменные экземпляра `width` и `height`. Кроме того, с помощью объектов `t1` и `t2` в методе `main()` можно непосредственно обращаться к переменным `width` и `height`, как будто они принадлежат классу `Triangle`. На рис. 7.1 схематически показано, каким образом суперкласс `TwoDShape` включается в состав класса `Triangle`.

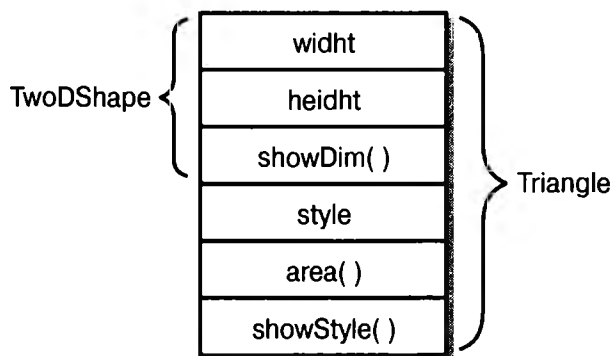


Рис. 7.1. Схематическое представление класса `Triangle`

Несмотря на то что `TwoDShape` является суперклассом для класса `Triangle`, он по-прежнему остается независимым классом. Тот факт, что один класс является суперклассом другого класса, совсем не означает, что он не может быть использован самостоятельно. Например, следующий фрагмент кода считается вполне допустимым:

```
TwoDShape shape = new TwoDShape();

shape.width = 10;
shape.height = 20;

shape.showDim();
```

Разумеется, объекту типа `TwoDShape` ничего не известно о подклассах своего класса `TwoDShape`, и он не может даже обратиться к ним.

Ниже приведена общая форма объявления класса, наследующего от суперкласса.

```
class имя_подкласса extends имя_суперкласса {
    // тело класса
}
```

Для каждого создаваемого подкласса можно указать только один суперкласс. Множественное наследование в Java не поддерживается, т.е. у подкласса не может быть

несколько суперклассов. (Этим Java отличается от языка C++, где можно создать класс, производный сразу от нескольких классов. Об этом не следует забывать, преобразуя код C++ в код Java.) С другой стороны, вполне допустима многоуровневая иерархия, в которой один подкласс является суперклассом другого подкласса. И конечно же, класс не может быть суперклассом для самого себя.

Главное преимущество наследования заключается в следующем: как только будет создан суперкласс, в котором определены общие для множества объектов атрибуты, он может быть использован для создания любого числа более конкретных подклассов. А в каждом подклассе может быть точно выстроена своя собственная классификация. В качестве примера ниже приведен еще один подкласс, производный от суперкласса TwoDShape и инкапсулирующий прямоугольники.

```
// Подкласс класса TwoDShape, представляющий прямоугольники.
class Rectangle extends TwoDShape {
    boolean isSquare() {
        if(width == height) return true;
        return false;
    }

    double area() {
        return width * height;
    }
}
```

В класс `Rectangle` входят все члены класса `TwoDShape`. Кроме того, он содержит метод `isSquare()`, определяющий, является ли прямоугольник квадратом, а также метод `area()`, вычисляющий площадь прямоугольника.

Доступ к членам класса и наследование

Как пояснялось в главе 6, члены класса зачастую объявляются закрытыми, чтобы исключить их несанкционированное или незаконное использование. Но наследование класса *не* отменяет ограничения, накладываемые на доступ к закрытым членам класса. Поэтому если в подкласс и входят все члены его суперкласса, то в нем все равно оказываются недоступными те члены суперкласса, которые являются закрытыми. Так, если сделать закрытыми переменные экземпляра `width` и `height` в классе `TwoDShape`, они станут недоступными в классе `Triangle`, как показано ниже.

[illegible]

```
// Подкласс, производный от класса TwoDShape,
// для представления треугольников.
class Triangle extends TwoDShape {
    String style;

    double area() {
        // Обратиться к членам суперкласса, объявленным
        // как закрытые, нельзя.
        return width * height / 2; // Ошибка! Доступ запрещен.
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}
```

Класс `Triangle` не будет скомпилирован, поскольку ссылки на переменные экземпляра `width` и `height` в методе `area()` нарушают правила доступа. Эти переменные объявлены закрытыми (`private`), и поэтому они доступны только членам собственного класса. А его подклассам запрещено обращаться к ним.

Напомним, что член класса, объявленный закрытым (`private`), недоступен за пределами своего класса. Это ограничение распространяется и на подклассы.

На первый взгляд, ограничение на доступ к закрытым членам суперкласса из подкласса кажется трудно преодолимым, поскольку оно не дает во многих случаях возможности пользоваться закрытыми членами этого класса. Но на самом деле это не так. Как пояснялось в главе 6, для обращения к закрытым членам класса в программах на Java обычно используются специальные методы доступа. Ниже в качестве примера приведены видоизмененные классы `TwoDShape` и `Triangle`, в которых обращение к переменным экземпляра `width` и `height` осуществляется с помощью специальных методов доступа.

```
// Применение методов доступа для установки и
/// получения значений закрытых переменных.

// Класс, описывающий двумерные объекты.
class TwoDShape {
    private double width; // Теперь эти переменные
    private double height; // объявлены как закрытые.

    // Методы доступа к переменным экземпляра width и height.
    double getWidth() { return width; }
    double getHeight() { return height; }
    void setWidth(double w) { width = w; }
    void setHeight(double h) { height = h; }

    void showDim() {
        System.out.println("Width and height are " +
                           width + " and " + height);
    }
}

// Подкласс, производный от класса TwoDShape,
// для представления треугольников.
```

```

class Triangle extends TwoDShape {
    String style;

    double area() {
        // Применение методов доступа, предоставляемых суперклассом.
        return getWidth() * getHeight() / 2;
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}

class Shapes2 {
    public static void main(String args[]) {
        Triangle t1 = new Triangle();
        Triangle t2 = new Triangle();

        t1.setWidth(4.0);
        t1.setHeight(4.0);
        t1.style = "isosceles";

        t2.setWidth(8.0);
        t2.setHeight(12.0);
        t2.style = "right";

        System.out.println("Info for t1: ");
        t1.showStyle();
        t1.showDim();
        System.out.println("Area is " + t1.area());

        System.out.println();

        System.out.println("Info for t2: ");
        t2.showStyle();
        t2.showDim();
        System.out.println("Area is " + t2.area());
    }
}

```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. В каких случаях переменную экземпляра следует объявлять закрытой?

ОТВЕТ. Не существует твердых правил, позволяющих принять безошибочное решение по данному вопросу. Следует лишь придерживаться двух общих принципов. Во-первых, если переменная экземпляра используется только методами, определенными в классе, то она должна быть закрытой. И во-вторых, если значение переменной экземпляра не должно выходить за определенные границы, ее следует объявить закрытой (`private`), а обращение к ней организовать с помощью специальных методов доступа. Подобным образом можно предотвратить присваивание недопустимых значений переменной.

Конструкторы и наследование

В иерархии классов допускается, чтобы у суперклассов и подклассов были свои собственные конструкторы. В связи с этим возникает следующий резонный вопрос: какой конструктор отвечает за построение объекта подкласса: конструктор суперкласса, конструктор подкласса или же оба вместе? На этот вопрос можно ответить так: конструктор суперкласса конструирует родительскую часть объекта, а конструктор подкласса — производную часть этого объекта. И в этом есть своя логика, поскольку суперклассу неизвестны и недоступны любые элементы подкласса, а следовательно, их конструирование должно происходить раздельно. В приведенных выше примерах данный вопрос не возникал, поскольку они опирались на автоматическое создание конструкторов, используемых в Java по умолчанию. Но на практике конструкторы определяются явным образом в большинстве классов. Ниже будет показано, каким образом разрешается подобная ситуация.

Если конструктор определен только в подклассе, то все происходит очень просто: конструируется объект подкласса, а родительская часть объекта автоматически конструируется конструктором суперкласса, используемым по умолчанию. В качестве примера ниже приведен переработанный вариант класса `Triangle`, в котором определяется конструктор, а член `style` этого класса делается закрытым, так как теперь он устанавливается конструктором.

```
// Добавление конструктора в класс Triangle.

// Класс, описывающий двумерные объекты.
class TwoDShape {
    private double width; // Теперь эти переменные
    private double height; // объявлены как закрытые.

    // Методы доступа к переменным экземпляра width и height.
    double getWidth() { return width; }
    double getHeight() { return height; }
    void setWidth(double w) { width = w; }
    void setHeight(double h) { height = h; }

    void showDim() {
        System.out.println("Width and height are " +
                           width + " and " + height);
    }
}

// Подкласс, производный от класса TwoDShape,
// для представления треугольников.
class Triangle extends TwoDShape {
    private String style;

    // Конструктор.
    Triangle(String s, double w, double h) {
        // Инициализация родительской части объекта,
        // соответствующей классу TwoDShape.
        setWidth(w);
        setHeight(h);
    }
}
```

```

        style = s;
    }

    double area() {
        return getWidth() * getHeight() / 2;
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}

class Shapes3 {
    public static void main(String args[]) {
        Triangle t1 = new Triangle("isosceles", 4.0, 4.0);
        Triangle t2 = new Triangle("right", 8.0, 12.0);

        System.out.println("Info for t1: ");
        t1.showStyle();
        t1.showDim();
        System.out.println("Area is " + t1.area());

        System.out.println();

        System.out.println("Info for t2: ");
        t2.showStyle();
        t2.showDim();
        System.out.println("Area is " + t2.area());
    }
}

```

Здесь конструктор класса `Triangle`, помимо поля `style`, инициализирует также унаследованные члены класса `TwoDClass`.

Если конструкторы объявлены как в подклассе, так и в суперклассе, то дело несколько усложняется, поскольку должны быть выполнены оба конструктора. В таком случае на помощь приходит ключевое слово `super`, доступное в двух общих формах. С помощью первой формы вызывается конструктор суперкласса. А вторая форма служит для доступа к членам суперкласса, скрываемым членами подкласса. Рассмотрим первое применение ключевого слова `super`.

Применение ключевого слова `super` для вызова конструктора суперкласса

Для вызова конструктора суперкласса служит следующая общая форма ключевого слова `super`:

```
super(список_параметров);
```

где *список_параметров* обозначает параметры, необходимые для нормальной работы конструктора суперкласса. Вызов конструктора `super()` должен быть первым оператором в теле конструктора подкласса. Для того чтобы лучше понять особенности вызова `super()`, рассмотрим вариант класса `TwoDShape` из следующего примера программы, где определен конструктор, инициализирующий переменные экземпляра `width` и `height`:

```
// Добавление конструкторов в класс TwoDShape.
class TwoDShape {
    private double width;
    private double height;

    // Параметризированный конструктор объектов класса TwoDShape.
    TwoDShape(double w, double h) {
        width = w;
        height = h;
    }

    // Методы доступа к переменным экземпляра width и height.
    double getWidth() { return width; }
    double getHeight() { return height; }
    void setWidth(double w) { width = w; }
    void setHeight(double h) { height = h; }

    void showDim() {
        System.out.println("Width and height are " +
                           width + " and " + height);
    }
}

// Подкласс, производный от класса TwoDShape,
// для представления треугольников.
class Triangle extends TwoDShape {
    private String style;

    Triangle(String s, double w, double h) {
        // Использование оператора super() для вызова
        // конструктора класса TwoDShape.
        super(w, h); // вызвать конструктор суперкласса

        style = s;
    }

    double area() {
        return getWidth() * getHeight() / 2;
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}

class Shapes4 {
    public static void main(String args[]) {
        Triangle t1 = new Triangle("isosceles", 4.0, 4.0);
        Triangle t2 = new Triangle("right", 8.0, 12.0);

        System.out.println("Info for t1: ");
        t1.showStyle();
        t1.showDim();
    }
}
```

```

    System.out.println("Area is " + t1.area());

    System.out.println();

    System.out.println("Info for t2: ");
    t2.showStyle();
    t2.showDim();
    System.out.println("Area is " + t2.area());
}
}

```

В конструкторе `Triangle` присутствует вызов конструктора `super()` с параметрами `w` и `h`. В результате управление получает конструктор `TwoDShape()`, инициализирующий переменные `width` и `height` значениями, передаваемыми ему в качестве параметров. Теперь класс `Triangle` уже не занимается инициализацией элементов суперкласса. Он должен инициализировать только собственную переменную экземпляра `style`. Конструктору `TwoDShape()` предоставляется возможность построить соответствующий подобъект так, как требуется для данного класса. Более того, в суперклассе `TwoDShape` можно реализовать функции, о которых не будут знать его подклассы. Благодаря этому код становится более устойчивым к ошибкам.

Любая форма конструктора, определенного в суперклассе, может быть вызвана с помощью оператора `super()`. Для выполнения выбирается тот вариант конструктора, который соответствует указываемым аргументам. В качестве примера ниже приведена расширенная версия классов `TwoDShape` и `Triangle`, содержащих конструкторы по умолчанию и конструкторы, принимающие один или более аргумент.

```

// Добавление дополнительных конструкторов в класс TwoDShape.
class TwoDShape {
    private double width;
    private double height;

    // Конструктор по умолчанию.
    TwoDShape() {
        width = height = 0.0;
    }

    // Параметризованный конструктор.
    TwoDShape(double w, double h) {
        width = w;
        height = h;
    }

    // Конструирование объекта с одинаковыми значениями
    // переменных экземпляра width и height.
    TwoDShape(double x) {
        width = height = x;
    }

    // Методы доступа к переменным экземпляра width и height.
    double getWidth() { return width; }
    double getHeight() { return height; }
    void setWidth(double w) { width = w; }
    void setHeight(double h) { height = h; }
}

```



```

void showDim() {
    System.out.println("Width and height are " +
        width + " and " + height);
}
}

// Подкласс, производный от класса TwoDShape,
// для представления треугольников.
class Triangle extends TwoDShape {
    private String style;

    // Использование оператора super() для вызова
    // разных вариантов конструктора TwoDShape().
    // Конструктор по умолчанию.
    Triangle() {

        super(); // вызвать конструктор суперкласса по умолчанию
        style = "null";
    }

    // Параметризованный конструктор.
    Triangle(String s, double w, double h) {
        super(w, h); // вызвать конструктор суперкласса с двумя аргументами

        style = s;
    }

    // Конструктор с одним аргументом.
    Triangle(double x) {
        super(x); // вызвать конструктор суперкласса с одним аргументом

        style = "isosceles";
    }

    double area() {
        return getWidth() * getHeight() / 2;
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}

class Shapes5 {
    public static void main(String args[]) {
        Triangle t1 = new Triangle();
        Triangle t2 = new Triangle("right", 8.0, 12.0);
        Triangle t3 = new Triangle(4.0);

        t1 = t2;

        System.out.println("Info for t1: ");
        t1.showStyle();
    }
}

```

```

t1.showDim();
System.out.println("Area is " + t1.area());

System.out.println();

System.out.println("Info for t2: ");
t2.showStyle();
t2.showDim();
System.out.println("Area is " + t2.area());

System.out.println();

System.out.println("Info for t3: ");
t3.showStyle();
t3.showDim();
System.out.println("Area is " + t3.area());

System.out.println();
}
}

```

Выполнение этой версии программы дает следующий результат:

```

Info for t1:
Triangle is right
Width and height are 8.0 and 12.0
Area is 48.0

```

```

Info for t2:
Triangle is right
Width and height are 8.0 and 12.0
Area is 48.0

```

```

Info for t3:
Triangle is isosceles
Width and height are 4.0 and 4.0
Area is 8.0

```

Еще раз напомним основные свойства вызова конструктора `super()`. Когда этот вызов присутствует в конструкторе подкласса, происходит обращение к конструктору его непосредственного суперкласса. Таким образом, вызывается конструктор того класса, который непосредственно породил вызывающий класс. Это справедливо и при многоуровневой иерархии. Кроме того, вызов конструктора `super()` должен быть первым оператором в теле конструктора подкласса.

Применение ключевого слова `super` для доступа к членам суперкласса

Существует еще одна общая форма ключевого слова `super`, которая применяется подобно ключевому слову `this`, но ссылается на суперкласс данного класса. Эта общая форма обращения к члену суперкласса имеет следующий вид:

```
super.член_класса
```

где `член_класса` обозначает метод или переменную экземпляра.

Данная форма ключевого слова `super` применяется в тех случаях, если член подкласса скрывает член суперкласса. Рассмотрим следующий пример несложной иерархии классов:

```
// Применение ключевого слова super для предотвращения сокрытия имен.
class A {
    int i;
}

// создать подкласс, расширяющий класс A.
class B extends A {
    int i; // Эта переменная i скрывает переменную i из класса A.

    B(int a, int b) {
        // Оператор super.i ссылается на переменную i из класса A.
        super.i = a; // переменная i из класса A
        i = b;       // переменная i из класса B
    }

    void show() {
        System.out.println("i in superclass: " + super.i);
        System.out.println("i in subclass: " + i);
    }
}

class UseSuper {
    public static void main(String args[]) {
        B subOb = new B(1, 2);

        subOb.show();
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
i in superclass: 1
i in subclass: 2
```

Несмотря на то что переменная экземпляра `i` в классе `B` скрывает одноименную переменную в классе `A`, ключевое слово `super` позволяет обращаться к переменной `i` из суперкласса. Аналогичным образом ключевое слово `super` можно использовать для вызова методов суперкласса, скрываемых методами подкласса.

Пример для опробования 7.1.

Расширение класса `Vehicle`

Исходный файл: `TruckDemo.java`

Для того чтобы продемонстрировать возможности наследования, расширим класс `Vehicle`, созданный в главе 4. Напомним, что класс `Vehicle` инкапсулирует данные о транспортных средствах и, в частности, сведения о количестве пассажиров, объеме топливного бака и потреблении топлива. Воспользуемся классом `Vehicle` в качестве заготовки для создания более специализированных классов. Например, транспортным средством, помимо прочих, является грузовик. Одной из важных характеристик грузовика является его грузоподъемность. Поэтому для создания класса `Truck` можно расширить класс `Vehicle`, добавив переменную экземпляра, хранящую сведения о допустимом весе перевозимого груза. В этом проекте переменные экземпляра будут объявлены в классе

Vehicle как закрытые (private), а для обращения к ним будут созданы специальные методы доступа.

Последовательность действий

1. Создайте новый файл TruckDemo.java и скопируйте в него исходный код последней версии класса Vehicle, разработанной в главе 4.
2. Создайте класс Truck, исходный код которого приведен ниже.

```
// Расширение класса Vehicle для грузовиков.
class Truck extends Vehicle {
    private int cargocap; // грузоподъемность в фунтах

    // Конструктор класса Truck.
    Truck(int p, int f, int m, int c) {
        /* Инициализация переменных из класса Vehicle
           с помощью вызываемого конструктора этого класса. */
        super(p, f, m);
        cargocap = c;
    }

    // Методы доступа к переменной cargocap.
    int getCargo() { return cargocap; }
    void putCargo(int c) { cargocap = c; }
}
```

Здесь класс Truck наследует от класса Vehicle. В класс Truck добавлены новые члены cargocap, getCargo() и putCargo(). Кроме того, класс Truck содержит все элементы, определенные в классе Vehicle.

3. Объявите закрытыми переменные экземпляра в классе Vehicle, как показано ниже.

```
private int passengers; // количество пассажиров
private int fuelcap;     // объем топливного бака в галлонах
private int mpg;         // потребление топлива в милях на галлон
```

4. Ниже приведен весь исходный код программы, в которой демонстрируется класс Truck.

```
// Пример для опробования 7.1.
//
// Создание подкласса класса Vehicle для грузовиков.
class Vehicle {
    private int passengers; // количество пассажиров
    private int fuelcap;    // объем топливного бака в галлонах
    private int mpg;        // потребление топлива в милях на галлон

    // Конструктор класса Vehicle.
    Vehicle(int p, int f, int m) {
        passengers = p;
        fuelcap = f;
        mpg = m;
    }

    // вернуть дальность действия транспортного средства
    int range() {
```

```

        return mpg * fuelcap;
    }

    // рассчитать объем топлива, требующегося
    // для прохождения заданного пути
    double fuelneeded(int miles) {
        return (double) miles / mpg;
    }

    // Методы доступа к переменным экземпляра.
    int getPassengers() { return passengers; }
    void setPassengers(int p) { passengers = p; }
    int getFuelcap() { return fuelcap; }
    void setFuelcap(int f) { fuelcap = f; }
    int getMpg() { return mpg; }
    void setMpg(int m) { mpg = m; }
}

// Расширение класса Vehicle для грузовиков.
class Truck extends Vehicle {
    private int cargocap; // грузоподъемность в фунтах

    // Конструктор класса Truck.
    Truck(int p, int f, int m, int c) {
        /* Инициализация переменных из класса Vehicle
           с помощью вызываемого конструктора этого класса. */
        super(p, f, m);

        cargocap = c;
    }

    // Методы доступа к переменной cargocap.
    int getCargo() { return cargocap; }
    void putCargo(int c) { cargocap = c; }
}

class TruckDemo {
    public static void main(String args[]) {
        // построить ряд новых объектов типа Truck
        Truck semi = new Truck(2, 200, 7, 44000);
        Truck pickup = new Truck(3, 28, 15, 2000);
        double gallons;
        int dist = 252;

        gallons = semi.fuelneeded(dist);

        System.out.println("Semi can carry " + semi.getCargo() +
                           " pounds.");
        System.out.println("To go " + dist + " miles semi needs " +
                           gallons + " gallons of fuel.\n");

        gallons = pickup.fuelneeded(dist);
    }
}

```

```

        System.out.println("Pickup can carry " + pickup.getCargo() +
                           " pounds.");
        System.out.println("To go " + dist + " miles pickup needs " +
                           gallons + " gallons of fuel.");
    }
}

```

5. Ниже приведен результат выполнения данной программы.

```

Semi can carry 44000 pounds.
To go 252 miles semi needs 36.0 gallons of fuel.

```

```

Pickup can carry 2000 pounds.
To go 252 miles pickup needs 16.8 gallons of fuel.

```

6. От класса Vehicle можно породить немало других подклассов. Например, в приведенной ниже заготовке класса, описывающего внедорожники, предусмотрена переменная, содержащая величину дорожного просвета для автомобиля.

```

// Создание класса, описывающего внедорожники.
class OffRoad extends Vehicle {
    private int groundClearance; // дорожный просвет в дюймах

    // ...
}

```

Итак, имея в своем распоряжении суперкласс, определяющий общие свойства некоторых объектов, можно создать на его основе специализированные подклассы. Каждый подкласс дополняет свойства суперкласса собственными уникальными свойствами. В этом и состоит вся сущность наследования.

Создание многоуровневой иерархии классов

В представленных до сих пор примерах программ использовались простые иерархии классов, состоявшие только из суперкласса и подкласса. Но в Java можно также строить иерархии, состоящие из любого числа уровней наследования. Как упоминалось выше, многоуровневая иерархия идеально подходит для использования одного подкласса в качестве суперкласса для другого подкласса. Так, если имеются три класса, А, В и С, то класс С может наследовать от класса В, а тот, в свою очередь, от класса А. В таком случае каждый подкласс наследует характерные особенности всех своих суперклассов. В частности, класс С наследует все члены классов В и А.

Для того чтобы стало понятнее назначение многоуровневой иерархии, рассмотрим следующий пример программы. В этой программе подкласс Triangle выступает в роли суперкласса для класса ColorTriangle. Класс ColorTriangle наследует все свойства классов Triangle и TwoDShape, а также содержит поле color, определяющее цвет треугольника.

```

// Многоуровневая иерархия.
class TwoDShape {
    private double width;
    private double height;

    // Конструктор по умолчанию.
    TwoDShape() {

```

```

    width = height = 0.0;
}

// Параметризированный конструктор.
TwoDShape(double w, double h) {
    width = w;
    height = h;
}

// построить объект с одинаковыми значениями
// переменных экземпляра width и height
TwoDShape(double x) {
    width = height = x;
}

// Методы доступа к переменным экземпляра width и height.
double getWidth() { return width; }
double getHeight() { return height; }
void setWidth(double w) { width = w; }
void setHeight(double h) { height = h; }

void showDim() {
    System.out.println("Width and height are " +
                       width + " and " + height);
}
}

// Подкласс, производный от класса TwoDShape.
class Triangle extends TwoDShape {
    private String style;

    // Конструктор по умолчанию.
    Triangle() {
        super();
        style = "null";
    }

    Triangle(String s, double w, double h) {
        super(w, h); // вызвать конструктор суперкласса

        style = s;
    }

    // Конструктор с одним аргументом для построения треугольника.
    Triangle(double x) {
        super(x); // вызвать конструктор суперкласса

        style = "isosceles";
    }

    double area() {
        return getWidth() * getHeight() / 2;
    }
}

```

```

void showStyle() {
    System.out.println("Triangle is " + style);
}
}

// Подкласс, производный от класса Triangle.
// Класс ColorTriangle является подклассом,
// производным от класса Triangle, который, в
// свою очередь, расширяет класс TwoDShape.
// Следовательно, в класс ColorTriangle входят переменные
// и методы как из класса Triangle, так из класса TwoDShape.
class ColorTriangle extends Triangle {
    private String color;

    ColorTriangle(String c, String s,
                  double w, double h) {
        super(s, w, h);

        color = c;
    }

    String getColor() { return color; }

    void showColor() {
        System.out.println("Color is " + color);
    }
}

class Shapes6 {
    public static void main(String args[]) {
        ColorTriangle t1 =
            new ColorTriangle("Blue", "right", 8.0, 12.0);
        ColorTriangle t2 =
            new ColorTriangle("Red", "isosceles", 2.0, 2.0);

        System.out.println("Info for t1: ");
        t1.showStyle();
        t1.showDim();
        t1.showColor();
        System.out.println("Area is " + t1.area());

        System.out.println();

        System.out.println("Info for t2: ");
        // Из объекта типа ColorTriangle можно вызывать как его
        // собственные методы, так и методы его суперклассов.
        t2.showStyle();
        t2.showDim();
        t2.showColor();
        System.out.println("Area is " + t2.area());
    }
}

```


Результат выполнения данной программы выглядит следующим образом:

```
Info for t1:
Triangle is right
Width and height are 8.0 and 12.0
Color is Blue
Area is 48.0

Info for t2:
Triangle is isosceles
Width and height are 2.0 and 2.0
Color is Red
Area is 2.0
```

Благодаря наследованию в классе `ColorTriangle` можно использовать ранее определенные классы `Triangle` и `TwoDShape`, дополняя их лишь данными, необходимыми для конкретного применения класса `ColorTriangle`. Таким образом, наследование способствует повторному использованию кода.

Данный пример демонстрирует еще одну важную деталь: оператор `super()` всегда обращается к конструктору ближайшего суперкласса. Иными словами, оператор `super()` в классе `ColorTriangle` означает вызов конструктора класса `Triangle`, а в классе `Triangle` — вызов конструктора класса `TwoDShape`. Если в иерархии классов для конструктора суперкласса предусмотрены параметры, то все суперклассы должны передавать их вверх по иерархической структуре. Это правило действует независимо от того, нужны ли параметры самому подклассу или не нужны.

Порядок вызова конструкторов

В связи с изложенным выше в отношении наследования и иерархии классов может возникнуть следующий резонный вопрос: когда создается объект подкласса и какой конструктор выполняется первым: тот, что определен в подклассе, или же тот, что определен в суперклассе? Так, если имеется суперкласс `A` и подкласс `B`, то вызывается ли конструктор класса `A` раньше конструктора класса `B`, или же наоборот? Ответ на этот вопрос состоит в том, что в иерархии классов конструкторы вызываются по порядку выведения классов: от суперкласса к подклассу. Более того, оператор `super()` должен быть первым в конструкторе подкласса, и поэтому порядок, в котором вызываются конструкторы, остается неизменным, независимо от того, используется ли оператор `super()` или нет. Если оператор `super()` отсутствует, то выполняется конструктор каждого суперкласса по умолчанию (т.е. конструктор без параметров). В следующем примере программы демонстрируется порядок вызова конструкторов:

```
// Демонстрация порядка вызова конструкторов.
```

```
// создать суперкласс
class A {
    A() {
        System.out.println("Constructing A.");
    }
}
```

```
// создать подкласс путем расширения класса A
class B extends A {
    B() {
```

```

        System.out.println("Constructing B.");
    }
}

// создать подкласс путем расширения класса B
class C extends B {
    C() {
        System.out.println("Constructing C.");
    }
}

class OrderOfConstruction {
    public static void main(String args[]) {
        C c = new C();
    }
}

```

Ниже приведен результат выполнения данной программы.

```

Constructing A.
Constructing B.
Constructing C.

```

Как видите, конструкторы вызываются по порядку выведения их классов.

По зрелом размышлении можно прийти к выводу, что вызов конструкторов по порядку выведения их классов имеет определенный смысл. Ведь суперклассу ничего не известно ни об одном из производных от него подклассов, и поэтому любая инициализация, которая требуется его членам, осуществляется совершенно отдельно от инициализации членов подкласса, а возможно, это и необходимое условие. Следовательно, она должна выполняться первой.

Ссылки на суперкласс и объекты подклассов

Как вам должно быть уже известно, Java является строго типизированным языком программирования. Помимо стандартных преобразований и автоматического продвижения простых типов данных, в этом языке строго соблюдается принцип совместимости типов. Это означает, что переменная ссылки на объект класса одного типа, как правило, не может ссылаться на объект класса другого типа. В качестве примера рассмотрим следующую простую программу:

```

// Этот код не подлежит компиляции.
class X {
    int a;

    X(int i) { a = i; }
}

class Y {
    int a;

    Y(int i) { a = i; }
}

class IncompatibleRef {

```

```

public static void main(String args[]) {
    X x = new X(10);
    X x2;
    Y y = new Y(5);

    x2 = x; // Допустимо, так как обе переменные одного типа.

    x2 = y; // Ошибка, так как переменные разных типов.
}
}

```

Несмотря на то что классы X и Y содержат одинаковые члены, переменной типа X нельзя присвоить ссылку на объект типа Y, поскольку типы объектов отличаются. Вообще говоря, переменная ссылки на объект может указывать только на объекты своего типа.

Но из этого строгого правила соблюдения типов имеется следующее важное исключение: переменной ссылки на объект суперкласса может быть присвоена ссылка на объект любого производного от него подкласса. Следовательно, по ссылке на объект суперкласса можно обращаться к объекту подкласса. Ниже приведен соответствующий пример.

// Обращение к объекту подкласса по ссылке на объект суперкласса.

```

class X {
    int a;

    X(int i) { a = i; }
}

class Y extends X {
    int b;

    Y(int i, int j) {
        super(j);
        b = i;
    }
}

class SupSubRef {
    public static void main(String args[]) {
        X x = new X(10);
        X x2;
        Y y = new Y(5, 6);

        x2 = x; // Допустимо, так как обе переменные одного типа.
        System.out.println("x2.a: " + x2.a);

        // Класс Y является подклассом X, поэтому переменные x2 и y
        // могут ссылаться на один и тот же объект производного класса.
        x2 = y; // По-прежнему допустимо по указанной выше причине.
        System.out.println("x2.a: " + x2.a);

        // В классе X известны только члены класса X.
        x2.a = 19; // Допустимо.
    }
}

```

```
// x2.b = 27; // Ошибка, так как переменная b не является членом класса X.
}
}
```

В данном примере класс *Y* является подклассом *X*. Следовательно, переменной *x2* можно присвоить ссылку на объект типа *Y*.

Следует особо подчеркнуть, что доступ к конкретным членам класса определяется типом переменной ссылки на объект, а не типом объекта, на который она ссылается. Это означает, что если ссылка на объект подкласса присваивается переменной ссылки на объект суперкласса, то доступ разрешается только к тем частям этого объекта, которые определяются суперклассом. Именно поэтому переменной *x2* недоступен член *b* класса *Y*, когда она ссылается на объект этого класса. И в этом есть своя логика, поскольку суперклассу ничего не известно о тех членах, которые добавлены в производный от него подкласс. Именно поэтому последняя строка кода в приведенном выше примере была закомментирована.

Несмотря на кажущийся несколько отвлеченным характер приведенных выше рассуждений, им можно найти ряд важных применений на практике. Одно из них рассматривается ниже, а другое — далее в этой главе, когда речь пойдет о переопределении методов.

Один из самых важных моментов для присваивания ссылок на объекты подкласса переменным суперкласса наступает тогда, когда конструкторы вызываются в иерархии классов. Как вам должно быть уже известно, в классе нередко определяется конструктор, принимающий объект своего класса в качестве параметра. Благодаря этому в классе может быть сконструирована копия его объекта. Этой особенностью можно выгодно воспользоваться в подклассах, производных от такого класса. В качестве примера рассмотрим очередные версии классов *TwoDShape* и *Triangle*. В оба класса добавлены конструкторы, принимающие объект своего класса в качестве параметра.

```
class TwoDShape {
    private double width;
    private double height;

    // Конструктор по умолчанию.
    TwoDShape() {
        width = height = 0.0;
    }

    // Параметризованный конструктор.
    TwoDShape(double w, double h) {
        width = w;
        height = h;
    }

    // построить объект с одинаковыми значениями
    // переменных экземпляра width и height
    TwoDShape(double x) {
        width = height = x;
    }

    // Построение одного объекта на основании другого объекта.
    TwoDShape(TwoDShape ob) {
        width = ob.width;
```

```

    height = ob.height;
}

// Методы доступа к переменным width и height.
double getWidth() { return width; }
double getHeight() { return height; }
void setWidth(double w) { width = w; }
void setHeight(double h) { height = h; }

void showDim() {
    System.out.println("Width and height are " +
        width + " and " + height);
}
}

// Подкласс класса TwoDShape для треугольников.
class Triangle extends TwoDShape {
    private String style;

    // Конструктор по умолчанию.
    Triangle() {
        super();
        style = "null";
    }

    // Конструктор класса Triangle.
    Triangle(String s, double w, double h) {
        super(w, h); // вызвать конструктор суперкласса

        style = s;
    }

    // Конструктор с одним аргументом для построения треугольника.
    Triangle(double x) {
        super(x); // вызвать конструктор суперкласса

        style = "isosceles";
    }

    // построить один объект на основании другого объекта
    Triangle(Triangle ob) {
        // Передача ссылки на объект Triangle конструктору класса TwoDShape.
        super(ob);
        style = ob.style;
    }

    double area() {
        return getWidth() * getHeight() / 2;
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}

```

```

class Shapes7 {
    public static void main(String args[]) {
        Triangle t1 =
            new Triangle("right", 8.0, 12.0);

        // создать копию объекта t1
        Triangle t2 = new Triangle(t1);

        System.out.println("Info for t1: ");
        t1.showStyle();
        t1.showDim();
        System.out.println("Area is " + t1.area());

        System.out.println();

        System.out.println("Info for t2: ");
        t2.showStyle();
        t2.showDim();
        System.out.println("Area is " + t2.area());
    }
}

```

В приведенном выше примере программы объект `t2` конструируется на основании объекта `t1`, и поэтому они идентичны. Результат выполнения данной программы выглядит следующим образом:

```

Info for t1:
Triangle is right
Width and height are 8.0 and 12.0
Area is 48.0

```

```

Info for t2:
Triangle is right
Width and height are 8.0 and 12.0
Area is 48.0

```

Обратите внимание на конструктор класса `Triangle`, код которого приведен ниже.

```

// построить один объект на основании другого объекта
Triangle(Triangle ob) {
    // Передача ссылки на объект Triangle конструктору класса TwoDShape.
    super(ob);
    style = ob.style;
}

```

В качестве параметра данному конструктору передается объект `Triangle`, который затем с помощью вызова `super()` передается конструктору `TwoDShape`, как показано ниже.

```

// Построение одного объекта на основании другого объекта.
TwoDShape(TwoDShape ob) {
    width = ob.width;
    height = ob.height;
}

```

Следует заметить, что конструктор `TwoDShape()` должен получить объект типа `TwoDShape`, но конструктор `Triangle()` передает ему объект типа `Triangle`. Тем не

менее никаких недоразумений не возникает. Ведь, как пояснялось ранее, переменная ссылки на суперкласс может ссылаться на объект производного от него подкласса. Следовательно, конструктор `TwoDShape()` можно передать ссылку на экземпляр подкласса, производного от класса `TwoDShape`. Конструктор `TwoDShape()` инициализирует лишь те части передаваемого ему объекта подкласса, которые являются членами класса `TwoDShape`, и поэтому не имеет значения, содержит ли этот объект дополнительные члены, добавленные в производных подклассах.

Переопределение методов

В иерархии классов часто присутствуют методы с одинаковой сигнатурой и одинаковым возвращаемым значением как в суперклассе, так и в подклассе. В этом случае говорят, что метод суперкласса *переопределяется* в подклассе. Если переопределяемый метод вызывается из подкласса, то всегда выбирается тот вариант метода, который определен в подклассе. А вариант метода, определенный в суперклассе, скрывается. Рассмотрим в качестве примера следующий фрагмент кода:

```
// Переопределение метода.
class A {
    int i, j;
    A(int a, int b) {
        i = a;
        j = b;
    }

    // отобразить переменные i и j
    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}

class B extends A {
    int k;

    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }

    // Отображение переменной k. Данный метод переопределяет
    // метод show() из класса A.
    void show() {
        System.out.println("k: " + k);
    }
}

class Override {
    public static void main(String args[]) {
        B subOb = new B(1, 2, 3);

        subOb.show(); // вызвать метод show() из класса B
    }
}
```

Выполнение этого фрагмента кода дает следующий результат:

```
k: 3
```

Когда метод `show()` вызывается для объекта типа `B`, выбирается вариант этого метода, определенный в классе `B`. Таким образом, вариант метода `show()` в классе `B` переопределяет вариант одноименного метода, объявленный в классе `A`.

Если требуется обратиться к исходному варианту переопределяемого метода, т.е. тому, который определен в суперклассе, следует воспользоваться ключевым словом `super`. Например, в приведенном ниже варианте класса `B` из метода `show()` вызывается вариант того же метода, определенный в суперклассе. При этом отображаются все переменные экземпляра.

```
class B extends A {
    int k;

    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }

    void show() {
        // Использование ключевого слова super для
        // вызова метода show(), определенного в классе A.
        super.show();
        System.out.println("k: " + k);
    }
}
```

Если подставить новый вариант метода `show()` в предыдущую версию рассматриваемого здесь фрагмента кода, результат его выполнения изменится и будет иметь следующий вид:

```
i and j: 1 2
k: 3
```

В данном случае `super.show()` — это вызов метода `show()`, определенного в суперклассе.

Переопределение метода происходит только в том случае, если сигнатуры переопределяемого и переопределяющего методов совпадают. В противном случае происходит обычная перегрузка методов. Рассмотрим следующую видоизмененную версию предыдущего примера:

```
/* Методы с разными сигнатурами не переопределяются,
   а перегружаются. */
class A {
    int i, j;

    A(int a, int b) {
        i = a;
        j = b;
    }

    // отобразить переменные i и j
    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}
```



```

    }
}

// создать подкласс путем расширения класса A
class B extends A {
    int k;

    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }

    // Сигнатуры данного метода и метода show() из класса A отличаются,
    // поэтому вместо переопределения происходит перегрузка метода.
    void show(String msg) {
        System.out.println(msg + k);
    }
}

class Overload {
    public static void main(String args[]) {
        B subOb = new B(1, 2, 3);

        subOb.show("This is k: "); // вызывается метод show() из класса B
        subOb.show(); // вызывается метод show() из класса A
    }
}

```

Выполнение этого фрагмента кода дает следующий результат:

```

This is k: 3
i and j: 1 2

```

На этот раз в варианте метода `show()` из класса `B` предусмотрен строковый параметр. И благодаря этому сигнатура данного метода отличается от сигнатуры метода `show()` из класса `A`, для которого параметры не предусмотрены. В результате переопределение метода не происходит.

Поддержка полиморфизма в переопределяемых методах

Примеры из предыдущего раздела демонстрируют переопределение методов, но по ним трудно судить, насколько богатые возможности предоставляет этот механизм. В самом деле, если переопределение методов используется только для соблюдения соглашений о пространствах имен, то его можно рассматривать как любопытную, но почти бесполезную особенность языка программирования. Но это совсем не так. Переопределение методов лежит в основе одного из наиболее эффективных языковых средств Java: *динамической диспетчеризации методов*, представляющей собой механизм вызова переопределяемых методов, когда выбор конкретного метода осуществляется не на этапе компиляции, а в процессе выполнения программы. Динамическая диспетчеризация методов имеет очень большое значение, поскольку именно с ее помощью принцип полиморфизма реализуется на стадии выполнения программ на Java.

Прежде всего напомним один очень важный принцип: переменная ссылки на суперкласс может ссылаться на объект подкласса. В Java этот принцип используется для вызова переопределяемых методов во время выполнения. Если переопределяемый метод вызывается по ссылке на суперкласс, то исполняющая система Java определяет по типу объекта, какой именно вариант метода следует вызвать, причем делает это во время выполнения программы. Если ссылки указывают на разные типы объектов, то и вызываться будут разные версии переопределенных методов. Иными словами, вариант переопределенного метода для вызова определяет не *тип переменной*, а *тип объекта*, на который она ссылается. Так, если суперкласс содержит метод, переопределяемый в подклассе, то вызывается метод, соответствующий тому типу объекта, на который указывает переменная ссылки на суперкласс.

Ниже приведен простой пример, демонстрирующий динамическую диспетчеризацию методов в действии.

// Демонстрация динамической диспетчеризации методов.

```
class Sup {
    void who() {
        System.out.println("who() in Sup");
    }
}

class Sub1 extends Sup {
    void who() {
        System.out.println("who() in Sub1");
    }
}

class Sub2 extends Sup {
    void who() {
        System.out.println("who() in Sub2");
    }
}

class DynDispDemo {
    public static void main(String args[]) {
        Sup superOb = new Sup();
        Sub1 subOb1 = new Sub1();
        Sub2 subOb2 = new Sub2();

        Sup supRef;

        // В каждом из приведенных ниже вызовов конкретный вариант
        // метода who() выбирается во время выполнения по типу
        // объекта, на который делается ссылка.
        supRef = superOb;
        supRef.who();

        supRef = subOb1;
        supRef.who();

        supRef = subOb2;
        supRef.who();
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
who() in Sup
who() in Sub1
who() in Sub2
```

В данном примере программы определяются суперкласс `Sup` и два его подкласса `Sub1` и `Sub2`. В классе `Sup` объявляется метод `who()`, переопределяемый в его подклассах. В методе `main()` создаются объекты типа `Sup`, `Sub1` и `Sub2`. Там же объявляется переменная `supRef` ссылки на объект типа `Sup`. Затем переменной `supRef` в методе `main()` поочередно присваиваются ссылки на объекты разного типа, и далее эти ссылки используются для вызова метода `who()`. Как следует из результата выполнения данной программы, вызываемый вариант метода `who()` определяется типом объекта, на который ссылается переменная `supRef` в момент вызова, а не типом самой этой переменной.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Переопределяемые методы напоминают виртуальные функции в C++. Есть ли у них сходство?

ОТВЕТ. Да, есть. Читатели, знакомые с языком C++, заметят, что переопределяемые методы в Java применяются с той же целью и таким же способом, как и виртуальные функции в C++.

Причины для переопределения методов

Как упоминалось выше, переопределяемые методы обеспечивают соблюдение принципа полиморфизма при выполнении программ на Java. Полиморфизм в объектно-ориентированных программах имеет большое значение потому, что благодаря ему появляется возможность объявить в суперклассе методы, общие для всех его подклассов, а в самих подклассах — определить специфические реализации всех этих методов или некоторых из них. Переопределение методов — один из способов, которыми в Java реализуется принцип полиморфизма “один интерфейс — множество методов”.

Залогом успешного применения полиморфизма является, в частности, понимание того, что суперклассы и подклассы образуют иерархию в направлении от меньшей специализации к большей. Если суперкласс организован правильно, он предоставляет своему подклассу все элементы, которыми тот может пользоваться непосредственно. В нем также определяются те методы, которые должны быть по-своему реализованы в порожденных классах. Таким образом, подклассы получают достаточную свободу определять собственные методы, реализуя в то же время согласованный интерфейс. Сочетая наследование с переопределением методов, в суперклассе можно определить общую форму методов для использования во всех его подклассах.

Демонстрация механизма переопределения методов на примере класса `TwoDShape`

Для того чтобы стало понятнее, насколько эффективным является механизм переопределения методов, продемонстрируем его на примере класса `TwoDShape`. В приведенных ранее примерах в каждом классе, порожденном от класса `TwoDShape`, определялся метод `area()`. Теперь мы знаем, что в этом случае имеет смысл включить метод `area()`

в состав класса `TwoDShape` и позволить каждому его подклассу переопределить этот метод: в частности, реализовать вычисление площади в зависимости от конкретного типа геометрической фигуры. Именно такой подход и воплощен в приведенном ниже примере программы. Для удобства в класс `TwoDShape` добавлено поле `name`, упрощающее написание демонстрационных программ.

```
// Применение динамической диспетчеризации методов.
class TwoDShape {
    private double width;
    private double height;
    private String name;

    // Конструктор по умолчанию.
    TwoDShape() {
        width = height = 0.0;
        name = "null";
    }

    // Параметризованный конструктор.
    TwoDShape(double w, double h, String n) {
        width = w;
        height = h;
        name = n;
    }

    // построить объект с одинаковыми значениями
    // переменных экземпляра width и height
    TwoDShape(double x, String n) {
        width = height = x;
        name = n;
    }

    // построить один объект на основании другого объекта
    TwoDShape(TwoDShape ob) {
        width = ob.width;
        height = ob.height;
        name = ob.name;
    }

    // Методы доступа к переменным width и height.
    double getWidth() { return width; }
    double getHeight() { return height; }
    void setWidth(double w) { width = w; }
    void setHeight(double h) { height = h; }

    String getName() { return name; }

    void showDim() {
        System.out.println("Width and height are " +
                           width + " and " + height);
    }

    // Метод area() определен в классе TwoDShape.
    double area() {
```

```

        System.out.println("area() must be overridden");
        return 0.0;
    }
}

// Подкласс, производный от класса TwoDShape,
// для представления треугольников.
class Triangle extends TwoDShape {
    private String style;

    // Конструктор по умолчанию.
    Triangle() {
        super();
        style = "null";
    }

    // Конструктор класса Triangle.
    Triangle(String s, double w, double h) {
        super(w, h, "triangle");

        style = s;
    }

    // Конструктор с одним аргументом для построения треугольника
    Triangle(double x) {
        super(x, "triangle"); // вызвать конструктор суперкласса

        style = "isosceles";
    }

    // построить один объект на основании другого объекта
    Triangle(Triangle ob) {
        super(ob); // передать объект конструктору класса TwoDShape
        style = ob.style;
    }

    // Переопределение метода area() для класса Triangle.
    double area() {
        return getWidth() * getHeight() / 2;
    }

    void showStyle() {
        System.out.println("Triangle is " + style);
    }
}

// Подкласс, производный от класса TwoDShape,
// для представления прямоугольников.
class Rectangle extends TwoDShape {
    // Конструктор по умолчанию.
    Rectangle() {
        super();
    }

    // Конструктор класса Rectangle.

```

```

Rectangle(double w, double h) {
    super(w, h, "rectangle"); // вызвать конструктор суперкласса
}

// построить квадрат
Rectangle(double x) {
    super(x, "rectangle"); // вызвать конструктор суперкласса
}

// построить один объект на основании другого объекта
Rectangle(Rectangle ob) {
    super(ob); // передать объект конструктору класса TwoDShape
}

boolean isSquare() {
    if(getWidth() == getHeight()) return true;
    return false;
}

// Переопределение метода area() для класса Rectangle.
double area() {
    return getWidth() * getHeight();
}
}

class DynShapes {
    public static void main(String args[]) {
        TwoDShape shapes[] = new TwoDShape[5];

        shapes[0] = new Triangle("right", 8.0, 12.0);
        shapes[1] = new Rectangle(10);
        shapes[2] = new Rectangle(10, 4);
        shapes[3] = new Triangle(7.0);
        shapes[4] = new TwoDShape(10, 20, "generic");

        for(int i=0; i < shapes.length; i++) {
            System.out.println("object is " + shapes[i].getName());
            // Требуемый вариант метода area() вызывается
            // для каждой геометрической фигуры в отдельности.
            System.out.println("Area is " + shapes[i].area());
            System.out.println();
        }
    }
}

```

Ниже приведен результат выполнения данной программы.

```

object is triangle
Area is 48.0

```

```

object is rectangle
Area is 100.0

```

```

object is rectangle

```

```
Area is 40.0
```

```
object is triangle
Area is 24.5
```

```
object is generic
area() must be overridden
Area is 0.0
```

Рассмотрим код данной программы более подробно. Теперь, как и предполагалось при написании программы, метод `area()` входит в состав класса `TwoDShape` и переопределяется в классах `Triangle` и `Rectangle`. В классе `TwoDShape` метод `area()` играет роль заполнителя и лишь уведомляет пользователя о том, что этот метод должен быть переопределен в подклассе. При каждом переопределении метода `area()` в нем реализуются средства, необходимые для того типа объекта, который инкапсулируется в подклассе. Так, если требуется реализовать класс для эллипсов, метод `area()` придется переопределить таким образом, чтобы он вычислял площадь этой фигуры.

У рассматриваемой здесь программы имеется еще одна важная особенность. Обратите внимание на то, что в методе `main()` геометрические фигуры объявляются как массив объектов типа `TwoDShape`. Но на самом деле элементами массива являются ссылки на объекты `Triangle`, `Rectangle` и `TwoDShape`. Это вполне допустимо. Ведь, как пояснялось ранее, переменная ссылки на суперкласс может ссылаться на объект его подкласса. В этой программе организован перебор элементов массива в цикле и вывод сведений о каждом объекте. Несмотря на всю простоту данного примера, он наглядно демонстрирует потенциальные возможности как наследования, так и переопределения методов. Тип объекта, на который указывает переменная ссылки на суперкласс, определяется на этапе выполнения программы и обрабатывается соответствующим образом. Если объект является производным от класса `TwoDShape`, его площадь вычисляется при вызове метода `area()`. Интерфейс для данной операции оказывается общим и не зависит от того, с какой именно геометрической фигурой приходится иметь дело каждый раз.

Применение абстрактных классов

Иногда требуется создать суперкласс, в котором определяется лишь самая общая форма для всех его подклассов, а наполнение ее деталями предоставляется каждому из этих подклассов. В таком классе определяется лишь характер методов, которые должны быть конкретно реализованы в подклассах, а не в самом суперклассе. Подобная ситуация возникает, например, в связи с невозможностью получить содержательную реализацию метода в суперклассе. Именно такая ситуация была продемонстрирована в варианте класса `TwoDShape` из предыдущего примера, где метод `area()` был просто определен как заполнитель. Такой метод не вычисляет и не выводит площадь двумерной геометрической формы любого типа.

Создавая собственные библиотеки классов, вы можете сами убедиться в том, что у метода зачастую отсутствует содержательное определение в контексте его суперкласса. Подобное затруднение разрешается двумя способами. Один из них, как показано в предыдущем примере, состоит в том, чтобы просто выдать предупреждающее сообщение. И хотя такой способ может пригодиться в некоторых случаях, например при отладке, в практике программирования он обычно не применяется. Ведь в суперклассе могут быть объявлены методы, которые должны быть переопределены в подклассе, чтобы этот класс стал содержательным. Рассмотрим для примера класс `Triangle`. Он был

бы неполным, если бы в нем не был переопределен метод `area()`. В подобных случаях требуется какой-то способ, гарантирующий, что в подклассе действительно будут переопределены все необходимые методы. И такой способ в Java имеется. Он состоит в использовании *абстрактного метода*.

Абстрактный метод создается с помощью указываемого модификатора типа `abstract`. У абстрактного метода отсутствует тело, и поэтому он не реализуется в суперклассе. Это означает, что он должен быть переопределен в подклассе, поскольку его вариант из суперкласса просто непригоден для использования. Для определения абстрактного метода служит приведенная ниже общая форма.

```
abstract тип имя(список_параметров);
```

Как видите, у абстрактного метода отсутствует тело. Модификатор `abstract` может применяться только к обычным методам, но не к статическим методам (`static`) и конструкторам.

Класс, содержащий один или более абстрактный метод, должен быть также объявлен как абстрактный, для чего перед его объявлением `class` указывается модификатор `abstract`. А поскольку реализация абстрактного класса не определяется полностью, то у него не может быть объектов. Следовательно, попытка создать объект абстрактного класса с помощью оператора `new` приведет к ошибке во время компиляции.

Когда подкласс наследует абстрактный класс, в нем должны быть реализованы все абстрактные методы суперкласса. В противном случае подкласс должен быть также определен как `abstract`. Таким образом, атрибут `abstract` наследуется до тех пор, пока не будет достигнута полная реализация класса.

Используя абстрактный класс, мы можем усовершенствовать рассматривавшийся ранее класс `TwoDShape`. Для неопределенной двумерной геометрической фигуры понятие площади не имеет никакого смысла, поэтому в приведенной ниже версии предыдущей программы метод `area()` и сам класс `TwoDShape` объявляются как `abstract`. Это, конечно, означает, что во всех классах, производных от класса `TwoDShape`, должен быть переопределен метод `area()`.

```
// Создание абстрактного класса.
```

```
// Теперь класс TwoDShape является абстрактным.
```

```
abstract class TwoDShape {
    private double width;
    private double height;
    private String name;

    // Конструктор по умолчанию.
    TwoDShape() {
        width = height = 0.0;
        name = "null";
    }

    // Параметризованный конструктор.
    TwoDShape(double w, double h, String n) {
        width = w;
        height = h;
        name = n;
    }
}
```



```

// построить объект с одинаковыми значениями
// переменных экземпляра width и height
TwoDShape(double x, String n) {
    width = height = x;
    name = n;
}

// построить один объект на основании другого объекта
TwoDShape(TwoDShape ob) {
    width = ob.width;
    height = ob.height;
    name = ob.name;
}

// Методы доступа к переменным width и height.
double getWidth() { return width; }
double getHeight() { return height; }
void setWidth(double w) { width = w; }
void setHeight(double h) { height = h; }

String getName() { return name; }

void showDim() {
    System.out.println("Width and height are " +
                       width + " and " + height);
}

// Теперь метод area() является абстрактным.
abstract double area();
}

// Подкласс, производный от класса TwoDShape,
// для представления треугольников.
class Triangle extends TwoDShape {
    private String style;

    // Конструктор по умолчанию.
    Triangle() {
        super();
        style = "null";
    }

    // Конструктор класса Triangle.
    Triangle(String s, double w, double h) {
        super(w, h, "triangle");

        style = s;
    }

    // Конструктор с одним аргументом для построения треугольника.
    Triangle(double x) {
        super(x, "triangle"); // вызвать конструктор суперкласса
        style = "isosceles";
    }
}

```

```

// построить один объект на основании другого объекта
Triangle(Triangle ob) {
    super(ob); // передать объект конструктору класса TwoDShape
    style = ob.style;
}

double area() {
    return getWidth() * getHeight() / 2;
}

void showStyle() {
    System.out.println("Triangle is " + style);
}
}

// Подкласс, производный от класса TwoDShape,
// для представления прямоугольников.
class Rectangle extends TwoDShape {
    // Конструктор по умолчанию.
    Rectangle() {
        super();
    }

    // Конструктор класса Rectangle.
    Rectangle(double w, double h) {
        super(w, h, "rectangle"); // вызвать конструктор суперкласса
    }

    // построить квадрат
    Rectangle(double x) {
        super(x, "rectangle"); // вызвать конструктор суперкласса
    }

    // построить один объект на основании другого объекта
    Rectangle(Rectangle ob) {
        super(ob); // передать объект конструктору класса TwoDShape
    }

    boolean isSquare() {
        if(getWidth() == getHeight()) return true;
        return false;
    }

    double area() {
        return getWidth() * getHeight();
    }
}

class AbsShape {
    public static void main(String args[]) {
        TwoDShape shapes[] = new TwoDShape[4];
        shapes[0] = new Triangle("right", 8.0, 12.0);
        shapes[1] = new Rectangle(10);
    }
}

```

```

shapes[2] = new Rectangle(10, 4);
shapes[3] = new Triangle(7.0);

for(int i=0; i < shapes.length; i++) {
    System.out.println("object is " +
        shapes[i].getName());
    System.out.println("Area is " + shapes[i].area());
    System.out.println();
}
}
}

```

Как показывает представленный выше пример программы, во всех подклассах, производных от класса `TwoDShape`, метод `area()` должен быть *непрерывно* переопределен. Убедитесь в этом сами, попробовав создать подкласс, в котором не переопределен метод `area()`. В итоге вы получите сообщение об ошибке во время компиляции. Конечно, возможность создавать ссылки на объекты типа `TwoDShape` по-прежнему существует, и это было сделано в приведенном выше примере программы, но объявлять объекты типа `TwoDShape` уже нельзя. Именно поэтому массив `shapes` сокращен в методе `main()` до 4 элементов, а объект типа `TwoDShape` для общей двумерной геометрической формы больше не создается.

И еще одно, последнее замечание. Обратите внимание на то, что в классе `TwoDShape` по-прежнему присутствуют определения методов `showDim()` и `getName()` и перед их именами нет модификатора `abstract`. В абстрактные классы вполне допускается (и часто практикуется) включать конкретные методы, которые могут быть использованы в своем исходном виде в подклассе. А переопределению в подклассах подлежат только те методы, которые объявлены как `abstract`.

Использование ключевого слова `final`

Какие бы богатые возможности ни представляли механизмы наследования и переопределения методов, иногда требуется запретить их действие. Допустим, создается класс, в котором инкапсулированы средства управления некоторым устройством. Кроме того, в этом классе пользователю может быть разрешено инициализировать устройство, чтобы воспользоваться некоторой секретной информацией. В таком случае пользователи данного класса не должны иметь возможность переопределять метод инициализации устройства. Для этой цели в Java предоставляется ключевое слово `final`, позволяющее без труда запретить переопределение метода или наследование класса.

Предотвращение переопределения методов

Для того чтобы предотвратить переопределение метода, в начале его объявления нужно указать модификатор доступа `final`. Переопределять объявленные подобным образом методы нельзя. Ниже приведен фрагмент кода, демонстрирующий использование ключевого слова `final` для подобных целей.

```

class A {
    final void meth() {
        System.out.println("This is a final method.");
    }
}

```

```
class B extends A {
    void meth() { // Ошибка! Переопределить метод нельзя.
        System.out.println("Illegal!");
    }
}
```

Поскольку метод `meth()` объявлен как `final`, его нельзя переопределить в классе `B`. Если вы попытаетесь сделать это, возникнет ошибка при компиляции программы.

Предотвращение наследования

Предотвратить наследование класса можно, указав в определении класса ключевое слово `final`. В этом случае считается, что данное ключевое слово применяется ко всем методам класса. Очевидно, что не имеет никакого смысла применять ключевое слово `final` к абстрактным классам. Ведь абстрактный класс не завершен по определению, и объявленные в нем методы должны быть реализованы в подклассах.

Ниже приведен пример класса, подклассы которого создавать запрещено.

```
final class A {
    // ...
}

// Следующее определение класса недопустимо.
class B extends A { // Ошибка! Создать подкласс от класса A нельзя.
    // ...
}
```

Как следует из комментариев к данному примеру, недопустимо, чтобы класс `B` наследовал от класса `A`, так как последний определен как `final`.

Применение ключевого слова `final` к переменным экземпляра

Помимо рассмотренных ранее примеров использования, ключевое слово `final` можно применять и к переменным экземпляра. Подобным способом создаются именованные константы. Если имени переменной предшествует модификатор `final`, то значение этой переменной не может быть изменено на протяжении всего времени выполнения программы. Очевидно, что подобным переменным нужно присваивать начальные значения. В главе 6 был рассмотрен простой класс `ErrorMsg` для обработки ошибок. В нем устанавливается соответствие между кодами ошибок и символьными строками сообщений об ошибках. Ниже приведен усовершенствованный вариант этого класса, в котором для создания именованных констант применяется модификатор `final`. Теперь, вместо того чтобы передавать методу `getErrorMsg()` числовое значение, например `2`, достаточно указать при его вызове именованную целочисленную константу `DISKERR`.

```
// Возврат объекта типа String.
class ErrorMsg {
    // Коды ошибок.
    // Константы объявляются с помощью ключевого слова final.
    final int OUTERR = 0;
    final int INERR = 1;
    final int DISKERR = 2;
    final int INDEXERR = 3;
```

```

String msgs[] = {
    "Output Error",
    "Input Error",
    "Disk Full",
    "Index Out-Of-Bounds"
};

// вернуть сообщение об ошибке
String getErrorMsg(int i) {
    if(i >=0 & i < msgs.length)
        return msgs[i];
    else
        return "Invalid Error Code";
}
}

class FinalD {
    public static void main(String args[]) {
        ErrorMsg err = new ErrorMsg();

        // При вызове метода используются константы,
        // объявленные с помощью ключевого слова final.
        System.out.println(err.getErrorMsg(err.OUTERR));
        System.out.println(err.getErrorMsg(err.DISKERR));
    }
}

```

Обратите внимание на то, как используются константы в методе `main()`. Они являются членами класса `ErrorMsg`, и поэтому для доступа к ним требуется ссылка на объект этого класса. Разумеется, константы могут быть унаследованы подклассами и непосредственно доступными в них.

Многие программирующие на Java пользуются именами констант типа `final`, составленными полностью из прописных букв, как в предыдущем примере. Но это не строгое правило, а только принятый стиль программирования.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Могут ли переменные типа `final` быть статическими? И можно ли использовать ключевое слово `final` при объявлении локальных переменных и параметров методов?

ОТВЕТ. Да, можно. Объявив константу таким образом, вы можете обращаться к ней по имени класса, не создавая конкретных объектов. Так, если при объявлении констант в классе `ErrorMsg` указать ключевое слово `static`, вызов метода `println()` в методе `main()` может приобрести следующий вид:

```

System.out.println(err.getErrorMsg(ErrorMsg.OUTERR));
System.out.println(err.getErrorMsg(ErrorMsg.DISKERR));

```

Благодаря объявлению параметра как `final` предотвращается его изменение в методе. А если объявить локальную переменную как `final`, ей нельзя будет присвоить значение больше одного раза.

Класс Object

В Java определен специальный класс `Object`. По умолчанию он считается супер-классом всех остальных классов. Иными словами, все классы являются подклассами, производными от класса `Object`. Это означает, что переменная ссылки на объект типа `Object` может ссылаться на объект любого класса. Более того, переменная ссылки на объект типа `Object` может также ссылаться на любой массив, поскольку массивы реализованы в виде классов.

В классе `Object` определены перечисленные ниже методы, доступные в любом объекте.

Метод	Назначение
<code>Object clone()</code>	Создает новый объект, аналогичный копируемому объекту
<code>boolean equals(Object объект)</code>	Определяет равнозначность объектов
<code>void finalize()</code>	Вызывается перед тем, как неиспользуемый объект будет удален системой "сборки мусора"
<code>Class<?> getClass()</code>	Определяет класс объекта во время выполнения
<code>int hashCode()</code>	Возвращает хеш-код, связанный с вызывающим объектом
<code>void notify()</code>	Возобновляет работу потока, ожидающего уведомления от вызывающего объекта
<code>void notifyAll()</code>	Возобновляет работу всех потоков, ожидающих уведомления от вызывающего объекта
<code>String toString()</code>	Возвращает символьную строку, описывающую объект
<code>void wait()</code> <code>void wait(long миллисекунды)</code> <code>void wait(long миллисекунды, int наносекунды)</code>	Ожидает исполнения другого потока

Методы `getClass()`, `notify()`, `notifyAll()` и `wait()` объявлены как `final`, а остальные методы можно переопределять в подклассах. Некоторые из этих методов будут описаны далее в этой книге. Два из них — `equals()` и `toString()` — заслуживают особого внимания. Метод `equals()` сравнивает два объекта. Если объекты равнозначны, то он возвращает логическое значение `true`, иначе — логическое значение `false`. Метод `toString()` возвращает символьную строку, содержащую описание того объекта, которому принадлежит этот метод. Он автоматически вызывается в том случае, если объект передается методу `println()` в качестве параметра. Во многих классах этот метод переопределяется. В этом случае описание специально подбирается для конкретных типов объектов, которые в них создаются.

Обратите внимание на необычный синтаксис, описывающий значение, возвращаемое методом `getClass()`. Это *обобщенный* тип. С помощью обобщений в Java можно указывать в качестве параметра тип данных, используемый в классе или методе. Более подробно обобщения рассматриваются в главе 13.

Упражнение для самопроверки по материалу главы 7

1. Имеет ли суперкласс доступ к членам подкласса? Имеет ли подкласс доступ к членам суперкласса?
2. Создайте подкласс `Circle`, производный от класса `TwoDShape`. В нем должен быть определен метод `area()`, вычисляющий площадь круга, а также конструктор с ключевым словом `super` для инициализации членов, унаследованных от класса `TwoDShape`.
3. Как предотвратить обращение к членам суперкласса из подкласса?
4. Опишите назначение и два варианта использования ключевого слова `super`.
5. Допустим, имеется следующая иерархия классов:

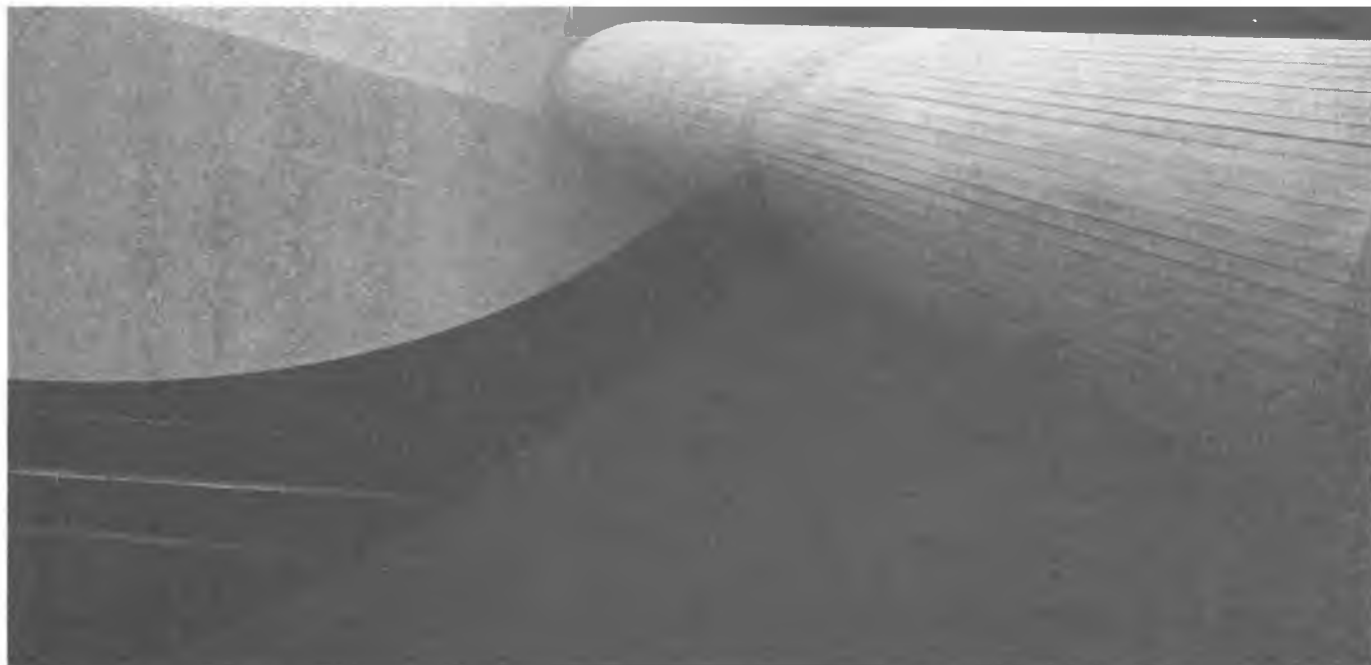
```
class Alpha { ...
```

```
class Beta extends Alpha { ...
```

```
class Gamma extends Beta { ...
```

В каком порядке вызываются конструкторы этих классов при создании объекта класса `Gamma`?

6. Переменная ссылки на суперкласс может указывать на объект подкласса. Объясните, почему это важно и как это связано с переопределением методов?
7. Что такое абстрактный класс?
8. Как предотвратить переопределение метода? И как предотвратить наследование класса?
9. Объясните, каким образом механизмы наследования, переопределения методов и абстрактные классы используются для поддержки полиморфизма.
10. Какой класс является суперклассом всех остальных классов?
11. Класс, который содержит хотя бы один абстрактный метод, должен быть объявлен абстрактным. Верно или не верно?
12. Какое ключевое слово следует использовать для создания именованной константы?



Глава 8

Пакеты и интерфейсы

Основные навыки и понятия

- Применение пакетов
- Влияние пакетов на доступ к членам класса
- Применение модификатора доступа `protected`
- Импорт пакетов
- Представление о стандартных пакетах Java
- Основные положения об интерфейсах
- Реализация интерфейсов
- Применение интерфейсных ссылок
- Представление о переменных интерфейса
- Наследование интерфейсов

Эта глава посвящена рассмотрению двух очень важных, передовых языковых средств Java: пакетов и интерфейсов. *Пакет* — это группа логически связанных классов. Пакеты помогают лучше организовать код и обеспечивают дополнительный уровень инкапсуляции. *Интерфейс* определяет набор методов, которые должны присутствовать в классе. В самом интерфейсе эти методы не реализуются, поэтому он представляет собой чисто логическую конструкцию. Пакеты и интерфейсы предоставляют дополнительную возможность для более рациональной организации программ и контроля их структуры.

Пакеты

Иногда взаимозависимые части программ оказывается удобно объединить в группу. В Java для этой цели предусмотрены пакеты. Во-первых, пакет предоставляет механизм объединения взаимосвязанных частей программы. При обращении к классам, входящим в пакет, указывается имя пакета. Таким образом, пакет предоставляет средства для именования коллекции классов. И во-вторых, пакет является частью механизма управления доступом в Java. Классы могут быть объявлены как закрытые для всех пакетов, кроме того, в который они входят. Следовательно, пакет предоставляет также средства для инкапсуляции классов. Рассмотрим все эти средства более подробно.

Как правило, при именовании класса для него выделяется имя в пространстве имен. *Пространство имен* определяет область объявлений. В Java нельзя присваивать двум классам одинаковые имена из одного и того же пространства имен. Иными словами, в пределах пространства имен каждый класс должен иметь уникальное имя. В примерах программ, представленных в предыдущих главах, по умолчанию использовалось глобальное пространство имен. Это удобно для небольших программ, но по мере увеличения объема кода могут возникнуть конфликты имен. В крупных программах бывает нелегко выбрать уникальное имя для класса. Более того, при использовании библиотек и кода, написанного другими программистами, приходится принимать специальные

меры для предотвращения конфликтов имен. Для разрешения подобных затруднений служат пакеты, позволяющие разделить пространство имен на отдельные области. Если класс определен в пакете, то имя пакета присоединяется к имени класса, в результате чего исключается конфликт между двумя классами, имеющими одинаковые имена, но принадлежащими к разным пакетам.

Пакет обычно содержит логически связанные классы, и поэтому в Java определены специальные права доступа к содержимому пакета. Так, в пакете можно определить один код, доступный другому коду из того же самого пакета, но недоступный из других пакетов. Это позволяет создавать автономные группы связанных вместе классов и делать операции в них закрытыми.

Определение пакета

Каждый класс в Java относится к тому или иному пакету. Если оператор `package` отсутствует в коде, то используется глобальный пакет, выбираемый по умолчанию. Пакет по умолчанию не имеет имени, что упрощает его применение. Именно поэтому в рассмотренных до сих пор примерах программ не нужно было беспокоиться о пакетах. Но если пакет по умолчанию подходит для очень простых программ, служащих в качестве примера, то для реальных приложений он малоприменим. Как правило, для разрабатываемого кода приходится определять один или несколько пакетов.

Для создания пакета достаточно поместить оператор `package` в начало файла с исходным кодом программы на Java. В результате классы, определенные в этом файле, будут принадлежать указанному пакету. А поскольку пакет определяет пространство имен, имена классов, содержащихся в файле, войдут в это пространство имен как составные его части.

Общая форма оператора `package` выглядит следующим образом:

```
package имя_пакета;
```

Например, приведенная ниже строка кода определяет пакет `mypack`.

```
package mypack;
```

Для управления пакетами в Java используется файловая система, в которой для хранения содержимого каждого пакета выделяется отдельный каталог. Например, файлы с расширением `.class`, содержащие классы и объявленные в пакете `mypack`, будут храниться в каталоге `mypack`.

Подобно другим именам в Java, имена пакетов зависят от регистра символов. Это означает, что каталог, предназначенный для хранения пакета, должен иметь имя, в точности совпадающее с именем пакета. Если у вас возникнут затруднения при опробовании примеров программ, представленных в этой главе, проверьте соответствие имен пакетов именам каталогов. Пакеты всегда именуется прописными буквами.

В разных файлах могут содержаться одинаковые операторы `package`. Этот оператор лишь определяет, какому именно пакету должны принадлежать классы, код которых содержится в данном файле, и не запрещает другим классам входить в состав того же самого пакета. Как правило, пакеты реальных программ распространяются на большое количество файлов.

В Java допускается создавать иерархию пакетов. Для этого достаточно разделить имена пакетов точками. Ниже приведена общая форма оператора `package` для определения многоуровневого пакета.

```
package пакет_1.пакет_2.пакет_3...пакет_N;
```

Очевидно, что для поддержки иерархии пакетов следует создать аналогичную иерархию каталогов. Например:

```
package alpha.beta.gamma;
```

Классы, содержащиеся в данном пакете, должны храниться в структуре каталогов `.../alpha/beta/gamma`, где многоточие обозначает путь к каталогу `alpha`.

Поиск пакета и переменная окружения CLASSPATH

Как пояснялось выше, иерархия каталогов пакетов должна отражать иерархию пакетов. В связи с этим возникает важный вопрос: как исполняющая система Java узнает, где искать созданные пакеты? Ответ на этот вопрос следует разделить на три части. Во-первых, по умолчанию исполняющая система Java обращается к текущему рабочему каталогу. Так, если поместить пакет в подкаталоге текущего каталога, он будет там найден. Во-вторых, один или несколько путей к каталогам можно задать в качестве значения переменной окружения `CLASSPATH`. И в-третьих, при вызове интерпретатора `java` и компилятора `javac` из командной строки можно указать параметр `-classpath`, а также путь к каталогам с классами.

Рассмотрим в качестве примера следующее определение пакета:

```
package mypack
```

Для того чтобы программа могла найти пакет `mypack`, должно быть выполнено одно из трех условий: программа должна быть запущена из каталога, содержащего пакет `mypack`; в переменной окружения `CLASSPATH` должен храниться полный путь к каталогу с пакетом `mypack`; либо при запуске программы интерпретатору `java` должен быть передан параметр `-classpath` и указан путь к каталогу с пакетом `mypack`.

Для опробования примеров программ, представленных далее в этой книге, проще всего создать в текущем каталоге, используемом для разработки программ, структуру каталогов, предназначенных для хранения пакетов, а затем разместить файлы с расширением `.class` в соответствующих каталогах и далее выполнять программы из каталога, выбранного для разработки.

Во избежание каких-либо осложнений все файлы с расширением `.java` и `.class` рекомендуется хранить в том каталоге, который выделен для содержащего их пакета. Кроме того, каждый исходный файл следует компилировать из того каталога, который находится по иерархии выше каталога с пакетом.

Простой пример применения пакета

С учетом всего сказанного выше, попробуем сначала написать несложную программу, использующую пакет, а затем запустить ее на выполнение. Эта программа предназначена для создания простой базы данных книг. Классы этой программы будут содержаться в пакете `bookpack`.

```
// Простая программа, демонстрирующая применение пакета.
// Этот файл является частью пакета bookpack.
package bookpack;
```

```
// Класс Book принадлежит пакету bookpack.
class Book {
    private String title;
    private String author;
    private int pubDate;
```

```

Book(String t, String a, int d) {
    title = t;
    author = a;
    pubDate = d;
}

void show() {
    System.out.println(title);
    System.out.println(author);
    System.out.println(pubDate);
    System.out.println();
}
}

// Класс BookDemo принадлежит пакету bookpack.
class BookDemo {
    public static void main(String args[]) {
        Book books[] = new Book[5];

        books[0] = new Book("Java: A Beginner's Guide",
                             "Schildt", 2011);
        books[1] = new Book("Java: The Complete Reference",
                             "Schildt", 2011);
        books[2] = new Book("The Art of Java",
                             "Schildt and Holmes", 2003);
        books[3] = new Book("Red Storm Rising",
                             "Clancy", 1986);
        books[4] = new Book("On the Road",
                             "Kerouac", 1955);

        for(int i=0; i < books.length; i++) books[i].show();
    }
}

```

Присвойте файлу с приведенным выше исходным кодом имя `BookDemo.java` и поместите его в каталог `bookpack`.

Скомпилируйте этот исходный файл из каталога, находящегося непосредственно над каталогом `bookpack`, указав в командной строке следующее:

```
javac bookpack/BookDemo.java
```

После этого попробуйте выполнить скомпилированную программу, указав в командной строке следующее:

```
java bookpack.BookDemo
```

Не забывайте, что для нормального выполнения указанных выше команд текущим должен быть каталог, являющийся родительским по отношению к каталогу `bookpack`. (Для компиляции и запуска программы из какого-нибудь другого каталога вам придется указать путь к каталогу `bookpack`, используя один из двух других описанных выше способов обращения к каталогам с пакетами.)

Теперь классы `BookDemo` и `Book` принадлежат пакету `bookpack`. Это означает, что при вызове интерпретатора нельзя ограничиваться передачей ему только имени класса `BookDemo`. Приведенная ниже команда не будет выполнена.

```
java BookDemo
```

Перед именем класса BookDemo следует непременно указать имя его пакета, как показано выше.

Пакеты и доступ к членам классов

В предыдущих главах были представлены основные механизмы управления доступом, в том числе модификаторы `private` и `public`. И теперь самое время продолжить обсуждение вопросов управления доступом к членам классов. Ведь и пакеты принимают участие в управлении доступом.

Область действия члена класса определяется указываемым модификатором доступа: `private`, `public` или `protected`, хотя модификатор может и отсутствовать. На формирование области действия оказывает также влияние принадлежность класса тому или иному пакету. Таким образом, область действия члена класса определяется его доступностью как в классе, так и в пакете. Столь сложный, многоуровневый подход к управлению доступом позволяет установить достаточно обширный набор прав доступа. В табл. 8.1 описаны разные уровни доступа к членам классов. Рассмотрим каждый из них в отдельности.

Таблица 8.1. Уровни доступа к членам классов

	Закрытый член	Член, доступный по умолчанию	Защищен- ный член	Открытый член
Доступен в том же классе	Да	Да	Да	Да
Доступен из подкласса в том же пакете	Нет	Да	Да	Да
Доступен из любого класса в том же пакете	Нет	Да	Да	Да
Доступен из подкласса в любом пакете	Нет	Нет	Да	Да
Доступен из всякого класса в любом пакете	Нет	Нет	Нет	Да

Если модификатор доступа явно не указан для члена класса, он доступен только в своем пакете, но не за его пределами. Следовательно, член класса, для которого не задан модификатор доступа, является открытым в текущем пакете и закрытым за его пределами.

Члены класса, объявленные как открытые (`public`), доступны из классов, принадлежащих любым пакетам. На доступ к ним никаких ограничений не накладывается. А члены класса, объявленные как закрытые (`private`), доступны только для членов того же самого класса. Другие классы, даже принадлежащие тому же самому пакету, не могут воздействовать на них. И наконец, члены класса, объявленные как защищенные (`protected`), доступны для классов, находящихся в том же самом пакете, а также для подклассов данного класса, независимо от того, каким пакетам эти подклассы принадлежат.

Правила доступа, приведенные в табл. 8.1, распространяются только на члены классов. Сами же классы могут быть объявлены как открытые или же доступные по умолчанию. Если в определении класса присутствует ключевое слово `public`, он доступен для других классов. Отсутствие модификатора доступа означает, что класс доступен только классам, находящимся в том же самом пакете. На классы, объявленные как открытые, накладывается следующее единственное ограничение: имя файла, в котором находится исходный код класса, должно совпадать с именем класса.

Пример доступа к пакету

В рассмотренном выше примере классы `Book` и `BookDemo` находились в одном и том же пакете, поэтому при организации доступа из класса `BookDemo` к классу `Book` не возникало никаких затруднений. По умолчанию все члены класса имеют право обращаться к членам других классов из того же самого пакета. Если бы класс `Book` находился в одном пакете, а класс `BookDemo` — в другом, ситуация оказалась бы немного сложнее. В этом случае доступ к классу `Book` по умолчанию был бы запрещен. Для того чтобы сделать класс `Book` доступным для других пакетов, в код программы нужно внести три изменения. Во-первых, сам класс `Book` должен быть объявлен открытым (`public`). Это позволит обращаться к нему из-за пределов пакета **bookpack**. Во-вторых, конструктор класса должен быть также объявлен открытым. И наконец, модификатор доступа `public` следует указать перед методом `show()`. Благодаря этому конструктор и метод `show()` станут доступными за пределами пакета **bookpack**. Следовательно, для использования класса `Book` в классах, принадлежащих другим пакетам, его следует объявить так, как показано ниже.

```
// Класс Book, видоизмененный для открытого доступа.
package bookpack;

// Класс Book и некоторые его члены должны быть объявлены открытыми,
// чтобы ими можно было пользоваться в других пакетах.
public class Book {
    private String title;
    private String author;
    private int pubDate;

    // Теперь конструктор объявлен открытым.
    public Book(String t, String a, int d) {
        title = t;
        author = a;
        pubDate = d;
    }

    // Теперь метод объявлен открытым.
    public void show() {
        System.out.println(title);
        System.out.println(author);
        System.out.println(pubDate);
        System.out.println();
    }
}
```

Для того чтобы воспользоваться классом `Book` в другом пакете, нужно применить оператор `import`, который будет рассматриваться в следующем разделе, либо указать полностью определенное имя класса, т.е. предварять имя класса именем пакета. Ниже приведен пример класса `UseBook`, содержащегося в пакете `bookpackext`. Для обращения к классу `Book` в нем используется полностью определенное имя этого класса.

```
// Этот класс принадлежит пакету bookpackext.
package bookpackext;

// использовать класс Book из пакета bookpack.
```

```

class UseBook {
    public static void main(String args[]) {
        // Перед именем класса Book указывается имя пакета backpack.
        backpack.Book books[] = new backpack.Book[5];
        books[0] = new backpack.Book("Java: A Beginner's Guide",
                                     "Schildt", 2011);
        books[1] = new backpack.Book("Java: The Complete Reference",
                                     "Schildt", 2001);
        books[2] = new backpack.Book("The Art of Java",
                                     "Schildt and Holmes", 2003);
        books[3] = new backpack.Book("Red Storm Rising",
                                     "Clancy", 1986);
        books[4] = new backpack.Book("On the Road",
                                     "Kerouac", 1955);

        for(int i=0; i < books.length; i++) books[i].show();
    }
}

```

Обратите внимание на то, что при каждом обращении к классу `Book` перед ним указывается имя пакета `backpack`. Если бы здесь не использовалось полностью определенное имя, то при компиляции класса `UseBook` класс `Book` не был бы найден.

Представление о защищенных членах классов

Начинающие программисты иногда неправильно пользуются модификатором доступа `protected`. Как пояснялось ранее, переменные и методы, объявленные защищенными (`protected`), доступны для классов, находящихся в том же самом пакете, а также для подклассов данного класса, независимо от того, каким пакетам эти подклассы принадлежат. Иными словами, член класса, объявленный как `protected`, доступен для подклассов, но защищен от доступа за пределами пакета.

Для того чтобы стало понятнее назначение модификатора доступа `protected`, рассмотрим следующий пример. Сначала изменим класс `Book`, объявив его переменные экземпляра защищенными, как показано ниже.

```

// Объявление защищенными переменных экземпляра в классе Book.
package BookPack;

public class Book {
    // При объявлении этих переменных использован
    // модификатор доступа protected.
    protected String title;
    protected String author;
    protected int pubDate;

    public Book(String t, String a, int d) {
        title = t;
        author = a;
        pubDate = d;
    }

    public void show() {
        System.out.println(title);
    }
}

```

Теперь создадим подкласс ExtBook класса Book, а также класс ProtectDemo, в котором будет использоваться класс ExtBook. В классе ExtBook содержится поле, предназначенное для хранения названия издательства, а также несколько методов доступа. Оба эти класса принадлежат пакету bookpackext. Их исходный код приведен ниже.

```
package bookpackext;
```

[illegible]


```

for(int i=0; i < books.length; i++) books[i].show();

// искать книги по автору
System.out.println("Showing all books by Schildt.");
for(int i=0; i < books.length; i++)
    if(books[i].getAuthor() == "Schildt")
        System.out.println(books[i].getTitle());

// Доступ к защищенному полю за пределами подклассов не разрешается.
// books[0].title = "test title"; // Ошибка: доступ запрещен!
}
}

```

Обратите внимание на код класса ExtBook. В связи с тем что класс ExtBook является подклассом, производным от класса Book, он имеет доступ к защищенным членам класса Book. Это правило действует, несмотря на то, что класс ExtBook находится в другом пакете. Следовательно, он может обращаться непосредственно к переменным экземпляра title, author и pubDate, что и было использовано при написании методов доступа. В то же время доступ к этим переменным экземпляра из класса ProtectDemo запрещен, поскольку класс ProtectDemo не является подклассом, производным от класса Book. Так, если удалить комментарии в приведенной ниже строке кода, рассматриваемая здесь программа не будет скомпилирована.

```
// books[0].title = "test title"; // Ошибка: доступ запрещен.
```

Импорт пакетов

При использовании класса из другого пакета необходимо полностью определять его имя, т.е. указывать перед именем класса имя пакета. Такой подход был принят в предыдущем примере. Но его соблюдение очень быстро становится утомительным для программирования, и особенно это касается глубоко вложенных пакетов. Язык Java был разработан программистами для программистов, и поэтому не удивительно, что в нем было предусмотрено более удобное средство доступа к содержимому пакета: оператор import. Используя этот оператор, можно упростить обращение к одному или нескольким членам пакета, чтобы пользоваться ими непосредственно, не указывая явно имя пакета.

Ниже приведена общая форма оператора import.

```
import имя_пакета.имя_класса;
```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Известно, что в языке C++ также имеется модификатор protected. Подобен ли он одноименному модификатору доступа в языке Java?

ОТВЕТ. Подобен, но не совсем. В языке C++ ключевое слово protected разрешает доступ к переменной или методу из подклассов, а для остальных классов данный член класса остается закрытым. А в Java таким способом создается член класса, который может быть доступен из любого класса в пределах пакета, но в других пакетах доступ сохраняется только для подклассов данного класса. Это отличие следует иметь в виду при переносе кода из C++ в Java и обратно.

Если требуется импортировать все содержимое пакета, вместо имени класса следует указать звездочку (*). Ниже приведены примеры обеих форм записи оператора `import`.

```
import mypack.MyClass
import mypack.*;
```

В первом случае из пакета `mypack` импортируется класс `MyClass`, а во втором — все классы из данного пакета. В исходном файле программы на Java операторы `import` должны следовать сразу же после оператора `package` (если таковой имеется) и перед определением классов.

С помощью оператора `import` можно организовать доступ к пакету `bookpack` и воспользоваться классом `Book`, не прибегая к полностью определенному имени. Оператор `import`, разрешающий данное затруднение, помещается в начало того файла, где требуется доступ к классу `Book`, в следующем виде:

```
import bookpack.*;
```

Например, так будет выглядеть исходный код класса `UseBook`, в котором используется механизм импорта пакетов:

```
// Использование ключевого слова import.
package bookpackext;
// Импорт пакета bookpack.
import bookpack.*;

// использовать класс Book из пакета bookpack
class UseBook {
    public static void main(String args[]) {
        // Теперь к членам класса Book можно обращаться непосредственно,
        // не указывая полностью определенное имя.
        Book books[] = new Book[5];

        books[0] = new Book("Java: A Beginner's Guide",
                           "Schildt", 2007);
        books[1] = new Book("Java: The Complete Reference",
                           "Schildt", 2007);
        books[2] = new Book("The Art of Java",
                           "Schildt and Holmes", 2003);
        books[3] = new Book("Red Storm Rising",
                           "Clancy", 1986);
        books[4] = new Book("On the Road",
                           "Kerouac", 1955);

        for(int i=0; i < books.length; i++) books[i].show();
    }
}
```

Как видите, теперь нет нужды предварять имя класса `Book` именем пакета.

Библиотечные классы Java, содержащиеся в пакетах

Как пояснялось ранее, в Java определено большое количество стандартных классов, доступных всем программам. Библиотека классов Java обычно называется Java API (Application Programming Interface — прикладной программный интерфейс). Классы,

входящие в состав библиотеки Java API, хранятся в пакетах. На верхней ступени иерархии находится пакет `java`. В его состав входят подчиненные пакеты, включая и перечисленные ниже.

Пакет	Описание
<code>java.lang</code>	Содержит большое количество классов общего назначения
<code>java.io</code>	Содержит классы, предназначенные для поддержки ввода-вывода
<code>java.net</code>	Содержит классы, предназначенные для поддержки сетевого взаимодействия
<code>java.applet</code>	Содержит классы, предназначенные для создания апплетов
<code>java.awt</code>	Содержит классы, обеспечивающие поддержку набора инструментальных средств Abstract Window Toolkit

В примерах программ, представленных в этой книге, с самого начала использовался пакет `java.lang`. Помимо прочего, он содержит класс `System` (к нему не раз приходилось обращаться при вызове метода `println()`). Пакет `java.lang` примечателен тем, что он автоматически включается в каждую программу на Java. А содержимое других пакетов приходится импортировать явным образом. Некоторые из этих пакетов будут рассмотрены в последующих главах книги.

Интерфейсы

Иногда в объектно-ориентированном программировании полезно определить, что именно должен делать класс, но не как он должен это делать. Примером тому может служить упоминавшийся ранее абстрактный метод. В абстрактном методе определяются возвращаемый тип и сигнатура метода, но не предоставляется его реализация. А в подклассе должна быть обеспечена своя собственная реализация каждого абстрактного метода, определенного в его суперклассе. Таким образом, абстрактный метод определяет *интерфейс*, но не реализацию метода. Конечно, абстрактные классы и методы приносят известную пользу, но положенный в их основу принцип может быть развит далее. В Java предусмотрено разделение интерфейса класса и его реализации с помощью ключевого слова `interface`.

С точки зрения синтаксиса интерфейсы подобны абстрактным классам. Но в интерфейсе ни у одного из методов не должно быть тела. Это означает, что в интерфейсе вообще не предоставляется никакой реализации. В нем указывается только, что именно следует делать, но не как это делать. Как только интерфейс будет определен, он может быть реализован в любом количестве классов. Кроме того, в одном классе может быть реализовано любое количество интерфейсов.

Для реализации интерфейса в классе должны быть предоставлены тела (т.е. конкретные реализации) методов, описанных в этом интерфейсе. Каждому классу предоставляется полная свобода для определения деталей своей собственной реализации интерфейса. Следовательно, один и тот же интерфейс может быть реализован в двух классах по-разному. Тем не менее в каждом из них должен поддерживаться один и тот же ряд методов данного интерфейса. А в том коде, где известен такой интерфейс, могут использоваться объекты любого из этих двух классов, поскольку интерфейс для всех этих объектов остается одинаковым. Благодаря поддержке интерфейсов в Java может быть в полной мере реализован главный принцип полиморфизма: “один интерфейс — множество методов”.

Интерфейсы объявляются с помощью ключевого слова `interface`. Ниже приведена упрощенная форма объявления интерфейса.

```
доступ interface имя {
    возвращаемый_тип имя_метода_1(список_параметров);
    возвращаемый_тип имя_метода_2(список_параметров);
    тип переменная_1 = значение;
    тип переменная_2 = значение;
    // ...
    возвращаемый_тип имя_метода_N(список_параметров);
    тип переменная_N = значение;
}
```

Здесь *доступ* обозначает тип доступа, который определяется модификатором доступа `public` или вообще не указывается. Если модификатор доступа отсутствует, применяется правило, предусмотренное по умолчанию, т.е. интерфейс оказывается доступным только членам своего пакета. Ключевое слово `public` указывает на то, что интерфейс может использоваться в любом другом пакете. (Код интерфейса, объявленного как `public`, должен храниться в файле, имя которого совпадает с именем интерфейса.) А *имя* интерфейса может быть любым допустимым идентификатором.

При объявлении методов указываются их сигнатуры и возвращаемые типы. Эти методы являются, по существу, абстрактными. Как упоминалось выше, реализация метода не может содержаться в составе интерфейса. Каждый класс, в определении которого указан интерфейс, должен реализовать все методы, объявленные в интерфейсе. Методы, объявленные в интерфейсе, неявно считаются открытыми (`public`).

Переменные, объявленные в интерфейсе, не являются переменными экземпляра. Они неявно обозначаются ключевыми словами `public`, `final` и `static` и обязательно подлежат инициализации. По существу, они являются константами. Ниже приведен пример определения интерфейса. Предполагается, что этот интерфейс должен быть реализован в классе, где формируется последовательный ряд числовых значений.

```
public interface Series {
    int getNext(); // вернуть следующее по порядку число
    void reset(); // начать все с самого сначала
    void setStart(int x); // задать начальное значение
}
```

Этот интерфейс объявляется открытым (`public`), а следовательно, он может быть реализован в классе, принадлежащем любому пакету.

Реализация интерфейсов

Определенный один раз интерфейс может быть реализован одним или несколькими классами. Для реализации интерфейса в определение класса следует ввести ключевое слово `implements`, а затем определить методы, объявленные в интерфейсе. Ниже приведена общая форма реализации интерфейса в классе.

```
class имя_класса extends суперкласс implements интерфейс {
    // тело класса
}
```

Если в классе должно быть реализовано несколько интерфейсов, то имена интерфейсов указываются через запятую. Разумеется, ключевое слово `extends` и имя суперкласса указывать не обязательно.

Реализуемые методы интерфейса должны быть объявлены открытыми (`public`). А сигнатура реализованного метода должна полностью соответствовать сигнатуре, объявленной в составе интерфейса. Ниже приведен пример класса `ByTwos`, реализующего рассмотренный ранее интерфейс `Series`. В этом классе формируется последовательный ряд числовых значений, каждое из которых на два больше предыдущего.

```
// Реализация интерфейса Series.
class ByTwos implements Series {
    int start;
    int val;

    ByTwos() {
        start = 0;
        val = 0;
    }

    public int getNext() {
        val += 2;
        return val;
    }

    public void reset() {
        start = 0;
        val = 0;
    }

    public void setStart(int x) {
        start = x;
        val = x;
    }
}
```

Обратите внимание на то, что методы `getNext()`, `reset()` и `setStart()` объявлены открытыми. Это нужно сделать непременно, поскольку любой метод интерфейса неявно считается открытым для доступа. Ниже приведен пример программы, демонстрирующий применение класса `ByTwos`.

```
class SeriesDemo {
    public static void main(String args[]) {
        ByTwos ob = new ByTwos();

        for(int i=0; i < 5; i++)
            System.out.println("Next value is " +
                               ob.getNext());

        System.out.println("\nResetting");
        ob.reset();
        for(int i=0; i < 5; i++)
            System.out.println("Next value is " +
                               ob.getNext());

        System.out.println("\nStarting at 100");
        ob.setStart(100);
        for(int i=0; i < 5; i++)
```

```

        System.out.println("Next value is " +
                           ob.getNext());
    }
}

```

Выполнение этой программы дает следующий результат:

```

Next value is 2
Next value is 4
Next value is 6
Next value is 8
Next value is 10

```

```

Resetting
Next value is 2
Next value is 4
Next value is 6
Next value is 8
Next value is 10

```

```

Starting at 100
Next value is 102
Next value is 104
Next value is 106
Next value is 108
Next value is 110

```

Класс, реализующий интерфейс, может содержать дополнительные переменные и методы, что вполне допустимо. Более того, именно так в большинстве случаев и поступают программирующие на Java. Например, в приведенную ниже версию класса `ByTwos` добавлен метод `getPrevious()`, возвращающий предыдущее числовое значение.

// Реализация интерфейса **Series** и добавление метода **getPrevious()**.

```

class ByTwos implements Series {
    int start;
    int val;
    int prev;

    ByTwos() {
        start = 0;
        val = 0;
        prev = -2;
    }

    public int getNext() {
        prev = val;
        val += 2;
        return val;
    }

    public void reset() {
        start = 0;
        val = 0;
        prev = -2;
    }
}

```

```

public void setStart(int x) {
    start = x;
    val = x;
    prev = x - 2;
}

// Добавление метода, не объявленного в интерфейсе Series.
int getPrevious() {
    return prev;
}
}

```

Обратите внимание на то, что для добавления метода `getPrevious()` пришлось изменить реализацию методов, объявленных в интерфейсе `Series`. Но сам интерфейс не претерпел никаких изменений. Эти изменения не видны за пределами класса и не влияют на его использование. В этом и состоит одно из преимуществ интерфейсов.

Как пояснялось ранее, интерфейс может быть реализован каким угодно количеством классов. В качестве примера ниже приведен код класса `ByThrees`. Этот класс формирует последовательный ряд числовых значений, каждое из которых на три больше предыдущего.

```

// Еще одна реализация интерфейса Series.
class ByThrees implements Series {
    int start;
    int val;

    ByThrees() {
        start = 0;
        val = 0;
    }

    public int getNext() {
        val += 3;
        return val;
    }

    public void reset() {
        start = 0;
        val = 0;
    }

    public void setStart(int x) {
        start = x;
        val = x;
    }
}

```

Следует также иметь в виду, что если в определении класса имеется ключевое слово `implements`, но он не реализует все методы указанного интерфейса, то этот класс должен быть объявлен абстрактным (`abstract`). Объект такого класса создать нельзя, но его можно использовать в качестве суперкласса, а завершить реализацию методов интерфейса в его подклассах.

Применение интерфейсных ссылок

Как это ни покажется странным, но в Java допускается объявлять переменные ссылочного интерфейсного типа, т.е. переменные ссылки на интерфейс. Такая переменная может ссылаться на любой объект, реализующий ее интерфейс. При вызове метода для объекта по интерфейсной ссылке выполняется вариант этого метода, реализованный в классе данного объекта. Этот процесс аналогичен применению ссылки на суперкласс для доступа к объекту подкласса, как пояснялось в главе 7.

Ниже приведен пример программы, демонстрирующий применение интерфейсной ссылки. По такой ссылке в данной программе будут вызываться методы, принадлежащие классам `ByTwos` и `ByThrees`.

```
// Применение интерфейсных ссылок.
```

```
class ByTwos implements Series {
    int start;
    int val;

    ByTwos() {
        start = 0;
        val = 0;
    }

    public int getNext() {
        val += 2;
        return val;
    }

    public void reset() {
        start = 0;
        val = 0;
    }

    public void setStart(int x) {
        start = x;
        val = x;
    }
}

class ByThrees implements Series {
    int start;
    int val;

    ByThrees() {
        start = 0;
        val = 0;
    }

    public int getNext() {
        val += 3;
        return val;
    }
}
```



```

public void reset() {
    start = 0;
    val = 0;
}

public void setStart(int x) {
    start = x;
    val = x;
}
}

class SeriesDemo2 {
    public static void main(String args[]) {
        ByTwos twoOb = new ByTwos();
        ByThrees threeOb = new ByThrees();
        Series ob;

        for(int i=0; i < 5; i++) {
            ob = twoOb;
            // Обращение к объекту по интерфейсной ссылке.
            System.out.println("Next ByTwos value is " +
                               ob.getNext());
            ob = threeOb;
            // Обращение к объекту по интерфейсной ссылке.
            System.out.println("Next ByThrees value is " +
                               ob.getNext());
        }
    }
}

```

В методе `main()` переменная `ob` объявляется как ссылка на интерфейс `Series`. Это означает, что в данной переменной может храниться ссылка на любой объект, реализующий интерфейс `Series`. В данном случае в переменной `ob` сохраняется ссылка на объекты `twoOb` и `threeOb`, т.е. в разные моменты времени переменная представляет собой ссылку на объект класса `ByTwos` или же на объект класса `ByThrees`. Оба эти класса реализуют интерфейс `Series`. Переменная ссылки на интерфейс содержит сведения только о методах, объявленных в этом интерфейсе. Следовательно, переменная `ob` не может быть использована для доступа к любым другим переменным и методам, которые поддерживаются в объекте, но не объявлены в интерфейсе.

Пример для опробования 8.1.

Создание интерфейса для очереди

Исходный файл: `ICharQ.java` и `IQDemo.java`

Для того чтобы продемонстрировать истинные возможности интерфейсов, обратимся к конкретному практическому примеру. В предыдущих главах был создан класс `Queue`, реализующий простую очередь фиксированного размера для хранения символов. Но обеспечить функционирование очереди можно разными способами. В частности, очередь может быть фиксированного размера или “растущей”, линейной (т.е. переполняться при достижении верхней границы выделенной памяти) или кольцевой (в этом случае при удалении символов из очереди освобождается место для новых элементов). Кроме того, очередь может быть реализована на базе массива, связанного списка, двоичного дерева и т.д. Как бы ни была воплощена очередь, интерфейс для нее остается без изменения, т.е. методы `put()` и `get()`, определяющие этот интерфейс, выполняют

одинаковые действия независимо от внутренней организации очереди. А поскольку интерфейс для очереди не зависит от конкретной ее реализации, его нетрудно определить, а конкретные детали разработать в каждой реализации очереди по отдельности.

В этом проекте предстоит сначала создать интерфейс для очереди, хранящей символы, а затем реализовать его тремя способами. Во всех трех реализациях для хранения символов будет использоваться массив. Одна из очередей будет линейной и фиксированного размера, т.е. такая же, как и реализованная ранее. Вторая очередь будет кольцевой. В кольцевой очереди при достижении границ массива значения индексов будут автоматически изменяться таким образом, чтобы указывать на начало очереди. Таким образом, в кольцевую очередь можно будет поместить любое количество элементов, но при условии своевременного удаления элементов, включенных в нее ранее. И наконец, третья очередь будет динамической. Это означает, что ее размеры будут увеличиваться по мере необходимости.

Последовательность действий

1. Создайте файл `ICharQ.java` и введите в него следующее определение интерфейса:

```
// Интерфейс для очереди символов.
public interface ICharQ {
    // поместить символ в очередь
    void put(char ch);

    // извлечь символ из очереди
    char get();
}
```

Как видите, этот интерфейс чрезвычайно прост: в нем объявлены только два метода. Эти методы должны быть определены в любом классе, реализующем интерфейс `ICharQ`.

2. Создайте файл `IQDemo.java`.
3. Начните написание примера программы в файле `IQDemo.java` с приведенного ниже кода класса `FixedQueue`.

```
// Класс, реализующий очередь фиксированного размера
// для хранения символов.
class FixedQueue implements ICharQ {
    private char q[]; // Массив для хранения элементов очереди.
    private int putloc, getloc; // Индексы размещения и извлечения
                               // элементов очереди.

    // создать пустую очередь заданного размера
    public FixedQueue(int size) {
        q = new char[size+1]; // выделить память для очереди
        putloc = getloc = 0;
    }

    // поместить символ в очередь
    public void put(char ch) {
        if(putloc==q.length-1) {
            System.out.println(" - Queue is full.");
            return;
        }
    }
```

```

        putloc++;
        q[putloc] = ch;
    }

    // извлечь символ из очереди
    public char get() {
        if(getloc == putloc) {
            System.out.println(" - Queue is empty.");
            return (char) 0;
        }

        getloc++;
        return q[getloc];
    }
}

```

Эта реализация интерфейса `ICharQ` выполнена на основе уже знакомого вам класса `Queue`, разработанного в главе 5.

4. Добавьте в файл `IQDemo.java` приведенный ниже класс `CircularQueue`. Он реализует кольцевую очередь для хранения символов.

```

// Кольцевая очередь.
class CircularQueue implements ICharQ {
    private char q[]; // Массив для хранения элементов очереди.
    private int putloc, getloc; // Индексы размещения и извлечения
                                // элементов очереди.

    // создать пустую очередь заданного размера
    public CircularQueue(int size) {
        q = new char[size+1]; // выделить память для очереди
        putloc = getloc = 0;
    }

    // поместить символ в очередь
    public void put(char ch) {
        /* Очередь считается полной, если индекс putloc на единицу
           меньше индекса getloc или если индекс putloc указывает
           на конец массива, а индекс getloc - на его начало. */
        if(putloc+1==getloc |
            ((putloc==q.length-1) & (getloc==0))) {
            System.out.println(" - Queue is full.");
            return;
        }

        putloc++;
        if(putloc==q.length) putloc = 0; // перейти в начало массива
        q[putloc] = ch;
    }

    // извлечь символ из очереди
    public char get() {
        if(getloc == putloc) {
            System.out.println(" - Queue is empty.");

```

```

        return (char) 0;
    }

    getloc++;
    if(getloc==q.length) getloc = 0; // вернуться в начало очереди
    return q[getloc];
}
}

```

В кольцевой очереди повторно используются элементы массива, освобожденные при извлечении символов. Поэтому в нее можно поместить неограниченное число элементов (при условии, что элементы, помещенные в очередь ранее, будут вовремя удалены). Отслеживание границ массива производится очень просто (достаточно обнулить индекс по достижении верхней границы), хотя условие достижения этих границ может, на первый взгляд, показаться не совсем понятным. Кольцевая очередь переполняется не тогда, когда достигается верхняя граница массива, а тогда, когда число элементов, ожидающих извлечения из очереди, становится слишком большим. Поэтому в методе `put()` проверяется ряд условий с целью определить момент переполнения очереди. Как следует из комментариев к коду, очередь считается заполненной, если индекс `putloc` оказывается на единицу меньше индекса `getloc` или если индекс `putloc` указывает на конец массива, а индекс `getloc` — на его начало. Как и прежде, очередь считается пустой, если индексы `getloc` и `putloc` равны.

5. Введите в файл `IQDemo.java` приведенный ниже код класса `DynQueue`. Этот код реализует динамическую, или “растущую”, очередь, т.е. такую очередь, размеры которой увеличиваются, когда в ней не хватает места для символов.

```

// Динамическая очередь.
class DynQueue implements ICharQ {
    private char q[]; // Массив для хранения элементов очереди.
    private int putloc, getloc; // Индексы размещения и извлечения
                                // элементов очереди.

    // создать пустую очередь заданного размера
    public DynQueue(int size) {
        q = new char[size+1]; // выделить память для очереди
        putloc = getloc = 0;
    }

    // поместить символ в очередь
    public void put(char ch) {
        if(putloc==q.length-1){
            // увеличить размер очереди
            char t[] = new char[q.length * 2];

            // скопировать элементы в новую очередь
            for(int i=0; i < q.length; i++)
                t[i] = q[i];

            q = t;
        }

        putloc++;
    }
}

```

```

        q[putloc] = ch;
    }

    // извлечь символ из очереди
    public char get() {
        if(getloc == putloc) {
            System.out.println(" - Queue is empty.");
            return (char) 0;
        }

        getloc++;
        return q[getloc];
    }
}

```

В данной реализации при попытке поместить в заполненную очередь еще один элемент создается новый массив, размеры которого в два раза превышают размеры исходного, текущее содержимое очереди копируется в новый массив, а ссылка на него помещается в переменную `q`.

6. Для того чтобы продемонстрировать все три реализации интерфейса `ICharQ`, добавьте в файл `IQDemo.java` приведенный ниже класс, в котором для доступа ко всем трем очередям используется переменная ссылки на интерфейс `ICharQ`.

```

// Демонстрация трех реализаций интерфейса ICharQ.
class IQDemo {
    public static void main(String args[]) {
        FixedQueue q1 = new FixedQueue(10);
        DynQueue q2 = new DynQueue(5);
        CircularQueue q3 = new CircularQueue(10);

        ICharQ iQ;

        char ch;
        int i;

        iQ = q1;
        // поместить ряд символов в очередь фиксированного размера
        for(i=0; i < 10; i++)
            iQ.put((char) ('A' + i));

        // отобразить содержимое очереди
        System.out.print("Contents of fixed queue: ");
        for(i=0; i < 10; i++) {
            ch = iQ.get();
            System.out.print(ch);
        }
        System.out.println();

        iQ = q2;
        // поместить ряд символов в динамическую очередь
        for(i=0; i < 10; i++)
            iQ.put((char) ('Z' - i));
    }
}

```

```

// отобразить содержимое очереди
System.out.print("Contents of dynamic queue: ");
for(i=0; i < 10; i++) {
    ch = iQ.get();
    System.out.print(ch);
}

System.out.println();

iQ = q3;
// поместить ряд символов в кольцевую очередь
for(i=0; i < 10; i++)
    iQ.put((char) ('A' + i));

// отобразить содержимое очереди
System.out.print("Contents of circular queue: ");
for(i=0; i < 10; i++) {
    ch = iQ.get();
    System.out.print(ch);
}

System.out.println();

// поместить больше символов в кольцевую очередь
for(i=10; i < 20; i++)
    iQ.put((char) ('A' + i));

// отобразить содержимое очереди
System.out.print("Contents of circular queue: ");
for(i=0; i < 10; i++) {
    ch = iQ.get();
    System.out.print(ch);
}

System.out.println("\nStore and consume from" +
                    " circular queue.");

// поместить символы в кольцевую очередь и извлечь их оттуда
for(i=0; i < 20; i++) {
    iQ.put((char) ('A' + i));
    ch = iQ.get();
    System.out.print(ch);
}
}
}

```

7. Выполнение этой программы дает следующий результат:

```

Contents of fixed queue: ABCDEFGHIJ
Contents of dynamic queue: ZYXWVUTSRQ
Contents of circular queue: ABCDEFGHIJ
Contents of circular queue: KLMNOPQRST
Store and consume from circular queue.
ABCDEFGHIJKLMNOPQRST

```

8. А теперь попробуйте самостоятельно поупражняться в организации очередей. Создайте кольцевой вариант очереди `DynQueue`. Добавьте в интерфейс `ICharQ` метод `reset()`, устанавливающий очередь в исходное состояние. Создайте статический метод для копирования содержимого одной очереди в другую.

Переменные в интерфейсах

Как упоминалось выше, в интерфейсах могут объявляться переменные, но они неявно считаются как `public`, `static` и `final`. На первый взгляд, такие переменные находят лишь ограниченное применение, но это не совсем так. В крупных программах часто используются константы, описывающие размеры массивов, граничные и специальные значения и т.п. Для крупных программ обычно создается несколько исходных файлов, а следовательно, требуется удобный способ доступа к константам из любого файла. В Java решить эту задачу помогают интерфейсы.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как обращаться со значениями, определяемыми операторами `#define` в заголовочном файле при преобразовании программы на C++ в код Java?

ОТВЕТ. Для переноса значений, определяемых операторами `#define`, достаточно преобразовать их в переменные интерфейса.

Для того чтобы определить набор общедоступных констант, достаточно создать интерфейс, в котором объявлялись бы не методы, а только нужные константы. Каждый класс, которому требуются эти константы, должен просто “реализовать” интерфейс, чтобы сделать константы доступными. Ниже приведен несложный пример, демонстрирующий такой подход.

```
// Интерфейс, содержащий только константы.
interface IConst {
    // Константы.
    int MIN = 0;
    int MAX = 10;
    String ERRORMSG = "Boundary Error";
}

class IConstD implements IConst {
    public static void main(String args[]) {
        int nums[] = new int[MAX];

        for(int i=MIN; i < 11; i++) {
            if(i >= MAX) System.out.println(ERRORMSG);
            else {
                nums[i] = i;
                System.out.print(nums[i] + " ");
            }
        }
    }
}
```

Наследование интерфейсов

Один интерфейс может наследовать другой интерфейс, для чего служит ключевое слово `extends`. Синтаксис наследования интерфейсов ничем не отличается от того, что употребляется для наследования классов. Если класс реализует один интерфейс, наследующий другой интерфейс, в нем следует определить все методы, объявленные в интерфейсах по всей цепочке наследования. Ниже приведен пример, демонстрирующий наследование интерфейсов.

```
// Наследование интерфейсов.
interface A {
    void meth1();
    void meth2();
}

// Интерфейс В содержит методы meth1() и meth2(), а
// кроме того, в него добавляется метод meth3().
interface B extends A { // Интерфейс В наследует интерфейс А.
    void meth3();
}

// Этот класс должен реализовать все методы,
// объявленные в интерфейсах А и В.
class MyClass implements B {
    public void meth1() {
        System.out.println("Implement meth1().");
    }

    public void meth2() {
        System.out.println("Implement meth2().");
    }

    public void meth3() {
        System.out.println("Implement meth3().");
    }
}

class IFExtend {
    public static void main(String arg[]) {
        MyClass ob = new MyClass();

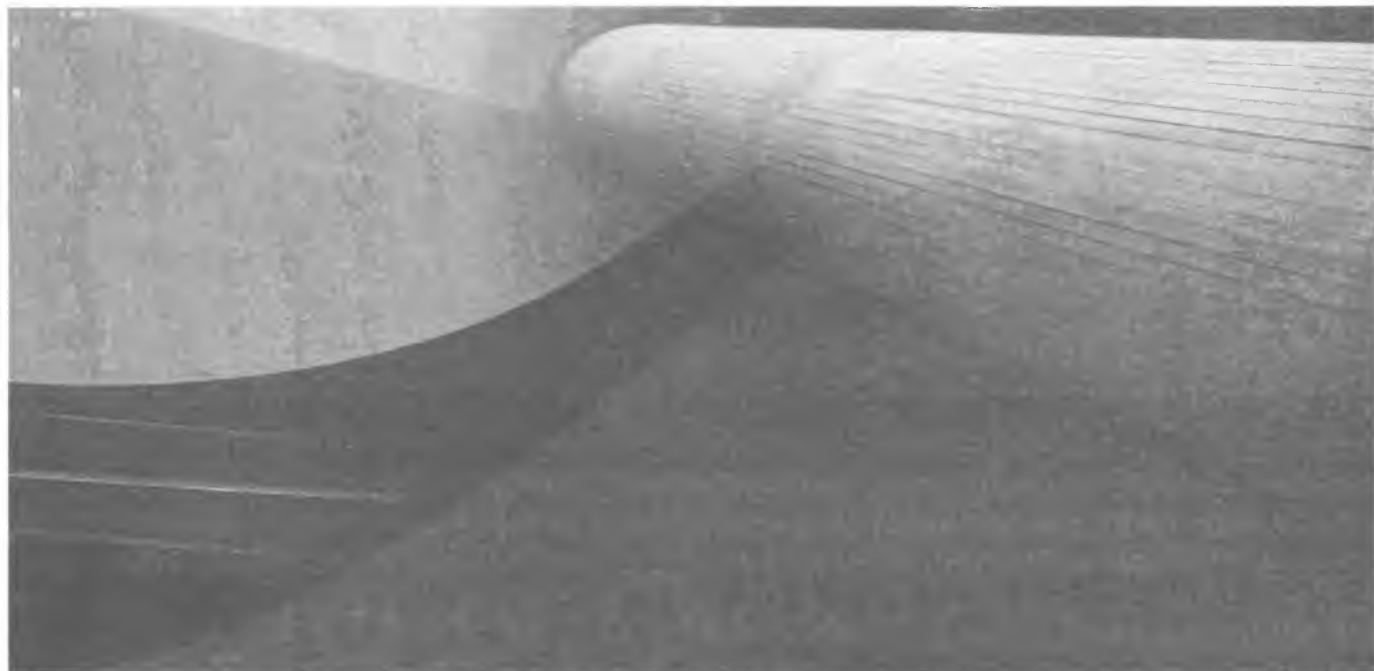
        ob.meth1();
        ob.meth2();
        ob.meth3();
    }
}
```

В качестве эксперимента можно попробовать удалить из класса `MyClass` реализацию метода `meth1()`. Это приведет к ошибке при компиляции. Как упоминалось выше, в каждом классе, реализующем интерфейс, должны быть определены все методы, объявленные в интерфейсе, в том числе те, которые были унаследованы от других интерфейсов.

И хотя пакеты и интерфейсы нечасто используются в примерах программ, представленных в этой книге, следует все же иметь в виду, что эти языковые средства являются важной частью Java. Практически во всех реальных программах и апплетах, написанных на Java, применяются пакеты, а многие классы реализуют интерфейсы. Поэтому эти языковые средства необходимо знать, чтобы уметь пользоваться ими в практике программирования на Java.

Упражнение для самопроверки по материалу главы 8

1. Используя код, созданный в примере для опробования 8.1, поместите в пакет `qrack` интерфейс `ICharQ` и все три реализующие его класса. Класс `IQDemo` должен остаться в пакете, используемом по умолчанию. Покажите, как импортировать и использовать классы из пакета `qrack`.
2. Что такое пространство имен? Почему так важна возможность его разделения на отдельные области в Java?
3. Содержимое пакетов хранится в _____.
4. В чем отличие доступа, определяемого ключевым словом `protected`, от доступа по умолчанию?
5. Допустим, классы, содержащиеся в одном пакете, требуется использовать в другом пакете. Какими двумя способами можно этого добиться?
6. “Один интерфейс — множество методов” — главный принцип Java. Какое языковое средство лучше всего демонстрирует этот принцип?
7. Сколько классов могут реализовать один и тот же интерфейс? Сколько интерфейсов может реализовать класс?
8. Может ли один интерфейс наследовать другой интерфейс?
9. Создайте интерфейс для класса `Vehicle`, рассмотренного в главе 7, назвав его `IVehicle`.
10. Переменные, объявленные в интерфейсе, неявно считаются как `static` и `final`. Какие преимущества это дает?
11. Пакет, по существу, является контейнером для классов. Верно или не верно?
12. Какой стандартный пакет импортируется по умолчанию в любую программу на Java?



Глава 9

Обработка исключений

Основные навыки и понятия

- Представление об иерархии исключений
- Использование ключевых слов `try` и `catch`
- Последствия перехвата исключений
- Применение нескольких операторов `catch`
- Перехват исключений, генерируемых подклассами
- Вложенные блоки `try`
- Генерирование исключений
- Представление о членах класса `Throwable`
- Использование ключевого слова `finally`
- Использование ключевого слова `throws`
- Представление о исключениях, встроенные в Java
- Создание специальных классов исключений

В этой главе речь пойдет об обработке исключительных ситуаций, или просто исключений. *Исключение* — это ошибка, возникающая в процессе выполнения программы. Используя подсистему обработки исключений Java, можно контролировать реакцию программы на появление ошибок в ходе ее выполнения. Средства обработки исключений в том или ином виде присутствуют практически во всех современных языках программирования. Можно смело утверждать, что в Java подобные инструментальные средства отличаются большей гибкостью, более понятны и удобны в употреблении по сравнению с большинством других языков программирования.

Преимущество обработки исключений заключается в том, что она автоматически предусматривает реакцию на многие ошибки, избавляя от необходимости писать ручную соответствующий код. Например, в некоторых старых языках программирования предусматривается возврат специального кода при возникновении ошибки в ходе выполнения метода. Этот код приходится проверять вручную при каждом вызове метода. Такой подход к обработке ошибок вручную трудоемок и чреват погрешностями. Обработка исключений упрощает этот процесс, давая возможность определять в программе блок кода, называемый *обработчиком исключения* и автоматически выполняющийся при возникновении ошибки. Это избавляет от необходимости проверять вручную, насколько удачно или неудачно была выполнена та или иная операция или вызов метода. Если возникнет ошибка, все необходимые действия по ее обработке выполнит обработчик исключений.

В Java определены стандартные исключения для наиболее часто встречающихся программных ошибок, в том числе деления на ноль или попытки открыть несуществующий файл. Для того чтобы обеспечить требуемую реакцию на конкретную ошибку, в программу следует включить соответствующий обработчик событий. Исключения широко применяются в библиотеке Java API.

Одним словом, для успешного программирования на Java нужно уметь обращаться с подсистемой обработки исключений, предусмотренной в этом языке программирования.

Иерархия исключений

В Java все исключения представлены отдельными классами. Все классы исключений являются потомками класса `Throwable`. Так, если в программе возникнет исключительная ситуация, будет сгенерирован объект класса, соответствующего определенному типу исключения. У класса `Throwable` имеются два непосредственных подкласса: `Exception` и `Error`. Исключения типа `Error` относятся к ошибкам, возникающим в виртуальной машине Java, а не в прикладной программе. Контролировать такие исключения невозможно, поэтому реакция на них в прикладной программе, как правило, не предусматривается. В связи с этим исключения данного типа не будут описываться в этой книге.

Ошибки, связанные с выполнением действий в программе, представлены отдельными подклассами, производными от класса `Exception`. К этой категории, в частности, относятся ошибки деления на ноль, выхода за границы массива и обращения к файлам. Подобные ошибки следует обрабатывать в самой программе. Важным подклассом, производным от `Exception`, является класс `RuntimeException`, который служит для представления различных видов ошибок, часто встречающихся при выполнении программ.

Общее представление об обработке исключений

Для обработки исключений в Java предусмотрены пять ключевых слов: `try`, `catch`, `throw`, `throws` и `finally`. Они образуют единую подсистему, в которой использование одного ключевого слова почти всегда автоматически влечет за собой употребление другого. Каждое из упомянутых выше ключевых слов будет подробно рассмотрено далее в этой главе. Но прежде следует дать общее представление об их роли в процессе обработки исключений. Поэтому ниже поясняется вкратце, каким образом они действуют.

Операторы, в которых требуется отслеживать появление исключений, помещаются в блок `try`. Если в блоке `try` будет сгенерировано исключение, его можно перехватить и обработать нужным образом. Системные исключения генерируются автоматически. А для того чтобы сгенерировать исключение вручную, следует воспользоваться ключевым словом `throw`. Иногда возникает потребность обрабатывать исключения за пределами метода, в котором они возникают, и для этой цели служит ключевое слово `throws`. Если же некоторый фрагмент кода должен быть выполнен обязательно и независимо от того, возникнет исключение или нет, его следует поместить в блок `finally`.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Можно ли подробнее описать условия, при которых генерируются исключения?

ОТВЕТ. Исключения генерируются при трех условиях. Во-первых, исключение может сгенерировать виртуальная машина Java в качестве реакции на некоторую внутреннюю ошибку, но такое исключение нельзя контролировать в прикладных программах. Во-вторых, причиной исключения может стать ошибка в коде программы. Примерами подобных ошибок являются деление на ноль и выход за границы массива. Такие исключения нужно обрабатывать в прикладных программах. И в-третьих, исключения можно сгенерировать вручную, используя ключевое слово `throw`. Но порядок обработки исключений не зависит от того, как они были сгенерированы.

Использование ключевых слов `try` и `catch`

Основными языковыми средствами обработки исключений являются ключевые слова `try` и `catch`. Они используются совместно. Это означает, что нельзя указать ключевое слово `catch` в коде, не указав ключевое слово `try`. Ниже приведена общая форма записи блоков `try/catch`, предназначенных для обработки исключений.

```
try {
    // Блок кода, в котором должны отслеживаться ошибки
}

catch (тип_исключения_1 объект_исключения) {
    // Обработчик исключения тип_исключения_1
}

catch (тип_исключения_2 объект_исключения) {
    // Обработчик исключения тип_исключения_2
}
.
.
.
```

В скобках, следующих за ключевым словом `catch`, указываются тип исключения и переменная, ссылающаяся на объект данного типа. Когда возникает исключение, оно перехватывается соответствующим оператором `catch`, обрабатывающим это исключение. Как следует из приведенной выше общей формы записи, с одним блоком `try` может быть связано несколько операторов `catch`. Тип исключения определяет, какой именно оператор `catch` будет выполняться. Так, если тип исключения соответствует типу оператора `catch`, то именно он и будет выполнен, а остальные операторы `catch` — пропущены. При перехвате исключения переменной, указанной в скобках после ключевого слова `catch`, присваивается ссылка на *объект_исключения*.

Следует иметь в виду, что если исключение не генерируется, блок `try` завершается обычным образом и ни один из его операторов `catch` не выполняется. Выполнение программы продолжается с первого оператора, следующего за последним оператором `catch`. Таким образом, операторы `catch` выполняются только при появлении исключения.

На заметку

В версии JDK 7 внедрена новая форма оператора `try`, поддерживающая автоматическое управления ресурсами и называемая оператором *try с ресурсами*. Более подробно она описывается в главе 10 при рассмотрении потоков ввода-вывода, в том числе и тех, что связаны с файлами, поскольку потоки ввода-вывода относятся к числу ресурсов, наиболее употребительных в прикладных программах.

Простой пример обработки исключений

Рассмотрим простой пример, демонстрирующий перехват и обработку исключения. Как известно, попытка обратиться за границы массива приводит к ошибке, и виртуальная машина Java генерирует соответствующее исключение `ArrayIndexOutOfBoundsException`. Ниже приведен код программы, в которой намеренно создаются условия для появления данного исключения, которое затем перехватывается.

```
// Демонстрация обработки исключений.
class ExcDemo1 {
    public static void main(String args[]) {
        int nums[] = new int[4];

        // Создание блока try.
        try {
            System.out.println("Before exception is generated.");

            // Попытка обратиться за границы массива.
            nums[7] = 10;
            System.out.println("this won't be displayed");
        }
        // Перехват исключения в связи с обращением за границы массива.
        catch (ArrayIndexOutOfBoundsException exc) {
            System.out.println("Index out-of-bounds!");
        }
        System.out.println("After catch statement.");
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
Before exception is generated.
Index out-of-bounds!
After catch statement.
```

Несмотря на всю простоту данного примера программы, он наглядно демонстрирует несколько важных особенностей обработки исключений. Во-первых, код, подлежащий проверке на наличие ошибок, помещается в блок `try`. И во-вторых, когда возникает исключение (в данном случае это происходит при попытке обратиться за границы массива), выполнение блока `try` прерывается и управление получает блок `catch`. Следовательно, явного обращения к блоку `catch` не происходит, но переход к нему осуществляется лишь при определенном условии, возникающем в ходе выполнения программы. Так, оператор вызова метода `println()`, следующий за выражением, в котором происходит обращение к несуществующему элементу массива, вообще не выполняется. По завершении блока `catch` выполнение программы продолжается с оператора, следующего за этим блоком. Таким образом, обработчик исключений предназначен для устранения программных ошибок, приводящих к исключительным ситуациям, а также для обеспечения нормального продолжения исполняемой программы.

Как упоминалось выше, если в блоке `try` не возникнут исключения, операторы в блоке `catch` не получают управление и выполнение программы продолжится после блока `catch`. Для того чтобы убедиться в этом, измените в предыдущей программе строку кода

```
nums[7] = 10;
```

на следующую строку кода:

```
nums[0] = 10;
```

Теперь исключение не возникнет и блок `catch` не выполнится.

Важно понимать, что исключения отслеживаются во всем коде в блоке `try`. К их числу относятся исключения, которые могут быть сгенерированы методом, вызываемым из блока `try`. Исключения, возникающие в вызываемом методе, перехватываются операторами в блоке `catch`, связанном с блоком `try`. Правда, это произойдет лишь в том

случае, если метод не обрабатывает исключения самостоятельно. Рассмотрим в качестве примера следующую программу:

```
/* Исключение может быть сгенерировано одним методом,
   а перехвачено другим. */

class ExcTest {
    // сгенерировать исключение
    static void genException() {
        int nums[] = new int[4];

        System.out.println("Before exception is generated.");

        // Здесь генерируется исключение в связи с
        // обращением за границы массива.
        nums[7] = 10;
        System.out.println("this won't be displayed");
    }
}

class ExcDemo2 {
    public static void main(String args[]) {

        try {
            ExcTest.genException();
        }
        // А здесь исключение перехватывается.
        catch (ArrayIndexOutOfBoundsException exc) {
            System.out.println("Index out-of-bounds!");
        }
        System.out.println("After catch statement.");
    }
}
```

Выполнение этой версии программы дает такой же результат, как и при выполнении ее предыдущей версии. Этот результат приведен ниже.

```
Before exception is generated.
Index out-of-bounds!
After catch statement.
```

Метод `genException()` вызывается из блока `try`, и поэтому генерируемое, но не перехватываемое в нем исключение перехватывается далее в блоке `catch` в методе `main()`. Если бы метод `genException()` сам перехватывал исключение, оно вообще не дошло бы до метода `main()`.

Последствия неперехвата исключений

Перехват стандартного исключения Java, продемонстрированный в предыдущем примере, позволяет предотвратить завершение программы вследствие ошибки. Генерируемое исключение должно быть перехвачено и обработано. Если исключение не обрабатывается в программе, оно будет обработано виртуальной машиной Java. Но дело в том, что по умолчанию виртуальная машина Java аварийно завершает программу, выводя сообщение об ошибке и трассировку стека исключений. Допустим, в предыдущем примере попытка

обращения за границы массива не отслеживается и исключение не перехватывается, как показано ниже.

```
// Обработка ошибки средствами виртуальной машины Java.
class NotHandled {
    public static void main(String args[]) {
        int nums[] = new int[4];

        System.out.println("Before exception is generated.");

        // Попытка обращения за границы массива.
        nums[7] = 10;
    }
}
```

При появлении ошибки, связанной с обращением за границы массива, выполнение программы прекращается и выводится следующее сообщение:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 7
    at NotHandled.main(NotHandled.java:9)
```

Оно полезно на этапе отладки, но пользователям программы эта информация вряд ли нужна. Именно поэтому очень важно, чтобы программы обрабатывали исключения самостоятельно и не поручали эту задачу виртуальной машине Java.

Как упоминалось выше, тип исключения должен соответствовать типу, указанному в операторе `catch`. В противном случае исключение не будет перехвачено. Так, в приведенном ниже примере программы делается попытка перехватить исключение, связанное с обращением за границы массива, с помощью оператора `catch`, в котором указан тип `ArithmeticException` еще одного встроенного в Java исключения. При неправильном обращении к массиву будет сгенерировано исключение `ArrayIndexOutOfBoundsException`, не соответствующее типу, указанному в операторе `catch`. В результате программа будет завершена аварийно.

```
// Эта программа не будет работать нормально!
class ExcTypeMismatch {
    public static void main(String args[]) {
        int nums[] = new int[4];

        try {
            System.out.println("Before exception is generated.");

            // При выполнении следующего оператора возникает
            // исключение ArrayIndexOutOfBoundsException
            nums[7] = 10;
            System.out.println("this won't be displayed");
        }

        /* Исключение, связанное с обращением за границы массива,
           нельзя обработать с помощью оператора catch, в котором
           указан тип исключения ArithmeticException. */
        catch (ArithmeticException exc) {
            System.out.println("Index out-of-bounds!");
        }
        System.out.println("After catch statement.");
    }
}
```


Ниже приведен результат выполнения данной программы.

Before exception is generated.

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 7
    at ExcTypeMismatch.main(ExcTypeMismatch.java:10)
```

Нетрудно заметить, что оператор `catch`, в котором указан тип исключения `ArithmeticException`, не может перехватить исключение `ArrayIndexOutOfBoundsException`.

Обработка исключений — изящный способ устранения программных ошибок

Одно из главных преимуществ обработки исключений заключается в том, что она позволяет вовремя отреагировать на ошибку в программе и затем продолжить ее выполнение. В качестве примера рассмотрим еще одну программу, в которой элементы одного массива делятся на элементы другого. Если при этом происходит деление на ноль, то генерируется исключение `ArithmeticException`. Обработка подобного исключения заключается в том, что программа уведомляет об ошибке и затем продолжает свое выполнение. Таким образом, попытка деления на ноль не приведет к аварийному завершению программы из-за ошибки при ее выполнении. Вместо этого ошибка обрабатывается изящно, не прерывая выполнение программы.

// Изящная обработка исключения и продолжение выполнения программы.

```
class ExcDemo3 {
    public static void main(String args[]) {
        int numer[] = { 4, 8, 16, 32, 64, 128 };
        int denom[] = { 2, 0, 4, 4, 0, 8 };

        for(int i=0; i<numer.length; i++) {
            try {
                System.out.println(numer[i] + " / " +
                                   denom[i] + " is " +
                                   numer[i]/denom[i]);
            }
            catch (ArithmeticException exc) {
                // перехватить исключение
                System.out.println("Can't divide by Zero!");
            }
        }
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
4 / 2 is 2
Can't divide by Zero!
16 / 4 is 4
32 / 4 is 8
Can't divide by Zero!
128 / 8 is 16
```

Данный пример демонстрирует еще одну важную особенность: обработанное исключение удаляется из системы. Иными словами, на каждом шаге цикла блок `try` выполня-

ется в программе сызнова, а все возникшие ранее исключения считаются обработанными. Благодаря этому в программе могут обрабатываться повторяющиеся ошибки.

Применение нескольких операторов `catch`

Как пояснялось ранее, с блоком `try` можно связать несколько операторов `catch`. Обычно разработчики так и поступают на практике. Каждый из операторов `catch` должен перехватывать отдельный тип исключений. Например, в приведенной ниже программе обрабатываются как исключения, связанные с обращением за границы массива, так и ошибки деления на ноль.

```
// Применение нескольких операторов catch.
class ExcDemo4 {
    public static void main(String args[]) {
        // Здесь массив numer длиннее массива denom.
        int numer[] = { 4, 8, 16, 32, 64, 128, 256, 512 };
        int denom[] = { 2, 0, 4, 4, 0, 8 };

        for(int i=0; i<numer.length; i++) {
            try {
                System.out.println(numer[i] + " / " +
                                   denom[i] + " is " +
                                   numer[i]/denom[i]);
            }
            // За блоком try следует несколько блоков catch подряд.
            catch (ArithmeticException exc) {
                // перехватить исключение
                System.out.println("Can't divide by Zero!");
            }
            catch (ArrayIndexOutOfBoundsException exc) {
                // перехватить исключение
                System.out.println("No matching element found.");
            }
        }
    }
}
```

Выполнение этой программы дает следующий результат:

```
4 / 2 is 2
Can't divide by Zero!
16 / 4 is 4
32 / 4 is 8
Can't divide by Zero!
128 / 8 is 16
No matching element found.
No matching element found.
```

Как подтверждает приведенный выше результат выполнения программы, в каждом блоке оператора `catch` обрабатывается свой тип исключения.

Вообще говоря, выражения с операторами `catch` проверяются в том порядке, в котором они встречаются в программе. И выполняется лишь тот из них, который соответствует типу возникшего исключения. А остальные блоки операторов `catch` просто игнорируются.

Перехват исключений, генерируемых подклассами

В отношении подклассов следует отметить еще одну интересную особенность применения нескольких операторов `catch`: условие перехвата исключений для суперкласса будет справедливо и для любых его подклассов. Например, класс `Throwable` является суперклассом для всех исключений, поэтому для перехвата всех возможных исключений в операторах `catch` следует указывать тип `Throwable`. Если же требуется перехватывать исключения типа суперкласса и типа подкласса, то в блоке операторов первым должен быть указан тип исключения, генерируемого подклассом. В противном случае вместе с исключением типа суперкласса будут перехвачены и все исключения производных от него классов. Это правило соблюдается автоматически, и если указать первым тип исключения, генерируемого суперклассом, то будет создан недостижимый код, поскольку условие перехвата исключения, генерируемого подклассом, никогда не будет выполнено. А ведь недостижимый код в Java считается ошибкой.

Рассмотрим в качестве примера следующую программу

```
// В операторах catch исключения типа подкласса должны
// предшествовать исключениям типа суперкласса.
class ExcDemo5 {
    public static void main(String args[]) {
        // Здесь массив numer длиннее массива denom.
        int numer[] = { 4, 8, 16, 32, 64, 128, 256, 512 };
        int denom[] = { 2, 0, 4, 4, 0, 8 };

        for(int i=0; i<numer.length; i++) {
            try {
                System.out.println(numer[i] + " / " +
                                   denom[i] + " is " +
                                   numer[i]/denom[i]);
            }
            // Перехват исключения от подкласса.
            catch (ArrayIndexOutOfBoundsException exc) {
                System.out.println("No matching element found.");
            }
            // Перехват исключения от суперкласса.
            catch (Throwable exc) {
                System.out.println("Some exception occurred.");
            }
        }
    }
}
```

Ниже приведен результат выполнения данной программы.

```
4 / 2 is 2
Some exception occurred.
16 / 4 is 4
32 / 4 is 8
Some exception occurred.
128 / 8 is 16
No matching element found.
No matching element found.
```

В данном случае оператор `catch(Throwable)` перехватывает все исключения, кроме `ArrayIndexOutOfBoundsException`. Соблюдение правильного порядка следования операторов `catch` приобретает особое значение в том случае, когда исключения генерируются в самой программе.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Зачем перехватывать исключения, генерируемые суперклассами?

ОТВЕТ. Для этого могут быть самые разные причины. Ограничимся рассмотрением двух из них. Во-первых, включив в программу блок `catch` для перехвата исключений типа `Exception`, вы получаете универсальный механизм обработки всех исключений, связанных с выполнением вашей программы. Такой универсальный обработчик исключений оказывается удобным, например, в тех случаях, когда требуется предотвратить аварийное завершение программы, независимо от возникшей ситуации. И во-вторых, для обработки некоторой категории исключений иногда подходит единый алгоритм. Перехватывая эти исключения, генерируемые суперклассом, можно избежать дублирования кода.

Вложенные блоки `try`

Блоки `try` могут быть вложенными друг в друга. Исключение, возникшее во внутреннем блоке `try` и не перехваченное связанным с ним блоком `catch`, распространяется далее во внешний блок `try` и обрабатывается связанным с ним блоком `catch`. Такой порядок обработки исключений демонстрируется в приведенном ниже примере программы, где исключение `ArrayIndexOutOfBoundsException` не перехватывается во внутреннем блоке `catch`, но обрабатывается во внешнем.

```
// Применение вложенных блоков try.
class NestTrys {
    public static void main(String args[]) {
        // Массив numer длиннее, чем массив denom.
        int numer[] = { 4, 8, 16, 32, 64, 128, 256, 512 };
        int denom[] = { 2, 0, 4, 4, 0, 8 };

        // Вложенные блоки try.
        try { // Внешний блок try.
            for(int i=0; i<numer.length; i++) {
                try { // Внутренний блок try.
                    System.out.println(numer[i] + " / " +
                                         denom[i] + " is " +
                                         numer[i]/denom[i]);
                }
                catch (ArithmeticException exc) {
                    // перехватить исключение
                    System.out.println("Can't divide by Zero!");
                }
            }
        }
        catch (ArrayIndexOutOfBoundsException exc) {
            // перехватить исключение
```

```

        System.out.println("No matching element found.");
        System.out.println("Fatal error - program terminated.");
    }
}

```

Выполнение этой программы может дать, например, следующий результат:

```

4 / 2 is 2
Can't divide by Zero!
16 / 4 is 4
32 / 4 is 8
Can't divide by Zero!
128 / 8 is 16
No matching element found.
Fatal error - program terminated.

```

В данном примере исключение, которое может быть обработано во внутреннем блоке `try` (в данном случае ошибка деления на нуль), не мешает дальнейшему выполнению программы. А вот ошибка превышения границ массива перехватывается во внешнем блоке `try`, что приводит к аварийному завершению программы.

Ситуация, продемонстрированная в предыдущем примере, является не единственной причиной для применения вложенных блоков `try`, хотя она встречается очень часто. В этом случае вложенные блоки `try` помогают по-разному обрабатывать разные типы ошибок. Одни ошибки невозможно устранить, а для других достаточно предусмотреть сравнительно простые действия. Внешний блок `try` чаще всего используется для перехвата критических ошибок, а менее серьезные ошибки обрабатываются во внутреннем блоке `try`.

Генерирование исключений

В предыдущих примерах программ обрабатывались исключения, автоматически генерируемые виртуальной машиной Java. Но генерировать исключения можно и вручную, используя для этого оператор `throw`. Ниже приведена общая форма этого оператора.

```
throw объект_исключения;
```

где `объект_исключения` должен быть объектом класса, производного от класса `Throwable`.

Ниже приведен пример программы, демонстрирующий применение оператора `throw`. В этой программе исключение `ArithmeticException` генерируется вручную.

```

// Генерирование исключения вручную.
class ThrowDemo {
    public static void main(String args[]) {
        try {
            System.out.println("Before throw.");
            // Генерирование исключения.
            throw new ArithmeticException();
        }
        catch (ArithmeticException exc) {
            // перехватить исключение
            System.out.println("Exception caught.");
        }
    }
}

```

```

        System.out.println("After try/catch block.");
    }
}

```

Выполнение этой программы дает следующий результат:

```

Before throw.
Exception caught.
After try/catch block.

```

Обратите внимание на то, что исключение `ArithmeticException` генерируется с помощью ключевого слова `new` в операторе `throw`. Дело в том, что оператор `throw` генерирует исключение в виде объекта. Поэтому после ключевого слова `throw` недостаточно указать только тип исключения, нужно еще создать объект для этой цели.

Повторное генерирование исключений

Исключение, перехваченное блоком `catch`, может быть повторно сгенерировано для обработки другим аналогичным блоком. Чаще всего повторное генерирование исключений применяется с целью предоставить разным обработчикам доступ к исключению. Так, например, повторное генерирование имеет смысл в том случае, если один обработчик оперирует одним свойством исключения, а другой обработчик ориентирован на другое его свойство. Повторно сгенерированное исключение не может быть перехвачено тем же самым блоком `catch`. Оно распространяется в другие блоки `catch`.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Зачем генерировать исключения вручную?

ОТВЕТ. Чаще всего генерируемые вручную исключения являются экземплярами создаваемых классов исключений. Как будет показано далее в этой главе, создание собственных классов исключений позволяет обрабатывать ошибки в рамках единой стратегии обработки исключений в разрабатываемой прикладной программе.

Ниже приведен пример программы, демонстрирующий повторное генерирование исключений.

```

// Повторное генерирование исключений.
class Rethrow {
    public static void genException() {
        // Массив numer длиннее массива denom.
        int numer[] = { 4, 8, 16, 32, 64, 128, 256, 512 };
        int denom[] = { 2, 0, 4, 4, 0, 8 };

        for(int i=0; i<numer.length; i++) {
            try {
                System.out.println(numer[i] + " / " +
                                   denom[i] + " is " +
                                   numer[i]/denom[i]);
            }
            catch (ArithmeticException exc) {
                // перехватить исключение
                System.out.println("Can't divide by Zero!");
            }
            catch (ArrayIndexOutOfBoundsException exc) {

```

```

        // перехватить исключение
        System.out.println("No matching element found.");
        throw exc; // Повторное генерирование исключения.
    }
}
}
}

class RethrowDemo {
    public static void main(String args[]) {
        try {
            Rethrow.genException();
        }
        catch (ArrayIndexOutOfBoundsException exc) {
            // Перехват повторно сгенерированного исключения.
            System.out.println("Fatal error - " +
                               "program terminated.");
        }
    }
}

```

В данной программе ошибка деления на нуль обрабатывается локально в методе `genException()`, а при попытке обращения за границы массива исключение генерируется повторно. На этот раз оно перехватывается в методе `main()`.

Подробнее о классе `Throwable`

В приведенных до сих примерах программ только перехватывались исключения, но не выполнялось никаких действий над представляющими их объектами. В выражении оператора `catch` указываются тип исключения и параметр, принимающий объект исключения. А поскольку все исключения представлены подклассами, производными от класса `Throwable`, то они поддерживают методы, определенные в этом классе. Некоторые наиболее употребительные методы из класса `Throwable` приведены в табл. 9.1.

Таблица 9.1. Наиболее употребительные методы из класса `Throwable`

Метод	Описание
<code>Throwable fillInStackTrace ()</code>	Возвращает объект типа <code>Throwable</code> , содержащий полную трассировку стека исключений. Этот объект пригоден для повторного генерирования исключений
<code>String getLocalizedMessage ()</code>	Возвращает описание исключения, локализованное по региональным стандартам
<code>String getMessage ()</code>	Возвращает описание исключения
<code>void printStackTrace ()</code>	Выводит трассировку стека исключений
<code>void printStackTrace (PrintStream stream)</code>	Выводит трассировку стека исключений в указанный поток
<code>void printStackTrace (PrintWriter stream)</code>	Направляет трассировку стека исключений в указанный поток
<code>String toString ()</code>	Возвращает объект типа <code>String</code> , содержащий полное описание исключения. Этот метод вызывается из метода <code>println()</code> при выводе объекта типа <code>Throwable</code>

Среди методов, определенных в классе `Throwable`, наибольший интерес представляют методы `printStackTrace()` и `toString()`. С помощью метода `printStackTrace()` можно вывести стандартное сообщение об ошибке и запись последовательности вызовов методов, которые привели к возникновению исключения. А метод `toString()` позволяет получить стандартное сообщение об ошибке. Этот метод также вызывается в том случае, когда объект исключения передается в качестве параметра методу `println()`. Применение этих методов демонстрируется в следующем примере программы:

```
// Применение методов из класса Throwable.
class ExcTest {
    static void genException() {
        int nums[] = new int[4];

        System.out.println("Before exception is generated.");

        // сгенерировать исключение в связи с попыткой
        // обращения за границы массива
        nums[7] = 10;
        System.out.println("this won't be displayed");
    }
}

class UseThrowableMethods {
    public static void main(String args[]) {

        try {
            ExcTest.genException();
        }
        catch (ArrayIndexOutOfBoundsException exc) {
            // перехватить исключение
            System.out.println("Standard message is: ");
            System.out.println(exc);
            System.out.println("\nStack trace: ");
            exc.printStackTrace();
        }
        System.out.println("After catch statement.");
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
Before exception is generated.
Standard message is:
java.lang.ArrayIndexOutOfBoundsException: 7

Stack trace:
java.lang.ArrayIndexOutOfBoundsException: 7
    at ExcTest.genException(UseThrowableMethods.java:10)
    at UseThrowableMethods.main(UseThrowableMethods.java:19)
After catch statement.
```


Использование ключевого слова **finally**

Иногда требуется определить кодовый блок, который должен выполняться по завершении блока `try/catch`. Допустим, в процессе работы программы возникло исключение, требующее ее преждевременного завершения. Но в программе открыт файл или установлено сетевое соединение, а следовательно, файл нужно закрыть, а соединение разорвать. Для выполнения подобных операций нормального завершения программы удобно воспользоваться ключевым словом `finally`.

Для того чтобы определить код, который должен выполняться по завершении блока `try/catch`, нужно указать блок `finally` в конце последовательности операторов `try/catch`. Ниже приведена общая форма записи блока `try/catch` вместе с блоком `finally`.

```
try {
    // Блок кода, в котором отслеживаются ошибки.
}

catch (тип_исключения_1 объект_исключения) {
    // Обработчик исключения тип_исключения_1
}

catch (тип_исключения_2 объект_исключения) {
    // Обработчик исключения тип_исключения_2
}

//...
finally {
    // Код блока finally
}
```

Блок `finally` выполняется всегда по завершении блока `try/catch` независимо от того, какое именно условие к этому привело. Следовательно, блок `finally` получит управление как при нормальной работе программы, так и при возникновении ошибки. Более того, он будет вызван даже в том случае, если в блоке `try` или в одном из блоков `catch` будет присутствовать оператор `return` для немедленного возврата из метода.

Ниже приведен краткий пример программы, демонстрирующий применение блока `finally`.

```
// Применение блока finally.
class UseFinally {
    public static void genException(int what) {
        int t;
        int nums[] = new int[2];

        System.out.println("Receiving " + what);
        try {
            switch(what) {
                case 0:
                    t = 10 / what; // сгенерировать ошибку деления на нуль
                    break;
                case 1:
                    nums[4] = 4; // сгенерировать ошибку обращения к массиву
                    break;
                case 2:
```

```

        return; // возвратиться из блока try
    }
}
catch (ArithmeticException exc) {
    // перехватить исключение
    System.out.println("Can't divide by Zero!");
    return; // возвратиться из блока catch
}
catch (ArrayIndexOutOfBoundsException exc) {
    // перехватить исключение
    System.out.println("No matching element found.");
}
// Этот блок выполняется независимо от того, каким
// образом завершается блок try/catch.
finally {
    System.out.println("Leaving try.");
}
}
}

class FinallyDemo {
    public static void main(String args[]) {

        for(int i=0; i < 3; i++) {
            UseFinally.genException(i);
            System.out.println();
        }
    }
}

```

В результате выполнения данной программы получается следующий результат:

```

Receiving 0
Can't divide by Zero!
Leaving try.

```

```

Receiving 1
No matching element found.
Leaving try.

```

```

Receiving 2
Leaving try.

```

Нетрудно заметить, что блок `finally` выполняется независимо от того, каким образом завершается блок `try/catch`.

Использование ключевого слова **throws**

Иногда исключения нецелесообразно обрабатывать в том методе, в котором они возникают. В таком случае их следует указывать с помощью ключевого слова `throws`. Ниже приведена общая форма объявления метода, в котором присутствует ключевое слово `throws`.

```
возвращаемый_тип имя_метода(список_параметров) throws список_исключений {
    // Тело метода
}
```

В списке исключений через запятую указываются исключения, которые может генерировать метод.

Возможно, вам покажется странным, что в ряде предыдущих примеров ключевое слово `throws` не указывалось при генерировании исключений за пределами методов. Дело в том, что исключения, генерируемые подклассом `Error` или `RuntimeException`, можно и не указывать в списке оператора `throws`. Исполняющая система Java по умолчанию предполагает, что метод может их генерировать. А исключения всех остальных типов следует *непрерывно* объявить с помощью ключевого слова `throws`. Если этого не сделать, возникнет ошибка при компиляции.

Пример применения оператора `throws` уже был представлен ранее в этой книге. Напомним, что при организации ввода с клавиатуры в метод `main()` потребовалось включить следующее выражение:

```
throws java.io.IOException
```

Теперь вы знаете, зачем это было нужно. При вводе данных может возникнуть исключение `IOException`, а на тот момент вы еще не знали, как оно обрабатывается. Поэтому мы и указали, что исключение должно обрабатываться за пределами метода `main()`. Теперь, ознакомившись с исключениями, вы сможете без труда обработать исключение `IOException` самостоятельно.

Рассмотрим пример, в котором осуществляется обработка исключения `IOException`. В методе `prompt()` отображается сообщение, а затем выполняется ввод символов с клавиатуры. Такой ввод данных может привести к возникновению исключения `IOException`. Но это исключение не обрабатывается в методе `prompt()`. Вместо этого в объявлении метода указан оператор `throws`, т.е. обязанности по обработке данного исключения поручаются вызывающему методу. В данном случае вызывающим является метод `main()`, в котором и перехватывается исключение.

```
// Применение ключевого слова throws.
class ThrowsDemo {
    // Обратите внимание на оператор throws в объявлении метода.
    public static char prompt(String str)
        throws java.io.IOException {

        System.out.print(str + ": ");
        return (char) System.in.read();
    }

    public static void main(String args[]) {
        char ch;

        try {
            // В методе prompt() может быть сгенерировано исключение,
            // поэтому данный метод следует вызывать в блоке try.
            ch = prompt("Enter a letter");
        }
        catch(java.io.IOException exc) {
            System.out.println("I/O exception occurred.");
            ch = 'X';
        }
    }
}
```

```

        System.out.println("You pressed " + ch);
    }
}

```

Обратите внимание на одну особенность приведенного выше примера. Класс `IOException` относится к пакету `java.io`. Как будет разъяснено в главе 10, в этом пакете содержатся многие языковые средства Java для организации ввода-вывода. Следовательно, пакет `java.io` можно импортировать, а в программе указать только имя класса `IOException`.

Новые средства обработки исключений, внедренные в версии JDK 7

С появлением версии JDK 7 механизм обработки исключений в Java был значительно усовершенствован благодаря внедрению трех новых средств. Первое из них поддерживает *автоматическое управление ресурсами*, позволяющее автоматизировать процесс освобождения таких ресурсов, как файлы, когда они больше не нужны. В основу этого средства положена расширенная форма оператора `try`, называемая *оператором try с ресурсами* и описываемая в главе 10 при рассмотрении файлов. Второе новое средство называется *многократным перехватом*, а третье — *окончательным* или *более точным повторным генерированием* исключений. Два последних средства рассматриваются ниже.

Многократный перехват позволяет перехватывать два или более исключения одним оператором `catch`. Как пояснялось ранее, после оператора `try` можно (и даже принято) указывать два или более оператора `catch`. И хотя каждый блок оператора `catch`, как правило, содержит свою особую кодовую последовательность, нередко в двух или более блоках оператора `catch` выполняется *одна и та же* кодовая последовательность, несмотря на то, что в них перехватываются разные исключения. Вместо того чтобы перехватывать каждый тип исключения в отдельности, теперь можно воспользоваться единым блоком оператора `catch` для обработки исключений, не дублируя код.

Для организации многократного перехвата следует указать список исключений в одном операторе `catch`, разделив их типы оператором поразрядного ИЛИ. Каждый параметр многократного перехвата неявно указывается как `final`. (По желанию модификатор доступа `final` можно указать и явным образом, но это совсем не обязательно.) А поскольку каждый параметр многократного перехвата неявно указывается как `final`, то ему нельзя присвоить новое значение.

В приведенной ниже строке кода показывается, каким образом многократный перехват исключений `ArithmeticException` и `ArrayIndexOutOfBoundsException` указывается в одном операторе `catch`.

```
catch(final ArithmeticException | ArrayIndexOutOfBoundsException e) {
```

Ниже приведен краткий пример программы, демонстрирующий применение многократного перехвата исключений.

```

// Применение средства многократного перехвата исключений.
// Примечание: для компиляции этого кода требуется JDK 7
// или более поздняя версия данного комплекта.
class MultiCatch {
    public static void main(String args[]) {
        int a=88, b=0;
        int result;
        char chrs[] = { 'A', 'B', 'C' };

```

```

for(int i=0; i < 2; i++) {
    try {
        if(i == 0)
            // сгенерировать исключение ArithmeticException
            result = a / b;
        else
            // сгенерировать исключение ArrayIndexOutOfBoundsException
            chrs[5] = 'X';
    }
    // В этом операторе catch организуется перехват обоих исключений.
    catch(ArithmeticException | ArrayIndexOutOfBoundsException e) {
        System.out.println("Exception caught: " + e);
    }
}

System.out.println("After multi-catch.");
}
}

```

В данном примере программы исключение `ArithmeticException` генерируется при попытке деления на нуль, а исключение `ArrayIndexOutOfBoundsException` — при попытке обращения за границы массива `chrs`. Оба исключения перехватываются одним оператором `catch`.

Средство более точного повторного генерирования исключений ограничивает этот процесс лишь теми проверяемыми типами исключений, которые генерируются в соответствующем блоке `try` и не обрабатываются в предыдущем блоке оператора `catch`, а также относятся к подтипу или супертипу указываемого параметра. И хотя такая возможность требуется нечасто, ничто не мешает теперь воспользоваться ею в полной мере. А для организации окончательного повторного генерирования исключений параметр оператора `catch` должен быть, по существу, указан как `final`. Это означает, что ему нельзя присвоить новое значение в блоке `catch`. Он может быть указан как `final` явным образом, хотя это и не обязательно.

Встроенные в Java исключения

В стандартном пакете `java.lang` определены некоторые классы, представляющие стандартные исключения Java. Часть из них использовалась в предыдущих примерах программ. Наиболее часто встречаются исключения из подклассов стандартного класса `RuntimeException`. А поскольку пакет `java.lang` импортируется по умолчанию во все программы на Java, то исключения, производные от класса `RuntimeException`, становятся доступными автоматически. Их даже обязательно включать в список оператора `throws`. В терминологии языка Java такие исключения называют *непроверяемыми*, поскольку компилятор не проверяет, обрабатываются или генерируются подобные исключения в методе. Непроверяемые исключения, определенные в пакете `java.lang`, приведены в табл. 9.2, тогда как в табл. 9.3 — те исключения из пакета `java.lang`, которые следует непременно включать в список оператора `throws` при объявлении метода, если, конечно, в методе содержатся операторы, способные генерировать эти исключения, а их обработка не предусмотрена в теле метода. Такие исключения принято называть *проверяемыми*. В Java предусмотрен также ряд других исключений, определения которых со-

держатся в различных библиотеках классов. К их числу можно отнести упоминавшееся ранее исключение `IOException`.

Таблица 9.2. Непроверяемые исключения, определенные в пакете `java.lang`

Исключение	Описание
<code>ArithmeticException</code>	Арифметическая ошибка, например попытка деления на нуль
<code>ArrayIndexOutOfBoundsException</code>	Попытка обращения за границы массива
<code>ArrayStoreException</code>	Попытка ввести в массив элемент, несовместимый с ним по типу
<code>ClassCastException</code>	Недопустимое приведение типов
<code>EnumConstNotPresentException</code>	Попытка использования нумерованного значения, которое не было определено ранее
<code>IllegalArgumentException</code>	Недопустимый параметр при вызове метода
<code>IllegalMonitorStateException</code>	Недопустимая операция контроля, например, ожидание разблокировки потока
<code>IllegalStateException</code>	Недопустимое состояние среды выполнения или приложения
<code>IllegalThreadStateException</code>	Запрашиваемая операция несовместима с текущим состоянием потока
<code>IndexOutOfBoundsException</code>	Недопустимое значение индекса
<code>NegativeArraySizeException</code>	Создание массива отрицательного размера
<code>NullPointerException</code>	Недопустимое использование пустой ссылки
<code>NumberFormatException</code>	Неверное преобразование символьной строки в число
<code>SecurityException</code>	Попытка нарушить систему защиты
<code>StringIndexOutOfBoundsException</code>	Попытка обращения к символьной строке за ее границами
<code>TypeNotPresentException</code>	Неизвестный тип
<code>UnsupportedOperationException</code>	Неподдерживаемая операция

Таблица 9.3. Проверяемые исключения, определенные в пакете `java.lang`

Исключение	Описание
<code>ClassNotFoundException</code>	Класс не найден
<code>CloneNotSupportedException</code>	Попытка клонирования объекта, не реализующего интерфейс <code>Cloneable</code>
<code>IllegalAccessException</code>	Доступ к классу запрещен
<code>InstantiationException</code>	Попытка создания объекта абстрактного класса или интерфейса
<code>InterruptedException</code>	Прерывание одного потока другим
<code>NoSuchFieldException</code>	Требуемое поле не существует
<code>NoSuchMethodException</code>	Требуемый метод не существует
<code>ReflectiveOperationException</code>	Суперкласс исключений, связанных с рефлексией (добавлен в версии JDK 7)

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Говорят, что в языке Java поддерживаются специальные исключения, называемые *цепочечными*. Что это такое?

ОТВЕТ. Цепочечные исключения — сравнительно недавнее дополнение Java. Это языковое средство позволяет указывать одно исключение как причину другого. Представьте себе ситуацию, когда метод генерирует исключение `ArithmeticException` как реакцию на попытку деления на ноль. Но на самом деле в программе возникает ошибка ввода-вывода, из-за которой делитель устанавливается неверно. Поэтому было бы желательно уведомить вызывающую часть программы, что истинной причиной является не попытка деления на ноль, а ошибка ввода-вывода, хотя исключение `ArithmeticException` безусловно должно быть сгенерировано. И это позволяют сделать цепочечные исключения. Они применимы и в других ситуациях, когда имеют место многоуровневые исключения.

Для поддержки цепочечных исключений в класс `Throwable` введены два конструктора и два метода. Ниже приведены общие формы объявления обоих конструкторов.

```
Throwable(Throwable причинное_исключение)
Throwable(String сообщение, Throwable причинное_исключение)
```

В первой форме конструктора *причинное_исключение* обозначает предыдущее исключение, послужившее причиной текущего исключения. Вторая форма конструктора позволяет указывать не только *причинное_исключение*, но и *сообщение*. Эти же конструкторы были введены в классы `Error`, `Exception` и `RuntimeException`.

Помимо конструкторов, в классе `Throwable` были также определены методы `getCause()` и `initCause()`. Ниже приведены общие формы объявления этих методов.

```
Throwable getCause()
Throwable initCause(Throwable причинное_исключение)
```

Метод `getCause()` возвращает исключение, которое стало причиной текущего исключения. Если такого исключения не было, возвращается пустое значение `null`. А метод `initCause()` связывает *причинное_исключение* с тем исключением, которое должно быть сгенерировано, возвращая ссылку на это исключение. Подобным способом можно связать причину с исключением уже после того, как исключение было сгенерировано. Как правило, метод `initCause()` применяется для установления истинной причины исключения, сгенерированного устаревшими классами, не поддерживающими описанные выше конструкторы.

Цепочечные исключения требуются далеко не в каждой программе. Но в тех случаях, когда необходимо выяснить истинную причину исключения, это языковое средство позволяет легко решить подобную задачу.

Создание подклассов, производных от класса `Exception`

Несмотря на то что встроенные в Java исключения позволяют обрабатывать большинство ошибок, механизм обработки исключений не ограничивается только этими ошибками. В частности, можно создавать исключения для обработки потенциальных ошибок в прикладной программе. Создать исключение несложно. Для этого достаточно определить подкласс, производный от класса `Exception`, который, в свою очередь, является подклассом, порожденным классом `Throwable`. В создаваемый подкласс не обязательно включать реализацию каких-то методов. Сам факт существования такого подкласса позволяет использовать его в качестве исключения.

В классе `Exception` не определены новые методы. Он лишь наследует методы, предоставляемые классом `Throwable`. Таким образом, все исключения, включая и создаваемые вами, содержат методы класса `Throwable`. Конечно же, вы вольны переопределить в создаваемом вами классе один или несколько методов.

Ниже приведен пример, в котором создается исключение `NonIntResultException`. Оно генерируется в том случае, если результатом деления двух целых чисел является дробное число. В классе `NonIntResultException` содержатся два поля, предназначенные для хранения целых чисел, а также конструктор. В нем также переопределен метод `toString()`, что дает возможность выводить описание исключения с помощью метода `println()`.

```
// Применение специально создаваемого исключения.

// создать исключение
class NonIntResultException extends Exception {
    int n;
    int d;

    NonIntResultException(int i, int j) {
        n = i;
        d = j;
    }

    public String toString() {
        return "Result of " + n + " / " + d +
            " is non-integer.";
    }
}

class CustomExceptDemo {
    public static void main(String args[]) {

        // В массиве numer содержатся нечетные числа.
        int numer[] = { 4, 8, 15, 32, 64, 127, 256, 512 };
        int denom[] = { 2, 0, 4, 4, 0, 8 };
        for(int i=0; i<numer.length; i++) {
            try {
                if((numer[i]%2) != 0)
                    throw new
                        NonIntResultException(numer[i], denom[i]);
            }
        }
    }
}
```



```

        System.out.println(numer[i] + " / " +
                           denom[i] + " is " +
                           numer[i]/denom[i]);
    }
    catch (ArithmeticException exc) {
        // перехватить исключение
        System.out.println("Can't divide by Zero!");
    }
    catch (ArrayIndexOutOfBoundsException exc) {
        // перехватить исключение
        System.out.println("No matching element found.");
    }
    catch (NonIntResultException exc) {
        System.out.println(exc);
    }
}
}
}

```

Результат выполнения данной программы выглядит следующим образом:

```

4 / 2 is 2
Can't divide by Zero!
Result of 15 / 4 is non-integer.
32 / 4 is 8
Can't divide by Zero!
Result of 127 / 8 is non-integer.
No matching element found.
No matching element found.

```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Когда следует предусматривать обработку исключений в программе?

И в каких случаях имеет смысл самостоятельно определять классы исключений?

ОТВЕТ. Поскольку в библиотеке Java API для формирования сообщений об ошибках широко применяются исключения, обработчики исключений должны присутствовать практически во всех прикладных программах. Ответ на вопрос, следует ли в конкретном случае обрабатывать исключения, практически очевиден.

Сложнее принять решение о том, следует ли определить собственное исключение. Обычно сообщения об ошибках формируются двумя способами: путем возврата специальных кодов и с помощью исключений. Какому из этих способов отдать предпочтение? Традиционный для программирования на Java подход состоит в использовании исключений. Разумеется, полностью отказываться от обработки возвращаемого кода ошибок не стоит. Ведь в некоторых случаях такой способ оказывается очень удобным. Но исключения предоставляют более эффективный и структурированный механизм обработки ошибок. Именно так грамотные программирующие на Java организуют в своих программах реакцию на потенциальные ошибки.

Пример для опробования 9.1.**Добавление исключений в класс очереди****Исходный файл: QExcDemo.java**

В этом проекте предстоит создать два класса исключений, которые будут использоваться классом очереди, разработанным в примере для опробования 8.1. Эти исключения должны указывать на переполнение и опустошение очереди, а генерировать их будут методы `put()` и `get()` соответственно. Ради простоты эти исключения добавляются в класс `FixedQueue`, но вы можете без труда внедрить их в любые другие классы очереди, разработанные в примере для опробования 8.1.

Последовательность действий

1. Создайте файл `QExcDemo.java`.
2. Определите следующие исключения в файле `QExcDemo.java`:

```
/*
    Пример для опробования 9.1.

    Добавление обработчиков исключений в класс очереди.
*/

// Исключение, указывающее на переполнение очереди.
class QueueFullException extends Exception {
    int size;

    QueueFullException(int s) { size = s; }

    public String toString() {
        return "\nQueue is full. Maximum size is " + size;
    }
}

// Исключение, указывающее на опустошение очереди.
class QueueEmptyException extends Exception {

    public String toString() {
        return "\nQueue is empty.";
    }
}
```

Исключение `QueueFullException` генерируется при попытке поместить элемент в уже заполненную очередь, а исключение `QueueEmptyException` — в ответ на попытку извлечь элемент из пустой очереди.

3. Измените класс `FixedQueue` таким образом, чтобы при возникновении ошибки он генерировал исключение. Соответствующий код приведен ниже. Введите этот код в файл `QExcDemo.java`.

```
// Класс, реализующий очередь фиксированного размера
// для хранения символов.
class FixedQueue implements ICharQ {
    private char q[]; // Массив для хранения элементов очереди.
    private int putloc, getloc; // Индексы размещения и извлечения
                                // элементов очереди.
```

```

// создать пустую очередь заданного размера
public FixedQueue(int size) {
    q = new char[size+1]; // выделить память для очереди
    putloc = getloc = 0;
}

// поместить символ в очередь
public void put(char ch)
    throws QueueFullException {

    if(putloc==q.length-1)
        throw new QueueFullException(q.length-1);

    putloc++;
    q[putloc] = ch;
}

// извлечь символ из очереди
public char get()
    throws QueueEmptyException {

    if(getloc == putloc)
        throw new QueueEmptyException();

    getloc++;
    return q[getloc];
}
}

```

Добавление исключений в класс `FixedQueue` выполняется в два этапа. Сначала в определении методов `get()` и `put()` указывается оператор `throws` с типом генерируемого исключения. А затем в этих методах организуется генерирование исключений при возникновении ошибок. Используя исключения, можно организовать обработку ошибок в вызывающей части программы наиболее рациональным способом. Как вы помните, в предыдущих версиях рассматриваемой здесь программы выводились только сообщения об ошибках. А генерирование исключений является более профессиональным подходом к разработке данной программы.

4. Для опробования усовершенствованного класса `FixedQueue` введите в файл `QExcDemo.java` приведенный ниже исходный код класса `QExcDemo`.

```

// Демонстрация исключений при обращении с очередью.
class QExcDemo {
    public static void main(String args[]) {
        FixedQueue q = new FixedQueue(10);
        char ch;
        int i;

        try {
            // Переполнение очереди.
            for(i=0; i < 11; i++) {
                System.out.print("Attempting to store : " +
                                (char) ('A' + i));
                q.put((char) ('A' + i));
            }
        }
    }
}

```

```

        System.out.println(" - OK");
    }
    System.out.println();
}
catch (QueueFullException exc) {
    System.out.println(exc);
}
System.out.println();

try {
    // Попытка извлечь символ из пустой очереди.
    for(i=0; i < 11; i++) {
        System.out.print("Getting next char: ");
        ch = q.get();
        System.out.println(ch);
    }
}
catch (QueueEmptyException exc) {
    System.out.println(exc);
}
}
}

```

5. Класс `FixedQueue` реализует интерфейс `ICharQ`, в котором определены методы `get()` и `put()`, и поэтому интерфейс `ICharQ` необходимо изменить таким образом, чтобы в нем отражалось наличие операторов `throws`. Ниже приведен видоизмененный соответственно код интерфейса `ICharQ`. Не забывайте о том, что он должен храниться в файле `ICharQ.java`.

```

// Интерфейс очереди для хранения символов с генерированием исключений.
public interface ICharQ {
    // поместить символ в очередь
    void put(char ch) throws QueueFullException;
    // извлечь символ из очереди
    char get() throws QueueEmptyException;
}

```

6. Скомпилируйте сначала новую версию исходного файла `IQChar.java`, а затем исходный файл `QExcDemo.java` и запустите программу `QExcDemo` на выполнение. В итоге вы получите следующий результат ее выполнения:

```

Attempting to store : A - OK
Attempting to store : B - OK
Attempting to store : C - OK
Attempting to store : D - OK
Attempting to store : E - OK
Attempting to store : F - OK
Attempting to store : G - OK
Attempting to store : H - OK
Attempting to store : I - OK
Attempting to store : J - OK
Attempting to store : K
Queue is full. Maximum size is 10

```

```

Getting next char: A

```

```

Getting next char: B
Getting next char: C
Getting next char: D
Getting next char: E
Getting next char: F
Getting next char: G
Getting next char: H
Getting next char: I
Getting next char: J
Getting next char:
Queue is empty.

```

Упражнение для самопроверки по материалу главы 9

1. Какой класс находится на вершине иерархии исключений?
2. Объясните вкратце, как пользоваться ключевыми словами `try` и `catch`?
3. Какая ошибка допущена в приведенном ниже фрагменте кода?

```

// ...
vals[18] = 10;
catch (ArrayIndexOutOfBoundsException exc) {
    // обработать ошибку
}

```

4. Что произойдет, если исключение не будет перехвачено?
5. Какая ошибка допущена в приведенном ниже фрагменте кода?

```

class A extends Exception { ...

class B extends A { ...

// ...

try {
    // ...
}
catch (A exc) { ... }
catch (B exc) { ... }

```

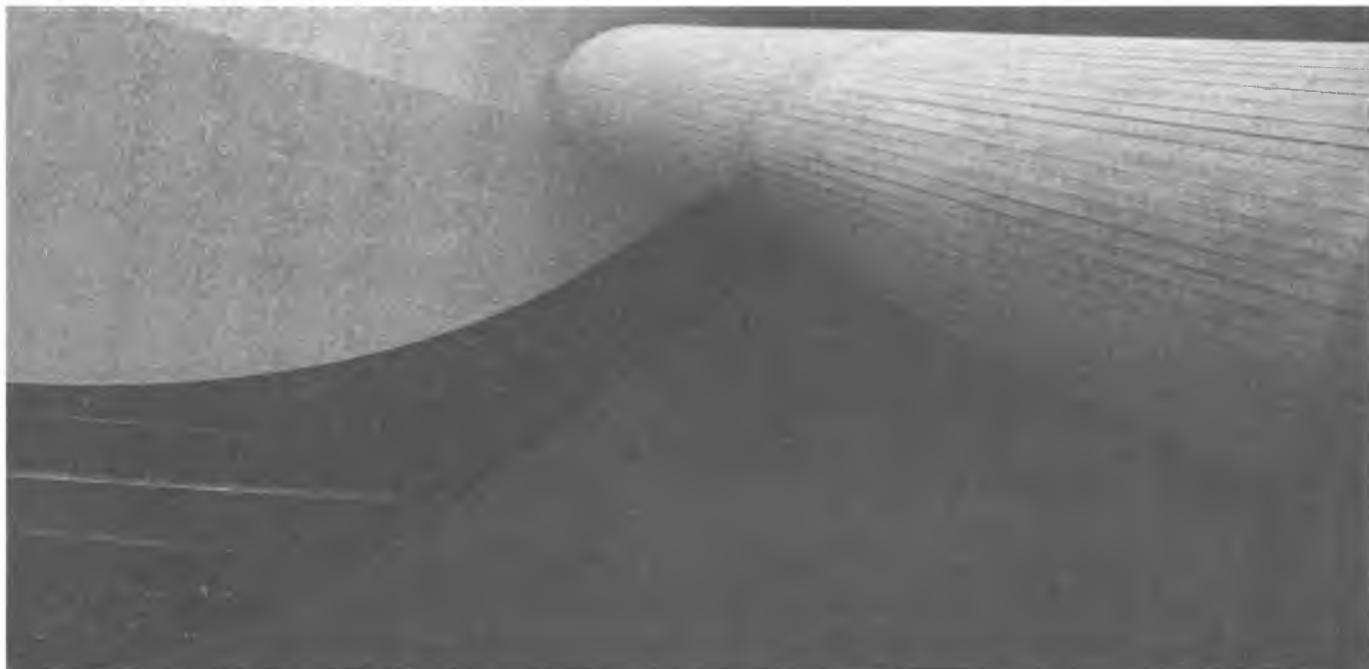
6. Может ли внутренний блок `catch` повторно генерировать исключение, которое будет обработано во внешнем блоке `catch`?
7. Блок `finally` — последний фрагмент кода, выполняемый перед завершением программы. Верно или неверно? Обоснуйте свой ответ.
8. Исключения какого типа необходимо явно объявлять с помощью оператора `throws`, включаемого в объявление метода?
9. Какая ошибка допущена в приведенном ниже фрагменте кода?

```

class MyClass { // ... }
// ...
throw new MyClass();

```

10. Отвечая на вопрос 3 упражнения для самопроверки по материалу главы 6, вы создали класс `Stack`. Добавьте в него специальные исключения для реагирования на попытку поместить элемент в переполненный стек и извлечь элемент из пустого стека.
11. Какими тремя способами можно сгенерировать исключение?
12. Назовите два подкласса, производных непосредственно от класса `Throwable`.
13. Что такое многократный перехват?
14. Следует ли перехватывать в программе исключения типа `Error`?



Глава 10

Ввод-вывод данных

Основные навыки и понятия

- Представление о потоках ввода-вывода
- Отличия байтовых и символьных потоков
- Классы для поддержки байтовых потоков
- Классы для поддержки символьных потоков
- Представление о встроенных потоках
- Применение байтовых потоков
- Использование байтовых потоков для файлового ввода-вывода
- Автоматическое закрытие файлов с помощью оператора `try` с ресурсами
- Чтение и запись двоичных данных
- Манипулирование файлами с произвольным доступом
- Применение символьных потоков
- Использование символьных потоков для файлового ввода-вывода
- Применение оболочек типов Java для преобразования символьных строк в числа

В примерах программ, приводившихся в предыдущих главах, уже применялись отдельные части системы ввода-вывода в Java, в частности метод `println()`, но делалось это без каких-либо формальных пояснений. Система ввода-вывода основана в Java на иерархии классов, поэтому ее функции и особенности нельзя было представлять до тех пор, пока не были рассмотрены классы, наследование и исключения. А теперь настал черед и для средств ввода-вывода.

Приступая к изучению системы ввода вывода Java, приготовьтесь к длительной и кропотливой работе. Система ввода-вывода в Java довольно обширна и содержит немало классов, интерфейсов и методов. Объясняется это, в частности, тем, что в Java, по существу, определены две полноценные системы ввода-вывода: одна — для обмена байтами, другая — для обмена символами. Здесь нет возможности рассмотреть все аспекты ввода-вывода в Java, ведь для этого бы потребовалась отдельная книга. Поэтому в данной главе будут рассмотрены лишь наиболее важные и часто используемые языковые средства ввода-вывода. Правда, элементы системы ввода-вывода в Java тесно взаимосвязаны, и поэтому, уяснив основы, вы легко освоите все остальные свойства этой системы.

Прежде чем приступать к рассмотрению системы ввода-вывода, необходимо сделать следующее замечание. Классы, описанные в этой главе, предназначены для консольного и файлового ввода-вывода. Они не применяются для создания графических пользовательских интерфейсов. Поэтому ими не имеет смысла пользоваться при создании оконных приложений. Для графических интерфейсов предусмотрены другие языковые средства. Они будут представлены в главе 14 при рассмотрении апплетов, а также в главе 15, служащей введением в библиотеку Swing. (Swing — это современный набор инструментальных средств, ориентированных на создание графических пользовательских интерфейсов приложений.)

Организация системы ввода-вывода в Java на потоках

Ввод-вывод в программах на Java осуществляется посредством потоков. *Поток* — это некая абстракция производства или потребления информации. С физическим устройством поток связывает система ввода-вывода. Все потоки действуют одинаково — даже если они связаны с разными физическими устройствами. Поэтому классы и методы ввода-вывода могут применяться к самым разным типам устройств. Например, методами вывода на консоль можно пользоваться и для вывода в файл на диске. Для реализации потоков используется иерархия классов, содержащихся в пакете `java.io`.

Байтовые и символьные потоки

В современных версиях Java определены два типа потоков: байтовые и символьные. (В первоначальных версиях Java были доступны только байтовые потоки, тогда как символьные потоки были реализованы в дальнейшем.) Байтовые потоки предоставляют удобные средства для ввода и вывода байтов. Они используются, например, при чтении и записи двоичных данных. В особенности они полезны для обращения с файлами. А символьные потоки ориентированы на обмен символьными данными. В них применяется кодировка в уникоде (Unicode), а следовательно, программы, в которых используются символьные потоки, легко поддаются локализации на разные языки мира. В некоторых случаях символьные потоки обеспечивают более высокую эффективность по сравнению с байтовыми.

Необходимость поддерживать два разных типа потоков ввода-вывода привела к созданию двух иерархий классов (одна для байтовых, другая для символьных данных). Из-за того что число классов достаточно велико, на первый взгляд система ввода-вывода кажется сложнее, чем она есть на самом деле. Но не следует забывать, что функциональные возможности для байтовых потоков дублируются соответствующими средствами для символьных потоков.

Следует также иметь в виду, что на самом нижнем уровне все средства ввода-вывода имеют байтовую организацию. А символьные потоки лишь предоставляют удобные и эффективные инструменты для обработки символов.

Классы байтовых потоков

Для определения байтовых потоков служат две иерархии классов. На их вершине находятся два абстрактных класса: `InputStream` и `OutputStream`. В классе `InputStream` определены свойства, общие для байтовых потоков ввода, а в классе `OutputStream` — свойства, общие для байтовых потоков вывода.

Производными от классов `InputStream` и `OutputStream` являются конкретные подклассы, реализующие различные функциональные возможности и учитывающие особенности обмена данными с разными устройствами, например ввода-вывода в файлы на диске. Классы байтовых потоков приведены в табл. 10.1. Не следует пугаться большого количества этих классов: изучив один из них, легко освоить остальные.

Таблица 10.1. Классы байтовых потоков

Класс байтового потока	Описание
<code>BufferedInputStream</code>	Буферизованный поток ввода
<code>BufferedOutputStream</code>	Буферизованный поток вывода
<code>ByteArrayInputStream</code>	Поток ввода для чтения из байтового массива
<code>ByteArrayOutputStream</code>	Поток вывода для записи в байтовый массив
<code>DataInputStream</code>	Поток ввода с методами для чтения стандартных типов данных Java
<code>DataOutputStream</code>	Поток вывода с методами для записи стандартных типов данных Java
<code>FileInputStream</code>	Поток ввода для чтения из файла
<code>FileOutputStream</code>	Поток вывода для записи в файл
<code>FilterInputStream</code>	Подкласс, производный от класса <code>InputStream</code>
<code>FilterOutputStream</code>	Подкласс, производный от класса <code>OutputStream</code>
<code>InputStream</code>	Абстрактный класс, описывающий потоковый ввод
<code>ObjectInputStream</code>	Поток для ввода объектов
<code>ObjectOutputStream</code>	Поток для вывода объектов
<code>OutputStream</code>	Абстрактный класс, описывающий потоковый вывод
<code>PipedInputStream</code>	Поток конвейерного ввода
<code>PipedOutputStream</code>	Поток конвейерного вывода
<code>PrintStream</code>	Поток вывода с методами <code>print()</code> и <code>println()</code>
<code>PushbackInputStream</code>	Поток ввода с возвратом прочитанных байтов в поток
<code>RandomAccessFile</code>	Класс, поддерживающий файловый ввод-вывод с произвольным доступом
<code>SequenceInputStream</code>	Поток ввода, сочетающий в себе несколько потоков ввода для поочередного чтения данных из них

Классы символьных потоков

Для определения символьных потоков служат две иерархические структуры классов, на вершине которых находятся абстрактные классы `Reader` и `Writer` соответственно. Класс `Reader` и его подклассы используются для чтения, а класс `Writer` и его подклассы — для записи данных. Конкретные классы, производные от классов `Reader` и `Writer`, оперируют символами в уникоде.

Классы, производные от классов `Reader` и `Writer`, предназначены для выполнения различных операций ввода-вывода символов. Символьные классы присутствуют в Java параллельно с байтовыми классами. Классы символьных потоков приведены в табл. 10.2.

Таблица 10.2. Классы символьных потоков

Класс символьного потока	Описание
<code>BufferedReader</code>	Буферизованный поток ввода символов
<code>BufferedWriter</code>	Буферизованный поток вывода символов
<code>CharArrayReader</code>	Поток ввода для чтения из символьного массива
<code>CharArrayWriter</code>	Поток вывода для записи в символьный массив

Окончание табл. 10.2

Класс символьного потока	Описание
<code>FileReader</code>	Поток ввода для чтения символов из файла
<code>FileWriter</code>	Поток вывода для записи символов в файл
<code>FilterReader</code>	Класс для чтения символов с фильтрацией
<code>FilterWriter</code>	Класс для записи символов с фильтрацией
<code>InputStreamReader</code>	Поток ввода с преобразованием байтов в символы
<code>LineNumberReader</code>	Поток ввода с подсчетом символьных строк
<code>OutputStreamWriter</code>	Поток вывода с преобразованием символов в байты
<code>PipedReader</code>	Поток конвейерного ввода
<code>PipedWriter</code>	Поток конвейерного вывода
<code>PrintWriter</code>	Поток вывода с методами <code>print()</code> и <code>println()</code>
<code>PushbackReader</code>	Поток ввода с возвратом прочитанных символов в поток
<code>Reader</code>	Абстрактный класс, описывающий потоковый ввод символов
<code>StringReader</code>	Поток ввода для чтения из символьной строки
<code>StringWriter</code>	Поток вывода для записи в символьную строку
<code>Writer</code>	Абстрактный класс, описывающий потоковый вывод символов

Встроенные потоки

Как вам должно быть уже известно, во все программы на Java автоматически импортируется пакет `java.lang`. В этом пакете определен класс `System`, инкапсулирующий некоторые элементы среды выполнения программ. Помимо прочего, в нем содержатся предопределенные переменные `in`, `out` и `err`, представляющие стандартные потоки ввода-вывода. Эти поля объявлены как `public`, `final` и `static`. А это означает, что ими можно пользоваться в любой другой части программы, не ссылаясь на конкретный объект типа `System`.

Переменная `System.out` ссылается на поток стандартного вывода. По умолчанию этот поток связан с консолью. А переменная `System.in` ссылается на поток стандартного ввода (по умолчанию с клавиатуры). И наконец, переменная `System.err` ссылается на поток стандартных сообщений об ошибках, которые по умолчанию выводятся на консоль. По мере необходимости все эти потоки могут быть перенаправлены на другие совместимые устройства ввода-вывода.

Поток `System.in` представляет собой объект типа `InputStream`, а потоки `System.out` и `System.err` — объекты типа `PrintStream`. Хотя эти потоки обычно используются для чтения и записи символов, они на самом деле являются байтовыми потоками. Дело в том, что эти потоки были определены в первоначальной спецификации Java, где символьные потоки вообще не были предусмотрены. Как станет ясно в дальнейшем, для этих потоков можно по необходимости создать оболочки, превратив их в символьные потоки.

Применение байтовых потоков

Начнем рассмотрение системы ввода-вывода в Java с байтовых потоков. Как пояснялось ранее, на вершине иерархии байтовых потоков находятся классы `InputStream`

и `OutputStream`. Методы из класса `InputStream` приведены в табл. 10.3, а методы из класса `OutputStream` — в табл. 10.4. При возникновении ошибок в процессе выполнения методы из классов `InputStream` и `OutputStream` могут генерировать исключения типа `IOException`. Методы, определенные в этих двух абстрактных классах, доступны во всех подклассах. Таким образом, они формируют минимальный набор функций ввода-вывода, общих для всех байтовых потоков.

Таблица 10.3. Методы, определенные в классе `InputStream`

Метод	Описание
<code>int available()</code>	Возвращает количество байтов, доступных для чтения
<code>void close()</code>	Закрывает поток ввода. При последующей попытке чтения из потока генерируется исключение <code>IOException</code>
<code>void mark(int numBytes)</code>	Ставит отметку на текущей позиции в потоке. Отметка доступна до тех пор, пока не будет прочитано количество байтов, определяемое параметром <code>numBytes</code>
<code>boolean markSupported()</code>	Возвращает логическое значение <code>true</code> , если методы <code>mark()</code> и <code>reset()</code> поддерживаются в вызывающем потоке
<code>int read()</code>	Возвращает целочисленное представление следующего байта в потоке. Если достигнут конец потока, возвращается значение <code>-1</code>
<code>int read(byte buffer[])</code>	Предпринимает попытку прочитать количество байтов, определяемое выражением <code>buffer.length</code> , в массив <code>buffer</code> и возвращает фактическое количество успешно прочитанных байтов. Если достигнут конец потока, возвращается значение <code>-1</code>
<code>int read(byte buffer[], int offset, int numBytes)</code>	Предпринимает попытку прочитать количество байтов, определяемое параметром <code>numBytes</code> , в массив <code>buffer</code> , начиная с элемента <code>buffer[offset]</code> . Если достигнут конец потока, возвращается значение <code>-1</code>
<code>void reset()</code>	Устанавливает указатель ввода на помеченной ранее позиции
<code>long skip(long numBytes)</code>	Пропускает количество байтов, определяемое параметром <code>numBytes</code> , в поток ввода. Возвращает фактическое количество пропущенных байтов

Таблица 10.4. Методы, определенные в классе `OutputStream`

Метод	Описание
<code>void close()</code>	Закрывает выходной поток. При последующей попытке записи в поток генерируется исключение <code>IOException</code>
<code>void flush()</code>	Выводит содержимое выходного буфера вывода в целевой поток. По завершении этой операции выходной буфер очищается
<code>void write(int b)</code>	Записывает один байт в поток вывода. Параметр <code>b</code> относится к типу <code>int</code> , что позволяет вызывать данный метод в выражениях, не приводя результат их вычисления к типу <code>byte</code>
<code>void write(byte buffer[])</code>	Записывает массив в поток вывода
<code>void write(byte buffer[], int offset, int numBytes)</code>	Записывает в поток вывода часть массива <code>buffer</code> длиной <code>numBytes</code> байтов, начиная с элемента <code>buffer[offset]</code>

Консольный ввод

Первоначально единственный способ реализовать консольный ввод в Java состоял в использовании байтовых потоков, и во многих программах на Java до сих пор используются исключительно потоки данного типа. В настоящее время для разработки прикладных программ доступны как байтовые, так и символьные потоки. В коммерческих приложениях для чтения данных с консоли в основном используются символьные потоки. Такой подход упрощает локализацию программ и их сопровождение. Ведь намного удобнее оперировать непосредственно символами и не тратить время и труд на преобразование символов в байты, а байтов — в символы. Но в простых служебных и прикладных программах, где данные, введенные с клавиатуры, обрабатываются непосредственно, удобно пользоваться байтовыми потоками. Именно поэтому они здесь и рассматриваются.

Поток `System.in` является экземпляром класса `InputStream`, и благодаря этому обеспечивается автоматический доступ к методам, определенным в классе `InputStream`. К сожалению, в классе `InputStream` определен только один метод ввода, `read()`, предназначенный для чтения байтов. Ниже приведены разные формы объявления этого метода.

```
int read() throws IOException
int read(byte data[]) throws IOException
int read(byte data[], int start, int max) throws IOException
```

В главе 3 было показано, как пользоваться первой формой метода `read()` для ввода отдельных символов с клавиатуры (а по существу, из потока стандартного ввода `System.in`). Достигнув конца потока, этот метод возвращает значение `-1`. Вторая форма метода `read()` предназначена для чтения данных из потока ввода в массив `data`. Чтение завершается по достижении конца потока, по заполнении массива или при возникновении ошибки. Метод возвращает количество прочитанных байтов или значение `-1`, если достигнут конец потока. И третья форма данного метода позволяет разместить прочитанные данные в массиве `data`, начиная с элемента, обозначаемого индексом `start`. Максимальное количество байтов, которые могут быть введены в массив, определяется параметром `max`. Метод возвращает число прочитанных байтов или значение `-1`, если достигнут конец потока. При возникновении ошибки в каждой из этих форм метода `read()` генерируется исключение `IOException`. Условие конца потока ввода `System.in` устанавливается при нажатии клавиши `<Enter>`.

Ниже приведен краткий пример программы, демонстрирующий чтение байтов из потока ввода `System.in` в массив. Следует иметь в виду, что исключения, которые могут быть сгенерированы при выполнении данной программы, обрабатываются за пределами метода `main()`. Такой подход часто используется при чтении данных с консоли. По мере необходимости вы сможете самостоятельно организовать обработку ошибок.

```
// Чтение байтов с клавиатуры в массив.
import java.io.*;

class ReadBytes {
    public static void main(String args[])
        throws IOException {
        byte data[] = new byte[10];

        System.out.println("Enter some characters.");
        // Чтение данных, введенных с клавиатуры,
```

```
// и размещение их в байтовом массиве.
System.in.read(data);
System.out.print("You entered: ");
for(int i=0; i < data.length; i++)
    System.out.print((char) data[i]);
}
}
```

Выполнение этой программы дает например, следующий результат:

```
Enter some characters.
Read Bytes
You entered: Read Bytes
```

Вывод на консоль

Как и для консольного ввода, в Java для консольного вывода первоначально были предусмотрены только байтовые потоки. Но уже в версии Java 1.1 были реализованы символьные потоки. Именно их и рекомендуется применять в прикладных программах, особенно в тех случаях, когда необходимо добиться переносимости кода. Но поскольку `System.out` является байтовым потоком вывода, он по-прежнему широко используется для побайтового вывода данных на консоль. Именно такой подход до сих пор применялся в примерах, представленных в этой книге. Поэтому он здесь и рассматривается.

Вывести данные на консоль проще всего с помощью уже знакомых вам методов `print()` и `println()`. Эти методы определены в классе `PrintStream` (на объект данного типа ссылается переменная потока стандартного вывода `System.out`). Несмотря на то что `System.out` является байтовым потоком вывода, пользоваться им вполне допустимо для организации элементарного вывода данных на консоль.

Класс `PrintStream` представляет собой выходной поток, производный от класса `OutputStream`, и поэтому в нем также реализуется метод `write()` низкоуровневого вывода. Следовательно, этот метод может быть использован для вывода данных на консоль. Самая простая форма метода `write()`, определенного в `PrintStream`, имеет следующий вид:

```
void write(int byteval)
```

Этот метод записывает в поток байтовое значение, указываемое в качестве параметра `byteval`. Несмотря на то что этот параметр объявлен как `int`, учитываются только 8 младших битов его значения. Ниже приведен простой пример программы, где метод `write()` используется для вывода символов **S** и новой строки на консоль.

```
// Применение метода System.out.write().
class WriteDemo {
    public static void main(String args[]) {
        int b;

        b = 'S';
        // Вывод байтов на экран.
        System.out.write(b);
        System.out.write('\n');
    }
}
```

На практике для вывода на консоль метод `write()` применяется достаточно редко. Для этой цели намного удобнее пользоваться методами `print()` и `println()`.

В классе `PrintStream` реализованы два дополнительных метода, `printf()` и `format()`, которые позволяют управлять форматом выводимых данных. Например, при выводе можно указать количество десятичных цифр, минимальную ширину поля или способ представления отрицательных числовых значений. И хотя эти методы не используются в примерах, представленных в данной книге, вам стоит обратить на них пристальное внимание, поскольку они могут оказаться очень полезными при написании прикладных программ.

Чтение и запись в файлы из байтовых потоков

В Java предоставляется большое количество классов и методов, позволяющих читать и записывать данные в файлы. Разумеется, чаще всего приходится обращаться к файлам, хранящимся на дисках. В Java все файлы имеют байтовую организацию, и поэтому для побайтового чтения и записи данных в такие файлы предоставляются соответствующие методы. Следовательно, организовывать чтение и запись данных в файлы из байтовых потоков приходится довольно часто. Кроме того, для байтовых потоков ввода-вывода в файлы в Java разрешено создавать специальные оболочки в виде символьных объектов. Такие оболочки будут рассмотрены далее в этой главе.

Для того чтобы создать байтовый поток и связать его с файлом, следует воспользоваться классом `FileInputStream` или `FileOutputStream`. А для открытия файла достаточно создать объект одного из этих классов, передав имя файла конструктору в качестве параметра. В открытый файл можно записывать данные или читать их из него.

Ввод данных из файла

Файл открывается для ввода созданием объекта типа `FileInputStream`. Для этой цели чаще всего используется приведенная ниже форма объявления конструктора данного класса.

```
FileInputStream(String имя_файла) throws FileNotFoundException
```

В качестве параметра этому конструктору передается *имя_файла*, который требуется открыть. Если указанный файл не существует, генерируется исключение `FileNotFoundException`.

Для чтения данных из файла служит метод `read()`. Ниже приведена форма объявления этого метода, которой мы будем пользоваться в дальнейшем.

```
int read() throws IOException
```

При каждом вызове метод `read()` читает байт из файла и возвращает его как целочисленное значение. По достижении конца файла этот метод возвращает значение `-1`. При возникновении ошибки метод генерирует исключение `IOException`. Как видите, в этой форме метод `read()` выполняет те же самые действия, что и одноименный метод для ввода данных с консоли.

Завершив операции с файлом, следует закрыть его с помощью метода `close()`, общая форма объявления которого выглядит следующим образом:

```
void close() throws IOException
```

При закрытии файла освобождаются связанные с ним системные ресурсы, чтобы использовать их для работы с другим файлом. Если же файл не будет закрыт, могут произойти “утечки памяти” из-за того, что часть памяти остается выделенной для неиспользуемых ресурсов.

Ниже приведен пример программы, где метод `read()` используется для ввода содержимого текстового файла. Имя файла задается с помощью параметра в командной строке при запуске программы на выполнение. Полученные данные выводятся на экран. Обратите внимание на то, что ошибки ввода-вывода обрабатываются с помощью блока `try/catch`.

/ Отображение текстового файла.*

При вызове этой программы следует указать имя файла, содержимое которого требуется просмотреть. Например, для вывода на экран содержимого файла `TEST.TXT`, в командной строке нужно указать следующее:

```
java ShowFile TEST.TXT
*/
import java.io.*;

class ShowFile {
    public static void main(String args[])
    {
        int i;
        FileInputStream fin;

        // Прежде всего следует убедиться, что файл был указан.
        if(args.length != 1) {
            System.out.println("Usage: ShowFile File");
            return;
        }

        try {
            // Открытие файла.
            fin = new FileInputStream(args[0]);
        } catch(FileNotFoundException exc) {
            System.out.println("File Not Found");
            return;
        }

        try {
            // читать из файла до тех пор, пока не встретится знак EOF.
            do {
                // Чтение из файла.
                i = fin.read();
                if(i != -1) System.out.print((char) i);
                // Если значение переменной i равно -1, значит,
                // достигнут конец файла.
            } while(i != -1);
        } catch(IOException exc) {
            System.out.println("Error reading file.");
        }

        try {
            // Закрытие файла.
            fin.close();
        }
    }
}
```

```

    } catch(IOException exc) {
        System.out.println("Error closing file.");
    }
}
}

```

В приведенном выше примере поток ввода из файла закрывается после того, как чтение данных из файла завершается в блоке `try`. Такой способ оказывается удобным не всегда, и поэтому в Java предоставляется более совершенный и чаще употребляемый способ. А состоит он в вызове метода `close()` в блоке `finally`. В этом случае все методы, получающие доступ к файлу, помещаются в блок `try`, а для закрытия файла используется блок `finally`. Благодаря этому файл закрывается независимо от того, как завершится блок `try`. Если продолжить предыдущий пример, то блок `try`, в котором выполняется чтение из файла, можно переписать следующим образом:

```

try {
    do {
        i = fin.read();
        if(i != -1) System.out.print((char) i);
    } while(i != -1);
} catch(IOException exc) {
    System.out.println("Error Reading File");
    // Блок finally используется для закрытия файла.
} finally {
    // закрыть файл при выходе из блока try.
    try {
        fin.close();
    } catch(IOException exc) {
        System.out.println("Error Closing File");
    }
}
}

```

Преимущество рассмотренного выше способа состоит, в частности, в том, что если программа, получающая доступ к файлу, завершается аварийно из-за какой-нибудь ошибки ввода-вывода, генерирующей исключение, файл все равно закрывается в блоке `finally`. И если с аварийным завершением простых программ, как в большинстве примеров в этой книге, из-за неожиданно возникающей исключительной ситуации еще можно как-то мириться, то в крупных программах подобная ситуация вряд ли вообще допустима. Именно ее и позволяет исключить блок `finally`.

Иногда оказывается проще заключить в оболочку те части программы, в которых открывается файл, чтобы получить доступ к нему из единственного блока `try`, не разделяя его на два блока, а для закрытия файла использовать отдельный блок `finally`. В качестве примера ниже приведена переделанная версия рассмотренной выше программы `ShowFile`.

```

/* В этой версии программы отображения текстового файла код,
   открывающий файл и получающий к нему доступ, заключается
   в единственный блок try. А закрывается файл в блоке finally.
*/
import java.io.*;

class ShowFile {
    public static void main(String args[])
    {

```

```

int i;
FileInputStream fin = null;

// Прежде всего следует убедиться, что файл был указан.
if(args.length != 1) {
    System.out.println("Usage: ShowFile filename");
    return;
}

// В следующем коде открывается файл, из которого читаются
// символы до тех пор, пока не встретится знак EOF, а затем
// файл закрывается в блоке finally.
try {
    fin = new FileInputStream(args[0]);

    do {
        i = fin.read();
        if(i != -1) System.out.print((char) i);
    } while(i != -1);
} catch(FileNotFoundException exc) {
    System.out.println("File Not Found.");
} catch(IOException exc) {
    System.out.println("An I/O Error Occurred");
} finally {
    // Файл закрывается в любом случае.
    try {
        if(fin != null) fin.close();
    } catch(IOException exc) {
        System.out.println("Error Closing File");
    }
}
}

```

Обратите внимание на то, что переменная `fin` инициализируется пустым значением `null`. А в блоке `finally` файл закрывается только в том случае, если значение переменной `fin` не является пустым. Такой способ оказывается вполне работоспособным, поскольку переменная `fin` не будет содержать пустое значение лишь в том случае, если файл был успешно открыт. Следовательно, метод `close()` не будет вызываться, если во время открытия файла возникнет исключение.

В приведенном выше примере блок `try/catch` можно сделать более компактным. Ведь исключение `FileNotFoundException` является подклассом исключения `IOException`, и поэтому его не нужно перехватывать отдельно. В качестве примера ниже приведен блок оператора `catch`, которым можно воспользоваться для перехвата обоих этих исключений, не прибегая к перехвату исключения `FileNotFoundException` в отдельности. В данном случае выводится стандартное сообщение о возникшем исключении с описанием характера ошибки.

```

...
} catch(IOException exc) {
    System.out.println("I/O Error: " + exc);
} finally {
    ...

```

В рассматриваемом здесь способе любая ошибка, в том числе и ошибка открытия файла, будет обработана единственным оператором `catch`. Благодаря своей компактности именно такой способ применяется в большинстве примеров ввода-вывода, представленных в этой книге. Следует, однако, иметь в виду, что он может оказаться не вполне пригодным в тех случаях, когда требуется отдельно обрабатывать ошибку открытия файла, например, вследствие того, что пользователь введет имя файла с опечаткой. В подобных случаях рекомендуется выдать сначала приглашение правильно ввести имя файла, а затем перейти к блоку `try` для доступа к файлу.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Метод `read()` возвращает значение `-1` по достижении конца файла, но для ошибки обращения к файлу специально возвращаемое значение не предусмотрено. Почему?

ОТВЕТ. В Java для обработки ошибок используются исключения. Поэтому если метод `read()` или любой другой метод возвращает конкретное значение, то это автоматически означает, что в процессе его работы ошибка не возникла. Такой подход многие считают гораздо более удобным, чем использование специальных кодов ошибок.

Вывод в файл

Для того чтобы открыть файл для вывода, следует создать объект типа `FileOutputStream`. Ниже приведены два наиболее часто употребляемых конструктора этого класса.

```
FileOutputStream(String имя_файла) throws FileNotFoundException
FileOutputStream(String имя_файла, boolean append)
    throws FileNotFoundException
```

Если файл не может быть создан, возникает исключение `FileNotFoundException`. В первой форме конструктора при открытии файла удаляется существовавший ранее файл с таким именем. Вторая форма отличается наличием параметра `append`. Если этот параметр принимает логическое значение `true`, записываемые данные добавляются в конец файл. В противном случае старые данные в файле перезаписываются новыми.

Для того чтобы записать данные в файл, следует вызвать метод `write()`. Наиболее простая форма этого метода приведена ниже.

```
void write(int byteval) throws IOException
```

Этот метод записывает в поток байтовое значение, указанное в качестве параметра `byteval`. Несмотря на то что этот параметр объявлен как `int`, учитываются только 8 младших битов его значения. Если в процессе записи возникнет ошибка, будет сгенерировано исключение `IOException`.

По завершении работы с файлом его нужно закрыть с помощью метода `close()`. Объявление этого метода выглядит следующим образом:

```
void close() throws IOException
```

При закрытии файла освобождаются связанные с ним системные ресурсы, чтобы использовать их для работы с другим файлом. Процедура закрытия файла также гарантирует, что данные, оставшиеся в буфере, будут записаны на диск.

В приведенном ниже примере программы осуществляется копирование текстового файла. Имена исходного и целевого файлов указываются в командной строке.

/* Копирование текстового файла.

При вызове этой программы следует указать имя исходного и целевого файлов. Например, для копирования файла FIRST.TXT в файл SECOND.TXT в командной строке нужно указать следующее:

```
java CopyFile FIRST.TXT SECOND.TXT
*/

import java.io.*;

class CopyFile {
    public static void main(String args[])
    {
        int i;
        FileInputStream fin;
        FileOutputStream fout;

        // Прежде всего следует убедиться, что оба файла были указаны.
        if(args.length !=2 ) {
            System.out.println("Usage: CopyFile From To");
            return;
        }

        // открыть исходный файл
        try {
            fin = new FileInputStream(args[0]);
        } catch(FileNotFoundException exc) {
            System.out.println("Input File Not Found");
            return;
        }

        // открыть целевой файл
        try {
            fout = new FileOutputStream(args[1]);
        } catch(FileNotFoundException exc) {
            System.out.println("Error Opening Output File");

            // закрыть исходный файл
            try {
                fin.close();
            } catch(IOException exc2) {
                System.out.println("Error closing input file.");
            }
            return;
        }

        // копировать файл
        try {
            do {
```

```

        // Чтение байтов из одного файла и запись их в другой файл.
        i = fin.read();
        if(i != -1) fout.write(i);
    } while(i != -1);
} catch(IOException exc) {
    System.out.println("File Error");
}

try {
    fin.close();
} catch(IOException exc) {
    System.out.println("Error closing input file.");
}

try {
    fout.close();
} catch(IOException exc) {
    System.out.println("Error closing output file.");
}

}
}

```

Автоматическое закрытие файлов

В примерах программ из предыдущего раздела метод `close()` вызывался явным образом для закрытия файла, когда он уже не был больше нужен. Подобным образом файлы закрывались с тех пор, как появилась первая версия Java. В итоге именно такой способ получил широкое распространение в существующих программах на Java. И до сих пор он остается вполне обоснованным и пригодным. Но в JDK 7 внедрено новое средство, предоставляющее другой, более рациональный способ управления ресурсами, в том числе и потоками файлового ввода-вывода, автоматизирующий процесс закрытия файлов. Этот способ основывается на новой разновидности оператора `try`, называемой *оператором try с ресурсами*, а иногда еще — *автоматическим управлением ресурсами*. Главное преимущество оператора `try с ресурсами` заключается в том, что он предотвращает ситуации, в которых файл (или другой ресурс) неумышленно остается неосвобожденным после того, как он уже больше не нужен. Как пояснялось ранее, если не позаботиться вовремя о закрытии файла в программе, это может привести к утечкам памяти и прочим осложнениям в работе программы.

Ниже приведена общая форма оператора `try с ресурсами`

```

try (описание_ресурса) {
    // использовать ресурс
}

```

где *описание_ресурса* обозначает оператор, в котором объявляется и инициализируется конкретный ресурс, например файл. По существу, он содержит объявление переменной, в котором переменная инициализируется ссылкой на объект управляемого ресурса. По завершении блока `try` объявленный ресурс автоматически освобождается. Если этим ресурсом является файл, то он автоматически закрывается, что избавляет от необходимости вызывать метод `close()` явным образом. В блок оператора `try с ресурсами` могут также входить операторы `catch` и `finally`.

Оператор `try` с ресурсами можно применять только к тем ресурсам, в которых реализуется интерфейс `AutoCloseable`, определенный в пакете `java.lang`. Этот интерфейс внедрен в JDK 7, и в нем определен метод `close()`. Интерфейс `AutoCloseable` наследует от интерфейса `Closeable`, определенного в пакете `java.io`. Оба интерфейса реализуются классами потоков, в том числе `FileInputStream` и `FileOutputStream`. Следовательно, оператор `try` с ресурсами может применяться вместе с потоками, включая и потоки файлового ввода-вывода.

В качестве примера ниже приведена переделанная версия программы `ShowFile`, в которой оператор `try` с ресурсами применяется для автоматического закрытия файла.

```
/* В этой версии программы ShowFile оператор try с ресурсами
   применяется для автоматического закрытия файла, когда он
   уже больше не нужен.
```

Примечание: для компиляции этого кода требуется JDK 7 или более поздняя версия данного комплекта.

```
*/
```

```
import java.io.*;
```

```
class ShowFile {
    public static void main(String args[])
    {
        int i;

        // Прежде всего следует убедиться, что оба файла были указаны.
        if(args.length != 1) {
            System.out.println("Usage: ShowFile filename");
            return;
        }

        // Ниже оператор try с ресурсами применяется сначала для открытия, а
        // затем для автоматического закрытия файла после выхода из блока try.
        try(FileInputStream fin = new FileInputStream(args[0])) {
            // Блок оператора try с ресурсами.
            do {
                i = fin.read();
                if(i != -1) System.out.print((char) i);
            } while(i != -1);

            } catch(IOException exc) {
                System.out.println("I/O Error: " + exc);
            }
        }
    }
}
```

Особое внимание в данной программе обращает на себя следующая строка кода, в которой файл открывается в операторе `try` с ресурсами.

```
try(FileInputStream fin = new FileInputStream(args[0])) {
```

Как видите, в той части оператора `try` с ресурсами, где указывается конкретный ресурс, объявляется переменная `fin` типа `FileInputStream`, которой затем присваивается ссылка на файл как объект, открываемый конструктором класса `FileInputStream`.

Следовательно, в данной версии программы переменная `fin` является локальной для блока `try` и создается при входе в этот блок. А при выходе из блока `try` файл, связанный с переменной `fin`, автоматически закрывается с помощью неявно вызываемого метода `close()`. Это означает, что метод `close()` не нужно вызывать явным образом, а следовательно, он избавляет от необходимости помнить, что файл нужно закрыть. Именно в этом и заключается главное преимущество автоматического управления ресурсами.

Следует иметь в виду, что ресурс, объявляемый в операторе `try` с ресурсами, неявно считается как `final`. Это означает, что ресурс нельзя присвоить после того, как он был создан. Кроме того, область действия ресурса ограничивается блоком оператора `try` с ресурсами.

С помощью одного оператора `try` с ресурсами можно управлять несколькими ресурсами. Для этого достаточно указать каждый из них через точку с запятой. В качестве примера ниже приведена переделанная версия рассмотренной ранее программы `CopyFile`. В этой версии оператор с ресурсами используется для управления переменными `fin` и `fout`, ссылающимися на два ресурса (в данном случае — оригинал и копию файла).

```
/* В этой версии программы CopyFile используется оператор try с
   ресурсами. В ней демонстрируется управление двумя ресурсами
   (в данном случае — файлами) с помощью единственного оператора try.
```

Примечание: для компиляции этого кода требуется JDK 7 или более поздняя версия данного комплекта.

```
*/
```

```
import java.io.*;
```

```
class CopyFile {
    public static void main(String args[]) throws IOException
    {
        int i;

        // Прежде всего следует убедиться, что оба файла были указаны.
        if(args.length != 2) {
            System.out.println("Usage: CopyFile from to");
            return;
        }

        // открыть оба файла для управления с помощью оператора try
        try (FileInputStream fin = new FileInputStream(args[0]);
            FileOutputStream fout = new FileOutputStream(args[1]))
            // Управление двумя ресурсами (в данном случае — файлами).
            {
                do {
                    i = fin.read();
                    if(i != -1) fout.write(i);
                } while(i != -1);

            } catch(IOException exc) {
                System.out.println("I/O Error: " + exc);
            }
    }
}
```


Обратите внимание на то, каким образом входной и выходной файлы открываются в операторе `try` с ресурсами, как показано ниже.

```
try (FileInputStream fin = new FileInputStream(args[0]);
    FileOutputStream fout = new FileOutputStream(args[1]))
{
```

По завершении этого блока `try` оба файла, на которые ссылаются переменные `fin` и `fout`, закрываются автоматически. Если сравнить эту версию программы с предыдущей, то можно заметить, что ее исходный код намного компактнее. Возможность писать более компактный код является еще одним, дополнительным преимуществом оператора `try` с ресурсами.

Следует также упомянуть о еще одной особенности оператора `try` с ресурсами. Вообще говоря, когда выполняется блок `try`, в нем может возникнуть одно исключение, приводящее к другому исключению при закрытии ресурса в блоке `finally`. И если это блок обычного оператора `try`, то исходное исключение теряется, прерываясь вторым исключением. А в блоке оператора `try` с ресурсами второе исключение *подавляется*. Но оно не теряется, а добавляется в список подавленных исключений, связанных с первым исключением. Этот список можно получить, вызвав метод `getSuppressed()`, определенный в классе `Throwable`.

В силу упомянутых выше преимуществ, присущих оператору `try` с ресурсами, можно ожидать, что он найдет широкое применение в программировании на Java. Поэтому именно он и будет использоваться в остальных примерах программ, представленных далее в этой главе. Но не менее важным остается и умение пользоваться рассмотренным ранее традиционным способом освобождения ресурсов с помощью вызываемого явным образом оператора `close()`. И на то имеется ряд веских оснований. Во-первых, уже существует немало написанных и повсеместно эксплуатируемых программ на Java, в которых применяется традиционный способ управления ресурсами. Поэтому все программирующие на Java должны как следует усвоить и уметь пользоваться этим традиционным способом для сопровождения устаревшего кода. Во-вторых, переход на JDK 7 может произойти не сразу, а следовательно, придется работать с предыдущей версией данного комплекта. В этом случае воспользоваться преимуществами оператора `try` с ресурсами не удастся и придется применять традиционный способ управления ресурсами. И наконец, в некоторых классах закрытие ресурса явным образом может оказаться более пригодным, чем его автоматическое освобождение. Но, несмотря на все сказанное выше, новый способ автоматического управления ресурсами считается более предпочтительным при переходе к JDK 7 или более поздней версии данного комплекта, поскольку он рациональнее и надежнее традиционного способа.

Чтение и запись двоичных данных

В приведенных до сих пор примерах программ читались и записывались байтовые значения, содержащие символы в коде ASCII. Но аналогичным образом можно также организовать чтение и запись любых типов данных. Допустим, требуется создать файл, содержащий значения типа `int`, `double` или `short`. Для чтения и записи простых типов данных в Java предусмотрены классы `DataInputStream` и `DataOutputStream`.

Класс `DataOutputStream` реализует интерфейс `DataOutput`, в котором определены методы, позволяющие записывать в файл значения любых простых типов. Следует, однако, иметь в виду, что данные записываются во внутреннем двоичном формате, а не в виде последовательности символов. Методы, наиболее часто применяемые для записи

простых типов данных в Java, приведены в табл. 10.5. Каждый из них генерирует исключение `IOException` при возникновении ошибки ввода-вывода.

Таблица 10.5. Наиболее часто употребляемые методы вывода данных, определенные в классе `DataOutputStream`

Метод вывода данных	Описание
<code>void writeBoolean(boolean val)</code>	Записывает логическое значение, определяемое параметром <code>val</code>
<code>void writeByte(int val)</code>	Записывает младший байт целочисленного значения, определяемого параметром <code>val</code>
<code>void writeChar(int val)</code>	Записывает значение, определяемое параметром <code>val</code> , интерпретируя его как символ
<code>void writeDouble(double val)</code>	Записывает значение типа <code>double</code> , определяемое параметром <code>val</code>
<code>void writeFloat(float val)</code>	Записывает значение типа <code>float</code> , определяемое параметром <code>val</code>
<code>void writeInt(int val)</code>	Записывает значение типа <code>int</code> , определяемое параметром <code>val</code>
<code>void writeLong(long val)</code>	Записывает значение типа <code>long</code> , определяемое параметром <code>val</code>
<code>void writeShort(int val)</code>	Записывает целочисленное значение, определяемое параметром <code>val</code> , преобразуя его в тип <code>short</code>

Ниже приведен конструктор класса `DataOutputStream`. Обратите внимание на то, что при вызове ему передается экземпляр класса `OutputStream`.

```
DataOutputStream(OutputStream outputStream)
```

где `outputStream` — это поток вывода, в который записываются данные. Для того чтобы организовать запись данных в файл, следует передать конструктору в качестве параметра `outputStream` объект типа `FileOutputStream`.

Класс `DataInputStream` реализует интерфейс `DataInput`, в котором объявлены методы для чтения всех простых типов данных в Java (табл. 10.6). В каждом из этих методов может быть сгенерировано исключение `IOException` при возникновении ошибки ввода-вывода. В качестве своего основания класс `DataInputStream` использует экземпляр класса `InputStream`, перекрывая его методами для чтения различных типов данных в Java. Однако в потоке типа `DataInputStream` данные читаются в двоичном виде, а не в удобной для чтения форме. Ниже приведен конструктор класса `DataInputStream`.

```
DataInputStream(InputStream inputStream)
```

где `inputStream` — это поток, связанный с создаваемым экземпляром класса `DataInputStream`. Для того чтобы организовать чтение данных из файла, следует передать конструктору в качестве параметра `inputStream` объект типа `FileInputStream`.

Таблица 10.6. Наиболее часто употребляемые методы ввода данных, определенные в классе `DataInputStream`

Метод ввода данных	Описание
<code>boolean readBoolean()</code>	Читает значение типа <code>boolean</code>

Метод ввода данных	Описание
<code>byte readByte()</code>	Читает значение типа <code>byte</code>
<code>char readChar()</code>	Читает значение типа <code>char</code>
<code>double readDouble()</code>	Читает значение типа <code>double</code>
<code>float readFloat()</code>	Читает значение типа <code>float</code>
<code>int readInt()</code>	Читает значение типа <code>int</code>
<code>long readLong()</code>	Читает значение типа <code>long</code>
<code>short readShort()</code>	Читает значение типа <code>short</code>

Ниже приведен пример программы, демонстрирующий применение классов `DataOutputStream` и `DataInputStream`. В этой программе данные разных типов сначала записываются в файл, а затем читаются из файла.

```
// Запись и чтение двоичных данных. Для компиляции этого кода
// требуется JDK 7 или более поздняя версия данного комплекта.

import java.io.*;

class RWData {
    public static void main(String args[])
    {
        int i = 10;
        double d = 1023.56;
        boolean b = true;

        // записать ряд значений
        try (DataOutputStream dataOut =
            new DataOutputStream(new FileOutputStream("testdata")))
        {
            // Запись двоичных данных в файл testdata.
            System.out.println("Writing " + i);
            dataOut.writeInt(i);

            System.out.println("Writing " + d);
            dataOut.writeDouble(d);

            System.out.println("Writing " + b);
            dataOut.writeBoolean(b);

            System.out.println("Writing " + 12.2 * 7.4);
            dataOut.writeDouble(12.2 * 7.4);
        }
        catch (IOException exc) {
            System.out.println("Write error.");
            return;
        }

        System.out.println();

        // а теперь прочитать записанные значения
```

```

try (DataInputStream dataIn =
    new DataInputStream(new FileInputStream("testdata")))
{
    // Чтение двоичных данных из файла testdata.
    i = dataIn.readInt();
    System.out.println("Reading " + i);

    d = dataIn.readDouble();
    System.out.println("Reading " + d);

    b = dataIn.readBoolean();
    System.out.println("Reading " + b);

    d = dataIn.readDouble();
    System.out.println("Reading " + d);
}
catch (IOException exc) {
    System.out.println("Read error.");
}
}

```

Выполнение этой программы дает следующий результат:

```

Writing 10
Writing 1023.56
Writing true
Writing 90.28

Reading 10
Reading 1023.56
Reading true
Reading 90.28

```

Пример для опробования 10.1.

Утилита сравнения файлов

Исходный файл: `CompFiles.java`

В этом проекте предстоит создать простую, но очень полезную утилиту для сравнения содержимого файлов. В ходе выполнения этой сервисной программы сначала открываются два сравниваемых файла, а затем данные читаются из них и сравниваются по соответствующему количеству байтов. Если на какой-то стадии операция сравнения дает отрицательный результат, это означает, что содержимое обоих файлов не одинаково. Если же конец обоих файлов достигается одновременно, это означает, что они содержат одинаковые данные.

Последовательность действий

1. Создайте файл `CompFiles.java`.
2. Введите в файл `CompFiles.java` приведенный ниже исходный код.

/*

Пример для опробования 10.1.

Сравнение двух файлов.

Для того чтобы воспользоваться этой программой, укажите имена сравниваемых файлов в командной строке, например:

```
java CompFile FIRST.TXT SECOND.TXT
```

Для компиляции этого кода требуется JDK 7 или более поздняя версия данного комплекта.

```
*/
```

```
import java.io.*;
```

```
class CompFiles {
    public static void main(String args[])
    {
        int i=0, j=0;

        // Прежде всего следует убедиться, что файлы были указаны.
        if(args.length !=2 ) {
            System.out.println("Usage: CompFiles f1 f2");
            return;
        }

        // сравнить файлы
        try (FileInputStream f1 = new FileInputStream(args[0]);
            FileInputStream f2 = new FileInputStream(args[1]))
        {
            // проверить содержимое каждого файла
            do {
                i = f1.read();
                j = f2.read();
                if(i != j) break;
            } while(i != -1 && j != -1);

            if(i != j)
                System.out.println("Files differ.");
            else
                System.out.println("Files are the same.");
        } catch(IOException exc) {
            System.out.println("I/O Error: " + exc);
        }
    }
}
```

3. Для опробования программы скопируйте сначала файл `CompFiles.java` во временный файл `temp`, а затем введите в командной строке следующее:

```
java CompFiles CompFiles.java temp
```

4. Программа сообщит, что файлы одинаковы. Далее сравните файл `CompFiles.java` с рассмотренным ранее файлом `CopyFile.java`, введя в командной строке следующее:

```
java CompFiles CompFiles.java CopyFile.java
```

5. Эти файлы содержат разные данные, о чем и сообщит программа `CompFiles`.
6. Попробуйте самостоятельно внедрить в программу `CompFiles` ряд дополнительных возможностей. В частности, введите в нее возможность выполнять сравнение

без учета регистра символов. Программу `CompFiles` можно также доработать таким образом, чтобы она выводила место, где обнаружено первое отличие сравниваемых файлов.

Файлы с произвольным доступом

До сих пор нам приходилось иметь дело с *последовательными файлами*, содержимое которых вводилось и выводилось побайтно, т.е. строго по порядку. Но в Java предоставляется также возможность обращаться к хранящимся в файле данным в произвольном порядке. Для этой цели предусмотрен класс `RandomAccessFile`, инкапсулирующий файл с произвольным доступом. Класс `RandomAccessFile` не является производным от класса `InputStream` или `OutputStream`. Вместо этого он реализует интерфейсы `DataInput` и `DataOutput`, в которых объявлены основные методы ввода-вывода. В нем поддерживаются также запросы позиционирования, т.е. возможность задавать положение *указателя файла* произвольным образом. Ниже приведен конструктор класса `RandomAccessFile`, которым мы будем пользоваться далее.

```
RandomAccessFile(String имя_файла, String доступ)
    throws FileNotFoundException
```

Здесь конкретный файл указывается с помощью параметра *имя_файла*, а параметр *доступ* определяет, какой именно тип доступа будет использоваться для обращения к файлу. Если параметр *доступ* принимает значение `"r"`, то данные могут читаться из файла, но не записываться в него. Если же указан тип доступа `"rw"`, то файл открывается как для чтения, так и для записи.

Метод `seek()`, общая форма объявления которого приведена ниже, служит для установки текущего положения указателя файла.

```
void seek(long newPos) throws IOException
```

Здесь параметр *newPos* определяет новое положение указателя файла в байтах относительно начала файла. Операция чтения или записи, следующая после вызова метода `seek()`, будет выполняться относительно нового положения указателя файла.

В классе `RandomAccessFile` определены методы `read()` и `write()`. Этот класс также реализует интерфейсы `DataInput` и `DataOutput`, т.е. в нем доступны методы чтения и записи простых типов, например `readInt()` и `writeDouble()`.

Ниже приведен пример программы, демонстрирующий ввод-вывод с произвольным доступом. В этой программе шесть значений типа `double` сначала записываются в файл, а затем читаются из него, причем порядок чтения их отличается от порядка записи.

```
// Демонстрация произвольного доступа к файлам.
```

```
// Для компиляции этого кода требуется JDK 7
// или более поздняя версия данного комплекта.
```

```
import java.io.*;
```

```
class RandomAccessDemo {
    public static void main(String args[])
    {
        double data[] = { 19.4, 10.1, 123.54, 33.0, 87.9, 74.25 };
        double d;
```

```

// открыть и использовать файл с произвольным доступом
// Файл с произвольным доступом открывается для записи и чтения.
try (RandomAccessFile raf = new RandomAccessFile("random.dat", "rw"))
{
    // записать значения в Файл
    for(int i=0; i < data.length; i++) {
        raf.writeDouble(data[i]);
    }

    // а теперь прочитать отдельные значения из файла
    // Для установки указателя файла служит метод seek().
    raf.seek(0); // найти первое значение типа double
    d = raf.readDouble();
    System.out.println("First value is " + d);

    raf.seek(8); // найти второе значение типа double
    d = raf.readDouble();
    System.out.println("Second value is " + d);

    raf.seek(8 * 3); // найти четвертое значение типа double
    d = raf.readDouble();
    System.out.println("Fourth value is " + d);

    System.out.println();

    // а теперь прочитать значения через одно
    System.out.println("Here is every other value: ");
    for(int i=0; i < data.length; i+=2) {
        raf.seek(8 * i); // найти i-е значение типа double
        d = raf.readDouble();
        System.out.print(d + " ");
    }
}
catch(IOException exc) {
    System.out.println("I/O Error: " + exc);
}
}

```

Результат выполнения данной программы выглядит следующим образом:

```

First value is 19.4
Second value is 10.1
Fourth value is 33.0

```

```

Here is every other value:
19.4 123.54 87.9

```

Обратите внимание на расположение каждого числового значения. Ведь значение типа `double` занимает 8 байтов, и поэтому каждое последующее число начинается на 8-байтовой границе предыдущего числа. Иными словами, первое числовое значение начинается на позиции нулевого байта, второе — на позиции 8-го байта, третье — на позиции 16-го байта и т.д. Поэтому для чтения четвертого числового значения нужно установить указатель файла на позиции 24-го байта при вызове метода `seek()`.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. В документации на JDK упоминается о классе `Console`. Можно ли воспользоваться этим классом для обмена данными с консолью?

ОТВЕТ. Да, можно. Класс `Console` был внедрен в версии Java 6 и служит для ввода-вывода данных на консоль. Этот класс создан в основном для удобства работы с консолью, поскольку большая часть его функциональных возможностей доступна в стандартных потоках ввода-вывода `System.in` и `System.out`. Но пренебрегать классом `Console` все же не следует, ведь с его помощью можно упростить некоторые типы операций с консолью, в том числе чтение символьных строк, вводимых с клавиатуры.

Класс `Console` не предоставляет конструкторы. Для получения объекта данного типа следует вызывать статический метод `System.console()`, который также был внедрен в версии Java 6. Ниже приведена общая форма объявления этого метода.

```
static Console console()
```

Если консоль доступна, этот метод возвращает ссылку на соответствующий объект. В противном случае возвращается пустое значение `null`. Консоль доступна не всегда: обращение к ней запрещено, если программа выполняется в фоновом режиме.

В классе `Console` определен ряд методов, поддерживающих ввод-вывод, например `readLine()` и `printf()`. В нем также содержится метод `readPassword()`, который служит для получения пароля. При вводе пароля с клавиатуры его символы отображаются на экране с помощью замещающих знаков, не раскрывая пароль. Средствами класса `Console` можно также получить ссылки на объекты типа `Reader` и `Writer`, связанные с консолью. Одним словом, класс `Console` может оказаться очень полезным при создании некоторых видов приложений.

Применение символьных потоков в Java

Как следует из предыдущих разделов этой главы, байтовые потоки в Java довольно эффективны и удобны в употреблении. Но что касается ввода-вывода символов, то байтовые потоки далеки от идеала. Поэтому для этих целей в Java определены классы символьных потоков. На вершине иерархии классов, поддерживающих символьные потоки, находятся абстрактные классы `Reader` и `Writer`. Методы класса `Reader` приведены в табл. 10.7, а методы класса `Writer` — в табл. 10.8. В большинстве этих методов может быть сгенерировано исключение `IOException`. Методы, определенные в указанных абстрактных классах `Reader` и `Writer`, доступны во всех их подклассах. Таким образом, они образуют минимальный набор функций ввода-вывода, необходимых для всех символьных потоков.

Таблица 10.7. Методы, определенные в классе `Reader`

Метод	Описание
<code>abstract void close()</code>	Закрывает поток ввода данных. При последующей попытке чтения генерируется исключение <code>IOException</code>

Метод	Описание
<code>void mark (int numChars)</code>	Ставит отметку на текущей позиции в потоке. Отметка доступна до тех пор, пока не будет прочитано количество символов, определяемое параметром <i>numChars</i>
<code>boolean markSupported()</code>	Возвращает логическое значение <code>true</code> , если поток поддерживает методы <code>mark()</code> и <code>reset()</code>
<code>int read()</code>	Возвращает целочисленное представление очередного символа из потока ввода. Если достигнут конец потока, возвращается значение <code>-1</code>
<code>int read(char buffer[])</code>	Предпринимает попытку прочитать количество байтов, определяемое выражением <i>buffer.length</i> , в массив <i>buffer</i> и возвращает фактическое количество успешно прочитанных символов. Если достигнут конец потока, возвращается значение <code>-1</code>
<code>abstract int read(char buffer[], int offset, int numChars)</code>	Предпринимает попытку прочитать количество символов, определяемое параметром <i>numChars</i> , в массив <i>buffer</i> , начиная с элемента <i>buffer[offset]</i> . Если достигнут конец потока, возвращается значение <code>-1</code>
<code>int read(CharBuffer buffer)</code>	Предпринимает попытку заполнить буфер, определяемый параметром <i>buffer</i> , символами, прочитанными из входного потока. Если достигнут конец потока, возвращается значение <code>-1</code> . <code>CharBuffer</code> — это класс, представляющий последовательность символов, например строку
<code>boolean ready()</code>	Возвращает логическое значение <code>true</code> , если следующий запрос на получение символа может быть выполнен немедленно. В противном случае возвращается логическое значение <code>false</code>
<code>void reset()</code>	Устанавливает указатель ввода на помеченной ранее позиции
<code>long skip(long numChars)</code>	Пропускает количество символов, определяемое параметром <i>numChars</i> , в поток ввода. Возвращает фактическое количество пропущенных символов

Таблица 10.8. Методы, определенные в классе `Writer`

Метод	Описание
<code>Writer append(char ch)</code>	Записывает символ <i>ch</i> в конец текущего потока. Возвращает ссылку на поток
<code>Writer append(CharSequence chars)</code>	Записывает символы <i>chars</i> в конец текущего потока. Возвращает ссылку на поток. <code>CharSequence</code> — это интерфейс, в котором описаны только операции чтения последовательности символов
<code>Writer append(CharSequence chars, int begin, int end)</code>	Записывает символы <i>chars</i> в конец текущего потока, начиная с позиции, определяемой параметром <i>begin</i> , и кончая позицией, определяемой параметром <i>end</i> . Возвращает ссылку на поток. <code>CharSequence</code> — это интерфейс, в котором описаны только операции чтения последовательности символов
<code>abstract void close()</code>	Закрывает поток вывода. При последующей попытке записи в поток генерируется исключение <code>IOException</code>

Окончание табл. 10.8

Метод	Описание
<code>abstract void flush()</code>	Выводит текущее содержимое буфера на устройство. В результате выполнения данной операции буфер очищается
<code>void write(int ch)</code>	Записывает в вызывающий поток вывода один символ. Параметр <i>ch</i> относится к типу <code>int</code> , что позволяет вызывать данный метод в выражениях, не приводя результат их вычисления к типу <code>char</code>
<code>void write(char buffer[])</code>	Записывает в вызывающий поток вывода массив символов <i>buffer</i>
<code>abstract void write(char buffer[], int offset, int numChars)</code>	Записывает в вызывающий поток вывода количество символов, определяемое параметром <i>numChars</i> , из массива <i>buffer</i> , начиная с элемента <i>buffer[offset]</i>
<code>void write(String str)</code>	Записывает в вызывающий поток вывода символьную строку <i>str</i>
<code>void write(String str, int offset, int numChars)</code>	Записывает в вызывающий поток вывода часть <i>numChars</i> символов из строки <i>str</i> , начиная с позиции, обозначаемой параметром <i>offset</i>

Консольный ввод из символьных потоков

Если программа подлежит локализации, то при организации ввода с консоли символьным потокам следует отдать предпочтение перед байтовыми. А поскольку `System.in` — это байтовый поток, то для него придется построить оболочку в виде класса, производного от класса `Reader`. Наиболее подходящим для ввода с консоли является класс `BufferedReader`, поддерживающий буферизованный поток ввода. Но объект типа `BufferedReader` нельзя построить непосредственно из потока стандартного ввода `System.in`. Сначала нужно преобразовать байтовый поток в символьный. И для этой цели служит класс `InputStreamReader`, преобразующий байты в символы. Для того чтобы получить объект типа `InputStreamReader`, связанный с потоком стандартного ввода `System.in`, нужно воспользоваться следующим конструктором:

```
InputStreamReader(InputStream inputStream)
```

Поток ввода `System.in` является экземпляром класса `InputStream`, и поэтому его можно указать в качестве параметра `inputStream` данного конструктора.

Затем на основании объекта типа `InputStreamReader` можно создать объект типа `BufferedReader`, используя следующий конструктор:

```
BufferedReader(Reader inputReader)
```

где `inputReader` — это поток, который связывается с создаваемым экземпляром класса `BufferedReader`. Объединяя обращения к указанным выше конструкторам в одну операцию, мы получаем приведенную ниже строку кода. В ней создается объект типа `BufferedReader`, связанный с клавиатурой.

```
BufferedReader br = new BufferedReader(new
    InputStreamReader(System.in));
```

После выполнения этого оператора присваивания переменная `br` будет содержать ссылку на символьный поток, связанный с консолью через поток ввода `System.in`.

Чтение символов

Прочитать символы из потока ввода `System.in` можно с помощью метода `read()`, определенного в классе `BufferedReader`. Чтение символов мало чем отличается от чтения данных из байтовых потоков. Ниже приведены общие формы объявления трех вариантов метода `read()`, предусмотренных в классе `BufferedReader`.

```
int read() throws IOException
int read(char data[]) throws IOException
int read(char data[], int start, int max) throws IOException
```

В первом варианте метод `read()` читает один символ в уникоде. По достижении конца потока этот метод возвращает значение `-1`. Во втором варианте метод `read()` читает данные из потока ввода и помещает их в массив. Чтение оканчивается по достижении конца потока, по заполнении массива `data` символами или при возникновении ошибки. В этом случае метод возвращает число прочитанных символов, а если достигнут конец потока, — значение `-1`. В третьем варианте метод `read()` помещает прочитанные символы в массив `data`, начиная с элемента, определяемого индексом `start`. Максимальное число символов, которые могут быть записаны в массив, определяется параметром `max`. В данном случае метод возвращает число прочитанных символов или значение `-1`, если достигнут конец потока. При возникновении ошибки в каждом из перечисленных выше вариантов метода `read()` генерируется исключение `IOException`. При чтении данных из потока ввода `System.in` конец потока устанавливается нажатием клавиши `<Enter>`.

Ниже приведен пример программы, демонстрирующий применение метода `read()` для чтения символов с консоли. Символы читаются до тех пор, пока пользователь не введет точку. Следует иметь в виду, что исключения, которые могут быть сгенерированы при выполнении данной программы, обрабатываются за пределами метода `main()`. Как пояснялось выше, подобный подход характерен для обработки ошибок при чтении данных с консоли. По желанию вы можете употребить другой механизм обработки ошибок.

```
// Применение класса BufferedReader для чтения символов с консоли.
import java.io.*;

class ReadChars {
    public static void main(String args[])
        throws IOException
    {
        char c;
        // Создание объекта типа BufferedReader, связанного
        // с потоком стандартного ввода System.in.
        BufferedReader br = new
            BufferedReader(new
                InputStreamReader(System.in));

        System.out.println("Enter characters, period to quit.");

        // читать символы
        do {
            c = (char) br.read();
            System.out.println(c);
        } while(c != '.');
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
Enter characters, period to quit.
One Two.
O
n
e
T
w
o
.
```

Чтение символьных строк

Для ввода символьной строки с клавиатуры следует воспользоваться методом `readLine()` из класса `BufferedReader`. Ниже приведена общая форма объявления этого метода.

```
String readLine() throws IOException
```

Этот метод возвращает объект типа `String`, содержащий прочитанные символы. При попытке прочитать символьную строку по окончании потока метод возвращает пустое значение `null`.

Ниже приведен пример программы, демонстрирующий применение класса `BufferedReader` и метода `readLine()`. В этой программе текстовые строки читаются и отображаются до тех пор, пока не будет введено слово **"stop"**.

```
// Чтение символьных строк с консоли средствами класса BufferedReader.
import java.io.*;

class ReadLines {
    public static void main(String args[])
        throws IOException
    {
        // создать объект типа BufferedReader, связанный с потоком System.in
        BufferedReader br = new BufferedReader(new
            InputStreamReader(System.in));

        String str;

        System.out.println("Enter lines of text.");
        System.out.println("Enter 'stop' to quit.");
        do {
            // использовать метод readLine() из класса BufferedReader
            // для чтения текстовой строки
            str = br.readLine();
            System.out.println(str);
        } while(!str.equals("stop"));
    }
}
```

Консольный вывод в символьные потоки

Несмотря на то что поток стандартного вывода `System.out` вполне может использоваться для вывода на консоль, такой подход скорее пригоден для целей отладки или

при создании очень простых программ, подобных тем, которые приводятся в качестве примеров в этой книге. Для реальных прикладных программ на Java вывод на консоль обычно организуется через поток `PrintWriter`, относящийся к одному из классов, представляющих символьные потоки. Как упоминалось ранее, применение символьных потоков упрощает локализацию прикладных программ.

В классе `PrintWriter` определен ряд конструкторов. Далее будет использоваться следующий конструктор:

```
PrintWriter(OutputStream outputStream, boolean flushOnNewline)
```

где в качестве первого параметра `outputStream` конструктору передается объект типа `OutputStream`, а второй параметр `flushOnNewline` указывает, должен ли производиться вывод данных из буфера в поток вывода при каждом вызове метода `println()`. Если параметр `flushOnNewline` принимает логическое значение `true`, данные выводятся из буфера автоматически.

В классе `PrintWriter` поддерживаются методы `print()` и `println()` для всех типов, включая `Object`. Следовательно, методы `print()` и `println()` можно использовать точно так же, как и вместе с потоком вывода `System.out`. Если значение аргумента не относится к простому типу, то методы из класса `PrintWriter` вызывают метод `toString()` для объекта, указываемого в качестве параметра, а затем выводят результат.

Для вывода данных на консоль через поток типа `PrintWriter` следует указать `System.out` в качестве потока вывода и обеспечить вывод данных из буфера после каждого вызова метода `println()`. Например, при выполнении следующей строки кода создается объект типа `PrintWriter`, связанный с консолью:

```
PrintWriter pw = new PrintWriter(System.out, true);
```

Ниже приведен пример прикладной программы, демонстрирующий применение класса `PrintWriter` для организации вывода на консоль.

```
// Применение класса PrintWriter.
import java.io.*;

public class PrintWriterDemo {
    public static void main(String args[]) {
        // Создание объекта типа PrintWriter, связанного
        // с потоком стандартного вывода System.out.
        PrintWriter pw = new PrintWriter(System.out, true);
        int i = 10;
        double d = 123.65;

        pw.println("Using a PrintWriter.");
        pw.println(i);
        pw.println(d);

        pw.println(i + " + " + d + " is " + (i+d));
    }
}
```

Выполнение этой программы дает следующий результат:

```
Using a PrintWriter.
10
123.65
10 + 123.65 is 133.65
```

Несмотря на все удобство символьных потоков, не следует забывать, что для изучения языка Java или отладки программ можно вполне пользоваться и потоком вывода `System.out`. Но если в программе применяется поток `PrintWriter`, то ее проще локализовать. Для кратких примеров программ, представленных в этой книге, применение потока `PrintWriter` не имеет существенных преимуществ перед потоком `System.out`, поэтому в и последующих примерах для вывода на консоль будет использоваться поток `System.out`.

Ввод-вывод в файлы через символьные потоки

На практике чаще всего приходится обращаться с файлами, имеющими байтовую организацию, тем не менее, для этой цели можно пользоваться символьными потоками. Преимущество символьных потоков заключается в том, что они оперируют непосредственно символами в уникоде. Так, если требуется сохранить текст в уникоде, для этой цели лучше всего воспользоваться символьными потоками. Как правило, для файлового ввода-вывода символов служат классы `FileReader` и `FileWriter`.

Применение класса `FileWriter`

Класс `FileWriter` представляет поток, через который можно осуществлять запись данных в файл. Ниже приведены общие формы объявления двух наиболее часто употребляемых конструкторов данного класса.

```
FileWriter(String имя_файла) throws IOException
FileWriter(String имя_файла, boolean append) throws IOException
```

Здесь `имя_файла` обозначает полный путь к файлу. Если параметр `append` принимает логическое значение `true`, данные записываются в конец файла, а иначе они перезаписывают прежние данные на том же месте в файле. При возникновении ошибки в каждом из указанных выше конструкторов генерируется исключение `IOException`. Класс `FileWriter` является производным от классов `OutputStreamWriter` и `Writer`. Следовательно, в нем доступны методы, объявленные в его суперклассах.

Ниже приведен краткий пример программы, демонстрирующий ввод текстовых строк с клавиатуры и последующий их вывод в файл `test.txt`. Набираемый текст читается до тех пор, пока пользователь не введет слово **"stop"**. Для вывода текстовых строк в файл используется класс `FileWriter`.

```
// Простой пример утилиты ввода с клавиатуры и вывода данных
// на диск, демонстрирующий применение класса FileWriter.
```

```
// Для компиляции этого кода требуется JDK 7
// или более поздняя версия данного комплекта.
```

```
import java.io.*;

class KtoD {
    public static void main(String args[])
    {
        String str;
        BufferedReader br =
            new BufferedReader(
                new InputStreamReader(System.in));
        System.out.println("Enter text ('stop' to quit).");
```

```
// Создание потока вывода типа FileWriter.
try (FileWriter fw = new FileWriter("test.txt"))
{
    do {
        System.out.print(": ");
        str = br.readLine();
        if(str.compareTo("stop") == 0) break;
        str = str + "\r\n"; // add newline
        // Запись текстовых строк в файл.
        fw.write(str);
    } while(str.compareTo("stop") != 0);
} catch(IOException exc) {
    System.out.println("I/O Error: " + exc);
}
}
```

Применение класса FileReader

В классе `FileReader` создается объект типа `Reader`, который можно использовать для чтения содержимого файла. Чаще всего употребляется такой конструктор этого класса:

`FileReader(String имя_файла) throws FileNotFoundException`

где `имя_файла` обозначает полный путь к файлу. Если указанный файл не существует, генерируется исключение `FileNotFoundException`. Класс `FileReader` является производным от классов `InputStreamReader` и `Reader`. Следовательно, в нем доступны методы, объявленные в его суперклассах.

Приведенный ниже пример демонстрирует простую утилиту для отображения на экране содержимого текстового файла `test.txt`. Она является своего рода дополнением к утилите, рассмотренной в предыдущем разделе.

```
// Простая утилита ввода с диска и вывода на экран,
// демонстрирующая применение класса FileReader.

// Для компиляции этого кода требуется JDK 7
// или более поздняя версия данного комплекта.

import java.io.*;

class DtoS {
    public static void main(String args[]) {
        String s;
        // Создание в классе BufferedReader оболочки с целью заключить
        // в нее класс FileReader и организовать чтение данных из файла.
        try (BufferedReader br = new BufferedReader(new FileReader("test.txt")))
        {
            while((s = br.readLine()) != null) {
                System.out.println(s);
            }
        } catch(IOException exc) {
            System.out.println("I/O Error: " + exc);
        }
    }
}
```

Обратите внимание на то, что для потока типа `FileReader` создана оболочка в классе `BufferedReader`. Благодаря этому появляется возможность обращаться к методу `readLine()`. Кроме того, закрытие потока типа `BufferedReader`, на который в данном примере ссылается переменная `br`, автоматически приводит к закрытию файла.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Говорят, что имеется отдельная система NIO с классами для поддержки ввода-вывода. Что она собой представляет?

ОТВЕТ. Система NIO, что означает *новый ввод-вывод*, была внедрена в версии JDK 1.4. В ней поддерживается канальный подход к организации операций ввода-вывода. Классы новой системы ввода-вывода относятся к пакету `java.nio` и его подчиненным пакетам, включая `java.nio.channels` и `java.nio.charset`.

Система NIO опирается на два основных понятия: *буферы* и *каналы*. Буфер содержит данные, а канал представляет собой соединение, устанавливаемое с устройством ввода-вывода, например с файлом на диске или сетевым сокетом. Как правило, для применения новой системы ввода-вывода нужно получить канал доступа к устройству и буфер для хранения данных. После этого можно выполнять необходимые операции с буфером, в частности вводить или выводить данные.

Кроме буфера и канала, в NIO используются также понятия набора символов и селектора. *Набор символов* определяет способ преобразования байтов в символы. Для представления последовательности символов в виде набора байтов используется *шифратор*. Обратное преобразование осуществляет *дешифратор*. А *селектор* поддерживает неблокирующий мультиплексированный ввод-вывод с ключом шифрования. Иными словами, селекторы позволяют выполнять обмен данными по нескольким каналам. Селекторы находят наибольшее применение в каналах, поддерживаемых сетевыми сокетами.

Начиная с версии JDK 7 новая система ввода-вывода была значительно усовершенствована, и поэтому нередко ее называют *NIO.2*. К числу ее усовершенствований относятся три новых пакета (`java.nio.file`, `java.nio.file.attribute` и `java.nio.file.spi`), ряд новых классов, интерфейсов и методов, а также непосредственная поддержка потокового ввода-вывода. Все эти дополнения в значительной степени расширили область применения NIO, и особенно это касается обработки файлов.

Однако новая система ввода-вывода не призвана заменить классы ввода-вывода, существующие в пакете `java.io`. Классы NIO лишь дополняют стандартную систему ввода-вывода, предлагая альтернативный подход, вполне уместный в ряде случаев.

Применение оболочек типов для преобразования символьных строк в числа

Прежде чем завершить обсуждение средств ввода-вывода, необходимо рассмотреть еще один способ, помогающий читать числовые строки. Как известно, метод `println()` предоставляет удобные средства для вывода на консоль различных типов данных, в том числе целых чисел и чисел с плавающей точкой. Он автоматически преобразует чис-

ловые значения в удобную для чтения форму. Но в Java отсутствует метод, который читал бы числовые строки и преобразовывал бы их во внутреннюю двоичную форму. Например, не существует варианта метода `read()`, который читал бы числовую строку "100" и автоматически преобразовывал ее в целое число, пригодное для хранения в переменной типа `int`. Но для этой цели в Java имеются другие средства. И проще всего подобное преобразование осуществляется с помощью так называемых *оболочек типов*.

Оболочки типов в Java представляют собой классы, которые инкапсулируют простые типы. Оболочки типов необходимы, поскольку простые типы не являются объектами, что ограничивает их применение. Так, простой тип нельзя передать методу по ссылке. Для того чтобы исключить ненужные ограничения, в Java были предусмотрены классы, соответствующие каждому из простых типов.

Оболочками типов являются классы `Double`, `Float`, `Long`, `Integer`, `Short`, `Byte`, `Character` и `Boolean`. Эти классы предоставляют обширный ряд методов, позволяющих полностью интегрировать простые типы в иерархию объектов Java. Кроме того, в классах-оболочках числовых типов содержатся методы, предназначенные для преобразования числовых строк в соответствующие двоичные эквиваленты. Эти методы приведены ниже. Каждый из них возвращает двоичное значение, соответствующее числовой строке.

Оболочка	Метод преобразования типа
----------	------------------------------

<code>Double</code>	<code>static double parseDouble(String str) throws NumberFormatException</code>
<code>Float</code>	<code>static float parseFloat(String str) throws NumberFormatException</code>
<code>Long</code>	<code>static long parseLong(String str) throws NumberFormatException</code>
<code>Integer</code>	<code>static int parseInt(String str) throws NumberFormatException</code>
<code>Short</code>	<code>static short parseShort(String str) throws NumberFormatException</code>
<code>Byte</code>	<code>static byte parseByte(String str) throws NumberFormatException</code>

Оболочки целочисленных типов также предоставляют дополнительный метод синтаксического анализа, позволяющий задавать основание системы счисления.

Методы синтаксического анализа позволяют без труда преобразовать во внутренний формат числовые значения, введенные в виде символьных строк с клавиатуры или из текстового файла. Ниже приведен пример программы, демонстрирующий применение для этих целей методов `parseInt()` и `parseDouble()`. В этой программе находится среднее арифметическое ряда чисел, введенных пользователем с клавиатуры. Сначала пользователю предлагается указать количество числовых значений для обработки, а затем программа вводит числа с клавиатуры, используя метод `readLine()`, а с помощью метода `parseInt()` преобразует символьную строку в целочисленное значение. Далее осуществляется ввод числовых значений и последующее их преобразование в тип `double` с помощью метода `parseDouble()`.

```
/* Эта программа находит среднее арифметическое для
   ряда чисел, введенных пользователем с клавиатуры. */
```

```
import java.io.*;
```

```
class AvgNums {
    public static void main(String args[])
        throws IOException
```

```

{
    // создать объект типа BufferedReader,
    // использующий поток ввода System.in
    BufferedReader br = new
        BufferedReader(new InputStreamReader(System.in));
    String str;
    int n;
    double sum = 0.0;
    double avg, t;

    System.out.print("How many numbers will you enter: ");
    str = br.readLine();
    try {
        // Преобразование символьной строки
        // в числовое значение типа int.
        n = Integer.parseInt(str);
    }
    catch(NumberFormatException exc) {
        System.out.println("Invalid format");
        n = 0;
    }

    System.out.println("Enter " + n + " values.");
    for(int i=0; i < n ; i++) {
        System.out.print(": ");
        str = br.readLine();
        try {
            // Преобразование символьной строки
            // в числовое значение типа double.
            t = Double.parseDouble(str);
        } catch(NumberFormatException exc) {
            System.out.println("Invalid format");
            t = 0.0;
        }
        sum += t;
    }
    avg = sum / n;
    System.out.println("Average is " + avg);
}
}

```

Выполнение этой программы может дать, например, следующий результат:

```

How many numbers will you enter: 5
Enter 5 values.
: 1.1
: 2.2
: 3.3
: 4.4
: 5.5
Average is 3.3

```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Могут ли оболочки простых типов выполнять другие функции, помимо описанных в этом разделе?

ОТВЕТ. Классы оболочек простых типов предоставляют ряд методов, помогающих интегрировать эти типы данных в иерархию объектов. Например, различные механизмы хранения данных, предусмотренные в библиотеке Java, включая отображения, списки и множества, взаимодействуют только с объектами. Поэтому для сохранения целочисленного значения в списке его следует сначала преобразовать в объект. Оболочки типов предоставляют также метод `compareTo()` для сравнения текущего значения с заданным, метод `equals()` для проверки равенства двух объектов, а также методы, возвращающие значение объекта в разных формах записи. К оболочкам типов мы еще вернемся в главе 12, когда речь пойдет об автоупаковке.

Пример для опробования 10.2.

Создание справочной системы, находящейся на диске

Исходный файл: `FileHelp.java`

В примере для опробования 4.1 был создан класс `Help`, позволяющий отображать сведения об операторах Java. Справочная информация хранилась в самом классе, а пользователь выбирал требуемые сведения из меню. И хотя такая справочная система выполняет свои функции, подход к ее разработке был выбран далеко не самый лучший. Так, если требуется добавить или изменить какие-нибудь сведения в подобной справочной системе, придется внести изменения в исходный код программы, которая ее реализует. Кроме того, выбирать пункт меню по его номеру не очень удобно, а если количество пунктов велико, то такой способ оказывается вообще непригодным. В этом проекте предстоит устранить недостатки, имеющиеся в справочной системе, разместив справочную информацию на диске.

В новом варианте справочная информация должна храниться в файле. Это будет обычный текстовый файл, который можно изменять, не затрагивая исходный код программы. Для того чтобы получить справку по конкретному вопросу, следует ввести название темы. Система будет искать соответствующий раздел в файле. Если поиск завершится успешно, справочная информация будет выведена на экран.

Последовательность действий

1. Создайте файл, в котором будет храниться справочная информация и который будет использоваться в справочной системе. Это должен быть обычный текстовый файл, организованный следующим образом:

```
#название_темы_1
Информация по теме
```

```
#название_темы_2
Информация по теме
```

```
#название_темы_N
Информация по теме
```

2. Название каждой темы располагается в отдельной строке и предваряется символом **#**. Наличие специального символа в строке (в данном случае — **#**) позволяет программе быстро найти начало раздела. Под названием темы может располагаться любая справочная информация. После окончания одного раздела и перед началом другого должна быть введена пустая строка. Кроме того, в конце строк не должно быть лишних пробелов.
3. Ниже приведен пример простого файла со справочной информацией, который можно использовать вместе с новой версией справочной системы. В нем хранятся сведения об операторах Java.

```
#if
if(condition) statement;
else statement;

#switch
switch(expression) {
    case constant:
        statement sequence
        break;
    // ...
}

#for
for(init; condition; iteration) statement;

#while
while(condition) statement;

#do
do {
    statement;
} while (condition);

#break
break; or break label;

#continue
continue; or continue label;
```

4. Присвойте этому файлу имя `helpfile.txt`.
5. Создайте файл `FileHelp.java`.
6. Начните создание новой версии класса `Help` со следующих строк кода:

```
class Help {
    String helpfile; // Имя файла со справочной информацией

    Help(String fname) {
        helpfile = fname;
    }
}
```

7. Имя файла со справочной информацией передается конструктору класса `Help` и запоминается в переменной экземпляра `helpfile`. А поскольку каждый экземпляр класса `Help` содержит отдельную копию переменной `helpfile`, то каждый из них

может взаимодействовать с отдельным файлом. Это дает возможность создавать отдельные наборы справочных файлов на разные темы.

8. Добавьте в класс `Help` метод `helpon()`, код которого приведен ниже. Этот метод извлекает справочную информацию по заданной теме.

```
// отобразить справочную информацию по заданной теме
boolean helpon(String what) {
    int ch;
    String topic, info;

    // открыть справочный файл
    try (BufferedReader helpRdr =
        new BufferedReader(new FileReader(helpfile)))
    {
        do {
            // читать символы до тех пор, пока не встретится знак #
            ch = helpRdr.read();

            // а теперь проверить, совпадают ли темы
            if(ch == '#') {
                topic = helpRdr.readLine();
                if(what.compareTo(topic) == 0) { // found topic
                    do {
                        info = helpRdr.readLine();
                        if(info != null) System.out.println(info);
                    } while((info != null) &&
                        (info.compareTo("") != 0));
                    return true;
                }
            }
        } while(ch != -1);
    }
    catch(IOException exc) {
        System.out.println("Error accessing help file.");
        return false;
    }
    return false; // тема не найдена
}
```

9. Прежде всего обратите внимание на то, что в методе `helpon()` обрабатываются все исключения, связанные с вводом-выводом, поэтому в заголовке метода не указано ключевое слово `throws`. Благодаря такому подходу упрощается разработка методов, в которых используется метод `helpon()`. В вызывающем методе достаточно обратиться к методу `helpon()`, не заключая его вызов в блок `try/catch`.
10. Для открытия файла со справочной информацией служит класс `FileReader`, оболочкой которого является класс `BufferedReader`. В справочном файле содержится текст, и поэтому справочную систему удобнее локализовать через символьные потоки ввода-вывода.
11. Метод `helpon()` действует следующим образом. Символьная строка, содержащая название темы, передается этому методу в качестве параметра. Метод открывает сначала файл со справочной информацией. Затем в файле производится поиск, т.е. проверяется совпадение содержимого переменной `what` и названия темы.

Напомним, что в файле заголовок темы предваряется символом #, поэтому метод сначала ищет данный символ. Если символ найден, производится сравнение следующего за ним названия темы с содержимым переменной `what`. Если сравниваемые строки совпадают, то отображается справочная информация по данной теме. И если заголовок темы найден, то метод `helpOn()` возвращает логическое значение `true`, иначе — логическое значение `false`.

12. В классе `Help` содержится также метод `getSelection()`, который предлагает задать тему и возвращает строку, введенную пользователем.

```
// получить тему
String getSelection() {
    String topic = "";

    BufferedReader br = new BufferedReader(
        new InputStreamReader(System.in));

    System.out.print("Enter topic: ");
    try {
        topic = br.readLine();
    }
    catch(IOException exc) {
        System.out.println("Error reading console.");
    }
    return topic;
}
```

13. В теле этого метода сначала создается объект типа `BufferedReader`, который связывается с потоком вывода `System.in`. Затем в нем запрашивается название темы, которое принимается и далее возвращается вызывающей части программы.

14. Ниже приведен весь исходный код программы, реализующей справочную систему, находящуюся на диске.

```
/*
    Пример для опробования 10.2.

    Справочная система, находящаяся на диске.

    Для компиляции этой программы требуется JDK 7
    или более поздняя версия данного комплекта.
*/

import java.io.*;

/* В классе Help открывается файл со справочной информацией,
   производится поиск названия темы, а затем отображается
   справочная информация по этой теме.
   Обратите внимание на то, что в этом классе поддерживаются
   все исключения, освобождая от этой обязанности вызывающий код. */
class Help {
    String helpfile; // Имя файла со справочной информацией

    Help(String fname) {
        helpfile = fname;
```

```

    }

    // отобразить справочную информацию по заданной теме
    boolean helpOn(String what) {
        int ch;
        String topic, info;

        // открыть справочный файл
        try (BufferedReader helpRdr =
            new BufferedReader(new FileReader(helpfile)))
        {
            do {
                // читать символы до тех пор, пока не встретится знак #
                ch = helpRdr.read();

                // а теперь проверить, совпадают ли темы
                if(ch == '#') {
                    topic = helpRdr.readLine();
                    if(what.compareTo(topic) == 0) { // тема найдена
                        do {
                            info = helpRdr.readLine();
                            if(info != null) System.out.println(info);
                        } while((info != null) &&
                            (info.compareTo("") != 0));
                        return true;
                    }
                }
            } while(ch != -1);
        }
        catch(IOException exc) {
            System.out.println("Error accessing help file.");
            return false;
        }
        return false; // тема не найдена
    }

    // получить тему
    String getSelection() {
        String topic = "";

        BufferedReader br = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.print("Enter topic: ");
        try {
            topic = br.readLine();
        }
        catch(IOException exc) {
            System.out.println("Error reading console.");
        }
        return topic;
    }
}

```

```
// продемонстрировать справочную систему, находящуюся на диске
class FileHelp {
    public static void main(String args[]) {
        Help hlpobj = new Help("helpfile.txt");
        String topic;

        System.out.println("Try the help system. " +
            "Enter 'stop' to end.");

        do {
            topic = hlpobj.getSelection();

            if(!hlpobj.helpon(topic))
                System.out.println("Topic not found.\n");

            } while(topic.compareTo("stop") != 0);
        }
    }
}
```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Имеется ли, помимо методов *синтаксического анализа*, определяемых в оболочках простых типов, другой простой способ преобразования числовой строки, вводимой с клавиатуры, в эквивалентную ей двоичную форму?

ОТВЕТ. Да, имеется. Другой способ преобразования числовой строки в ее внутреннее представление в двоичной форме состоит в использовании одного из методов, определенных в классе `Scanner` из пакета `java.util`. Этот класс внедрен в версии JDK 5 и читает данные, вводимые в удобном для чтения виде, преобразуя их в двоичную форму. Средствами класса `Scanner` можно организовать чтение данных, вводимых из самых разных источников, в том числе с консоли и из файлов. Следовательно, его можно использовать для чтения числовой строки, введенной с клавиатуры, присвоив полученное значение переменной. И хотя в классе `Scanner` содержится слишком много средств, чтобы описать их подробно, ниже демонстрируются основные примеры его применения.

Для организации ввода с клавиатуры средствами класса `Scanner` необходимо сначала создать объект этого класса, связанный с потоком ввода с консоли. Для этой цели служит следующий конструктор:

```
Scanner(InputStream from)
```

Этот конструктор создает объект типа `Scanner`, использующий поток ввода, определяемый параметром `from`, в качестве источника ввода данных. С помощью этого конструктора можно создать объект типа `Scanner`, связанный с потоком ввода с консоли, как показано ниже.

```
Scanner conin = new Scanner(System.in);
```

Это оказывается возможным благодаря тому, что поток `System.in` является объектом типа `InputStream`. После выполнения этой строки кода переменную `conin` ссылки на объект типа `Scanner` можно использовать для чтения данных, вводимых с клавиатуры.

ОБРАЩЕНИЕ К ЗНАТОКУ

Как только будет создан объект типа `Scanner`, им нетрудно воспользоваться для чтения числовой строки, вводимой с клавиатуры. Ниже приведен общий порядок выполняемых для этого действий.

1. Определить, имеются ли вводимые данные конкретного типа, вызвав один из методов `hasNextX` класса `Scanner`, где `X` — нужный тип вводимых данных.
2. Если вводимые данные имеются, прочитать их, вызвав один из методов `nextX` класса `Scanner`.

Как следует из приведенного выше порядка действий, в классе `Scanner` определены две группы методов, предназначенных для чтения вводимых данных. К первой из них относятся методы `hasNextX`, в том числе `hasNextInt()` и `hasNextDouble()`. Каждый из методов `hasNextX` возвращает логическое значение `true`, если очередной элемент данных, имеющийся в потоке ввода, относится к нужному типу данных, а иначе — логическое значение `false`. Так, логическое значение `true` возвращается при вызове метода `hasNextInt()` лишь в том случае, если очередной элемент данных в потоке ввода является целочисленным значением, представленным в удобном для чтения виде. Если данные нужного типа присутствуют в потоке ввода, их можно прочитать, вызвав один из методов класса `Scanner`, относящихся к группе `next`, например метод `nextInt()` или `nextDouble()`. Эти методы преобразуют данные соответствующего типа из удобной для чтения формы во внутреннее их представление в двоичном виде, возвращая полученный результат. Так, для чтения целочисленного значения, введенного с клавиатуры, следует вызвать метод `nextInt()`.

В приведенном ниже фрагменте кода показано, каким образом организуется чтение целочисленного значения с клавиатуры.

```
Scanner conin = new Scanner(System.in);
int i;
```

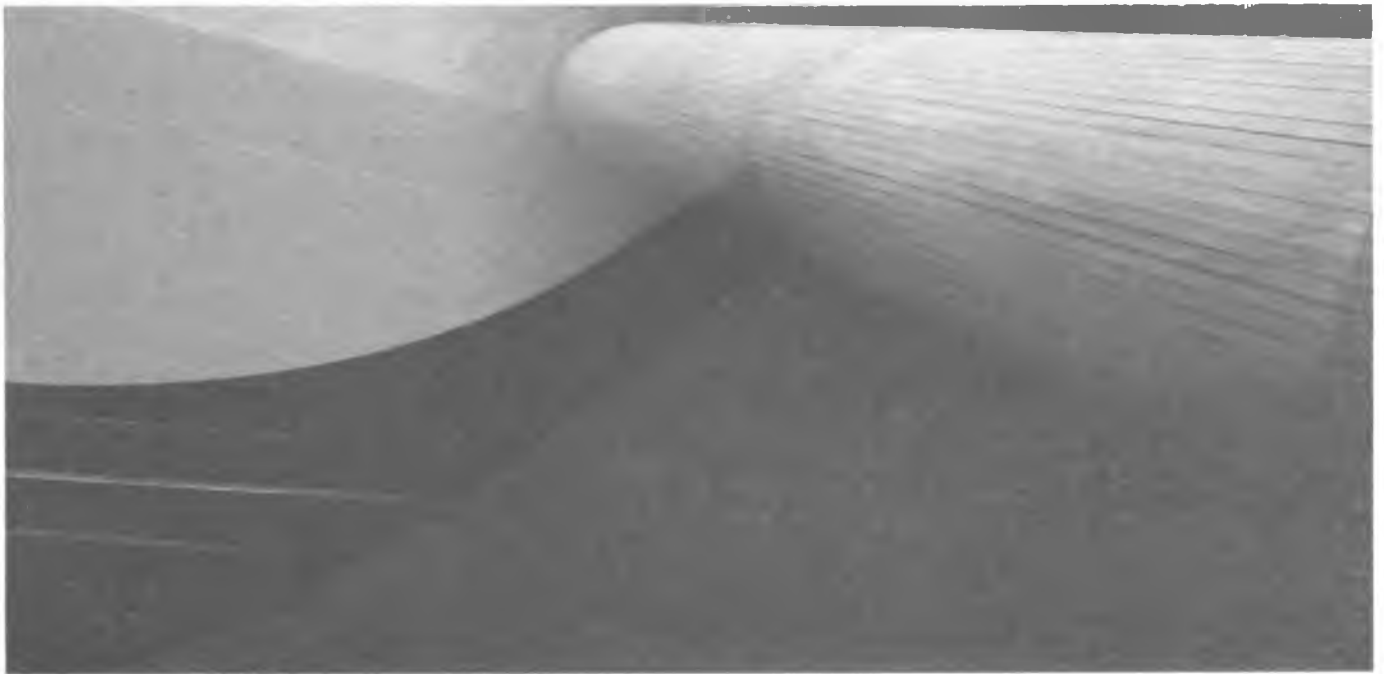
```
if (conin.hasNextInt()) i = conin.nextInt();
```

Если ввести с клавиатуры целое число **123**, то в результате выполнения приведенного выше фрагмента кода переменная `i` будет содержать целочисленное значение **123**.

Формально методы из группы `next` можно вызывать без предварительного вызова методов из группы `hasNext`, но делать этого все же не рекомендуется. Ведь если методу из группы `next` не удастся обнаружить данные искомого типа, он сгенерирует исключение `InputMismatchException`. Поэтому лучше сначала убедиться, что данные нужного типа присутствуют в потоке ввода, вызвав подходящий метод `hasNext`, и только после этого вызывать соответствующий метод `next`.

Упражнение для самопроверки по материалу главы 10

1. Для чего в Java определены как байтовые, так и символьные потоки?
2. Как известно, ввод-вывод данных на консоль осуществляется в текстовом виде. Почему же в Java для этой цели используются байтовые потоки?
3. Как открыть файл для чтения байтов?
4. Как открыть файл для чтения символов?
5. Как открыть файл для ввода-вывода с произвольным доступом?
6. Как преобразовать числовую строку "123.23" в двоичный эквивалент?
7. Напишите программу, которая будет копировать текстовые файлы. Видоизмените ее таким образом, чтобы все пробелы заменялись дефисами. Используйте при написании программы классы, представляющие байтовые потоки, а также традиционный способ закрытия файла явным вызовом метода `close()`.
8. Перепишите программу, созданную в ответ на предыдущий вопрос, таким образом, чтобы в ней использовались классы, представляющие символьные потоки. На этот раз воспользуйтесь оператором `try` с ресурсами для автоматического закрытия файла.
9. К какому типу относится поток `System.in`?
10. Что возвращает метод `read()` из класса `InputStream` по достижении конца потока?
11. Поток какого типа используется для чтения двоичных данных?
12. Классы `Reader` и `Writer` находятся на вершине иерархии классов _____.
13. Оператор `try` без ресурсов служит для _____.
14. Если для закрытия файла используется традиционный способ, то это лучше всего делать в блоке `finally`. Верно или неверно?



Глава 11

Многопоточное программирование

Основные навыки и понятия

- Общее представление о многопоточной обработке
- Класс Thread и интерфейс Runnable
- Создание потока
- Создание нескольких потоков
- Определение момента завершения потока
- Использование приоритетов потоков
- Представление о синхронизации потоков
- Применение синхронизированных блоков
- Взаимодействие потоков
- Приостановка, возобновление и остановка потоков

Среди многих замечательных свойств языка Java особое место принадлежит поддержке *многопоточного программирования*. Многопоточная программа состоит из двух или более частей, выполняемых параллельно. Каждая часть такой программы называется *потоком* и определяет отдельный путь выполнения команд. Таким образом, многопоточная обработка является особой формой многозадачности.

Общее представление о многопоточной обработке

Различают две разновидности многозадачности: на основе процессов и на основе потоков. В связи с этим важно понимать отличия между ними. *Процесс* фактически представляет собой исполняемую программу. Поэтому *многозадачность на основе процессов* — это средство, благодаря которому на компьютере могут параллельно выполняться две программы или более. Так, многозадачность на основе процессов позволяет одновременно выполнять программы текстового редактора, электронных таблиц и просмотра содержимого в Интернете. При организации многозадачности на основе процессов программа является наименьшей единицей кода, выполнение которой может координировать планировщик задач.

Поток представляет собой координируемую единицу исполняемого кода. Своим происхождением этот термин обязан понятию “поток исполнения”. При организации *многозадачности на основе потоков* у каждого процесса должен быть по крайней мере один поток, хотя их может быть и больше. Это означает, что в одной программе одновременно можно решать две и более задачи. Например, текст может форматироваться в редакторе текста одновременно с его выводом на печать, при условии, что оба эти действия выполняются в двух отдельных потоках. Несмотря на то что программы на Java выполняются в среде, поддерживающей многозадачность на основе процессов, в самих программах управлять процессами нельзя. В них можно управлять только потоками.

Главное преимущество многопоточной обработки заключается в том, что она позволяет писать программы, которые работают очень эффективно благодаря возможности выгодно использовать время простоя, неизбежно возникающее в ходе выполнения большинства программ. Как известно, большинство устройств ввода-вывода, будь то устройства, подключенные к сетевым портам, накопители на дисках или клавиатура, работают намного медленнее, чем центральный процессор (ЦП). Поэтому большую часть своего времени программе приходится ожидать отправки данных на устройство ввода-вывода или приема информации из него. А благодаря многопоточной обработке программа может решать какую-нибудь другую задачу во время вынужденного простоя. Например, в то время как одна часть программы отправляет файл через соединение с Интернетом, другая ее часть может выполнять чтение текстовой информации, вводимой с клавиатуры, а третья — осуществлять буферизацию очередного блока отправляемых данных.

Как известно, за последние несколько лет широкое распространение нашли многопроцессорные или многоядерные вычислительные системы, хотя по-прежнему повсеместно используются и однопроцессорные системы. В этой связи следует иметь в виду, что языковые средства организации многопоточной обработки в Java пригодны для обеих разновидностей вычислительных систем. В одноядерной системе параллельно выполняющиеся потоки разделяют ресурсы одного ЦП, получая по очереди квант времени ЦП. Поэтому в одноядерной системе два или более потока на самом деле не выполняются параллельно, а лишь используют время простоя ЦП. С другой стороны, в многопроцессорных или многоядерных системах два потока или более могут выполняться параллельно. Это, как правило, позволяет повысить производительность программ и скорость выполнения отдельных операций.

Поток может находиться в одном из нескольких состояний. В целом поток может быть *выполняющимся*; *готовым к выполнению*, как только он получит время и ресурсы ЦП; *приостановленным*, т.е. временно не выполняющимся; *возобновленным* в дальнейшем; *заблокированным* в ожидании ресурсов для своего выполнения; а также *завершенным*, когда его выполнение окончено и не может быть возобновлено.

В связи с организацией многозадачности на основе потоков возникает потребность в особом рода режиме, который называется *синхронизацией* и позволяет координировать выполнение потоков вполне определенным образом. Для такой синхронизации в Java предусмотрена отдельная подсистема, основные средства которой рассматриваются в этой главе.

Если вы пишете программы для таких операционных систем, как Windows, то принципы многопоточного программирования вам должны быть уже знакомы. Но то обстоятельство, что в Java имеются языковые средства для поддержки потоков, упрощает организацию многопоточной обработки, поскольку избавляет от необходимости реализовывать ее во всех деталях.

Класс Thread и интерфейс Runnable

В основу системы многопоточной обработки в Java положены класс Thread и интерфейс Runnable, входящие в пакет `java.lang`. Класс Thread инкапсулирует поток исполнения. Для того чтобы образовать новый поток, нужно создать класс, являющийся подклассом Thread или реализующий интерфейс Runnable.

В классе Thread определен ряд методов, позволяющих управлять потоками. Некоторые из этих наиболее употребительных методов описаны ниже. По мере их представления в последующих примерах программ вы ознакомитесь с ними поближе.

Метод	Описание
<code>final String getName()</code>	Получает имя потока
<code>final int getPriority ()</code>	Получает приоритет потока
<code>final boolean isAlive()</code>	Определяет, выполняется ли поток
<code>final void join()</code>	Ожидает завершения потока
<code>void run()</code>	Определяет точку входа в поток
<code>static void sleep(long миллисекунд)</code>	Приостанавливает исполнение потока на указанное число миллисекунд
<code>void start()</code>	Запускает поток, вызывая его метод <code>run()</code>

В каждом процессе имеется как минимум один поток исполнения, который называется *основным потоком*. Он получает управление уже при запуске программы. Следовательно, во всех рассматривавшихся до сих пор примерах программ использовался основной поток. От основного потока могут быть порождены другие, подчиненные потоки.

Создание потока

Для того чтобы создать поток, нужно построить объект типа `Thread`. Класс `Thread` инкапсулирует объект, который может стать исполняемым. Как пояснялось ранее, пригодные для исполнения объекты можно создавать в Java двумя способами:

- реализуя интерфейс `Runnable`;
- создавая подкласс класса `Thread`.

В большинстве примеров, представленных в этой главе, будет применяться первый способ. Хотя в примере для опробования 11.1 будет продемонстрировано, каким образом поток реализуется путем расширения класса `Thread`. Но независимо от выбранного способа создание экземпляра потока, организация доступа к нему и управление потоком осуществляется средствами класса `Thread`. Единственное отличие обоих способов состоит в том, как создается класс, активизирующий поток.

Интерфейс `Runnable` дает абстрактное описание единицы исполняемого кода. Для формирования потока подходит любой объект, реализующий этот интерфейс. В интерфейсе `Runnable` объявлен только один метод, `run()`:

```
public void run()
```

В теле метода `run()` определяется код, соответствующий новому потоку. Из этого метода можно вызывать другие методы, использовать в нем различные классы и объявлять переменные таким же образом, как это делается в основном потоке. Единственное отличие состоит в том, что метод `run()` создает точку входа в поток, исполняемый в программе параллельно с основным. Этот поток исполняется до тех пор, пока не произойдет возврат из метода `run()`.

После создания класса, реализующего интерфейс `Runnable`, следует создать экземпляр объекта типа `Thread` на основе объекта данного класса. В классе `Thread` определен ряд конструкторов. В дальнейшем будет использоваться следующий конструктор:

```
Thread(Runnable threadOb)
```

В качестве параметра *threadOb* этому конструктору передается экземпляр класса, реализующего интерфейс *Runnable*. Благодаря этому определяется место для исполнения потока.

Созданный поток не начнет исполнение до тех пор, пока не будет вызван метод *start()*, объявленный в классе *Thread*. По существу, единственным назначением метода *start()* является вызов метода *run()*. А объявляется метод *start()* следующим образом:

```
void start()
```

Ниже приведен пример программы, в которой создается и запускается на исполнение новый поток.

```
// Создание потока путем реализации интерфейса Runnable.
class MyThread implements Runnable {
    String thrdName;

    // Объекты типа MyThread выполняются в отдельных потоках, так как
    // класс MyThread реализует интерфейс Runnable.
    MyThread(String name) {
        thrdName = name;
    }

    // Точка входа в поток.
    public void run() {
        // Здесь начинают исполняться потоки.
        System.out.println(thrdName + " starting.");
        try {
            for(int count=0; count < 10; count++) {
                Thread.sleep(400);
                System.out.println("In " + thrdName +
                                   ", count is " + count);
            }
        }
        catch(InterruptedException exc) {
            System.out.println(thrdName + " interrupted.");
        }
        System.out.println(thrdName + " terminating.");
    }
}

class UseThreads {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");

        // сначала построить объект типа MyThread
        MyThread mt = new MyThread("Child #1"); // Создание исполняемого объекта.

        // далее сформировать поток из этого объекта
        Thread newThrd = new Thread(mt); // Формирование потока из этого объекта.

        // и, наконец, начать исполнение потока
        newThrd.start(); // Начало исполнения потока.
```



```

    for(int i=0; i<50; i++) {
        System.out.print(".");
        try {
            Thread.sleep(100);
        }
        catch(InterruptedException exc) {
            System.out.println("Main thread interrupted.");
        }
    }

    System.out.println("Main thread ending.");
}
}

```

Рассмотрим исходный код приведенной выше программы более подробно. Как видите, класс `MyThread` реализует интерфейс `Runnable`. Это означает, что объект типа `MyThread` подходит для использования в качестве потока, а следовательно, его можно передать конструктору класса `Thread`.

В теле метода `run()` присутствует цикл, в котором производится отсчет от 0 до 9. Обратите внимание на вызов метода `sleep()`. Этот метод приостанавливает поток, из которого он был вызван на указанное число миллисекунд. Ниже приведена общая форма объявления данного метода.

```
static void sleep(long миллисекунд) throws InterruptedException
```

Единственный параметр метода `sleep()` задает время задержки, определяемое числом миллисекунд. Как следует из объявления этого метода, в нем может быть сгенерировано исключение `InterruptedException`. Следовательно, его нужно вызывать в блоке `try`. Имеется и другой вариант метода `sleep()`, позволяющий точнее указывать время задержки в миллисекундах и дополнительно в наносекундах. Когда метод `sleep()` вызывается в методе `run()`, исполнение потока приостанавливается на 400 миллисекунд на каждом шаге цикла. Благодаря этому поток исполняется достаточно медленно, чтобы можно проследить за ним.

В методе `main()` создается новый объект типа `Thread`. Для этой цели служит приведенная ниже последовательность операторов.

```

// сначала построить объект типа MyThread
MyThread mt = new MyThread("Child #1");

// далее сформировать поток из этого объекта
Thread newThrd = new Thread(mt);

// и, наконец, начать исполнение потока
newThrd.start();

```

Как видите, сначала создается объект типа `MyThread`, а затем он используется для построения объекта типа `Thread`. Его можно передать конструктору класса `Thread` в качестве параметра, поскольку класс `MyThread` реализует интерфейс `Runnable`. И наконец, начинается исполнение нового потока, для чего вызывается метод `start()`, что приводит к вызову метода `run()` из порожденного потока. После вызова метода `start()` управление возвращается к методу `main()`, где начинается выполнение цикла `for`. Этот цикл повторяется 50 раз, приостанавливая на 100 миллисекунд исполнение потока на каждом своем шаге. Оба потока продолжают исполняться, разделяя ресурсы ЦП в однопроцессорной системе до тех пор, пока циклы в них не завершатся. Ниже

приведен результат выполнения данной программы. Вследствие отличий в вычислительных средах у вас может получиться несколько иной результат.

```
Main thread starting.
.Child #1 starting.
....In Child #1, count is 0
.....In Child #1, count is 1
.....In Child #1, count is 2
.....In Child #1, count is 3
.....In Child #1, count is 4
.....In Child #1, count is 5
.....In Child #1, count is 6
.....In Child #1, count is 7
.....In Child #1, count is 8
.....In Child #1, count is 9
Child #1 terminating.
Main thread ending.
```

В рассматриваемом здесь первом примере организации многопоточной обработки любопытно также отметить следующее обстоятельство: для демонстрации того факта, что основной и порожденный потоки исполняются одновременно, необходимо задержать завершение метода `main()` до тех пор, пока не окончится порожденный поток `mt`. В данном примере это достигается благодаря отличиям во временных характеристиках обоих потоков. Вызовы метода `sleep()` из цикла `for` в методе `main()` приводят в итоге к задержке на 5 секунд (50 шагов цикла $\times$ 100 миллисекунд), тогда как общая задержка с помощью того же самого метода в аналогичном цикле в методе `run()` составляет лишь 4 секунды (10 шагов цикла $\times$ 400 миллисекунд). Поэтому метод `run()` завершится приблизительно на 1 секунду раньше, чем метод `main()`. В итоге основной и порожденный потоки будут выполняться параллельно до тех пор, пока не завершится порожденный поток `mt`. А приблизительно через одну секунду завершится и основной поток в методе `main()`.

Отличий во временных характеристиках обоих потоков в данном и ряде последующих простых примеров оказывается достаточно для того, чтобы основной поток в методе `main()` завершился последним, но на практике этого, как правило, оказывается недостаточно. В Java предоставляются намного более совершенные способы, позволяющие организовать ожидание завершения потока. Далее в этой главе будет продемонстрирован более совершенный способ организации ожидания одним потоком завершения другого.

И последнее замечание: многопоточная программа обычно разрабатывается с таким расчетом, чтобы последним завершал свою работу основной поток. Как правило, выполнение программы продолжается до тех пор, пока все потоки не завершат работу. Поэтому завершение основного потока является не требованием, а рекомендуемой для наследования нормой, особенно для тех, кто лишь начинает осваивать многопоточное программирование.

Несложные усовершенствования многопоточной программы

Рассмотренная выше многопоточная программа вполне работоспособна, тем не менее ей не помешает небольшая доработка, повышающая ее эффективность. Во-первых, можно сделать так, чтобы поток начинал исполняться сразу после создания. Эта цель достигается созданием экземпляра объекта типа `Thread` в конструкторе класса

MyThread. И во-вторых, нет никакой нужды хранить в объекте типа MyThread имя потока, но присвоить имя потоку при его создании. Эту задачу позволяет решить следующий вариант конструктора Thread:

```
Thread(Runnable threadOb, String имя)
```

где *имя* обозначает конкретное наименование потока.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Почему основной поток рекомендуется завершать последним?

ОТВЕТ. Если основной поток завершался раньше порожденного, исполняющая система Java прежних версий могла зависнуть. Правда, в последних версиях исполняющей системы Java это явление уже не наблюдается. Но поскольку не всегда известно, в какой именно среде будет выполняться прикладная программа, то имеет смысл подстраховаться и не выбирать программные решения, чреватые потенциальными ошибками. Кроме того, в основном потоке удобно выполнять действия по подготовке к завершению программы, например закрывать файлы. Таким образом, основной поток желательно завершать последним. К счастью, организовать ожидание в основном потоке завершения порожденных потоков совсем не сложно.

Получить имя потока можно, используя метод `getName()`, определенный в классе Thread. Ниже приведено объявление этого метода.

```
final String getName()
```

В приведенной ниже программе имя присваивается потоку после его создания с помощью метода `setName()`. И хотя в этом нет особой необходимости, такое решение выбрано лишь для того, чтобы продемонстрировать возможности класса Thread. Объявление метода `setName()` имеет следующий вид:

```
final void setName(String имя_потока)
```

где *имя_потока* обозначает имя, которое присваивается потоку.

Ниже приведена видоизмененная версия предыдущей программы.

// Видоизменение класса **MyThread**.

```
class MyThread implements Runnable {
    Thread thrd; // В этой переменной хранится ссылка на поток.

    // построить новый поток
    MyThread(String name) {
        thrd = new Thread(this, name); // Поток именуется при его создании.
        thrd.start(); // Начало исполнения потока.
    }

    // начать исполнение нового потока
    public void run() {
        System.out.println(thrd.getName() + " starting.");
        try {
            for(int count=0; count<10; count++) {
                Thread.sleep(400);
            }
        }
    }
}
```

```

        System.out.println("In " + thrd.getName() +
                           ", count is " + count);
    }
}
catch (InterruptedException exc) {
    System.out.println(thrd.getName() + " interrupted.");
}
System.out.println(thrd.getName() + " terminating.");
}
}

class UseThreadsImproved {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");

        // Теперь поток начинается при его создании.
        MyThread mt = new MyThread("Child #1");

        for (int i=0; i < 50; i++) {
            System.out.print(".");
            try {
                Thread.sleep(100);
            }
            catch (InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        }
        System.out.println("Main thread ending.");
    }
}

```

Эта версия программы дает такой же результат, как и предыдущая. Обратите внимание на то, что ссылка на поток хранится в переменной `thrd` экземпляра класса `MyThread`.

Пример для опробования 11.1. Расширение класса `Thread`

Исходный файл: `ExtendThread.java` Реализация интерфейса `Runnable` — это лишь один из способов получения экземпляров потоковых объектов. Другой способ состоит в создании подкласса, производного от класса `Thread`. В этом проекте будет продемонстрировано, каким образом расширение класса `Thread` позволяет реализовать такие же функциональные возможности, как и у рассмотренной выше программы `UseThreadsImproved`.

В подклассе, производном от класса `Thread`, нужно переопределить метод `run()`, который является точкой входа в новый поток. Для того чтобы начать исполнение нового потока, следует вызвать метод `start()`. Можно также переопределить и другие методы из класса `Thread`, но делать это не обязательно.

Последовательность действий

1. Создайте файл `ExtendThread.java`. Скопируйте в этот файл исходный код второго рассмотренного ранее примера программы (файл `UseThreadsImproved.java`).

2. Измените объявление класса `MyThread`. Теперь он должен быть подклассом, производным от класса `Thread`, как показано ниже.

```
class MyThread extends Thread {
```

3. Удалите следующую строку кода:

```
Thread thrd;
```

Переменная `thrd` уже не нужна, поскольку класс `MyThread` включает в себя экземпляр класса `Thread` и может ссылаться на самого себя.

4. Внесите в конструктор класса `Thread` следующие изменения:

```
// построить новый поток.
MyThread(String name) {
    super(name); // присвоить потоку имя
    start(); // начать поток
}
```

Как видите, в данном конструкторе присутствует ключевое слово `super`, которое используется для вызова следующего варианта конструктора `Thread`:

```
Thread(String имя);
```

где *имя* обозначает присваиваемое потоку конкретное имя.

5. Внесите приведенные ниже изменения в метод `run()`, чтобы он вызывал метод `getName()` непосредственно, не предваряя его именем переменной `thrd`.

```
// начать исполнение нового метода
public void run() {
    System.out.println(getName() + " starting.");
    try {
        for(int count=0; count < 10; count++) {
            Thread.sleep(400);
            System.out.println("In " + getName() +
                               ", count is " + count);
        }
    }
    catch(InterruptedException exc) {
        System.out.println(getName() + " interrupted.");
    }

    System.out.println(getName() + " terminating.");
}
```

6. Ниже приведен весь исходный код программы, в которой вместо реализации интерфейса `Runnable` используется подкласс, производный от класса `Thread`. Выполнение этой программы дает такой же результат, как и предыдущие ее версии.

```
/*
    Пример для опробования 11.1.

    Расширение класса Thread.
*/
class MyThread extends Thread {
    // построить новый поток
    MyThread(String name) {
        super(name); // присвоить потоку имя
```

```

        start(); // начать поток
    }

    // начать исполнение нового потока
    public void run() {
        System.out.println(getName() + " starting.");
        try {
            for(int count=0; count < 10; count++) {
                Thread.sleep(400);
                System.out.println("In " + getName() +
                                   ", count is " + count);
            }
        }
        catch (InterruptedException exc) {
            System.out.println(getName() + " interrupted.");
        }

        System.out.println(getName() + " terminating.");
    }
}

class ExtendThread {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");

        MyThread mt = new MyThread("Child #1");

        for(int i=0; i < 50; i++) {
            System.out.print(".");
            try {
                Thread.sleep(100);
            }
            catch (InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        }

        System.out.println("Main thread ending.");
    }
}

```

Создание нескольких потоков

В предыдущем примере был создан только один порожденный поток. Но в программе можно породить столько потоков, сколько требуется. Например, в приведенной ниже программе формируются три порожденных потока.

// Создание нескольких потоков.

```

class MyThread implements Runnable {
    Thread thrd;

```

```

// построить новый поток
MyThread(String name) {
    thrd = new Thread(this, name);

    thrd.start(); // начать поток
}

// начать исполнение нового потока
public void run() {
    System.out.println(thrd.getName() + " starting.");
    try {
        for(int count=0; count < 10; count++) {
            Thread.sleep(400);
            System.out.println("In " + thrd.getName() +
                               ", count is " + count);
        }
    }
    catch(InterruptedException exc) {
        System.out.println(thrd.getName() + " interrupted.");
    }

    System.out.println(thrd.getName() + " terminating.");
}
}

class MoreThreads {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");

        // Создание и запуск на исполнение трех потоков.
        MyThread mt1 = new MyThread("Child #1");
        MyThread mt2 = new MyThread("Child #2");
        MyThread mt3 = new MyThread("Child #3");

        for(int i=0; i < 50; i++) {
            System.out.print(".");
            try {
                Thread.sleep(100);
            }
            catch(InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        }

        System.out.println("Main thread ending.");
    }
}

```

Ниже приведен результат выполнения данной программы.

```

Main thread starting.
Child #1 starting.
Child #2 starting.
Child #3 starting.

```

```

...In Child #3, count is 0
In Child #2, count is 0
In Child #1, count is 0
....In Child #1, count is 1
In Child #2, count is 1
In Child #3, count is 1
....In Child #2, count is 2
In Child #3, count is 2
In Child #1, count is 2
...In Child #1, count is 3
In Child #2, count is 3
In Child #3, count is 3
....In Child #1, count is 4
In Child #3, count is 4
In Child #2, count is 4
....In Child #1, count is 5
In Child #3, count is 5
In Child #2, count is 5
...In Child #3, count is 6
.In Child #2, count is 6
In Child #1, count is 6
...In Child #3, count is 7
In Child #1, count is 7
In Child #2, count is 7
....In Child #2, count is 8
In Child #1, count is 8
In Child #3, count is 8
....In Child #1, count is 9
Child #1 terminating.
In Child #2, count is 9
Child #2 terminating.
In Child #3, count is 9
Child #3 terminating.
.....Main thread ending.

```

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Зачем нужны два способа создания порожденных потоков (расширение класса `Thread` и реализация интерфейса `Runnable`)? Какой из этих способов предпочтительнее?

ОТВЕТ. В классе `Thread` имеется несколько методов, которые можно переопределить в порожденном классе. Из них обязательному переопределению подлежит только метод `run()`. Этот же метод, безусловно, должен быть определен и при реализации интерфейса `Runnable`. Некоторые программирующие на Java считают, что создавать подкласс, порожденный от класса `Thread`, следует только в том случае, если нужно дополнить его новыми функциями. Так, если переопределять любые другие методы из класса `Thread` не нужно, то можно ограничиться только реализацией интерфейса `Runnable`. Кроме того, реализация интерфейса `Runnable` позволяет создаваемому потоку наследовать класс, отличающийся от `Thread`.

Как видите, после запуска на исполнение все три потока совместно используют ресурсы ЦП. Следует иметь в виду, что потоки в данном примере запускаются на исполнение в том порядке, в каком они были созданы. Но так происходит не всегда. Исполняющая система Java сама планирует исполнение потоков. Вследствие отличий в вычислительных средах у вас может получиться несколько иной результат.

Определение момента завершения потока

Нередко требуется знать, когда завершится поток. Так, в приведенных выше примерах ради большей наглядности нужно было поддерживать основной поток действующим до тех пор, пока не завершатся остальные потоки. Для этой цели основной поток переводился в состояние ожидания на более продолжительное время, чем порожденные им потоки. Но такое решение вряд ли можно считать удовлетворительным или общепотребительным!

Правда, в классе `Thread` предусмотрены два средства, позволяющие определить, завершился ли поток. Первым из них является метод `isAlive()`, объявление которого приведено ниже.

```
final boolean isAlive()
```

Этот метод возвращает логическое значение `true`, если поток, для которого он вызывается, все еще выполняется. В противном случае он возвращает логическое значение `false`. Для того чтобы опробовать метод `isAlive()` на практике, замените в предыдущей программе класс `MoreThreads` новой версией, исходный код которой приведен ниже.

```
// Применение метода isAlive().
class MoreThreads {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");

        MyThread mt1 = new MyThread("Child #1");
        MyThread mt2 = new MyThread("Child #2");
        MyThread mt3 = new MyThread("Child #3");

        do {
            System.out.print(".");
            try {
                Thread.sleep(100);
            }
            catch (InterruptedException exc) {
                System.out.println("Main thread interrupted.");
            }
        } while (mt1.thrd.isAlive() ||
                mt2.thrd.isAlive() ||
                mt3.thrd.isAlive());
        System.out.println("Main thread ending.");
    }
}
```

Эта версия дает такой же результат, как и предыдущая. Единственное отличие состоит в том, что в данном случае ожидание завершения порожденного потока организовано

с помощью метода `isAlive()`. Вторым средством, позволяющим определить, завершился ли поток, является метод `join()`, объявление которого приведено ниже.

```
final void join() throws InterruptedException
```

Этот метод ожидает завершения потока, для которого он был вызван. Его имя `join` выбрано потому, что вызывающий поток ожидает, когда указанный поток *присоединится* (англ. *join*) к нему. Имеется и другой вариант метода `join()`, позволяющий указать максимальное время ожидания момента, когда поток завершится.

В приведенном ниже примере программы наличие метода `join()` гарантирует, что основной поток завершит работу последним.

```
// Применение метода join().
```

```
class MyThread implements Runnable {
    Thread thrd;

    // построить новый поток
    MyThread(String name) {
        thrd = new Thread(this, name);
        thrd.start(); // начать поток
    }

    // начать исполнение нового потока
    public void run() {
        System.out.println(thrd.getName() + " starting.");
        try {
            for(int count=0; count < 10; count++) {
                Thread.sleep(400);
                System.out.println("In " + thrd.getName() +
                                   ", count is " + count);
            }
        }
        catch(InterruptedException exc) {
            System.out.println(thrd.getName() + " interrupted.");
        }
        System.out.println(thrd.getName() + " terminating.");
    }
}

class JoinThreads {
    public static void main(String args[]) {
        System.out.println("Main thread starting.");

        MyThread mt1 = new MyThread("Child #1");
        MyThread mt2 = new MyThread("Child #2");
        MyThread mt3 = new MyThread("Child #3");

        try {
            // Ожидание до тех пор, пока указанный метод не завершится.
            mt1.thrd.join();
            System.out.println("Child #1 joined.");
            mt2.thrd.join();
            System.out.println("Child #2 joined.");
```

```

        mt3.thrd.join();
        System.out.println("Child #3 joined.");
    }
    catch (InterruptedException exc) {
        System.out.println("Main thread interrupted.");
    }
    System.out.println("Main thread ending.");
}
}

```

Результат выполнения данной программы приведен ниже. Вследствие отличий в вычислительных средах он может получиться у вас несколько иным.

```

Main thread starting.
Child #1 starting.
Child #2 starting.
Child #3 starting.
In Child #2, count is 0
In Child #1, count is 0
In Child #3, count is 0
In Child #2, count is 1
In Child #3, count is 1
In Child #1, count is 1
In Child #2, count is 2
In Child #1, count is 2
In Child #3, count is 2
In Child #2, count is 3
In Child #3, count is 3
In Child #1, count is 3
In Child #3, count is 4
In Child #2, count is 4
In Child #1, count is 4
In Child #3, count is 5
In Child #1, count is 5
In Child #2, count is 5
In Child #3, count is 6
In Child #2, count is 6
In Child #1, count is 6
In Child #3, count is 7
In Child #1, count is 7
In Child #2, count is 7
In Child #3, count is 8
In Child #2, count is 8
In Child #1, count is 8
In Child #3, count is 9
Child #3 terminating.
In Child #2, count is 9
Child #2 terminating.
In Child #1, count is 9
Child #1 terminating.
Child #1 joined.
Child #2 joined.
Child #3 joined.
Main thread ending.

```

Как видите, после того как вызываемый метод `join()` возвращает управление, исполнение потока прекращается.

Приоритеты потоков

У каждого потока имеется свой приоритет, который отчасти определяет, насколько часто поток получает доступ к ЦП. Вообще говоря, низкоприоритетные потоки получают доступ к ЦП реже, чем высокоприоритетные. Таким образом, в течение заданного промежутка времени низкоприоритетному потоку будет доступно меньше времени ЦП, чем высокоприоритетному. Как и следовало ожидать, время ЦП, получаемое потоком, оказывает определяющее влияние на характер его исполнения и взаимодействия с другими потоками, исполняемыми в настоящий момент в системе.

Следует иметь в виду, что, помимо приоритета, на частоту доступа потока к ЦП оказывают влияние и другие факторы. Так, если высокоприоритетный поток ожидает доступа к некоторому ресурсу, например для ввода с клавиатуры, он блокируется, а вместо него исполняется низкоприоритетный поток. Но когда высокоприоритетный поток получит доступ к ресурсам, он прервет низкоприоритетный поток и возобновит свое исполнение. На планирование работы потоков оказывает также влияние способ, посредством которого в операционной системе поддерживается многозадачность (см. врезку “Обращение к знатоку” в конце этого раздела). Следовательно, если один поток имеет более высокий приоритет, чем другой поток, это еще не означает, что первый поток будет исполняться быстрее второго. Высокий приоритет потока лишь означает, что потенциально он может получить больше времени ЦП.

При запуске порожденного потока его приоритет устанавливается равным приоритету родительского потока. Изменить приоритет можно, вызвав метод `setPriority()` из класса `Thread`. Ниже приведено объявление этого метода.

```
final void setPriority(int уровень)
```

В качестве параметра *уровень* данному методу передается новый приоритет для потока. Значение параметра *уровень* должно находиться в пределах от `MIN_PRIORITY` до `MAX_PRIORITY`. В настоящее время этим константам соответствуют числовые значения от 1 до 10. Для того чтобы восстановить приоритет потока по умолчанию, следует указать значение 5, которому соответствует константа `NORM_PRIORITY`. Константы, определяющие приоритеты потоков, определены как `static final` в классе `Thread`.

Получить текущий приоритет можно с помощью метода `getPriority()` из класса `Thread`, объявляемого следующим образом:

```
final int getPriority()
```

Ниже приведен пример программы, демонстрирующий использование двух потоков с разными приоритетами. Потоки создаются как экземпляры класса `Priority`. В методе `run()` содержится цикл, отсчитывающий число своих шагов. Этот цикл завершает работу, когда значение счетчика достигает 10000000 или же когда статическая переменная `stop` принимает логическое значение `true`. Первоначально переменной `stop` присваивается логическое значение `false`, но первый же поток, заканчивающий отсчет, устанавливает в ней логическое значение `true`. В результате второй поток завершится, как только ему будет выделен квант времени. В цикле производится проверка символьной строки в переменной `currentName` на совпадение с именем исполняемого потока. Если они не совпадают, это означает, что произошло переключение задач. При этом отображается имя нового потока, которое присваивается переменной `currentName`. Это дает

возможность следить за тем, насколько часто каждый поток получает время ЦП. После остановки обоих потоков выводится число шагов, выполненных в каждом цикле.

// Демонстрация потоков с разными приоритетами.

```
class Priority implements Runnable {
    int count;
    Thread thrd;

    static boolean stop = false;
    static String currentName;

    /* Построение нового потока. Обратите внимание на то,
       что конструктор не запускает поток на исполнение. */
    Priority(String name) {
        thrd = new Thread(this, name);
        count = 0;
        currentName = name;
    }

    // начать исполнение нового потока
    public void run() {
        System.out.println(thrd.getName() + " starting.");
        do {
            count++;

            if(currentName.compareTo(thrd.getName()) != 0) {
                currentName = thrd.getName();
                System.out.println("In " + currentName);
            }
            // Первый же поток, в котором достигнуто значение 10000000,
            // завершает остальные потоки.
        } while(stop == false && count < 10000000);
        stop = true;

        System.out.println("\n" + thrd.getName() +
                           " terminating.");
    }
}

class PriorityDemo {
    public static void main(String args[]) {
        Priority mt1 = new Priority("High Priority");
        Priority mt2 = new Priority("Low Priority");

        // задать приоритеты
        // Поток mt1 получает более высокий приоритет, чем поток mt2.
        mt1.thrd.setPriority(Thread.NORM_PRIORITY+2);
        mt2.thrd.setPriority(Thread.NORM_PRIORITY-2);

        // запустить потоки на исполнение
        mt1.thrd.start();
        mt2.thrd.start();

        try {
```

```

        mt1.thrd.join();
        mt2.thrd.join();
    }
    catch (InterruptedException exc) {
        System.out.println("Main thread interrupted.");
    }
    System.out.println("\nHigh priority thread counted to " +
        mt1.count);
    System.out.println("Low priority thread counted to " +
        mt2.count);
}
}

```

Результат выполнения данной программы выглядит следующим образом:

```

High Priority starting.
In High Priority
Low Priority starting.
In Low Priority
In High Priority

High Priority terminating.

Low Priority terminating.

High priority thread counted to 10000000
Low priority thread counted to 8183

```

В данном примере большую часть времени ЦП получает высокоприоритетный поток. Очевидно, что результат выполнения программы существенно зависит от быстродействия ЦП и их количества, типа операционной системы и наличия прочих задач, выполняющихся в системе.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Оказывает ли влияние конкретная реализация многозадачности на то, сколько времени ЦП получает поток?

ОТВЕТ. Если не учитывать приоритет потока, то наиболее важным фактором, оказывающим влияние на исполнение потока, является способ реализации многозадачности и планирования заданий в операционной системе. В одних операционных системах применяется вытесняющая многозадачность, т.е. каждый поток получает квант времени. А в других системах планирование заданий осуществляется по-другому. В частности, второй поток иногда получает управление только после завершения первого. В системах с невытесняющей многозадачностью один поток будет господствовать над остальными, препятствуя их исполнению.

Синхронизация

Если в программе используется несколько потоков, то иногда приходится координировать действия двух потоков или более. Процесс достижения такой координации называется *синхронизацией*. Самой распространенной причиной для синхронизации является необходимость разделять среди двух или более потоков общий ресурс, который может

быть одновременно доступен только одному потоку. Например, когда в одном потоке выполняется запись информации в файл, второму потоку должно быть запрещено делать это в тот же самый момент времени. Синхронизация требуется и в том случае, если один поток ожидает событие, вызываемое другим потоком. В подобной ситуации требуются какие-то средства, позволяющие приостановить один из потоков до тех пор, пока не произойдет событие в другом потоке. После этого ожидающий поток может возобновить свое выполнение.

Главным для синхронизации в Java является понятие *монитора*, контролирующего доступ к объекту. Монитор реализует принцип *блокировки*. Если объект заблокирован одним потоком, то он оказывается недоступным для других потоков. В какой-то момент объект разблокируется, и другие потоки могут обращаться к нему.

У каждого объекта в Java имеется свой монитор. Этот механизм встроен в сам язык. Следовательно, все объекты поддаются синхронизации. Для поддержки синхронизации в Java предусмотрено ключевое слово `synchronized` и ряд вполне определенных методов у каждого из объектов. А поскольку средства синхронизации встроены в язык, то пользоваться ими на практике очень просто — гораздо проще, чем может показаться на первый взгляд. Для многих программ средства синхронизации объектов по сути прозрачны.

Синхронизировать код можно двумя способами. Оба способа рассматриваются ниже, и в обоих используется ключевое слово `synchronized`.

Применение синхронизированных методов

Для того чтобы синхронизировать метод, в его объявлении следует указать ключевое слово `synchronized`. Когда такой метод получает управление, вызывающий поток активизирует монитор, что приводит к блокированию объекта. Если объект заблокирован, он недоступен из другого потока, а кроме того, его нельзя вызвать из других синхронизированных методов, определенных в классе данного объекта. Когда выполнение синхронизированного метода завершается, монитор разблокирует объект, что позволяет другому потоку использовать этот метод. Таким образом, для достижения синхронизации программирующему на Java не приходится прилагать каких-то особых усилий.

Ниже приведен пример программы, демонстрирующий контролируемый доступ к методу `sumArray()`. Этот метод суммирует элементы целочисленного массива.

// Применение ключевого слова **synchronize** для управления доступом.

```
class SumArray {
    private int sum;

    // Метод sumArray() синхронизирован.
    synchronized int sumArray(int nums[]) {
        sum = 0; // обнулить сумму

        for(int i=0; i<nums.length; i++) {
            sum += nums[i];
            System.out.println("Running total for " +
                               Thread.currentThread().getName() +
                               " is " + sum);
        }
        try {
            Thread.sleep(10); // разрешить переключение задач
```

```

    }
    catch(InterruptedException exc) {
        System.out.println("Main thread interrupted.");
    }
}
return sum;
}
}

class MyThread implements Runnable {
    Thread thrd;
    static SumArray sa = new SumArray();
    int a[];
    int answer;

    // построить новый поток
    MyThread(String name, int nums[]) {
        thrd = new Thread(this, name);
        a = nums;
        thrd.start(); // начать поток
    }

    // начать исполнение нового потока
    public void run() {
        int sum;

        System.out.println(thrd.getName() + " starting.");

        answer = sa.sumArray(a);
        System.out.println("Sum for " + thrd.getName() +
                           " is " + answer);

        System.out.println(thrd.getName() + " terminating.");
    }
}

class Sync {
    public static void main(String args[]) {
        int a[] = {1, 2, 3, 4, 5};

        MyThread mt1 = new MyThread("Child #1", a);
        MyThread mt2 = new MyThread("Child #2", a);
    }
}

```

Выполнение этой программы дает следующий результат:

```

Child #1 starting.
Running total for Child #1 is 1
Child #2 starting.
Running total for Child #1 is 3
Running total for Child #1 is 6
Running total for Child #1 is 10
Running total for Child #1 is 15

```



```

Sum for Child #1 is 15
Child #1 terminating.
Running total for Child #2 is 1
Running total for Child #2 is 3
Running total for Child #2 is 6
Running total for Child #2 is 10
Running total for Child #2 is 15
Sum for Child #2 is 15
Child #2 terminating.

```

Рассмотрим подробнее эту программу. В ней определены три класса. Имя первого — `SumArray`. В нем содержится метод `sumArray()`, вычисляющий сумму элементов целочисленного массива. Во втором классе `MyThread` используется статический объект `sa` типа `SumArray` для получения суммы элементов массива. А поскольку он статический, то все экземпляры класса `MyThread` используют одну его копию. И наконец, в классе `Sync` создаются два потока, в каждом из которых должна вычисляться сумма элементов массива.

В методе `sumArray()` вызывается метод `sleep()`. Он нужен лишь для того, чтобы обеспечить переключение задач. Метод `sumArray()` синхронизирован, и поэтому в каждый момент времени он может использоваться только одним потоком. Следовательно, когда второй порожденный поток начинает свое исполнение, он не может вызвать метод `sumArray()` до тех пор, пока этот метод не завершится в первом потоке. Благодаря этому обеспечивается правильность получаемого результата.

Для того чтобы лучше понять эффект от использования ключевого слова `synchronized`, попробуйте удалить его из объявления метода `sumArray()`. В итоге метод `sumArray()` потеряет синхронизацию и может быть использован в нескольких потоках одновременно. Это приведет к затруднению в связи с тем, что результат расчета суммы сохраняется в переменной `sum`, значение которой изменяется при каждом вызове метода `sumArray()` для статического объекта `sa`. Так, если в двух потоках одновременно сделать вызов `sa.sumArray()`, расчет суммы окажется неверным, поскольку в переменной `sum` накапливаются результаты суммирования, выполняемого одновременно в двух потоках. Ниже приведен результат выполнения той же самой программы, где из объявления метода `sumArray()` удалено ключевое слово `synchronized`. (Вследствие отличий в вычислительных средах у вас может получиться несколько иной результат.)

```

Child #1 starting.
Running total for Child #1 is 1
Child #2 starting.
Running total for Child #2 is 1
Running total for Child #1 is 3
Running total for Child #2 is 5
Running total for Child #2 is 8
Running total for Child #1 is 11
Running total for Child #2 is 15
Running total for Child #1 is 19
Running total for Child #2 is 24
Sum for Child #2 is 24
Child #2 terminating.
Running total for Child #1 is 29
Sum for Child #1 is 29
Child #1 terminating.

```

Нетрудно заметить, что вследствие одновременного вызова `sa.sumArray()` из разных потоков результат искажается.

Прежде чем переходить к рассмотрению следующей темы, перечислим основные свойства синхронизированных методов.

- Синхронизированный метод создается путем указания ключевого слова `synchronized` в его объявлении.
- Как только синхронизированный метод любого объекта получает управление, объект блокируется и ни один синхронизированный метод этого объекта не может быть вызван другим потоком.
- Потоки, которым требуется синхронизированный метод, используемый другим потоком, ожидают до тех пор, пока не будет разблокирован объект, для которого он вызывается.
- Когда синхронизированный метод завершается, разблокируется объект, для которого он вызывается.

Синхронизированные блоки

Несмотря на то что создание синхронизированных методов в классах — простой и эффективный способ управления потоками, такой способ оказывается пригодным далеко не всегда. Иногда возникает потребность синхронизировать доступ к методам, в объявлении которых отсутствует ключевое слово `synchronized`. Подобная ситуация часто возникает при использовании классов, которые были созданы независимыми разработчиками и исходный код которых недоступен. В таком случае ввести в объявление нужного метода ключевое слово `synchronized` вряд ли удастся. Как же тогда синхронизировать объект класса, содержащего этот метод? К счастью, данное затруднение разрешается очень просто. Достаточно ввести вызов метода в синхронизированный кодовый блок типа `synchronized`.

Синхронизированный блок определяется следующим образом:

```
synchronized(ссылка_на_объект) {
    // синхронизируемые операторы
}
```

где *ссылка_на_объект* обозначает ссылку на конкретный объект, который должен быть синхронизирован. Как только содержимое синхронизированного блока получит управление, ни один другой поток не сможет вызвать метод для объекта, на который делается *ссылка_на_объект*, до тех пор, пока этот кодовый блок не завершится.

Следовательно, обращение к методу `sumArray()` можно синхронизировать, вызвав его из синхронизированного блока. Такой способ демонстрируется в приведенной ниже переделанной версии предыдущей программы.

```
// Применение синхронизированного блока
// для управления доступом к методу sumArray().
class SumArray {
    private int sum;

    // Здесь метод sumArray() не синхронизирован.
    int sumArray(int nums[]) {
        sum = 0; // обнулить сумму
```

```

    for(int i=0; i<nums.length; i++) {
        sum += nums[i];
        System.out.println("Running total for " +
                           Thread.currentThread().getName() +
                           " is " + sum);

        try {
            Thread.sleep(10); // разрешить переключение задач
        }
        catch(InterruptedException exc) {
            System.out.println("Main thread interrupted.");
        }
    }
    return sum;
}

class MyThread implements Runnable {
    Thread thrd;
    static SumArray sa = new SumArray();
    int a[];
    int answer;

    // построить новый поток
    MyThread(String name, int nums[]) {
        thrd = new Thread(this, name);
        a = nums;
        thrd.start(); // начать поток
    }

    // начать исполнение нового потока
    public void run() {
        int sum;

        System.out.println(thrd.getName() + " starting.");

        // Здесь вызовы метода sumArray() для объекта sa синхронизированы.
        synchronized(sa) {
            answer = sa.sumArray(a);
        }
        System.out.println("Sum for " + thrd.getName() +
                           " is " + answer);

        System.out.println(thrd.getName() + " terminating.");
    }
}

class Sync {
    public static void main(String args[]) {
        int a[] = {1, 2, 3, 4, 5};

        MyThread mt1 = new MyThread("Child #1", a);
        MyThread mt2 = new MyThread("Child #2", a);
    }
}

```

```

try {
    mt1.thrd.join();
    mt2.thrd.join();
} catch (InterruptedException exc) {
    System.out.println("Main thread interrupted.");
}
}
}

```

Выполнение этой версии программы дает такой же правильный результат, как и предыдущей ее версии, в которой использовался синхронизированный метод.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Говорят, что существуют так называемые “утилиты параллелизма”. Что это такое? И что такое Fork/Join Framework?

ОТВЕТ. Утилиты параллелизма, входящие в пакет `java.util.concurrent` (и подчиненные ему пакеты), служат для поддержки параллельного программирования. Среди прочего они предоставляют синхронизаторы, пулы потоков, диспетчеры исполнения и блокировки, расширяющие возможности контроля над исполнением потока. К числу самых привлекательных средств прикладного интерфейса параллельного программирования относится среда Fork/Join Framework, включенная в состав JDK 7.

В Fork/Join Framework поддерживается так называемое *параллельное программирование*. Этим термином обычно обозначаются приемы программирования, использующие преимущества компьютеров с двумя и более процессорами, включая и многоядерные системы, разделяя задачи на более мелкие подзадачи, выполняемые отдельным процессором. Нетрудно себе представить, что такой подход позволяет существенно повысить производительность и пропускную способность. Главное преимущество среды Fork/Join Framework заключается в простоте ее использования. Она упрощает создание многопоточных программ с автоматическим масштабированием для использования нескольких процессоров в системе. Следовательно, она облегчает разработку параллельных решений таких распространенных задач, как выполнение операций над элементами массива. Утилиты параллелизма вообще и Fork/Join Framework в частности относятся к тем средствам, которые следует освоить, приобретя некоторый опыт многопоточного программирования.

Организация взаимодействия потоков с помощью методов `notify()`, `wait()` и `notifyAll()`

Рассмотрим для примера следующую ситуацию. В потоке `T` выполняется синхронизированный метод, которому необходим доступ к ресурсу `R`. Этот ресурс временно недоступен. Что должен предпринять поток `T`? Если он будет ожидать в цикле освобождения ресурса `R`, объект будет по-прежнему заблокирован и другие потоки не смогут обратиться к нему. Такое решение малоприспособно, поскольку оно сводит на нет все преимущества программирования в многопоточной среде. Намного лучше, если поток `T` временно разблокирует объект и позволит другим потокам воспользоваться его методами. Когда

ресурс R станет доступным, поток T получит об этом уведомление и возобновит свое исполнение. Но для того чтобы такое решение можно было реализовать, необходимы средства взаимодействия потоков, с помощью которых один поток мог бы уведомить другой поток о том, что он приостановил свое исполнение, а также получить уведомление о том, что его исполнение может быть возобновлено. Для организации подобного взаимодействия потоков в Java предусмотрены методы `wait()`, `notify()` и `notifyAll()`.

Эти методы реализованы в классе `Object`, поэтому они доступны для любого объекта. Но обратиться к ним можно только из синхронизированного контекста. А применяются они следующим образом. Когда поток временно приостанавливает свое исполнение, он вызывает метод `wait()`. При этом поток переходит в состояние ожидания и монитор данного объекта освобождается, позволяя другим потокам использовать объект. Впоследствии ожидающий поток возобновит свое выполнение, когда другой поток войдет в тот же самый монитор и вызовет метод `notify()` или `notifyAll()`.

В классе `Object` определены различные формы объявления метода `wait()`, как показано ниже.

```
final void wait() throws InterruptedException
final void wait(long миллисекунд) throws InterruptedException
final void wait(long миллисекунд, int наносекунд) throws InterruptedException
```

В первой своей форме метод `wait()` переводит поток в режим ожидания до поступления уведомления. Во второй форме метода организуется ожидание уведомления или до тех пор, пока не истечет указанный период времени. А третья форма позволяет точнее задавать период времени в наносекундах.

Ниже приведены общие формы объявления методов `notify()` и `notifyAll()`.

```
final void notify()
final void notifyAll()
```

При вызове метода `notify()` возобновляется исполнение одного ожидающего потока. А метод `notifyAll()` уведомляет все потоки об освобождении объекта, и тот поток, который имеет наивысший приоритет, получает доступ к объекту.

Прежде чем рассматривать конкретный пример, демонстрирующий применение метода `wait()`, необходимо сделать важное замечание. Несмотря на то что метод `wait()` должен переводить поток в состояние ожидания до тех пор, пока не будет вызван метод `notify()` или `notifyAll()`, иногда поток выводится из состояния ожидания вследствие так называемой *ложной активизации*. Условия для ложной активизации сложны, возникают редко, а их обсуждение выходит за рамки этой книги. Но в компании Oracle рекомендуют учитывать вероятность проявления ложной активизации и помещать вызов метода `wait()` в цикл. В этом цикле должно проверяться условие, по которому поток переводится в состояние ожидания. Именно такой подход и применяется в рассматриваемом ниже примере.

Пример применения методов `wait()` и `notify()`

Для того чтобы стала понятнее потребность в применении методов `wait()` и `notify()` в многопоточном программировании, рассмотрим пример программы, имитирующей работу часов и выводящей на экран слова "Tick" (Тик) и "Tock" (Так). Для этой цели создадим класс `TickTock`, который будет содержать два метода: `tick()` и `tock()`. Метод `tick()` выводит слово "Tick", а метод `tock()` — слово "Tock". При запуске программы, имитирующей часы, создаются два потока: в одном из них вызывается метод `tick()`, а в другом — метод `tock()`. В результате взаимодействия двух потоков

на экран будет выводиться набор повторяющихся сообщений "Tick Tock", т.е. после слова "Tick", обозначающего один такт, должно следовать слово "Tock", обозначающее другой такт часов.

// Применение методов **wait()** и **notify()** для имитации часов.

```
class TickTock {

    String state; // содержит сведения о состоянии часов

    synchronized void tick(boolean running) {
        if(!running) { // остановить часы
            state = "ticked";
            notify(); // уведомить ожидающие потоки
            return;
        }

        System.out.print("Tick ");

        state = "ticked"; // установить текущее состояние после такта "тик"
        notify(); // Метод tick() уведомляет метод tock()
                   // о возможности продолжить выполнение.
        try {
            while(!state.equals("tocked"))
                wait();// Метод tick() ожидает завершения метода tock().
        }
        catch(InterruptedException exc) {
            System.out.println("Thread interrupted.");
        }
    }

    synchronized void tock(boolean running) {
        if(!running) { // остановить часы
            state = "tocked";
            notify(); // уведомить ожидающие потоки
            return;
        }

        System.out.println("Tock");

        state = "tocked"; // установить текущее состояние после такта "так"
        notify(); // Метод tock() уведомляет метод tick()
                   // о возможности продолжить выполнение.
        try {
            while(!state.equals("ticked"))
                wait(); // Метод tock() ожидает завершения метода tick().
        }
        catch(InterruptedException exc) {
            System.out.println("Thread interrupted.");
        }
    }
}

class MyThread implements Runnable {
```

```

Thread thrd;
TickTock ttOb;

// построить новый поток
MyThread(String name, TickTock tt) {
    thrd = new Thread(this, name);
    ttOb = tt;
    thrd.start(); // начать поток
}

// начать исполнение нового потока
public void run() {
    if(thrd.getName().compareTo("Tick") == 0) {
        for(int i=0; i<5; i++) ttOb.tick(true);
        ttOb.tick(false);
    }
    else {
        for(int i=0; i<5; i++) ttOb.tock(true);
        ttOb.tock(false);
    }
}

}

class ThreadCom {
    public static void main(String args[]) {
        TickTock tt = new TickTock();
        MyThread mt1 = new MyThread("Tick", tt);
        MyThread mt2 = new MyThread("Tock", tt);

        try {
            mt1.thrd.join();
            mt2.thrd.join();
        } catch (InterruptedException exc) {
            System.out.println("Main thread interrupted.");
        }
    }
}

```

В результате выполнения этой программы на экране появляются следующие сообщения:

```

Tick Tock
Tick Tock
Tick Tock
Tick Tock
Tick Tock

```

Рассмотрим более подробно исходный код программы, имитирующей работу часов. В ее основу положен класс `TickTock`. В нем содержатся два метода `tick()` и `tock()`, которые взаимодействуют друг с другом. Это взаимодействие организовано таким образом, чтобы за словом "Tick" всегда следовало слово "Tock", затем слово "Tick" и т.д. Обратите внимание на переменную `state`. В процессе работы имитатора часов в данной переменной хранится строка "ticked" или "tocked", определяющая текущее состоя-

ние часов после такта “тик” или “так” соответственно. В методе `main()` создается объект `tt` типа `TickTock`, используемый для запуска двух потоков на исполнение.

Потоки строятся на основе объектов типа `MyThread`. Конструктору `MyThread()` передаются два параметра. Первый из них задает имя потока (в данном случае — “Tick” или “Tock”), а второй — ссылку на объект типа `TickTock` (в данном случае — объект `tt`). В методе `run()` из класса `MyThread` вызывается метод `tick()`, если поток называется “Tick”, или же метод `tock()`, если поток называется “Tock”. Каждый из этих методов вызывается пять раз с параметром, принимающим логическое значение `true`. Работа имитатора часов продолжается до тех пор, пока методу передается параметр с логическим значением `true`. Последний вызов каждого из методов с параметром, принимающим логическое значение `false`, останавливает имитатор работы часов.

Самая важная часть программы находится в теле методов `tick()` и `tock()` из класса `TickTock`. Начнем с метода `tick()`. Для удобства анализа ниже представлен исходный код этого метода.

```
synchronized void tick(boolean running) {
    if(!running) { // остановить часы
        state = "ticked";
        notify(); // уведомить ожидающие потоки
        return;
    }

    System.out.print("Tick ");

    state = "ticked"; // установить текущее состояние после такта "тик"
    notify(); // уведомить метод tock() о возможности продолжить выполнение
    try {
        while(!state.equals("tocked"))
            wait(); // ожидать завершения метода tock()
    }
    catch(InterruptedException exc) {
        System.out.println("Thread interrupted.");
    }
}
```

Прежде всего обратите внимание на то, что в объявлении метода `tick()` присутствует ключевое слово `synchronized`, указываемое в качестве модификатора доступа. Как пояснялось ранее, действие методов `wait()` и `notify()` распространяется только на синхронизированные методы. В начале метода `tick()` проверяется значение параметра `running`. Этот параметр служит для корректного завершения программы, имитирующей работу часов. Если он принимает логическое значение `false`, имитатор работы часов должен быть остановлен. Если же параметр `running` принимает логическое значение `true`, а переменная `state` — значение “ticked”, вызывается метод `notify()`, разрешающий ожидающему потоку возобновить свое исполнение. Мы еще вернемся к этому вопросу несколько ниже.

По ходу работы имитируемых часов в методе `tick()` выводится слово “Tick”, переменная `state` принимает значение “ticked”, а затем вызывается метод `notify()`. Вызов метода `notify()` возобновляет исполнение ожидающего потока. Далее в цикле `while` вызывается метод `wait()`. В итоге выполнение метода `tick()` будет приостановлено до тех пор, пока другой поток не вызовет метод `notify()`. Таким образом, очередной шаг цикла не будет выполнен до тех пор, пока другой поток не вызовет метод

`notify()` для того же самого объекта. Поэтому когда вызывается метод `tick()`, на экран выводится слово "Tick" и другой поток получает возможность продолжить свое исполнение, а затем выполнение этого метода приостанавливается.

В том цикле `while`, в котором вызывается метод `wait()`, проверяется значение переменной `state`. Значение "ticked", означающее завершение цикла, будет установлено только после выполнения метода `tock()`. Этот цикл предотвращает продолжение исполнения потока в результате ложной активизации. Если по окончании ожидания в переменной `state` не будет присутствовать значение "ticked", значит, имела место ложная активизация, и метод `wait()` будет вызван снова.

Метод `tock()` является почти точной копией метода `tick()`. Его отличие состоит лишь в том, что он выводит на экран слово "Tock" и присваивает переменной `state` значение "ticked". Следовательно, когда метод `tock()` вызывается, он выводит на экран слово "Tock", вызывает метод `notify()`, а затем переходит в состояние ожидания. Если проанализировать работу сразу двух потоков, то станет ясно, что за вызовом метода `tick()` тотчас следует вызов метода `tock()`, после чего снова вызывается метод `tick()`, и т.д. В итоге оба метода синхронизируют друг друга.

При остановке имитатора работы часов вызывается метод `notify()`. Это нужно для того, чтобы возобновить исполнение ждущего потока. Как упоминалось выше, в обоих методах, `tick()` и `tock()`, после вывода сообщения на экран вызывается метод `wait()`. В результате при остановке имитатора работы часов один из потоков обязательно будет находиться в состоянии ожидания. Следовательно, последний вызов метода `notify()` необходим. В качестве эксперимента попробуйте удалить вызов метода `notify()` и посмотрите, что при этом произойдет. Вы увидите, что программа зависнет, и вам придется завершить ее нажатием комбинации клавиш <Ctrl+C>. Дело в том, что когда метод `tock()` в последний раз получает управление, он вызывает метод `wait()`, после чего не происходит вызов метода `notify()`, позволяющего завершиться методу `tock()`. В итоге метод `tock()` остается в состоянии бесконечного ожидания.

Если у вас еще остаются сомнения по поводу того, что методы `wait()` и `notify()` необходимы для организации нормального выполнения программы, имитирующей работу часов, замените в ее исходном коде класс `TickTock` приведенным ниже его вариантом. Он отличается тем, что в нем удалены вызовы методов `wait()` и `notify()`.

// В этой версии вызовы методов `wait()` и `notify()` отсутствуют.

```
class TickTock {

    String state; // содержит сведения о состоянии часов

    synchronized void tick(boolean running) {
        if(!running) { // остановить часы
            state = "ticked";
            return;
        }

        System.out.print("Tick ");

        state = "ticked"; // установить текущее состояние после такта "тик"
    }

    synchronized void tock(boolean running) {
        if(!running) { // остановить часы
```

```

        state = "tocked";
        return;
    }

    System.out.println("Tock");

    state = "tocked"; // установить текущее состояние после такта "так"
}
}

```

Теперь программа выводит на экран следующие сообщения:

```

Tick Tick Tick Tick Tock
Tock
Tock
Tock
Tock

```

Это происходит потому, что методы `tick()` и `tock()` не взаимодействуют друг с другом.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Неверное поведение многопоточных программ называется *взаимоблокировкой* и *состоянием гонки*. Что это такое и как избежать подобного поведения?

ОТВЕТ. Взаимоблокировка возникает в том случае, если один поток ожидает завершения некоторых действий другим потоком, а другой — того же самого от первого. В итоге оба потока оказываются в состоянии ожидания, а их исполнение не возобновляется. Это можно сравнить с поведением двух слишком вежливых людей, каждый из которых ни за что не соглашается пройти в дверь раньше другого.

На первый взгляд, предотвратить взаимоблокировку не трудно, но на самом деле это совсем не так. Например, взаимоблокировка может возникнуть косвенным образом. Причины ее не всегда понятны, поскольку между потоками нередко организуется довольно сложное взаимодействие. Единственный способ предотвратить взаимоблокировку — аккуратно программировать и тщательно тестировать создаваемый исходный код. Если многопоточная программа время от времени зависает, то, скорее всего, имеет место взаимоблокировка.

Состояние гонки возникает в том случае, когда два потока или более пытаются одновременно получить доступ к общему ресурсу без должной синхронизации. Так, в одном потоке может сохраняться значение в переменной, а в другом — инкрементировать текущее значение этой же переменной. В отсутствие синхронизации конечный результат будет зависеть от того, в каком именно порядке выполняются потоки: инкрементируется ли значение переменной во втором потоке или же оно сохраняется в первом. О подобной ситуации говорят, что потоки «гонятся друг за другом», причем конечный результат зависит от того, какой из потоков завершится первым. Возникающее состояние гонок, как и взаимоблокировку, непросто обнаружить. Поэтому его лучше предотвратить, синхронизируя должным образом доступ к общим ресурсам на стадии программирования.

Приостановка, возобновление и остановка потоков

Иногда оказывается полезно приостановить или даже полностью прекратить исполнение потока. Допустим, отдельный поток используется для отображения времени. Если пользователю не нужны часы на экране, то отображающий их поток можно приостановить. Независимо от причин, по которым требуется временная остановка потока, сделать это нетрудно, как, впрочем, и возобновить исполнение потока.

Механизмы приостановки, возобновления и остановки потоков менялись в разных версиях Java. До появления версии Java 2 для этих целей применялись методы `suspend()`, `resume()` и `stop()`, определенные в классе `Thread`. Ниже приведены общие формы их объявления.

```
final void resume()
final void suspend()
final void stop()
```

На первый взгляд кажется, что упомянутые выше методы удобны для управления потоками, но пользоваться ими все же не рекомендуется по следующим причинам. При выполнении метода `suspend()` иногда возникают серьезные осложнения, приводящие к взаимоблокировке. Метод `resume()` сам по себе безопасен, но применяется только в сочетании с методом `suspend()`. Что же касается метода `stop()` из класса `Thread`, то и он не рекомендуется к применению, начиная с версии Java 2, поскольку может вызывать порой серьезные осложнения в работе многопоточных программ.

Если методы `suspend()`, `resume()` и `stop()` нельзя использовать для управления потоками, то может показаться, что приостановить, возобновить и остановить поток вообще нельзя. Но это, к счастью, не так. Поток следует разрабатывать таким образом, чтобы в методе `run()` периодически осуществлялась проверка, следует ли приостановить, возобновить или остановить поток. Обычно для этой цели используются две флаговые переменные: одна — для приостановки и возобновления потока, другая — для остановки потока. Если флаговая переменная, управляющая приостановкой потока, установлена в состояние исполнения, то метод `run()` должен обеспечить продолжение исполнения потока. Если же эта флаговая переменная находится в состоянии приостановки, в работе потока должна произойти пауза. А если переменная, управляющая остановкой потока, находится в состоянии остановки, исполнение потока должно прекратиться.

Следующий пример программы демонстрирует один из способов реализации собственных версий методов `suspend()`, `resume()` и `stop()`.

```
// Приостановка, возобновление и остановка потока.

class MyThread implements Runnable {
    Thread thrd;

    // Если эта переменная принимает логическое значение
    // true, исполнение потока приостанавливается.
    volatile boolean suspended;
    // Если эта переменная принимает логическое значение
    // true, исполнение потока прекращается.
    volatile boolean stopped;
```

```

MyThread(String name) {
    thrd = new Thread(this, name);
    suspended = false;
    stopped = false;
    thrd.start();
}

// Точка входа в поток
public void run() {
    System.out.println(thrd.getName() + " starting.");
    try {
        for(int i = 1; i < 1000; i++) {
            System.out.print(i + " ");
            if((i%10)==0) {
                System.out.println();
                Thread.sleep(250);
            }

            // Для проверки условий приостановки и остановки потока
            // используется следующий синхронизированный блок.
            synchronized(this) {
                while(suspended) {
                    wait();
                }
                if(stopped) break;
            }
        }
    } catch (InterruptedException exc) {
        System.out.println(thrd.getName() + " interrupted.");
    }
    System.out.println(thrd.getName() + " exiting.");
}

// остановить поток
synchronized void mystop() {
    stopped = true;

    // Следующие операторы обеспечивают полную
    // остановку приостановленного потока.
    suspended = false;
    notify();
}

// приостановить поток
synchronized void mysuspend() {
    suspended = true;
}

// возобновить поток
synchronized void myresume() {
    suspended = false;
    notify();
}

```

```

}

class Suspend {
    public static void main(String args[]) {
        MyThread obl = new MyThread("My Thread");

        try {
            Thread.sleep(1000); // позволить потоку obl начать исполнение

            obl.mysuspend();
            System.out.println("Suspending thread.");
            Thread.sleep(1000);

            obl.myresume();
            System.out.println("Resuming thread.");
            Thread.sleep(1000);

            obl.mysuspend();
            System.out.println("Suspending thread.");
            Thread.sleep(1000);

            obl.myresume();
            System.out.println("Resuming thread.");
            Thread.sleep(1000);

            obl.mysuspend();
            System.out.println("Stopping thread.");
            obl.mystop();
        } catch (InterruptedException e) {
            System.out.println("Main thread Interrupted");
        }

        // ожидать завершения потока
        try {
            obl.thrd.join();
        } catch (InterruptedException e) {
            System.out.println("Main thread Interrupted");
        }

        System.out.println("Main thread exiting.");
    }
}

```

Ниже приведен результат выполнения данной программы.

```

My Thread starting.
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
Suspending thread.
Resuming thread.
41 42 43 44 45 46 47 48 49 50
51 52 53 54 55 56 57 58 59 60

```

```

61 62 63 64 65 66 67 68 69 70
71 72 73 74 75 76 77 78 79 80
Suspending thread.
Resuming thread.
81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100
101 102 103 104 105 106 107 108 109 110
111 112 113 114 115 116 117 118 119 120
Stopping thread.
My Thread exiting.
Main thread exiting.

```

Эта программа работает следующим образом. В классе потока `MyThread` определены две логические переменные, `suspended` и `stopped`, управляющие временной и полной остановкой потока. В конструкторе этого класса обеим переменным присваивается логическое значение `false`. Метод `run()` содержит синхронизированный блок, в котором проверяется состояние переменной `suspended`. Если эта переменная принимает логическое значение `true`, вызывается метод `wait()`, приостанавливающий исполнение потока. Логическое значение `true` присваивается переменной `suspended` в методе `mysuspend()`, и поэтому данный метод следует вызвать для приостановки потока. Для возобновления потока служит метод `myresume()`, в котором переменной `suspended` присваивается логическое значение `false` и вызывается метод `notify()`.

Для остановки потока следует вызвать метод `mystop()`, в котором переменной `stopped` присваивается логическое значение `true`. Кроме того, в методе `mystop()` переменной `suspended` присваивается логическое значение `false` и вызывается метод `notify()`. Это необходимо для прекращения работы потока, исполнение которого ранее было приостановлено.

В отношении рассматриваемой здесь программы нужно сделать еще одно, последнее замечание. В объявлении переменных `suspended` и `stopped` используется ключевое слово `volatile`. Этот модификатор подробно описывается в главе 14, а до тех пор вкратце поясним его назначение. Он сообщает компилятору о том, что значение переменной может быть неожиданно изменено другими частями программы, в том числе и другим потоком.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Многопоточное программирование, по-видимому, является эффективным средством повышения производительности программ. Какие рекомендации можно дать по его эффективному применению?

ОТВЕТ. Самое главное для эффективного многопоточного программирования — мыслить категориями параллельного, а не последовательного исполнения кода. Так, если в одной программе имеются две подсистемы, которые могут работать параллельно, их следует организовать в отдельные потоки. Но делать это следует очень внимательно и аккуратно, поскольку если создать слишком много потоков, то тем самым можно значительно снизить, а не повысить производительность программы. Следует также иметь в виду дополнительные издержки, связанные с переключением контекста. Так, если создать слишком много потоков, то на смену контекста уйдет больше времени ЦП, чем на выполнение самой программы!

Пример для опробования 11.2.**Применение основного потока****Исходный файл:** UseMain.java

В каждой программе на Java присутствует хотя бы один поток, называемый *основным*. Этот поток получает управление автоматически при запуске программы на выполнение. В этом проекте будет продемонстрировано, что основным потоком можно управлять таким образом же, как и любым другим.

Последовательность действий

1. Создайте файл UseMain.java.
2. Для доступа к основному потоку нужно получить ссылающийся на него объект типа Thread. Для этого следует вызвать метод `currentThread()`, являющийся статическим членом класса Thread. Ниже приведено объявление этого метода.

```
static Thread currentThread()
```

Метод `currentThread()` возвращает ссылку на тот поток, из которого он вызывается. Так, если вызвать метод `currentThread()` из основного потока, можно получить ссылку на этот поток. А имея ссылку на основной поток, можно управлять им.

3. Введите в файл UseMain.java приведенный ниже исходный код программы. В процессе ее выполнения сначала извлекается ссылка на основной поток, затем определяется и устанавливается имя и приоритет потока.

```
/*
```

```
    Пример для опробования 11.2.
```

```
    Управление основным потоком.
```

```
*/
```

```
class UseMain {
    public static void main(String args[]) {
        Thread thrd;

        // получить основной поток
        thrd = Thread.currentThread();

        // отобразить имя основного потока
        System.out.println("Main thread is called: " +
                           thrd.getName());

        // отобразить приоритет основного потока
        System.out.println("Priority: " +
                           thrd.getPriority());

        System.out.println();

        // установить имя и приоритет основного потока
        System.out.println("Setting name and priority.\n");
        thrd.setName("Thread #1");
        thrd.setPriority(Thread.NORM_PRIORITY+3);

        System.out.println("Main thread is now called: " +
                           thrd.getName());
    }
}
```

```

        System.out.println("Priority is now: " +
                           thrd.getPriority());
    }
}

```

4. Ниже приведен результат выполнения данной программы.

```

Main thread is called: main
Priority: 5

```

```

Setting name and priority.

```

```

Main thread is now called: Thread #1
Priority is now: 8

```

5. Выполняя операции над основным потоком, необходимо соблюдать осторожность. Так, если добавить в конце метода `main()` приведенный ниже код, программа никогда не завершится, потому что будет ожидать завершения основного потока!

```

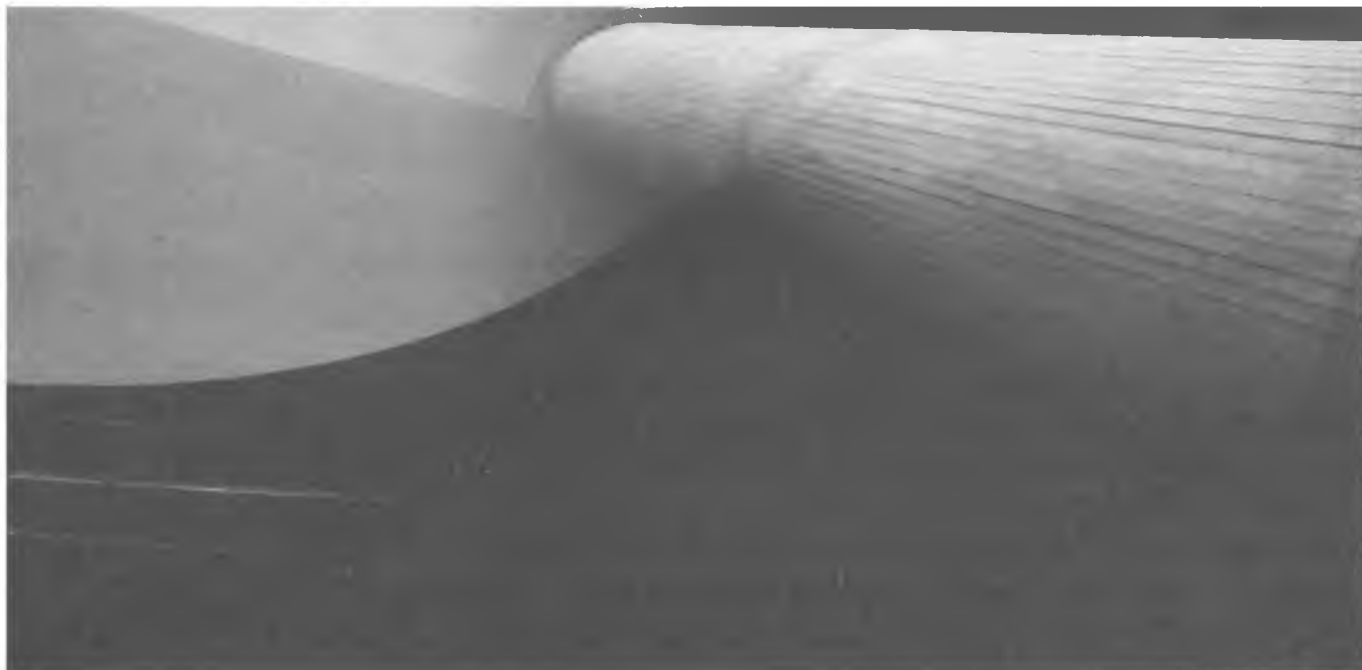
try {
    thrd.join();
} catch (InterruptedException exc) {
    System.out.println("Interrupted");
}

```

Упражнение для самопроверки по материалу главы 11

1. Каким образом имеющиеся в Java средства многопоточного программирования позволяют писать более эффективные программы?
2. Для поддержки многопоточного программирования в Java предусмотрен класс _____ и интерфейс _____.
3. В каких случаях следует отдать предпочтение расширению класса `Thread` над реализацией интерфейса `Runnable`?
4. Покажите, как с помощью метода `join()` можно организовать ожидание завершения потокового объекта `MyThrd`.
5. Покажите, как установить приоритет потока `MyThrd` на три уровня выше нормального приоритета.
6. Что произойдет, если в объявлении метода указать ключевое слово `synchronized`?
7. Методы `wait()` и `notify()` служат для _____.
8. Внесите в класс `TickTock` изменения для организации настоящего отчета времени. Первую половину секунды должен занимать вывод на экран слова "Tick", а вторую — вывод слова "Tock". Таким образом, сообщение "Tick-Tock" должно соответствовать одной секунде отсчитываемого времени. (Время переключения контекстов можно не учитывать.)
9. Почему в новых программах на Java не следует применять методы `suspend()`, `resume()` и `stop()`?
10. С помощью какого метода из класса `Thread` можно получить имя потока?

- 11.** Какое значение возвращает метод `isAlive()`?
- 12.** Попробуйте самостоятельно реализовать средства синхронизации в классе `Queue`, разработанном в предыдущих главах. В результате доработки класс должен действовать правильно, когда он используется для многопоточной обработки.



Глава 12

**Перечисления,
автоупаковка,
статический импорт
и аннотации**

Основные навыки и понятия

- Представление о перечислениях
- Применение свойств перечислений, основанных на классах
- Применение методов `values()` и `valueOf()` к перечислениям
- Создание перечислений с конструкторами, переменными экземпляра и методами
- Применение методов `ordinal()` и `compareTo()`, наследуемых перечислениями от класса `Enum`
- Использование оболочек типов Java
- Основные положения об автоупаковке и автораспаковке
- Применение автоупаковки в методах
- Употребление автоупаковки в выражениях
- Применение статического импорта
- Основные положения об аннотациях

В этой главе рассматриваются четыре относительно новые языковые средства Java: перечисления, автоупаковка, статический импорт и аннотации. И хотя ни одно из них не вошло в первоначальное описание Java, каждое из них расширяет возможности и область применения этого языка программирования. Так, перечисления и автоупаковка удовлетворяют давно назревшим потребностям, статический импорт упрощает применение статических членов класса, тогда как аннотации расширяют виды информации, которую можно встраивать в исходный файл. А сообщая все эти средства обеспечивают более совершенное решение насущных задач программирования. В этой главе рассматриваются также оболочки типов Java.

Перечисления

Несмотря на то что перечисления применяются во многих языках программирования, в первоначальном описании Java они не поддерживались. Это, в частности, объясняется тем, что перечисление является скорее удобным, чем обязательным языковым средством. Но со временем программирующие на Java все чаще стали ощущать потребность в перечислениях, потому что с их помощью они могли реализовать структурно изысканные решения многих задач программирования. И эта потребность была учтена в версии JDK 5, начиная с которой перечисления внедрены в Java.

В простейшем виде *перечисление* представляет собой список именованных констант, определяющих новый тип данных. Объект перечислимого типа может принимать лишь значения, содержащиеся в списке. Таким образом, перечисления предоставляют возможность точно определить новый тип данных, имеющий строго фиксированный ряд допустимых значений.

В повседневной жизни перечисления встречаются довольно часто. Например, к ним можно отнести ряд монет, имеющих хождение в стране. А месяцы года и дни недели перечисляются по названиям.

С точки зрения программирования перечисления оказываются удобными в тех случаях, когда требуется определить ряд значений, обозначающих совокупность элементов. Например, с помощью перечисления можно представить набор кодов состояния (успешное завершение, ошибка, необходимость повторной попытки). Конечно, такие значения можно определить и с помощью констант типа `final`, но перечисления обеспечивают более структурированный подход к решению подобной задачи.

Основные положения о перечислениях

Для создания перечисления служит ключевое слово `enum`. Ниже приведен пример простого перечисления разных видов транспортных средств.

```
// Перечисление видов транспортных средств.
enum Transport {
    CAR, TRUCK, AIRPLANE, TRAIN, BOAT
}
```

Идентификаторы `CAR`, `TRUCK` и так далее называются *константами перечислимого типа*. Каждый из них автоматически неявно объявляется как открытый (`public`), статический (`static`) член перечисления `Transport`. Тип этих констант соответствует типу перечисления (в данном случае — `Transport`). В терминологии Java подобные константы называются *самотипизированными* (приставка “само” означает, что в качестве типа константы принимается тип перечисления).

Определив перечисление, можно создать переменную данного типа. Но, несмотря на то, что перечисление определяется как тип класса, получить экземпляр объекта типа `enum` с помощью оператора `new` нельзя. Переменная перечислимого типа создается подобно переменной простого типа. Например, для объявления переменной `tp` упомянутого выше перечислимого типа `Transport` служит следующее выражение:

```
Transport tp;
```

Переменная `tp` относится к типу `Transport`, и поэтому ей можно присваивать только те значения, которые определены в данном перечислении. Например, в следующей строке кода переменной `tp` присваивается значение `AIRPLANE`:

```
tp = Transport.AIRPLANE;
```

Обратите внимание на то, что идентификатор `AIRPLANE` полностью определяется как относящийся к типу `Transport`.

Для проверки равенства констант перечислимого типа служит оператор сравнения `==`. Например, в приведенной ниже строке кода содержимое переменной `tp` сравнивается с константой `TRAIN`.

```
if(tp == Transport.TRAIN) // ...
```

Перечислимые значения можно также использовать в операторе `switch`. Очевидно, что в ветвях `case` этого оператора могут присутствовать только константы того же самого перечислимого типа, что и в выражении `switch`. Например, следующий фрагмент кода составлен правильно:

```
// Применение перечисления для управления оператором switch.
switch(tp) {
    case CAR:
```

```
// ...
case TRUCK:
// ...
```

Как видите, в ветвях `case` используются константы без полного определения имени типа. Например, вместо `Transport.TRUCK` указано просто `TRUCK`. Это допустимо потому, что перечислимый тип в выражении `switch` неявно определяет тип констант в ветвях `case`. Более того, если попытаться указать тип константы явно, при компиляции возникнет ошибка.

При отображении константы перечислимого типа, например, с помощью метода `println()`, выводится ее имя. Так, в результате выполнения следующего оператора отобразится имя `BOAT`:

```
System.out.println(Transport.BOAT);
```

Ниже приведен пример программы, демонстрирующий все особенности применения перечисления `Transport`.

```
// Особенности применения перечисления Transport.
```

```
// Объявление перечисления.
```

```
enum Transport {
    CAR, TRUCK, AIRPLANE, TRAIN, BOAT
}
```

```
class EnumDemo {
    public static void main(String args[])
    {
```

```
        // Объявление ссылки на перечисление Transport.
        Transport tp;
```

```
        // Присваивание переменной tp константы AIRPLANE.
        tp = Transport.AIRPLANE;
```

```
        // вывести перечислимое значение
        System.out.println("Value of tp: " + tp);
        System.out.println();
```

```
        tp = Transport.TRAIN;
```

```
        // Проверка равенства двух объектов типа Transport.
```

```
        if(tp == Transport.TRAIN) // сравнить два перечислимых значения
            System.out.println("tp contains TRAIN.\n");
```

```
        // Использование перечисления для управления оператором switch.
```

```
        switch(tp) {
            case CAR:
                System.out.println("A car carries people.");
                break;
            case TRUCK:
                System.out.println("A truck carries freight.");
                break;
            case AIRPLANE:
                System.out.println("An airplane flies.");
```

```
        break;
    case TRAIN:
        System.out.println("A train runs on rails.");
        break;
    case BOAT:
        System.out.println("A boat sails on water.");
        break;
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

Value of tp: AIRPLANE

tp contains TRAIN.

A train runs on rails.

Прежде чем переходить к рассмотрению следующей темы, необходимо сделать одно замечание. Имена констант в перечислении `Transport` указываются прописными буквами (например, одна из них называется `CAR`, а не `car`). Но это требование не является обязательным. Не существует никаких ограничений на использование регистра символов в именах констант перечислимого типа. Но поскольку константы перечислимого типа, как правило, предназначены для замены переменных, объявленных как `final`, имена которых по традиции обозначаются прописными буквами, то и имена констант перечислимого типа, как правило, обозначаются прописными буквами. И хотя на этот счет имеются разные точки зрения, в примерах программ, представленных в этой книге, для констант перечислимого типа будут выбираться имена, обозначаемые прописными буквами.

Перечисления в Java относятся к типам классов

Приведенные выше примеры демонстрируют создание и использование перечислений, но они не дают полного представления обо всех их возможностях. В отличие от других языков программирования *перечисления* в Java реализованы как *типы классов*. И хотя создать экземпляр объекта типа `enum` с помощью оператора `new` нельзя, во всем остальном перечисления ничем не отличаются от классов. Такой способ реализации перечислений наделяет их богатыми возможностями, принципиально недостижимыми в других языках. Это, в частности, позволяет определять конструкторы перечислений, добавлять в них переменные экземпляра, методы и даже создавать перечисления, реализующие интерфейсы.

Методы `values()` и `valueOf()`

У всех перечислений имеются два предопределенных метода: `values()` и `valueOf()`. Ниже приведены общие формы их объявления.

```
public static перечислимый_тип[] values()
public static перечислимый_тип valueOf(String str)
```

Метод `values()` возвращает массив, содержащий константы перечислимого типа, а метод `valueOf()` — константу того же типа, значение которой соответствует символической строке `str`, передаваемой этому методу в качестве параметра. В обоих слу-

чаях *перечислимый* тип обозначает тип используемого перечисления. Так, если воспользоваться рассмотренным выше перечислением `Transport`, то при вызове метода `Transport.valueOf("TRAIN")` будет возвращено значение `TRAIN` типа `Transport`. Ниже приведен пример программы, демонстрирующий применение методов `values()` и `valueOf()`.

```
// Применение встроенных в перечисление методов.

// Перечисление разных видов транспорта.
enum Transport {
    CAR, TRUCK, AIRPLANE, TRAIN, BOAT
}

class EnumDemo2 {
    public static void main(String args[])
    {
        Transport tp;

        System.out.println("Here are all Transport constants");

        // воспользоваться методом values()
        // Получение массива констант типа Transport.
        Transport allTransports[] = Transport.values();
        for(Transport t : allTransports)
            System.out.println(t);

        System.out.println();

        // воспользоваться методом valueOf()
        // Получение константы под названием AIRPLANE.
        tp = Transport.valueOf("AIRPLANE");
        System.out.println("tp contains " + tp);
    }
}
```

Выполнение этой программы дает следующий результат:

```
Here are all Transport constants
CAR
TRUCK
AIRPLANE
TRAIN
BOAT

tp contains AIRPLANE
```

Обратите внимание на то, что в данном примере программы для перебора массива констант, полученного с помощью метода `values()`, используется вариант `for-each` цикла `for`. Ради большей наглядности данного примера в его коде создается переменная `allTransports`, которой присваивается ссылка на массив констант перечислимого типа. Но делать это совсем не обязательно, а цикл `for` можно написать так, как показано ниже. В этом случае необходимость в дополнительной переменной `allTransports` отпадает.

```
for(Transport t : Transport.values())  
    System.out.println(t);
```

Обратите также внимание на то, что значение, соответствующее имени AIRPLANE, было получено в результате вызова метода `valueOf()`:

```
tp = Transport.valueOf("AIRPLANE");
```

Как пояснялось выше, метод `valueOf()` возвращает значение перечислимого типа, связанное с именем константы, которое передается в виде символьной строки при вызове этого метода.

Конструкторы, методы, переменные экземпляра и перечисления

Следует иметь в виду, что каждая константа перечислимого типа является объектом этого же типа, а следовательно, в перечислении можно определить конструкторы, ввести методы и объявить переменные экземпляра. Если определить для объекта перечислимого типа `enum` конструктор, он будет вызываться при создании каждой константы этого типа. А каждая константа перечислимого типа позволяет вызвать любой метод, определенный в перечислении. И у каждой константы перечислимого типа имеется собственная копия любой переменной экземпляра, определенной в перечислении. Ниже приведен пример с новой версией перечисления `Transport`, демонстрирующий применение конструктора, переменной экземпляра и метода. Благодаря им появляется возможность определить обычную скорость передвижения различных транспортных средств.

```
// Применение конструктора, переменной экземпляра и  
// метода в перечислении.  
enum Transport {  
    // Обратите внимание на инициализирующие значения констант.  
    CAR(65), TRUCK(55), AIRPLANE(600), TRAIN(70), BOAT(22);  
  
    // Объявление переменной экземпляра.  
    private int speed; // обычная скорость каждого транспортного средства  
  
    // Объявление конструктора.  
    Transport(int s) { speed = s; }  
  
    // Определение метода.  
    int getSpeed() { return speed; }  
}  
  
class EnumDemo3 {  
    public static void main(String args[])  
    {  
        Transport tp;  
  
        // отобразить скорость самолета  
        // Скорость определяется с помощью метода getSpeed().  
        System.out.println("Typical speed for an airplane is " +  
            Transport.AIRPLANE.getSpeed() +  
            " miles per hour.\n");  
    }  
}
```



```
// отобразить все виды транспорта и скорости их передвижения
System.out.println("All Transport speeds: ");
for(Transport t : Transport.values())
    System.out.println(t + " typical speed is " +
        t.getSpeed() +
        " miles per hour.");
}
```

Выполнение этой программы дает следующий результат:

Typical speed for an airplane is 600 miles per hour.

All Transport speeds:

CAR typical speed is 65 miles per hour.

TRUCK typical speed is 55 miles per hour.

AIRPLANE typical speed is 600 miles per hour.

TRAIN typical speed is 70 miles per hour.

BOAT typical speed is 22 miles per hour.

В эту версию перечисления `Transport` внесен ряд дополнений. Во-первых, появилась переменная экземпляра `speed`, используемая для хранения скорости передвижения каждого вида транспортных средств. Во-вторых, в перечисление `Transport` добавлен конструктор, которому передается значение скорости. И в-третьих, в это перечисление добавлен метод `getSpeed()`, возвращающий значение переменной `speed`, т.е. скорость передвижения конкретного транспортного средства.

Когда переменная `tp` объявляется в методе `main()`, конструктор `Transport()` автоматически вызывается для каждой указанной константы. Аргументы, передаваемые конструктору, указываются в скобках после имени константы, как показано ниже.

```
CAR(65), TRUCK(55), AIRPLANE(600), TRAIN(70), BOAT(22);
```

Числовые значения, передаваемые конструктору `Transport()` в качестве параметра `s`, присваиваются переменной `speed`. Обратите также внимание на то, что список констант перечислимого типа завершается точкой с запятой. Последней в этом списке указана константа `BOAT`. Точка с запятой требуется в том случае, если, помимо констант, в перечислении присутствуют и другие члены.

У каждой константы перечислимого типа имеется собственная копия переменной `speed`, что дает возможность выяснить скорость передвижения конкретного транспортного средства, вызвав метод `getSpeed()`. Например, в методе `main()` скорость самолета определяется при следующем вызове:

```
Transport.AIRPLANE.getSpeed()
```

Скорость каждого транспортного средства определяется путем перебора констант перечислимого типа в цикле `for`. А поскольку у каждой такой константы имеется своя копия переменной `speed`, то значения скорости, связанные с разными константами, отличаются друг от друга. Это довольно эффективный принцип организации перечислений, но он возможен только в том случае, если перечисления реализованы в виде классов, как это сделано в Java.

В предыдущем примере использовался только один конструктор, но перечисления, как и обычные классы, допускают любое число конструкторов.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Теперь, когда перечисления внедрены в Java, можно ли считать, что переменные типа `final` больше не нужны?

ОТВЕТ. Нет, нельзя. Перечисления оказываются удобными в тех случаях, когда приходится иметь дело со списками элементов, которые должны быть обозначены идентификаторами. А переменные типа `final` целесообразно применять для обозначения постоянных значений, например размеров массивов, которые многократно используются в программе. Таким образом, у каждого из этих языковых средств имеется своя область применения. Преимущество перечислений проявляется в тех случаях, когда переменные типа `final` не совсем удобны.

Два важных ограничения

На перечисления накладываются два ограничения. Во-первых, перечисление не может быть подклассом другого класса. И во-вторых, перечисление не может выступать в роли суперкласса. Иными словами, перечислимый тип `enum` нельзя расширять. Если бы это было не так, перечисления действовали бы как обычные классы. Основной же особенностью перечислений является создание констант в виде объектов того класса, в котором они были объявлены.

Наследование перечислений от класса `Enum`

Несмотря на то что при объявлении перечислимого типа `enum` нельзя указывать суперкласс, все перечисления автоматически наследуют переменные и методы от класса `java.lang.Enum`. В этом классе определен ряд методов, доступных для использования всеми перечислениями. И хотя большинство этих методов используются редко, тем не менее два из них иногда применяются в программах на Java. Это методы `ordinal()` и `compareTo()`.

Метод `ordinal()` принимает значение, обозначающее положение константы перечислимого типа в списке. Это значение принято называть *порядковым*. Ниже приведена общая форма объявления метода `ordinal()`.

```
final int ordinal()
```

Этот метод возвращает порядковое значение вызывающей константы. Отсчет порядковых значений начинается с нуля. Следовательно, в упоминавшемся выше перечислении `Transport` порядковое значение константы `CAR` равно нулю, константы `TRUCK` — 1, константы `AIRPLANE` — 2 и т.д.

Для сравнения порядковых значений двух констант в одном и том же перечислении можно воспользоваться методом `compareTo()`. Ниже приведена общая форма объявления этого метода.

```
final int compareTo(перечислимый_тип e)
```

Здесь в качестве параметра `e` задается константа, сравниваемая с вызывающей константой, а перед ней указывается *перечислимый_тип*, к которому эта константа относится. Следует иметь в виду, что вызывающая константа и константа `e` должны относиться к одному и тому же перечислимому типу. Так, если порядковое значение вызывающей константы оказывается меньше, чем у константы `e`, метод `compareTo()` возвращает отрицательное значение. Если же их порядковые значения совпадают, воз-

возвращается нулевое значение. И наконец, если порядковое значение вызывающей константы больше, чем у константы `e`, метод возвращает положительное значение.

Ниже приведен пример программы, демонстрирующий применение методов `ordinal()` и `compareTo()`.

```
// Применение методов ordinal() и compareTo().

// Перечисление разных видов транспорта.
enum Transport {
    CAR, TRUCK, AIRPLANE, TRAIN, BOAT
}

class EnumDemo4 {
    public static void main(String args[])
    {
        Transport tp, tp2, tp3;

        // получить все порядковые значения с помощью метода ordinal()
        System.out.println("Here are all Transport constants" +
                           " and their ordinal values: ");
        for(Transport t : Transport.values())
            // Получение порядковых значений констант.
            System.out.println(t + " " + t.ordinal());

        tp = Transport.AIRPLANE;
        tp2 = Transport.TRAIN;
        tp3 = Transport.AIRPLANE;

        System.out.println();

        // продемонстрировать применение метода compareTo()
        // Сравнение порядковых значений констант.
        if(tp.compareTo(tp2) < 0)
            System.out.println(tp + " comes before " + tp2);

        if(tp.compareTo(tp2) > 0)
            System.out.println(tp2 + " comes before " + tp);

        if(tp.compareTo(tp3) == 0)
            System.out.println(tp + " equals " + tp3);
    }
}
```

Результат выполнения данной программы выглядит следующим образом:

```
Here are all Transport constants and their ordinal values:
CAR 0
TRUCK 1
AIRPLANE 2
TRAIN 3
BOAT 4

AIRPLANE comes before TRAIN
AIRPLANE equals AIRPLANE
```

Пример для опробования 12.1. Автоматизированный светофор**Исходный файл:****TrafficLightDemo.java**

Перечисления оказываются полезными в тех случаях, когда в программе требуется набор констант, конкретные значения которых не важны, — достаточно, чтобы они отличались друг от друга. Необходимость в подобных наборах констант часто возникает при написании программ. Одним из характерных тому примеров служит поддержка устройств, которые могут находиться в нескольких фиксированных состояниях. Допустим, требуется программа, управляющая светофором, переключающимся в три состояния, обозначаемые зеленым, желтым и красным цветом. Необходимо также, чтобы программа могла определять текущий цвет и устанавливать светофор в исходное состояние. Таким образом, нужно как-либо представлять три возможных состояния светофора. И хотя для этой цели вполне допустимо применять целочисленные значения, например, 1, 2 и 3 или символьные строки "red" (красный), "green" (зеленый) и "yellow" (желтый), вместо них лучше всего воспользоваться перечислением. С помощью перечисления можно написать более эффективный и структурированный код, чем тот, в котором применяются символьные строки и целочисленные значения.

В этом проекте предстоит симитировать автоматизированный светофор. А по ходу дела будет продемонстрировано применение не только перечислений, но и многопоточной обработки и синхронизации потоков.

Последовательность действий

1. Создайте файл TrafficLightDemo.java.
2. Начните с создания перечисления TrafficLightColor, которое представляет три состояния светофора.

```
// Перечисление, представляющее состояния светофора.
enum TrafficLightColor {
    RED, GREEN, YELLOW
}
```

Каждая из констант в этом перечислении соответствует одному цвету светофора.

3. Далее начните определение класса TrafficLightSimulator так, как показано ниже. Этот класс инкапсулирует имитацию светофора.

```
// Имитация автоматизированного светофора.
class TrafficLightSimulator implements Runnable {
    private Thread thrd; // Поток для имитации светофора
    private TrafficLightColor tlc; // Текущий цвет светофора
    boolean stop = false; // Остановка имитации, если истинно
    boolean changed = false; // Переключение светофора, если истинно

    TrafficLightSimulator(TrafficLightColor init) {
        tlc = init;

        thrd = new Thread(this);
        thrd.start();
    }

    TrafficLightSimulator() {
        tlc = TrafficLightColor.RED;
    }
}
```

```

        thrd = new Thread(this);
        thrd.start();
    }

```

Как видите, класс `TrafficLightSimulator` реализует интерфейс `Runnable`. Благодаря этому каждое состояние светофора и его цвет регулируется в отдельном потоке. Как следует из приведенного выше исходного кода, в классе `TrafficLightSimulator` определены два конструктора. Первый из них служит для указания исходного цвета светофора. А второй принимает в качестве исходного красный цвет. Оба конструктора запускают на исполнение новый поток.

А теперь рассмотрим переменные экземпляра. Ссылка на поток, регулирующий состояние и цвет светофора, хранится в переменной `thrd`. А сведения о текущем цвете хранятся в переменной `tlc`. Переменная `stop` служит для остановки имитации автоматизированного светофора. Первоначально она принимает логическое значение `false`. Имитация светофора будет действовать до тех пор, пока эта переменная не примет логическое значение `true`. И наконец, переменная `changed` принимает логическое значение `true` при переключении светофора, когда его цвет меняется.

4. Введите приведенный ниже метод `run()`, начинающий имитацию автоматизированного светофора.

```

// Запуск имитации автоматизированного светофора.
public void run() {
    while(!stop) {

        try {
            switch(tlc) {
                case GREEN:
                    Thread.sleep(10000); // Зеленый на 10 секунд
                    break;
                case YELLOW:
                    Thread.sleep(2000); // Желтый на 2 секунды
                    break;
                case RED:
                    Thread.sleep(12000); // Красный на 12 секунд
                    break;
            }
        } catch (InterruptedException exc) {
            System.out.println(exc);
        }
        changeColor();
    }
}

```

Этот метод переключает цвета светофора по очереди. Сначала выполнение потока приостанавливается на заданный промежуток времени, который выбирается в зависимости от конкретного цвета светофора. Затем вызывается метод `changeColor()`, переключающий цвет светофора.

5. Введите приведенный ниже метод `changeColor()`, переключающий цвет светофора.

```

// Переключение цвета светофора.
synchronized void changeColor() {

    switch(tlc) {

```

```
        case RED:
            tlc = TrafficLightColor.GREEN;
            break;
        case YELLOW:
            tlc = TrafficLightColor.RED;
            break;
        case GREEN:
            tlc = TrafficLightColor.YELLOW;
    }

    changed = true;
    notify(); // уведомить о переключении цвета светофора
}
```

6. В операторе `switch` проверяются сведения о цвете светофора, хранящиеся в переменной `tlc`, после чего этой переменной присваивается другой цвет. Обратите внимание на то, что этот метод синхронизирован. Это необходимо потому, что он вызывает метод `notify()`, уведомляющий о смене цвета. (Напомним, что обратиться к методу `notify()` можно только из синхронизированного контекста.)

7. Далее введите метод `waitForChange()`, ожидающий переключения цвета светофора.

// Ожидание переключения цвета светофора.

```
synchronized void waitForChange() {
    try {
        while(!changed)
            wait(); // ожидать переключения цвета светофора
        changed = false;
    } catch (InterruptedException exc) {
        System.out.println(exc);
    }
}
```

Действие этого метода ограничивается вызовом метода `wait()`. Возврат из него не произойдет до тех пор, пока в методе `changeColor()` не будет вызван метод `notify()`. Следовательно, метод `waitForChange()` не завершится до переключения цвета светофора.

8. И наконец, введите метод `getColor()`, возвращающий текущий цвет светофора, а вслед за ним — метод `cancel()`, останавливающий имитацию светофора, присваивая переменной `stop` логическое значение `true`. Ниже приведен исходный код обоих методов.

```
// Возврат текущего цвета.
TrafficLightColor getColor() {
    return tlc;
}

// Прекращение имитации светофора.
void cancel() {
    stop = true;
}
```

9. Ниже приведен весь исходный код программы, имитирующей автоматизированный светофор с помощью перечисления.

```

// Пример для опробования 12.1.

// Имитация автоматизированного светофора с помощью
// перечисления, описывающего переключаемые цвета светофора.

// Перечисление, представляющее состояния светофора.
enum TrafficLightColor {
    RED, GREEN, YELLOW
}

// Имитация автоматизированного светофора.
class TrafficLightSimulator implements Runnable {
    private Thread thrd; // Поток для имитации светофора
    private TrafficLightColor tlc; // Текущий цвет светофора
    boolean stop = false; // Остановка имитации, если истинно
    boolean changed = false; // Переключение светофора, если истинно

    TrafficLightSimulator(TrafficLightColor init) {
        tlc = init;

        thrd = new Thread(this);
        thrd.start();
    }

    TrafficLightSimulator() {
        tlc = TrafficLightColor.RED;

        thrd = new Thread(this);
        thrd.start();
    }

    // Запуск имитации автоматизированного светофора.
    public void run() {
        while(!stop) {

            try {
                switch(tlc) {
                    case GREEN:
                        Thread.sleep(10000); // Зеленый на 10 секунд
                        break;
                    case YELLOW:
                        Thread.sleep(2000); // Желтый на 2 секунды
                        break;
                    case RED:
                        Thread.sleep(12000); // Красный на 12 секунд
                        break;
                }
            } catch (InterruptedException exc) {
                System.out.println(exc);
            }
            changeColor();
        }
    }
}

```

```

// Переключение цвета светофора.
synchronized void changeColor() {

    switch(tlc) {
        case RED:
            tlc = TrafficLightColor.GREEN;
            break;
        case YELLOW:
            tlc = TrafficLightColor.RED;
            break;
        case GREEN:
            tlc = TrafficLightColor.YELLOW;
    }

    changed = true;
    notify(); // уведомить о переключении цвета светофора
}

// Ожидание переключения цвета светофора.
synchronized void waitForChange() {
    try {
        while(!changed)
            wait(); // ожидать переключения цвета светофора
        changed = false;
    } catch(InterruptedException exc) {
        System.out.println(exc);
    }
}

// Возврат текущего цвета.
TrafficLightColor getColor() {
    return tlc;
}

// Прекращение имитации светофора.
void cancel() {
    stop = true;
}

class TrafficLightDemo {
    public static void main(String args[]) {
        TrafficLightSimulator tl =
            new TrafficLightSimulator(TrafficLightColor.GREEN);

        for(int i=0; i < 9; i++) {
            System.out.println(tl.getColor());
            tl.waitForChange();
        }

        tl.cancel();
    }
}

```


При выполнении этой программы на экран выводится приведенный ниже результат. Как видите, цвета светофора переключаются в требуемой очередности: зеленый, желтый, красный.

```
GREEN
YELLOW
RED
GREEN
YELLOW
RED
GREEN
YELLOW
RED
```

Обратите внимание на то, что перечисление позволяет сделать исходный код данной программы более структурированным. Светофор может находиться в одном из трех состояний, и для этой цели в перечислении предусмотрены только три константы. Благодаря этому исключается случайное переключение имитируемого светофора в недопустимое состояние.

10. Используя тот факт, что перечисления реализуются в виде классов, можете усовершенствовать рассмотренную здесь программу. Соответствующее задание будет предложено в упражнении для самопроверки по материалу этой главы в самом ее конце.

Автоупаковка

В версии JDK 5 были реализованы два очень важных языковых средства, недостаток которых долгое время ощущали программирующие на Java. Речь идет об *автоупаковке* и *автораспаковке*, существенно упрощающих и ускоряющих создание кода, в котором приходится преобразовывать простые типы данных в объекты, и наоборот. А поскольку такие потребности возникают в программах довольно часто, то появление автоупаковки и автораспаковки положительно сказалось на работе практически всех программирующих на Java. Как будет показано в главе 13, автоупаковка и автораспаковка способствовали практическому применению обобщений — еще одного языкового средства, реализованного в Java.

Автоупаковка и автораспаковка непосредственно связаны с оболочками типов и способами внедрения и извлечения значений из экземпляров оболочек. Поэтому рассмотрим сначала оболочки типов и способы упаковки и распаковки значений вручную.

Оболочки типов

Как вам должно быть уже известно, в Java предусмотрены простые типы данных, в том числе `int` и `double`. Простые типы позволяют добиться более высокой эффективности вычислений по сравнению с объектами. Но простые типы не являются частью иерархии объектов и не наследуют свойства и методы класса `Object`.

Несмотря на высокую эффективность простых типов, возникают такие ситуации, когда для представления данных желательно использовать объекты. Например, переменную простого типа нельзя передать методу по ссылке. Кроме того, многие стандартные структуры данных, реализованные в Java, предполагают работу с объектами, и поэтому в них нельзя хранить данные простых типов. Для преодоления затруднений, возникающих в подобных и во многих других ситуациях, в Java предусмотрены *оболочки*

типов — классы, инкапсулирующие простые типы данных. Классы оболочек типов упоминались в главе 10, а здесь они будут рассмотрены более подробно.

Оболочки типов реализуются в классах `Double`, `Float`, `Long`, `Integer`, `Short`, `Byte`, `Character` и `Boolean`, входящих в пакет `java.lang`. Эти классы предоставляют методы, позволяющие полностью интегрировать простые типы данных в иерархию объектов Java.

Чаще всего применяются оболочки типов, представляющие числовые типы данных: `Byte`, `Short`, `Integer`, `Long`, `Float` и `Double`. Все оболочки числовых типов данных являются производными от абстрактного класса `Number`. В классе `Number` определены методы, возвращающие значение объекта для каждого числового типа данных. Эти методы перечислены ниже.

```
byte byteValue()
double doubleValue()
float floatValue()
int intValue()
long longValue()
short shortValue()
```

Например, метод `doubleValue()` возвращает значение объекта как `double`, метод `floatValue()` — как `float` и т.д. Перечисленные выше методы реализуются каждым классом оболочки числового типа.

В каждом классе оболочки числового типа предусмотрены конструкторы, позволяющие сформировать объект на основе соответствующего простого типа данных или его строкового представления. Например, в классах `Integer` и `Double` имеются следующие конструкторы:

```
Integer(int num)
Integer(String str)
Double(double num)
Double(String str)
```

Если параметр `str` не содержит допустимое строковое представление числового значения, генерируется исключение `NumberFormatException`.

Во всех оболочках типов переопределен метод `toString()`. Он возвращает из оболочки значение в удобной для чтения форме. Это позволяет выводить значения на экран, передавая объекты оболочек в качестве параметра методу, например `println()`, и не преобразуя их предварительно в простые типы данных.

Процесс инкапсуляции значения в оболочке типа называется *упаковкой*. До появления версии JDK 5 упаковка производилась вручную, т.е. программирующий на Java строил явным образом экземпляр класса оболочки с нужным значением. Например, для ручной упаковки значения `100` в объект типа `Integer` требовалась следующая строка кода:

```
Integer iOb = new Integer(100);
```

В данном примере явно создается объект типа `Integer`, в который упаковывается значение `100`, а ссылка на этот объект присваивается переменной `iOb`.

Процесс извлечения значения из объекта оболочки называется *распаковкой*. До появления версии JDK 5 распаковка также производилась вручную, т.е. программирующему на Java приходилось вызывать явным образом соответствующий метод для объекта оболочки, чтобы извлечь значение, упакованное в этом объекте. Например, для распаковки значения из объекта `iOb` вручную и присваивания результата переменной `int` требовалась следующая строка кода:

```
int i = iOb.intValue();
```

В данном примере метод `intValue()` возвращает значение, упакованное в объект `iOb` как `int`.

Рассмотренные выше механизмы упаковки и распаковки демонстрируются в приведенном ниже примере программы.

```
// Упаковка и распаковка значений вручную.
class Wrap {
    public static void main(String args[]) {

        // Упаковка значения 100 производится вручную.
        Integer iOb = new Integer(100);

        // Распаковка значения 100 производится вручную.
        int i = iOb.intValue();

        System.out.println(i + " " + iOb); // отображает значения 100и100
    }
}
```

В данной программе целочисленное значение **100** упаковывается в объект типа `Integer`, на который ссылается переменная `iOb`. Для извлечения упакованного числового значения вызывается метод `intValue()`. Полученное значение сохраняется в переменной `i`. А в конце программы на экран выводятся значения переменных `i` и `iOb`, каждое из которых равно **100**.

Аналогичная процедура использовалась в программах для упаковки и распаковки значений, начиная с ранних версий Java и до появления JDK 5. Но это не совсем удобно. Более того, создание объектов оболочек разных типов вручную может сопровождаться ошибками. Но теперь, с появлением автоупаковки и автораспаковки, обращаться с оболочками типов стало значительно проще.

Основные положения о об автоупаковке

Автоупаковка — это процесс автоматической инкапсуляции (упаковки) простого типа данных в объекте оболочки соответствующего типа всякий раз, когда в этом возникает потребность, причем создавать такой объект явным образом не нужно. **Автораспаковка** — это процесс автоматического извлечения (распаковки) из объекта оболочки упакованного в нем значения соответствующего типа всякий раз, когда в этом возникает потребность. Благодаря автораспаковке отпадает необходимость в вызове таких методов, как `intValue()` и `doubleValue()`.

Поддержка автоупаковки и автораспаковки существенно упрощает реализацию целого ряда алгоритмов, так как в этом случае все рутинные операции по упаковке и распаковке значений простых типов берет на себя исполняющая система Java, что позволяет уменьшить вероятность возникновения программных ошибок. Автоупаковка освобождает программирующего на Java от необходимости создавать вручную объекты для заключения в них простых типов данных. Достаточно присвоить упаковываемое значение переменной ссылки на объект оболочки соответствующего типа, а нужный объект будет автоматически создан исполняющей системой Java. Ниже приведен пример создания объекта типа `Integer`, в который автоматически упаковывается целочисленное значение **100**.

```
Integer iOb = 100; // Автоупаковка целочисленного значения
```

Обратите внимание на то, что в данном примере отсутствует оператор `new`, конструирующий объект явным образом. Создание объекта происходит автоматически.

Для распаковки значения из объекта достаточно присвоить переменной простого типа ссылку на этот объект. Например, для распаковки значения, упакованного в объекте `iOb`, нужно лишь ввести в код следующую единственную строку:

```
int i = iOb; // Автораспаковка
```

А все остальное возьмет на себя исполняющая система Java. Ниже приведен пример программы, демонстрирующий автоупаковку и автораспаковку.

```
// Применение автоупаковки и автораспаковки.
class AutoBox {
    public static void main(String args[]) {

        // Автоупаковка и автораспаковка значения 100.
        Integer iOb = 100;
        int i = iOb;

        System.out.println(i + " " + iOb); // displays 100 100
    }
}
```

Автоупаковка и методы

Автоупаковка и автораспаковка происходят не только в простых операциях присваивания, но и в тех случаях, когда простой тип требуется преобразовать в объект, и наоборот. Следовательно, автоупаковка и автораспаковка могут происходить при передаче аргумента методу и при возврате значения последним. Рассмотрим в качестве примера следующую программу:

```
// Автоупаковка и автораспаковка при передаче
// параметров и возврате значений из методов.

class AutoBox2 {
    // Этот метод принимает параметр типа Integer.
    static void m(Integer v) {
        System.out.println("m() received " + v);
    }

    // Этот метод возвращает значение типа int.
    static int m2() {
        return 10;
    }

    // Этот метод возвращает значение типа Integer.
    static Integer m3() {
        return 99; // Автоупаковка значения 99 в объект типа Integer.
    }

    public static void main(String args[]) {

        // Передача методу m() значения типа int.
        // Метод m() принимает параметр типа Integer,
```

```
// поэтому значение int автоматически упаковывается.
m(199);

// Здесь объект iOb получает значение типа int, возвращаемое
// методом m2(). Это значение автоматически упаковывается,
// чтобы его можно было присвоить объекту iOb.
Integer iOb = m2();
System.out.println("Return value from m2() is " + iOb);

// А здесь метод m3() возвращает значение типа Integer, которое
// автоматически распаковывается и преобразуется в тип int.
int i = m3();
System.out.println("Return value from m3() is " + i);

// Здесь методу Math.sqrt() в качестве параметра передается
// объект iOb, который автоматически распаковывается, а его
// значение продвигается к типу double, требующемуся для
// выполнения данного метода.
iOb = 100;
System.out.println("Square root of iOb is " + Math.sqrt(iOb));
}
}
```

Результат выполнения данной программы выглядит так:

```
m() received 199
Return value from m2() is 10
Return value from m3() is 99
Square root of iOb is 10.0
```

В объявлении метода `m()` указывается, что ему должен передаваться параметр типа `Integer`. В методе `main()` целочисленное значение **199** передается методу `m()` в качестве параметра. В итоге происходит автоупаковка этого целочисленного значения. Далее в программе вызывается метод `m2()`, возвращающий целочисленные значения **10**. Это значение присваивается переменной ссылки на объект `iOb` в методе `main()`. А поскольку объект `iOb` относится к типу `Integer`, то целочисленное значение, возвращаемое методом `m2()`, автоматически упаковывается. Затем в методе `main()` вызывается метод `m3()`. Он возвращает объект типа `Integer`, который посредством автораспаковки преобразуется в тип `int`. И наконец, в методе `main()` вызывается метод `Math.sqrt()`, которому в качестве аргумента передается объект `iOb`. В данном случае происходит автораспаковка данного объекта, а его значение продвигается к типу `double`, поскольку параметр именно этого типа должен быть передан методу `Math.sqrt()`.

Автоупаковка и автораспаковка в выражениях

Автоупаковка и автораспаковка выполняются всякий раз, когда объект необходимо преобразовать в простой тип, а простой тип — в объект. Так, автораспаковка производится при вычислении выражений, и если требуется, то результат вычисления упаковывается. Рассмотрим в качестве примера приведенную ниже программу.

```
// Автоупаковка и автораспаковка в выражениях.

class AutoBox3 {
    public static void main(String args[]) {
```

```

Integer iOb, iOb2;
int i;

iOb = 99;
System.out.println("Original value of iOb: " + iOb);

// В следующем выражении объект iOb автоматически
// распаковывается, производятся вычисления, а результат
// снова упаковывается в объект iOb.
++iOb;
System.out.println("After ++iOb: " + iOb);

// В последующем выражении производится автораспаковка
// объекта iOb, к полученному значению прибавляется число 10,
// а результат снова упаковывается в объект iOb.
iOb += 10;
System.out.println("After iOb += 10: " + iOb);

// И в следующем выражении объект iOb автоматически
// распаковывается, выполняются вычисления, а результат
// снова упаковывается в объект iOb.
iOb2 = iOb + (iOb / 3);
System.out.println("iOb2 after expression: " + iOb2);

// А в этом случае вычисляется то же самое выражение,
// но повторная упаковка не производится.
i = iOb + (iOb / 3);

System.out.println("i after expression: " + i);
}
}

```

Выполнение этой программы дает следующий результат:

```

Original value of iOb: 99
After ++iOb: 100
After iOb += 10: 110
iOb2 after expression: 146
i after expression: 146

```

В данной программе особое внимание обратите на следующую строку кода:

```
++iOb;
```

В ней значение объекта **iOb** должно быть увеличено на единицу. Происходит это следующим образом: объект **iOb** распаковывается, полученное значение инкрементируется, а результат снова упаковывается в объект **iOb**.

Благодаря автораспаковке объекты оболочек целочисленных типов, например **Integer**, можно использовать в операторах **switch**. В качестве примера рассмотрим следующий фрагмент кода:

```

Integer iOb = 2;

switch(iOb) {
    case 1: System.out.println("one");
        break;
}

```

```

case 2: System.out.println("two");
    break;
default: System.out.println("error");
}

```

При вычислении выражения в операторе `switch` объект `iOb` распаковывается и последующей обработке подвергается значение типа `int`, упакованное в этом объекте.

Как следует из приведенных выше примеров, выражения, в которых применяются объекты оболочек простых типов, становятся интуитивно понятными благодаря автоупаковке и автораспаковке. До появления версии JDK 5 для достижения аналогичного результата в программе приходилось прибегать к приведению типов и вызовам специальных методов вроде `intValue()`.

Предупреждение относительно автоупаковки и автораспаковки

Теперь, когда автоупаковка и автораспаковка предельно упрощают обращение с оболочками простых типов, может возникнуть сильное искушение пользоваться вместо простых типов только их оболочками, например `Integer` или `Double`. Так, например, автоупаковка и автораспаковка позволяют создавать код, аналогичный приведенному ниже.

```

// Неоправданное использование автоупаковки и автораспаковки.
Double a, b, c;

```

```

a = 10.2;
b = 11.4;
c = 9.8;

```

```

Double avg = (a + b + c) / 3;

```

В данном примере в объектах типа `Double` хранятся три значения, используемые для вычисления арифметического среднего, а полученный результат присваивается другому объекту типа `Double`. И хотя такой код формально считается корректным, а следовательно, будет выполняться правильно, тем не менее, автоупаковка и автораспаковка применяются в нем совершенно не оправданно. Ведь подобный код значительно менее эффективен аналогичного кода, написанного только с использованием переменных типа `double`. А каждая распаковка и упаковка связана с издержками, которые простые типы не налагают на вычислительные ресурсы.

Вообще говоря, в программировании на Java желательно поменьше пользоваться оболочками простых типов. Прибегать к ним следует лишь в тех случаях, когда действительно требуется объектное представление простых типов. Ведь автоупаковка и автораспаковка внедрены в Java не в качестве “лазейки”, употребляемой в обход простых типов данных.

Статический импорт

Начиная с версии JDK 5 в Java была расширена область применения ключевого слова `import`, а именно: реализован механизм *статического импорта*. Указав после `import` ключевое слово `static`, можно сформировать выражение для импорта статических членов класса или интерфейса. Используя статический импорт, можно также ссылаться на статические члены непосредственно по именам, не указывая перед ними имена классов.

Благодаря этому упрощается синтаксис и сокращается запись выражений, в которых применяются статические члены классов.

Для того чтобы оценить по достоинству возможности статического импорта, начнем его рассмотрение с примера, в котором это языковое средство не используется. Ниже проведен пример программы для решения следующего квадратного уравнения:

$$ax^2 + bx + c = 0$$

В этой программе применяются два статических метода — `Math.pow()` и `Math.sqrt()` — из класса `Math`, который, в свою очередь, входит в пакет `java.lang`. Первый из них возвращает значение, возведенное в заданную степень, а второй — квадратный корень значения своего параметра.

```
// Решение квадратного уравнения.
class Quadratic {
    public static void main(String args[]) {

        // Переменные a, b и c обозначают коэффициенты
        // квадратного уравнения  $ax^2 + bx + c = 0$ 
        double a, b, c, x;

        // решить квадратное уравнение  $4x^2 + x - 3 = 0$ 
        a = 4;
        b = 1;
        c = -3;

        // найти первое решение
        x = (-b + Math.sqrt(Math.pow(b, 2) - 4 * a * c)) / (2 * a);
        System.out.println("First solution: " + x);

        // найти второе решение
        x = (-b - Math.sqrt(Math.pow(b, 2) - 4 * a * c)) / (2 * a);
        System.out.println("Second solution: " + x);
    }
}
```

Методы `pow()` и `sqrt()` являются статическими, а следовательно, их нужно вызывать, ссылаясь на имя класса `Math`. Их вызов осуществляется в приведенном ниже выражении, хотя и нельзя не признать, что оно получается довольно громоздким.

```
x = (-b + Math.sqrt(Math.pow(b, 2) - 4 * a * c)) / (2 * a);
```

В таких выражениях приходится постоянно следить за тем, чтобы перед методами `pow()` и `sqrt()` (и другими подобными методами, например `sin()`, `cos()` и `tan()`) было указано имя класса, что неудобно и чревато ошибками.

Утомительной обязанности указывать всякий раз имя класса перед статическим методом позволяет избежать статический импорт. Его применение демонстрирует приведенная ниже версия предыдущей программы.

```
// Применение статического импорта с целью
// упростить вызовы методов sqrt() и pow().

// Средствами статического импорта обеспечивается
// непосредственный доступ к методам sqrt() и pow().
import static java.lang.Math.sqrt;
import static java.lang.Math.pow;
```



```

class Quadratic {
    public static void main(String args[]) {

        // Переменные a, b и c обозначают коэффициенты
        // квадратного уравнения  $ax^2 + bx + c = 0$ 
        double a, b, c, x;

        // решить квадратное уравнение  $4x^2 + x - 3 = 0$ 
        a = 4;
        b = 1;
        c = -3;

        // найти первое решение
        x = (-b + sqrt(pow(b, 2) - 4 * a * c)) / (2 * a);
        System.out.println("First solution: " + x);

        // найти второе решение
        x = (-b - sqrt(pow(b, 2) - 4 * a * c)) / (2 * a);
        System.out.println("Second solution: " + x);
    }
}

```

В данной версии программы имена методов `sqrt` и `pow` уже не нужно указывать полностью (т.е. вместе с именем их класса). И достигается это благодаря статическому импорту обоих методов в приведенных ниже операторах, делающих оба метода непосредственно доступными.

```

import static java.lang.Math.sqrt;
import static java.lang.Math.pow;

```

После статического импорта отпадает необходимость предварять имена методов `sqrt()` и `pow()` именем их класса. В итоге выражение для решения квадратного уравнения принимает следующий вид:

```

x = (-b + sqrt(pow(b, 2) - 4 * a * c)) / (2 * a);

```

Теперь оно выглядит проще и воспринимается намного лучше.

В Java предусмотрены две общие формы оператора `import static`. В первой форме, использованной в предыдущем примере, непосредственно доступным для программы делается единственное имя. Ниже приведена эта общая форма статического импорта.

```

import static пакет.имя_типа.имя_статического_члена;

```

где *имя_типа* обозначает класс или интерфейс, содержащий требуемый статический член, на который указывает *имя_статического_члена*. Вторая общая форма оператора статического импорта выглядит следующим образом:

```

import static пакет.имя_типа.*;

```

Если предполагается использовать несколько статических методов или полей, определенных в классе, то данная общая форма записи позволяет импортировать все эти члены одновременно. Таким образом, обеспечить непосредственный доступ к методам `pow()` и `sqrt()` в предыдущей версии программы (а также к другим статическим членам класса `Math`) без указания имени класса можно с помощью следующей единственной строки кода:

```

import static java.lang.Math.*;

```

Очевидно, что статический импорт не ограничивается только классом `Math` и его методами. Так, если требуется сделать непосредственно доступным статическое поле `System.out` потока стандартного вывода, в программу достаточно ввести следующую строку кода:

```
import static java.lang.System.out;
```

После этого данные можно выводить на консоль, не указывая перед статическим полем `out` имя его класса `System`:

```
out.println("After importing System.out, you can use out directly.");
```

Насколько целесообразно поступать именно так — вопрос спорный. С одной стороны, размер исходного кода в этом случае сокращается. А с другой стороны, тем, кто просматривает исходный код программы, может быть непонятно, что конкретно обозначает имя `out`: поток стандартного вывода `System.out` или нечто иное.

Каким бы удобным ни был статический импорт, важно следить за тем, чтобы он применялся правильно. Как известно, библиотеки классов Java организованы в пакеты именно для того, чтобы исключить конфликты имен. Если импортируются статические члены класса, они переносятся в глобальное пространство имен. Вследствие этого увеличивается вероятность конфликтов и неумышленного сокрытия имен. Если статический член используется в программе один или два раза, то импортировать его нет никакого смысла. Кроме того, некоторые имена статических членов (например, `System.out`) настолько знакомы всем программирующим на Java, что они окажутся менее узнаваемыми, если будут употребляться без имени своего класса. Статический импорт был внедрен в расчете на те программы, в которых постоянно применяются определенные статические члены, например при математических расчетах. Следовательно, статическим импортом следует пользоваться аккуратно, не злоупотребляя им.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Допускается ли статический импорт статических членов создаваемых классов?

ОТВЕТ. Допускается. Это средство можно применять для импорта статических членов любых создаваемых классов и интерфейсов. Им особенно удобно пользоваться в тех случаях, когда в классе определено несколько статических членов, часто употребляемых в крупной программе. Так, если в классе определен ряд констант, обозначающих граничные значения, статический импорт позволит избежать излишнего упоминания их классов.

Аннотации (метаданные)

Начиная с версии JDE 5 в Java внедрено средство, позволяющее включать в исходный файл дополнительные сведения, которые называются *аннотацией* и не изменяют поведение программы. Эти сведения можно использовать в различных инструментальных средствах как на этапе разработки, так и в процессе развертывания прикладных программ. В частности, аннотации могут быть обработаны генератором исходного кода, компилятором или средствами развертывания прикладных программ. Дополнительные сведения, включаемые в исходный файл, иногда еще называют *метаданными*, но термин *аннотация* точнее отражает их назначение.

Вопросы составления и применения аннотаций выходят за рамки этой книги. Для подробного их рассмотрения здесь просто недостаточно места. Поэтому ограничимся лишь кратким описанием самого понятия и назначения аннотаций.

На заметку

Подробнее с метаданными и аннотациями можно ознакомиться в книге *Java. Полное руководство*, 8-е издание, ИД "Вильямс", 2012 г.

Для составления аннотаций служит ключевое слово `interface`. Ниже приведен простой пример объявления аннотации.

```
// Простой пример аннотации.
@interface MyAnno {
    String str();
    int val();
}
```

В данном примере объявляется аннотация `MyAnno`. Обратите внимание на то, что ключевое слово `interface` предваряется знаком `@`. Таким образом компилятору сообщается об объявлении аннотации. Обратите также внимание на два члена — `str()` и `val()`. Все аннотации содержат лишь объявления методов, а их тела указывать нельзя. Объявленные методы реализует исполняющая система Java, причем они действуют подобно полям.

Аннотации всех типов автоматически расширяют интерфейс `Annotation`. Следовательно, `Annotation` служит в качестве суперинтерфейса для всех аннотаций. Он входит в пакет `java.lang.annotation`.

Объявив аннотацию, ее можно использовать для аннотирования объявления. Аннотацию можно связать с любым объявлением. В частности, аннотированными могут быть объявления классов, методов, полей, параметров, констант перечислимого типа и даже самих аннотаций. Но в любом случае аннотация предшествует остальной части объявления.

Применяя аннотацию, вы тем самым задаете значения ее членов. Ниже приведен пример аннотации `MyAnno`, применяемой к методу.

```
// Аннотирование метода.
@MyAnno(str = "Annotation Example", val = 100)
public static void myMeth() { // ...
```

Эта аннотация связывается с методом `myMeth()`. Обратите внимание на синтаксис аннотации. Имени аннотации предшествует знак `@`, а после имени следует список, предназначенный для инициализации ее членов. Для того чтобы задать значение члена аннотации, следует присвоить это значение имени данного члена. В рассматриваемом здесь примере символьная строка `"Annotation Example"` (Пример аннотации) присваивается члену `str` аннотации `MyAnno`. Обратите внимание на то, что в операторе присваивания скобки после члена `str` не указываются. При присваивании значения члену аннотации используется только его имя. В этом отношении члены аннотации похожи на поля.

Аннотация без параметров называется *маркером*. При определении маркера круглые скобки не указываются. Главное назначение маркера — пометить объявление некоторым атрибутом.

В Java определено немало встроенных аннотаций. Некоторые из них считаются аннотациями общего назначения, но большинство являются специализированными. В па-

кет `java.lang.annotation` входят аннотации `@Retention`, `@Documented`, `@Target` и `@Inherited`. Аннотации `@Override`, `@Deprecated` и `@SuppressWarnings` включены в пакет `java.lang`. Назначение этих семи аннотаций приведено в табл. 12.1.

Таблица 12.1. Встроенные аннотации

Аннотация	Описание
<code>@Retention</code>	Определяет правила сохранения, связываемые с аннотацией. Правила сохранения определяют, как долго аннотация будет присутствовать в процессе компиляции и развертывания прикладной программы
<code>@Documented</code>	Маркер, сообщающий инструментальному средству о том, что аннотация документируется. Желательно использовать только как аннотацию к аннотации объявления
<code>@Target</code>	Задаёт виды объявлений, к которым может быть применена аннотация. Желательно использовать только в качестве одной аннотации к другой. Данная аннотация принимает аргумент в виде константы перечислимого типа <code>ElementType</code> , в котором определены различные константы, в том числе <code>CONSTRUCTOR</code> , <code>FIELD</code> и <code>METHOD</code> . Аргумент определяет виды объявлений, к которым может быть применена аннотация
<code>@Inherited</code>	Маркер, указывающий на то, что аннотация суперкласса должна наследоваться подклассом
<code>@Override</code>	Метод, аннотированный как <code>@Override</code> , должен переопределять метод суперкласса. Если это условие не выполняется, возникает ошибка при компиляции. Данная аннотация представляет собой маркер и позволяет убедиться в том, что метод суперкласса действительно переопределен, а не перегружен
<code>@Deprecated</code>	Маркер, указывающий на то, что объявление устарело и было заменено новым
<code>@SuppressWarnings</code>	Указывает на то, что одно или несколько предупреждающих сообщений, которые могут быть сгенерированы при компиляции, должны быть подавлены. Предупреждающие сообщения задаются именами, представленными в строковом виде

Ниже приведен пример, в котором аннотацией `@Deprecated` помечаются класс `MyClass` и метод `getMsg()`. При попытке скомпилировать программу будет выведено сообщение о том, что в исходном коде содержатся устаревшие и не рекомендованные к применению элементы.

```
// Пример использования аннотации @Deprecated.

// Пометить класс как не рекомендованный к применению.
@Deprecated
class MyClass {
    private String msg;

    MyClass(String m) {
        msg = m;
    }

    // Пометить метод в классе как
    // не рекомендованный к применению.
```

```

@Deprecated
String getMsg() {
    return msg;
}

// ...
}

class AnnoDemo {
    public static void main(String args[]) {
        MyClass myObj = new MyClass("test");

        System.out.println(myObj.getMsg());
    }
}

```

Упражнение для самопроверки по материалу главы 12

1. Константы перечислимого типа иногда называют *самотипизированными*. Что это означает?
2. Какой класс автоматически наследуют перечисления?
3. Напишите для приведенного ниже перечисления программу, в которой метод `values()` служит для отображения списка констант и их значений.

```

enum Tools {
    SCREWDRIVER, WRENCH, HAMMER, PLIERS
}

```

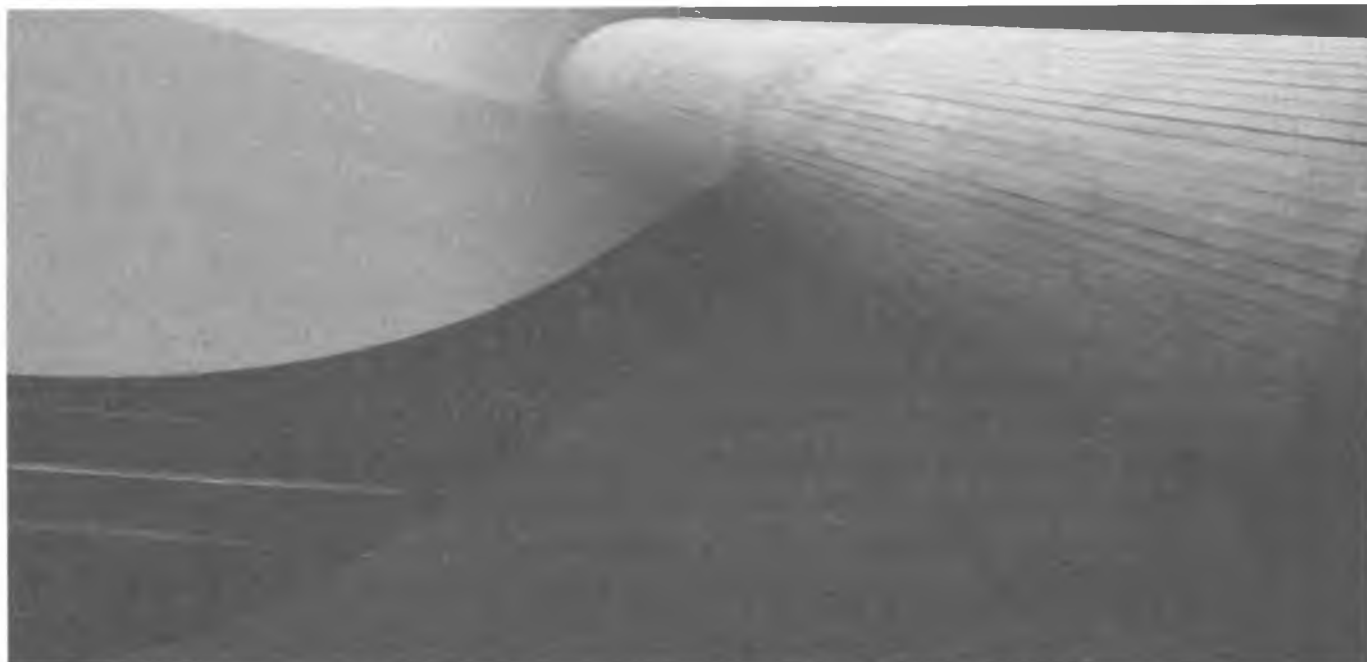
4. Программу, имитирующую автоматизированный светофор и созданную в примере для опробования 12.1, можно усовершенствовать, внося ряд простых изменений, чтобы выгодно воспользоваться возможностями перечислений. В исходной версии этой программы продолжительность отображения каждого цвета светофора регулировалась в классе `TrafficLightSimulator`, причем величины задержек были жестко запрограммированы в методе `run()`. Измените исходный код программы таким образом, чтобы продолжительность отображения каждого цвета светофора задавалась константами перечислимого типа `TrafficLightColor`. Для этого вам понадобятся конструктор, переменная экземпляра, объявленная как `private`, а также метод `getDelay()`. Подумайте о том, как еще можно улучшить данную программу. (Подсказка: попробуйте отказаться от оператора `switch` и воспользоваться порядковыми значениями каждого цвета для переключения светофора.)
5. Что такое упаковка и распаковка? В каких случаях производится автоупаковка и автораспаковка?
6. Измените следующий фрагмент кода таким образом, чтобы в нем производилась автоупаковка:

```
Short val = new Short(123);
```

7. Объясните, что такое статический импорт?
8. Какие действия выполняет приведенный ниже оператор?

```
import static java.lang.Integer.parseInt;
```

9. Следует ли употреблять статический импорт от случая к случаю или желательно импортировать статические члены всех классов?
10. Синтаксис аннотации основывается на _____.
11. Какая аннотация называется маркером?
12. Аннотации применимы только к методам. Верно или неверно?



Глава 13

Обобщения

Основные навыки и понятия

- Преимущества обобщений
- Создание обобщенного класса
- Применение ограниченных параметров типов
- Использование метасимвольных аргументов
- Применение ограниченных метасимвольных аргументов
- Создание обобщенного метода
- Создание обобщенного конструктора
- Создание обобщенного интерфейса
- Использование базовых типов
- Выводимость типов с помощью ромбовидного оператора
- Стирание
- Исключение ошибок неоднозначности
- Наложение ограничений на обобщения

Начиная с версии 1.0 в Java было реализовано много новых языковых средств. Все они очень полезны и расширяют область применения Java, но на одном из них следует остановиться особо, поскольку оно оказывает огромное влияние на язык в целом. Речь идет об *обобщениях* — совершенно новой синтаксической конструкции, появление которой вызвало существенные изменения во многих классах и методах базового интерфейса API. Не будет преувеличением сказать, что введение обобщений коренным образом изменило сам язык Java.

Обобщения — слишком обширная тема, чтобы подробно рассматривать ее в этой книге. Тем не менее всякий программирующий на Java должен иметь хотя бы общее представление об этом языковом средстве. На первый взгляд синтаксис обобщений может показаться непонятным, но на самом деле пользоваться ими совсем не трудно. К тому моменту, когда вы завершите проработку материала этой главы, вы не только усвоите основы обобщений, но и научитесь успешно применять их в своих программах.

Основные положения об обобщениях

Термин *обобщение*, по существу, означает *параметризованный тип*. Особая роль параметризованных типов состоит в том, что они позволяют создавать классы, интерфейсы и методы, в которых обрабатываемые данные указываются в виде параметра. С помощью обобщений можно, например, создать единый класс, который автоматически становится пригодным для обработки разнотипных данных. Класс, интерфейс или метод, оперирующий параметризованным типом данных, называется *обобщенным*, как, например, *обобщенный класс* или *обобщенный метод*.

Главное преимущество обобщенного кода состоит в том, что он автоматически настраивается на работу с нужным типом данных. Многие алгоритмы выполняются одинаково, независимо от того, к данным какого типа они должны применяться. Например, быстрая сортировка не зависит от типа данных, в качестве которого можно использовать `Integer`, `String`, `Object` и даже `Thread`. Используя обобщения, можно реализовать алгоритм один раз, а затем применять его без особого труда к любому типу данных.

Следует особо подчеркнуть, что в Java всегда имелась возможность создавать обобщенный код, оперируя ссылками типа `Object`. А поскольку класс `Object` является суперклассом для всех остальных классов, то по ссылке типа `Object` можно обращаться к объекту любого типа. Таким образом, до появления обобщений для оперирования разнотипными объектами в программах служил обобщенный код, в котором для этой цели использовались ссылки типа `Object`.

Но дело в том, что в таком коде трудно было соблюсти типовую безопасность, поскольку для преобразования типа `Object` в конкретный тип данных требовалось приведение типов. А это служило потенциальным источником ошибок из-за того, что приведение типов могло быть неумышленно выполнено неверно. Это затруднение позволяют преодолеть обобщения, обеспечивая типовую безопасность, которой раньше так не хватало. Кроме того, обобщения упрощают весь процесс, поскольку исключают необходимость выполнять приведение типов для преобразования объекта или другого типа обрабатываемых данных. Таким образом, обобщения расширяют возможности повторного использования кода и позволяют делать это надежно и просто.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Говорят, что обобщения в Java аналогичны шаблонам в C++. Так ли это?

ОТВЕТ. Действительно, обобщения в Java похожи на шаблоны в C++. То, что в Java называют *параметризованными типами*, в C++ называют *шаблонами*. Но они не полностью равнозначны. У них имеется ряд принципиальных отличий, а в целом обобщения в Java намного более просты в употреблении.

Тем, у кого имеется опыт программирования на C++, важно помнить, что навыки применения шаблонов нельзя механически переносить на обобщения в Java. У этих языковых средств имеются отличия, которые слабо проявляются внешне, но довольно значительны по существу.

Простой пример обобщений

Прежде чем приступить к более подробному рассмотрению обобщений, полезно рассмотреть простой пример их применения. Ниже приведен исходный код программы, в которой объявлены два класса. Первым из них является обобщенный класс `Gen`, вторым — класс `GenDemo`, в котором используется класс `Gen`.

```
// Простой обобщенный класс.
// Здесь T - это параметр типа, заменяемый именем
// подлинного типа при создании объекта класса Gen.

// В объявлении этого класса T означает обобщенный тип.
class Gen<T> {
    T ob; // объявить объект типа T
```

```

// передать конструктору ссылку на объект типа T.
Gen(T o) {
    ob = o;
}

// вернуть объект ob из метода
T getob() {
    return ob;
}

// отобразить тип T
void showType() {
    System.out.println("Type of T is " +
                       ob.getClass().getName());
}
}

// продемонстрировать обобщенный класс
class GenDemo {
    public static void main(String args[]) {
        // Создание ссылки на объект типа Gen<Integer>.
        Gen<Integer> iOb;

        // Создать объект типа Gen<Integer> и присвоить ссылку на
        // него переменной iOb. Обратите внимание на автоупаковку при
        // инкапсуляции значения 88 в объекте типа Integer.
        iOb = new Gen<Integer>(88); // получить экземпляр типа Gen<Integer>

        // отобразить тип данных, используемых в объекте iOb
        iOb.showType();

        // Получение значения из объекта iOb. Обратите внимание
        // на то, что приведение типов здесь не требуется.
        int v = iOb.getob();
        System.out.println("value: " + v);

        System.out.println();

        // Создание объекта типа Gen для символьных строк.
        // Здесь создается ссылка и сам объект типа Gen<String>.
        Gen<String> strOb = new Gen<String>("Generics Test");

        // отобразить тип данных, используемых в объекте strOb
        strOb.showType();

        // Получение значения из объекта strOb.
        // И здесь приведение типов не требуется.
        String str = strOb.getob();
        System.out.println("value: " + str);
    }
}

```

Выполнение данной программы дает следующий результат:

```

Type of T is java.lang.Integer
value: 88

```

```
Type of T is java.lang.String
value: Generics Test
```

Рассмотрим исходный код данной программы более подробно. Прежде всего обратите внимание на то, как объявляется класс `Gen`. Для этого используется следующая строка кода:

```
class Gen<T> {
```

где `T` — имя *параметра типа*. Это имя служит в качестве метки-заполнителя конкретного типа, который указывается при создании объекта класса `Gen`. Следовательно, имя `T` используется в классе `Gen` всякий раз, когда требуется параметр типа. Обратите внимание на то, что имя `T` заключается в угловые скобки (`< >`). Этот синтаксис можно обобщить: всякий раз, когда объявляется параметр типа, он указывается в угловых скобках. А поскольку параметр типа используется в классе `Gen`, то такой класс считается *обобщенным*.

В объявлении класса `Gen` можно указывать любое имя параметра типа, но по традиции выбирается имя `T`. К числу других наиболее употребительных имен параметров типа относятся `V` и `E`. А вообще, обозначать параметры типа рекомендуется одной прописной буквой.

Далее имя `T` используется для объявления объекта `ob`, как показано в следующей строке кода:

```
T ob; // объявить объект типа T
```

Как пояснялось выше, имя параметра типа `T` служит меткой-заполнителем конкретного типа, указываемого при создании объекта класса `Gen`. Поэтому объект `ob` будет иметь тип, передаваемый в качестве параметра типа `T` при получении экземпляра объекта класса `Gen`. Так, если в качестве параметра типа `T` указывается `String`, то экземпляр объекта `ob` будет отнесен к типу `String`.

Рассмотрим далее конструктор класса `Gen`.

```
Gen(T o) {
    ob = o;
}
```

Как видите, параметр `o` этого конструктора относится к типу `T`. Это означает, что конкретный тип параметра `o` определяется типом, передаваемым в качестве параметра типа `T` при создании объекта класса `Gen`. А поскольку параметр `o` и переменная экземпляра `ob` относятся к типу `T`, то после создания объекта класса `Gen` их конкретный тип окажется одним и тем же.

С помощью параметра типа `T` можно также указывать тип, возвращаемый методом, как показано ниже на примере метода `getob()`.

```
T getob() {
    return ob;
}
```

Переменная экземпляра `ob` также относится к типу `T`, поэтому ее тип совпадает с типом, возвращаемым методом `getob()`.

Метод `showType()` отображает тип `T`. С этой целью метод `getName()` вызывается для объекта типа `Class`, возвращаемого методом `getClass()`, вызываемым для объекта `ob`. Это средство еще не применялось в представленных до сих пор примерах программ, поэтому рассмотрим его подробнее. Как пояснялось в главе 7, в классе `Object` определен метод `getClass()`, автоматически являющийся членом каждого производного

класса. Он возвращает объект типа `Class`, соответствующий типу класса текущего объекта. Класс `Class` относится к пакету `java.lang` и инкапсулирует сведения о текущем классе. В нем определено несколько методов, которые позволяют получать сведения о классах по ходу выполнения программы. К их числу принадлежит метод `getName()`, возвращающий строковое представление имени класса.

В классе `GenDemo` демонстрируется применение обобщенного класса `Gen`. Прежде всего, в нем создается версия класса `Gen` для целых чисел, как показано ниже.

```
Gen<Integer> iOb;
```

Внимательно проанализируем это объявление. В первую очередь обратите внимание на то, что тип `Integer` указывается в угловых скобках после имени класса `Gen`. В данном случае `Integer` служит *аргументом типа*, передаваемым в качестве параметра типа `T` класса `Gen`. В рассматриваемом здесь объявлении создается версия класса `Gen`, в которой тип `T` заменяется типом `Integer` везде, где он встречается. Следовательно, после этого объявления `Integer` становится типом переменной `ob` и возвращаемым типом метода `getob()`.

Прежде чем продолжить рассмотрение обобщений, следует принять во внимание то обстоятельство, что компилятор Java на самом деле не создает разные версии `Gen` или другого обобщенного класса, а просто удаляет данные обобщенного типа, заменяя их приведением типов. Получаемый в итоге объект ведет себя так, как *будто* в программе была создана конкретная версия класса `Gen`. Таким образом, в программе фактически присутствует лишь одна версия класса `Gen`. Процесс удаления данных обобщенного типа называется *стиранием*, более подробно рассматриваемым в конце этой главы.

В следующей строке кода переменной `iOb` присваивается ссылка на экземпляр в версии класса `Gen` для типа `Integer`:

```
iOb = new Gen<Integer>(88);
```

Обратите внимание на то, что при вызове конструктора класса `Gen` указывается также аргумент типа `Integer`. Это необходимо потому, что тип объекта, на который указывает ссылка (в данном случае — `iOb`), должен соответствовать `Gen<Integer>`. Если тип ссылки, возвращаемой оператором `new`, будет отличаться от `Gen<Integer>`, возникнет ошибка при компиляции. Сообщение о такой ошибке будет, например, получено при попытке скомпилировать следующую строку кода:

```
iOb = new Gen<Double>(88.0); // Ошибка!
```

Переменная `iOb` относится к типу `Gen<Integer>`, а следовательно, ее нельзя использовать для хранения ссылки на объект типа `Gen<Double>`. Возможность проверки на соответствие типов — одно из основных преимуществ обобщенных типов, поскольку они обеспечивают типовую безопасность.

Как следует из комментариев к программе, в рассматриваемом здесь операторе присваивания

```
iOb = new Gen<Integer>(88);
```

производится автоупаковка целочисленного значения **88** в объект типа `Integer`. Это происходит потому, что обобщение `Gen<Integer>` создает конструктор, которому передается аргумент типа `Integer`. А поскольку предполагается создание объекта типа `Integer`, то в нем автоматически упаковывается целочисленное значение **88**. Разумеется, это можно было бы явно указать в операторе присваивания, как показано ниже.

```
iOb = new Gen<Integer>(new Integer(88));
```

Но в данном случае столь длинная строка кода не дает никаких преимуществ по сравнению с предыдущей, более компактной записью.

Затем в программе отображается тип переменной `ob` в объекте `iOb` (в данном случае это тип `Integer`). А значение переменной `ob` получается в следующей строке кода:

```
int v = iOb.getob();
```

Метод `getob()` возвращает значение типа `T`, замененное на `Integer` при объявлении переменной ссылки на объект `iOb`, а следовательно, метод `getob()` фактически возвращает значение того же самого типа `Integer`. Это значение автоматически распаковывается перед присваиванием переменной `v` типа `int`.

И наконец, в классе `GenDemo` объявляется объект типа `Gen<String>`.

```
Gen<String> strOb = new Gen<String>("Generics Test");
```

В этом объявлении указывается аргумент типа `String`, поэтому в объекте класса `Gen` вместо `T` подставляется тип `String`. В итоге создается версия класса `Gen` для типа `String`, как демонстрируют остальные строки кода рассматриваемой здесь программы.

Действие обобщений распространяется только на объекты

При определении экземпляра обобщенного класса аргумент типа, передаваемый в качестве параметра типа, должен обозначать тип класса. Для этой цели *нельзя* использовать простой тип, например `int` или `char`. В примере с классом `Gen` в качестве параметра типа `T` можно передать любой класс, но не простой тип данных. Иными словами, следующее объявление недопустимо:

```
Gen<int> strOb = new Gen<int>(53); // Ошибка. Использовать простой тип нельзя!
```

Очевидно, что запрет на использование простых типов не является серьезным ограничением, поскольку всегда можно воспользоваться классом оболочки типа, инкапсулировав в нем значение простого типа, что и было продемонстрировано в предыдущем примере программы. А поддержка в Java автоупаковки и автораспаковки еще больше упрощает применение оболочек типов в обобщениях.

Различение обобщений по аргументам типа

Для лучшего усвоения обобщенных типов следует иметь в виду, что ссылка на один вариант некоторого обобщенного типа несовместима с другим вариантом того же самого обобщенного типа. Так, если бы в рассмотренном выше примере программы присутствовала приведенная ниже строка кода, компилятор выдал бы сообщение об ошибке.

```
iOb = strOb; // Ошибка!
```

Несмотря на то что обе переменные, `iOb` и `strOb`, относятся к типу `Gen<T>`, они являются ссылками на объекты разного типа, поскольку при их объявлении указаны разные аргументы типа. Это часть той типовой безопасности обобщений, благодаря которой предотвращаются программные ошибки.

Обобщенный класс с двумя параметрами типа

В обобщенном классе можно задать несколько параметров типа. В этом случае параметры типа разделяются запятыми. Например, приведенный ниже класс `TwoGen` является переделанной версией класса `Gen`, в которой определены два параметра типа.

```
// Простой обобщенный класс с двумя параметрами типа: T и V.
class TwoGen<T, V> { // Применение двух параметров типа
```

```

T ob1;
V ob2;

// передать конструктору класса ссылки на объекты типов T и V
TwoGen(T o1, V o2) {
    ob1 = o1;
    ob2 = o2;
}

// отобразить типы T и V
void showTypes() {
    System.out.println("Type of T is " +
                       ob1.getClass().getName());

    System.out.println("Type of V is " +
                       ob2.getClass().getName());
}

T getob1() {
    return ob1;
}

V getob2() {
    return ob2;
}
}

// продемонстрировать класс TwoGen
class SimpGen {
    public static void main(String args[]) {
        // Здесь в качестве параметра типа T передается тип
        // Integer, а в качестве параметра типа V - тип String.
        TwoGen<Integer, String> tgObj =
            new TwoGen<Integer, String>(88, "Generics");

        // отобразить конкретные типы
        tgObj.showTypes();

        // получить и отобразить отдельные значения
        int v = tgObj.getob1();
        System.out.println("value: " + v);

        String str = tgObj.getob2();
        System.out.println("value: " + str);
    }
}

```

Выполнение этой программы дает следующий результат:

```

Type of T is java.lang.Integer
Type of V is java.lang.String
value: 88
value: Generics

```

Обратите внимание на приведенное ниже объявление класса TwoGen.

```

class TwoGen<T, V> {

```

Здесь определяются два параметра типа, *T* и *V*, разделяемые запятыми. А поскольку в этом классе используются два параметра типа, то при создании его объекта следует непременно указывать оба аргумента типа, как показано ниже.

```
TwoGen<Integer, String> tgObj =
    new TwoGen<Integer, String>(88, "Generics");
```

В данном случае тип *Integer* передается в качестве параметра типа *T*, а тип *String* — в качестве параметра типа *V*. И хотя в этом примере аргументы типа отличаются, они могут в принципе и совпадать. Например, следующая строка кода считается вполне допустимой:

```
TwoGen<String, String> x = new TwoGen<String, String>("A", "B");
```

В данном случае в качестве обоих параметров типа *T* и *V* передается один и тот же тип *String*. Очевидно, что если аргументы типа совпадают, то определять два параметра типа в обобщенном классе нет никакой надобности.

Общая форма обобщенного класса

Синтаксис обобщений, представленных в предыдущих примерах, может быть сведен к общей форме. Ниже приведена общая форма объявления обобщенного класса.

```
class имя_класса<список_параметров_типа> { // ...
```

А вот как выглядит синтаксис объявления ссылки на обобщенный класс:

```
имя_класса<список_аргументов_типа> имя_переменной =
    new имя_класса<список_аргументов_типа> (список_аргументов_конструктора);
```

Ограниченные типы

В предыдущих примерах параметры типа могли заменяться любым типом класса. Такая подстановка оказывается пригодной для многих целей, но иногда бывает полезно ограничить допустимый ряд типов, передаваемых в качестве параметра типа. Допустим, требуется создать обобщенный класс для хранения числовых значений и выполнения над ними различных математических операций, включая получение обратной величины или извлечение дробной части. Допустим также, что в этом классе предполагается выполнение математических операций над данными любых числовых типов: как целочисленных, так и с плавающей точкой. В таком случае будет вполне логично указывать числовой тип данных обобщенно, т.е. с помощью параметра типа. Для создания такого класса можно было бы написать код, аналогичный приведенному ниже.

```
// Класс NumericFns как пример неудачной попытки создать
// обобщенный класс для выполнения различных математических
// операций, включая получение обратной величины или
// извлечение дробной части числовых значений любого типа.
class NumericFns<T> {
    T num;

    // передать конструктору ссылку на числовой объект
    NumericFns(T n) {
        num = n;
    }

    // вернуть обратную величину
```



```

double reciprocal() {
    return 1 / num.doubleValue(); // Ошибка!
}

// вернуть дробную часть
double fraction() {
    return num.doubleValue() - num.intValue(); // Ошибка!
}

// ...
}

```

К сожалению, класс `NumericFns` в таком виде, в каком он приведен выше, не компилируется, так как оба метода, определенные в этом классе, содержат программную ошибку. Рассмотрим сначала метод `reciprocal()`, который пытается вернуть величину, обратную его параметру `num`. Для этого нужно разделить 1 на значение переменной `num`, которое определяется при вызове метода `doubleValue()`, возвращающего вариант `double` числового объекта, хранящегося в переменной `num`. Как известно, все числовые классы, в том числе `Integer` и `Double`, являются подклассами, производными от класса `Number`, в котором определен метод `doubleValue()`, что делает его доступным для всех классов оболочек числовых типов. Но дело в том, что компилятору неизвестно, что объекты класса `NumericFns` предполагается создавать только для числовых типов данных. Поэтому при попытке скомпилировать класс `NumericFns` возникает ошибка, а соответствующее сообщение уведомляет о том, что метод `doubleValue()` неизвестен. Аналогичная ошибка возникает дважды при компиляции метода `fraction()`, где вызываются методы `doubleValue()` и `intValue()`. При вызовах обоих этих методов компилятор также сообщает о том, что они неизвестны. Для того чтобы разрешить данное затруднение, нужно каким-то образом сообщить компилятору, что в качестве параметра типа `T` предполагается передавать только числовые типы. И нужно еще убедиться, что в действительности передаются *только* эти типы данных. Для подобных случаев в Java предусмотрены *ограниченные типы*. При указании параметра типа можно задать верхнюю границу, объявив суперкласс, который должны наследовать все аргументы типа. И делается это с помощью оператора `extends`, указываемого при определении параметра типа, как показано ниже.

```
<T extends суперкласс>
```

В этом объявлении компилятору указывается, что параметр типа `T` может быть заменен только суперклассом или его подклассами. Таким образом, *суперкласс* определяет верхнюю границу в иерархии классов Java.

С помощью ограниченных типов можно устранить программные ошибки в классе `NumericFns`. Для этого следует указать верхнюю границу так, как показано ниже.

```

// В этой версии класса NumericFns аргументом типа,
// заменяющим параметр типа T, должен стать класс Number
// или производный от него подкласс, как показано ниже.
class NumericFns<T extends Number> {
    T num;

    // передать конструктору ссылку на числовой объект
    NumericFns(T n) {
        num = n;
    }
}

```

```

// вернуть обратную величину
double reciprocal() {
    return 1 / num.doubleValue();
}

// вернуть дробную часть
double fraction() {
    return num.doubleValue() - num.intValue();
}

// ...
}

// продемонстрировать класс NumericFns
class BoundsDemo {
    public static void main(String args[]) {

        // Применение класса Integer вполне допустимо, так как он
        // является подклассом, производным от класса Number.
        NumericFns<Integer> iOb =
            new NumericFns<Integer>(5);

        System.out.println("Reciprocal of iOb is " +
            iOb.reciprocal());
        System.out.println("Fractional component of iOb is " +
            iOb.fraction());

        System.out.println();

        // Применение класса Double также допустимо.
        NumericFns<Double> dOb =
            new NumericFns<Double>(5.25);

        System.out.println("Reciprocal of dOb is " +
            dOb.reciprocal());
        System.out.println("Fractional component of dOb is " +
            dOb.fraction());

        // Следующая строка кода не будет компилироваться, так как
        // класс String не является производным от класса Number.
        // NumericFns<String> strOb = new NumericFns<String>("Error");
    }
}

```

Ниже приведен результат выполнения данной программы.

```

Reciprocal of iOb is 0.2
Fractional component of iOb is 0.0

Reciprocal of dOb is 0.19047619047619047
Fractional component of dOb is 0.25

```

Как видите, для объявления класса `NumericFns` в данном примере служит следующая строка кода:

```
class NumericFns<T extends Number> {
```

Теперь тип `T` ограничен классом `Number`, а следовательно, компилятору Java известно, что для всех объектов типа `T` доступен метод `doubleValue()`, а также другие методы, определенные в классе `Number`. И хотя это само по себе дает немалые преимущества, кроме того, предотвращает создание объектов класса `NumericFns` для нечисловых типов. Так, если попытаться удалить комментарии из строки кода в конце рассматриваемой здесь программы, а затем повторно скомпилировать ее, то будет получено сообщение об ошибке, поскольку класс `String` не является подклассом, производным от класса `Number`.

Ограниченные типы оказываются особенно полезными в тех случаях, когда нужно обеспечить совместимость одного параметра типа с другим. Рассмотрим в качестве примера представленный ниже класс `Pair`. В нем хранятся два объекта, которые должны быть совместимы друг с другом.

// Тип `V` должен совпадать с типом `T` или быть его подклассом.

```
class Pair<T, V extends T> {
    T first;
    V second;

    Pair(T a, V b) {
        first = a;
        second = b;
    }

    // ...
}
```

В классе `Pair` определяются два параметра типа `T` и `V`, причем `V` расширяет тип `T`. Это означает, что тип `V` должен быть либо того же типа, что и `T`, либо его подклассом. Благодаря такому объявлению гарантируется, что два параметра типа, передаваемые конструктору класса `Pair`, будут совместимы друг с другом. Например, приведенные ниже строки кода составлены правильно.

// Эта строка кода верна, так как `T` и `V` относятся типу `Integer`.
`Pair<Integer, Integer> x = new Pair<Integer, Integer>(1, 2);`

// И эта строка кода верна, так как `Integer` является подклассом `Number`.
`Pair<Number, Integer> y = new Pair<Number, Integer>(10.4, 12);`

А следующий фрагмент кода содержит ошибку:

// Эта строка кода недопустима, так как `String` не является подклассом `Number`.
`Pair<Number, String> z = new Pair<Number, String>(10.4, "12");`

В данном случае класс `String` не является производным от класса `Number`, что нарушает граничное условие, указанное в объявлении класса `Pair`.

Использование метасимвольных аргументов

Несмотря на всю полезность типовой безопасности в обобщениях, иногда она может помешать использованию идеально подходящих языковых конструкций. Допустим, требуется реализовать метод `absEqual()`, возвращающий логическое значение `true` в том случае, если два объекта рассмотренного выше класса `NumericFns` содержат одинаковые абсолютные значения. Допустим также, что этот метод должен оперировать любыми типами числовых данных, которые могут храниться в сравниваемых объектах. Так,

если один объект содержит значение 1,25 типа Double, а другой — значение -1,25 типа Float, метод `absEqual()` должен возвращать логическое значение `true`. Один из способов реализации метода `absEqual()` состоит в том, чтобы передавать этому методу параметр типа `NumericFns`, а затем сравнивать его абсолютное значение с абсолютным значением текущего объекта и возвращать логическое значение `true`, если эти значения совпадают. Например, вызов метода `absEqual()` может выглядеть следующим образом:

```
NumericFns<Double> dOb = new NumericFns<Double>(1.25);
NumericFns<Float> fOb = new NumericFns<Float>(-1.25);

if(dOb.absEqual(fOb))
    System.out.println("Absolute values are the same.");
else
    System.out.println("Absolute values differ.");
```

На первый взгляд может показаться, что при выполнении метода `absEqual()` не должно возникнуть никаких затруднений, но это совсем не так. Затруднения начнутся при первой же попытке объявить параметр типа `NumericFns`. Каким он должен быть? Казалось бы, подходящим должно быть следующее решение, где `T` указывается в качестве параметра типа:

```
// Не пройдет!

// определить равенство абсолютных значений в двух объектах
boolean absEqual(NumericFns<T> ob) {
    if(Math.abs(num.doubleValue()) ==
        Math.abs(ob.num.doubleValue())) return true;
    return false;
}
```

В данном случае для определения абсолютного значения каждого числа используется стандартный метод `Math.abs()`. Полученные значения сравниваются. Но дело в том, что рассматриваемое здесь решение окажется пригодным лишь в том случае, если объект класса `NumericFns`, передаваемый в качестве параметра, имеет тот же тип, что и текущий объект. Так, если текущий объект относится к типу `NumericFns<Integer>`, параметр `ob` также должен быть типа `NumericFns<Integer>`, а следовательно, сравнить текущий объект с объектом типа `NumericFns<Double>` не удастся. Таким образом, выбранное решение не является обобщенным.

Для того чтобы создать обобщенный метод `absEqual()`, придется воспользоваться еще одним свойством обобщений в Java, называемым *метасимвольным аргументом*. Для указания такого аргумента служит знак `?`, обозначающий неизвестный тип данных. Используя метасимвольный аргумент, можно переписать метод `absEqual()` следующим образом:

```
// определить равенство абсолютных значений в двух объектах
boolean absEqual(NumericFns<?> ob) { // обратите внимание на метасимвол
    if(Math.abs(num.doubleValue()) ==
        Math.abs(ob.num.doubleValue())) return true;
    return false;
}
```

В данном случае выражение `NumericFns<?>` соответствует любому типу объекта из класса `NumericFns` и позволяет сравнивать абсолютные значения в двух произвольных объектах класса `NumericFns`. Ниже приведен пример программы, демонстрирующий применение метасимвольного аргумента.

```
// Применение метасимвольного аргумента.
class NumericFns<T extends Number> {
    T num;

    // передать конструктору ссылку на числовой объект
    NumericFns(T n) {
        num = n;
    }

    // вернуть обратную величину
    double reciprocal() {
        return 1 / num.doubleValue();
    }

    // вернуть дробную часть
    double fraction() {
        return num.doubleValue() - num.intValue();
    }

    // определить равенство абсолютных значений в двух объектах
    boolean absEqual(NumericFns<?> ob) {
        if(Math.abs(num.doubleValue()) ==
            Math.abs(ob.num.doubleValue())) return true;

        return false;
    }

    // ...
}

// продемонстрировать применение метасимвольного аргумента
class WildcardDemo {
    public static void main(String args[]) {

        NumericFns<Integer> iOb =
            new NumericFns<Integer>(6);
        NumericFns<Double> dOb =
            new NumericFns<Double>(-6.0);
        NumericFns<Long> lOb =
            new NumericFns<Long>(5L);

        System.out.println("Testing iOb and dOb.");
        // В этом вызове метода тип метасимвольного
        // аргумента совпадает с типом Double.
        if(iOb.absEqual(dOb))
            System.out.println("Absolute values are equal.");
        else
            System.out.println("Absolute values differ.");

        System.out.println();

        System.out.println("Testing iOb and lOb.");
        // А в этом вызове метода тип метасимвольного
```

```
// аргумента совпадает с типом Long.
if(iOb.absEqual(lOb))
    System.out.println("Absolute values are equal.");
else
    System.out.println("Absolute values differ.");
}
}
```

Выполнение этой программы дает следующий результат:

```
Testing iOb and dOb.
Absolute values are equal.
```

```
Testing iOb and lOb.
Absolute values differ.
```

Обратите внимание на два следующих вызова метода `absEqual()`:

```
if(iOb.absEqual(dOb))

if(iOb.absEqual(lOb))
```

В первом вызове переменная `iOb` указывает на объект типа `NumericFns<Integer>`, а переменная `dOb` — на объект типа `NumericFns<Double>`. Благодаря применению метасимвольного аргумента по ссылке на объект `iOb` удастся передать объект `dOb` методу `absEqual()`. Подобным образом формируется и другой вызов, в котором методу передается объект типа `NumericFns<Long>`.

И последнее замечание: не следует забывать, что метасимвольные аргументы не оказывают влияния на тип создаваемого объекта в классе `NumericFns`. Для этой цели служит оператор `extends`, указываемый в объявлении класса `NumericFns`. Метасимвольный аргумент лишь указывает на соответствие *любому* допустимому объекту класса `NumericFns`.

Ограниченные метасимвольные аргументы

Метасимвольные аргументы можно ограничивать таким же образом, как и любой параметр типа. Ограниченные метасимвольные аргументы приобретают особое значение при написании методов, которые должны оперировать только объектами подклассов отдельного суперкласса. Для того чтобы стало понятнее назначение метасимвольных аргументов, обратимся к простому примеру. Допустим, имеется следующий ряд классов:

```
class A {
    // ...
}

class B extends A {
    // ...
}

class C extends A {
    // ...
}

// Обратите внимание на то, что D не является подклассом A.
```

```
class D {
    // ...
}
```

Здесь класс A является суперклассом для классов B и C, но не для класса D.

Теперь рассмотрим очень простой обобщенный класс.

// Простой обобщенный класс.

```
class Gen<T> {
    T ob;

    Gen(T o) {
        ob = o;
    }
}
```

В классе Gen предусмотрен один параметр типа, который определяет тип объекта, хранящегося в переменной ob. Как видите, на тип T не накладывается никаких ограничений. Следовательно, параметр типа T может обозначать любой класс.

А теперь допустим, что требуется создать метод, принимающий аргумент любого типа, соответствующего объекту класса Gen, при условии, что в качестве параметра типа этого объекта указывается класс A или его подклассы. Иными словами, требуется создать метод, который оперирует только объектами типа Gen<тип>, где тип — это класс A или его подклассы. Для этой цели нужно воспользоваться ограниченным метасимвольным аргументом. Ниже для примера приведено объявление метода test(), которому в качестве аргумента может быть передан только объект класса Gen, на параметр типа которого накладываются следующие ограничения: соответствие классу A или его подклассам.

```
// Здесь знак ? устанавливает соответствие
// классу A или производным от него подклассам.
static void test(Gen<? extends A> o) {
    // ...
}
```

А приведенный ниже пример класса демонстрирует типы объектов класса Gen, которые могут быть переданы методу test().

```
class UseBoundedWildcard {
    // Здесь знак ? устанавливает соответствие
    // классу A или производным от него подклассам.

    // В объявлении этого метода используется ограниченный
    // метасимвольный аргумент.
    static void test(Gen<? extends A> o) {
        // ...
    }

    public static void main(String args[]) {
        A a = new A();
        B b = new B();
        C c = new C();
        D d = new D();

        Gen<A> w = new Gen<A>(a);
```

```

Gen<B> w2 = new Gen<B>(b);
Gen<C> w3 = new Gen<C>(c);
Gen<D> w4 = new Gen<D>(d);

// Эти вызовы метода test() допустимы, так как
// объекты w, w2 и w3 относятся к подклассам A.
test(w);
test(w2);
test(w3);

// А этот вызов метода test() недопустим, так как
// объект w4 не относится к подклассу A.
// test(w4); // Ошибка!
}
}

```

В методе `main()` создаются объекты классов `A`, `B`, `C` и `D`. Затем они используются для создания четырех объектов класса `Gen` (по одному на каждый тип). После этого метод `test()` вызывается четыре раза, причем последний его вызов закомментирован. Первые три вызова вполне допустимы, поскольку `w`, `w2` и `w3` являются объектами класса `Gen`, типы которых определяются классом `A` или производными от него классами. А последний вызов метода `test()` недопустим, потому что `w4` — это объект класса `D`, не являющегося производным от класса `A`. Следовательно, ограниченный метасимвольный аргумент в методе `test()` не позволяет передавать ему объект `w4` в качестве параметра.

В целом верхняя граница для метасимвольного аргумента задается в следующей общей форме:

```
<? extends суперкласс >
```

где после ключевого слова `extends` указывается *суперкласс*, т.е. имя класса, определяющего верхнюю границу, включая и его самого. Это означает, что в качестве аргумента допускается указывать не только подклассы данного класса, но и сам этот класс.

По мере необходимости можно также указать нижнюю границу для метасимвольного аргумента. Для этой цели служит ключевое слово `super`, указываемое в следующей общей форме:

```
<? extends подкласс >
```

В данном случае в качестве аргумента допускается использовать только суперклассы, от которых наследует *подкласс*, исключая его самого. Это означает, что *подкласс*, определяющий нижнюю границу, не относится к числу классов, передаваемых в качестве аргумента.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Можно ли привести один экземпляр обобщенного класса к другому?

ОТВЕТ. Да, можно. Но только в том случае, если типы обоих классов совместимы и у них одинаковые аргументы типа. Рассмотрим в качестве примера обобщенный класс `Gen`, объявляемый следующим образом:

```
class Gen<T> { // ...
```

Далее допустим, что переменная `x` объявлена так, как показано ниже.

```
Gen<Integer> x = new Gen<Integer>();
```


В этом случае следующее приведение типов может быть выполнено, поскольку переменная `x` указывает на экземпляр класса `Gen<Integer>`:

```
(Gen<Integer>) x // Допустимо
```

А следующее приведение типов не может быть выполнено, поскольку переменная `x` не указывает на экземпляр класса `Gen<Long>`:

```
(Gen<Long>) x // Недопустимо
```

Обобщенные методы

Как было показано в предыдущих примерах, методы в обобщенных классах могут быть объявлены с параметром типа своего класса, а следовательно, такие методы автоматически становятся обобщенными относительно параметра типа. Но можно также объявить обобщенный метод с одним или несколькими параметрами его собственного типа. Более того, такой метод может быть объявлен в обычном, а не обобщенном классе.

Ниже приведен пример программы, в которой объявляется класс `GenericMethodDemo`, не являющийся обобщенным. В этом классе объявляется статический обобщенный метод `arraysEqual()`, в котором определяется, содержатся ли в двух массивах одинаковые элементы, расположенные в том же самом порядке. Такой метод можно использовать для сравнения двух массивов одинаковых или совместимых между собой типов.

```
// Пример простого обобщенного метода.
```

```
class GenericMethodDemo {
```

```
    // Этот обобщенный метод определяет,
    // совпадает ли содержимое двух массивов.
```

```
    static <T, V extends T> boolean arraysEqual(T[] x, V[] y) {
        // Если массивы имеют разную длину, они не могут быть одинаковыми.
        if(x.length != y.length) return false;
```

```
        for(int i=0; i < x.length; i++)
            if(!x[i].equals(y[i]))
                return false; // Массивы отличаются.
```

```
        return true; // Содержимое массивов совпадает.
```

```
    }
```

```
    public static void main(String args[]) {
```

```
        Integer nums[] = { 1, 2, 3, 4, 5 };
        Integer nums2[] = { 1, 2, 3, 4, 5 };
        Integer nums3[] = { 1, 2, 7, 4, 5 };
        Integer nums4[] = { 1, 2, 7, 4, 5, 6 };
```

```
        // Аргументы типа T и V неявно определяются при вызове метода.
```

```
        if(arraysEqual(nums, nums))
            System.out.println("nums equals nums");
```

```
        if(arraysEqual(nums, nums2))
```

```

        System.out.println("nums equals nums2");

    if(arrayEquals(nums, nums3))
        System.out.println("nums equals nums3");

    if(arrayEquals(nums, nums4))
        System.out.println("nums equals nums4");

    // создать массив объектов типа Double
    Double dvals[] = { 1.1, 2.2, 3.3, 4.4, 5.5 };

    // Следующая строка не будет скомпилирована, так как
    // типы массивов nums и dvals не совпадают.
    // if(arrayEquals(nums, dvals))
    //     System.out.println("nums equals dvals");
    }
}

```

Результат выполнения данной программы выглядит следующим образом:

```

nums equals nums
nums equals nums2

```

Рассмотрим подробнее исходный код метода `arrayEquals()`. Посмотрите прежде всего, как он объявляется:

```

static <T, V extends T> boolean arrayEquals(T[] x, V[] y) {

```

Параметры типа указываются *перед* возвращаемым типом. Обратите далее внимание на то, что верхней границей для типа параметра `V` является тип параметра `T`. Таким образом, тип параметра `V` должен быть таким же, как и у параметра `T`, или же быть его подклассом. Такая связь гарантирует, что при вызове метода `arrayEquals()` могут быть указаны только совместимые друг с другом параметры. И наконец, обратите внимание на то обстоятельство, что метод `arrayEquals()` объявлен как `static`, т.е. его можно вызывать независимо от любого объекта. Но обобщенные методы не обязательно должны быть статическими. В этом смысле на них не накладывается никаких ограничений.

А теперь проанализируем, каким образом метод `arrayEquals()` вызывается в методе `main()`. Для этого используется обычный синтаксис, а параметры типа не указываются. И это становится возможным потому, что типы аргументов данного метода распознаются автоматически, а типы параметров `T` и `V` настраиваются соответствующим образом. Рассмотрим в качестве примера первый вызов метода `arrayEquals()`:

```

if(arrayEquals(nums, nums))

```

В данном случае типом первого аргумента является `Integer`, который и заменяет тип параметра `T`. Таким же является и тип второго аргумента, а следовательно, тип параметра `V` также заменяется на `Integer`. Следовательно, выражение для вызова метода `arrayEquals()` составлено правильно, и оба массива можно сравнить друг с другом.

Обратите далее внимание на следующие закомментированные строки:

```

// if(arrayEquals(nums, dvals))
//     System.out.println("nums equals dvals");

```

Если удалить в них символы комментариев и попытаться скомпилировать программу, то компилятор выдаст сообщение об ошибке. Дело в том, что верхней границей для типа параметра `V` является тип параметра `T`. Этот тип указывается после ключевого сло-

ва `extends`, т.е. тип параметра `V` может быть таким же, как и у параметра `T`, или быть его подклассом. В данном случае типом первого аргумента рассматриваемого здесь метода является `Integer`, заменяющий тип параметра `T`, тогда как типом второго аргумента — `Double`, не являющийся подклассом `Integer`. Таким образом, вызов метода `arraysEqual()` оказывается недопустимым, что и приводит к ошибке при компиляции.

Синтаксис объявления метода `arraysEqual()` может быть обобщен. Ниже приведена общая форма объявления обобщенного метода.

```
<параметры_типа> возвращаемый_тип имя_метода(параметры) { // ...
```

Как и при вызове обычного метода, `параметры_типа` разделяются запятыми. В обобщенном методе их список предваряет `возвращаемый_тип`.

Обобщенные конструкторы

Конструктор может быть обобщенным, даже если сам класс не является таковым. Например, в приведенной ниже программе класс `Summation` не является обобщенным, но в нем используется обобщенный конструктор.

```
// Применение обобщенного конструктора.
class Summation {
    private int sum;

    // Обобщенный конструктор.
    <T extends Number> Summation(T arg) {
        sum = 0;

        for(int i=0; i <= arg.intValue(); i++)
            sum += i;
    }

    int getSum() {
        return sum;
    }
}

class GenConsDemo {
    public static void main(String args[]) {
        Summation ob = new Summation(4.0);

        System.out.println("Summation of 4.0 is " +
                           ob.getSum());
    }
}
```

В классе `Summation` вычисляется и инкапсулируется сумма всех чисел от 0 до `N`, причем значение `N` передается конструктору. Для конструктора `Summation()` указан параметр типа, ограниченный сверху классом `Number`, и поэтому объект типа `Summation` может быть создан с использованием любого числового типа, в том числе `Integer`, `Float` и `Double`. Независимо от используемого числового типа, соответствующее значение преобразуется в тип `Integer` при вызове `intValue()`, а затем вычисляется требуемая сумма. Таким образом, класс `Summation` совсем не обязательно объявлять обобщенным — достаточно сделать обобщенным только его конструктор.

Обобщенные интерфейсы

Наряду с обобщенными классами и методами существуют также обобщенные интерфейсы. Такие интерфейсы определяются подобно обобщенным классам. Их применение демонстрируется в приведенном ниже примере программы. В ней создается интерфейс `Containment`, который может быть реализован классами, хранящими одно или несколько значений. Кроме того, в этой программе объявляется метод `contains()`, в котором определяется, содержится ли указанное значение в текущем объекте.

```
// Пример обобщенного интерфейса.

// В этом интерфейсе подразумевается, что реализующий
// его класс содержит одно или несколько значений.
interface Containment<T> { // обобщенный интерфейс
    // Метод contains() проверяет, содержится ли
    // некоторый элемент в объекте класса,
    // реализующего интерфейс Containment.
    boolean contains(T o);
}

// реализовать интерфейс Containment с помощью массива,
// предназначенного для хранения значений.
// Любой класс, реализующий обобщенный интерфейс,
// также должен быть обобщенным.
class MyClass<T> implements Containment<T> {
    T[] arrayRef;

    MyClass(T[] o) {
        arrayRef = o;
    }

    // реализовать метод contains()
    public boolean contains(T o) {
        for(T x : arrayRef)
            if(x.equals(o)) return true;
        return false;
    }
}

class GenIFDemo {
    public static void main(String args[]) {
        Integer x[] = { 1, 2, 3 };

        MyClass<Integer> ob = new MyClass<Integer>(x);

        if(ob.contains(2))
            System.out.println("2 is in ob");
        else
            System.out.println("2 is NOT in ob");

        if(ob.contains(5))
            System.out.println("5 is in ob");
        else
            System.out.println("5 is NOT in ob");
    }
}
```

```
// Следующие строки кода недопустимы, так как объект ob
// является вариантом реализации интерфейса Containment для
// типа Integer, а значение 9.25 относится к типу Double.
// if(ob.contains(9.25)) // Недопустимо!
// System.out.println("9.25 is in ob");
}
}
```

Выполнение этой программы дает следующий результат:

```
2 is in ob
5 is NOT in ob
```

Большую часть исходного кода этой программы нетрудно понять, но на некоторых ее особенностях следует все же остановиться. Обратите прежде всего внимание на то, как объявляется интерфейс `Containment`:

```
interface Containment<T> {
```

Обобщенные интерфейсы объявляются таким же образом, как и обобщенные классы. В данном случае параметр типа `T` задает тип включаемого объекта.

Интерфейс `Containment` реализуется классом `MyClass`. Объявление этого класса выглядит следующим образом:

```
class MyClass<T> implements Containment<T> {
```

Если класс реализует обобщенный интерфейс, то он также должен быть обобщенным. В нем должен быть объявлен как минимум тот же параметр типа, который указан в объявлении интерфейса. Например, приведенный ниже вариант объявления класса `MyClass` недопустим.

```
class MyClass implements Containment<T> { // Ошибка!
```

В данном случае ошибка заключается в том, что в классе `MyClass` не объявлен параметр типа, а это означает, что передать параметр типа интерфейсу `Containment` нельзя. Если идентификатор `T` останется неизвестным, компилятор выдаст сообщение об ошибке. Класс, реализующий обобщенный интерфейс, может не быть обобщенным только в одном случае: если при объявлении класса для интерфейса указывается конкретный тип. Такой способ объявления класса приведен ниже.

```
class MyClass implements Containment<Double> { // Допустимо
```

Вас теперь вряд ли удивит, что один или несколько параметров типа для универсального интерфейса могут быть ограничены. Это позволяет указывать, какие именно типы данных допустимы для интерфейса. Так, если требуется запретить передачу интерфейсу `Containment` значений, не являющихся числовыми, для этой цели интерфейс можно объявить следующим образом:

```
interface Containment<T extends Number> {
```

Теперь любой класс, реализующий интерфейс `Containment`, должен передавать ему значение типа, удовлетворяющее указанным выше ограничениям. Например, класс `MyClass`, реализующий данный интерфейс, должен объявляться следующим образом:

```
class MyClass<T extends Number> implements Containment<T> {
```

Обратите особое внимание на то, как параметр типа `T` объявляется в классе `MyClass`, а затем передается интерфейсу `Containment`. На этот раз интерфейсу `Containment` требуется тип, расширяющий тип `Number`, поэтому в классе `MyClass`, реализующем этот интерфейс, должны быть указаны соответствующие ограничения. Если верхняя граница

задана в объявлении класса, то ее нет необходимости указывать еще раз в операторе `implements`. Если же попытаться сделать это, будет получено сообщение об ошибке. Например, следующее выражение составлено неверно и не будет скомпилировано:

```
// Ошибка!
class MyClass<T extends Number>
    implements Containment<T extends Number> {
```

Если параметр типа задан в объявлении класса, он лишь передается интерфейсу без дальнейших видоизменений.

Ниже приведена общая форма объявления обобщенного интерфейса.

```
interface имя_интерфейса<параметры_типа> { // ...
```

где *параметры_типа* указываются списком через запятую. При реализации обобщенного интерфейса в объявлении класса также должны быть указаны параметры типа. Общая форма объявления класса, реализующего обобщенный интерфейс, приведена ниже.

```
class имя_класса<параметры_типа>
    implements имя_интерфейса<параметры_типа> {
```

Пример для опробования 13.1.

Создание обобщенного класса очереди

Исходный файл: IGenQ.java,
QExc.java, GenQueue.java,
GenQDemo.java

Главным преимуществом обобщенных классов является возможность создания надежного кода, пригодного для повторного использования. Как пояснялось в начале главы, многие алгоритмы могут быть реализованы одинаково независимо от типа данных. Например, очередь в равной степени пригодна для хранения целых чисел, символьных строк, объектов типа `File` и других типов данных. Вместо того чтобы создавать отдельный класс очереди для объектов каждого типа, можно разработать единое обобщенное решение, пригодное для обращения с объектами любого типа. В итоге цикл проектирования, программирования, тестирования и отладки кода будет выполняться только один раз, не повторяясь всякий раз, когда потребуется организовать очередь для нового типа данных.

В этом проекте предстоит в очередной и последний раз видоизменить класс очереди, разработка которого была впервые начата в главе 5. Для этой цели будет объявлен обобщенный интерфейс, определяющий операции над очередью, созданы два класса исключений и реализована очередь фиксированного размера. Разумеется, вам ничто не мешает поэкспериментировать с другими разновидностями обобщенных очередей, например, создать динамическую или циклическую очередь, следуя приведенным ниже рекомендациям.

Кроме того, исходный код, реализующий очередь в этом проекте, будет организован в виде ряда отдельных файлов. С этой целью код интерфейса, исключений, реализации очереди фиксированного размера и программы, демонстрирующей очередь в действии, будет распределен по отдельным исходным файлам. Такая организация исходного кода отвечает подходу, принятому в работе над большинством реальных проектов.

Последовательность действий

1. Первым этапом создания обобщенной очереди станет формирование обобщенного интерфейса, описывающего две операции над очередью: размещение и извлечение. Обобщенная версия интерфейса очереди будет называться `IGenQ`, ее исходный код приведен ниже. Введите этот код во вновь созданный файл `IGenQ.java`.

```
// Обобщенный интерфейс очереди.
public interface IGenQ<T> {
    // поместить элемент в очередь
    void put(T ch) throws QueueFullException;

    // извлечь элемент из очереди
    T get() throws QueueEmptyException;
}
```

Обратите внимание на то, что тип данных, предназначенных для хранения в очереди, определяется параметром типа T.

2. Далее создайте файл QExc.java. Введите в него два приведенных ниже класса, в которых определяются исключения, возникающие в работе с очередью.

```
// Исключение в связи с ошибками переполнения очереди.
class QueueFullException extends Exception {
    int size;

    QueueFullException(int s) { size = s; }

    public String toString() {
        return "\nQueue is full. Maximum size is " +
            size;
    }
}

// Исключение в связи с ошибками опустошения очереди.
class QueueEmptyException extends Exception {

    public String toString() {
        return "\nQueue is empty.";
    }
}
```

В этих классах определяются две ошибки, которые могут возникнуть в работе с очередью: попытки поместить элемент в заполненную очередь и извлечь элемент из пустой очереди. Эти классы не являются обобщенными, поскольку они действуют одинаково, независимо от типа данных, хранящихся в очереди.

3. Создайте файл GenQueue.java. Введите в него приведенный ниже код, в котором реализуется очередь фиксированного размера.

```
// Обобщенный класс, реализующий очередь фиксированного размера.
class GenQueue<T> implements IGenQ<T> {
    private T q[]; // Массив для хранения элементов очереди.
    private int putloc, getloc; // Индексы размещения и извлечения
    // элементов очереди.

    // построить пустую очередь из заданного массива
    public GenQueue(T[] aRef) {
        q = aRef;
        putloc = getloc = 0;
    }

    // поместить элемент в очередь
```

```

public void put(T obj)
    throws QueueFullException {
    if(putloc==q.length-1)
        throw new QueueFullException(q.length-1);

    putloc++;
    q[putloc] = obj;
}

// извлечь элемент из очереди
public T get()
    throws QueueEmptyException {

    if(getloc == putloc)
        throw new QueueEmptyException();

    getloc++;
    return q[getloc];
}
}

```

Класс `GenQueue` объявляется как обобщенный с параметром типа `T`. Этот параметр определяет тип данных, хранящихся в очереди. Обратите внимание на то, что параметр типа `T` также передается интерфейсу `IGenQ`.

Конструктору `GenQueue` передается ссылка на массив, используемый для хранения элементов очереди. Следовательно, для построения объекта класса `GenQueue` нужно сначала сформировать массив, тип которого будет совместим с типом объектов, сохраняемых в очереди, а его размер достаточен для размещения этих объектов в очереди. В рассматриваемом здесь коде первый элемент массива не используется, поэтому длина массива должна быть на один элемент больше, чем количество элементов, которые допускается хранить в очереди.

Например, в следующих строках кода демонстрируется создание очереди для хранения символьных строк:

```

String strArray[] = new String[10];
GenQueue<String> strQ = new GenQueue<String>(strArray);

```

4. Создайте файл `GenQDemo.java` и введите в него приведенный ниже код. В этом коде демонстрируется работа обобщенной очереди.

```

/*
    Проект 13.1.

    Демонстрация обобщенного класса очереди.
*/

class GenQDemo {
    public static void main(String args[]) {
        // создать очередь для хранения целых чисел
        Integer iStore[] = new Integer[10];
        GenQueue<Integer> q = new GenQueue<Integer>(iStore);

        Integer iVal;
    }
}

```



```

System.out.println("Demonstrate a queue of Integers.");
try {
    for(int i=0; i < 5; i++) {
        System.out.println("Adding " + i + " to the q.");
        q.put(i); // ввести целочисленное значение в очередь q
    }
}
catch (QueueFullException exc) {
    System.out.println(exc);
}
System.out.println();

try {
    for(int i=0; i < 5; i++) {
        System.out.print("Getting next Integer from q: ");
        iVal = q.get();
        System.out.println(iVal);
    }
}
catch (QueueEmptyException exc) {
    System.out.println(exc);
}

System.out.println();

// создать очередь для хранения чисел с плавающей точкой
Double dStore[] = new Double[10];
GenQueue<Double> q2 = new GenQueue<Double>(dStore);

Double dVal;

System.out.println("Demonstrate a queue of Doubles.");
try {
    for(int i=0; i < 5; i++) {
        System.out.println("Adding " + (double)i/2 +
                           " to the q2.");
        q2.put((double)i/2); // ввести значение типа double в очередь q2
    }
}
catch (QueueFullException exc) {
    System.out.println(exc);
}
System.out.println();

try {
    for(int i=0; i < 5; i++) {
        System.out.print("Getting next Double from q2: ");
        dVal = q2.get();
        System.out.println(dVal);
    }
}
catch (QueueEmptyException exc) {
    System.out.println(exc);
}

```

```

    }
}

```

5. Скомпилируйте программу и запустите ее на выполнение. В итоге на экране отобразится следующий результат:

```

Demonstrate a queue of Integers.
Adding 0 to the q.
Adding 1 to the q.
Adding 2 to the q.
Adding 3 to the q.
Adding 4 to the q.

```

```

Getting next Integer from q: 0
Getting next Integer from q: 1
Getting next Integer from q: 2
Getting next Integer from q: 3
Getting next Integer from q: 4

```

```

Demonstrate a queue of Doubles.
Adding 0.0 to the q2.
Adding 0.5 to the q2.
Adding 1.0 to the q2.
Adding 1.5 to the q2.
Adding 2.0 to the q2.

```

```

Getting next Double from q2: 0.0
Getting next Double from q2: 0.5
Getting next Double from q2: 1.0
Getting next Double from q2: 1.5
Getting next Double from q2: 2.0

```

6. Попробуйте самостоятельно сделать обобщенными классы `CircularQueue` и `DynQueue`, созданные в примере для опробования 8.1.

Базовые типы и устаревший код

Обобщенные типы появились лишь в версии JDK 5, поэтому в Java нужно было принять меры для обеспечения совместимости с созданным ранее и впоследствии устаревшим кодом. Короче говоря, устаревший код нужно было оставить вполне работоспособным и совместимым с обобщениями. А это означало, что устаревший код должен был нормально взаимодействовать с обобщениями, и наоборот.

Для обеспечения перехода к обобщенным типам в Java предусмотрена возможность создания обобщенных классов, используемых без указания аргументов типа. Это подразумевает создание для класса *базового типа*, иногда еще называемого *сырым*. Такой тип совместим с устаревшим кодом, в котором ничего не известно об обобщенных классах. Единственный недостаток базового типа заключается в том, что при его использовании теряются преимущества типовой безопасности, присущие обобщениям.

Ниже приведен пример программы, демонстрирующий применение базового типа.

```

// Применение базового типа.
class Gen<T> {

```

```

T ob; // объявить объект типа T

// передать конструктору ссылку на объект типа T
Gen(T o) {
    ob = o;
}

// вернуть объект ob
T getob() {
    return ob;
}
}

// продемонстрировать использование базового типа
class RawDemo {
    public static void main(String args[]) {

        // создать объект класса Gen для типа Integer
        Gen<Integer> iOb = new Gen<Integer>(88);

        // создать объект класса Gen для типа String
        Gen<String> strOb = new Gen<String>("Generics Test");

        // создать объект класса Gen для базового типа
        // и передать ему значение типа Double
        // Если аргумент типа не передается, создается базовый тип.
        Gen raw = new Gen(new Double(98.6));

        // Здесь требуется приведение типов, так как тип неизвестен.
        double d = (Double) raw.getob();
        System.out.println("value: " + d);

        // Использование базового типа может привести
        // к исключениям при выполнении программы.
        // Ниже представлен ряд тому примеров.

        // Следующее приведение типов вызывает ошибку
        // при выполнении программы!
        // int i = (Integer) raw.getob();

        // Следующее присваивание чревато ошибкой.
        // Базовые типы отменяют типовую безопасность.
        strOb = raw; // Допустимо, но потенциально ошибочно.
        // String str = strOb.getob(); // Ошибка при выполнении программы.

        // Следующее присваивание также чревато ошибкой.
        raw = iOb; // Допустимо, но потенциально ошибочно.
        // d = (Double) raw.getob(); // Ошибка при выполнении программы.
    }
}

```

У этой программы имеется ряд интересных особенностей. Прежде всего, базовый тип обобщенного класса Gen создается в следующем объявлении:

```
Gen raw = new Gen(new Double(98.6));
```

В данном случае аргументы типа не указываются. В итоге создается объект класса Gen, тип T которого замещается типом Object.

Базовые типы не обеспечивают типовую безопасность. Переменной базового типа может быть присвоена ссылка на любой тип объекта класса Gen. Справедливо и обратное: переменной конкретного типа из класса Gen может быть присвоена ссылка на объект класса Gen базового типа. Обе операции потенциально опасны, поскольку они действуют в обход механизма проверки типов, обязательной для обобщений.

Недостаточный уровень типовой безопасности демонстрируют примеры в закомментированных строках кода в конце данной программы. Рассмотрим их по отдельности. Сначала проанализируем следующую строку кода:

```
// int i = (Integer) raw.getob();
```

В этом операторе присваивания в объекте raw определяется значение переменной ob, которое приводится к типу Integer. Но дело в том, что в объекте raw содержится не целое число, а значение типа Double. На стадии компиляции этот факт выявить невозможно, поскольку тип объекта raw неизвестен. Следовательно, ошибка возникнет на стадии выполнения программы.

В следующих строках кода ссылка на объект класса Gen базового типа присваивается переменной strOb (ссылки на объект типа Gen<String>).

```
strOb = raw; // Допустимо, но потенциально ошибочно.  
// String str = strOb.getob(); // Ошибка при выполнении программы.
```

Само по себе присваивание синтаксически правильно, но логически сомнительно. Переменная strOb ссылается на объект типа Gen<String>, а следовательно, она должна содержать ссылку на объект, содержащий значение типа String, но после присваивания объект, на который ссылается переменная strOb, содержит значение типа Double. Поэтому при выполнении программы, когда предпринимается попытка присвоить переменной str содержимое объекта, на который ссылается переменная strOb, возникает ошибка. А причина ее заключается в том, что переменная strOb ссылается в этот момент на объект, содержащий значение типа Double. Таким образом, присваивание ссылки на объект базового типа переменной ссылки на объект обобщенного типа делается в обход механизма типовой безопасности.

В следующих строках кода демонстрируется ситуация, обратная предыдущей:

```
raw = iOb; // Допустимо, но потенциально ошибочно.  
// d = (Double) raw.getob(); // Ошибка при выполнении программы.
```

В данном случае ссылка на объект обобщенного типа присваивается переменной базового типа. И это присваивание синтаксически правильно, но приводит к ошибке, возникающей во второй строке кода. В частности, переменная raw указывает на объект, содержащий значение типа Integer, но при приведении типов предполагается, что он содержит значение типа Double. И эту ошибку также нельзя выявить на стадии компиляции, а проявляется она при выполнении программы.

В связи с тем что использование базовых типов сопряжено с потенциальными ошибками, проявляющимися при выполнении программы, компилятор javac выводит в подобных случаях сообщения, предупреждающие об *отсутствии проверки типов*, когда базовый тип используется в обход механизма типовой безопасности. Такие сообщения будут получены и при компиляции рассматриваемой здесь программы. Причиной их появления станут следующие строки кода:

```
Gen raw = new Gen(new Double(98.6));
```

```
strOb = raw; // Допустимо, но потенциально ошибочно.
```

В первой из этих строк кода содержится обращение к конструктору класса `Gen` без указания аргумента типа, что приводит к выдаче компилятором соответствующего предупреждения. А при компиляции второй строки предупреждающее сообщение возникнет из-за попытки присвоить переменной, ссылающейся на объект обобщенного типа, ссылки на объект базового типа.

На первый взгляд может показаться, что предупреждение об отсутствии проверки типов может вызвать и приведенная ниже строка кода, но этого не произойдет.

```
raw = iOb; // Допустимо, но потенциально ошибочно.
```

В этом случае компилятор не предупреждает пользователя, потому что такое присваивание не приводит к еще *большей* потере типовой безопасности по сравнению с той, которая произошла при создании переменной `raw` базового типа.

Из всего изложенного выше можно сделать следующий вывод: пользоваться базовыми типами следует весьма ограниченно и только в тех случаях, когда устаревший код требуется объединить с новым, обобщенным кодом. Базовые типы — лишь средство для обеспечения совместимости с устаревшим кодом. Они не должны присутствовать во вновь разрабатываемом коде.

Выводимость типов с помощью ромбовидного оператора

Начиная с версии JDK 7 появилась возможность сократить синтаксис, применяемый для создания экземпляра обобщенного типа. В качестве примера обратимся к классу `TwoGen`, представленному в начале этой главы. Ниже для удобства приведена часть его объявления. Обратите внимание на то, что в нем определяются два обобщенных типа данных.

```
class TwoGen<T, V> {
    T ob1;
    V ob2;
    // передать конструктору ссылку на объект типа T.
    TwoGen(T o1, V o2) {
        ob1 = o1;
        ob2 = o2;
    }
    // ...
}
```

До появления версии JDK 7 для создания экземпляра класса `TwoGen` приходилось прибегать к оператору присваивания, аналогичному приведенному ниже.

```
TwoGen<Integer, String> tgOb =
    new TwoGen<Integer, String>(42, "testing");
```

Здесь аргументы типа (в данном случае — `Integer` и `String`) указываются дважды: сначала при объявлении переменной `tgOb`, а затем при создании экземпляра класса `TwoGen` с помощью оператора `new`. Начиная с версии JDK 5, в которой были внедрены обобщения, подобная форма создания объектов обобщенного типа требовалась при написании всех программ на Java, вплоть до появления версии JDK 7. И хотя в этой

форме, по существу, нет ничего ошибочного, все же она выглядит несколько многословной. Ведь в операторе `new` типы аргументов могут быть выведены без особого труда, и поэтому нет никаких оснований указывать их еще раз. И для этой цели в версии JDK 7 внедрен специальный синтаксический элемент.

В версии JDK 7 приведенное выше объявление может быть переписано следующим образом:

```
TwoGen<Integer, String> tgOb = new TwoGen<>(42, "testing");
```

Как видите, в той части, где создается экземпляр объекта обобщенного типа, просто указываются угловые скобки (`< >`), обозначающие пустой список аргументов типа. Это так называемый *ромбовидный* оператор, предписывающий компилятору вывести типы аргументов, требующиеся конструктору в операторе `new`. Главное преимущество такого синтаксиса вывода типов состоит в том, что он позволяет сделать более краткими те операторы объявления, которые порой выглядят слишком длинными и громоздкими. Это оказывается особенно удобным для объявления обобщенных типов, определяющих границы наследования в иерархии классов Java.

Приведенную выше форму объявления экземпляра класса можно обобщить. Для вывода типов служит следующая общая синтаксическая форма объявления обобщенной ссылки и создания экземпляра объекта обобщенного типа:

```
class-name<аргументы_типа> имя_переменной =  
    new class-name< >(аргументы_конструктора);
```

Здесь список аргументов типа в операторе `new` указывается пустым.

В операторах объявления выводение типов может, как правило, применяться и при передаче параметров. Так, если объявить в классе `TwoGen` следующий метод:

```
boolean isSame(TwoGen<T, V> o) {  
    if(ob1 == o.ob1 && ob2 == o.ob2) return true;  
    else return false;  
}
```

то приведенный ниже вызов окажется вполне допустимым в версии JDK 7.

```
if(tgOb.isSame(new TwoGen<>(42, "testing"))) System.out.println("Same");
```

В данном случае опускаются аргументы типа, которые должны передаваться методу `isSame()`. Их типы могут быть выведены автоматически, а следовательно, указывать их еще раз не нужно.

Ромбовидный оператор является новым языковым средством в версии JDK 7, но непригодным для компиляции кода, написанного в предыдущих версиях Java. Поэтому в примерах программ, приведенных далее в этой книге, будет использоваться прежний несокращенный синтаксис объявления экземпляров обобщенных классов, чтобы эти программы можно было скомпилировать любым компилятором Java, поддерживающим обобщения. Кроме того, несокращенный синтаксис объявления экземпляров обобщенных классов яснее дает понять, что именно создается, и благодаря этому представленные в книге примеры становятся более наглядными и полезными. Разумеется, в своих программах на Java вы вольны пользоваться синтаксисом вывода типов, чтобы упростить их объявления.

Стирание

Как правило, программирующему на Java совсем не обязательно знать во всех подробностях, каким образом компилятор преобразует исходный код программы в объект-

ный. Но что касается обобщенных типов, то важно иметь хотя бы общее представление о процессе их преобразования. Это помогает лучше понять, почему обобщенные классы и методы действуют именно так, а не иначе, и почему их поведение порой ставит в тупик непосвященных. Поэтому ниже поясняется вкратце, каким образом обобщенные типы реализуются в Java.

При внедрении обобщенных типов в Java пришлось учитывать следующее важное условие, накладывавшее определенные ограничения на их реализацию: совместимость с предыдущими версиями Java. Иными словами, обобщенный код должен был быть совместим с предыдущими версиями кода, на момент написания которого обобщенные типы еще не были доступны. Таким образом, любые изменения в синтаксисе языка Java или механизме JVM не должны были оказывать влияние на уже существующий код. Поэтому для реализации обобщенных типов с учетом указанных ограничений был выбран механизм, получивший название *стирание*.

Механизм стирания действует следующим образом. При компиляции кода, написанного на Java, все сведения об обобщенных типах удаляются, т.е. стираются. Это означает, что параметры типа заменяются верхними границами их типа, а если границы не указаны, то их функции выполняет класс `Object`. После этого производится приведение типов, определяемых аргументами типа. Подобная совместимость типов соблюдается компилятором. Благодаря такому подходу к реализации обобщений параметры типа при выполнении программы вообще отсутствуют, но действуют только на стадии компиляции исходного кода.

Ошибки неоднозначности

Появление обобщенных типов стало причиной возникновения новой разновидности ошибок, связанных с *неоднозначностью*. Ошибки неоднозначности возникают в тех случаях, когда в результате стирания два, на первый взгляд, отличающихся объявления обобщенных типов преобразуются в один тип, вызывая тем самым конфликтную ситуацию. Рассмотрим пример, в котором используется перегрузка методов.

```
// Неоднозначность, вызванная стиранием перегруженных методов.
class MyGenClass<T, V> {
    T ob1;
    V ob2;

    // ...

    // Два следующих метода конфликтуют друг с другом,
    // поэтому код не компилируется.
    // Эти методы существенно неоднозначны.
    void set(T o) {
        ob1 = o;
    }

    void set(V o) {
        ob2 = o;
    }
}
```

В классе `MyGenClass` объявлены два обобщенных типа: `T` и `V`. В этом классе предпринимается попытка перегрузить метод `set()`. Перегружаемые методы отличаются па-

параметрами типа *T* и *V*. Казалось бы, это не должно приводить к ошибке, поскольку типы *T* и *V* отличаются. Но здесь возникают два затруднения в связи с неоднозначностью.

Прежде всего, в классе `MyGenClass` не указано никаких требований, чтобы типы *T* и *V* действительно отличались. В частности, не является принципиальной ошибкой создание объекта типа `MyGenClass` так, как показано ниже.

```
MyGenClass<String, String> obj = new MyGenClass<String, String>()
```

В данном случае типы *T* и *V* заменяются типом `String`. В результате оба варианта метода `set()` становятся совершенно одинаковыми, что, безусловно, считается ошибкой.

Второе, более серьезное затруднение возникает в связи с тем, что при стирании обобщенных типов оба варианта метода `set()` преобразуются в следующий вид:

```
void set(Object o) { // ...
```

Таким образом, попытка перегрузить метод `set()` в классе `MyGenClass` является существенно неоднозначной. В качестве выхода из этого затруднительного положения может стать отказ от перегрузки и использование двух разных имен методов.

Ограничения, накладываемые на обобщения

На обобщения накладывается ряд ограничений, которые следует учитывать при их употреблении в программах на Java. К числу подобных ограничений относится создание объектов, определяемых параметром типа, использование статических членов класса, генерирование исключений и обращение с массивами. Рассмотрим все эти ограничения более подробно.

Невозможность получить экземпляры параметров типа

Экземпляр параметра типа получить невозможно. Рассмотрим в качестве примера следующий класс:

```
// Экземпляр типа T получить нельзя.
class Gen<T> {
    T ob;
    Gen() {
        ob = new T(); // Недопустимо!!!
    }
}
```

В данном примере любая попытка получить экземпляр типа *T* приводит к ошибке. Ее причину понять нетрудно: компилятору ничего не известно о типе создаваемого объекта, поскольку тип *T* является заполнителем, который стирается во время компиляции.

Ограничения, накладываемые на статические члены класса

В статических членах нельзя использовать параметры типа, объявленные в содержащем их классе. Так, все объявления статических членов в приведенном ниже классе недопустимы.

```
class Wrong<T> {
    // Статическую переменную типа T создать нельзя.
    static T ob;

    // В статическом методе нельзя использовать тип T.
    static T getob() {
```



```

    return ob;
}

// Статический метод не может обращаться к объекту типа T.
static void showob() {
    System.out.println(ob);
}
}

```

Несмотря на наличие описанного выше ограничения, допускается все же объявлять обобщенные статические методы, в которых используются собственные параметры типа. Примеры таких объявлений приводились ранее в этой главе.

Ограничения, накладываемые на обобщенные массивы

На массивы обобщенного типа накладываются два существенных ограничения. Во-первых, нельзя получить экземпляр массива, тип элементов которого определяется параметром типа. И во-вторых, нельзя создать массив обобщенных ссылок на объекты конкретного типа. Оба эти ограничения демонстрируются в приведенном ниже кратком примере программы.

```

// Обобщенные типы и массивы.
class Gen<T extends Number> {
    T ob;

    T vals[]; // Допустимо.

    Gen(T o, T[] nums) {
        ob = o;

        // Следующее выражение недопустимо:
        // vals = new T[10]; // Нельзя создать массив типа T.

        // Следующее выражение составлено верно.
        vals = nums; // Переменной допускается присваивать ссылку
                     // на существующий массив.
    }
}

class GenArrays {
    public static void main(String args[]) {
        Integer n[] = { 1, 2, 3, 4, 5 };

        Gen<Integer> iOb = new Gen<Integer>(50, n);

        // Нельзя создать массив обобщенных ссылок
        // на объекты конкретного типа.
        // Gen<Integer> gens[] = new Gen<Integer>[10]; // Ошибка!

        // Следующее выражение составлено верно.
        Gen<?> gens[] = new Gen<?>[10];
    }
}

```

Как следует из исходного кода приведенной выше программы, допускается создавать ссылку на массив типа `T`. Это демонстрируется в следующей строке кода:

```
T vals[]; // Допустимо.
```

Но получить экземпляр самого массива типа `T` нельзя. Именно поэтому приведенная ниже строка кода закомментирована.

```
// vals = new T[10]; // Нельзя создать массив типа T.
```

В данном случае ограничение на массив типа `T` состоит в том, что компилятору не известно, какого типа массив следует создавать на самом деле. Но в то же время конструктору `Gen()` можно передать ссылку на массив совместимого типа при создании объекта, а также присвоить это значение переменной `vals`. Примером тому может служить следующая строка кода:

```
vals = nums; // Переменной допускается присваивать ссылку
              // на существующий массив.
```

Это выражение составлено верно, поскольку тип массива, передаваемого конструктору `Gen()` при создании объекта, известен и совпадает с типом `T`. В теле метода `main()` содержится выражение, демонстрирующее невозможность объявить массив обобщенных ссылок на объекты конкретного типа. Поэтому приведенная ниже строка кода не будет скомпилирована.

```
// Gen<Integer> gens[] = new Gen<Integer>[10]; // Ошибка!
```

Ограничения, накладываемые на обобщенные исключения

Обобщенный класс не может расширять класс `Throwable`. Это означает, что создавать обобщенные классы исключений нельзя.

Дальнейшее изучение обобщений

Как пояснялось в начале этой главы, приведенных в ней сведений достаточно для того, чтобы эффективно пользоваться обобщениями в создаваемых на Java программах. Но у обобщений имеется немало особенностей, которые не нашли отражения в этой главе.

Читатели, которых заинтересовала эта тема, вероятно, захотят узнать побольше о том влиянии, которое обобщения оказывают на иерархию классов и, в частности, каким образом осуществляется сравнение типов при выполнении программы, как переопределяются методы и т.д. Все эти и многие другие вопросы употребления обобщений подробно освещены в книге *Java. Полное руководство*, 8-е издание, ИД “Вильямс”, 2012 г.

Упражнение для самопроверки по материалу главы 13

- Обобщения очень важны, поскольку они позволяют создавать код, который:
 - обеспечивает типовую безопасность;
 - пригоден для повторного использования;
 - отличается высокой надежностью;
 - обладает всеми перечисленными выше свойствами.
- Можно ли указывать простой тип в качестве аргумента типа?
- Как объявить класс `FlightSched` с двумя параметрами типа?

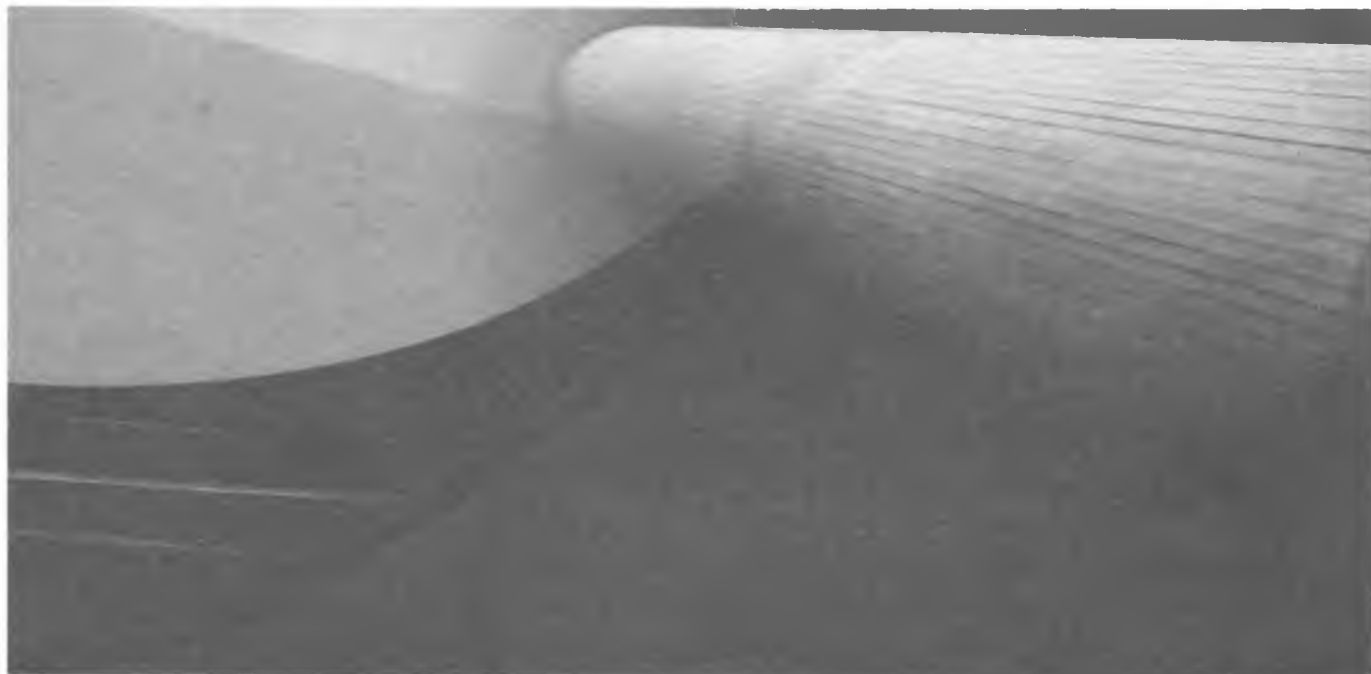
4. Измените ваш ответ на вопрос 3 таким образом, чтобы второй параметр типа обозначал подкласс, производный от класса `Thread`.
5. Внесите изменения в класс `FlightSched` таким образом, чтобы второй параметр типа стал подклассом первого параметра типа.
6. Что обозначает знак `?` в обобщениях?
7. Может ли метасимвольный аргумент быть ограниченным?
8. У обобщенного метода `MyGen()` имеется один параметр типа, определяющий тип передаваемого ему аргумента. Этот метод возвращает также объект, тип которого соответствует параметру типа. Как должен быть объявлен метод `MyGen()`?
9. Допустим, обобщенный интерфейс объявлен так, как показано ниже.

```
interface IGenIF<T, V extends T> { // ...
```

Составьте объявление класса `MyClass`, который реализует интерфейс `IGenIF`.

10. Допустим, имеется обобщенный класс `Counter<T>`. Как создать объект его базового типа?
11. Существуют ли параметры типа на стадии выполнения программы?
12. Видоизмените ответ на вопрос 10 в упражнении по материалу главы 9 таким образом, чтобы сделать класс обобщенным. По ходу дела создайте интерфейс стека `IGenStack`, объявив в нем обобщенные методы `push()` и `pop()`.
13. Что обозначают угловые скобки `< >`?
14. Как упростить приведенную ниже строку кода в версии JDK 7?

```
MyClass<Double,String> obj = new MyClass<Double,String>(1.1,"Hi");
```



Глава 14

**Апплеты, события
и прочее**

Основные навыки и понятия

- Представление об апплетах
- Архитектура апплетов
- Создание заготовок апплетов
- Инициализация и завершение апплетов
- Перерисовка содержимого окон апплетов
- Вывод данных в окне состояния
- Передача параметров апплетам
- Представление о классе Applet
- Применение модели делегирования событий
- Описание остальных ключевых слов Java

Цель данной книги — поведать читателю об основных элементах Java, и мы близки к завершению этого повествования. Тринадцать предыдущих глав были посвящены тем средствам, которые определяют Java как язык программирования, в том числе ключевым словам, синтаксису, блочной структуре, правилам преобразования типов и т.д. Теперь у вас должно быть достаточно знаний, чтобы писать реальные программы на Java. Но еще остаются вопросы программирования, которые невозможно разрешить с помощью ключевых слов, и поэтому приходится обращаться к классам API и специальным технологиям. В этой главе речь пойдет об апплетах и событиях.

Следует, однако, иметь в виду, что темы апплетов и событий довольно обширны, а полное и подробное их изложение выходит за рамки этой книги. Поэтому апплеты и события будут рассмотрены в этой главе лишь в самых общих чертах, хотя и проиллюстрированы на ряде примеров. Но не следует забывать, что это только самые основы. И тем не менее, усвоив эти основы, вы сможете опереться на них как на прочный фундамент для пополнения своих знаний и совершенствования навыков программирования на Java.

В конце этой главы приводится описание ключевых слов Java, которые еще не рассматривались в данной книге, например `instanceof` и `native`. Эти языковые средства предназначены для решения более сложных задач программирования на Java, а здесь они рассматриваются ради полноты представления об этом языке.

Общее представление об апплетах

Апплеты существенно отличаются от всех программ, примеры которых были рассмотрены в предыдущих главах. Как пояснялось в главе 1, апплеты — это небольшие прикладные программы, предназначенные для передачи через Интернет и выполнения в веб-браузере. В виртуальной машине Java поддерживаются все типы программ на Java, в том числе и апплеты, что дает возможность обеспечить достаточный уровень безопасности при динамической загрузке апплетов и последующем их выполнении в веб-браузере.

Прежде всего, апплеты подразделяются на две категории: те, что создаются на основе библиотеки Abstract Window Toolkit (AWT), и те, что создаются на основе библиотеки Swing. В апплетах обеих категорий поддерживается графический пользовательский интерфейс. Причем AWT позволяет работать с обычными элементами пользовательского интерфейса, а Swing — с альтернативными им упрощенными компонентами. В этой главе рассматриваются апплеты, создаваемые на основе библиотеки AWT, тогда как библиотеке Swing посвящена глава 15. Следует, однако, иметь в виду, что в апплетах на основе Swing используется та же самая архитектура, что и в апплетах на основе AWT. Более того, библиотека Swing построена на основе библиотеки AWT. Поэтому сведения и приемы программирования, представленные в этой главе, в равной степени относятся к обеим категориям апплетов.

Перед тем как приступить к подробному изложению теоретических вопросов, рассмотрим простой пример апплета. Он выполняет лишь одно действие: отображает в окне строку "Java makes applets easy" (Java упрощает создание апплетов).

```
// Апплет с минимальными функциональными возможностями.
import java.awt.*; // Обратите внимание на эти операторы.
import java.applet.*;

public class SimpleApplet extends Applet {
    public void paint(Graphics g) {
        // Вывод символьной строки в окне апплета.
        g.drawString("Java makes applets easy.", 20, 20);
    }
}
```

В начале апплета находятся два оператора `import`. С помощью первого из них импортируются классы из библиотеки Abstract Window Toolkit. Благодаря тому что в состав этой библиотеки входят средства поддержки оконного графического пользовательского интерфейса, апплеты (прямо или косвенно) взаимодействуют с пользователями именно через такой интерфейс, а не через консольный ввод-вывод. Как и следовало ожидать, размеры библиотеки AWT достаточно велики, поэтому подробное обсуждение всех ее возможностей выходит за рамки этой книги. Но поскольку в этой главе речь идет о создании лишь самых простых апплетов, то средства AWT будут использоваться в ней очень ограниченно. Во втором операторе `import` импортируется пакет `applet`, в который входит класс `Applet`. Любой апплет должен быть (прямо или косвенно) подклассом, производным от этого класса.

В следующей строке кода объявляется класс `SimpleApplet`, который должен быть открытым (`public`), поскольку доступ к нему будет осуществляться извне.

В классе `SimpleApplet` объявляется метод `paint()`. Этот метод является членом класса `Component` из библиотеки AWT, являющегося суперклассом по отношению к классу `Applet`, и должен переопределяться в апплете. Метод `paint()` вызывается всякий раз, когда в апплете должны быть обновлены выводимые данные. Такой вызов может быть сделан по разным причинам. Например, окно, в котором присутствует апплет, может быть перекрыто другим окном, а затем снова переведено на передний план. Окно может быть также свернуто, а затем развернуто до обычных размеров. Метод `paint()` всегда вызывается в начале выполнения апплета. Этому методу передается один параметр, который представляет собой объект типа `Graphics`. В этом объекте содержится графический контекст, описывающий среду, в которой выполняется апплет. Этот контекст используется в тех случаях, когда требуется вывести данные в окне апплета.

В теле метода `paint()` вызывается метод `drawString()`, который является членом класса `Graphics`. Этот метод служит для вывода текстовой строки в окне апплета, начиная с точки с координатами X, Y . Общая форма его объявления имеет следующий вид:

```
void drawString(String сообщение, int x, int y)
```

где *сообщение* — это символьная строка, которая должна быть выведена начиная с точки, имеющей координаты x, y . В Java верхний левый угол окна принято обозначать координатами $0, 0$. Вызов метода `drawString()` в рассматриваемом здесь апплете приводит к выводу в его окне сообщения, начиная с точки, имеющей координаты $20, 20$.

Обратите внимание на то, что в рассматриваемом здесь апплете отсутствует метод `main()`. Выполнение апплета начинается не так, как выполнение программ из примеров, рассмотренных ранее в этой книге. Сигналом к началу работы апплета служит передача имени класса браузеру или другой программе, допускающей выполнение апплетов.

Исходный код класса `SimpleApplet` компилируется таким же образом, как и примеры предыдущих программ. А запуск апплета на выполнение осуществляется иначе: средствами браузера или инструментального средства разработки, специально предназначенного для просмотра апплетов. Такое инструментальное средство входит в стандартный пакет JDK и называется `appletviewer`. Именно им следует пользоваться для выполнения апплетов, примеры которых приведены в этой главе. Разумеется, ничто не мешает вам употреблять для этой цели браузер, но на стадии разработки пользоваться средством просмотра `appletviewer` намного удобнее.

Для выполнения апплета (средствами браузера или `appletviewer`) можно, в частности, составить небольшой файл HTML-документа, который содержит дескриптор для загрузки апплета. В настоящее время компания Oracle рекомендует использовать для этой цели дескриптор `APPLET`. (С той же целью можно также воспользоваться дескриптором `OBJECT` или другими доступными методиками развертывания прикладных программ. За более подробными сведениями по данному вопросу обращайтесь к документации на Java.) Ниже приведен код из HTML-документа, с помощью которого запускается апплет `SimpleApplet`.

```
<applet code="SimpleApplet" width=200 height=60>
</applet>
```

Атрибуты `width` и `height` задают размеры области отображения, используемой в апплете.

Для того чтобы запустить апплет `SimpleApplet` на выполнение средствами `appletviewer`, последнему нужно передать для открытия файл соответствующего HTML-документа. Так, если приведенный выше код HTML находится в файле `StartApp.html`, то для запуска апплета `SimpleApplet` в командной строке необходимо ввести следующее:

```
C:\>appletviewer StartApp.html
```

И хотя пользоваться отдельным файлом HTML-документа для запуска апплета вполне допустимо, существует и более простой способ добиться той же самой цели. Для этого достаточно ввести в самом начале исходного кода Java строку комментариев и указать в ней дескриптор `APPLET`. Если обратиться к примеру апплета `SimpleApplet`, то его исходный код будет выглядеть следующим образом:

```
import java.awt.*;
import java.applet.*;
// Следующий код HTML используется в appletviewer для запуска апплета.
```

```

/*
  <applet code="SimpleApplet" width=200 height=60>
  </applet>
*/

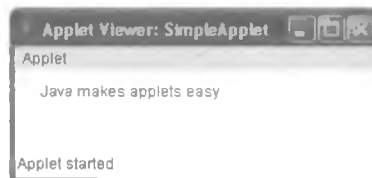
public class SimpleApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("Java makes applets easy.", 20, 20);
    }
}

```

Теперь апплет можно запустить на выполнение, передав имя его исходного файла средству просмотра `appletviewer`. Для запуска на выполнение апплета `SimpleApplet` в командной строке нужно ввести следующее:

C:>appletviewer SimpleApplet.java

Окно, формируемое апплетом `SimpleApplet` в `appletviewer`, будет выглядеть так, как показано на приведенном ниже рисунке.



Пользуясь `appletviewer`, не следует забывать, что это инструментальное средство просмотра апплетов отображает окно с обрамлением. А при запуске апплета в браузере обрамление у окна отсутствует.

Итак, подведем краткий итог того, что в настоящий момент известно об апплетах.

- Каждый апплет является подклассом, производным от класса `Applet`.
- Для выполнения апплета метод `main()` не требуется.
- Апплет должен выполняться средствами специальной программы просмотра или совместимого с Java веб-браузера.
- Для обмена данными с пользователем апплета потоки ввода-вывода не используются. Вместо этого применяются средства, предоставляемые библиотекой AWT или Swing.

Организация апплета и его основные элементы

Несмотря на то что рассмотренный ранее апплет удовлетворяет всем требованиям, он настолько прост, что ему вряд ли можно найти практическое применение. Но прежде чем приступить к созданию апплета, пригодного для применения на практике, следует дать хотя бы самое общее представление об организации апплетов, применяемых в них методах, и об их взаимодействии с исполняющей системой.

Архитектура апплетов

Как упоминалось ранее, апплет — это прикладная программа на Java с графическим пользовательским интерфейсом. Его архитектура отличается от консольных программ из примеров, рассмотренных ранее в этой книге. Если вы знакомы с основными принципами создания графических пользовательских интерфейсов, то можете считать себя

полностью готовыми к написанию апплетов. В противном случае вам придется сначала усвоить эти принципы.

Прежде всего следует заметить, что апплет — это прикладная программа, управляемая событиями, и в этом отношении он сродни программе обработки прерываний. Взаимодействие апплета с исполняющей системой и пользователем происходит следующим образом. Апплет ожидает наступления некоторого события. Исполняющая система оповещает апплет о наступлении события, вызывая его обработчик, предусмотренный в апплете. При наступлении события апплет должен предпринять соответствующие действия, а затем вернуть управление исполняющей системе, что очень важно. Как правило, апплет не выполняет продолжительные операции, а в ответ на событие предпринимает несложные действия и затем возвращает управление исполняющей системе. Если же в апплете требуется выполнить конкретное повторяющееся задание (например, отображать сообщение, прокручиваемое в окне), то для этой цели придется запустить отдельный поток исполнения.

Следует также иметь в виду, что взаимодействие с апплетом происходит только по инициативе пользователя, а не наоборот. Когда консольной программе требуются данные от пользователя, она выводит на экран специальное приглашение, а затем вызывает один из методов ввода данных с консоли. А в апплете все происходит иначе. Пользователь обращается к апплету, когда у него есть в этом потребность. Запрос на взаимодействие доставляется апплету в виде события, которое последний должен обработать. Так, если щелкнуть мышью в окне апплета, будет сформировано событие от мыши. Если же нажать клавишу на клавиатуре в тот момент, когда фокус ввода находится в окне апплета, будет сформировано событие от клавиатуры. Кроме того, в окне апплета могут присутствовать элементы управления пользовательского интерфейса, например кнопки или флажки. Когда пользователь активизирует такой элемент, наступает соответствующее событие.

Несмотря на то что архитектура апплетов немного сложнее архитектуры консольной программы, понять ее совсем не трудно. Если у вас имеется опыт написания прикладных программ для Windows (или аналогичных систем с графическим пользовательским интерфейсом), значит, вам должно быть известно, каким сложным может быть их окружение. Правда, в Java предоставляется более простой подход к разработке прикладных программ (в данном случае апплетов), который усваивается легче и быстрее.

Заготовка апплета

Хотя рассмотренный ранее класс `SimpleApplet` представляет собой подлинный апплет, он все же не содержит всех тех элементов, которые требуются для большинства апплетов. Почти во всех апплетах, кроме самых простых, переопределяется ряд методов, с помощью которых браузер или другая программа просмотра апплетов может контролировать их выполнение. Срок действия апплета определяется методами `init()`, `start()`, `stop()` и `destroy()`, которые относятся к классу `Applet`. Пятый метод, `paint()`, часто переопределяется в апплетах, создаваемых на основе AWT. Этот метод наследуется от класса `Component` из библиотеки AWT. Четыре упомянутых выше метода собраны вместе с методом `paint()` в приведенной ниже заготовке апплета.

```
// Заготовка апплета.  
import java.awt.*;  
import java.applet.*;  
/*
```

```

<applet code="AppletSkel" width=300 height=100>
</applet>
*/

public class AppletSkel extends Applet {
    // Этот метод вызывается первым.
    public void init() {
        // инициализировать апплет
    }

    /* Этот метод вызывается вторым после метода init(). Он вызывается
       и в тех случаях, когда требуется возобновить работу апплета. */
    public void start() {
        // запустить или возобновить выполнение апплета
    }

    // Этот метод вызывается для остановки работы апплета.
    public void stop() {
        // приостановить выполнение апплета
    }

    /* Этот метод вызывается последним для завершения работы апплета. */
    public void destroy() {
        // выполнить действия, завершающие выполнение апплета.
    }

    /* Этот метод вызывается для обновления данных в окне апплета. */
    public void paint(Graphics g) {
        // обновить содержимое окна апплета
    }
}

```

Несмотря на то что код этой заготовки не выполняет никаких действий, его можно скомпилировать и запустить на выполнение, а в дальнейшем создавать на его основе настоящие апплеты.

На заметку

Переопределять метод `paint()` приходится главным образом в апплетах, создаваемых на основе библиотеки AWT. А в апплетах, создаваемых на основе библиотеки Swing, для перерисовки содержимого окна апплета чаще всего используется другой механизм.

Инициализация и завершение апплетов

Необходимо ясно представлять себе, в каком именно порядке вызываются методы, входящие в состав заготовки апплета. При запуске апплета на выполнение вызываются три метода в следующей последовательности.

1. Метод `init()`.
2. Метод `start()`.
3. Метод `paint()`.

По завершении работы апплета вызываются следующие методы.

1. Метод `stop()`.
2. Метод `destroy()`.

Рассмотрим все эти методы более подробно.

Метод `init()` вызывается первым при запуске апплета на выполнение. В теле этого метода, как правило, инициализируются переменные и выполняются другие действия, связанные с подготовкой апплета к выполнению.

После метода `init()` вызывается метод `start()`. С его помощью выполнение апплета возобновляется после остановки. Обычно это происходит в тех случаях, когда пользователь возвращается к ранее посещавшейся веб-странице, содержащей апплет. Таким образом, в течение срока действия апплета метод `start()` может быть вызван несколько раз.

Метод `paint()` вызывается всякий раз, когда данные, выводимые апплетом, должны быть обновлены в его окне, как пояснялось ранее.

Когда пользователь покидает веб-страницу, содержащую апплет, вызывается метод `stop()`. Это может быть сделано для того, чтобы приостановить любые порожденные потоки, созданные в апплете, а также выполнить другие действия для благополучного перевода апплета в режим простоя. Не следует, однако, забывать, что вызов метода `stop()` совсем не означает завершение работы самого апплета. Когда пользователь вернется к веб-странице с апплетом, его выполнение будет возобновлено благодаря вызову метода `start()`.

Метод `destroy()` вызывается в том случае, когда апплет больше не нужен. Этот метод служит для выполнения операций по завершении работы апплета.

Запрос на перерисовку окна апплета

Вывод данных в окне апплета происходит только в том случае, если исполняющая система вызывает метод `paint()`. В связи с этим возникает вопрос: можно ли по инициативе апплета произвести перерисовку окна? Необходимость в этом чаще всего возникает при изменении данных, предназначенных для отображения. Допустим, в апплете отображается движущееся рекламное сообщение. Какой механизм должен быть использован для обновления окна всякий раз, когда внешний вид сообщения должен измениться? Напомним, что одним из основных ограничений, налагаемых на апплет, является требование как можно быстрее возвращать управление исполняющей системе. Поэтому в методе `paint()` нельзя создать цикл, в котором постоянно изменялся бы вид сообщения. Принимая во внимание это ограничение, можно прийти к заключению, что вывод данных в окне апплета — дело непростое. К счастью, это совсем не так. Если требуется изменить данные, отображаемые в окне апплета, достаточно вызвать метод `repaint()`.

Этот метод определен в классе `Component`, входящем в состав библиотеки AWT. Он сообщает исполняющей системе о том, что метод `paint()` должен быть вызван из апплета при первой же возможности. Если, например, в апплете возникнет потребность вывести данные, их нужно сначала сохранить, а затем вызвать метод `repaint()`. В итоге будет вызван метод `paint()`, чтобы отобразить сохраненные изменившиеся данные. Так, если в апплете требуется вывести символьную строку, ее нужно сначала сохранить в переменной типа `String`, а затем вызвать метод `repaint()`. А в самом методе `paint()` символьная строка будет выведена с помощью метода `drawString()`.

Ниже приведена простейшая форма вызова метода `repaint()`.

```
void repaint()
```

Этот метод производит перерисовку всего окна апплета.

В другой форме вызова метода `repaint()` задается конкретная область для перерисовки.

```
void repaint(int слева, int сверху, int ширина, int высота)
```

Координаты верхнего левого угла перерисовываемой области задаются с помощью параметров *слева* и *сверху*, а параметры *ширина* и *высота* определяют конкретную ширину и высоту этой области. Все размеры задаются в пикселях. Указывая области для перерисовки, можно значительно увеличить быстродействие программы, поскольку для обновления содержимого окна требуется немало времени. Если же требуется перерисовать лишь небольшую область, то задавать обновление всего окна нет никакой надобности.

Применение метода `repaint()` будет продемонстрировано в примере для опробования 14.1.

Метод `update()`

Перерисовку окна апплета можно организовать и с помощью метода `update()`, который переопределяется во многих апплетах. Этот метод объявляется в классе `Component` и вызывается в том случае, если требуется перерисовать часть окна апплета. В используемом по умолчанию варианте метода `update()` просто вызывается метод `paint()`. Но, переопределив метод `update()`, можно выполнить и более сложные операции, рассмотрение которых, впрочем, выходит за рамки этой книги. Переопределение метода `update()` осуществляется только в апплетах, создаваемых на основе библиотеки AWT.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Можно ли выводить данные в окне апплета, не вызывая метод `paint()` или `update()`?

ОТВЕТ. Да, можно. Для этого нужно сначала получить графический контекст, вызвав метод `getGraphics()`, определенный в классе `Component`, а затем использовать полученный контекст для вывода данных в окне. Но в большинстве прикладных программ, создаваемых на основе библиотеки AWT, применяется намного более удобный способ, в соответствии с которым все действия по отображению данных в окне реализуются в методе `paint()`, а для вызова последнего служит метод `repaint()`.

Пример для опробования 14.1.

Простой апплет для формирования крупного заголовка на веб-странице

Исходный файл: `Banner.java`

Работу метода `repaint()` удобно продемонстрировать на примере следующего простого апплета. Он выполняет прокрутку крупного заголовка в окне слева направо. А поскольку такую прокрутку нужно выполнять постоянно, то реализовать ее следует в отдельном потоке, который создается при инициализации апплета. Подобные крупные заголовки называются еще *баннерами* и часто встречаются на веб-страницах, а в этом проекте демонстрируется, каким образом можно создать такой заголовок, используя апплет Java.

Последовательность действий

1. Создайте файл `Banner.java`.
2. Начните создание апплета со следующих строк кода:

```
/*
    Пример для опробования 14.1.

    Простой апплет, отображающий крупный заголовок.

    В этом апплете создается поток, управляющий прокруткой
    крупного заголовка, хранящегося в переменной msg,
    справа налево в окне апплета.
*/
import java.awt.*;
import java.applet.*;
/*
    <applet code="Banner" width=300 height=50>
    </applet>
*/

public class Banner extends Applet implements Runnable {
    String msg = " Java Rules the Web "; // "Java правит Паутиной"
    Thread t;
    boolean stopFlag;

    // инициализировать переменную t пустым значением null
    public void init() {
        t = null;
    }
}
```

Обратите внимание на то, что класс `Banner` не только является подклассом, производным от класса `Applet`, но и реализует интерфейс `Runnable`. Это необходимо потому, что апплет создает еще один поток исполнения, который служит для прокрутки крупного заголовка по горизонтали. Текст крупного заголовка хранится в переменной `msg` типа `String`, ссылка на поток, в котором выполняется апплет, хранится в переменной `t`, а логическая переменная `stopFlag` служит для остановки апплета. В методе `init()` переменной `t` исходно присваивается пустое значение `null`.

3. Введите метод `start()`, исходный код которого приведен ниже.

```
// запустить поток
public void start() {
    t = new Thread(this);
    stopFlag = false;
    t.start();
}
```

Исполняющая система вызывает метод `start()` для запуска апплета на выполнение. В этом методе создается новый поток исполнения, а ссылка на него присваивается переменной `t` типа `Thread`. Затем переменная `stopFlag` инициализируется логическим значением `false`. Далее поток запускается при вызове `t.start()`. Это, в свою очередь, приводит к вызову метода `run()`. Следует, однако, иметь в виду, что метод `start()` из класса `Thread` не имеет ничего общего с одноименным методом, определенным в классе `Applet`.

4. Введите метод `run()`, исходный код которого приведен ниже.

```
// Точка входа в поток, манипулирующий крупным заголовком.
public void run() {
    // отобразить крупный заголовок снова
    for( ; ; ) {
        try {
            repaint();
            Thread.sleep(250);
            if(stopFlag)
                break;
        } catch (InterruptedException exc) {}
    }
}
```

В методе `run()` вызывается метод `repaint()`, что в конечном итоге приводит к вызову метода `paint()` и отображению текущего содержимого переменной `msg` на экране. Между последовательными шагами цикла выполнение метода `run()` приостанавливается на четверть секунды. В результате содержимое переменной `msg` постоянно прокручивается справа налево. На каждом шаге цикла проверяется содержимое переменной `stopFlag`. И как только она примет логическое значение `true`, выполнение метода `run()` завершится.

5. Введите методы `stop()` и `paint()`, исходный код которых приведен ниже.

```
// остановить выполнение апплета
public void stop() {
    stopFlag = true;
    t = null;
}

// отобразить крупный заголовок
public void paint(Graphics g) {
    char ch;
    ch = msg.charAt(0);
    msg = msg.substring(1, msg.length());
    msg += ch;
    g.drawString(msg, 50, 30);
}
```

Если пользователь браузера переходит на другую веб-страницу, вызывается метод `stop()`, в котором переменной `stopFlag` присваивается логическое значение `true`, завершая тем самым выполнение метода `run()`. В этом же методе переменной `t` присваивается пустое значение `null`. После этого она уже не ссылается на объект типа `Thread`, а следовательно, этот объект будет удален системой “сборки мусора”. Этот механизм используется для остановки потока, когда страница с апплетом не просматривается. Когда же она снова появляется в окне браузера, вызывается метод `start()`, запускающий новый поток манипулирования крупным заголовком.

В методе `paint()` символы строки крупного заголовка, хранящегося в переменной `msg`, постоянно сдвигаются влево. После каждого сдвига вызывается метод `drawString()`, выводящий текущее содержимое переменной `msg` на экран.

6. Ниже приведен весь исходный код апплета для формирования крупного заголовка на веб-странице.

/*

Пример для опробования 14.1.

Простой апплет, отображающий крупный заголовок.

В этом апплете создается поток, управляющий прокруткой крупного заголовка, хранящегося в переменной **msg**, справа налево в окне апплета.

*/

```
import java.awt.*;
```

```
import java.applet.*;
```

/*

```
<applet code="Banner" width=300 height=50>
```

```
</applet>
```

*/

```
public class Banner extends Applet implements Runnable {
    String msg = " Java Rules the Web "; // "Java правит Паутиной"
    Thread t;
    boolean stopFlag;
```

```
    // инициализировать переменную t пустым значением null
```

```
    public void init() {
```

```
        t = null;
```

```
    }
```

```
    // запустить поток
```

```
    public void start() {
```

```
        t = new Thread(this);
```

```
        stopFlag = false;
```

```
        t.start();
```

```
    }
```

```
    // Точка входа в поток, манипулирующий крупным заголовком.
```

```
    public void run() {
```

```
        // отобразить крупный заголовок снова
```

```
        for( ; ; ) {
```

```
            try {
```

```
                repaint();
```

```
                Thread.sleep(250);
```

```
                if(stopFlag)
```

```
                    break;
```

```
            } catch (InterruptedException exc) {}
```

```
        }
```

```
    }
```

```
    // остановить выполнение апплета
```

```
    public void stop() {
```

```
        stopFlag = true;
```

```
        t = null;
```

```
    }
```

```
// отобразить крупный заголовок
public void paint(Graphics g) {
    char ch;
    ch = msg.charAt(0);
    msg = msg.substring(1, msg.length());
    msg += ch;
    g.drawString(msg, 50, 30);
}
}
```

Ниже приведен результат выполнения данного апплета.



Применение строки состояния

Помимо отображения информации в основном окне, апплет может также выводить сообщения в строке состояния браузера или другого средства просмотра апплетов. Для вывода сообщения в строке состояния следует вызвать метод `showStatus()`, определенный в классе `Applet`. При вызове этому методу в качестве параметра передается выводимая символьная строка. Ниже приведена общая форма объявления метода `showStatus()`.

```
void showStatus(String сообщение)
```

где *сообщение* обозначает выводимую символьную строку.

В строке состояния удобно отображать сведения о том, что происходит при выполнении апплета, подсказывать пользователю дальнейшие действия или выводить сообщения об ошибках. Строка состояния незаменима при отладке, поскольку дает возможность без труда выводить всю необходимую информацию об апплете.

Ниже приведен пример апплета, демонстрирующий применение метода `showStatus()`.

```
// Использование строки состояния.
import java.awt.*;
import java.applet.*;
/*
    <applet code="StatusWindow" width=300 height=50>
    </applet>
*/

public class StatusWindow extends Applet{
    // отобразить содержимое переменной msg в окне апплета
    public void paint(Graphics g) {
        g.drawString("This is in the applet window.", 10, 20);
        showStatus("This is shown in the status window.");
    }
}
```

Выполнение этого апплета дает приведенный ниже результат.



Передача параметров апплету

По мере необходимости апплету можно передать один или более параметр. Для этой цели предусмотрен атрибут PARAM дескриптора APPLET. С его помощью задается имя и значение параметра. А для извлечения параметра служит метод `getParameter()`, определенный в классе `Applet`. Он объявляется следующим образом:

```
String getParameter(String имя_параметра)
```

где *имя_параметра* обозначает конкретное имя передаваемого параметра. Этот метод возвращает значение указанного параметра в виде объекта типа `String`. Так, если апплету требуется передать числовое или логическое значение, его строковое представление придется преобразовать во внутренний формат. Если же указанный параметр не найден, метод возвращает пустое значение `null`. Поэтому в исходный код апплетов следует включать проверку правильности значений, возвращаемых методом `getParameter()`. Желательно также проверять, правильно ли было выполнено преобразование числового значения параметра, передаваемого апплету.

Ниже приведен пример, демонстрирующий процесс передачи параметров апплету.

```
// Передача параметров апплету.
import java.awt.*;
import java.applet.*;

// Параметры передаются апплету в коде HTML.
/*
    <applet code="Param" width=300 height=80>
        <param name=author value="Herb Schildt">
        <param name=purpose value="Demonstrate Parameters">
        <param name=version value=2>
    </applet>
*/

public class Param extends Applet {
    String author;
    String purpose;
    int ver;

    public void start() {
        String temp;

        author = getParameter("author");
        // Важно убедиться в существовании параметра.
        if(author == null) author = "not found";

        purpose = getParameter("purpose");
        if(purpose == null) purpose = "not found";
    }
}
```

```

temp = getParameter("version");
try {
    if(temp != null)
        ver = Integer.parseInt(temp);
    else
        ver = 0;
    // Не менее важно убедиться, что преобразование
    // числового значения параметра выполнено правильно.
} catch (NumberFormatException exc) {
    ver = -1; // Код ошибки
}

public void paint(Graphics g) {
    g.drawString("Purpose: " + purpose, 10, 20);
    g.drawString("By: " + author, 10, 40);
    g.drawString("Version: " + ver, 10, 60);
}
}

```

Выполнение этого апплета дает приведенный ниже результат.



Класс Applet

Как пояснялось ранее, все апплеты представляют собой подклассы, производные от класса Applet. Класс Applet наследует переменные и методы следующих классов из библиотеки AWT: Component, Container и Panel. В результате апплет получает все функциональные возможности, доступные в библиотеке AWT.

Помимо методов, рассмотренных в предыдущих разделах, класс Applet содержит ряд других методов, которые предоставляют средства для более полного управления процессом выполнения апплетов. Все методы, определенные в классе Applet, приведены в табл. 14.1.

Таблица 14.1. Методы, определенные в классе Applet

Метод	Описание
<code>void destroy()</code>	Вызывается браузером непосредственно перед завершением апплета. Если же перед завершением апплета требуется освободить ресурсы или выполнить какие-нибудь другие подготовительные действия, этот метод следует переопределить
<code>AccessibleContext getAccessibleContext()</code>	Возвращает контекст доступности для вызывающего объекта
<code>AppletContext getAppletContext()</code>	Возвращает контекст, связанный с апплетом
<code>String getAppletInfo()</code>	Возвращает строку, описывающую апплет

Метод	Описание
AudioClip getAudioClip(URL url)	Возвращает объект типа AudioClip, инкапсулирующий аудиоклип, доступный по адресу, задаваемому параметром url
AudioClip getAudioClip(URL url, String clipName)	Возвращает объект типа AudioClip, инкапсулирующий аудиоклип, доступный по адресу, задаваемому параметром url, и называемый по имени, которое указывает параметр clipName
URL getCodeBase()	Возвращает веб-адрес (URL), связанный с апплетом
URL getDocumentBase()	Возвращает веб-адрес (URL) того HTML-документа, который вызывает апплет
Image getImage(URL url)	Возвращает объект типа Image, инкапсулирующий изображение, доступное в месте, определяемом параметром url
Image getImage(URL url, String imageName)	Возвращает объект типа Image, инкапсулирующий изображение, доступное в месте, определяемом параметром url, и называемое по имени, указываемому параметром imageName
Locale getLocale()	Возвращает объект типа Locale, который используется классами и методами, реагирующими на настройки региональных и языковых параметров системы
String getParameter(String paramName)	Возвращает параметр по имени, указываемому параметром paramName. Если параметр не найден, возвращается пустое значение null
String[][] getParameterInfo()	Переопределенный вариант этого метода возвращает таблицу объектов типа String с описанием параметров, распознаваемых апплетом. Каждый элемент таблицы должен состоять из трех строк, содержащих имя параметра, описание его типа и/или диапазона допустимых значений, а также краткое пояснение назначения параметра. В исходном варианте реализации этот метод возвращает пустое значение null
void init()	Этот метод выполняется первым при запуске любого апплета на выполнение
boolean isActive()	Возвращает логическое значение true, если апплет выполняется. А если выполнение апплета приостановлено, то возвращает логическое значение false
boolean isValidateRoot()	Возвращает логическое значение true, указывающее на то, что апплет является достоверным корнем, т.е. он прошел проверку на достоверность. (Добавлен в версии JDK 7.)
static final AudioClip newAudioClip(URL url)	Возвращает объект типа AudioClip, инкапсулирующий аудиоклип, доступный по адресу, задаваемому параметром url. Этот метод подобен методу getAudioClip(), за исключением того, что он является статическим и для его вызова не требуется ссылка на объект типа Applet
void play(URL url)	Воспроизводит аудиоклип, если он найден по адресу, задаваемому параметром url
void play(URL url, String clipName)	Воспроизводит аудиоклип, если он найден по адресу, задаваемому параметром url, под именем, обозначаемым параметром clipName

Окончание табл. 14.1

Метод	Описание
<code>void resize (Dimension dim)</code>	Изменяет размеры апплета по значениям, указываемым параметром <i>dim</i> . Класс <i>Dimension</i> входит в пакет <i>java.awt</i> . В нем содержатся два целочисленных поля, <i>width</i> и <i>height</i> , в которых задаются ширина и высота апплета соответственно
<code>void resize (int width, int height)</code>	Изменяет размеры апплета по значениям, указываемым параметрами <i>width</i> и <i>height</i>
<code>final void setStub (AppletStub stubObj)</code>	Задаёт <i>stubObj</i> в качестве заглушки для апплета. Используется в основном исполняющей системой и не вызывается из апплета. Заглушкой называется фрагмент кода, устанавливающий связь между апплетом и браузером
<code>void showStatus (String str)</code>	Отображает символьную строку <i>str</i> в строке состояния браузера или другом средстве просмотра апплетов. Если строка состояния отсутствует в браузере, никакие действия не выполняются
<code>void start ()</code>	Вызывается браузером в том случае, если апплет должен начать (или возобновить) свое выполнение. Этот метод вызывается при запуске апплета на выполнение после метода <i>init()</i>
<code>void stop ()</code>	Вызывается браузером с целью приостановить выполнение апплета. Выполнение приостановленного апплета может быть возобновлено вызовом метода <i>start()</i>

Обработка событий

Все программы, написанные на Java с графическим пользовательским интерфейсом, в том числе и апплеты, управляются событиями. Следовательно, обработка событий служит основанием для успешного программирования графического пользовательского интерфейса. Большинство событий, на которые должна реагировать подобная программа, наступают в результате действий пользователя. Эти события передаются программе разными способами. События разделяются на отдельные категории, например события от мыши, клавиатуры, а также события, наступающие в результате взаимодействия пользователя с элементами пользовательского интерфейса. Классы поддержки событий в библиотеке AWT относятся к пакету *java.awt.event*.

Прежде чем приступить к подробному рассмотрению событий, следует заметить, что обсудить столь обширную тему более или менее подробно в рамках этой книги просто невозможно. Тем не менее приведенный ниже краткий обзор данной темы позволит вам успешно создавать несложные обработчики событий при написании реальных программ.

Модель делегирования событий

Современный подход к обработке событий основывается на *модели делегирования событий*. Эта модель описывает стандартный механизм формирования событий и их обработки. Она действует по очень простому принципу: событие формируется *источником* и передается одному или нескольким *приемникам*. В соответствии с этим принципом приемники лишь ожидают события. При наступлении событий они обрабатывают их и возвращают управление. Преимущество такого подхода заключается в том, что логика обработки событий четко отделена от логики пользовательского интерфейса, формиру-

ющего эти события. Элемент пользовательского интерфейса способен делегировать (т.е. поручать) обработку события отдельному фрагменту кода. В модели делегирования событий их приемники должны быть зарегистрированы в источнике, чтобы получать уведомления о событиях для их последующей обработки.

События

Согласно модели делегирования, событие является объектом, описывающим изменения в состоянии источника. Событие может быть сформировано в результате действий пользователя, в том числе активизации элементов графического пользовательского интерфейса. Такими действиями может быть нажатие кнопки, ввод символа с клавиатуры, выбор пункта списка или щелчок кнопкой мыши.

Источники событий

Источник события — это объект, сформировавший его. Для того чтобы приемник получил уведомление о конкретном виде события, он должен быть зарегистрирован в его источнике. Для каждого вида события определен собственный метод регистрации. Ниже приведена общая форма объявления методов регистрации.

```
public void addТипListener(ТипListener элемент)
```

где *Тип* — это имя события, а *элемент* — ссылка на приемник события. Например, метод, регистрирующий приемник событий от клавиатуры, объявляется как `addKeyListener()`. А для регистрации приемника событий от перемещения мыши служит метод `addMouseMotionListener()`. При наступлении события все приемники оповещаются и получают копию, описывающую его объект.

Источник должен также предоставлять метод, позволяющий отменить регистрацию приемника событий определенного вида. Ниже приведена общая форма объявления такого метода.

```
public void removeТипListener(ТипListener элемент)
```

Здесь, как и прежде, *Тип* — это имя события, а *элемент* — ссылка на его приемник. Например, для того чтобы удалить приемник событий от клавиатуры, следует вызвать метод `removeKeyListener()`.

Методы, добавляющие или удаляющие приемники событий, предоставляются источниками, формирующими эти события. Например, в классе `Component` предоставляются методы, позволяющие добавлять или удалять приемники событий от клавиатуры и мыши.

Приемники событий

Приемник — это объект, уведомляемый о наступлении события. К нему предъявляются два основных требования. Во-первых, он должен быть зарегистрирован в одном или нескольких источниках, чтобы получать от них уведомления о конкретных видах событий. И во-вторых, он должен реализовать методы для получения и обработки уведомлений о событиях.

Методы, получающие уведомления о событиях из библиотеки AWT и обрабатывающие их, определены в интерфейсах, относящихся к пакету `java.awt.event`. Например, в интерфейсе `MouseMotionListener` объявлены методы, получающие уведомления о перемещении и перетаскивании мыши. Всякий объект, реализующий этот интерфейс, может получать уведомления о подобных событиях и обрабатывать их.

Классы событий

Классы, представляющие события, положены в основу механизма обработки событий в Java. Они образуют иерархическую структуру, на вершине которой находится класс `EventObject`, относящийся к пакету `java.util`. Он служит суперклассом для всех событий. Класс `AWTEvent`, относящийся к пакету `java.awt`, является подклассом, производным от класса `EventObject`. Он, в свою очередь, выступает в роли родительского класса для всех событий из библиотеки AWT, используемых в модели делегирования событий.

В пакете `java.awt.event` определено несколько видов событий, формируемых различными элементами пользовательского интерфейса. Краткое описание некоторых из наиболее употребительных событий подобного рода приведено в табл. 14.2.

Таблица 14.2. Основные классы событий из пакета `java.awt.event`

Класс события	Описание
<code>ActionEvent</code>	Формируется после щелчка на кнопке, двойного щелчка на элементе списка или выбора пункта меню
<code>AdjustmentEvent</code>	Формируется при манипулировании полосой прокрутки
<code>ComponentEvent</code>	Формируется, когда компонент становится видимым или невидимым, а также при изменении его размеров и перемещении
<code>ContainerEvent</code>	Формируется, когда компонент добавляется в контейнер или удаляется из него
<code>FocusEvent</code>	Формируется, когда компонент получает или теряет фокус ввода
<code>InputEvent</code>	Абстрактный суперкласс для всех классов событий, связанных с вводом данных
<code>ItemEvent</code>	Формируется по щелчку на флажке или элементе списка, а также наступает при выборе или отмене выбора пункта меню
<code>KeyEvent</code>	Формируется при вводе данных с клавиатуры
<code>MouseEvent</code>	Формируется при перемещении или перетаскивании мыши, а также по нажатию, отпусканию или щелчку кнопкой мыши или же в том случае, когда курсор мыши наводится на элемент интерфейса или перемещается с него
<code>TextEvent</code>	Формируется при изменении содержимого области или поля ввода текста
<code>WindowEvent</code>	Формируется, если окно делается активным или неактивным, сворачивается или разворачивается, открывается или закрывается либо покидается

Интерфейсы приемников событий

Приемники событий получают уведомления об их наступлении. Приемники событий из библиотеки AWT реализуют один или несколько интерфейсов, относящихся к пакету `java.awt.event`. При наступлении события источник вызывает соответствующий метод, определенный приемником, и передает ему объект события в качестве аргумента. В табл. 14.3 перечислены наиболее употребительные интерфейсы приемников событий и кратко описаны объявленные в них методы.

Таблица 14.3. Наиболее употребительные интерфейсы приемников событий

Интерфейс	Описание
<code>ActionListener</code>	Определяет один метод для получения событий действий. Такого рода события наступают при нажатии кнопки, выборе пункта меню и т.д.
<code>AdjustmentListener</code>	Определяет один метод для получения событий настройки, подобных тем, которые наступают при манипулировании полосой прокрутки

Интерфейс	Описание
ComponentListener	Определяет четыре метода для выявления факта сокрытия, показа, изменения размеров или перемещения компонента пользовательского интерфейса
ContainerListener	Определяет два метода для выявления факта добавления компонента в контейнер или удаления из него
FocusListener	Определяет два метода для выявления факта получения и потери компонентом фокуса ввода
ItemListener	Определяет один метод для выявления факта изменения состояния элемента пользовательского интерфейса. Событие от элемента может формироваться, например, флажком
KeyListener	Определяет три метода для выявления нажатия, отпущения клавиши и ввода с клавиатуры
MouseListener	Определяет пять методов для выявления щелчка кнопкой мыши, ее нажатия и отпущения, а также наведения курсора мыши на элемент пользовательского интерфейса или перемещения с него
MouseMotionListener	Определяет два метода для выявления факта перемещения или перетаскивания мыши
MouseWheelListener	Определяет один метод для выявления факта прокрутки колесика мыши
TextListener	Определяет один метод для выявления факта изменения текстового значения
WindowListener	Определяет семь методов для выявления факта открытия и закрытия, сворачивания и разворачивания, активизации и деактивизации или покидания окна

Применение модели делегирования событий

А теперь, когда вы имеете общее представление о модели делегирования событий, перейдем к вопросам ее практического применения. Создавать апплеты, используя эту модель, совсем не трудно. По существу, написание кода для обработки событий сводится к следующему.

1. Реализация соответствующего интерфейса в приемнике событий нужного вида.
2. Написание кода для регистрации приемника событий, а если потребуется, то и отмены его регистрации.

Не следует, однако, забывать, что источник, как правило, может формировать несколько видов событий. И для каждого из этих видов событий требуется регистрация отдельного приемника. Кроме того, один объект можно зарегистрировать для приема на обработку событий нескольких видов, но тогда он должен реализовать все интерфейсы, соответствующие этим видам событий.

Для того чтобы стало понятнее, как пользоваться моделью делегирования событий на практике, рассмотрим простой пример, в котором обрабатывается одни из самых распространенных видов событий: события от мыши. Этот пример демонстрирует порядок обработки элементарных событий от мыши. (Следует также иметь в виду, что обрабатывать можно и события от прокрутки колесика мыши. Но эта возможность оставляется вам в качестве упражнения для самостоятельного выполнения.)

Обработка событий от мыши

Для обработки событий от мыши следует реализовать интерфейсы `MouseListener` и `MouseMotionListener`. В интерфейсе `MouseListener` объявлено пять методов. По щелчку кнопкой мыши вызывается метод `mouseClicked()`. Если курсор мыши наводится на компонент пользовательского интерфейса, вызывается метод `mouseEntered()`, а если курсор мыши перемещается с этого компонента — метод `mouseExited()`. Методы `mousePressed()` и `mouseReleased()` вызываются, когда кнопка мыши нажимается и отпускается соответственно.

Ниже приведены общие формы объявления упомянутых выше методов.

```
void mouseClicked(MouseEvent me)
void mouseEntered(MouseEvent me)
void mouseExited(MouseEvent me)
void mousePressed(MouseEvent me)
void mouseReleased(MouseEvent me)
```

В интерфейсе `MouseMotionListener` объявлены два метода. Метод `mouseDragged()` многократно вызывается при перетаскивании мыши, когда нажата ее левая кнопка. А обычное перемещение мыши приводит к такому же многократному вызову метода `mouseMoved()`. Ниже приведены общие формы объявления этих методов.

```
void mouseDragged(MouseEvent me)
void mouseMoved(MouseEvent me)
```

Событие описывается объектом типа `MouseEvent`, передаваемым каждому из методов в качестве параметра `me`. В классе `MouseEvent` определен целый ряд методов, которые можно использовать для получения подробных сведений о наступившем событии. Вероятно, наиболее употребительными в классе `MouseEvent` являются методы `getX()` и `getY()`, возвращающие координаты текущего положения курсора мыши (относительно окна) в момент наступления события. Ниже приведены общие формы объявления этих методов.

```
int getX()
int getY()
```

В примере, приведенном в следующем разделе, эти методы будут использованы для отображения сведений о текущем положении курсора мыши.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как упоминалось выше, методы `getX()` и `getY()`, определенные в классе `MouseEvent`, возвращают координаты текущего положения курсора мыши относительно окна. Существуют ли методы, предоставляющие те же самые сведения в экранных, т.е. абсолютных, координатах?

ОТВЕТ. Да, существуют. Начиная с версии JDK 6 в классе `MouseEvent` определены методы `int getXOnScreen()` и `int getYOnScreen()` для получения абсолютных координат X и Y соответственно, определяющих текущее положение курсора мыши относительно экрана.

Можете поэкспериментировать с этими методами, заменив ими методы `getX()` и `getY()` в примере апплета, представленном в следующем разделе.

Простой пример апплета, демонстрирующий обработку событий от мыши

В этом разделе представлен пример апплета, в котором обрабатываются элементарные события от мыши, а в строке состояния отображаются координаты текущего положения курсора мыши. При нажатии кнопки мыши в месте расположения курсора на экран выводится слово "Down" (Нажато), а при отпускании кнопки — слово "Up" (Отпущено). И наконец, по щелчку кнопкой мыши в верхнем левом углу окна апплета появляется сообщение "Mouse clicked" (Произведен щелчок кнопкой мыши).

Когда курсор мыши наводится на окно апплета или отводится от него, в левом верхнем его углу выводится соответствующее сообщение. При перетаскивании курсора мыши его сопровождает символ *. Кроме того, при нажатии и отпускании кнопки мыши или перетаскивании курсора координаты его текущего положения сначала сохраняются в переменных `mouseX` и `mouseY`, а затем используются в методе `paint()` для вывода сообщения в той точке экрана, где произошло событие от мыши.

```
// Демонстрация обработки событий от мыши.
import java.awt.event.*;
import java.applet.*;
/*
    <applet code="MouseEvents" width=300 height=100>
    </applet>
*/

public class MouseEvents extends Applet
    implements MouseListener, MouseMotionListener {

    String msg = "";
    int mouseX = 0, mouseY = 0; // Координаты курсора мыши

    public void init() {
        // Этот класс регистрируется в качестве приемника событий от мыши.
        addMouseListener(this);
        addMouseMotionListener(this);
    }

    // обработать событие, наступающее по щелчку кнопкой мыши
    // Этот и другие методы обработки событий вызываются при
    // наступлении разных событий от мыши.
    public void mouseClicked(MouseEvent me) {
        mouseX = 0;
        mouseY = 10;
        msg = "Mouse clicked.";
        repaint();
    }

    // обработать событие, наступающее при наведении курсора
    // мыши на компонент пользовательского интерфейса
    public void mouseEntered(MouseEvent me) {
        mouseX = 0;
        mouseY = 10;
        msg = "Mouse entered.";
```

```

    repaint();
}

// обработать событие, наступающее при отведении курсора
// мыши от компонента пользовательского интерфейса
public void mouseExited(MouseEvent me) {
    mouseX = 0;
    mouseY = 10;
    msg = "Mouse exited.";
    repaint();
}

// обработать событие, наступающее при нажатии кнопки мыши
public void mousePressed(MouseEvent me) {
    // сохранить координаты текущего положения курсора
    mouseX = me.getX();
    mouseY = me.getY();
    msg = "Down";
    repaint();
}

// обработать событие, наступающее при отпускании кнопки мыши
public void mouseReleased(MouseEvent me) {
    // сохранить координаты текущего положения курсора
    mouseX = me.getX();
    mouseY = me.getY();
    msg = "Up";
    repaint();
}

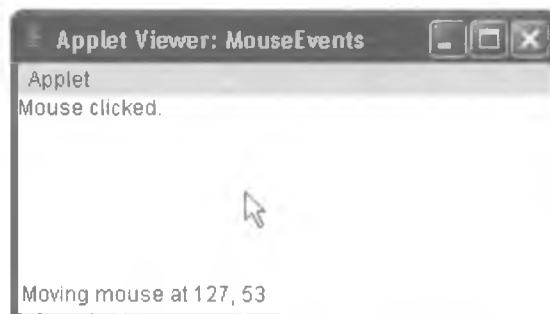
// обработать событие, наступающее при перетаскивании курсора мыши
public void mouseDragged(MouseEvent me) {
    // сохранить координаты текущего положения курсора
    mouseX = me.getX();
    mouseY = me.getY();
    msg = "**";
    showStatus("Dragging mouse at " + mouseX + ", " + mouseY);
    repaint();
}

// обработать событие, наступающее при перемещении курсора мыши
public void mouseMoved(MouseEvent me) {
    // отобразить текущее положение курсора в строке состояния
    showStatus("Moving mouse at " + me.getX() + ", " +
        me.getY());
}

// отобразить сообщение из переменной msg в окне апплета
// по координатам текущего положения курсора
public void paint(Graphics g) {
    g.drawString(msg, mouseX, mouseY);
}
}

```

Выполнение этого апплета дает приведенный ниже результат.



Рассмотрим приведенный выше исходный код апплета более подробно. Класс `MouseEvents` расширяет класс `Applet` и реализует интерфейсы `MouseListener` и `MouseMotionListener`. В этих интерфейсах объявлены методы, получающие и обрабатывающие различные виды событий от мыши. В данном примере апплет одновременно выступает в роли источника и приемника событий. И это вполне допустимо, поскольку `Applet` является подклассом, производным от суперкласса `Component`, в котором определены методы `addMouseListener()` и `addMouseMotionListener()`. Ситуация, когда один и тот же класс одновременно служит источником и приемником событий, является типичной для апплетов.

В методе `init()` апплет регистрируется как приемник событий от мыши. Для этой цели используются методы `addMouseListener()` и `addMouseMotionListener()`, являющиеся членами класса `Component`. Ниже приведены общие формы объявления этих методов.

```
void addMouseListener(MouseListener ml)
void addMouseMotionListener(MouseMotionListener mml)
```

где `ml` и `mml` — ссылки на объекты, принимающие события от мыши и перемещения мыши соответственно. В данном примере для приема и обработки обоих видов событий используется один и тот же объект.

В рассматриваемом здесь апплете реализованы все методы, объявленные в интерфейсах `MouseListener` и `MouseMotionListener`. Они выступают в роли обработчиков разных событий, после чего возвращают управление.

Другие ключевые слова Java

И в завершение этой главы будет сделан краткий обзор остальных, не упоминавшихся ранее ключевых слов Java. К их числу относятся следующие ключевые слова:

- `transient`
- `volatile`
- `instanceof`
- `native`
- `strictfp`
- `assert`

Перечисленные выше ключевые слова используются в более сложных программах, чем те, которые представлены для примера в этой книге. Рассмотрим эти ключевые слова вкратце, чтобы дать хотя бы самое общее представление об их назначении.

Модификаторы `transient` и `volatile`

Ключевые слова `transient` и `volatile` имеют весьма специфическое назначение. Если переменная экземпляра объявляется как `transient`, то при сохранении объекта ее содержимое не запоминается. Иными словами, поле `transient` не влияет на состояние объекта.

Модификатор `volatile` упоминался в главе 11, но он заслуживает более пристального внимания. Объявив переменную как `volatile`, вы сообщаете компилятору, что ее значение может быть неожиданно изменено в результате действий, выполняемых в других частях программы. В главе 11 было показано, что подобная ситуация может возникнуть в многопоточных программах. В них одна и та же переменная часто используется несколькими потоками. Из соображений эффективности каждый поток может содержать собственную копию разделяемой переменной. Подлинная копия (или оригинал) переменной обновляется в различные моменты времени, например, при выполнении метода, объявленного как `synchronized`. Как правило, такой подход себя оправдывает, но иногда он оказывается неуместным. Ведь иногда требуется, чтобы оригинал переменной отражал текущее состояние, используемое во всех потоках. И для того чтобы обеспечить выполнение этого условия, переменную нужно объявить как `volatile`.

Ключевое слово `instanceof`

Иногда в процессе выполнения программы требуется выяснить тип того или иного объекта. Допустим, в одном потоке формируются объекты разных типов, а в другом потоке они обрабатываются. В таком случае обрабатывающему потоку должен быть известен тип того или иного объекта. Об этом должно быть известно и в том случае, когда приведение типов производится в процессе выполнения программы. В Java недопустимое приведение типов вызывает ошибку при выполнении программы. Многие недопустимые попытки приведения типов могут быть выявлены еще на стадии компиляции. Но если в приведение типов вовлекается иерархия классов, то такое приведение типов может оказаться недопустимым, хотя обнаружить его удастся лишь после запуска программы на выполнение. Так, переменная ссылки на суперкласс может ссылаться и на объекты его подклассов, поэтому на стадии компиляции не всегда удастся определить, допустимо ли приведение типов, в которое вовлечена ссылка на суперкласс. В качестве выхода из подобных ситуаций служит использование ключевого слова `instanceof`. Ниже приведена общая форма оператора с этим ключевым словом.

экземпляр instanceof тип

Здесь *экземпляр* обозначает ссылку на экземпляр класса, а *тип* — имя класса или интерфейса. Если *экземпляр* имеет заданный *тип* или может быть приведен к нему, то в результате вычисления оператора `instanceof` получается логическое значение `true`, в противном случае — логическое значение `false`. Таким образом, с помощью ключевого слова `instanceof` можно получать сведения о типе объекта в процессе выполнения программы.

Ключевое слово `strictfp`

Ключевое слово `strictfp` относится к числу самых скрытых средств языка. После выпуска версии Java 2 модель вычислений с плавающей точкой претерпела некоторые изменения, став менее строгой. В частности, эта модель теперь не требует усечения промежуточных результатов вычислений. В ряде случаев это позволяет избежать перепол-

нения или потери значимости. Указав перед классом, методом или интерфейсом модификатор `strictfp`, можно сообщить системе, что вычисления с плавающей точкой (и усечение промежуточных результатов соответственно) должны выполняться точно так же, как это происходило в ранних версиях Java. Если же класс помечен ключевым словом `strictfp`, то им же автоматически помечаются и все методы этого класса.

Ключевое слово `assert`

Ключевое слово `assert` используется на стадии разработки программ для создания *утверждений*, т.е. условий, которые, как предполагается, должны быть истинными при выполнении программы. Допустим, создан метод, который должен возвращать только положительное целочисленное значение. Проверить выполнение этого условия можно с помощью утверждения в операторе `assert`. Если при выполнении программы условие становится истинным, то никакие специальные меры не принимаются. А если условие становится ложным, то генерируется исключение `AssertionError`. Утверждения зачастую используются при тестировании программ, а в готовом коде они обычно не применяются.

Утверждение, создаваемое с помощью ключевого слова `assert`, может быть записано в двух общих формах. Первая из них выглядит следующим образом:

```
assert условие;
```

где *условие* обозначает выражение, результатом вычисления которого является логическое значение. Так, если *условие* оказывается истинным (`true`), то и утверждение истинно, и никаких действий не предпринимается. А если *условие* оказывается ложным (`false`), то и утверждение ложно, и в этом случае по умолчанию генерируется исключение `AssertionError`. Рассмотрим следующий пример:

```
assert n > 0;
```

В данном примере исключение возникнет лишь в том случае, если значение переменной `n` окажется меньше или равно нулю. В противном случае программа будет выполнена в обычном порядке.

Вторая общая форма утверждения имеет следующий вид:

```
assert условие : выражение;
```

В данном случае *выражение* дает значение, которое передается конструктору класса `AssertionError`. Это значение преобразуется в свое строковое представление и выводится в том случае, если утверждение оказывается ложным. Обычно *выражение* указывается в виде символьной строки, но оно может быть любого типа, кроме `void`, лишь бы оно допускало корректное преобразование его результата в строковое представление.

Для того чтобы утверждения проверялись при выполнении программы, интерпретатор нужно запустить с параметром `-ea`. Так, если требуется разрешить проверку утверждений в программе `Sample`, в командной строке необходимо указать следующее:

```
java -ea Sample
```

Утверждения очень полезны на стадии разработки программ, поскольку они упрощают и ускоряют поиск ошибок, что и является целью тестирования. Но, применяя данное средство, нужно соблюдать осторожность. В частности, нельзя полагаться на утверждения, собираясь выполнить любые операций, фактически требующиеся в программе. Дело в том, что готовый код может быть запущен и без параметра, разрешающего проверку утверждений, а следовательно требуемые действия не будут выполнены, поскольку выражения в утверждениях не вычисляются.

Собственные методы

Иногда при выполнении программ на Java приходится вызывать подпрограммы, написанные на других языках программирования. Чаще всего такие подпрограммы существуют в исполняемом коде для конкретного ЦП и рабочей среды, т.е. в собственном коде. В одних случаях подпрограммы в собственном коде вызываются для ускорения работы приложения, а в других случаях приходится обращаться к средствам специализированных библиотек независимых производителей, например, к пакету для статистических расчетов. Но поскольку программы на Java компилируются в байт-код, а затем интерпретируются исполняющей системой (или компилируются динамически), то на первый взгляд может показаться, что вызвать подпрограмму в собственном коде из программы на Java невозможно. К счастью, это совсем не так. Для этой цели в Java предусмотрено ключевое слово `native`, с помощью которого объявляются собственные методы. Если метод объявлен как собственный (`native`), он может быть вызван из программы на Java точно так же, как и обычный метод.

Для того чтобы объявить собственный метод, перед его именем следует указать ключевое слово `native`, но не определять тело метода, как показано ниже.

```
public native int meth();
```

Как только собственный метод будет объявлен, необходимо предоставить сам этот метод и предпринять ряд сложных действий по его связыванию с кодом Java.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Раз уж речь зашла о ключевых словах, вернемся к ключевому слову `this`.

Иногда после него указывается выражение в круглых скобках. Например, что означает выражение `this(x);`?

ОТВЕТ. Такая форма записи ключевого слова `this` используется для того, чтобы обеспечить конструктору возможность вызвать другой конструктор того же класса. В общем виде эта форма записи выглядит следующим образом:

```
this(список_аргументов)
```

Когда в программе встречается выражение `this()`, то сначала выполняется перегружаемый конструктор с таким же списком параметров, как и `список_аргументов`. После этого выполняется тело текущего конструктора. Вызов `this()` должен быть первым выражением в теле конструктора. Ниже приведен простой пример использования выражения `this()`.

```
class MyClass {
    int a;
    int b;

    // инициализировать переменные a и b
    MyClass(int i, int j) {
        a = i;
        b = j;
    }

    // использовать выражение this() для инициализации
    // переменных a, b и c одинаковыми значениями
}
```

```

MyClass(int i) {
    this(i, i); // вызвать конструктор MyClass(i, i)
}
}

```

В классе `MyClass` только первый конструктор фактически присваивает значения переменным `a` и `b`, тогда как второй конструктор просто вызывает первый. Таким образом, при вычислении выражения

```
MyClass mc = new MyClass(8);
```

вызов конструктора `MyClass(8)` приводит к вычислению выражения `this(8, 8)`, которое преобразуется в вызов конструктора `MyClass(8, 8)`.

Вызывать перегружаемый конструктор из выражения `this()` нередко оказывается очень удобно, поскольку предотвращается дублирование кода. Но при этом необходимо соблюдать осторожность. Конструкторы, использующие выражения `this()`, выполняются медленнее, чем те, которые содержат весь необходимый инициализирующий код. Причина такого замедления объясняется дополнительными издержками, связанными с вызовом конструктора и возвратом из него. Не следует забывать, что создание объекта оказывает влияние на весь использующий его код. Если предполагается создание большого количества экземпляров одного класса, следует тщательно взвесить, что важнее: получить более компактный код или увеличить время создания объекта. Когда вы приобретете достаточный опыт программирования на Java, вам будет легче принимать подобные решения.

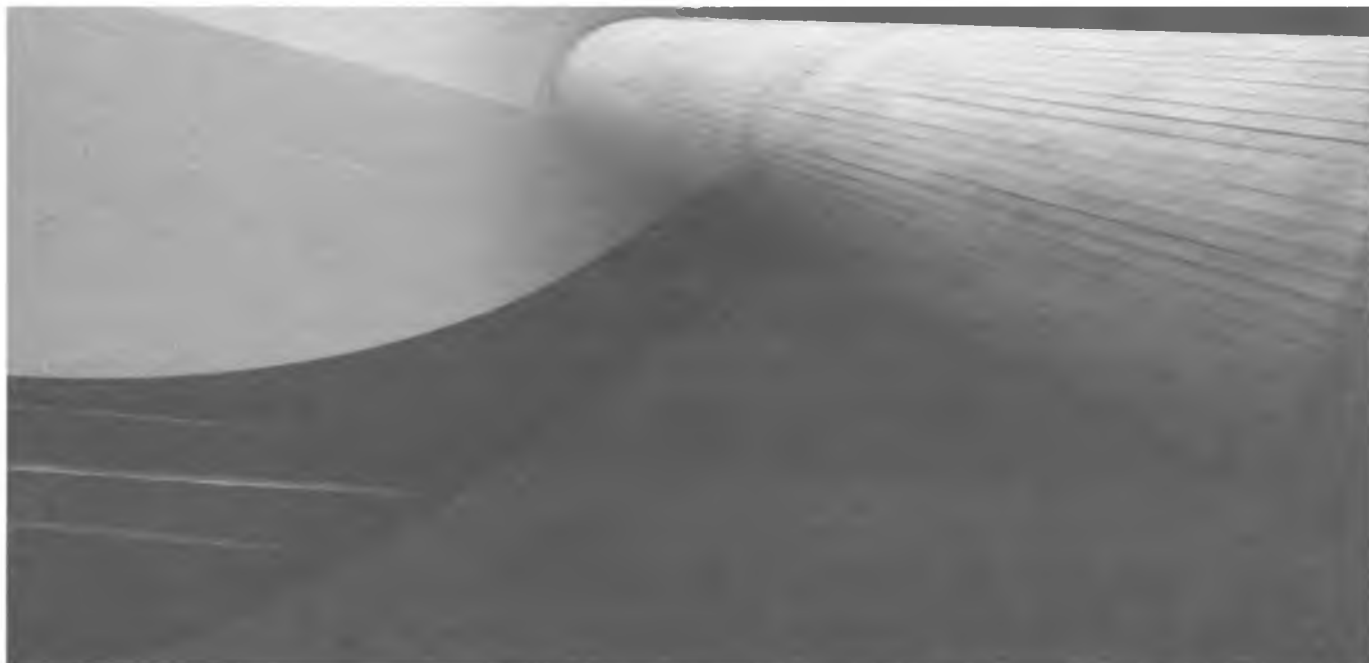
Имеются два ограничения, которые следует учитывать, используя выражение `this()`. Во-первых, при вызове `this()` нельзя использовать переменные экземпляра из того класса, к которому относится конструктор. И во-вторых, в одном конструкторе не могут одновременно присутствовать вызовы `super()` и `this()`, поскольку каждый из них должен быть первым выражением конструктора.

Упражнение для самопроверки по материалу главы 14

1. Какой метод вызывается первым в начале выполнения апплета? Какой метод вызывается, когда апплет должен быть удален из системы?
2. Почему в апплете, который должен работать постоянно, следует организовать многопоточный режим?
3. Видоизмените апплет, созданный в примере для опробования 14.1, таким образом, чтобы в нем отображалась символьная строка, передаваемая ему в качестве параметра. Добавьте еще один параметр, чтобы задавать время задержки (в миллисекундах) между последовательными сдвигами символов строки.
4. Дополнительное задание. Создайте апплет, который отображал бы текущее время, обновляя содержимое своего окна каждую секунду. Для того чтобы справиться с этим заданием, вам придется провести дополнительные изыскания. Для начала примите к сведению следующую подсказку: получить текущее время можно, воспользовавшись объектом класса `Calendar`, относящегося к пакету `java.util`. (Напомним, что компания Oracle предоставляет оперативно доступную документа-

цию на все стандартные классы Java.) Приобретенных вами до сих пор знаний должно быть достаточно для того, чтобы самостоятельно изучить класс `Calendar` и использовать его методы для выполнения этого задания.

5. Поясните вкратце, каким образом действует модель делегирования событий в Java.
6. Должен ли приемник событий быть зарегистрирован в их источнике?
7. Дополнительное задание. Среди языковых средств Java для отображения информации имеется метод `drawLine()`. Он рисует текущим цветом прямую линию между двумя точками. Этот метод относится к классу `Graphics`. Используя метод `drawLine()`, напишите апплет, отслеживающий перемещение мыши. Если кнопка мыши нажата, апплет должен рисовать на экране непрерывную линию до тех пор, пока кнопка не будет отпущена.
8. Объясните назначение ключевого слова `assert`.
9. Приведите хотя бы одну причину, по которой может возникнуть потребность в использовании собственного метода в некоторых программах.
10. Дополнительное задание. Попробуйте ввести поддержку события типа `MouseEvent` в пример апплета `MouseEvents`, рассмотренный ранее в разделе “Применение модели делегирования событий”. С этой целью реализуйте интерфейс `MouseListener` и сделайте апплет приемником события, связанного с прокруткой колесика мыши, воспользовавшись методом `addMouseListener()`. Для выполнения этого задания вам придется обратиться к документации на API языка Java и ознакомиться с упомянутыми выше средствами обработки подобных событий. На этот вопрос ответа не дается, поэтому вам придется призвать на помощь свои знания и навыки, чтобы найти собственное решение.



Глава 15

Введение в Swing

Основные навыки и понятия

- Происхождение и принципы конструирования Swing
- Представление о компонентах и контейнерах Swing
- Основные положения о диспетчерах компоновки
- Написание, компиляция и запуск на выполнение простого Swing-приложения
- Применение компонента JButton
- Работа с компонентом JTextField
- Создание флажков с помощью компонента JCheckBox
- Работа с компонентом Jlist
- Применение анонимных внутренних классов для обработки событий
- Создание апплета средствами Swing

Все программы, приведенные в предыдущих главах в качестве примера, за исключением апплетов, рассмотренных в главе 14, были консольными. Это означает, что в них не применялись средства графического пользовательского интерфейса. Консольные программы очень удобны для изучения основ Java, но в реальных программах без графического пользовательского интерфейса просто не обойтись. Поэтому в последней главе этой книги будет рассмотрена библиотека Swing современных инструментальных средств Java, предназначенных для создания пользовательских интерфейсов в программах на Java.

Библиотека Swing предоставляет богатую коллекцию классов и интерфейсов для создания визуальных компонентов, в том числе кнопок, полей ввода текста, полос прокрутки, флажков и таблиц, способных придать пользовательскому интерфейсу программ на Java современный вид. Средствами Swing можно разработать на высоком профессиональном уровне графический интерфейс прикладной программы, в наибольшей степени удовлетворяющий потребностям пользователей.

Тема Swing весьма обширна, и для подробного ее рассмотрения потребовалась бы отдельная книга, поэтому в одной главе удастся коснуться лишь самых основных вопросов применения Swing. Но и представленного здесь материала окажется достаточно, чтобы составить общее представление о Swing. В этой главе будут рассмотрены пять наиболее употребительных компонентов пользовательского интерфейса, создаваемых средствами Swing: метки, кнопки, поля ввода текста, флажки и списки. А в конце главы будет создан апплет на основе Swing. И хотя в данной главе описана лишь небольшая часть инструментальных средств Swing, изучив их, вы сможете писать простые программы с графическим пользовательским интерфейсом. А полученные знания послужат вам прочным основанием для дальнейшего изучения Swing.

На заметку

Более подробные сведения о Swing можно найти в книге *Swing: Руководство для начинающих*, ИД "Вильямс", 2007 г.

Происхождение и принципы конструирования Swing

В ранних версиях Java средства Swing отсутствовали. Можно считать, что они появились в результате попытки преодолеть трудности, связанные с использованием в Java первой оконной подсистемы — библиотеки Abstract Window Toolkit (AWT). В этой библиотеке определен базовый набор компонентов для создания вполне работоспособных пользовательских интерфейсов, но с ограниченными возможностями. Одним из ограничений AWT стало превращение ее визуальных компонентов в платформенно-зависимые эквиваленты, или *равноправные компоненты*. Это означает, что внешний вид компонентов AWT определялся не средствами Java, а используемой платформой. А поскольку в компонентах AWT применялся в качестве ресурса собственный код, то они назывались *тяжеловесными*.

Применение собственных равноправных компонентов послужило причиной целого ряда осложнений. Во-первых, из-за отличий в операционных системах компоненты по-разному выглядели и даже иначе вели себя на разных платформах. Это было прямым нарушением главного принципа Java: код, написанный один раз, должен работать везде. Во-вторых, внешний вид каждого компонента был фиксированным, и изменить его было очень трудно (причина та же — зависимость от конкретной платформы). И в-третьих, на применение тяжеловесных компонентов накладывался целый ряд обременительных ограничений. В частности, такие компоненты всегда имели прямоугольную форму и были непрозрачны.

Потребовалось не слишком много времени, чтобы осознать, что ограничения AWT слишком серьезны и нужен новый подход. В результате в 1997 году появилась библиотека компонентов Swing, включенная в состав библиотеки классов JFC (Java Foundation Classes). Первоначально Swing использовалась в версии Java 1.1 как отдельная библиотека. Но начиная с версии Java 1.2 средства Swing (как, впрочем, и остальные элементы JFC) были полностью интегрированы в Java.

Swing устраняет ограничения, присущие компонентам AWT, благодаря использованию двух основных средств: *легковесных компонентов* и *подключаемых стилей оформления*. И хотя эти средства практически незаметны для программирующего на Java, именно они составляют основные принципы конструирования Swing и в значительной мере обуславливают возможности и удобство применения этой библиотеки. Рассмотрим каждое из них в отдельности.

Все компоненты Swing, за небольшим исключением, являются *легковесными*, а это означает, что они полностью написаны на Java и не зависят от конкретной платформы, поскольку не опираются на платформенно-зависимые равноправные компоненты. Легковесные компоненты обладают рядом существенных преимуществ, к числу которых относятся эффективность и гибкость. Например, легковесный компонент может быть прозрачным и непрямоугольным по форме. Легковесные компоненты не преобразуются в платформенно-зависимые равноправные компоненты, и поэтому их внешний вид определяет библиотека Swing, а не базовая операционная система. Следовательно, элементы пользовательского интерфейса, созданные средствами Swing, выглядят одинаково на разных платформах.

Благодаря тому что каждый компонент Swing воспроизводится в коде Java, а не опирается на платформенно-зависимые равноправные компоненты, стиль оформления компонента можно отделить от логики его функционирования, что и делается средствами Swing. Такое разделение позволяет изменить внешний вид компонента, не затрагивая других его свойств. Иными словами, появляется возможность менять стиль оформления любого компонента без побочных эффектов, оказывающих влияние на его применение.

Каждому пользователю Swing изначально доступны различные стили оформления, к числу которых относятся металлический (Metal) и Motif (стандартный для ОС UNIX). Металлический стиль называется также *стилем оформления Java*. Это платформенно-независимый стиль оформления, доступный во всех средах выполнения программ на Java. Он же принимается по умолчанию, поэтому именно он и будет использоваться в примерах, представленных в этой главе.

Реализация подключаемых стилей оформления в Swing стала возможной благодаря тому, что при создании Swing был использован видоизмененный вариант классической архитектуры *модель—представление—контроллер* (MVC). В терминологии MVC *модель* определяет состояние компонента. Например, для флажка модель содержит поле, обозначающее состояние установки или сброса флажка. *Представление* определяет порядок отображения компонента на экране, не исключая ничего, на что может оказывать влияние текущее состояние модели. А *контроллер* определяет порядок реагирования компонента на действия пользователя. Так, если пользователь щелкнет кнопкой мыши на флажке, контроллер отреагирует, изменив модель таким образом, чтобы отразить выбор пользователя (установку или сброс флажка). В ответ на действия пользователя обновляется и представление. Разделяя компонент на модель, представление и контроллер, можно добиться того, что реализация одной из этих составляющих компонента не будет оказывать влияние на две другие. Например, в некоторых реализациях представления один и тот же компонент может отображаться разными способами, а модель и контроллер — оставаться без изменения.

Несмотря на то что принципы построения архитектуры MVC выглядят очень привлекательно, для компонентов Swing разделение функций между представлением и контроллером не дает ощутимых преимуществ. Поэтому в Swing применяется видоизмененный вариант MVC, в котором представление и контроллер объединены в единую логическую сущность, называемую *представителем* пользовательского интерфейса. В связи с этим принятый в Swing подход называется архитектурой *модель—представитель*, а иначе — архитектурой *с разделенной моделью*. Таким образом, компоненты Swing нельзя рассматривать как классическую реализацию архитектуры MVC, хотя их архитектура и опирается на нее. В процессе разработки вам не придется иметь дело непосредственно с моделями или представителями пользовательского интерфейса, но они будут незримо присутствовать в создаваемых вами программах.

Прорабатывая материал этой главы, вы обнаружите, что библиотека Swing очень проста в использовании, несмотря на то, что она построена на довольно сложных прин-

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как пояснялось выше, Swing определяет внешний вид современных Java-приложений с графическим пользовательским интерфейсом. Означает ли это, что средства Swing фактически заменили AWT?

ОТВЕТ. Нет, не означает. Swing нельзя рассматривать как полную замену AWT. Напротив, библиотека Swing была построена на основе библиотеки AWT, а следовательно, AWT остается важной составной частью Java. В Swing применяется тот же самый механизм обработки событий, что и в AWT (см. главу 14). Для изучения материала этой главы знать AWT не обязательно, но если вы стремитесь изучить Swing углубленно, вам придется основательно разобраться в структуре и средствах AWT, чтобы в полной мере овладеть Swing.

циях конструирования. Одним из аргументов в пользу Swing служит то обстоятельство, что эта библиотека позволяет сделать вполне управляемым такой сложный процесс, как построение пользовательского интерфейса. Это дает разработчикам возможность сосредоточить основное внимание на самом графическом пользовательском интерфейсе приложения, не отвлекаясь на детали его реализации.

Компоненты и контейнеры

В состав графического пользовательского интерфейса, создаваемого средствами Swing, входят две основные разновидности элементов: *компоненты* и *контейнеры*. Такое разделение во многом условно, поскольку контейнеры являются в то же время компонентами, а отличаются друг от друга своим назначением. Компонент — это независимый элемент (например, кнопка или поле ввода текста), а контейнер может содержать в себе несколько компонентов. Следовательно, контейнер — это особая разновидность компонента. Для того чтобы отобразить компонент на экране, его следует поместить в контейнер. Поэтому в графическом пользовательском интерфейсе должен присутствовать хотя бы один контейнер. А так как контейнеры являются в то же время компонентами, то один контейнер может содержать в себе другой. Это дает возможность сформировать так называемую *иерархию контейнеров*, на вершине которой должен находиться *контейнер верхнего уровня*.

Компоненты

подавляющее большинство компонентов Swing создаются с помощью классов, производных от класса `JComponent`. (Единственным исключением из этого правила являются четыре контейнера верхнего уровня, которые будут описаны в следующем разделе.) В классе `JComponent` реализуются функциональные возможности, общие для всех компонентов. Например, в классе `JComponent` поддерживаются подключаемые стили оформления. Этот класс наследует свойства классов `Container` и `Component` из библиотеки AWT. Таким образом, компоненты Swing создаются на основе компонентов AWT и совместимы с ними.

Классы, представляющие все компоненты Swing, входят в пакет `javax.swing`. В приведенной ниже таблице перечислены имена классов всех компонентов Swing (включая компоненты, используемые как контейнеры).

<code>JApplet</code>	<code>JButton</code>	<code>JCheckBox</code>	<code>JCheckBoxMenuItem</code>
<code>JColorChooser</code>	<code>JComboBox</code>	<code>JComponent</code>	<code>JDesktopPane</code>
<code>JDialog</code>	<code>JEditorPane</code>	<code>JFileChooser</code>	<code>JFormattedTextField</code>
<code>JFrame</code>	<code>JInternalFrame</code>	<code>JLabel</code>	<code>JLayeredPane</code>
<code>JList</code>	<code>JMenu</code>	<code>JMenuBar</code>	<code>JMenuItem</code>
<code>JOptionPane</code>	<code>JPanel</code>	<code>JPasswordField</code>	<code>JPopupMenu</code>
<code>JProgressBar</code>	<code>JRadioButton</code>	<code>JRadioButtonMenuItem</code>	<code>JRootPane</code>
<code>JScrollBar</code>	<code>JScrollPane</code>	<code>JSeparator</code>	<code>JSlider</code>
<code>JSpinner</code>	<code>JSplitPane</code>	<code>JTabbedPane</code>	<code>JTable</code>
<code>JTextArea</code>	<code>TextField</code>	<code>JTextPane</code>	<code>JToggleButton</code>
<code>JToolBar</code>	<code>JToolTip</code>	<code>JTree</code>	<code>JViewport</code>
<code>JWindow</code>			

Как видите, имена всех классов начинаются с буквы “J”. Например, метке соответствует класс `JLabel`, кнопке — класс `JBUTTON`, а флажку — класс `JCheckBox`.

Как пояснялось в начале этой главы, в рамках данной книги нет возможности рассмотреть все компоненты Swing: для этого потребовалась бы отдельная книга. Но в этой главе будут представлены пять наиболее употребительных компонентов: `JLabel`, `JBUTTON`, `JTextField`, `JCheckBox` и `JList`. Если вы уясните их принцип действия, вам будет легче овладеть другими компонентами.

Контейнеры

В Swing определены две разновидности контейнеров. К первой из них относятся следующие контейнеры верхнего уровня: `JFrame`, `JApplet`, `JWindow` и `JDialog`. Они не наследуют переменные и методы от класса `JComponent`, тем не менее являются производными от классов `Component` и `Container`. В отличие от других, легковесных компонентов Swing, контейнеры верхнего уровня являются тяжеловесными. Именно поэтому контейнеры верхнего уровня составляют отдельную группу в библиотеке Swing.

Как следует из названия контейнеров верхнего уровня, они должны находиться на вершине иерархии контейнеров и не могут содержаться в других контейнерах. Более того, любая иерархия должна начинаться именно с контейнера верхнего уровня. В прикладных программах чаще всего используется контейнер типа `JFrame`, а в апплетах — контейнер типа `JApplet`.

Контейнеры второго рода являются легковесными и производными от класса `JComponent`. В качестве примера легковесных контейнеров можно привести классы `JPanel`, `JScrollPane` и `JRootPane`. Легковесные контейнеры могут содержаться в других контейнерах, и поэтому они нередко используются для объединения группы взаимосвязанных компонентов.

Панели контейнеров верхнего уровня

В каждом контейнере верхнего уровня определен набор панелей. На вершине иерархии находится *корневая панель* — экземпляр класса `JRootPane`, который представляет собой легковесный контейнер, предназначенный для управления другими панелями. Он также позволяет управлять строкой меню. Корневая панель включает в себя *прозрачную панель*, *панель содержимого* и *многослойную панель*.

Прозрачная панель — это панель верхнего уровня. Она расположена над всеми остальными панелями, скрывая их. Прозрачная панель позволяет управлять событиями от мыши, относящимися ко всему контейнеру, а не только к содержащимся в нем компонентам. С ее помощью можно также прорисовывать любые компоненты. Как правило, разработчикам нет особой необходимости пользоваться непосредственно прозрачной панелью.

Многослойная панель позволяет отображать компоненты в глубину, определяя порядок перекрытия одних компонентов другими. (Это означает, что панель слоя позволяет задавать расположение компонентов в глубину, хотя такая возможность используется сравнительно редко.) В состав многослойной панели входит панель содержимого и может также включаться строка меню.

Несмотря на то что прозрачная и многослойная панели являются неотъемлемыми частями контейнера верхнего уровня и выполняют важные функции, их действия по большей части скрыты не только от пользователей, но и от разработчиков прикладных программ.

Прикладная программа взаимодействует в основном с панелью содержимого, поскольку именно в нее включаются визуальные компоненты. Иными словами, добавляя

компонент, например кнопку, в контейнер верхнего уровня, вы на самом деле вводите его на панели содержимого.

Диспетчеры компоновки

Прежде чем приступить к написанию программ средствами Swing, необходимо составить себе хотя бы общее представление о диспетчерах компоновки. *Диспетчер компоновки* управляет размещением компонентов в контейнере. В Java определено несколько таких диспетчеров. Большинство из них входит в состав AWT (т.е. в пакет `java.awt`), но в Swing также предоставляется ряд дополнительных диспетчеров компоновки. Все диспетчеры компоновки являются экземплярами классов, реализующих интерфейс `LayoutManager`. (Некоторые из диспетчеров компоновки реализуют интерфейс `LayoutManager2`.) Ниже перечислен ряд диспетчеров компоновки, доступных для разработчиков прикладных программ, пользующихся средствами Swing.

<code>FlowLayout</code>	Простой диспетчер компоновки, размещающий компоненты слева направо и сверху вниз. (Для некоторых региональных настроек компоненты располагаются справа налево.)
<code>BorderLayout</code>	Располагает компоненты по центру и по краям контейнера. Этот диспетчер компоновки принимается по умолчанию для панели содержимого
<code>GridLayout</code>	Располагает компоненты сеткой, как в таблице
<code>GridBagLayout</code>	Располагает компоненты разных размеров настраиваемой сеткой, как в таблице
<code>BoxLayout</code>	Располагает компоненты в блоке по вертикали или по горизонтали
<code>SpringLayout</code>	Располагает компоненты с учетом ряда ограничений

Диспетчеры компоновки, как и многие другие компоненты Swing, невозможно обсудить во всех подробностях в одной главе. Поэтому ограничимся рассмотрением только двух из них: `BorderLayout` и `FlowLayout`.

Для панели содержимого по умолчанию принимается диспетчер компоновки `BorderLayout`. Этот диспетчер компоновки определяет в составе контейнера пять областей, в которые могут помещаться компоненты. Первая область располагается по середине и называется *центральной*. Остальные четыре размещаются по сторонам света и соответственно называются *северной*, *южной*, *восточной* и *западной*. По умолчанию компонент, добавляемый на панели содержимого, располагается в центральной области. Для того чтобы расположить компонент в другой области, следует указать ее имя.

Несмотря на то что возможностей, предоставляемых диспетчером компоновки `BorderLayout`, зачастую оказывается достаточно, иногда возникает потребность в других диспетчерах компоновки. К числу самых простых относится диспетчер компоновки `FlowLayout`. Он размещает компоненты построчно: слева направо и сверху вниз. Заполнив текущую строку, этот диспетчер компоновки переходит к следующей. Такая компоновка предоставляет лишь ограниченный контроль над расположением компонентов, хотя и проста в употреблении. Однако при изменении размеров контейнера расположение компонентов может измениться.

Первая простая Swing-программа

Программы, создаваемые средствами Swing, называемые в дальнейшем для краткости Swing-программами, отличаются от консольных программ, примеры которых были

рассмотрены ранее в этой книге. Они отличаются и от апплетов, создаваемых на основе AWT и обсуждавшихся в главе 14. Swing-программы не просто настраивают применение компонентов Swing, обеспечивающих взаимодействие с пользователем, но и должны отвечать особым требованиям, связанным с организацией поточной обработки. Для того чтобы стала понятнее структура Swing-программы, лучше всего обратиться к конкретному примеру. Программы, в которых применяются компоненты Swing, можно условно разделить на две категории. К первой категории относятся обычные программы для настольных систем, которые зачастую называются настольными приложениями, а ко второй категории — апплеты. В этом разделе будет показано, каким образом создается Swing-приложение. А создание Swing-апплета будет рассмотрено далее в главе.

Несмотря на то что рассматриваемый здесь пример программы довольно прост, он наглядно демонстрирует один из приемов написания Swing-приложений. В данной программе используются два компонента Swing: классы `JFrame` и `JLabel`. Класс `JFrame` представляет собой контейнер верхнего уровня, нередко применяемый в Swing-приложениях, а класс `JLabel` — компонент Swing, с помощью которого создается метка, используемая для отображения информации. Метка является самым простым компонентом Swing, поскольку она не реагирует на действия пользователя, а только помечает отображаемую информацию. Контейнер `JFrame` служит для размещения экземпляра компонента `JLabel`. С помощью метки отображается короткое текстовое сообщение.

```
// Простая Swing-программа.
```

```
// Для каждой Swing-программы импортируется пакет javax.swing.
import javax.swing.*;
```

```
class SwingDemo {
```

```
    SwingDemo() {
```

```
        // Создание нового контейнера JFrame.
```

```
        JFrame jfrm = new JFrame("A Simple Swing Application");
```

```
        // Установка начальных размеров рамки окна.
```

```
        jfrm.setSize(275, 100);
```

```
        // При закрытии окна программа должна завершиться.
```

```
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        // Создание текстовой метки с помощью компонента Swing.
```

```
        JLabel jlab = new JLabel(" Swing defines the modern Java GUI.");
```

```
        // Добавление метки на панели содержимого.
```

```
        jfrm.add(jlab);
```

```
        // Отображение рамки окна.
```

```
        jfrm.setVisible(true);
```

```
    }
```

```
public static void main(String args[]) {
```

```
    // Объект SwingDemo должен быть создан в потоке
```

```
    // диспетчеризации событий.
```

```
    SwingUtilities.invokeLater(new Runnable() {
```

```

        public void run() {
            new SwingDemo();
        }
    });
}
}

```

Эта программа компилируется и запускается таким же образом, как и любое другое Java-приложение. Для ее компиляции в командной строке нужно ввести следующее:

```
javac SwingDemo1.java
```

А для запуска программы на выполнение в командной строке нужно ввести следующее:

```
java SwingDemo1
```

При выполнении данной программы отображается окно, приведенное на рис. 15.1.

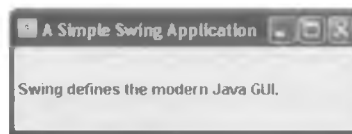


Рис. 15.1. Окно, отображаемое при выполнении программы SwingDemo

Построчный анализ первой Swing-программы

Пример программы SwingDemo демонстрирует основные понятия Swing, поэтому имеет смысл проанализировать ее подробно, уделив внимание каждой строке кода. В начале программы осуществляется импорт пакета, как показано ниже.

```
import javax.swing.*;
```

Этот пакет содержит компоненты и модели Swing. В частности, в нем определены классы, реализующие метки, кнопки, поля ввода текста и меню. Этот пакет должен быть непременно включен в каждую программу, использующую библиотеку Swing.

Далее в программе объявляется класс SwingDemo и его конструктор. Именно в конструкторе выполняется большая часть действий этой программы. Код конструктора начинается с создания объекта типа JFrame. Для этой цели служит следующая строка кода:

```
JFrame jfrm = new JFrame("A Simple Swing Application.");
```

Подобным образом создается контейнер jfrm, который определяет прямоугольное окно, содержащее строку заголовка, кнопки, предназначенные для закрытия, сворачивания, разворачивания и восстановления нормальных размеров окна, а также системное меню. Иными словами, в приведенной выше строке кода создается стандартное окно верхнего уровня, а строка заголовка передается конструктору в качестве параметра.

Далее задаются размеры окна в пикселях, как показано ниже.

```
jfrm.setSize(275, 100);
```

Для этой цели вызывается метод setSize(). Ниже приведена общая форма объявления этого метода.

```
void setSize(int ширина, int высота)
```

В рассматриваемом здесь примере задается ширина окна — 275 пикселей и высота — 100 пикселей.

По умолчанию, когда окно верхнего уровня закрывается (например, по щелчку на кнопке в его верхнем правом углу), оно удаляется с экрана, но приложение не завершает на этом свою работу. И хотя такое поведение подходит для некоторых ситуаций, в большинстве случаев оно неприемлемо. Ведь, как правило, желательно, чтобы при закрытии окна верхнего уровня приложение завершалось. И добиться этого можно несколькими способами. Самый простой из них состоит в том, чтобы вызвать метод `setDefaultCloseOperation()`. Именно такой способ и применен в анализируемой здесь программе, как показано ниже.

```
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

После выполнения данного метода закрытие окна приведет к завершению работы всего приложения. Ниже приведена общая форма объявления метода `setDefaultCloseOperation()`.

```
void setDefaultCloseOperation(int what)
```

Значение, передаваемое в качестве параметра *what*, определяет, что именно должно произойти при закрытии окна. Помимо константы `JFrame.EXIT_ON_CLOSE`, данному методу можно передавать и другие, приведенные ниже константы.

```
JFrame.DISPOSE_ON_CLOSE  
JFrame.HIDE_ON_CLOSE  
JFrame.DO_NOTHING_ON_CLOSE
```

Имена констант отражают их назначение: освободить, скрыть, ничего не делать после закрытия окна соответственно. Все они определены в интерфейсе `WindowConstants`, входящем в пакет `javax.swing`. Этот интерфейс реализуется в классе `JFrame`.

В следующей строке кода создается компонент `JLabel` из библиотеки `Swing`:

```
JLabel jlab = new JLabel(" Swing defines the modern Java GUI.");
```

Компонент `JLabel` — самый простой в использовании среди всех компонентов `Swing`, поскольку он не предполагает обработку событий, связанных с действиями пользователя, а только отображает информацию: текст, изображение или и то и другое. Метка, созданная в данной программе, содержит только текст. А строка, содержащая этот текст, передается конструктору класса `JLabel`.

В следующей строке кода метка вводится на панели содержимого рамки окна:

```
jfrm.add(jlab);
```

Как пояснялось ранее, каждый контейнер верхнего уровня включает в себя панель содержимого, на которой располагаются компоненты. Поэтому для размещения компонента внутри рамки окна его следует добавить на панели содержимого. С этой целью вызывается метод `add()` из класса `JFrame` (в данном случае ссылка на объект типа `JFrame` содержится в переменной `jfrm`). Существует несколько вариантов метода `add()`. Чаще других используется такой вариант:

```
Component add(Component компонент)
```

По умолчанию на панели содержимого, связанной с контейнером `JFrame`, используется диспетчер компоновки `BorderLayout`. В приведенном выше варианте метода `add()` компонент (в данном случае — метка) располагается в центральной области. Имеются также варианты `add()`, позволяющие располагать компоненты в других областях. При размещении в центральной области размеры компонента автоматически приводятся к размерам этой области.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Мне встречались Swing-программы, в которых при добавлении компонента на панели содержимого вызывался метод `getContentPane()`. Что это за метод и зачем он вообще нужен?

ОТВЕТ. Этот вопрос напрямую связан с историей развития Java. До появления версии JDK 5 при добавлении компонента на панели содержимого нельзя было вызвать метод `add()` непосредственно для объекта типа `JFrame`. Этот метод приходилось вызывать, ссылаясь явным образом на панель содержимого с объектом типа `JFrame`. А для получения ссылки на панель содержимого вызывался метод `getContentPane()` из класса `JFrame`. Ниже приведена общая форма объявления метода `getContentPane()`.

```
Container getContentPane()
```

Этот метод возвращает ссылку типа `Container` на панель содержимого. Полученная ссылка позволяет вызвать метод `add()` и добавить с его помощью компонент на панели содержимого. Следовательно, для того чтобы добавить компонент `jlab` в контейнер `jfrm`, в ранних версиях Java потребовался бы следующий вызов:

```
jfrm.getContentPane().add(jlab); // требуется для ранних версий Java
```

В этом вызове с помощью метода `getContentPane()` сначала извлекается ссылка на панель содержимого, а затем с помощью метода `add()` компонент добавляется в контейнер, связанный с данной панелью. Аналогичная процедура требуется для вызова метода `remove()`, удаляющего компонент, а также метода `setLayout()`, задающего диспетчер компоновки для панели содержимого. Явные вызовы метода `getContentPane()` использовались до появления версии JDK 5, но теперь потребность в его вызове отпала. Методы `add()`, `remove()` и `setLayout()` можно вызывать непосредственно для объекта типа `JFrame`, поскольку эти методы теперь автоматически обращаются к панели содержимого.

И последний оператор в конструкторе класса `SwingDemo` обеспечивает отображение окна, как показано ниже.

```
jfrm.setVisible(true);
```

Общая форма объявления метода `setVisible()` выглядит следующим образом:

```
void setVisible(boolean флаг)
```

Если параметр *флаг* принимает логическое значение `true`, окно отображается на экране, в противном случае оно остается скрытым. По умолчанию рамку окна (объект типа `JFrame`) не видно, поэтому для ее отображения требуется вызов `setVisible(true)`.

В методе `main()` создается объект класса `SwingDemo`, в результате чего окно и метка отображаются на экране. Конструктор класса `SwingDemo` вызывается в следующем фрагменте кода:

```
SwingUtilities.invokeLater(new Runnable() {
    public void run() {
        new SwingDemo();
    }
});
```

В этом фрагменте кода объект типа `SwingDemo` создается не в основном потоке приложения, а в *потоке диспетчеризации событий*. Такое решение принимается по ряду причин. Прежде всего, Swing-программы, как правило, управляются событиями. Так, если пользователь активизирует компонент пользовательского интерфейса, формируется соответствующее событие. Оно передается прикладной программе путем вызова обработчика событий, определенного в этой программе. Но этот обработчик выполняется в специальном потоке диспетчеризации событий, формируемом средствами Swing, а не в главном потоке прикладной программы. Таким образом, поток, в котором выполняется этот обработчик событий, создается другими средствами, хотя все обработчики событий определяются в самой программе. Во избежание осложнений, связанных, например, с попытками двух потоков одновременно обновить один и тот же компонент, все компоненты пользовательского интерфейса из библиотеки Swing должны создаваться и обновляться не в основном потоке приложения, а в потоке диспетчеризации событий. Но ведь метод `main()` выполняется в основном потоке, и поэтому в нем нельзя непосредственно создавать объект класса `SwingDemo`. Сначала следует построить объект типа `Runnable`, выполняемый в потоке диспетчеризации событий, а затем предоставить ему возможность построить графический пользовательский интерфейс.

Для того чтобы активизировать код построения графического пользовательского интерфейса в потоке диспетчеризации событий, следует воспользоваться одним из двух методов, определенных в классе `SwingUtilities`: `invokeLater()` или `invokeAndWait()`. Эти методы объявляются следующим образом:

```
static void invokeLater(Runnable obj)
```

```
static void invokeAndWait(Runnable obj)
    throws InterruptedException, InvocationTargetException
```

Параметр `obj` обозначает объект типа `Runnable`, метод `run()` которого вызывается в потоке диспетчеризации событий. Оба упомянутых выше метода отличаются тем, что метод `invokeLater()` сразу же возвращает управление вызывающему методу, тогда как метод `invokeAndWait()` ожидает завершения метода `obj.run()`. Их можно использовать для вызова метода, выполняющего построение пользовательского интерфейса Swing-приложения, а также в тех случаях, когда требуется изменить состояние этого интерфейса из метода, выполняющегося за пределами потока диспетчеризации событий. Для этой цели обычно используется метод `invokeLater()`, что и было сделано в рассматриваемом здесь примере. Но при создании пользовательского интерфейса для апплета лучше воспользоваться методом `invokeAndWait()`. (О создании Swing-апплета речь пойдет далее в этой главе.)

Следует отметить еще одну особенность анализируемой здесь программы: она не реагирует на действия пользователя, поскольку в компоненте `JLabel` не предусмотрены соответствующие средства. Иными словами, в этой программе не были предусмотрены обработчики событий, потому что компонент `JLabel` вообще не формирует события. А все остальные компоненты Swing формируют события, на которые программа должна каким-то образом реагировать, что и будет продемонстрировано на конкретных примерах далее в главе.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как упоминалось выше, компоненты можно добавлять и в другие области, помимо центральной, и для этого предусмотрены специальные варианты метода `add()`. Нельзя ли рассказать об этом подробнее?

ОТВЕТ. Как пояснялось ранее, диспетчер компоновки `BorderLayout` формирует в контейнере пять областей, в которых можно размещать компоненты. Первой является центральная область, а четыре остальные области располагаются по сторонам света и называются северной, южной, восточной и западной соответственно. По умолчанию компонент, добавляемый на панель содержимого, располагается в центральной области. Для того чтобы указать другое расположение компонента, следует воспользоваться приведенным ниже вариантом метода `add()`.

```
void add(Component comp, Object loc)
```

Здесь параметр `comp` задает компонент, добавляемый на панели содержимого, а параметр `loc` определяет область, в которой этот компонент будет размещен. Допускаются следующие значения параметра `loc`:

```
BorderLayout.CENTER      BorderLayout.EAST      BorderLayout.NORTH
BorderLayout.SOUTH      BorderLayout.WEST
```

Чаще всего диспетчер компоновки `BorderLayout` используется при создании контейнера `JFrame`, содержащего центральный компонент (или группу компонентов, размещаемых в легковесном контейнере), а также связанные с ним компоненты верхнего и нижнего колонтитулов. А в остальных случаях более пригодными оказываются другие диспетчеры компоновки.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Мне встречались Swing-программы, в которых не применялся ни метод `invokeLater()`, ни `invokeAndWait()`. Насколько эти методы нужны?

ОТВЕТ. На ранних этапах развития библиотеки Swing считалось допустимым отображать элементы графического пользовательского интерфейса в основном потоке приложения. Поэтому созданные достаточно давно Swing-приложения не содержали вызовы методов `invokeLater()` и `invokeAndWait()` при построении пользовательского интерфейса. Но с тех пор точка зрения разработчиков данной библиотеки изменилась. Теперь принято считать, что построение пользовательского интерфейса некорректно начинать в основном потоке, поскольку это может иногда приводить к серьезным осложнениям. Существует немало Swing-приложений, в которых элементы пользовательского интерфейса инициализируются в основном потоке и, тем не менее, оказываются вполне работоспособными. (Такое решение можно было принять и в рассмотренной выше программе в силу ее чрезвычайной простоты.) Но поскольку пользовательский интерфейс рекомендуется создавать в потоке диспетчеризации событий, именно такой подход и применяется в примерах программ, представленных в этой главе.

Применение компонента `JButton`

Кнопка является одним из наиболее употребительных компонентов Swing. Нажимаемая кнопка представлена в Swing экземпляром класса `JButton`. Этот класс является производным от абстрактного класса `AbstractButton`, в котором определены функции, общие для всех кнопок. На кнопке может отображаться текст надписи,

изображение или и то и другое, но здесь и далее будут рассматриваться только кнопки с текстовыми надписями.

Класс `JButton` содержит три конструктора. Один из них имеет следующий вид:

```
JButton(String сообщение)
```

Параметр *сообщение* определяет символьную строку, которая должна отображаться в виде надписи на кнопке.

После щелчка на кнопке формируется событие `ActionEvent`. Класс `ActionEvent` определен в библиотеке `AWT`, но используется также и в библиотеке `Swing`. В классе `JButton` предоставляются методы, позволяющие зарегистрировать приемник событий или отменить его регистрацию:

```
void addActionListener(ActionListener al)
void removeActionListener(ActionListener al)
```

где параметр *al* задает объект, который будет уведомляться о наступлении событий. Объект должен представлять собой экземпляр класса, реализующего интерфейс `ActionListener`.

В интерфейсе `ActionListener` определен только один метод: `actionPerformed()`. Ниже приведена общая форма объявления этого метода.

```
void actionPerformed(ActionEvent ae)
```

Данный метод вызывается при щелчке на кнопке. Следовательно, в этом методе осуществляется обработка событий, связанных с действиями пользователя над кнопкой. Реализуя метод `actionPerformed()`, необходимо позаботиться о том, чтобы он быстро выполнял свои функции и возвращал управление. Как пояснялось ранее, обработчики событий не должны выполнять длительные операции, поскольку это может привести к замедлению работы приложения в целом. Если же обработка события предполагает действия, требующие времени, их следует выполнять в отдельном потоке, специально создаваемом для этой цели.

С помощью объекта типа `ActionEvent`, передаваемого методу `actionPerformed()`, можно получить важные сведения о событии, связанном с щелчком на кнопке. В этой главе для этой цели будет использоваться символьная строка *команды действия*, связанная с кнопкой. По умолчанию именно эта символьная строка отображается внутри кнопки. Получить команду действия можно, вызвав метод `getActionCommand()` для объекта события, который объявляется следующим образом:

```
String getActionCommand()
```

Команда действия идентифицирует кнопку. Когда в пользовательском интерфейсе приложения имеется несколько кнопок, команда действия позволяет достаточно просто определить, какая из них была выбрана.

Ниже приведен пример программы, демонстрирующий применение кнопки, реагирующей на действия пользователя. А на рис. 15.2 показано окно, отображаемое данной программой на экране.

```
// Демонстрация нажатия кнопки и обработки событий действия.
```

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
class ButtonDemo implements ActionListener {
```

```

JLabel jlab;

ButtonDemo() {

    // создать новый контейнер JFrame
    JFrame jfrm = new JFrame("A Button Example");

    // установить диспетчер компоновки FlowLayout
    jfrm.setLayout(new FlowLayout());

    // задать исходные размеры рамки окна
    jfrm.setSize(220, 90);

    // завершить программу после закрытия окна
    jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // Создание двух кнопок.
    JButton jbtnUp = new JButton("Up");
    JButton jbtnDown = new JButton("Down");

    // Добавление приемников событий от кнопки.
    jbtnUp.addActionListener(this);
    jbtnDown.addActionListener(this);

    // Добавление кнопок на панели содержимого.
    jfrm.add(jbtnUp);
    jfrm.add(jbtnDown);

    // создать метку
    jlab = new JLabel("Press a button.");

    // добавить метку в рамке окна
    jfrm.add(jlab);

    // отобразить рамку окна
    jfrm.setVisible(true);
}

// Обработка событий от кнопки.
public void actionPerformed(ActionEvent ae) {
    // Для определения нажатой кнопки используется команда действия.
    if(ae.getActionCommand().equals("Up"))
        jlab.setText("You pressed Up.");
    else
        jlab.setText("You pressed down. ");
}

public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new ButtonDemo();
        }
    });
}
}

```




Рис. 15.2. Окно, отображаемое при выполнении программы ButtonDemo

Проанализируем приведенную выше программу, обратив внимание на новые и рассматривавшиеся до сих пор компоненты. Сначала в этой программе импортируются пакеты `java.awt` и `java.awt.event`. Пакет `java.awt` необходим, поскольку он содержит класс диспетчера компоновки `FlowLayout`, а пакет `java.awt.event` — потому, что в нем определены интерфейс `ActionListener` и класс `ActionEvent`.

Далее в программе объявляется класс `ButtonDemo`, который реализует интерфейс `ActionListener`. Это означает, что объекты типа `ButtonDemo` могут быть использованы для приема и обработки событий действия. Затем объявляется ссылка на объект типа `JLabel`. Она будет использована в методе `actionPerformed()` для отображения сведений о том, какая именно кнопка была нажата.

Конструктор класса `ButtonDemo` начинается с создания контейнера `jfrm` типа `JFrame`. Затем в качестве диспетчера компоновки для панели содержимого контейнера `jfrm` устанавливается `FlowLayout`, как показано ниже.

```
jfrm.setLayout(new FlowLayout());
```

Как пояснялось ранее, по умолчанию на панели содержимого в качестве диспетчера компоновки используется `BorderLayout`, но для многих приложений лучше подходит диспетчер компоновки `FlowLayout`. Он размещает компоненты построчно: слева направо и сверху вниз. После заполнения текущей строки этот диспетчер компоновки переходит к следующей строке. Такая компоновка предоставляет лишь ограниченный контроль над расположением компонентов, хотя и проста в употреблении. Но следует иметь в виду, что при изменении размеров контейнера расположение компонентов может измениться.

После установки размеров рамки окна и определения операции, завершающей программу, в конструкторе `ButtonDemo()` создаются две кнопки:

```
JButton jbbtnUp = new JButton("Up");
JButton jbbtnDown = new JButton("Down");
```

На первой кнопке отображается надпись `Up` (Нажато), а на второй — `Down` (Отпущено).

Далее с кнопками связывается приемник событий действия, в роли которого выступает экземпляр класса `ButtonDemo`, а ссылка на него передается с помощью ключевого слова `this`. Соответствующие строки кода приведены ниже.

```
jbbtnUp.addActionListener(this);
jbbtnDown.addActionListener(this);
```

В результате выполнения этих строк кода объект, создающий кнопки, будет также получать уведомление об их нажатии.

Всякий раз, когда кнопка нажимается, формируется событие, о котором зарегистрированные приемники уведомляются в результате вызова метода `actionPerformed()`. Объект типа `ActionEvent`, представляющий событие от кнопки, передается этому методу в качестве параметра. В программе `ButtonDemo` это событие передается следующей реализации метода `actionPerformed()`:

```
// Обработка событий от кнопки.
public void actionPerformed(ActionEvent ae) {
```

```

if(ae.getActionCommand().equals("Up"))
    jlab.setText("You pressed Up.");
else
    jlab.setText("You pressed down. ");
}

```

Для передачи события служит параметр *ae*. В теле метода извлекается команда действия, которая соответствует кнопке, сформировавшей событие. Для получения команды действия вызывается метод `getActionCommand()`. (Напомним: по умолчанию команда действия совпадает с текстом, отображаемым на кнопке.) В зависимости от содержания символьной строки, представляющей команду действия, устанавливается текст надписи на кнопке, указывающий на то, какая именно кнопка была нажата.

Следует также иметь в виду, что метод `actionPerformed()` вызывается в потоке диспетчеризации событий. Он должен завершаться как можно быстрее, чтобы не замедлять работу приложения.

Работа с компонентом `JTextField`

К числу широко используемых компонентов Swing относится также компонент `JTextField`, который дает пользователю возможность вводить и редактировать текстовую строку. Компонент `JTextField` является подклассом, производным от абстрактного класса `JTextComponent`, который выступает в роли суперкласса не только для компонента `JTextField`, но и для всех текстовых компонентов вообще. В классе `JTextField` определен ряд конструкторов. Здесь и далее будет использоваться следующий конструктор:

```
JTextField(int cols)
```

где *cols* обозначает ширину текстового поля, выраженную в столбцах. Однако длина вводимой строки не ограничивается шириной поля, отображаемого на экране. Поэтому, если в конструкторе присутствует параметр *cols*, он задает лишь упомянутый выше физический размер компонента.

Для завершения ввода текста в поле пользователь нажимает клавишу `<Enter>`, в результате чего формируется событие `ActionEvent`. В классе `JTextField` предоставляются методы `addActionListener()` и `removeActionListener()`. Для обработки событий действия необходимо реализовать метод `actionPerformed()`, объявленный в интерфейсе `ActionListener`. Обработка событий от поля ввода текста осуществляется таким же образом, как и обработка событий от кнопки, о которых шла речь ранее.

Как и с компонентом `JButton`, с компонентом `JTextField` связывается конкретная команда действия в виде символьной строки. По умолчанию она соответствует текущему содержимому поля ввода текста, хотя в таком значении используется редко. Чаще всего устанавливается фиксированное значение команды действия с помощью метода `setActionCommand()`, который объявляется следующим образом:

```
void setActionCommand(String cmd)
```

Символьная строка, передаваемая в качестве параметра *cmd*, становится новой командой действия, а текст в поле ввода текста не меняется. Установленная символьная строка команды действия остается постоянной, независимо от того, какой именно текст вводится в поле ввода текста. Как правило, к явной установке команды действия прибегают для того, чтобы обеспечить распознавание поля ввода текста как источника, сформировавшего событие действия. Поступать подобным образом приходится в том случае, если в рамке окна находится несколько элементов управления, для которых определен

общий обработчик событий. Установив команду действия, вы получаете в свое распоряжение удобное средство для различения компонентов. Если же вы не установите явно команду действия для поля ввода текста, то можете испытать затруднения при распознавании источника события, так как пользователь может ввести в поле произвольный текст, совпадающий с командой действия другого компонента.

Для того чтобы получить символьную строку, отображаемую в поле ввода текста, следует обратиться к экземпляру класса `JTextField` и вызвать метод `getText()`. Объявление этого метода приведено ниже.

```
String getText()
```

Задать текст для компонента `JTextField` можно с помощью метода `setText()`, объявляемого следующим образом:

```
void setText(String текст)
```

где *текст* — это символьная строка, размещаемая в поле ввода текста.

ОБРАЩЕНИЕ К ЗНАТОКУ

ВОПРОС. Как пояснялось выше, команду действия для поля ввода текста можно явно установить с помощью метода `setActionCommand()`. Можно ли воспользоваться этим методом, чтобы установить команду действия для кнопки?

ОТВЕТ. Да, можно. По умолчанию команда действия для кнопки совпадает с текстом надписи на кнопке. Но можно установить и другое ее значение, воспользовавшись методом `setActionCommand()` из класса `JButton`. Он выполняет те же действия, что и одноименный метод из класса `JTextField`.

Ниже приведен пример программы, демонстрирующий применение компонента `JTextField`. В окне этой программы содержатся поле ввода текста, кнопка и две метки. Одна из меток подсказывает пользователю ввести текст в поле. Когда пользователь нажмет клавишу `<Enter>` (при условии, что фокус ввода находится в поле ввода текста), введенные данные будут извлечены и отображены на второй метке. На кнопке отображается надпись `Reverse` (Обратить). При нажатии этой кнопки содержимое поля ввода текста преобразуется и заменяется на обратное. Окно, отображаемое на экране при выполнении данной программы, приведено на рис. 15.3.

```
// Применение поля ввода текста.
```

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
class TFDemo implements ActionListener {
```

```
    JTextField jtf;
    JButton jbtnRev;
    JLabel jlabPrompt, jlabContents;
```

```
    TFDemo() {
```

```
        // создать новый контейнер JFrame
```

```
        JFrame jfrm = new JFrame("Use a Text Field");
```

```

// установить диспетчер компоновки FlowLayout
jfrm.setLayout(new FlowLayout());

// задать исходные размеры рамки окна
jfrm.setSize(240, 120);

// завершить программу после закрытия окна
jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

// Создание поля ввода текста шириной 10 символов.
jtf = new JTextField(10);

// Установка команды действия для поля ввода текста.
jtf.setActionCommand("myTF");

// создать кнопку Reverse
JButton jbtnRev = new JButton("Reverse");

// Добавление приемников событий от поля ввода и кнопки.
jtf.addActionListener(this);
jbtnRev.addActionListener(this);

// создать метки
jlabPrompt = new JLabel("Enter text: ");
jlabContents = new JLabel("");

// добавить компоненты на панели содержимого
jfrm.add(jlabPrompt);
jfrm.add(jtf);
jfrm.add(jbtnRev);
jfrm.add(jlabContents);

// отобразить рамку окна
jfrm.setVisible(true);
}

// Обработка событий от кнопки и поля ввода текста.
public void actionPerformed(ActionEvent ae) {

    // Для определения компонента, сформировавшего событие,
    // используется команда действия.
    if(ae.getActionCommand().equals("Reverse")) {
        // Нажатие кнопки Reverse.
        String orgStr = jtf.getText();
        String resStr = "";

        // обратить символьную строку в поле ввода текста
        for(int i=orgStr.length()-1; i >=0; i--)
            resStr += orgStr.charAt(i);

        // сохранить обращенную строку в поле ввода текста
        jtf.setText(resStr);
    } else

```

```

// Нажатие клавиши <Enter> в тот момент, когда фокус
// ввода находится в поле ввода текста.
jlabContents.setText("You pressed ENTER. Text is: " +
                    jtf.getText());
}

public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new TFDemo();
        }
    });
}
}

```

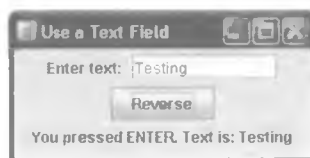


Рис. 15.3. Окно, отображаемое при выполнении программы TFDemo

Большая часть исходного кода приведенной выше программы вам уже знакома, но некоторые его фрагменты необходимо рассмотреть отдельно. Прежде всего обратите внимание на то, что с полем ввода текста связана команда действия "myTF". Такое связывание осуществляется в следующей строке кода:

```
jtf.setActionCommand("myTF");
```

После выполнения этой строки кода символьная строка команды действия всегда принимает значение "myTF" независимо от того, какой именно текст введен в поле. Благодаря этому исключается ситуация, при которой команда действия, связанная с полем ввода текста, будет вступать в конфликт с командой действия, связанной с кнопкой **Reverse**. В методе `actionPerformed()` установленная команда действия используется для распознавания того компонента, который стал источником события. Если символьная строка команды действия принимает значение "Reverse", это может означать только одно: событие наступило в результате щелчка на кнопке **Reverse**. Иначе следует сделать вывод, что событие наступило в результате нажатия пользователем клавиши <Enter> в тот момент, когда фокус ввода находился в поле ввода текста.

И наконец, обратите внимание на следующую строку кода в теле метода `actionPerformed()`:

```

jlabContents.setText("You pressed ENTER. Text is: " +
                    jtf.getText());

```

Как пояснялось выше, при нажатии клавиши <Enter> в тот момент, когда фокус ввода находился в поле ввода текста, формируется событие `ActionEvent`, которое пересылается всем зарегистрированным приемникам событий действия с помощью метода `actionPerformed()`. В программе TFDemo этот метод лишь вызывает метод `getText()`, извлекая текст, содержащийся в компоненте `jtf` (поле ввода текста). После этого текст отображается с помощью метки, на которую ссылается переменная `jlabContents`.

Создание флажков с помощью компонента JCheckBox

Если обычные кнопки используются чаще других элементов пользовательского интерфейса, то на втором месте по частоте употребления, безусловно, стоят флажки. В Swing эти элементы пользовательского интерфейса реализуются с помощью компонента типа JCheckBox. Класс JCheckBox является производным от классов AbstractButton и JToggleButton. Следовательно, флажок — это особая разновидность кнопки.

В классе JCheckBox определен ряд конструкторов. Один из них имеет следующий вид:
`JCheckBox(String str)`

Он создает флажок с пояснительной надписью в виде символьной строки, передаваемой в качестве параметра *str*.

При установке или сбросе флажка формируется событие от элемента, представленное классом ItemEvent. Для обработки событий от элементов используются классы, реализующие интерфейс ItemListener. В этом интерфейсе объявлен лишь один метод, `itemStateChanged()`, объявляемый следующим образом:

```
void itemStateChanged(ItemEvent ie)
```

Здесь событие от элемента передается в качестве параметра *ie*.

Для того чтобы получить ссылку на элемент, состояние которого изменилось, следует вызвать метод `getItem()` для объекта ItemEvent. Ниже приведена общая форма объявления этого метода.

```
Object getItem()
```

Возвращаемая этим методом ссылка должна быть приведена к типу оперируемого компонента, а в данном случае — к классу JCheckBox.

Текст, связанный с флажком, можно получить, вызвав метод `getText()`, а задать текст пояснительной надписи, вызвав метод `setText()`. Эти методы действуют таким же образом, как и одноименные методы из рассмотренного ранее класса JButton.

Самый простой способ определить состояние флажка — вызвать метод `isSelected()`, который объявляется следующим образом:

```
boolean isSelected()
```

Этот метод возвращает логическое значение `true`, если флажок установлен, иначе — логическое значение `false`.

Ниже приведен пример программы, демонстрирующий манипулирование флажками. В ней создаются три флажка: Alpha, Beta и Gamma. Всякий раз, когда состояние флажка изменяется, в окне программы появляются сведения о произведенном действии, а также перечисляются те флажки, которые установлены в данный момент. Окно, отображаемое на экране при выполнении данной программы, приведено на рис. 15.4.

```
// Демонстрация флажков.
```

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
class CBDemo implements ItemListener {
```

```

JLabel jlabSelected;
JLabel jlabChanged;
JCheckBox jcbAlpha;
JCheckBox jcbBeta;
JCheckBox jcbGamma;

CBDemo() {
    // создать новый контейнер JFrame
    JFrame jfrm = new JFrame("Demonstrate Check Boxes");

    // установить диспетчер компоновки FlowLayout
    jfrm.setLayout(new FlowLayout());

    // задать исходные размеры рамки окна
    jfrm.setSize(280, 120);

    // завершить программу после закрытия окна
    jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // создать пустые метки
    jlabSelected = new JLabel("");
    jlabChanged = new JLabel("");

    // Создание флажков.
    jcbAlpha = new JCheckBox("Alpha");
    jcbBeta = new JCheckBox("Beta");
    jcbGamma = new JCheckBox("Gamma");

    // События, формируемые компонентами JCheckBox, обрабатываются
    // одним методом itemStateChanged(), реализованным в классе CBDemo.
    jcbAlpha.addItemListener(this);
    jcbBeta.addItemListener(this);
    jcbGamma.addItemListener(this);

    // добавить флажки и метки на панели содержимого
    jfrm.add(jcbAlpha);
    jfrm.add(jcbBeta);
    jfrm.add(jcbGamma);
    jfrm.add(jlabChanged);
    jfrm.add(jlabSelected);

    // отобразить рамку окна
    jfrm.setVisible(true);
}

// Обработчик событий от элементов (в данном случае – флажков).
public void itemStateChanged(ItemEvent ie) {
    String str = "";

    // Получение ссылки на компонент флажка, сформировавший событие.
    JCheckBox cb = (JCheckBox) ie.getItem();

    // сообщить об изменении состояния флажка

```

```

if(cb.isSelected()) // Определение состояния флажка.
    jlabChanged.setText(cb.getText() + " was just selected.");
else
    jlabChanged.setText(cb.getText() + " was just cleared.");

// сообщить о всех установленных флажках
if(jcbAlpha.isSelected()) {
    str += "Alpha ";
}
if(jcbBeta.isSelected()) {
    str += "Beta ";
}
if(jcbGamma.isSelected()) {
    str += "Gamma";
}

jlabSelected.setText("Selected check boxes: " + str);
}

public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new CBDemo();
        }
    });
}
}

```

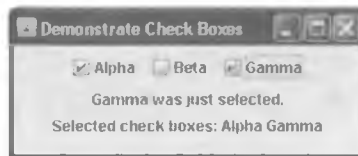


Рис. 15.4. Окно, отображаемое при выполнении программы CBDemo

Наибольший интерес в рассматриваемом здесь примере представляет метод `itemStateChanged()`, предназначенный для обработки событий от элементов (в данном случае — флажков). Он выполняет две функции: во-первых, сообщает, установлен или сброшен флажок; а во-вторых, отображает перечень установленных флажков. В начале этого метода определяется ссылка на компонент, сформировавший событие `ItemEvent`. Это происходит в следующей строке кода:

```
JCheckBox cb = (JCheckBox) ie.getItem();
```

Приведение к типу `JCheckBox` необходимо потому, что метод `getItem()` возвращает ссылку на объект типа `Object`. Далее метод `itemStateChanged()` обращается к методу `isSelected()` по ссылке `cb`, чтобы определить текущее состояние флажка. Если метод `isSelected()` возвращает логическое значение `true`, значит, флажок установлен, а логическое значение `false` соответствует сброшенному состоянию флажка. Затем с помощью метки `jlabChanged` отображаются сведения о выполненном действии.

И наконец, метод `itemStateChanged()` проверяет состояние каждого флажка и формирует символьную строку с именами установленных флажков. Эта символьная строка отображается в окне программы с помощью метки `jlabSelected`.

Работа с компонентом JList

Последним в этой главе будет рассмотрен компонент JList. Он является основным классом Swing для работы со списками и позволяет выбирать один или несколько элементов из списка. И хотя элементы списка являются символьными строками, это не мешает создать список, включающий практически любые объекты, которые могут быть отображены на экране. Компонент JList настолько широко применяется в реальных программах на Java, что его трудно было бы не заметить в них прежде.

Раньше элементы списка были представлены в компоненте JList в виде ссылок на объекты типа Object. А после выпуска версии JDK 7 компонент JList стал обобщенным и теперь объявляется следующим образом:

```
class JList<E>
```

где E обозначает тип элементов списка. Таким образом, в компоненте JList теперь обеспечивается типовая безопасность.

На заметку

Далее рассматривается обобщенный вариант компонента JList и демонстрируется пример его применения. Поэтому если вы пользуетесь компилятором более ранней версии, чем JDK 7, вам придется выбрать предыдущий, необобщенный вариант компонента JList.

В классе компонента JList предоставляется ряд конструкторов. Один из них имеет следующий вид:

```
JList(E[] элементы)
```

Этот конструктор создает компонент JList со списком элементов, хранящихся в массиве, на который указывает параметр *элементы*.

Компонент JList и сам позволяет решить немало задач построения списков, но чаще всего он помещается в контейнер JScrollPane, автоматически обеспечивающий прокрутку своего содержимого. Ниже приведен конструктор этого контейнера.

```
JScrollPane(Component компонент)
```

Здесь *компонент* обозначает конкретный компонент, передаваемый конструктору в качестве параметра для прокрутки (в данном случае это компонент JList). Если поместить компонент JList в контейнер JScrollPane, то тем самым будет обеспечена автоматическая прокрутка длинных списков. Благодаря этому упрощается построение графического пользовательского интерфейса вообще и изменение числа элементов списка в частности, не затрагивая при этом размеры самого компонента JList.

Когда пользователь делает или изменяет выбор элемента в списке, компонент JList формирует событие ListSelectionEvent. Это же событие формируется при отмене выбора. Для его обработки используется объект приемника событий из класса, реализующего интерфейс ListSelectionListener. Этот интерфейс относится к пакету javax.swing.event. В этом интерфейсе объявлен только один метод, valueChanged(), объявляемый следующим образом:

```
void valueChanged(ListSelectionEvent le)
```

где *le* обозначает ссылку на объект, сформировавший событие. И хотя в самом классе ListSelectionEvent определен ряд методов, для выяснения того, что же произошло со списком, обычно приходится опрашивать сам объект типа JList. Класс ListSelectionEvent также относится к пакету javax.swing.event.

По умолчанию компонент `JList` дает пользователю возможность выбрать несколько элементов из списка. Изменить такое поведение можно, вызвав метод `setSelectionMode()`, определенный в классе `JList`. Этот метод объявляется так:

```
void setSelectionMode(int режим)
```

где параметр *режим* задает порядок выбора элементов из списка. Значение этого параметра должно совпадать с одной из приведенных ниже констант, определенных в интерфейсе `ListSelectionModel`, входящем в пакет `javax.swing`.

```
SINGLE_SELECTION  
SINGLE_INTERVAL_SELECTION  
MULTIPLE_INTERVAL_SELECTION
```

По умолчанию устанавливается режим `MULTIPLE_INTERVAL_SELECTION`, и пользователь может выбирать ряд элементов из списка через несколько промежутков. В режиме `SINGLE_INTERVAL_SELECTION` можно выбирать ряд элементов из списка только через один промежуток. А в режиме `SINGLE_SELECTION` каждый раз можно выбрать только один элемент из списка. Очевидно, что единственный элемент может быть выбран из списка и в двух других режимах, где допускается также выбирать одновременно целый ряд элементов.

Индекс первого элемента, выбранного из списка, а в режиме `SINGLE_SELECTION` — это индекс единственного выбранного элемента, можно получить, вызвав метод `getSelectedIndex()`. Ниже показано, каким образом он объявляется.

```
int getSelectedIndex()
```

Индексирование начинается с нуля. Так, если выбран первый элемент в списке, этот метод возвращает значение 0. Если же ни один из элементов не выбран, возвращается значение `-1`.

Получить массив, содержащий все выбранные из списка элементы, можно, вызвав метод `getSelectedIndices()`:

```
int[] getSelectedIndices()
```

В возвращаемом массиве индексы расположены по возрастающей. Если же получен массив нулевой длины, это означает, что ни один из элементов не выбран из списка.

Ниже приведен пример программы, демонстрирующий применение простого компонента `JList`, содержащего список имен. Всякий раз, когда пользователь выбирает имя из списка, формируется событие `ListSelectionEvent`, которое обрабатывается методом `valueChanged()`, объявленным в интерфейсе `ListSelectionListener`. Этот метод определяет индекс выбранного элемента и отображает соответствующее имя. Окно, отображаемое на экране при выполнении данной программы, приведено на рис. 15.5.

```
// Демонстрация простого компонента JList.
```

```
// Для компиляции этой программы требуется JDK 7  
// или более поздняя версия данного комплекта.
```

```
import javax.swing.*;  
import javax.swing.event.*;  
import java.awt.*;  
import java.awt.event.*;
```

```
class ListDemo implements ListSelectionListener {
```

```
    JList<String> jlst;
```

```

JLabel jlab;
JScrollPane jscrlp;

// создать массив имен
String names[] = { "Sherry", "Jon", "Rachel", // Этот массив имен
                  "Sasha", "Josselyn", "Randy", // будет отображаться
                  "Tom", "Mary", "Ken", // списком в компоненте JList.
                  "Andrew", "Matt", "Todd" };

ListDemo() {
    // создать новый контейнер JFrame
    JFrame jfrm = new JFrame("JList Demo");

    // установить диспетчер компоновки FlowLayout
    jfrm.setLayout(new FlowLayout());

    // задать исходные размеры рамки окна
    jfrm.setSize(200, 160);

    // завершить программу после закрытия окна
    jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // создать компонент JList
    jlst = new JList<String>(names); // Создание списка имен.

    // задать режим выбора элементов из списка
    // Переход в режим выбора элементов из списка по одному.
    jlst.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

    // добавить список на панели прокрутки.
    // Компонент списка помещается в контейнер панели прокрутки.
    jscrlp = new JScrollPane(jlst);

    // задать предпочтительные размеры панели прокрутки
    jscrlp.setPreferredSize(new Dimension(120, 90));

    // создать метку для отображения результатов выбора из списка.
    jlab = new JLabel("Please choose a name");

    // добавить обработчик событий, связанных с выбором из списка
    // Прием событий, наступающих при выборе элементов из списка.
    jlst.addListSelectionListener(this);

    // добавить список и метку на панели содержимого
    jfrm.add(jscrlp);
    jfrm.add(jlab);

    // отобразить рамку окна
    jfrm.setVisible(true);
}

// обработать события, связанные с выбором элементов из списка
public void valueChanged(ListSelectionEvent le) {
    // получить индекс того элемента, выбор которого был сделан
    // или отменен в списке

```

```

int idx = jlst.getSelectedIndex();

// отобразить результат выбора, если элемент был выбран
if(idx != -1)
    jlab.setText("Current selection: " + names[idx]);
else // иначе еще раз предложить сделать выбор
    jlab.setText("Please choose a name");
}

public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new ListDemo();
        }
    });
}
}

```



Рис. 15.5. Окно, отображаемое при выполнении программы ListDemo

Рассмотрим исходный код данной программы более подробно. Обратите внимание на то, что в начале программы объявляется массив `names`. Он инициализируется символьными строками, содержащими разные имена. В конструкторе `ListDemo()` массив `names` используется для создания компонента `JList`. Конструктор, которому в качестве параметра передается массив, как это имеет место в данном случае, автоматически создает экземпляр класса `JList`, содержащий элементы массива. Следовательно, формируемый список будет состоять из имен, хранящихся в массиве `names`.

Далее устанавливается режим, допускающий выбор только одного элемента из списка. Затем компонент `jlst` помещается в контейнер `JScrollPane`, а для панели прокрутки задаются предпочтительные размеры `120 × 90`. Это делается ради компактности и удобства использования данного компонента. Для задания предпочтительных размеров компонента служит метод `setPreferredSize()`. Как правило, предпочтительные размеры определяют фактические размеры компонента, но не следует забывать, что некоторые диспетчеры компоновки могут игнорировать подобные запросы на установку размеров компонентов.

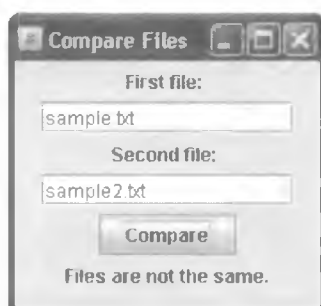
Когда пользователь выбирает элемент из списка или изменяет свой выбор, формируется связанное с этим событие. Для получения индекса выбранного элемента в обработчике подобных событий, а в данном случае в его роли выступает метод `valueChanged()`, вызывается метод `getSelectedIndex()`. И поскольку для списка был задан режим, ограничивающий выбор только одним элементом, то индекс однозначно определяет этот элемент. Затем индекс используется для обращения к массиву `names` и получения имени выбранного элемента. Обратите внимание на то, что в данной программе проверяется, равен ли индекс значению `-1`. Как упоминалось выше, это значение возвращается при условии, что ни один из элементов не выбран из списка. Нечто подобное может произойти в том случае, если событие было сформировано в результате отмены пользовате-

лем своего выбора. Напомним, что событие, связанное с выбором из списка, формируется, когда пользователь выбирает элемент списка или же отменяет свой выбор.

Пример для опробования 15.1. Утилита сравнения файлов, создаваемая на основе Swing

Исходный файл: SwingFC.java

Несмотря на то что вы ознакомились лишь с небольшой частью компонентов Swing, это не мешает вам применить свои знания на практике и создать реальное приложение средствами этой библиотеки. В примере для опробования 10.1 была создана консольная утилита сравнения файлов. А в этом проекте предстоит снабдить ее пользовательским интерфейсом, построенным из компонентов Swing. Это позволит значительно улучшить внешний вид данной утилиты и сделать ее более удобной в употреблении. Ниже показано, как выглядит рабочее окно утилиты сравнения файлов, создаваемой на основе Swing.



В процессе работы над данным проектом вы сможете сами убедиться, насколько библиотека Swing упрощает создание приложений с графическим пользовательским интерфейсом.

Последовательность действий

1. Создайте файл SwingFC.java и введите приведенные ниже комментарии и операторы import.

```
/*
```

```
    Пример для опробования 15-1.
```

```
    Утилита сравнения файлов, создаваемая на основе Swing.
```

```
    Для компиляции этой утилиты требуется JDK 7  
    или более поздняя версия данного комплекта.
```

```
*/
```

```
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
import java.io.*;
```

2. Создайте класс SwingFC, начав с приведенного ниже исходного кода.

```
class SwingFC implements ActionListener {  
  
    JTextField jtfFirst; // Переменная для хранения имени первого файла  
    JTextField jtfSecond; // Переменная для хранения имени второго файла  
  
    JButton jbtnComp; // Кнопка для сравнения файлов
```

```
JLabel jlabFirst, jlabSecond; // Подсказки для пользователя
JLabel jlabResult; // Сведения о результатах и сообщения об ошибках
```

Имена сравниваемых файлов указываются в полях ввода текста `jtfFirst` и `jtfSecond`. Для того чтобы начать сравнение файлов, указанных в этих полях, пользователь должен щелкнуть на кнопке `jbtnComp`. По ходу сравнения с помощью меток `jlabFirst` и `jlabSecond` должны отображаться наводящие сообщения. А результаты сравнения или сообщения об ошибках должны отображаться с помощью метки `jlabResult`.

3. Создайте конструктор класса `SwingFC`, как показано ниже.

```
SwingFC() {

    // создать новый контейнер JFrame
    JFrame jfrm = new JFrame("Compare Files");

    // установить диспетчер компоновки FlowLayout
    jfrm.setLayout(new FlowLayout());

    // задать исходные размеры рамки окна
    jfrm.setSize(200, 190);

    // завершить программу после закрытия окна
    jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // создать поля для ввода имен файлов
    jtfFirst = new JTextField(14);
    jtfSecond = new JTextField(14);

    // установить команды действия для полей ввода текста
    jtfFirst.setActionCommand("fileA");
    jtfSecond.setActionCommand("fileB");

    // создать кнопку Compare
    JButton jbtnComp = new JButton("Compare");

    // добавить приемник событий действия от кнопки Compare
    jbtnComp.addActionListener(this);

    // создать метки
    jlabFirst = new JLabel("First file: ");
    jlabSecond = new JLabel("Second file: ");
    jlabResult = new JLabel("");

    // добавить компоненты на панели содержимого
    jfrm.add(jlabFirst);
    jfrm.add(jtfFirst);
    jfrm.add(jlabSecond);
    jfrm.add(jtfSecond);
    jfrm.add(jbtnComp);
    jfrm.add(jlabResult);

    // отобразить рамку окна
    jfrm.setVisible(true);
}
```

Большая часть исходного кода этого конструктора должна быть вам уже знакома. Обратите внимание лишь на следующую особенность: приемник событий действия вводится только для нажимаемой кнопки `jbtnCompare`, а приемники событий действия для полей ввода текста не добавляются. Дело в том, что содержимое полей ввода текста требуется только в тот момент, когда нажимается кнопка `Compare` (Сравнить), а в остальное время в их содержимом нет особой нужды. Поэтому и нет никакого смысла предусматривать реагирование утилиты на любые события от полей ввода текста. Когда вы напишете хотя бы несколько реальных программ с использованием библиотеки `Swing`, вы обнаружите, что потребность в обработке событий от полей ввода текста возникает очень редко.

4. Начните создание обработчика событий `actionPerformed()` так, как показано ниже. Этот метод вызывается при нажатии кнопки `Compare`.

```
// сравнить файлы после нажатия кнопки Compare
public void actionPerformed(ActionEvent ae) {
    int i=0, j=0;

    // сначала убедиться, что введены имена обоих файлов
    if(jtfFirst.getText().equals("")) {
        jlabResult.setText("First file name missing.");
        return;
    }
    if(jtfSecond.getText().equals("")) {
        jlabResult.setText("Second file name missing.");
        return;
    }
}
```

В начале этого метода проверяется, ввел ли пользователь имена файлов в каждом из полей ввода текста. Если какое-то из этих полей осталось пустым, выводится соответствующее сообщение и обработка события завершается.

5. Завершите создание обработчика событий, введя приведенный ниже исходный код, в котором файлы сначала открываются, а затем сравниваются.

```
// сравнить файлы, используя оператор try с ресурсами
try (FileInputStream f1 = new FileInputStream(jtfFirst.getText());
     FileInputStream f2 = new FileInputStream(jtfSecond.getText()))

    // проверить содержимое каждого файла
    do {
        i = f1.read();
        j = f2.read();
        if(i != j) break;
    } while(i != -1 && j != -1);

    if(i != j)
        jlabResult.setText("Files are not the same.");
    else
        jlabResult.setText("Files compare equal.");
} catch(IOException exc) {
    jlabResult.setText("File Error");
}
}
```

6. И наконец, введите в класс SwingFC метод main(), как показано ниже.

```
public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new SwingFC();
        }
    });
}
```

7. Ниже приведен весь исходный код утилиты сравнения файлов.

```
/*
    Пример для опробования 15-1.

    Утилита сравнения файлов, создаваемая на основе Swing.

    Для компиляции этой утилиты требуется JDK 7
    или более поздняя версия данного комплекта.
*/

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.io.*;

class SwingFC implements ActionListener {

    JTextField jtfFirst; // Переменная для хранения имени первого файла
    JTextField jtfSecond; // Переменная для хранения имени второго файла

    JButton jbtnComp; // Кнопка для сравнения файлов

    JLabel jlabFirst, jlabSecond; // Подсказки для пользователя
    JLabel jlabResult; // Сведения о результатах и сообщения об ошибках

    SwingFC() {

        // создать новый контейнер JFrame
        JFrame jfrm = new JFrame("Compare Files");

        // установить диспетчер компоновки FlowLayout
        jfrm.setLayout(new FlowLayout());

        // задать исходные размеры рамки окна
        jfrm.setSize(200, 190);

        // завершить программу после закрытия окна
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // создать поля для ввода имен файлов
        jtfFirst = new JTextField(14);
        jtfSecond = new JTextField(14);
```



```

// установить команды действия для полей ввода текста
jtfFirst.setActionCommand("fileA");
jtfSecond.setActionCommand("fileB");

// создать кнопку Compare
JButton jbtnComp = new JButton("Compare");

// добавить приемник событий действия от кнопки Compare
jbtnComp.addActionListener(this);

// создать метки
jlabFirst = new JLabel("First file: ");
jlabSecond = new JLabel("Second file: ");
jlabResult = new JLabel("");

// добавить компоненты на панели содержимого
jfrm.add(jlabFirst);
jfrm.add(jtfFirst);
jfrm.add(jlabSecond);
jfrm.add(jtfSecond);
jfrm.add(jbtnComp);
jfrm.add(jlabResult);

// отобразить рамку окна
jfrm.setVisible(true);
}

// сравнить файлы после нажатия кнопки Compare
public void actionPerformed(ActionEvent ae) {
    int i=0, j=0;

    // сначала убедиться, что введены имена обоих файлов
    if(jtfFirst.getText().equals("")) {
        jlabResult.setText("First file name missing.");
        return;
    }
    if(jtfSecond.getText().equals("")) {
        jlabResult.setText("Second file name missing.");
        return;
    }

    // сравнить файлы, используя оператор try с ресурсами
    try (FileInputStream f1 = new FileInputStream(jtfFirst.getText());
        FileInputStream f2 = new FileInputStream(jtfSecond.getText()))

    // проверить содержимое каждого файла
    do {
        i = f1.read();
        j = f2.read();
        if(i != j) break;
    } while(i != -1 && j != -1);

    if(i != j)

```

```

        jlabResult.setText("Files are not the same.");
    else
        jlabResult.setText("Files compare equal.");
} catch(IOException exc) {
    jlabResult.setText("File Error");
}
}

public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new SwingFC();
        }
    });
}
}

```

Применение анонимных внутренних классов для обработки событий

Программы из примеров, рассмотренных до сих пор в этой главе, не отличались особой сложностью. Такими же несложными были и используемые в них обработчики событий, в роли которых выступал основной класс приложения, реализующий интерфейс соответствующего приемника событий, а все события передавались для обработки экземпляру этого класса. И хотя такой подход вполне пригоден для написания прикладных программ с пользовательским интерфейсом, его нельзя рассматривать как единственно возможный. В подобных программах могут применяться и другие способы обработки событий, происходящих в пользовательском интерфейсе. Во-первых, для каждого события можно реализовать приемник в отдельном классе. Благодаря этому разнородные события будут обрабатываться в разных классах. И во-вторых, приемники событий можно реализовать с помощью *анонимных внутренних классов*.

У анонимного внутреннего класса нет имени, а экземпляр такого класса получается динамически по мере необходимости. Анонимные внутренние классы позволяют значительно упростить создание обработчиков для некоторых видов событий. Допустим, имеется компонент `jbtn` типа `JButton`. Приемник событий действия от кнопки можно реализовать следующим образом:

```

jbtn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent ae) {
        // обработать событие здесь
    }
});

```

В данном примере используется анонимный внутренний класс, реализующий интерфейс `ActionListener`. Обратите особое внимание на синтаксис, используемый при создании этого класса. Тело внутреннего класса начинается после символа `{`, следующего за выражением `new ActionListener()`. Обратите также внимание на то, что вызов метода `addActionListener()` завершается закрывающей скобкой и точкой с запятой, т.е. как обычно. Такой синтаксис используется при создании анонимных внутренних

классов, предназначенных для обработки любых событий. Очевидно, что для разнородных событий задаются разные приемники и реализуются разные методы.

Преимущество анонимного внутреннего класса заключается, в частности, в том, что компонент, вызывающий методы этого класса, заранее известен. Так, в предыдущем примере не было никакой необходимости вызывать метод `getActionCommand()`, чтобы выяснить, какой именно компонент сформировал событие, поскольку метод `actionPerformed()` может быть вызван в подобной реализации только при наступлении событий, сформированных компонентом `jbtn`. С реальным примером применения анонимных внутренних классов вы ознакомитесь в следующем разделе при создании Swing-апплета.

Создание апплета средствами Swing

Ранее в этой главе рассматривались примеры Swing-программ. Но компоненты Swing нередко применяются и для создания апплетов. Swing-апплеты похожи на апплеты, создаваемые на основе библиотеки AWT (см. главу 14), но у них имеется существенное отличие: Swing-апплет расширяет класс `JApplet`, производный от класса `Applet`, а не сам этот класс. Таким образом, подкласс `JApplet` наследует все функциональные возможности своего суперкласса `Applet`, а кроме того, в него добавлены средства поддержки библиотеки Swing. Класс `JApplet` служит в качестве контейнера верхнего уровня, а следовательно, он содержит различные панели, описанные в начале этой главы. Поэтому все компоненты Swing-апплета добавляются на панели содержимого контейнера `JApplet` таким же образом, как это делалось ранее на панели содержимого контейнера `JFrame`.

Срок действия Swing-апплета определяется теми же четырьмя методами, что и срок действия AWT-апплета: `init()`, `start()`, `stop()` и `destroy()` (см. главу 14). Очевидно, что переопределять необходимо только те методы, в которых нужно реализовать функциональные возможности, требующиеся для создаваемого апплета. Следует также иметь в виду, что рисование в окне выполняется в Swing- и AWT-апплетах по-разному. Именно поэтому в Swing-апплетах метод `paint()` обычно не переопределяется.

Не следует также забывать, что все действия над компонентами в Swing-апплете должны выполняться в потоке диспетчеризации событий, как пояснялось ранее в этой главе. А это означает, что организовать многопоточную обработку необходимо в Swing-программах любого типа.

Ниже приведен пример Swing-апплета. Он выполняет те же действия, что и Swing-программа из примера, демонстрирующего нажатие кнопок ранее в этой главе, но в данном случае программа реализована в виде апплета. Для обработки событий в этом апплете используются анонимные внутренние классы. Результат выполнения этого Swing-апплета в средстве просмотра апплетов `appletviewer` показан на рис. 15.6.

```
// Простой Swing-апплет.
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

```
/*
```

Этот код HTML может быть использован для загрузки апплета:

```
<object code="MySwingApplet" width=200 height=80>
</object>
```

```

*/

// Swing-апплет должен расширять класс JApplet.
public class MySwingApplet extends JApplet {
    JButton jbtnUp;
    JButton jbtnDown;

    JLabel jlab;

    // инициализировать апплет
    public void init() {
        try {
            // Для создания графического пользовательского интерфейса
            // апплета используется метод invokeAndWait().
            SwingUtilities.invokeAndWait(new Runnable () {
                public void run() {
                    makeGUI(); // инициализировать графический интерфейс
                }
            });
        } catch (Exception exc) {
            System.out.println("Can't create because of " + exc);
        }
    }

    // В этом апплете нет нужды переопределять
    // методы start(), stop() и destroy().

    // установить и инициализировать графический интерфейс
    private void makeGUI() {
        // установить диспетчер компоновки FlowLayout для апплета
        setLayout(new FlowLayout());

        // создать две кнопки
        jbtnUp = new JButton("Up");
        jbtnDown = new JButton("Down");

        // добавить приемник событий от кнопки Up
        // Для обработки событий от кнопки Up
        // используется анонимный внутренний класс.
        jbtnUp.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                jlab.setText("You pressed Up.");
            }
        });

        // добавить приемник событий от кнопки Down
        // Для обработки событий от кнопки Down
        // используется анонимный внутренний класс.
        jbtnDown.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                jlab.setText("You pressed down.");
            }
        });
    }
}

```

```

// добавить кнопки на панели содержимого
add(jbtnUp);
add(jbtnDown);

// создать текстовую метку
jlab = new JLabel("Press a button.");

// добавить метку на панели содержимого
add(jlab);
}
}

```



Рис. 15.6. Результат выполнения Swing-апплета

У исходного кода этого апплета имеются некоторые особенности, заслуживающие отдельного рассмотрения. Во-первых, класс `MySwingApplet` является производным от класса `JApplet`. Как пояснялось ранее, все Swing-апплеты расширяют класс `JApplet` вместо класса `Applet`. Во-вторых, в методе `init()` компоненты Swing инициализируются в отдельном потоке диспетчеризации событий, где вызывается метод `makeGUI()`. Для организации такого потока служит метод `invokeAndWait()`. В апплетах рекомендуется пользоваться методом `init()`, а не методом `invokeLater()`, поскольку он не должен возвращать управление до завершения инициализирующего процесса. По существу, сам метод `start()` не должен вызываться до конца инициализации, т.е. до тех пор, пока графический пользовательский интерфейс не будет полностью построен.

В методе `makeGUI()` создаются две кнопки и метка, а кроме того, с кнопками связываются приемники событий действия. Обратите внимание на то, что обработчики событий реализуются в виде анонимных внутренних классов. Этот пример реализации обработчиков событий можно использовать в качестве образца при разработке обработчиков других событий. Одно из главных преимуществ такого подхода заключается в том, что объект, вызывающий событие, заранее известен, поскольку именно в нем создается экземпляр анонимного внутреннего класса. Поэтому нет никакой необходимости получать и анализировать команду действия, чтобы выяснить, какая именно кнопка породила событие. И наконец, компоненты, предназначенные для отображения в окне апплета, добавляются на панели содержимого. Несмотря на всю простоту данного примера, он демонстрирует общий подход к созданию любого Swing-апплета с графическим пользовательским интерфейсом.

Что делать дальше

Примите поздравления! Если вы прочитали и проработали материал всех 15 глав этой книги, то можете смело считать себя программирующим на Java. Конечно, вам предстоит узнать еще немало о самом языке Java, его библиотеках и подсистемах, но вы уже владеете базовыми знаниями, чтобы на их прочном основании приобретать новые знания и опыт программирования на Java.

В дальнейшем вам, вероятнее всего, придется самостоятельно изучить следующие темы.

- **Swing.** Полученных в этой книге знаний о библиотеке Swing явно недостаточно, чтобы воспользоваться в полной мере этим важным элементом Java.
- **AWT.** На этой библиотеке построена библиотека Swing.
- **Обработка событий.** Эта тема рассмотрена в данной книге лишь в самых общих чертах, поэтому многие вопросы обработки событий вам еще предстоит изучить самостоятельно.
- **Сетевые классы Java.** Служат для организации взаимодействия в сети.
- **Служебные классы Java,** в особенности из библиотеки коллекций *Collections Framework*. Упрощают решение очень многих задач программирования.
- **Прикладной интерфейс Concurrent API.** Обеспечивает doskonaльный контроль над высокопроизводительными многопоточными приложениями.
- **JavaBeans.** Инструментальное средство, предназначенное для создания программных компонентов на Java.
- **Создание собственных методов.**
- **Сервлеты.** Если вам придется участвовать в создании сложных веб-приложений, то вы вряд ли сможете обойтись без знаний и навыков разработки сервлетов. Сервлеты выполняют те же функции на стороне сервера, что и апплеты на стороне клиента, в роли которого чаще всего выступает браузер.

Для дальнейшего изучения Java рекомендуется книга *Полный справочник по Java*, 8-е издание, ИД “Вильямс”, 2012 г.

В ней вы найдете подробные сведения о языке программирования Java и его основных библиотеках, а также сотни примеров программ, демонстрирующих языковые средства Java.

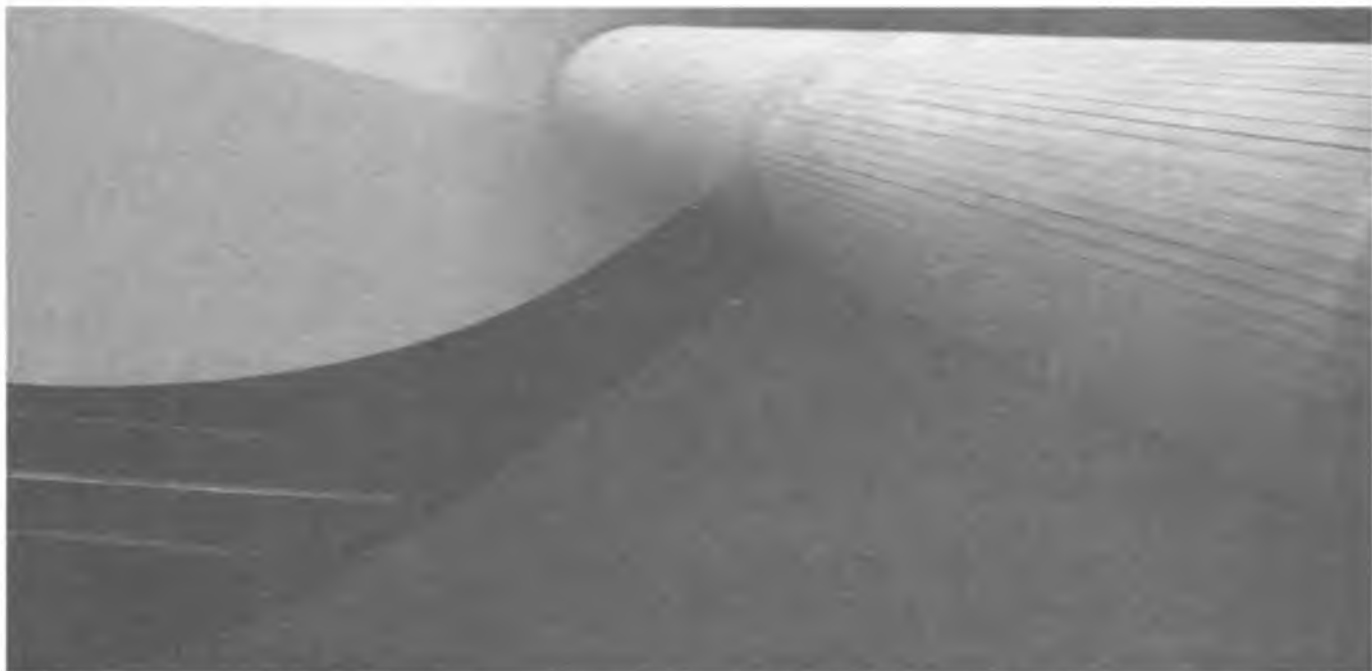
Упражнения для самопроверки по материалу главы 15

1. Компоненты AWT являются тяжеловесными, а компоненты Swing — _____.
2. Может ли изменяться стиль оформления компонента Swing? Если да, то какое средство позволяет это сделать?
3. Какой контейнер верхнего уровня чаще всего используется в приложениях?
4. Контейнер верхнего уровня содержит несколько панелей. На какой панели размещаются компоненты?
5. Как создать ссылку, отображающую сообщение "Select an entry from the list" (Выберите элемент из списка)?
6. В каком потоке должно происходить все взаимодействие с компонентами графического пользовательского интерфейса?
7. Какая команда действия связывается по умолчанию с компонентом `JButton`? Как изменить команду действия?
8. Какое событие формируется при нажатии кнопки?

9. Как создать поле ввода текста шириной до 32 символов?
10. Можно ли установить команду действия для компонента `JTextField`? Если можно, то как это сделать?
11. С помощью какого компонента `Swing` можно создать флажок? Какое событие формируется при установке или сбросе флажка?
12. Компонент `JList` отображает список элементов, которые может выбирать пользователь. Верно или неверно?
13. Какое событие формируется при выборе пользователем элемента из списка типа `JList` или отмене выбора?
14. В каком методе задается режим выбора элементов списка типа `JList`? С помощью какого метода можно получить индекс первого выбранного элемента?
15. Подкласс какого класса нужно создать при разработке `Swing`-апплета?
16. Обычно при построении исходного пользовательского интерфейса в `Swing`-апплетах используется метод `invokeAndWait()`. Верно или неверно?
17. Добавьте в утилиту сравнения файлов, созданную в примере для опробования 15.1, флажок со следующей пояснительной надписью: **Show position of mismatch** (Показывать место несовпадения). Если этот флажок установлен, программа должна отображать место, в котором обнаружено первое расхождение в содержимом сравниваемых файлов.
18. Измените программу `ListDemo` таким образом, чтобы она допускала выбор нескольких элементов из списка.
19. Дополнительное задание. Преобразуйте класс `Help`, созданный в примере для опробования 4.1, в `Swing`-программу с графическим пользовательским интерфейсом. Сведения о ключевых словах (`for`, `while`, `switch` и т.д.) должны отображаться с помощью компонента `JList`. При выборе пользователем элемента из списка должно выводиться описание синтаксиса выбранного ключевого слова. Для отображения многострочного текста на месте метки можно воспользоваться средствами `HTML`. В этом случае текст должен начинаться с дескриптора `<html>` и завершаться дескриптором `</html>`. В итоге текст будет автоматически размечен в виде `HTML`-документа. Помимо прочих преимуществ, такая разметка текста позволяет создавать многострочные метки. В качестве примера ниже приведена строка кода, в которой создается метка, отображающая две текстовые строки: первой выводится строка "Top" (Верх), а под ней — вторая строка "Bottom" (Низ).


```
JLabel jlabhtml = new JLabel("<html>Top<br>Bottom</html>");
```

На этот вопрос ответа не дается. Ведь вы достигли такого уровня, который позволяет самостоятельно разрабатывать программы на Java!
20. Продолжайте изучать Java. Желательно ознакомиться с возможностями стандартных пакетов, например `java.lang`, `java.util` и `java.net`. Пишите небольшие программы, демонстрирующие применение различных классов и интерфейсов. Наилучший способ достичь мастерства в программировании на Java — написать как можно больше разных программ.



Приложение А

**Ответы на вопросы
для самопроверки**

Глава 1. Основы Java

1. Что такое байт-код и почему он так важен для интернет-программирования на языке Java?

Байт-код — это высокооптимизированный набор инструкций, выполняемых под управлением интерпретатора Java. Байт-код позволяет добиться переносимости и безопасности программ, написанных на этом языке программирования.

2. Каковы три основных принципа объектно-ориентированного программирования? Инкапсуляция, полиморфизм и наследование.

3. С чего начинается выполнение программы на Java?

Выполнение программы на Java начинается с метода `main()`.

4. Что такое переменная?

Переменная — это именованная область памяти. Содержимое переменной может изменяться в процессе выполнения программы.

5. Какое из перечисленных ниже имен переменных недопустимо?

- A. `count`
- B. `$count`
- C. `count27`
- D. `67count`

Недопустимо имя переменной `67count` (пункт D). Имя переменной не может начинаться с цифры.

6. Как создать однострочный комментарий? И как создать многострочный комментарий?

Однострочные комментарии должны начинаться с символов `//`. В этом случае комментариями считаются эти и все последующие символы до конца строки. А многострочные комментарии должны начинаться символами `/*` и заканчиваться символами `*/`.

7. Как выглядит общая форма условного оператора `if`? И как выглядит общая форма цикла `for`?

Общая форма оператора `if` выглядит следующим образом:

```
if (условие) оператор;
```

А общая форма цикла `for` имеет следующий вид:

```
for(инициализация; условие; итерация) оператор;
```

8. Как создать кодовый блок?

Кодовый блок должен начинаться с символа `{` и завершаться символом `}`.

9. Сила тяжести на Луне составляет около 17% земной. Напишите программу, которая вычисляла бы ваш вес на Луне.

```
/*
```

```
    Вычисление веса на Луне.
```

```
    Присвоить ее исходному файлу имя Moon.java.
```

```
*/
```

```

class Moon {
    public static void main(String args[]) {
        double earthweight; // Вес на Земле
        double moonweight; // Вес на Луне

        earthweight = 165;

        moonweight = earthweight * 0.17;

        System.out.println(earthweight +
                            " earth-pounds is equivalent to " +
                            moonweight + " moon-pounds.");
    }
}

```

- 10.** Видоизмените программу, созданную в примере для опробования 1.2, таким образом, чтобы она выводила таблицу преобразования дюймов в метры. Выводите значения длины до 12 футов через каждый дюйм. После каждых 12 дюймов выводите пустую строку. (Один метр приблизительно равен 39,37 дюйма.)

```

/*
    Эта программа отображает таблицу преобразования дюймов в метры.

    Присвоить ее исходному файлу имя InchToMeterTable.java.
*/

```

```

class InchToMeterTable {
    public static void main(String args[]) {
        double inches, meters;
        int counter;

        counter = 0;
        for(inches = 1; inches <= 144; inches++) {
            meters = inches / 39.37; // преобразовать в метры
            System.out.println(inches + " inches is " +
                               meters + " meters.");

            counter++;
            // Каждая 12-я выводимая строка должна быть пустой
            if(counter == 12) {
                System.out.println();
                counter = 0; // сбросить счетчик строк
            }
        }
    }
}

```

- 11.** Если при вводе кода программы вы допустите опечатку, то какого рода сообщение об ошибке вы получите?

Сообщение о синтаксической ошибке.

- 12.** Имеет ли значение, в каком именно месте строки находится оператор? Не имеет. В Java допускается произвольное форматирование исходного кода.

Глава 2. Введение в типы данных и операторы

1. Почему в Java строго определены диапазоны допустимых значений и области действия простых типов?

Диапазоны допустимых значений и области действия простых типов строго определены в Java для того, чтобы обеспечить переносимость программ с одной платформы на другую.

2. Что собой представляет символьный тип в Java и чем он отличается от символьного типа в ряде других языков программирования?

Символьный тип задается ключевым словом `char`. В Java для представления символов применяется уникод (Unicode), в отличие от многих других языков программирования, в которых для этой цели применяется код ASCII.

3. Переменная типа `boolean` может иметь любое значение, поскольку любое ненулевое значение интерпретируется как истинное. Верно или неверно?

Неверно. Переменная типа `boolean` может принимать лишь одно из двух логических значений: `true` или `false`.

4. Допустим, результат выполнения программы выглядит следующим образом:

```
One
Two
Three
```

Напишите строку кода с вызовом метода `println()`, где этот результат выводится в одной символьной строке.

```
System.out.println("One\nTwo\nThree");
```

5. Какая ошибка допущена в следующем фрагменте кода:

```
for(i = 0; i < 10; i++) {
    int sum;
    sum = sum + i;
}
System.out.println("Sum is: " + sum);
```

В этом фрагменте кода имеются две существенные ошибки. Во-первых, переменная `sum` создается на каждом шаге цикла `for`, а следовательно, в промежутке между последовательными итерациями предыдущее значение подсчитываемой суммы не будет сохраняться в этой переменной. И во-вторых, переменная `sum` недоступна за пределами кодового блока, в котором она объявлена. Поэтому ссылка на нее при вызове метода `println()` недопустима.

6. Поясните отличие между префиксной и постфиксной формами оператора инкремента.

Если оператор инкремента предшествует операнду, исполняющая система Java выполнит операцию до извлечения значения операнда и использования его в остальной части выражения. Если же оператор инкремента следует за операндом, исполняющая система сначала извлечет значение операнда и лишь затем инкрементирует сам операнд.

7. Покажите, каким образом укороченный логический оператор `И` может предотвратить деление на нуль.

```
if((b != 0) && (val / b)) ...
```

8. К какому типу продвигаются типы byte и short при вычислении выражения?

В выражении типы byte и short продвигаются к типу int.

9. Когда возникает потребность в явном приведении типов?

Явное приведение типов требуется при обращении с несовместимыми типами или в том случае, если преобразование типов приводит к сужению диапазона допустимых значений.

10. Напишите программу, которая находила бы простые числа в пределах от 2 до 100.

// Нахождение простых чисел в пределах от 2 до 100.

```
class Prime {
    public static void main(String args[]) {
        int i, j;
        boolean isprime;

        for(i=2; i < 100; i++) {
            isprime = true;

            // проверить, делится ли число без остатка
            for(j=2; j < i/j; j++)
                // если число делится без остатка, значит, оно простое
                if((i%j) == 0) isprime = false;

            if(isprime)
                System.out.println(i + " is prime.");
        }
    }
}
```

11. Влияют ли лишние скобки на эффективность выполнения программ?

Нет, не влияют.

12. Определяет ли кодовый блок область действия переменных?

Да, определяет.

Глава 3. Управляющие операторы

1. Напишите программу, которая вводила бы символы с клавиатуры до тех пор, пока не встретится точка. Предусмотрите в программе счетчик числа пробелов. Сведения о количестве пробелов должны выводиться в конце программы.

// Подсчет пробелов.

```
class Spaces {
    public static void main(String args[])
        throws java.io.IOException {

        char ch;
        int spaces = 0;

        System.out.println("Enter a period to stop.");

        do {
            ch = (char) System.in.read();
```

```

        if(ch == ' ') spaces++;
    } while(ch != '.');

    System.out.println("Spaces: " + spaces);
}
}

```

2. Какова общая форма многоступенчатой конструкции if-else-if?

```

if(условие)
    оператор;
else if(условие)
    оператор;
else if(условие)
    оператор;
.
.
.
else
    оператор;

```

3. Допустим, имеется следующий фрагмент кода:

```

if(x < 10)
    if(y > 100) {
        if(!done) x = z;
        else y = z;
    }
else System.out.println("error"); // что если?

```

С каким из операторов if связан последний оператор else?

Последний оператор else соответствует ближайшему оператору if того же уровня, т.е. if(y > 100).

4. Напишите цикл for, в котором перебирались бы значения от 1000 до 0 с шагом -2.

```
for(int i = 1000; i >= 0; i -= 2) // ...
```

5. Корректен ли следующий фрагмент кода?

```

for(int i = 0; i < num; i++)
    sum += i;

count = i;

```

Нет, не корректен. Переменная i недоступна за пределами цикла for, в котором она объявлена.

6. Какие действия выполняет оператор break? Опишите оба варианта этого оператора.

Оператор break без метки вызывает немедленное завершение текущего цикла или выражения switch. А оператор break с меткой передает управление в конец помеченного блока.

7. Какое сообщение будет выведено после выполнения оператора break в приведенном ниже фрагменте кода?

```

for(i = 0; i < 10; i++) {
    while(running) {
        if(x<y) break;
        // ...
    }
}

```

```

    }
    System.out.println("after while");
}
System.out.println("After for");

```

После выполнения оператора `break` будет выведено сообщение "after while" (после цикла `while`).

8. Что будет выведено на экран в результате выполнения следующего фрагмента кода?

```

for(int i = 0; i<10; i++) {
    System.out.print(i + " ");
    if((i%2) == 0) continue;
    System.out.println();
}

```

На экране появится следующий результат:

```

0 1
2 3
4 5
6 7
8 9

```

9. Итерационное выражение для цикла `for` не обязательно должно изменять переменную цикла на фиксированную величину. Эта переменная может принимать произвольные значения. Напишите программу, использующую цикл `for` для вывода чисел в геометрической прогрессии 1, 2, 4, 8, 16, 32 и т.д.

```

/* Применение цикла for для формирования
   геометрической прогрессии 1, 2, 4, 8, 16, 32 и т.д.
*/
class Progress {
    public static void main(String args[]) {

        for(int i = 1; i < 100; i += i)
            System.out.print(i + " ");
    }
}

```

10. Код ASCII-символов нижнего регистра отличается от кода соответствующих символов верхнего регистра на величину 32. Следовательно, для преобразования строчной буквы в прописную нужно уменьшить ее код на 32. Используйте это обстоятельство для написания программы, осуществляющей ввод символов с клавиатуры. При выводе результатов данная программа должна преобразовывать строчные буквы в прописные, а прописные — в строчные. Остальные символы не должны изменяться. Работа программы должна завершаться после того, как пользователь введет с клавиатуры точку. И наконец, сделайте так, чтобы программа отображала число символов, для которых был изменен регистр.

```

// Смена регистра символов.
class CaseChg {
    public static void main(String args[])
        throws java.io.IOException {
        char ch;
        int changes = 0;

```

```

        System.out.println("Enter period to stop.");

        do {
            ch = (char) System.in.read();
            if(ch >= 'a' & ch <= 'z') {
                ch -= 32;
                changes++;
                System.out.println(ch);
            }
            else if(ch >= 'A' & ch <= 'Z') {
                ch += 32;
                changes++;
                System.out.println(ch);
            }
        } while(ch != '.');
        System.out.println("Case changes: " + changes);
    }
}

```

11. Что такое бесконечный цикл?

Бесконечным называется такой цикл, выполнение которого не оканчивается.

12. Должна ли метка, используемая вместе с оператором `break`, быть определена в кодовом блоке, содержащем этот оператор?

Да, должна.

Глава 4. Введение в классы, объекты и методы

1. В чем отличие класса от объекта?

Класс — это абстрактное логическое описание формы и поведения объекта. А объект — это вещественный экземпляр класса.

2. Как определяется класс?

Класс определяется с помощью ключевого слова `class`. В операторе `class` указываются код и данные, составляющие класс.

3. Чью собственную копию содержит каждый объект?

Каждый объект класса содержит собственную копию переменных экземпляра этого класса.

4. Покажите, как объявить объект `counter` класса `MyCounter`, используя два отдельных оператора.

```

MyCounter counter;
counter = new MyCounter();

```

5. Как должен быть объявлен метод `myMeth`, принимающий два параметра `a` и `b` типа `int` и возвращающий значение типа `double`?

```

double myMeth(int a, int b) { // ...

```

6. Как должно завершаться выполнение метода, возвращающего некоторое значение?

Для завершения метода служит оператор `return`. Он же передает возвращаемое значение вызывающей части программы.

7. Каким должно быть имя конструктора?

Имя конструктора должно совпадать с именем класса.

8. Какие действия выполняет оператор `new`?

Оператор `new` выделяет память для объекта и выполняет инициализацию объекта, используя конструктор соответствующего класса.

9. Что такое “сборка мусора” и какие действия она выполняет? Зачем нужен метод `finalize()`?

“Сборка мусора” — это механизм удаления неиспользуемых объектов для повторного использования памяти, освобождаемой от объектов. Метод `finalize()` вызывается для объекта непосредственно перед его удалением из памяти.

10. Что означает ключевое слово `this`?

Ключевое слово `this` означает ссылку на объект, для которого вызывается метод. Она автоматически передается методу.

11. Может ли конструктор иметь один или несколько параметров?

Да, может.

12. Если метод не возвращает значения, то как следует объявить тип этого метода?

Как `void`.

Глава 5. Дополнительные сведения о типах данных и операторах

1. Покажите два способа объявления одномерного массива, состоящего из 12 элементов типа `double`.

```
double x[] = new double[12];
double[] x = new double[12];
```

2. Покажите, как инициализировать одномерный массив целочисленными значениями от 1 до 5.

```
int x[] = { 1, 2, 3, 4, 5 };
```

3. Напишите программу, в которой массив используется для нахождения среднего арифметического десяти значений типа `double`. Используйте любые десять чисел.

```
// Среднее арифметическое 10 значений типа double.
class Avg {
    public static void main(String args[]) {
        double nums[] = { 1.1, 2.2, 3.3, 4.4, 5.5,
                          6.6, 7.7, 8.8, 9.9, 10.1 };
        double sum = 0;

        for(int i=0; i < nums.length; i++)
            sum += nums[i];

        System.out.println("Average: " + sum / nums.length);
    }
}
```

4. Измените программу, созданную в примере для опробования 5.1, таким образом, чтобы она сортировала массив символьных строк. Продемонстрируйте ее работоспособность.


```
// Демонстрация пузырьковой сортировки строк.
class StrBubble {
    public static void main(String args[]) {
        String strs[] = {
            "this", "is", "a", "test",
            "of", "a", "string", "sort"
        };

        int a, b;
        String t;
        int size;

        size = strs.length; // Количество сортируемых элементов

        // отобразить исходный массив
        System.out.print("Original array is:");
        for(int i=0; i < size; i++)
            System.out.print(" " + strs[i]);
        System.out.println();

        // Пузырьковая сортировка строк.
        for(a=1; a < size; a++)
            for(b=size-1; b >= a; b--) {
                // поменять элементы местами при нарушении порядка их следования
                if(strs[b-1].compareTo(strs[b]) > 0) {
                    t = strs[b-1];
                    strs[b-1] = strs[b];
                    strs[b] = t;
                }
            }

        // отобразить отсортированный массив
        System.out.print("Sorted array is:");
        for(int i=0; i < size; i++)
            System.out.print(" " + strs[i]);
        System.out.println();
    }
}
```

5. В чем отличие методов `indexOf()` и `lastIndexOf()` из класса `String`?

Метод `indexOf()` находит первое вхождение указанной подстроки, а метод `lastIndexOf()` — ее последнее вхождение в текущей символьной строке.

6. Все символьные строки являются объектами типа `String`. Покажите, как вызываются методы `length()` и `charAt()` для строкового литерала "I like Java" (Мне нравится Java).

Как ни странно, приведенный ниже вызов метода `length()` вполне допустим.

```
System.out.println("I like Java".length());
```

В результате этого вызова на экран выводится значение **11**. Аналогичным образом вызывается и метод `charAt()`.

7. Расширьте класс `Encode` таким образом, чтобы в качестве ключа шифрования использовалась строка из восьми символов.

```
// Видоизмененный способ шифрования и дешифрования сообщений
// с помощью операции поразрядного исключающего ИЛИ.
class Encode {
    public static void main(String args[]) {
        String msg = "This is a test";
        String encmsg = "";
        String decmsg = "";
        String key = "abcdefghi";
        int j;

        System.out.print("Original message: ");
        System.out.println(msg);

        // зашифровать сообщение
        j = 0;
        for(int i=0; i < msg.length(); i++) {
            encmsg = encmsg + (char) (msg.charAt(i) ^ key.charAt(j));
            j++;
            if(j==8) j = 0;
        }

        System.out.print("Encoded message: ");
        System.out.println(encmsg);

        // дешифровать сообщение
        j = 0;
        for(int i=0; i < msg.length(); i++) {
            decmsg = decmsg + (char) (encmsg.charAt(i) ^ key.charAt(j));
            j++;
            if(j==8) j = 0;
        }

        System.out.print("Decoded message: ");
        System.out.println(decmsg);
    }
}
```

- 8. Можно ли применять поразрядные операторы к значениям типа double?**

Нет, нельзя.

- 9. Перепишите приведенную ниже последовательность операторов, воспользовавшись оператором ?.**

```
if(x < 0) y = 10;
else y = 20;
```

Ответ:

```
y = x < 0 ? 10 : 20;
```

- 10. В приведенном ниже фрагменте кода содержится знак &. Какой оператор он обозначает: поразрядный или логический? Обоснуйте свой ответ.**

```
boolean a, b;
// ...
if(a & b) ...
```

Это логический оператор, поскольку оба его операнда относятся к типу `boolean`.

11. Является ли ошибкой превышение верхней границы массива?

Да.

Является ли ошибкой использование отрицательных значений для доступа к элементам массива?

Да. Значения индексов массива начинаются с нуля.

12. Как обозначается оператор сдвига вправо без знака?

`>>>`

13. Перепишите рассмотренный ранее в этой главе класс `MinMax` таким образом, чтобы в нем использовалась разновидность `for-each` цикла `for`.

// Нахождение минимального и максимального значений в массиве.

```
class MinMax {
    public static void main(String args[]) {
        int nums[] = new int[10];
        int min, max;

        nums[0] = 99;
        nums[1] = -10;
        nums[2] = 100123;
        nums[3] = 18;
        nums[4] = -978;
        nums[5] = 5623;
        nums[6] = 463;
        nums[7] = -9;
        nums[8] = 287;
        nums[9] = 49;

        min = max = nums[0];
        for(int v : nums) {
            if(v < min) min = v;
            if(v > max) max = v;
        }
        System.out.println("min and max: " + min + " " + max);
    }
}
```

14. В примере для опробования 5.1 была реализована пузырьковая сортировка. Можно ли в программе из этого примера заменить обычный цикл `for` его разновидностью `for-each`? Если нельзя, то почему?

Циклы `for`, выполняющие сортировку в классе `Bubble`, нельзя преобразовать в вариант `for-each`. Что касается внешнего цикла, то текущее значение его переменной используется во внутреннем цикле. А что касается внутреннего цикла, то для перестановки следующих не по порядку элементов требуются операции присваивания значений элементам массива, чего нельзя добиться в варианте `for-each`.

15. Можно ли управлять оператором `switch` с помощью объектов типа `String`?

Можно, начиная с версии `JDK 7`.

Глава 6. Дополнительные сведения о методах и классах

1. Допустим, имеется следующий фрагмент кода:

```
class X {
    private int count;
```

Является ли допустимым приведенный ниже фрагмент кода?

```
class Y {
    public static void main(String args[]) {
        X ob = new X();

        ob.count = 10;.
```

Нет. Закрытый (private) член недоступен за пределами своего класса.

2. Модификатор доступа должен _____ объявлению члена класса.
предшествовать

3. Помимо очереди, в программах часто используется структура данных, которая называется стеком. Обращение к стеку осуществляется по принципу “первым пришел — последним обслужен”. Стек можно сравнить со стопкой тарелок, стоящих на столе. Последней берется тарелка, поставленная на стол первой. Создайте класс Stack, реализующий стек для хранения символов. Используйте методы push() и pop() для манипулирования содержимым стека. Пользователь класса Stack должен иметь возможность задавать размер стека при его создании. Все члены класса Stack, кроме методов push() и pop(), должны быть объявлены как private. (Подсказка: в качестве заготовки можете воспользоваться классом Queue, изменив в нем лишь способ доступа к данным.)

// Класс, реализующий стек для хранения символов.

```
class Stack {
    private char stck[]; // Массив для хранения элементов стека
    private int tos;     // Вершина стека.
```

```
    // построить пустой стек заданного размера
    Stack(int size) {
        stck = new char[size]; // выделить память для стека
        tos = 0;
    }
```

```
    // построить один стек из другого стека
    Stack(Stack ob) {
        tos = ob.tos;
        stck = new char[ob.stck.length];
```

```
        // скопировать элементы
        for(int i=0; i < tos; i++)
            stck[i] = ob.stck[i];
    }
```

```
    // построить стек с исходными значениями
    Stack(char a[]) {
```

```

    stck = new char[a.length];

    for(int i = 0; i < a.length; i++) {
        push(a[i]);
    }
}

// поместить символы в стек
void push(char ch) {
    if(tos==stck.length) {
        System.out.println(" -- Stack is full.");
        return;
    }

    stck[tos] = ch;
    tos++;
}

// извлечь символы из стека
char pop() {
    if(tos==0) {
        System.out.println(" -- Stack is empty.");
        return (char) 0;
    }

    tos--;
    return stck[tos];
}

// продемонстрировать применение класса Stack
class SDemo {
    public static void main(String args[]) {
        // создать пустой стек на 10 элементов
        Stack stk1 = new Stack(10);

        char name[] = {'T', 'o', 'm'};

        // построить стек из массива
        Stack stk2 = new Stack(name);

        char ch;
        int i;

        // поместить символы в стек stk1
        for(i=0; i < 10; i++)
            stk1.push((char) ('A' + i));

        // построить один стек из другого стека
        Stack stk3 = new Stack(stk1);

        // отобразить стеки
        System.out.print("Contents of stk1: ");
    }
}

```

```

for(i=0; i < 10; i++) {
    ch = stk1.pop();
    System.out.print(ch);
}

System.out.println("\n");

System.out.print("Contents of stk2: ");
for(i=0; i < 3; i++) {
    ch = stk2.pop();
    System.out.print(ch);
}

System.out.println("\n");
System.out.print("Contents of stk3: ");
for(i=0; i < 10; i++) {
    ch = stk3.pop();
    System.out.print(ch);
}
}
}

```

Ниже приведен результат выполнения данной программы.

```

Contents of stk1: JINGFEDCBA
Contents of stk2: moT
Contents of stk3: JINGFEDCBA

```

4. Допустим, имеется следующий класс:

```

class Test {
    int a;
    Test(int i) { a = i; }
}

```

Напишите метод `swap()`, реализующий обмен содержимым между двумя объектами типа `Test`, на которые ссылаются две переменные данного типа.

```

void swap(Test ob1, Test ob2) {
    int t;

    t = ob1.a;
    ob1.a = ob2.a;
    ob2.a = t;
}

```

5. Правильно ли написан следующий фрагмент кода?

```

class X {
    int meth(int a, int b) { ... }
    String meth(int a, int b) { ... }
}

```

Нет, неправильно. Перегружаемые методы могут возвращать значения разного типа, но это не играет никакой роли для определения подобных методов во время их перегрузки. Перегружаемые методы должны иметь в своих списках *разнотипные* параметры.

6. Напишите рекурсивный метод, отображающий строку задом наперед.

```
// Отображение строки задом наперед с помощью рекурсии.
class Backwards {
    String str;

    Backwards(String s) {
        str = s;
    }

    void backward(int idx) {
        if(idx != str.length()-1) backward(idx+1);

        System.out.print(str.charAt(idx));
    }
}

class BWDemo {
    public static void main(String args[]) {
        Backwards s = new Backwards("This is a test");

        s.backward(0);
    }
}
```

7. Допустим, что все объекты класса должны совместно пользоваться одной и той же переменной. Как объявить такую переменную?
Переменная, предназначенная для совместного использования, должна быть объявлена как `static`.
8. Для чего может понадобиться статический блок?
Статический блок служит для выполнения любых инициализирующих действий в классе до создания конкретных объектов.
9. Что такое внутренний класс?
Внутренний класс — это нестатический вложенный класс.
10. Допустим, требуется член класса, к которому могут обращаться только другие члены этого же класса. Какой модификатор доступа следует использовать в его объявлении?
Модификатор доступа `private`.
11. Имя метода и список его параметров вместе составляют _____ метода.
сигнатуру
12. Если методу передается значение типа `int`, то в этом случае используется передача параметра по _____.
значению
13. Создайте метод `sum()` с аргументами переменной длины для суммирования передаваемых ему значений типа `int`. Метод должен возвращать результат суммирования. Протестируйте его в действии.
Существует много вариантов решения данной задачи. Ниже представлен один из них.

```

class SumIt {
    int sum(int ... n) {
        int result = 0;
        for(int i = 0; i < n.length; i++)
            result += n[i];

        return result;
    }
}

class SumDemo {
    public static void main(String args[]) {

        SumIt siObj = new SumIt();

        int total = siObj.sum(1, 2, 3);
        System.out.println("Sum is " + total);

        total = siObj.sum(1, 2, 3, 4, 5);
        System.out.println("Sum is " + total);
    }
}

```

14. Можно ли перегружать метод с аргументами переменной длины?

Да, можно.

15. Приведите пример неоднозначного вызова перегружаемого метода с переменным числом аргументов.

Ниже приведен один из вариантов перегружаемого метода с переменным числом аргументов, при вызове которого проявляется неоднозначность.

```

double myMeth(double ... v ) { // ...
double myMeth(double d, double ... v) { // ...

```

Если попытаться вызвать метод `myMeth()` с одним аргументом следующим образом:
`myMeth(1.1);`

то компилятор не сможет определить, какой именно метод вызывается.

Глава 7. Наследование

1. Имеет ли суперкласс доступ к членам подкласса? Имеет ли подкласс доступ к членам суперкласса?

Нет, не имеет. Суперклассу ничего не известно о существовании подклассов. Подклассы действительно могут обращаться ко всем членам суперкласса, кроме тех, которые объявлены как закрытые (`private`).

2. Создайте подкласс `Circle`, производный от класса `TwoDShape`. В нем должен быть определен метод `area()`, вычисляющий площадь круга, а также конструктор с ключевым словом `super` для инициализации членов, унаследованных от класса `TwoDShape`.

```

// Подкласс, производный от класса TwoDShape для формы круга.
class Circle extends TwoDShape {
    // Конструктор по умолчанию

```



```

Circle() {
    super();
}

// Конструктор класса Circle
Circle(double x) {
    super(x, "circle"); // вызвать конструктор суперкласса
}

// создать новый объект из имеющегося объекта
Circle(Circle ob) {
    super(ob); // передать объект конструктору класса TwoDShape
}

double area() {
    return (getWidth() / 2) * (getWidth() / 2) * 3.1416;
}
}

```

3. Как предотвратить обращение к членам суперкласса из подкласса?

Для того чтобы предотвратить доступ к членам суперкласса из подкласса, эти члены следует объявить как закрытые (`private`).

4. Опишите назначение и два варианта использования ключевого слова `super`.

Ключевое слово `super` используется в двух случаях. Во-первых, с его помощью вызывается конструктор суперкласса. В этом случае общая форма вызова имеет следующий вид:

```
super(список_параметров);
```

И во-вторых, это ключевое слово обеспечивает доступ к членам суперкласса. Ниже приведена общая форма такого доступа.

```
super.член_класса
```

5. Допустим, имеется следующая иерархия классов:

```
class Alpha { ...
```

```
class Beta extends Alpha { ...
```

```
Class Gamma extends Beta { ...
```

В каком порядке вызываются конструкторы этих классов при создании объекта класса `Gamma`?

Конструкторы всегда вызываются в порядке наследования. Таким образом, при создании экземпляра класса `Gamma` будет сначала вызван конструктор `Alpha`, затем `Beta` и, наконец, `Gamma`.

6. Переменная ссылки на суперкласс может указывать на объект подкласса. Объясните, почему это важно и как это связано с переопределением методов?

Когда переопределяемый метод вызывается по ссылке на суперкласс, его вариант определяется по типу объекта, на который делается ссылка.

7. Что такое абстрактный класс?

Абстрактным называется такой класс, который содержит хотя бы один абстрактный метод.

8. Как предотвратить переопределение метода? И как предотвратить наследование класса?

Для того чтобы метод нельзя было переопределить, его нужно объявить как `final`. А для того чтобы предотвратить наследование от класса, его следует объявить как `final`.

9. Объясните, каким образом механизмы наследования, переопределения методов и абстрактные классы используются для поддержки полиморфизма.

Наследование, переопределение методов и абстрактные классы поддерживают полиморфизм и позволяют создать обобщенную структуру, реализуемую различными классами. Так, абстрактный класс определяет согласованный интерфейс, общий для всех реализующих его классов. Такой подход соответствует принципу “один интерфейс — множество методов”.

10. Какой класс является суперклассом всех остальных классов?

Класс `Object`.

11. Класс, который содержит хотя бы один абстрактный метод, должен быть объявлен абстрактным. Верно или не верно?

Верно.

12. Какое ключевое слово следует использовать для создания именованной константы?

Ключевое слово `final`.

Глава 8. Пакеты и интерфейсы

1. Используя код, созданный в примере для опробования 8.1, поместите в пакет `qpack` интерфейс `ICharQ` и все три реализующие его класса. Класс `IQDemo` должен остаться в пакете, используемом по умолчанию. Покажите, как импортировать и использовать классы из пакета `qpack`.

Для того чтобы включить интерфейс `ICharQ` и реализующие его классы в пакет `qpack`, следует поместить каждый из них в отдельный файл, объявить все классы, реализующие данный интерфейс, как `public`, а в начале каждого файла ввести следующий оператор:

```
package qpack;
```

После этого можно воспользоваться пакетом `qpack`, добавив в интерфейс `IQDemo` следующий оператор `import`:

```
import qpack.*;
```

2. Что такое пространство имен? Почему так важна возможность его разделения на отдельные области в Java?

Пространство имен — это область объявлений. Разделяя пространство имен на отдельные области, можно предотвратить конфликты имен.

3. Содержимое пакетов хранится в _____.

каталогах

4. В чем отличие доступа, определяемого ключевым словом `protected`, от доступа по умолчанию?

Член класса с доступом типа `protected` может быть использован в пределах текущего пакета, а также в подклассах данного класса, относящихся к любому пакету.

А член класса с доступом по умолчанию может быть использован только в пределах текущего пакета.

5. Допустим, классы, содержащиеся в одном пакете, требуется использовать в другом пакете. Какими двумя способами можно этого добиться?

Для того чтобы воспользоваться членом пакета, нужно указать его имя полностью или же импортировать этот член с помощью оператора `import`.

6. “Один интерфейс — множество методов” — главный принцип Java. Какое языковое средство лучше всего демонстрирует этот принцип?

Этот принцип объектно-ориентированного программирования лучше всего демонстрирует интерфейс.

7. Сколько классов могут реализовать один и тот же интерфейс? Сколько интерфейсов может реализовать класс?

Один интерфейс может быть реализован любым количеством классов. Класс может реализовать произвольное число интерфейсов.

8. Может ли один интерфейс наследовать другой интерфейс?

Да, может. Механизм наследования распространяется и на интерфейсы.

9. Создайте интерфейс для класса `Vehicle`, рассмотренного в главе 7, назвав его `IVehicle`.

```
interface IVehicle {

    // вернуть дальность действия транспортного средства
    int range();

    // рассчитать объем топлива, требуемого
    // для прохождения заданного пути
    double fuelneeded(int miles);

    // Методы доступа к переменным экземпляра.
    int getPassengers();
    void setPassengers(int p);
    int getFuelcap();
    void setFuelcap(int f);
    int getMpg();
    void setMpg(int m);
}
```

10. Переменные, объявленные в интерфейсе, неявно считаются как `static` и `final`. Какие преимущества это дает?

Переменные, объявленные в интерфейсе, могут служить в качестве именованных констант, общих для всех файлов программы. Доступ к ним обеспечивается путем импорта того интерфейса, в котором они объявлены.

11. Пакет, по существу, является контейнером для классов. Верно или не верно?

Верно.

12. Какой стандартный пакет импортируется по умолчанию в любую программу на Java?
Пакет `java.lang`.

Глава 9. Обработка исключений

1. Какой класс находится на вершине иерархии исключений?

На вершине иерархии исключений находится класс `Throwable`.

2. Объясните вкратце, как пользоваться ключевыми словами `try` и `catch`?

Ключевые слова `try` и `catch` используются совместно. Операторы программы для отслеживания исключений помещаются в блок `try`. А перехват и обработка исключений осуществляются в блоке `catch`.

3. Какая ошибка допущена в приведенном ниже фрагменте кода?

```
// ...
vals[18] = 10;
catch (ArrayIndexOutOfBoundsException exc) {
    // обработать ошибку
}
```

Блоку `catch` не предшествует блок `try`.

4. Что произойдет, если исключение не будет перехвачено?

Если исключение не будет перехвачено, произойдет аварийное завершение программы.

5. Какая ошибка допущена в приведенном ниже фрагменте кода?

```
class A extends Exception { ...

class B extends A { ...

// ...

try {
    // ...
}
catch (A exc) { ... }
catch (B exc) { ... }
```

В данном фрагменте кода оператор `catch` для суперкласса предшествует оператору `catch` для подкласса. А поскольку оператор `catch` для суперкласса может обработать также исключения, относящиеся к подклассу, то в программе окажется код, недоступный для выполнения.

6. Может ли внутренний блок `catch` повторно генерировать исключение, которое будет обработано во внешнем блоке `catch`?

Да, исключения могут генерироваться повторно.

7. Блок `finally` — последний фрагмент кода, выполняемый перед завершением программы. Верно или неверно? Обоснуйте свой ответ.

Неверно. Блок `finally` выполняется по завершении блока `try`.

8. Исключения какого типа необходимо явно объявлять с помощью оператора `throws`, включаемого в объявление метода?

С помощью оператора `throws` объявляются все исключения, кроме `RuntimeException` и `Error`.

9. Какая ошибка допущена в приведенном ниже фрагменте кода?

```
class MyClass { // ... }
// ...
throw new MyClass();
```

Класс `MyClass` не является производным от класса `Throwable`. С помощью оператора `throw` могут генерироваться лишь те исключения, которые являются подклассами, производными от класса `Throwable`.

10. Отвечая на вопрос 3 упражнения для самопроверки по материалу главы 6, вы создали класс `Stack`. Добавьте в него специальные исключения для реагирования на попытку поместить элемент в переполненный стек и извлечь элемент из пустого стека.

// Исключение, возникающее при переполнении стека.

```
class StackFullException extends Exception {
    int size;

    StackFullException(int s) { size = s; }

    public String toString() {
        return "\nStack is full. Maximum size is " +
            size;
    }
}
```

// Исключение, возникающее при обращении к пустому стеку.

```
class StackEmptyException extends Exception {

    public String toString() {
        return "\nStack is empty.";
    }
}
```

// Класс, реализующий стек для хранения символов.

```
class Stack {
    private char stck[]; // Массив для хранения элементов стека
    private int tos;     // Вершина стека

    // построить пустой стек заданного размера
    Stack(int size) {
        stck = new char[size]; // выделить память для стека
        tos = 0;
    }

    // построить один стек из другого стека
    Stack(Stack ob) {
        tos = ob.tos;
        stck = new char[ob.stck.length];
    }
}
```

```
// скопировать элементы
for(int i=0; i < tos; i++)
    stck[i] = ob.stck[i];
}

// построить стек с исходными значениями
Stack(char a[]) {
    stck = new char[a.length];

    for(int i = 0; i < a.length; i++) {
        try {
            push(a[i]);
        }
        catch(StackFullException exc) {
            System.out.println(exc);
        }
    }
}

// поместить символы в стек
void push(char ch) throws StackFullException {
    if(tos==stck.length)
        throw new StackFullException(stck.length);

    stck[tos] = ch;
    tos++;
}

// извлечь символы из стека
char pop() throws StackEmptyException {
    if(tos==0)
        throw new StackEmptyException();

    tos--;
    return stck[tos];
}
}
```

11. Какими тремя способами можно сгенерировать исключение?

Исключение может быть сгенерировано в результате ошибки в виртуальной машине Java, ошибки в программе или явным образом с помощью оператора `throw`.

12. Назовите два подкласса, производных непосредственно от класса `Throwable`.

Классы `Error` и `Exception`

13. Что такое многократный перехват?

Многократным называется такой перехват, который позволяет перехватывать два и больше исключений одним оператором `catch`.

14. Следует ли перехватывать в программе исключения типа `Error`?

Нет, не следует.

Глава 10. Ввод-вывод данных

1. Для чего в Java определены как байтовые, так и символьные потоки?

Байтовые потоки с самого начала были определены в Java. Они особенно удобны для ввода-вывода двоичных данных и поддерживают произвольный доступ к ним в файлах. А символьные потоки оптимизированы для представления в уникоде.

2. Как известно, ввод-вывод данных на консоль осуществляется в текстовом виде. Почему же в Java для этой цели используются байтовые потоки?

Стандартные потоки ввода-вывода `System.in`, `System.out` и `System.err` были определены в Java прежде символьных потоков.

3. Как открыть файл для чтения байтов?

Ниже приведен один из способов открытия файла для ввода данных типа `byte`.

```
FileInputStream fin = new FileInputStream("test");
```

4. Как открыть файл для чтения символов?

Ниже приведен один из способов открытия файла для ввода символов.

```
FileReader fr = new FileReader("test");
```

5. Как открыть файл для ввода-вывода с произвольным доступом?

Ниже приведен один из способов открытия файла для ввода-вывода с произвольным доступом.

```
randfile = new RandomAccessFile("test", "rw");
```

6. Как преобразовать числовую строку "123.23" в двоичный эквивалент?

Для того чтобы преобразовать числовую строку в двоичный эквивалент, следует воспользоваться одним из методов синтаксического анализа, определенных в классах оболочек типов, например `Integer` или `Double`.

7. Напишите программу, которая будет копировать текстовые файлы. Видоизмените ее таким образом, чтобы все пробелы заменялись дефисами. Используйте при написании программы классы, представляющие байтовые потоки, а также традиционный способ закрытия файла явным вызовом метода `close()`.

```
/* Копирование текстового файла с заменой пробелов дефисами.
```

В этой версии программы используются байтовые потоки.

Для того чтобы воспользоваться этой программой, укажите в командной строке имена исходного и целевого файлов. Например:

```
java Hyphen source target
*/

import java.io.*;

class Hyphen {
    public static void main(String args[])
    {
        int i;
        FileInputStream fin = null;
        FileOutputStream fout = null;
```

```
// проверить сначала, указаны ли оба файла
if(args.length !=2 ) {
    System.out.println("Usage: Hyphen From To");
    return;
}

// скопировать файл и заменить в нем пробелы дефисами
try {
    fin = new FileInputStream(args[0]);
    fout = new FileOutputStream(args[1]);

    do {
        i = fin.read();
        // преобразовать пробел в дефис
        if((char)i == ' ') i = '-';

        if(i != -1) fout.write(i);
    } while(i != -1);
} catch(IOException exc) {
    System.out.println("I/O Error: " + exc);
} finally {
    try {
        if(fin != null) fin.close();
    } catch(IOException exc) {
        System.out.println("Error closing input file.");
    }

    try {
        if(fout != null) fout.close();
    } catch(IOException exc) {
        System.out.println("Error closing output file.");
    }
}
}
```

8. Перепишите программу, созданную в ответ на предыдущий вопрос, таким образом, чтобы в ней использовались классы, представляющие символьные потоки. На этот раз воспользуйтесь оператором `try` с ресурсами для автоматического закрытия файла.

/* Копирование текстового файла с заменой пробелов дефисами.

В этой версии программы используются символьные потоки.

Для того чтобы воспользоваться этой программой, укажите в командной строке имена исходного и целевого файлов. Например:

java Hyphen2 source target

Для компиляции этого кода требуется JDK 7 или более поздняя версия данного комплекта.

*/


```
import java.io.*;

class Hyphen2 {
    public static void main(String args[])
        throws IOException
    {
        int i;

        // проверить сначала, указаны ли оба файла
        if(args.length !=2 ) {
            System.out.println("Usage: CopyFile From To");
            return;
        }

        // скопировать файл и заменить в нем пробелы дефисами,
        // используя оператор try с ресурсами
        try (FileReader fin = new FileReader(args[0]);
            FileWriter fout = new FileWriter(args[1]))
        {
            do {
                i = fin.read();

                // преобразовать пробел в дефис
                if((char)i == ' ') i = '-';

                if(i != -1) fout.write(i);
            } while(i != -1);
        } catch(IOException exc) {
            System.out.println("I/O Error: " + exc);
        }
    }
}
```

9. К какому типу относится поток `System.in`?

К типу `InputStream`.

10. Что возвращает метод `read()` из класса `InputStream` по достижении конца потока?

Значение `-1`.

11. Поток какого типа используется для чтения двоичных данных?

Поток типа `DataInputStream`.

12. Классы `Reader` и `Writer` находятся на вершине иерархии классов

_____.

символьного ввода-вывода

13. Оператор `try` без ресурсов служит для _____.

автоматического управления ресурсами

14. Если для закрытия файла используется традиционный способ, то это лучше всего делать в блоке `finally`. Верно или неверно?

Верно.

Глава 11. Многопоточное программирование

1. Каким образом имеющиеся в Java средства многопоточного программирования позволяют писать более эффективные программы?

Средства многопоточного программирования дают возможность использовать периоды простоя, наступающие практически в любой программе. Когда операции в одном потоке по каким-то причинам не выполняются, в действие вступают другие потоки. В многоядерных системах два и больше потоков могут исполняться одновременно.

2. Для поддержки многопоточного программирования в Java предусмотрен класс _____ и интерфейс _____.

Для поддержки многопоточного программирования в Java предусмотрен класс `Thread` и интерфейс `Runnable`.

3. В каких случаях следует отдать предпочтение расширению класса `Thread` над реализацией интерфейса `Runnable`?

Подклассы, производные от класса `Thread`, целесообразно создавать в тех случаях, когда, помимо метода `run()`, требуется переопределить другие методы данного класса.

4. Покажите, как с помощью метода `join()` можно организовать ожидание завершения потокового объекта `MyThrd`.

```
MyThrd.join();
```

5. Покажите, как установить приоритет потока `MyThrd` на три уровня выше нормального приоритета.

```
MyThrd.setPriority(Thread.NORM_PRIORITY+3);
```

6. Что произойдет, если в объявлении метода указать ключевое слово `synchronized`?

Если указать ключевое слово `synchronized` в объявлении метода, то в каждый момент времени этот метод будет вызываться только в одном потоке для любого заданного объекта его класса.

7. Методы `wait()` и `notify()` служат для _____.
взаимодействия потоков

8. Внесите в класс `TickTock` изменения для организации настоящего отчета времени. Первую половину секунды должен занимать вывод на экран слова "Tick", а вторую — вывод слова "Tock". Таким образом, сообщение "Tick-Tock" должно соответствовать одной секунде отсчитываемого времени. (Время переключения контекстов можно не учитывать.)

Для организации отчета времени достаточно ввести в классе `TickTock` вызовы метода `sleep()`, как показано ниже.

```
// Вариант класса TickTock, в который введены вызовы  
// метода sleep() для организации отсчета времени.
```

```
class TickTock {  
  
    String state; // Состояние часов  
  
    synchronized void tick(boolean running) {
```

```

    if(!running) { // остановить часы
        state = "ticked";
        notify(); // уведомить ожидающие потоки
        return;
    }

    System.out.print("Tick ");

    // ожидать 1/2 секунды
    try {
        Thread.sleep(500);
    } catch (InterruptedException exc) {
        System.out.println("Thread interrupted.");
    }

    state = "ticked"; // установить текущее состояние после такта "тик"

    notify(); // разрешить выполнение метода tock()
    try {
        while(!state.equals("tocked"))
            wait(); // ожидать завершения метода tock()
    }
    catch (InterruptedException exc) {
        System.out.println("Thread interrupted.");
    }
}

synchronized void tock(boolean running) {
    if(!running) { // остановить часы
        state = "tocked";
        notify(); // уведомить ожидающие потоки
        return;
    }

    System.out.println("Tock");

    // ожидать 1/2 секунды
    try {
        Thread.sleep(500);
    } catch (InterruptedException exc) {
        System.out.println("Thread interrupted.");
    }

    state = "tocked"; // установить текущее состояние после такта "так"

    notify(); // разрешить выполнение метода tick()
    try {
        while(!state.equals("ticked"))
            wait(); // ожидать завершения метода tick()
    }
    catch (InterruptedException exc) {
        System.out.println("Thread interrupted.");
    }
}

```

```
}
}
```

9. Почему в новых программах на Java не следует применять методы `suspend()`, `resume()` и `stop()`?

Методы `suspend()`, `resume()` и `stop()` не рекомендуется применять, поскольку они могут стать причиной серьезных осложнений при выполнении программы.

10. С помощью какого метода из класса `Thread` можно получить имя потока?

С помощью метода `getName()`.

11. Какое значение возвращает метод `isAlive()`?

Он возвращает логическое значение `true`, если вызывающий поток выполняется, или логическое значение `false`, если поток завершен.

Глава 12. Перечисления, автоупаковка, статический импорт и аннотации

1. Константы перечислимого типа иногда называются *самотипизированными*. Что это означает?

Часть “само” в термине *самотипизированный* означает тип перечисления, в котором определена константа. Следовательно, константа перечислимого типа является объектом того перечисления, в которое она входит.

2. Какой класс автоматически наследуют перечисления?

Все перечисления наследуют от класса `Enum`.

3. Напишите для приведенного ниже перечисления программу, в которой метод `values()` служит для отображения списка констант и их значений.

```
enum Tools {
    SCREWDRIVER, WRENCH, HAMMER, PLIERS
}
```

Это задание имеет следующее решение:

```
enum Tools {
    SCREWDRIVER, WRENCH, HAMMER, PLIERS
}
```

```
class ShowEnum {
    public static void main(String args[]) {
        for(Tools d : Tools.values())
            System.out.print(d + " has ordinal value of " +
                             d.ordinal() + '\n');
    }
}
```

4. Программу, имитирующую автоматизированный светофор и созданную в примере для опробования 12.1, можно усовершенствовать, внося ряд простых изменений, чтобы выгодно воспользоваться возможностями перечислений. В исходной версии этой программы продолжительность отображения каждого цвета светофора регулировалась в классе `TrafficLightSimulator`, причем величины задержек были жестко запрограммированы в методе `run()`. Измените исходный код программы

таким образом, чтобы продолжительность отображения каждого цвета светофора задавалась константами перечислимого типа `TrafficLightColor`. Для этого вам понадобятся конструктор, переменная экземпляра, объявленная как `private`, а также метод `getDelay()`. Подумайте о том, как еще можно улучшить данную программу. (Подсказка: попробуйте отказаться от оператора `switch` и воспользоваться порядковыми значениями каждого цвета для переключения светофора.)

Усовершенствованная версия программы, имитирующей работу светофора, приведена ниже. В нее внесены два существенных изменения. Во-первых, величина задержки переключения связана теперь со значением перечислимого типа, что улучшает структуру кода. И во-вторых, в методе `run()` удалось обойтись без оператора `switch`. Вместо этого методу `sleep()` теперь передается вызов `tlc.getDelay()`, и благодаря этому автоматически устанавливается задержка, соответствующая текущему цвету светофора.

// Усовершенствованная версия программы, имитирующей работу светофора.
// Величины задержки теперь хранятся в классе **TrafficLightColor**.

```
// Перечисление цветов переключения светофора.
enum TrafficLightColor {
    RED(12000), GREEN(10000), YELLOW(2000);

    private int delay;

    TrafficLightColor(int d) {
        delay = d;
    }

    int getDelay() { return delay; }
}

// Имитация автоматизированного светофора.
class TrafficLightSimulator implements Runnable {
    private Thread thrd; // Поток для имитации светофора
    private TrafficLightColor tlc; // Текущее значение цвета
    boolean stop = false; // Остановка имитации, если истинно
    boolean changed = false; // Переключение светофора, если истинно

    TrafficLightSimulator(TrafficLightColor init) {
        tlc = init;

        thrd = new Thread(this);
        thrd.start();
    }

    TrafficLightSimulator() {
        tlc = TrafficLightColor.RED;

        thrd = new Thread(this);
        thrd.start();
    }

    // Запуск имитации автоматизированного светофора.
```

```

public void run() {
    while(!stop) {
        // По сравнению с предыдущей версией программы
        // код значительно упростился!
        try {
            Thread.sleep(tlc.getDelay());
        } catch (InterruptedException exc) {
            System.out.println(exc);
        }

        changeColor();
    }
}

// Переключение цвета светофора.
synchronized void changeColor() {
    switch(tlc) {
        case RED:
            tlc = TrafficLightColor.GREEN;
            break;
        case YELLOW:
            tlc = TrafficLightColor.RED;
            break;
        case GREEN:
            tlc = TrafficLightColor.YELLOW;
    }

    changed = true;
    notify(); // уведомить о переключении цвета светофора
}

// Ожидание переключения цвета светофора.
synchronized void waitForChange() {
    try {
        while(!changed)
            wait(); // ожидать переключения цвета светофора
        changed = false;
    } catch (InterruptedException exc) {
        System.out.println(exc);
    }
}

// Возврат текущего цвета.
TrafficLightColor getColor() {
    return tlc;
}

// Прекращение имитации светофора.
void cancel() {
    stop = true;
}
}

```

```

class TrafficLightDemo {
    public static void main(String args[]) {
        TrafficLightSimulator tl =
            new TrafficLightSimulator(TrafficLightColor.GREEN);

        for(int i=0; i < 9; i++) {
            System.out.println(tl.getColor());
            tl.waitForChange();
        }

        tl.cancel();
    }
}

```

5. Что такое упаковка и распаковка? В каких случаях производится автоупаковка и автораспаковка?

Упаковка означает включение значения простого типа в объект оболочки, а распаковка — извлечение значения из объекта оболочки. Автоупаковка означает автоматическую упаковку значения без явного создания объекта, тогда как при автораспаковке значение простого типа автоматически извлекается из объекта оболочки без явного вызова соответствующего метода, например `intValue()`.

6. Измените следующий фрагмент кода таким образом, чтобы в нем производилась автоупаковка:

```
Short val = new Short(123);
```

Это задание имеет следующее решение:

```
Short val = 123;
```

7. Объясните, что такое статический импорт?

Статический импорт означает размещение статических членов класса или интерфейса в глобальном пространстве имен. Это позволяет использовать статические члены без указания имени соответствующего класса или интерфейса.

8. Какие действия выполняет приведенный ниже оператор?

```
import static java.lang.Integer.parseInt;
```

Этот оператор помещает в глобальное пространство имен метод `parseInt()` из класса оболочки типа `Integer`.

9. Следует ли употреблять статический импорт от случая к случаю или желательно импортировать статические члены всех классов?

Статический импорт уместен только в отдельных случаях. Если доступным окажется слишком много статических членов, это может привести к конфликтам имен и нарушению структуры кода.

10. Синтаксис аннотации основывается на _____.
интерфейсе

11. Какая аннотация называется маркером?

Маркер — это аннотация без аргументов.

12. Аннотации применимы только к методам. Верно или неверно?

Неверно. Любое объявление может быть аннотировано.

Глава 13. Обобщения

1. Обобщения очень важны, поскольку они позволяют создавать код, который:

- а) обеспечивает типовую безопасность;
- б) пригоден для повторного использования;
- в) отличается высокой надежностью;
- г) обладает всеми перечисленными выше свойствами.

Ответ: г) код обладает всеми перечисленными выше свойствами.

2. Можно ли указывать простой тип в качестве аргумента типа?

Нет, нельзя. В качестве аргументов типа можно указывать только типы объектов.

3. Как объявить класс `FlightSched` с двумя параметрами типа?

Это задание имеет следующее решение:

```
class FlightSched<T, V> {
```

4. Измените ваш ответ на вопрос 3 таким образом, чтобы второй параметр типа обозначал подкласс, производный от класса `Thread`.

Это задание имеет следующее решение:

```
class FlightSched<T, V extends Thread> {
```

5. Внесите изменения в класс `FlightSched` таким образом, чтобы второй параметр типа стал подклассом первого параметра типа.

Это задание имеет следующее решение:

```
class FlightSched<T, V extends T> {
```

6. Что обозначает знак `?` в обобщениях?

Знак `?` обозначает метасимвольный аргумент, который соответствует любому допустимому типу.

7. Может ли метасимвольный аргумент быть ограниченным?

Да. Метасимвольный аргумент может ограничиваться как сверху, так и снизу.

8. У обобщенного метода `MyGen()` имеется один параметр типа, определяющий тип передаваемого ему аргумента. Этот метод возвращает также объект, тип которого соответствует параметру типа. Как должен быть объявлен метод `MyGen()`?

Это задание имеет следующее решение:

```
<T> T MyGen(T o) { // ...
```

9. Допустим, обобщенный интерфейс объявлен так, как показано ниже.

```
interface IGenIF<T, V extends T> { // ...
```

Составьте объявление класса `MyClass`, который реализует интерфейс `IGenIF`.

Это задание имеет следующее решение:

```
class MyClass<T, V extends T> implements IGenIF<T, V> { // ...
```

10. Допустим, имеется обобщенный класс `Counter<T>`. Как создать объект его базового типа?

Для того чтобы получить базовый тип из обобщенного класса `Counter<T>`, достаточно указать его имя, не обозначая тип, как показано ниже.

```
Counter x = new Counter;
```


11. Существуют ли параметры типа на стадии выполнения программы?

Нет. Все параметры типа удаляются на стадии компиляции и заменяются соответствующими приводимыми типами. Этот процесс называется стиранием.

12. Видоизмените ответ на вопрос 10 в упражнении по материалу главы 9 таким образом, чтобы сделать класс обобщенным. По ходу дела создайте интерфейс стека IGenStack, объявив в нем обобщенные методы push() и pop().

// Обобщенный стек.

```
interface IGenStack<T> {
    void push(T obj) throws StackFullException;
    T pop() throws StackEmptyException;
}
```

// Исключение, возникающее при переполнении стека.

```
class StackFullException extends Exception {
    int size;

    StackFullException(int s) { size = s; }

    public String toString() {
        return "\nStack is full. Maximum size is " + size;
    }
}
```

// Исключение, возникающее при обращении к пустому стеку.

```
class StackEmptyException extends Exception {

    public String toString() {
        return "\nStack is empty.";
    }
}
```

// Класс, реализующий стек для хранения объектов обобщенного типа.

```
class GenStack<T> implements IGenStack<T> {
    private T stck[]; // Массив для хранения элементов стека
    private int tos; // Вершина стека
```

// построить пустой стек заданного размера

```
GenStack(T[] stckArray) {
    stck = stckArray;
    tos = 0;
}
```

// построить один стек из другого стека

```
GenStack(T[] stckArray, GenStack<T> ob) {
```

```
    tos = ob.tos;
    stck = stckArray;
```

```
    try {
```

```

        if(stck.length < ob.stck.length)
            throw new StackFullException(stck.length);
    }
    catch(StackFullException exc) {
        System.out.println(exc);
    }

    // скопировать элементы
    for(int i=0; i < tos; i++)
        stck[i] = ob.stck[i];
}

// построить стек с исходными значениями
GenStack(T[] stckArray, T[] a) {
    stck = stckArray;

    for(int i = 0; i < a.length; i++) {
        try {
            push(a[i]);
        }
        catch(StackFullException exc) {
            System.out.println(exc);
        }
    }
}

// поместить объекты в стек
public void push(T obj) throws StackFullException {
    if(tos==stck.length)
        throw new StackFullException(stck.length);

    stck[tos] = obj;
    tos++;
}

// извлечь объекты из стека
public T pop() throws StackEmptyException {
    if(tos==0)
        throw new StackEmptyException();

    tos--;
    return stck[tos];
}

// продемонстрировать применение класса GenStack
class GenStackDemo {
    public static void main(String args[]) {
        // создать пустой стек на 10 элементов типа Integer
        Integer iStore[] = new Integer[10];
        GenStack<Integer> stk1 = new GenStack<Integer>(iStore);

        // построить стек из массива
    }
}

```

```

String name[] = {"One", "Two", "Three"};
String strStore[] = new String[3];
GenStack<String> stk2 =
    new GenStack<String>(strStore, name);

String str;
int n;

try {

    // поместить ряд значений в стек stk1
    for(int i=0; i < 10; i++)
        stk1.push(i);
    } catch(StackFullException exc) {
        System.out.println(exc);
    }

    // построить один стек из другого стека
    String strStore2[] = new String[3];
    GenStack<String> stk3 =
        new GenStack<String>(strStore2, stk2);

    try {
        // отобразить стеки
        System.out.print("Contents of stk1: ");
        for(int i=0; i < 10; i++) {
            n = stk1.pop();
            System.out.print(n + " ");
        }

        System.out.println("\n");

        System.out.print("Contents of stk2: ");
        for(int i=0; i < 3; i++) {
            str = stk2.pop();
            System.out.print(str + " ");
        }

        System.out.println("\n");

        System.out.print("Contents of stk3: ");
        for(int i=0; i < 3; i++) {
            str = stk3.pop();
            System.out.print(str + " ");
        }

    } catch(StackEmptyException exc) {
        System.out.println(exc);
    }

    System.out.println();
}
}

```

13. Что обозначают угловые скобки (< >)?

Угловые скобки обозначают ромбовидный оператор.

14. Как упростить приведенную ниже строку кода в версии JDK 7?

```
MyClass<Double,String> obj = new MyClass<Double,String>(1.1,"Hi");
```

Эту строку кода можно упростить, используя ромбовидный оператор следующим образом:

```
MyClass<Double,String> obj = new MyClass<>(1.1,"Hi");
```

Глава 14. Апплеты, события и прочее

1. Какой метод вызывается первым в начале выполнения апплета? Какой метод вызывается, когда апплет должен быть удален из системы?

Выполнение апплета начинается с вызова метода `init()`. А перед удалением апплета вызывается метод `destroy()`.

2. Почему в апплете, который должен работать постоянно, следует организовать многопоточный режим?

Многопоточный режим необходим в постоянно выполняющемся апплете, потому что апплет — это управляемая событиями прикладная программа, в которой не предусмотрены длительные операции. Например, метод `paint()` не будет вызван до тех пор, пока метод `start()` не возвратит управление.

3. Видоизмените апплет, созданный в примере для опробования 14.1, таким образом, чтобы в нем отображалась символьная строка, передаваемая ему в качестве параметра. Добавьте еще один параметр, чтобы задавать время задержки (в миллисекундах) между последовательными сдвигами символов строки.

```
/* Простой апплет, отображающий крупный
   заголовок с использованием параметров.
*/
```

```
import java.awt.*;
import java.applet.*;
```

```
/*
<applet code="ParamBanner" width=300 height=50>
<param name=message value=" I like Java! ">
<param name=delay value=500>
</applet>
*/
```

```
public class ParamBanner extends Applet implements Runnable {
    String msg;
    int delay;
    Thread t;
    boolean stopFlag;
```

```
    // инициализировать переменную t пустым значением null
    public void init() {
        String temp;
```

```

msg = getParameter("message");
if(msg == null) msg = " Java Rules the Web ";

temp = getParameter("delay");
try {
    if(temp != null)
        delay = Integer.parseInt(temp);
    else
        delay = 250; // Если параметр не задан,
                     // используется значение по умолчанию.
} catch(NumberFormatException exc) {
    delay = 250 ; // Если допущена ошибка, используется
                 // значение по умолчанию.
}

t = null;
}

// запустить поток
public void start() {
    t = new Thread(this);
    stopFlag = false;
    t.start();
}

// Точка входа в поток, манипулирующий крупным заголовком.
public void run() {
    char ch;

    // отобразить крупный заголовок снова
    for( ; ; ) {
        try {
            repaint();
            Thread.sleep(delay);
            ch = msg.charAt(0);
            msg = msg.substring(1, msg.length());
            msg += ch;
            if(stopFlag)
                break;
        } catch(InterruptedException exc) {}
    }
}

// остановить отображение крупного заголовка
public void stop() {
    stopFlag = true;
    t = null;
}

// отобразить крупный заголовок
public void paint(Graphics g) {
    g.drawString(msg, 50, 30);
}

```

```
}
}
```

- 4. Дополнительное задание.** Создайте апплет, который отображал бы текущее время, обновляя содержимое своего окна каждую секунду. Для того чтобы справиться с этим заданием, вам придется провести дополнительные изыскания. Для начала примите к сведению следующую подсказку: получить текущее время можно, воспользовавшись объектом класса `Calendar`, относящегося к пакету `java.util`. (Напомним, что компания Oracle предоставляет оперативно доступную документацию на все стандартные классы Java.) Приобретенных вами до сих пор знаний должно быть достаточно для того, чтобы самостоятельно изучить класс `Calendar` и использовать его методы для выполнения этого задания.

// Простой апплет, отображающий текущее время.

```
import java.util.*;
import java.awt.*;
import java.applet.*;
/*
<object code="Clock" width=200 height=50>
</object>
*/

public class Clock extends Applet implements Runnable {
    String msg;
    Thread t;
    Calendar clock;

    boolean stopFlag;

    // инициализировать апплет
    public void init() {
        t = null;
        msg = "";
    }

    // запустить поток
    public void start() {
        t = new Thread(this);
        stopFlag = false;
        t.start();
    }

    // Точка входа в поток, отображающий время.
    public void run() {
        // отобразить время повторно
        for( ; ; ) {
            try {
                repaint();
                Thread.sleep(1000);
                if(stopFlag)
                    break;
            } catch (InterruptedException exc) {}
        }
    }
}
```

```

    }
}

// приостановить поток, отображающий время
public void stop() {
    stopFlag = true;
    t = null;
}

// отобразить время
public void paint(Graphics g) {
    clock = Calendar.getInstance();

    msg = "Current time is " +
        Integer.toString(clock.get(Calendar.HOUR));
    msg = msg + ":" +
        Integer.toString(clock.get(Calendar.MINUTE));
    msg = msg + ":" +
        Integer.toString(clock.get(Calendar.SECOND));
    g.drawString(msg, 30, 30);
}
}

```

5. Поясните вкратце, каким образом действует модель делегирования событий в Java.

В модели делегирования событий *источник* формирует событие и передает его одному или нескольким *приемникам*, которые лишь ожидают наступления события. Получив уведомление, приемник обрабатывает событие, а затем возвращает управление.

6. Должен ли приемник событий быть зарегистрирован в их источнике?

Должен. Для того чтобы приемник получал уведомления о событиях, он должен быть зарегистрирован в источнике.

7. Дополнительное задание. Среди языковых средств Java для отображения информации имеется метод `drawLine()`. Он рисует текущим цветом прямую линию между двумя точками. Этот метод относится классу `Graphics`. Используя метод `drawLine()`, напишите апплет, отслеживающий перемещение мыши. Если кнопка мыши нажата, апплет должен рисовать на экране непрерывную линию до тех пор, пока кнопка не будет отпущена.

```

/* Отслеживание перемещения мыши путем рисования линии.
   Признаками начала и конца линии являются нажатие
   и отпускание кнопки мыши. */

```

```

import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="TrackM" width=300 height=100>
</applet>
*/

public class TrackM extends Applet
    implements MouseListener, MouseMotionListener {

```

```

int curX = 0, curY = 0; // Текущие координаты
int oldX = 0, oldY = 0; // Предыдущие координаты
boolean draw;

public void init() {
    addMouseListener(this);
    addMouseMotionListener(this);
    draw = false;
}

/* Следующие три метода не используются, но их пустые реализации
   нужны, так как эти методы объявлены в интерфейсе MouseListener. */

// обработать наведение курсора мыши на элемент
// пользовательского интерфейса
public void mouseEntered(MouseEvent me) {
}

// обработать отведение мыши от элемента
// пользовательского интерфейса
public void mouseExited(MouseEvent me) {
}

// обработать щелчок кнопкой мыши
public void mouseClicked(MouseEvent me) {
}

// обработать нажатие кнопки мыши
public void mousePressed(MouseEvent me) {
    // сохранить координаты
    oldX = me.getX();
    oldY = me.getY();
    draw = true;
}

// обработать отпускание кнопки мыши
public void mouseReleased(MouseEvent me) {
    draw = false;
}

// обработать перетаскивание мыши
public void mouseDragged(MouseEvent me) {
    // сохранить координаты
    curX = me.getX();
    curY = me.getY();
    repaint();
}

// обработать перемещение мыши
public void mouseMoved(MouseEvent me) {
    // отобразить состояние
    showStatus("Moving mouse at " + me.getX() + ", " + me.getY());
}

```



```
// отобразить линию в окне апплета
public void paint(Graphics g) {
    if(draw)
        g.drawLine(oldX, oldY, curX, curY);
    }
}
```

8. Объясните назначение ключевого слова `assert`.

Ключевое слово `assert` создает *утверждение*, т.е. условие, которое должно быть истинным при выполнении программы. Если же утверждение ложно, генерируется исключение `AssertionError`.

9. Приведите хотя бы одну причину, по которой может возникнуть потребность в использовании собственного метода в некоторых программах.

Собственный метод может понадобиться для сопряжения с подпрограммами, написанными на других языках программирования, или для оптимизации кода с учетом конкретной среды исполнения.

Глава 15. Введение в Swing

1. Компоненты AWT являются тяжеловесными, а компоненты Swing — _____.

легковесными

2. Может ли изменяться стиль оформления компонента Swing? Если да, то какое средство позволяет это сделать?

Да, может. Это позволяют сделать подключаемые стили оформления в Swing.

3. Какой контейнер верхнего уровня чаще всего используется в приложениях?

Контейнер `JFrame`.

4. Контейнер верхнего уровня содержит несколько панелей. На какой панели размещаются компоненты?

На панели содержимого.

5. Как создать ссылку, отображающую сообщение "Select an entry from the list" (Выберите элемент из списка)?

`JLabel("Select an entry from the list")`

6. В каком потоке должно происходить все взаимодействие с компонентами графического пользовательского интерфейса?

В потоке диспетчеризации событий.

7. Какая команда действия связывается по умолчанию с компонентом `JButton`? Как изменить команду действия?

По умолчанию символьная строка команды действия содержит текст надписи на кнопке. Команду действия можно изменить, вызвав метод `setActionCommand()`.

8. Какое событие формируется при нажатии кнопки?

Событие `ActionEvent`.

9. Как создать поле ввода текста шириной до 32 символов?

`JTextField(32)`

10. Можно ли установить команду действия для компонента `TextField`? Если можно, то как это сделать?

Да, можно. Для этого достаточно вызвать метод `setActionCommand()`.

11. С помощью какого компонента `Swing` можно создать флажок? Какое событие формируется при установке или сбросе флажка?

Флажок создается с помощью компонента `JCheckBox`. При установке или сбросе флажка формируется событие `ItemEvent`.

12. Компонент `JList` отображает список элементов, которые может выбирать пользователь. Верно или неверно?

Верно.

13. Какое событие формируется при выборе пользователем элемента из списка типа `JList` или отмене выбора?

Событие `ListSelectionEvent`.

14. В каком методе задается режим выбора элементов списка типа `JList`? С помощью какого метода можно получить индекс первого выбранного элемента?

Режим выбора элементов списка задается в методе `setSelectionMode()`. А метод `getSelectedIndex()` возвращает индекс первого выбранного элемента.

15. Подкласс какого класса нужно создать при разработке `Swing`-апплета?

Подкласс `JApplet`.

16. Обычно при построении исходного пользовательского интерфейса в `Swing`-апплетах используется метод `invokeAndWait()`. Верно или неверно?

Верно.

17. Добавьте в утилиту сравнения файлов, созданную в примере для опробования 15.1, флажок со следующей пояснительной надписью: **Show position of mismatch** (Показывать место несовпадения). Если этот флажок установлен, программа должна отображать место, в котором обнаружено первое расхождение в содержимом сравниваемых файлов.

/*

Пример для опробования 15-1.

Утилита сравнения файлов, создаваемая на основе **Swing**.

В этой версии присутствует флажок для установки режима показа места первого несовпадения содержимого сравниваемых файлов.

Для компиляции этой утилиты требуется `JDK 7` или более поздняя версия данного комплекта.

*/

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import java.io.*;
```

```
class SwingFC implements ActionListener {
```

```

JTextField jtfFirst; // Переменная для хранения имени первого файла
JTextField jtfSecond; // Переменная для хранения имени второго файла

JButton jbtnComp; // Кнопка для сравнения файлов

JLabel jlabFirst, jlabSecond; // Подсказки для пользователя
JLabel jlabResult; // Сведения о результатах и сообщения об ошибках

JCheckBox jcbLoc; // флажок для показа места первого несовпадения
файлов

SwingFC() {

    // создать новый контейнер JFrame
    JFrame jfrm = new JFrame("Compare Files");

    // установить диспетчер компоновки FlowLayout
    jfrm.setLayout(new FlowLayout());

    // задать исходные размеры рамки окна
    jfrm.setSize(200, 190);

    // завершить программу после закрытия окна
    jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // создать поля для ввода имен файлов
    jtfFirst = new JTextField(14);
    jtfSecond = new JTextField(14);

    // установить команды действия для полей ввода текста
    jtfFirst.setActionCommand("fileA");
    jtfSecond.setActionCommand("fileB");

    // создать кнопку Compare
    JButton jbtnComp = new JButton("Compare");

    // добавить приемник событий действия от кнопки Compare
    jbtnComp.addActionListener(this);

    // создать метки
    jlabFirst = new JLabel("First file: ");
    jlabSecond = new JLabel("Second file: ");
    jlabResult = new JLabel("");

    // создать флажок
    jcbLoc = new JCheckBox("Show position of mismatch");

    // добавить компоненты на панели содержимого
    jfrm.add(jlabFirst);
    jfrm.add(jtfFirst);
    jfrm.add(jlabSecond);
    jfrm.add(jtfSecond);
    jfrm.add(jcbLoc);

```

```

jfrm.add(jbtnComp);
jfrm.add(jlabResult);

// отобразить рамку окна
jfrm.setVisible(true);
}

// сравнить файлы после нажатия кнопки Compare
public void actionPerformed(ActionEvent ae) {
    int i=0, j=0;

    // сначала убедиться, что введены имена обоих файлов
    if(jtfFirst.getText().equals("")) {
        jlabResult.setText("First file name missing.");
        return;
    }
    if(jtfSecond.getText().equals("")) {
        jlabResult.setText("Second file name missing.");
        return;
    }

    // сравнить файлы, используя оператор try с ресурсами
    try (FileInputStream f1 = new FileInputStream(jtfFirst.getText());
        FileInputStream f2 = new FileInputStream(jtfSecond.getText()))

        // проверить содержимое каждого файла
        do {
            i = f1.read();
            j = f2.read();
            if(i != j) break;
        } while(i != -1 && j != -1);

        if(i != j) {
            if(jcbLoc.isSelected())
                jlabResult.setText("Files differ at location " + count);
            else
                jlabResult.setText("Files are not the same.");
        }
        else
            jlabResult.setText("Files compare equal.");
    } catch(IOException exc) {
        jlabResult.setText("File Error");
    }
}

public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new SwingFC();
        }
    });
}
}

```

18. Измените программу ListDemo таким образом, чтобы она допускала выбор нескольких элементов из списка.

```
// Демонстрация выбора нескольких элементов из списка
// с помощью компонента JList.

// Для компиляции этой программы требуется JDK 7
// или более поздняя версия данного комплекта.

import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;
import java.awt.event.*;

class ListDemo implements ListSelectionListener {

    JList<String> jlst;
    JLabel jlab;
    JScrollPane jscrlp;

    // создать массив имен
    String names[] = { "Sherry", "Jon", "Rachel",
                      "Sasha", "Josselyn", "Randy",
                      "Tom", "Mary", "Ken",
                      "Andrew", "Matt", "Todd" };

    ListDemo() {
        // создать новый контейнер JFrame
        JFrame jfrm = new JFrame("JList Demo");

        // установить диспетчер компоновки FlowLayout
        jfrm.setLayout(new FlowLayout());

        // задать исходные размеры рамки окна
        jfrm.setSize(200, 160);

        // завершить программу после закрытия окна
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // создать компонент JList
        jlst = new JList<String>(names);

        // Удалив следующую строку кода, можно задать режим выбора
        // нескольких элементов из списка, поскольку этот режим
        // устанавливается для компонента JList по умолчанию.
        // jlst.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

        // добавить список на панели прокрутки.
        jscrlp = new JScrollPane(jlst);

        // задать предпочтительные размеры панели прокрутки
        jscrlp.setPreferredSize(new Dimension(120, 90));
    }
}
```

```

// создать метку для отображения результатов выбора из списка.
jlab = new JLabel("Please choose a name");

// добавить обработчик событий, связанных с выбором из списка
jlst.addListSelectionListener(this);

// добавить список и метку на панели содержимого
jfrm.add(jscrlp);
jfrm.add(jlab);

// отобразить рамку окна
jfrm.setVisible(true);
}

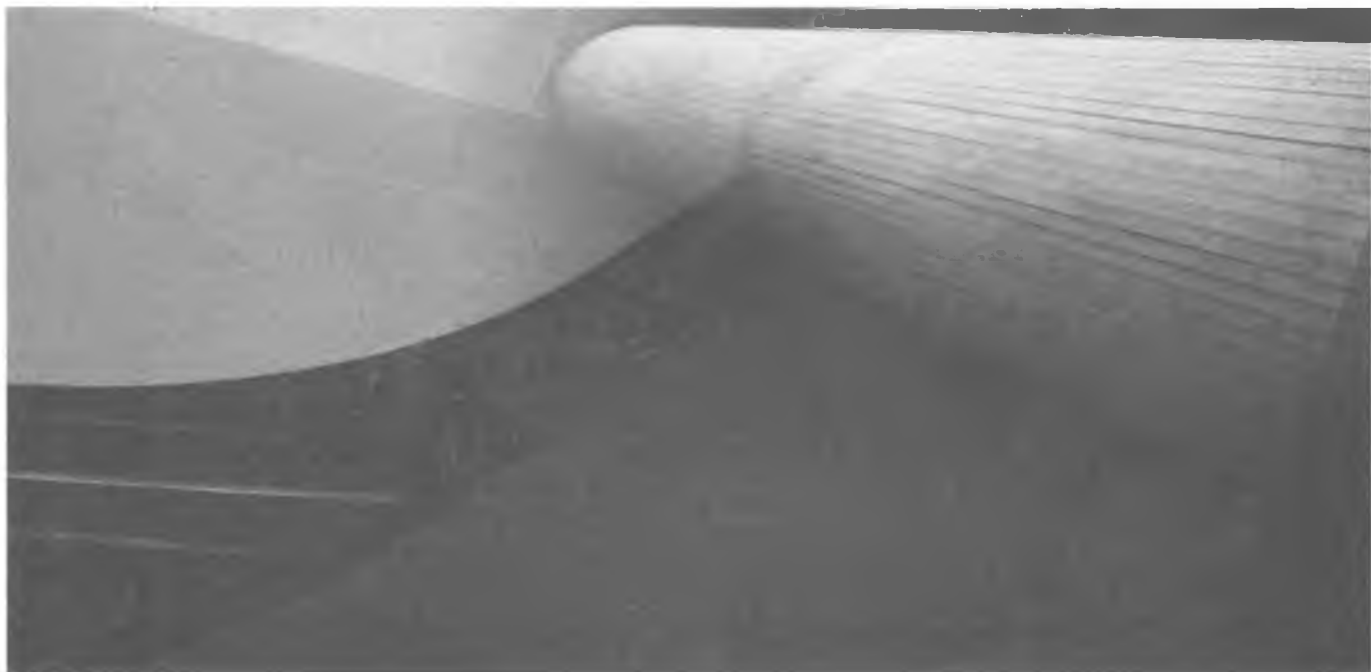
// обработать события, связанные с выбором элементов из списка
public void valueChanged(ListSelectionEvent le) {
    // получить индексы тех элементов, выбор которых был сделан
    // или отменен в списке
    int indices[] = jlst.getSelectedIndices();

    // отобразить результат выбора, если был выбран один
    // или несколько элементов из списка
    if(indices.length != 0) {
        String who = "";

        // построить символьную строку из выбранных имен
        for(int i : indices)
            who += names[i] + " ";
        jlab.setText("Current selections: " + who);
    }
    else // иначе еще раз предложить сделать выбор
        jlab.setText("Please choose a name");
}

public static void main(String args[]) {
    // создать рамку окна в потоке диспетчеризации событий
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new ListDemo();
        }
    });
}
}

```

Приложение Б

**Применение
документирующих
комментариев в Java**

Как пояснялось в главе 1, в Java поддерживаются три вида комментариев. Первые два вида обозначаются символами `//` и `/* */`, а третий их вид называется *документирующими комментариями*. Такие комментарии начинаются символами `/**` и оканчиваются символами `*/`. Документирующие комментарии позволяют включать сведения о программе в исходный код самой программы. Для извлечения этих сведений и последующего их преобразования в формат HTML-документа служит утилита `javadoc`, входящая в состав JDK. Документирующие комментарии — удобный способ документирования прикладных программ. Вам, вероятно, уже встречалась документация, сформированная утилитой `javadoc`, поскольку именно такой способ применяется для составления документации на библиотеку Java API.

Дескрипторы javadoc

Утилита `javadoc` распознает и обрабатывает в документирующих комментариях следующие дескрипторы

Дескриптор	Описание
<code>@author</code>	Обозначает автора программы
<code>{@code}</code>	Отображает данные шрифтом, предназначенным для вывода исходного кода, не выполняя преобразований в формат HTML-документа
<code>@deprecated</code>	Указывает на то, что элемент программы не рекомендован к применению
<code>{@docRoot}</code>	Указывает путь к корневому каталогу документации
<code>@exception</code>	Обозначает исключение, генерируемое методом
<code>{@inheritDoc}</code>	Наследует комментарии от ближайшего суперкласса
<code>{@link}</code>	Вставляет ссылку на другую тему
<code>{@linkplain}</code>	Вставляет ссылку на другую тему, но ссылка отображается тем же шрифтом, что и простой текст
<code>{@literal}</code>	Отображает данные, не выполняя преобразований в формат HTML-документа
<code>@param</code>	Документирует параметр метода
<code>@return</code>	Документирует значение, возвращаемое методом
<code>@see</code>	Указывает ссылку на другую тему
<code>@serial</code>	Документирует поле, упорядочиваемое по умолчанию
<code>@serialData</code>	Документирует данные, записываемые методом <code>writeObject()</code> или <code>writeExternal()</code>
<code>@serialField</code>	Документирует компонент <code>ObjectStreamField</code>
<code>@since</code>	Обозначает версию, в которой были внесены определенные изменения
<code>@throws</code>	То же, что и дескриптор <code>@exception</code>
<code>{@value}</code>	Отображает значение константы, которая должна быть определена как поле типа <code>static</code>
<code>@version</code>	Обозначает версию класса

Дескрипторы, начинающиеся с символа `@`, называются *автономными* и помечают строку комментариев. А дескрипторы, заключенные в фигурные скобки, называются

встраиваемыми и могут быть использованы в других дескрипторах. В документирующих комментариях можно также использовать стандартные HTML-дескрипторы. Но некоторые HTML-дескрипторы, например дескрипторы заголовков, применять не следует, поскольку они могут испортить внешний вид HTML-документа, составляемого утилитой `javadoc`.

Что касается документирования исходного кода, то документирующие комментарии можно использовать для описания классов, интерфейсов, полей, конструкторов и методов. Но в любом случае документирующие комментарии должны предшествовать непосредственно описываемому элементу исходного кода. Одни дескрипторы, в том числе `@see`, `@since` и `@deprecated`, могут быть использованы для документирования любых элементов исходного кода, а другие — только для документирования соответствующих элементов. Каждый дескриптор документирующих комментариев рассматривается далее по отдельности.

На заметку

Документирующие комментарии можно также использовать для составления документации и краткого обзора разрабатываемого пакета, но делается это иначе, чем документирование исходного кода. Подробнее об этом можно узнать из документации на утилиту `javadoc`.

Дескриптор `@author`

Дескриптор `@author` описывает автора класса или интерфейса и имеет следующий синтаксис:

```
@author описание
```

где *описание*, как правило, обозначает имя автора. Для того чтобы сведения, указываемые в поле `@author`, были включены в результирующий HTML-документ, при вызове утилиты `javadoc` из командной строки следует указать параметр `-author`.

Дескриптор `{@code}`

Дескриптор `{@code}` позволяет включать в комментарии текст, в том числе и отдельные фрагменты кода. Такой текст будет выводиться специальным шрифтом, используемым для форматирования кода, и не подлежит дальнейшей обработке по правилам форматирования HTML-документов. Этот дескриптор имеет следующий синтаксис:

```
{@code фрагмент_кода}
```

Дескриптор `@deprecated`

Дескриптор `@deprecated` указывает на то, что класс, интерфейс или метод не рекомендован к применению. В описание рекомендуется включать дескриптор `@see` или `{@link}`, чтобы уведомить программиста о других возможных решениях. У этого дескриптора имеется следующий синтаксис:

```
@deprecated описание
```

где *описание* обозначает сообщение, описывающее причины, по которым данное языковое средство Java не рекомендуется к применению. Дескриптор `@deprecated` можно применять для документирования полей, методов, конструкторов, классов и интерфейсов.

Дескриптор `{@docRoot}`

Дескриптор `{@docRoot}` указывает путь к корневому каталогу документации.

Дескриптор `@exception`

Дескриптор `@exception` описывает исключение, которое может возникнуть при выполнении метода. У него имеется следующий синтаксис:

```
@exception имя_исключения пояснение
```

где *имя_исключения* обозначает полностью определенное имя исключения, а *пояснение* — символьную строку, в которой поясняется, при каких условиях исключение может возникнуть. Дескриптор `@exception` можно применять только для документирования методов.

Дескриптор `{@inheritDoc}`

Этот дескриптор наследует комментарии от ближайшего суперкласса.

Дескриптор `{@link}`

Дескриптор `{@link}` предоставляет встраиваемую ссылку на дополнительные сведения. У него имеется следующий синтаксис:

```
{@link пакет.класс#член текст}
```

где *пакет.класс#член* обозначает имя класса или метода, на который делается встраиваемая ссылка, а *текст* — символьную строку, отображаемую в виде встраиваемой ссылки.

Дескриптор `{@linkplain}`

Дескриптор `{@linkplain}` вставляет встраиваемую ссылку на другую тему. Эта ссылка отображается обычным шрифтом. А в остальном данный дескриптор подобен дескриптору `{@link}`.

Дескриптор `{@literal}`

Дескриптор `{@literal}` позволяет включать текст в комментарии. Этот текст отображается без дополнительной обработки по правилам форматирования HTML-документов. У него имеется следующий синтаксис:

```
@literal описание
```

где *описание* обозначает текст, включаемый в комментарии.

Дескриптор `@param`

Дескриптор `@param` описывает параметр. У него имеется следующий синтаксис:

```
@parameter имя_параметра пояснение
```

где *имя_параметра* обозначает конкретное наименование параметра, а *пояснение* — поясняемое назначение параметра. Дескриптор `@param` можно применять для документирования метода, конструктора, а также обобщенного класса или интерфейса.

Дескриптор @return

Дескриптор @return описывает значение, возвращаемое методом. У него имеется следующий синтаксис:

@return *пояснение*

где *пояснение* обозначает тип и назначение возвращаемого значения. Дескриптор @return применяется только для документирования методов.

Дескриптор @see

Дескриптор @see предоставляет ссылку на дополнительные сведения. Ниже приведены две наиболее употребительные формы этого дескриптора.

@see *ссылка*

@see *пакет.класс#член текст*

В первой форме *ссылка* обозначает абсолютный или относительный веб-адрес (URL). А во второй форме *пакет.класс#член* обозначает имя элемента, тогда как *текст* — отображаемые сведения об этом элементе. Параметр *текст* указывать необязательно, а в его отсутствие отображается элемент, определяемый параметром *пакет.класс#член*. Имя члена также может быть опущено. Этот дескриптор дает возможность указать ссылку не только на метод или поле, но и на класс или интерфейс. Имя элемента может быть указано полностью или частично. Но если имени члена предшествует точка, она должна быть заменена знаком #.

Дескриптор @serial

Дескриптор @serial определяет комментарии к полю, упорядочиваемому по умолчанию. У этого дескриптора имеется следующий синтаксис:

@serial *описание*

где *описание* обозначает комментарии к данному полю.

Дескриптор @serialData

Дескриптор @serialData предназначен для документирования данных, которые были записаны с помощью методов writeObject() и writeExternal(). Синтаксис этого дескриптора приведен ниже.

@serialData *описание*

где *описание* обозначает комментарии к записанным данным.

Дескриптор @serialField

Этот дескриптор предназначен для документирования классов, реализующих интерфейс Serializable. Он предоставляет комментарии к компоненту ObjectOutputStreamField. У этого дескриптора имеется следующий синтаксис:

@serialField *имя тип описание*

где *имя* и *тип* обозначают конкретное наименование и тип поля соответственно, а *описание* — комментарии к этому полю.

Дескриптор @since

Дескриптор @since устанавливает, что данный элемент был внедрен, начиная с указанной версии программы. Синтаксис этого дескриптора приведен ниже.

@since *версия*

Здесь *версия* обозначает символьную строку, указывающую версию или выпуск программы, где был внедрен данный элемент.

Дескриптор @throws

Дескриптор @throws выполняет те же действия, что и дескриптор @exception.

Дескриптор @value

Этот дескриптор применяется в двух основных формах. В первой форме отображается значение константы, которой предшествует этот дескриптор. Константа должна быть полем типа static. Ниже приведена первая форма этого дескриптора.

{@value}

Во второй форме отображается значение указываемого статического поля. Эта форма выглядит следующим образом:

{@value *пакет.класс#член* }

где *пакет.класс#член* обозначает имя статического поля.

Дескриптор @version

Дескриптор @version описывает версию класса. Ниже приведен синтаксис этого дескриптора.

@version *информация*

Здесь *информация* обозначает символьную строку, содержащую сведения о версии программы. Как правило, это номер версии, например 2.2. Для того чтобы сведения в поле дескриптора @version были включены в результирующий HTML-документ, при вызове утилиты javadoc из командной строки следует указать параметр **-version**.

Общая форма документирующих комментариев

После символов /** следует одна или несколько строк с общим описанием класса, интерфейса, переменной или метода. Далее можно ввести любое количество дескрипторов, начинающихся со знака @. Каждый такой дескриптор должен начинаться с новой строки или следовать после одной или нескольких звездочек (*) в начале строки. Несколько однотипных дескрипторов должны быть объединены вместе. Так, если требуется использовать три дескриптора @see, их следует расположить друг за другом. Встраиваемые дескрипторы (начинающиеся с фигурной скобки) можно применять в любом описании.

Ниже приведен пример, демонстрирующий применение документирующих комментариев для описания класса.

```
/**
 * Класс для отображения гистограммы.
 * @author Herbert Schildt
 * @version 3.2
 */
```

Результат, выводимый утилитой javadoc

Утилита `javadoc` читает данные из исходного файла программы на Java и формирует несколько HTML-файлов, содержащих документацию на эту программу. Сведения о каждом классе помещаются в отдельный файл. В результате выполнения утилиты `javadoc` составляется также предметный указатель и дерево иерархии. Кроме того, могут быть сформированы и другие HTML-файлы.

Пример применения документирующих комментариев

Ниже приведен пример программы, в исходном тексте которой имеются документирующие комментарии. Обратите внимание на то, что каждый такой комментарий непосредственно предшествует описываемому элементу программы. После обработки утилитой `javadoc` документация на класс `SquareNum` помещается в файл `SquareNum.html`.

```
import java.io.*;

/**
 * Класс, демонстрирующий применение документирующих комментариев.
 * @author Herbert Schildt
 * @version 1.2
 */

public class SquareNum {
    /**
     * Этот метод возвращает квадрат значения параметра num.
     * Это описание состоит из нескольких строк. Число строк
     * не ограничивается.
     * @param num Значение, которое требуется возвести в квадрат.
     * @return Квадрат числового значения параметра num.
     */
    public double square(double num) {
        return num * num;
    }

    /**
     * Этот метод получает значение, введенное пользователем.
     * @return Введенное значение типа double.
     * @exception IOException Искключение при ошибке ввода.
     * @see IOException
     */
    public double getNumber() throws IOException {
        // создать поток BufferedReader из стандартного потока System.in.
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader inData = new BufferedReader(isr);
        String str;

        str = inData.readLine();
        return (new Double(str)).doubleValue();
    }
}
```

```
/**
 * В этом методе демонстрируется применение метода square().
 * @param args Не используется.
 * @exception IOException Исключение при ошибке ввода.
 * @see IOException
 */
public static void main(String args[])
    throws IOException
{
    SquareNum ob = new SquareNum();
    double val;

    System.out.println("Enter value to be squared: ");
    val = ob.getNumber();
    val = ob.square(val);

    System.out.println("Squared value is " + val);
}
}
```

Предметный указатель

А

Автоупаковка и автораспаковка
механизм 438
п применение реимущества 438
применение
в выражениях 440
в методах 439
в операторах присваивания 439
неоправданное, предупреждение 442

Аннотации

встроенные 446
как маркеры 446
объявление 446
определение 445
применение 446

Апплеты

архитектура 491
безопасность 28
вывод данных в окне 494
заготовка 492
запуск на выполнение 490
инициализация и завершение 493
категории 489
методы
основные 492
порядок вызова 493
назначение 27
на основе
AWT 489
Swing 550
ограничения 494
определение 488
передача параметров 500
переносимость 28
свойства 491
строка состояния, применение 499

Аргументы

командной строки 184
метасимвольные
метасимвольныеограниченные 465
метасимвольныеприменение 463
метасимвольныеуказание 463
метода 130
определение 135
передача
по значению 208

по ссылке 208
переменной длины
и неоднозначность 238
назначение 233
типа
обозначение 456
передача 457

Б

Байт-код

динамическая компиляция 29
интерпретация 28
назначение 28

Библиотеки

AWT

назначение 489
ограничения 519
равноправные, тяжеловесные компонен-
ты 519

Swing

Swing-апплеты
особенности структуры 550
создание 550
Swing-программы
завершение, способы 526
настольные приложения, создание 524
особенности структуры 524
архитектура MVC, видоизмененная 520
диспетчеры компоновки
BorderLayout, назначение 523
FlowLayout, назначение 523
назначение и состав 523
компоненты
Jbutton, применение 529
JCheckBox, применение 537
JFrame, применение 524
JLabel, применение 524
JList, применение 540
JTextField, применение 533
назначение 521
контейнеры
JScrollPane, применение 540
верхнего уровня, тяжеловесные 522
иерархия 521
легковесные 522
панели 522
легковесные компоненты 519
назначение 518
подключаемые стили оформления 519

происхождение и принципы конструирования 519

классов Java

JFC 519

назначение 52

состав 294

В

Ввод

данных из файла

в байтовые потоки 347

в символьные потоки 370

консольный

из байтовых потоков 345

из символьных потоков 365

символов с клавиатуры, организация 86

Взаимоблокировка

появление 413

предотвращение 413

Виртуальная машина Java

интерпретатор байт-кода 28

назначение 28

Выведение типов, применение 481

Вывод

данных в файл

из байтовых потоков 351

из символьных потоков 369

консольный

в байтовые потоки 346

в символьные потоки 367

Выражения

назначение 81

правила продвижения типов 82

пробелы и скобки, употребление 83

условные 88

Д

Дескрипторы

@author 605

{@code} 605

@deprecated 605

{@docRoot} 606

@exception 606

{@inheritDoc} 606

{@link} 606

{@linkplain} 606

{@literal} 606

@param 606

@return 607

@see 607

@serial 607

@serialData 607

@serialField 607

@since 608

@throws 608

@value 608

@version 608

З

Заглушка, определение 503

Закрытие файлов, автоматическое 353

И

Идентификаторы

назначение 51

обозначение 51

Иерархии классов, многоуровневые

назначение 257

порядок вызова конструкторов 260

создание 257

Инкапсуляция

назначение 32

поддержка в Java 32

Интегрированные среды разработки программ на Java 34

Интерфейсы

ActionListener, методы и реализация 530

Annotation, расширение 446

ItemListener, реализация и методы 537

LayoutManager, реализация 523

ListSelectionListener, реализация и методы 540

ListSelectionModel, константы режимов выбора из списка 541

Runnable, назначение 385

WindowConstants, реализация 526

назначение 284

наследование 307

обобщенные

объявление, общая форма 473

применение 471

объявление 295

переменные и константы 306

приемников событий 505

реализация 295

ссылки, применение 299

сходство с абстрактными классами 294

Исключения

- более точное повторное генерирование 328
 - вложенные блоки try 319
 - генерируемые
 - вручную 311, 320
 - повторно 321
 - иерархия классов 311
 - многократный перехват 327
 - обработка
 - в Java 311
 - в блоке try/catch 312
 - в другом методе 326
 - завершение в блоке finally 324
 - изящный способ устранения ошибок 316
 - особенности 313
 - преимущества 310
 - при вводе-выводе в файлы 350
 - с помощью нескольких операторов catch 317
 - обработчики 310
 - определение 310
 - порядок перехвата от подклассов 318
 - последствия неперехвата 314
 - проверяемые и непроверяемые 328
 - системные 311
 - создание в виде подклассов 331
 - условия генерирования 311
 - цепочечные 330
 - Исходные коды примеров программ
 - ввод вручную 35
 - загрузка 35
- ## К
- ### Классы
- ActionEvent, назначение 530
 - Applet
 - методы 501
 - назначение 501
 - Class
 - методы 456
 - назначение 456
 - Console, конструкторы и методы 363
 - Enum, методы 429
 - FileReader, применение 370
 - FileWriter, применение 369
 - JApplet, назначение 550
 - JButton
 - конструкторы и методы 530
 - назначение 529
 - JCheckBox
 - конструкторы и методы 537
 - назначение 537
 - JComponent, назначение 521
 - JFrame, назначение 524
 - JLabel, назначение 526
 - JList
 - конструкторы 540
 - методы 541
 - назначение 540
 - объявление 540
 - JTextComponent, назначение 533
 - JTextField
 - конструкторы и методы 533
 - назначение 533
 - ListSelectionEvent, назначение 540
 - Object
 - методы 281
 - назначение 281
 - Scanner, применение 379
 - String
 - методы 179
 - назначение 178
 - StringBuffer, назначение 182
 - System, назначение 52
 - Thread
 - конструкторы 386
 - методы 385
 - назначение 385
 - Throwable
 - методы 322
 - потомки 311
 - абстрактные 275
 - анонимные, внутренние
 - назначение 549
 - преимущества 550
 - применение 550
 - базового типа 477
 - байтовых потоков
 - иерархия 341
 - методы 344
 - вложенные
 - внутренние 230
 - определение 230
 - статические 230
 - назначение 32
 - обобщенные
 - объявление, общая форма 459
 - определение 452
 - реализующие обобщенный интерфейс, объявление 473

- с несколькими параметрами типа 457
- оболочек типов 372
 - конструкторы 437
 - методы 437
- общая форма определения 125
- объявление 126
- определение 124
- потоков ввода-вывода
 - в файлы, конструкторы и методы 347
 - двоичных данных, конструкторы и методы 356
- произвольного доступа к файлам
 - конструкторы 361
 - методы 361
- символьных потоков 342
 - иерархия 363
 - конструкторы 365, 368
 - методы 363, 368
- событий, иерархия 505
- спецификация 32
- члены
 - защищенные 290
 - методы и данные 125
 - определение 32
 - открытые и закрытые 200
 - статические 224
 - управление доступом 200
 - уровни доступа 288
- Ключевые слова
 - assert 512
 - catch, применение 312
 - class 125
 - enum, назначение 423
 - extends 242, 307
 - final 278
 - finally, применение 324
 - implements 295
 - import, применение 442
 - instanceof 511
 - interface 294, 446
 - Java, полный перечень 51
 - native 513
 - static 224
 - strictfp 511
 - super
 - вызов конструктора суперкласса 249
 - доступ к членам суперкласса 253
 - synchronized, назначение 402
 - this 151, 513
 - throws, применение 325

- transient 511
- try, применение 312
- void 132
- volatile 511
- зарезервированные 51
- назначение 51
- Кодовые блоки
 - назначение 47
 - область действия 67
 - применение 47
 - синхронизированные 405
 - создание 47
 - статические 227
- Комментарии
 - документирующие
 - назначение 604
 - общая форма 608
 - применение 605, 609
 - многострочные 37
 - назначение 37
 - однострочные 37

- Комплект JDK
 - интерпретатор Java 34
 - компилятор Java 34
 - установка 33

- Константы
 - восьмеричные 63
 - именованные, создание 279
 - перечислимого типа 423
 - самотипизированные 423
 - символьные 63
 - с обратной косой чертой 64
 - с плавающей точкой 63
 - целочисленные 63
 - шестнадцатеричные 63

- Конструкторы
 - назначение 144
 - обобщенные 470
 - параметризованные 145
 - перегрузка 217
 - порядок вызова 260
 - по умолчанию 144
 - применение 144

Л

- Литералы
 - назначение 62
 - со знаками подчеркивания 63
 - с плавающей точкой 63

строковые 64, 179
 целочисленные
 в двоичной форме 64
 в обычной форме 63
 шестнадцатеричные с плавающей точкой 64

Ложная активизация
 меры борьбы 408
 условия появления 408

М

Массивы
 главное преимущество 156
 двумерные 162
 индексирование 157
 инициализация 159
 многомерные
 инициализация 164
 объявление 164
 циклическое обращение 176
 назначение 156
 нерегулярные 162
 объявление
 альтернативная форма 165
 обычная форма 157
 одномерные 157
 определение 156
 присваивание ссылок 166
 реализация в виде объектов 156
 символьных строк 181
 соблюдение границ 159
 сортировка 160
 Метаданные, определение 445
 Методы
 main(), назначение 37
 print() и println(), назначение 41
 абстрактные 275
 апплетов, основные 494
 возврат
 без значения 133
 конкретного значения 133
 объектов 210
 вызов
 по значению 208
 порядок 132
 по ссылке 208
 динамическая диспетчеризация 268
 завершения 148
 назначение 130
 обобщенные

объявление 468
 определение 452
 обработки уведомлений о событиях 504
 общая форма объявления 130
 определение 32
 организации организация потоков 408
 отмены регистрации приемников событий 504
 параметризованные 137
 параметры
 определение 135
 передача 130
 применение 135
 указание 38, 136
 перегрузка 212
 переопределение
 назначение 266
 предотвращение 278
 соблюдение принципа полиморфизма 270
 эффективность механизма 270
 приостановки, возобновления и остановки потоков 414
 регистрации приемников событий 504
 рекурсивные 221
 с аргументами переменной длины
 объявление 233
 перегрузка 236
 сигнатуры 216
 синхронизированные, свойства 405
 собственные 513
 статические 225
 тело 132
 Многозадачность
 вытесняющая и невытесняющая 401
 на основе
 потоков 384
 процессов 384
 Многопоточная обработка
 в многоядерных системах 385
 в одноядерных системах 385
 главное преимущество 385
 языковые средства Java 385
 Модель делегирования событий
 источники и приемники событий 503
 применение 506
 принцип действия 503
 Модификаторы доступа
 abstract 275
 final 279, 327

private 38, 201
 protected 290
 public 37, 201
 synchronized 411
 transient 511
 volatile 511
 назначение и вызов 37
 указание 201

Н

Наследование

в конструкторах 248
 доступ к членам класса 245
 интерфейсов 307
 определение 33
 поддержка иерархической структуры 33
 предотвращение 279
 свойства 242
 ссылки на суперклассы и объекты под-
 классов 262
 суперклассы и подклассы 242

О

Области действия

вложенные 67
 переменных и кодовых блоков 67

Обобщения

аргументы типа
 обозначение 456
 передача 457
 ограничения 483
 определение 452
 параметры типа
 обозначение 455
 применение в классах и методах 455
 преимущества 453
 различие по аргументам типа 457
 соблюдение типовой безопасности 453
 только для объектов 457

Оболочки типов

классы 372, 437
 методы
 преобразования 372
 синтаксического анализа 372
 назначение 372, 437
 числовых 437

Объектно-ориентированное программи- рование

инкапсуляция 32

наследование 33
 организация программ, главный прин-
 цип 31
 полиморфизм 32

Объекты

возврат из методов 210
 как экземпляры класса 32
 копии переменных экземпляра 127
 объявление ссылок 129
 передача методам 206
 построение 32, 126
 создание 129

Операторы

break
 в операторе switch, применение 93
 в циклах, применение 109
 с меткой, применение 111
 continue
 в циклах, применение 114
 с меткой, применение 115
 extends, применение 460
 import 292
 new 129, 147
 package 285
 return 133
 switch
 вложенные 94
 назначение 91
 общая форма 91
 throw, применение 320
 try с ресурсами, применение 353
 арифметические 41, 70
 инкремента и декремента 46, 71
 логические
 обычные 72
 укороченные 73
 условные 74
 определение 69
 отношения 72
 поразрядные
 обычные 185
 составные 192
 порядок предшествования в Java 79
 присваивания
 обычные 40, 75
 по цепочке 75
 составные 76
 укороченные 75
 ромбовидные, применение 481
 сдвига 190

- сравнения 44
- тернарные, ? 195
- точки 126
- управляющие
 - выбора 86
 - итерационные 86
 - перехода 86
- условные
 - вложенные if 89
 - многоступенчатая конструкция if-else-if 90
 - обычные if 44, 87

Оформление исходного текста программ 49

Очереди

- динамические 301
- истощающие 169
- кольцевые 301
- линейные 301
- организация в виде массивов 170
- принцип организации 169
- реализация 300
- циклические и нециклические 169

П

Пакеты

- java.awt 505, 532
- java.awt.event 503, 505, 532
- java.io 341
- java.lang
 - особенности 294
 - стандартные исключения 328
- java.lang.annotation 447
- java.nio 371
- java.util 505
- javax.swing 521
- javax.swing.event 540
- доступ к содержимому 285
- иерархии, создание 285
- импорт 293
- исключение конфликтов имен 285
- назначение 284
- определение 201, 285
- поиск 286
- по умолчанию 285
- применение 286

Первая программа на Java

- ввод исходного кода 34
- запуск 36

- компиляция 35
- построчный анализ 36

Переменные

- инициализация
 - динамическая 66
 - обычная 66
- интерационные 173
- область и срок действия 67
- объявление 40, 65
- определение 39
- перечислимого типа 423
- ссылочного типа 129
- статические 224
- флаговые, назначение 414
- цикла 46, 98
- экземпляра
 - length 167
 - обращение 126
 - определение 32, 125
 - сокрытие 153

Перечисления

- как типы классов 425
- конструкторы 427
- методы 425
- назначение 423
- наследование от класса Enum 429
- ограничения 429
- определение 422
- переменные экземпляра 427
- порядковые значения констант 429
- создание 423

Полиморфизм

- главный принцип 33
- определение 32

Потоки

- ввода-вывода
 - байтовые 341, 343
 - в файлы 347
 - определение 341
 - символьные 341, 363, 369
 - стандартные 343
- диспетчеризации событий, назначение 528
- исполнения
 - момент завершения, определение 396
 - определение 384
 - организация взаимодействия, методы 408
 - основные и подчиненные 386
 - приоритеты 399

- приостановка, возобновление и остановка 414
- синхронизация 385, 401
- создание нескольких потоков 393.
- состояния 385
- способы создания 386
- Правила продвижения типов 82
- Преобразование типов
 - в выражениях 82
 - неявное, автоматическое 77
 - при присваивании 77
 - расширяющее 77
 - сужающее 78
 - явное 78
- Приведение типов
 - назначение 78
 - общая форма 78
- Программирование
 - история развития 31
 - многопоточное 384
 - объектно-ориентированное 31
 - параллельное 407
 - структурное 31
- Пространства имен
 - глобальные 284
 - назначение 284
 - разделение 285
- Процесс, определение 384
- Р**
- Рекурсия
 - главное преимущество 223
 - классический пример 222
 - определение 221
 - особенности реализации 223
- С**
- Сервлеты, назначение 29, 553
- Символьные строки
 - как объекты 178
 - команд действия 530, 533
 - операции 179
 - определение 64
 - постоянство 182
 - построение 178
 - сцепление 181
 - управление оператором switch 183
- Синтаксические ошибки, обработка 39
- Синхронизация
 - кодовых блоков 405
 - методов 402
 - мониторы и принцип блокировки 402
 - объектов 402
 - определение 401
 - потоков, причины 401
- Система
 - ввода-вывода
 - NIO, особенности организации 371
 - в Java, особенности организации 340
 - сборки мусора
 - демонстрация действия 149
 - методы завершения 148
 - назначение 148
 - особенности 148
 - счисления
 - восьмеричная 63
 - десятичная 63
 - шестнадцатеричная 63
- События
 - выбора из списка, обработка 540
 - действия, обработка 530
 - источники 504
 - категории 503
 - определение 504
 - от кнопки, обработка 532
 - от мыши
 - классы, интерфейсы и методы обработки 507
 - обработка 507
 - пример обработки в апплете 508
 - от поля ввода текста, обработка 533
 - от элементов, обработка 537
 - приемники 504
 - процедура обработки 506
- Сортировка
 - алгоритмы 160
 - быстрая 227
 - пузырьковая 160
- Состояние гонки
 - появление 413
 - предотвращение 413
- Статический импорт
 - механизм 442
 - общие формы 444
 - применение 443
- Стек
 - принцип организации 169
 - реализация 239
- Стирание

механизм 482
ошибки неоднозначности 482

Т

Темы для самостоятельного изучения Java 553

Типы данных

базовые
главный недостаток 477
ограниченное применение 480
совместимые с устаревшим кодом 477
логические 61
назначение 56
обобщенные, реализация 482
объектно-ориентированные 56
ограниченные
задание 460
назначение 460
применение 462
простые 57
символьные 59
с плавающей точкой 58
целочисленные 57

Точка с запятой, назначение и применение 48

У

Упаковка и распаковка

вручную 437
механизм 437

Управление ресурсами

автоматическое 353, 356
традиционный способ 356

Управляющие последовательности, назначение 64

Утверждения

определение 512
применение 512
создание, общие формы 512

Утилиты

javadoc
дескрипторы 604
применение 604
параллелизм 407

Ф

Файлы программ на Java

выходные
выполнение 36

расширение 36
исходные
как единицы компиляции 35
присваивание имен 35

Ц

Циклы

do-while
общая форма 104
применение 104
for
без тела 101
бесконечные 101
для единственного оператора 97
для кодового блока 97
объявление управляющих переменных 102
пропуск отдельных частей 99
простейшая форма 46
разновидности 98
расширенные 102
расширенные, типа for-each 173
while
общая форма 103
применение 103
вложенные, применение 119
назначение 46

Ч

Чтение

символов из потока 366
строки, символьной из потока 367

Я

Язык Java

взаимосвязь с другими языками
С# 26
C/C++ 26
влияние на развитие Интернета 27
история развития 25
назначение 17
нововведения в версии
J2SE 5 18
Java SE 6 19
Java SE 7 19
основные свойства 30
учет регистра символов 39
эволюция 17

JAVA ДЛЯ ЧАЙНИКОВ

5-е издание

Барри Берд



Java — современный объектно-ориентированный язык программирования. Программа, написанная на Java, способна выполняться практически на любом компьютере. Зная Java, можно создавать мощные мультимедийные приложения для любой платформы. Десятки тысяч программистов начинали изучать Java с помощью предыдущих изданий этой книги. Теперь ваша очередь! Независимо от того, на каком языке вы программировали раньше (и даже если вы никогда прежде не программировали), вы быстро научитесь создавать современные кроссплатформенные приложения.

Основные темы книги:

- ключевые концепции Java;
- грамматика языка;
- повторное использование кода;
- циклы и условные конструкции;
- принципы объектно-ориентированного программирования;
- обработка исключений;
- создание апплетов Java;
- как избежать распространенных ошибок.

www.dialektika.com

ISBN 978-5-8459-1769-0 **в продаже**

JAVA ПОЛНОЕ РУКОВОДСТВО 8-е издание

Герберт Шилдт



www.williamspublishing.com

Эта книга предназначена для всех программистов — как для новичков, так и для профессионалов.

Начинающий программист найдет в ней подробные пошаговые описания и множество чрезвычайно полезных примеров. А углубленное рассмотрение более сложных функций и библиотек Java должно удовлетворить ожидания профессиональных программистов. Для обеих категорий читателей в книге указаны действующие ресурсы и полезные ссылки.

В этой книге автор бестселлеров по программированию Герберт Шилдт знакомит вас со всем необходимым для разработки, компиляции, отладки и запуска Java-программ. Полностью обновленное для платформы Java Platform, Standard Edition 7 (Java SE 7), это исчерпывающее издание рассматривает язык Java в целом, включая его синтаксис, ключевые слова и фундаментальные принципы программирования. Здесь вы найдете информацию о ключевых элементах библиотеки Java API, рассмотрите JavaBeans, сервлеты, апплеты и Swing и ознакомитесь с работой Java в реальных ситуациях. Кроме того, в этой книге подробно обсуждаются такие новые средства Java SE 7, как оператор try-c-ресурсами, строки в операторе switch, вывод типов с оператором <>, NIO.2 и Fork/Join Framework.

ISBN 978-5-8459-1759-1

в продаже

JAVA™ 2

БИБЛИОТЕКА ПРОФЕССИОНАЛА

ТОМ 1. ОСНОВЫ

8-Е ИЗДАНИЕ

Кей Хорстманн
Гари Корнелл



www.williamspublishing.com

Книга ведущих специалистов по программированию на языке Java представляет собой обновленное издание фундаментального труда, учитывающее всю специфику новой версии платформы Java SE 6. Подробно рассматриваются такие темы, как организация и настройка среды программирования на Java, фундаментальные структуры данных, объектно-ориентированное программирование и его реализация в Java, интерфейсы, программирование графики, обработка событий, Swing, развертывание приложений и апплетов, отладка, обобщенное программирование, коллекции и построение многопоточных приложений. Книга изобилует множеством примеров, которые не только иллюстрируют концепции, но также демонстрируют способы правильной разработки, применяемые в реальных условиях. Книга рассчитана на программистов разной квалификации, а также будет полезна студентам и преподавателям.

ISBN 978-5-8459-1378-4

в продаже

JAVA™ 2

БИБЛИОТЕКА ПРОФЕССИОНАЛА

ТОМ 2. ТОНКОСТИ ПРОГРАММИРОВАНИЯ

ВОСЬМОЕ ИЗДАНИЕ

Кей Хорстманн
Гари Корнелл



www.williamspublishing.com

Книга ведущих специалистов по программированию на языке Java представляет собой обновленное издание фундаментального труда, учитывающее всю специфику новой версии платформы Java SE 6. Подробно рассматриваются такие темы, как новые средства ввода-вывода, использование XML, API-интерфейсы работы в сети и взаимодействия с базами данных (JDBC), интернационализация, построение графических интерфейсов, безопасность, JavaBeans, взаимодействие с распределенными объектами и кодом, написанным на языке сценариев, а также платформенно-ориентированные методы, позволяющие обращаться на низком уровне к функциям операционной системы. Книга изобилует множеством примеров, которые не только иллюстрируют концепции, но также демонстрируют способы правильной разработки, применяемые в реальных условиях.

ISBN 978-5-8459-1482-8

в продаже

JAVASERVER™ FACES

БИБЛИОТЕКА ПРОФЕССИОНАЛА

3-Е ИЗДАНИЕ

**Дэвид Гери
Кей Хорстманн**



www.williamspublishing.com

Книга содержит исчерпывающие сведения по созданию веб-приложений на основе технологии JavaServer™ Faces (JSF). В ней показано, по какому принципу платформа JSF обеспечивает разделение дизайна страницы и логики приложения и как можно без особых затруднений реализовать веб-приложения, применяя заранее подготовленные компоненты в сочетании с кодом Java. Подробно описано, какое место JSF занимает среди общего набора средств создания приложений Java EE и как расширить возможности JSF, если встроенных средств недостаточно для решения конкретных задач. Программное обеспечение, необходимое для работы с книгой, доступно бесплатно. В книге приведены многочисленные сценарии, файлы конфигурации, файлы определения ресурсов, веб-страницы и готовые программы, которые подробно описаны, тщательно прокомментированы и составляют основу практически значимых приложений.

ISBN 978-5-8459-1706-5

в продаже



Полностью
обновлено
для
JDK 7

Приобрести основные навыки — проще простого!

Это учебное пособие для начинающих составлено Гербертом Шилдтом, автором популярных во всем мире книг по языкам программирования, таким образом, чтобы читатель смог быстро овладеть основными навыками программирования на Java. Полностью обновленное по версии Java Platform, Standard Edition 7, пятое издание этого учебного пособия начинается с рассмотрения самых основ, включая компилирование и выполнение простых программ на Java. Затем в нем описываются ключевые слова и синтаксические конструкции, составляющие основу Java как языка программирования. Далее следует изложение самых передовых языковых средств Java, включая обобщения и многопоточное программирование. И завершается книга введением в библиотеку Swing. Представленный в книге учебный и справочный материал позволяет легко и быстро научиться программировать на Java.

Особенности книги

- **Основные навыки и понятия.** Каждая глава начинается с перечня тем, которые предстоит усвоить читателю.
- **Обращение к знатоку.** Во врезках под этим заголовком даются полезные рекомендации в форме вопросов и ответов.
- **Примеры для опробования.** Это примеры небольших проектов, наглядно показывающие, как применять приобретенные знания и навыки на практике.
- **Упражнения для самопроверки.** В конце каждой главы приводятся контрольные вопросы и задания для проверки прочности усвоенного материала.
- **Подробные комментарии к примерам программ.** Все примеры программ в этой книге снабжены подробными комментариями, описывающими демонстрируемые языковые средства и приемы программирования на Java.

Категория: программирование
Предмет рассмотрения: язык Java
Уровень: начальный/промежуточный

ISBN 978-5-8459-1770-6



Издательский дом "ВИЛЬЯМС"
www.williamspublishing.com



MHPROFESSIONAL.COM



9 785845 917706



12007